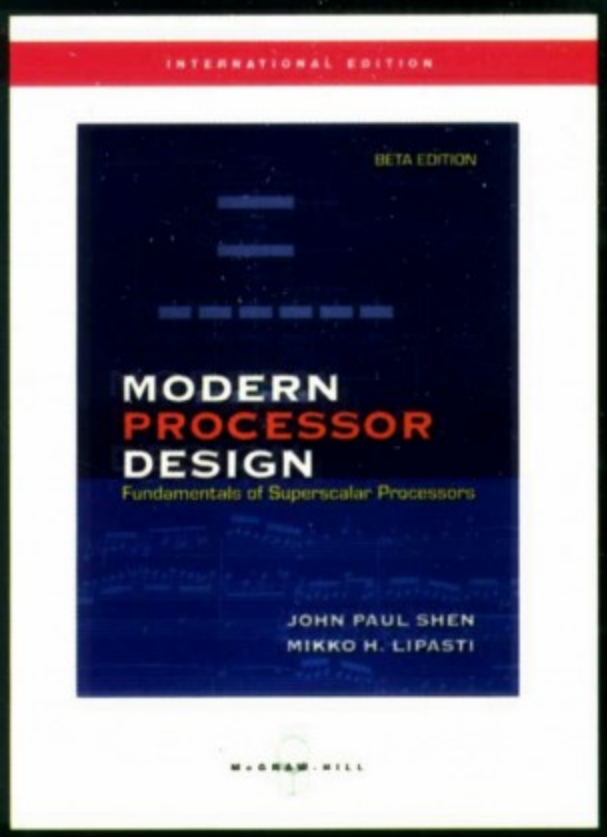
国 外 计 算 机 科 学 教 材 系 列

现代处理器设计

—— 超标量处理器基础

Modern Processor Design

Fundamentals of Superscalar Processors



[美] John Paul Shen 著
Mikko H. Lipasti

张承义 邓 宇 王 蕾 等译
戴 葵 审校

Mc Graw Hill

電子工業出版社
Publishing House of Electronics Industry
http://www.phei.com.cn

威权经典

现代处理器设计

——超标量处理器基础

Modern Processor Design

Fundamentals of Superscalar Processors

本书是一部有关超标量处理器设计的教科书，是卡内基·梅隆大学超标量处理器设计课程的教材。本书的特点是：突出关键的概念和基本的原理，隐藏复杂的技术细节；论述深入浅出，易于理解；内容全面并且新颖。书中内容涵盖了指令集、流水线等处理器设计的基本概念和超标量的结构以及主要的技术途径，同时提供了超标量处理器的实例，并对当前的超标量处理器产品进行了全面的分析和总结。本书还包含了一些高级讨论专题，并介绍了一些最新的超标量技术。

本书可供计算机系统结构专业的高年级本科生、研究生作为教科书和参考书，同时也可供从事处理器设计等领域研究的人员参考。

John Paul Shen

卡内基·梅隆大学电子与计算机工程系副教授，1973年毕业于密歇根州立大学，获学士学位；分别于1975年和1981年在南加州大学获得硕士和博士学位，之后进入卡内基·梅隆大学任教至今。主要研究领域为：现代处理器设计及评测技术，指令级并行的体系结构和编译技术，以及容错计算等。

Mikko H. Lipasti

美国威斯康星大学麦迪逊分校电子与计算机工程系助理教授，1992年在卡内基·梅隆大学电子计算机工程系获硕士学位，1997年获博士学位。主要研究方向为值预测和推测执行。

ISBN 7-5053-9806-7

9 787505 398061 >

责任编辑：李秦华
熊　健
封面设计：毛惠庚

本书贴有激光防伪标志，凡没有防伪标志者，属盗版图书

ISBN 7-5053-9806-7　定价：29.00元

国外计算机科学教材系列

现代处理器设计

——超标量处理器基础

Modern Processor Design

Fundamentals of Superscalar Processors

[美] John Paul Shen
Mikko H. Lipasti 著

张承义 邓 宇 王 蕾 等译
戴 葵 审校

電子工業出版社
Publishing House of Electronics Industry
北京 · BEIJING

内 容 简 介

本书是一部有关超标量处理器设计的教科书，是卡内基·梅隆大学超标量处理器设计课程的教材。本书的特点是：突出关键的概念和基本的原理，隐藏复杂的技术细节；论述深入浅出、易于理解；内容全面并且新颖。书中内容涵盖了指令集、流水线等处理器设计的基本概念和超标量的结构以及主要的技术途径，同时提供了超标量处理器的实例，并对当前的超标量处理器产品进行了全面的分析和总结。本书还包含了一些高级讨论专题，并介绍了一些最新的超标量技术。

本书可供计算机系统结构专业的高年级本科生、研究生作为教科书和参考书，同时也可供从事处理器设计等领域研究的人员参考。

John Paul Shen, Mikko H. Lipasti: Modern Processor Design: Fundamentals of Superscalar Processors
ISBN 0-07-057064-7
Copyright © 2003 by The McGraw-Hill Companies, Inc.
Original language published by The McGraw-Hill Companies, Inc. All rights reserved. No part of this publication may be reproduced or distributed in any means, or stored in a database or retrieval system, without the prior written permission of the publisher.
Simplified Chinese translation edition jointly published by McGraw-Hill Education (Asia) Co. and Publishing House of Electronics Industry. Copyright © 2004.

本书中文简体字翻译版由电子工业出版社和美国麦格劳-希尔教育出版（亚洲）公司合作出版。未经出版者预先书面许可，不得以任何方式复制或抄袭本书的任何部分。
本书封面贴有 McGraw-Hill 公司激光防伪标签，无标签者不得销售。
版权贸易合同登记号 图字：01-2003-3676

图书在版编目（CIP）数据

现代处理器设计——超标量处理器基础 /（美）沈（Shen, J. P.）著；张承义等译.
-北京：电子工业出版社，2004.5
（国外计算机科学教材系列）
书名原文：Modern Processor Design: Fundamentals of Superscalar Processors
ISBN 7-5053-9806-7

Ⅰ.现 … Ⅱ.①沈 … ②张 … Ⅲ.微处理器-设计-教材 Ⅳ.TP332

中国版本图书馆 CIP 数据核字（2004）第 026966 号

责任编辑：李秦华 熊 健
印 刷：北京增富印刷有限公司
出版发行：电子工业出版社
北京市海淀区万寿路 173 信箱 邮编：100036
经 销：各地新华书店
开 本：787 × 1092 1/16 印张：19.5 字数：530 千字
印 次：2004 年 5 月第 1 次印刷
定 价：29.00 元

凡购买电子工业出版社的图书，如有缺损问题，请向购买书店调换；若书店售缺，请与本社发行部联系。联系电话：（010）68279077 质量投诉请发邮件至 zlts@phei.com.cn，盗版侵权举报请发邮件至 dbqq@phei.com.cn。

出 版 说 明

21世纪初的5至10年是我国国民经济和社会发展的重要时期，也是信息产业快速发展的关键时期。在我国加入WTO后的今天，培养一支适应国际化竞争的一流IT人才队伍是我国高等教育的重要任务之一。信息科学和技术方面人才的优劣与多寡，是我国面对国际竞争时成败的关键因素。

当前，正值我国高等教育特别是信息科学领域的教育调整、变革的重大时期，为使我国教育体制与国际化接轨，有条件的高等院校正在为某些信息学科和技术课程使用国外优秀教材和优秀原版教材，以使我国在计算机教学上尽快赶上国际先进水平。

电子工业出版社秉承多年来引进国外优秀图书的经验，翻译出版了“国外计算机科学教材系列”丛书，这套教材覆盖学科范围广、领域宽、层次多，既有本科专业课程教材，也有研究生课程教材，以适应不同院系、不同专业、不同层次的师生对教材的需求，广大师生可自由选择和自由组合使用。这些教材涉及的学科方向包括网络与通信、操作系统、计算机组织与结构、算法与数据结构、数据库与信息处理、编程语言、图形图像与多媒体、软件工程等。同时，我们也适当引进了一些优秀英文原版教材，本着翻译版本和英文原版并重的原则，对重点图书既提供英文原版又提供相应的翻译版本。

在图书选题上，我们大都选择国外著名出版公司出版的高校教材，如Pearson Education培生教育出版集团、麦格劳－希尔教育出版集团、麻省理工学院出版社、剑桥大学出版社等。撰写教材的许多作者都是蜚声世界的教授、学者，如道格拉斯·科默（Douglas E. Comer）、威廉·斯托林斯（William Stallings）、哈维·戴特尔（Harvey M. Deitel）、尤利斯·布莱克（Uyless Black）等。

为确保教材的选题质量和翻译质量，我们约请了清华大学、北京大学、北京航空航天大学、复旦大学、上海交通大学、南京大学、浙江大学、哈尔滨工业大学、华中科技大学、西安交通大学、国防科学技术大学、解放军理工大学等著名高校的教授和骨干教师参与了本系列教材的选题、翻译和审校工作。他们中既有讲授同类教材的骨干教师、博士，也有积累了几十年教学经验的老教授和博士生导师。

在该系列教材的选题、翻译和编辑加工过程中，为提高教材质量，我们做了大量细致的工作，包括对所选教材进行全面论证；选择编辑时力求达到专业对口；对排版、印制质量进行严格把关。对于英文教材中出现的错误，我们通过与作者联络和网上下载勘误表等方式，逐一进行了修订。

此外，我们还将与国外著名出版公司合作，提供一些教材的教学支持资料，希望能为授课老师提供帮助。今后，我们将继续加强与各高校教师的密切联系，为广大师生引进更多的国外优秀教材和参考书，为我国计算机科学教学体系与国际教学体系的接轨做出努力。

电子工业出版社

教材出版委员会

主　任　杨芙清　北京大学教授
中国科学院院士
北京大学信息与工程学部主任
北京大学软件工程研究所所长

委　员　王　珊　中国人民大学信息学院院长、教授

胡道元　清华大学计算机科学与技术系教授
国际信息处理联合会通信系统中国代表

钟玉琢　清华大学计算机科学与技术系教授
中国计算机学会多媒体专业委员会主任

谢希仁　中国人民解放军理工大学教授
全军网络技术研究中心主任、博士生导师

尤晋元　上海交通大学计算机科学与工程系教授
上海分布计算技术中心主任

施伯乐　上海国际数据库研究中心主任、复旦大学教授
中国计算机学会常务理事、上海市计算机学会理事长

邹　鹏　国防科学技术大学计算机学院教授、博士生导师
教育部计算机基础课程教学指导委员会副主任委员

张昆藏　青岛大学信息工程学院教授

译 者 序

毫无疑问，微处理器是计算机系统最为关键的部分之一，其设计及制造也是计算机技术的核心。从第一块微处理器芯片Intel 4004于1971年诞生以来，微处理器按照著名的摩尔定律飞速地发展着，主频从最初的不到1MHz提高到1 GHz，集成的晶体管数目也从几千只增加到上亿只。在微处理器短短30多年的发展史上，超标量是一座重要的里程碑，它使微处理器的性能有了质的飞跃，从而引发了一场微体系结构的技术革命。目前为止，超标量技术依然是微体系结构中最为重要的技术之一。本书是一部关于超标量微处理器设计的教材并辅以大量实例，在微体系结构这一层次上比较全面地讲述了超标量微处理器的设计技术。在本书中，体系结构是指指令集体系结构，也就是指令集的规范，而微体系结构是指体系结构的具体逻辑实现，同一种指令集体系结构，例如IA32，可以有不同的微体系结构，并采用不同的流水线设计，不同的分支预测算法等。微体系结构的多样性使得同一种体系结构能够不断地推陈出新，并利用新出现的微体系结构技术来提高微处理器的性能，同时又保持代码的兼容性。因此，各种微体系结构技术将成为本书讨论的重点。丰富的实例是本书的一大特色，其详尽程度几乎可以称为一部资料大全，其中收录了超标量处理器发展史上绝大部分处理器。作者在对内容的处理上并不是简单地堆砌和罗列技术细节，而是有选择地强调每种设计在微体系结构上的不同之处，使读者能够领略到前人天才般的设计思想和独具特色的实现方式，并对当代一些经典的微体系结构技术的产生和发展形成充分的认识。前人的成果是一笔宝贵的财富，通过吸取前人的经验和教训，将使后来的设计者如同站在巨人的肩膀上，从而在更高的起点上施展抱负。本书中的许多内容译者也是首次接触，因此其翻译过程也是译者的学习过程，我们尽可能做到先理解之后再翻译，力求准确表达作者的思想。第6章由Intel公司负责Intel P6的技术人员执笔，其中有些地方比较晦涩难懂，我们参考了P6的其他资料来帮助理解其中的内容，但可能有些地方仍然翻译得不准确，欢迎读者与我们讨论。最后两章介绍一些新出现的微体系结构技术，其中部分术语比较生僻，中文尚无统一的译法，我们都将其英文原文标注出来，以最大程度地避免引起读者的混淆。

在本书的编辑出版过程中，夏宇闻老师给我们提出了宝贵意见和建议，在此表示衷心的感谢。

本书主要由张承义、邓宇、王蕾、王圣翻译，戴葵老师审校。限于译者的水平，书中若有不妥之处，敬请广大读者批评指正。

前　言

超标量处理器设计是一门面向高年级本科生和一年级研究生的课程，但是，相当一部分有能力的学生已经在大学三年级就选修了这门课。学习这门课程之前需要对计算机系统结构有一个初步的了解。超标量处理器设计课程的目标包括：（1）在微体系结构的抽象层次上讲述现代处理器设计中的技巧；（2）涵盖了通过开发指令级并行（ILP）来获取高性能的微体系结构技术；（3）讲述在开发高性能微处理器的过程中形成的深刻见解和经验。这门课程还包括一个实践项目——下一代超标量微处理器的微体系结构设计。该项目覆盖了本书所有的内容。

在20世纪90年代的10年中，为了获得更高的性能，人们提出了许多用于加快时钟频率以及开发指令级并行的微体系结构技术，并且将它们应用到了实际的处理器中。本书将把这些庞杂的知识加以系统化，这些技术包括深度流水、分支预测、动态寄存器重命名、多指令分派和发射、乱序执行和基于推断的存储器读写处理等内容。自20世纪90年代初以来，在这一领域内曾发表了数以百计的论文，其中许多思想都已经在商品化的超标量处理器中得到实现。为了便于读者理解，本书在一个清晰的框架中组织和阐述了很多技术细节，强调并突出了隐藏在这些技术细节背后的基本原理。

虽然本书一般作为研究生教材，但是我们在编写本书时，尽量使本书对于本科生也易于接受，从而使复杂的技术在阅读时更加直观。最后，我们希望本书所介绍的知识体系不仅对于微体系结构和处理器设计者有所裨益，同时也能够对该领域感兴趣的本科生和研究生有一定的帮助。

各章简要概述：

第1章　处理器设计　这一章介绍了处理器设计方面的技术。作为处理器规范的指令集体系结构（ISA，Instruction Set Architecture）和作为处理器实现的微体系结构，本章定义并讨论了软件和硬件之间的动态/静态接口。本章的目的并不是深入地讨论有关ISA的传统设计，而是为读者能够更好地理解现代处理器设计而建立一个合适的框架。

第2章　流水线处理器　这一章集中讲述了流水线的概念、指令流水设计并阐述流水线所带来的性能优势。流水线一般在第一门计算机体系结构课程中进行介绍。流水线是现代超标量技术的基础，本章将以一种新颖独特的方式进行阐述。我们力图避免使用大量的图表，而将重点放在指令流水线基本原理上。

第3章　超标量结构　这一章介绍超标量处理器的主要概念和结构。本章将以“总体概览”的方式将读者逐步引导到下一章关于超标量技术的详细讨论中去。同时，还强调了超标量处理器结构的主要特点。在第7章中将对真实的处理器进行详细的研究。

第4章　超标量技术　这一章是本书的重点，介绍了在当代超标量处理器设计中为了获得高性能而采取的主要微体系结构技术。本章介绍了加强指令流、寄存器数据流、存储器数据流的各种技术并对其进行了分类。为了便于读者理解，本章将大量的技术组织到了一个比较系统的框架中。

第5章　PowerPC 620　这一章给出了关于PowerPC 620微体系结构的详细分析，并把它作为一个实例来讨论前面各章所提到的许多问题和设计中的折中。本章还包括一个以乱序执行的设计方案，以及其完备的性能数据

第6章　Intel P6微体系结构　本章将针对Intel的P6机器进行实例讲解。P6大概是当代超标量微体系结构中在商业上最为成功的一个。本章由Bob Colwell领导的Intel P6设计组编写，深入地介绍了P6的微体系结构，这将有助于读者理解Pentium Pro、Pentium Ⅱ、Pentium Ⅲ等微处理器。本章给读者提供了一个机会以了解顶尖微处理器设计小组的设计思想。

第7章　超标量处理器概览　这一章讲述超标量处理器的发展历史，并对现有的超标量微处理器进行了总结。本章由Clemson大学的Mark Smotherman教授编写，最初完成于1998年，并且从那时起，不断地进行着修改和更新。本章包含了许多很宝贵的信息。

第8章　高级寄存器数据流技术　这一章着重介绍一些刚出现的微体系结构技术，这些技术利用程序中的值局部性（value locality）的特点来提高性能。程序中的值局部性是最近才被发现的。本章介绍的技术包括软件记忆法、指令重用以及各种形式的值预测等技术。尽管这些技术没有在真实的处理器中得到应用，但是未来的设计很有可能使用其中的若干种技术。

第9章　执行多线程　这一章介绍了线程级并行技术，并对许多其他技术进行了基本的介绍，包括多处理技术、cache一致性以及在多处理机中保证时序或者松弛存储器排序的高效实现技术。本章讨论了单芯片技术中的多线程技术和片上多处理技术，这两种技术目的都是开发线程级并行。最后，本章还介绍了两种新出现的技术：隐含多线程（implicit multithreading）技术和预执行（pre-execution）技术，这两种技术用于自动开发单线程程序中的线程级并行性。

第1章到第4章涵盖了一些基础概念和基本技术。第5章、第6章、第7章提供了超标量处理器的实例，并对真实的商用超标量处理器进行了全面的总结。第8章、第9章包含了一些高级讨论的专题，这两章突出介绍了一些最新出现的技术。

目　录

第 1 章　处理器设计 1
1.1　微处理器的发展史 1
1.2　指令集处理器设计 2
1.2.1　数字系统设计 2
1.2.2　体系结构、逻辑实现和物理实现 3
1.2.3　指令集体系结构 4
1.2.4　动态静态界面 5
1.3　处理器性能法则 6
1.3.1　处理器性能公式 7
1.3.2　处理器性能优化 7
1.3.3　性能评价方法 8
1.4　指令级并行处理 10
1.4.1　从标量到超标量 10
1.4.2　指令级并行的极限 15
1.4.3　指令级并行的机器 17
1.5　小结 21
1.6　习题 22

第 2 章　流水线处理器 24
2.1　流水线基础 24
2.1.1　流水线设计 24
2.1.2　算术流水线的实例 27
2.1.3　流水线理想假设 30
2.1.4　指令流水线 32
2.2　流水线处理器设计 34
2.2.1　保持流水段均衡 34
2.2.2　统一指令类型 38
2.2.3　减少流水线停顿 44
2.2.4　产品化的流水线处理器 54
2.3　深流水线处理器 59
2.4　小结 61
2.5　习题 61

第 3 章　超标量结构 63
3.1　标量流水线的局限性 63

3.1.1 标量流水线吞吐率的上限 63
3.1.2 低效的统一流水线 64
3.1.3 严格流水线导致的性能损失 64
3.2 从标量流水线到超标量流水线 65
3.2.1 并行流水线 65
3.2.2 多配置流水线 67
3.2.3 动态流水线 69
3.3 超标量流水线综述 71
3.3.1 取指 72
3.3.2 指令译码 74
3.3.3 指令分派 76
3.3.4 指令执行 79
3.3.5 指令的完成和提交 81
3.4 小结 82
3.5 习题 83

第 4 章 超标量技术 85
4.1 指令流技术 85
4.1.1 程序控制流和控制相关 85
4.1.2 分支造成的性能损失 86
4.1.3 分支预测技术 88
4.1.4 分支预测失败的恢复 91
4.1.5 先进的分支预测技术 93
4.1.6 其他指令流技术 96
4.2 寄存器数据流技术 97
4.2.1 寄存器重用和假数据相关 97
4.2.2 寄存器重命名技术 98
4.2.3 数据相关和数据流极限 101
4.2.4 经典的Tomasulo算法 102
4.2.5 动态执行内核 107
4.2.6 保留站和再定序缓冲 109
4.2.7 动态指令调度器 111
4.2.8 其他寄存器数据流技术 111
4.3 存储器数据流技术 112
4.3.1 存储器访问指令 113
4.3.2 存储器层次结构 114
4.3.3 存储器访问的排序 120
4.3.4 载入旁路和载入定向 121
4.3.5 其他存储器数据流技术 124

4.4 小结 127
4.5 习题 128

第 5 章 PowerPC 620 136
5.1 简介 136
5.2 实验框架 138
5.3 取指 139
5.3.1 分支预测 140
5.3.2 取指和推测 141
5.4 指令分派 142
5.4.1 指令缓冲 142
5.4.2 分派停顿 142
5.4.3 分派效率 144
5.5 指令执行 145
5.5.1 发射停顿 145
5.5.2 并行执行 145
5.5.3 执行延迟 146
5.6 指令完成 146
5.6.1 完成并行度 146
5.6.2 cache 的影响 147
5.7 结论和评价 148
5.8 IBM POWER3 和 POWER4 149
5.9 小结 151
5.10 习题 151

第 6 章 Intel P6 微体系结构 153
6.1 简介 153
6.1.1 P6 微体系结构基础 155
6.2 流水线 156
6.2.1 按序执行的前端流水线 157
6.2.2 乱序执行的内核流水线 157
6.2.3 指令提交流水线 158
6.3 按序执行的前端 159
6.3.1 指令缓存与 ITLB 159
6.3.2 分支预测 161
6.3.3 指令译码器（ID）..... 163
6.3.4 寄存器别名表 165
6.3.5 分配器 170
6.4 乱序执行的内核 171
6.4.1 保留站 171

6.5 指令提交 172
6.5.1 再定序缓冲 172
6.6 存储子系统 175
6.6.1 存储器访问顺序 176
6.6.2 存储器 load 操作 176
6.6.3 基本存储器存储操作 177
6.6.4 延期存储器操作 177
6.6.5 页故障 177
6.7 小结 178
6.8 习题 178

第 7 章 超标量处理器概览 180
7.1 超标量微处理器的发展 180
7.1.1 单处理器并行的早期发展：IBM Stretch 180
7.1.2 第一个超标量设计：IBM 高级计算机系统 182
7.1.3 指令级并行研究 186
7.1.4 DAE 的副产品：第一个多指令译码的实现 186
7.1.5 IBM 的 Cheetah, Panther 和 America 187
7.1.6 分离的微体系结构 188
7.1.7 20 世纪 80 年代其他的设计方案 188
7.1.8 超标量被广泛接受 189
7.2 对目前设计的分类 190
7.2.1 对 RISC 和 CISC 的翻新 190
7.2.2 Alpha：一种侧重于时钟周期的体系结构 192
7.2.3 POWER 系列：侧重于增强指令的体系结构 192
7.2.4 体系结构修订 193
7.3 处理器介绍 193
7.3.1 Compaq/DEC Alpha 194
7.3.2 HP 的 PA-RISC 1.0 版本 197
7.3.3 HP 的 PA RISC 2.0 版本 200
7.3.4 Intel i960 201
7.3.5 Intel IA32 203
7.3.6 MIPS 210
7.3.7 Motorola 68060 / 1993 213
7.3.8 Motorola 88110/1991 214
7.3.9 IBM POWER 215
7.3.10 PowerPC 219
7.3.11 SPARC 第 8 版 222
7.3.12 SPARC 第 9 版本 224
7.3.13 其他超标量处理器 226

第 8 章　高级寄存器数据流技术 235
8.1　简介 235
8.2　值局部性和冗余执行 237
8.2.1　值局部性的缘由 237
8.2.2　量化值局部性 238
8.3　非预测的值局部性利用 239
8.3.1　记忆法 240
8.3.2　指令重用 241
8.3.3　基本块和 trace 重用 244
8.3.4　数据流区域重用 244
8.3.5　结论 244
8.4　带预测的值局部性利用 245
8.4.1　弱相关模型 245
8.4.2　值预测 245
8.4.3　值预测单元 246
8.4.4　使用预测值的推断执行 249
8.4.5　值预测的性能 255
8.4.6　结论 256
8.5　小结 257
8.6　习题 257

第 9 章　执行多线程 259
9.1　介绍 259
9.2　共享存储器线程的同步 261
9.3　多处理机系统介绍 263
9.3.1　完全共享存储器，单位延迟以及无竞争 263
9.3.2　写操作的瞬时传播 264
9.3.3　一致的共享存储器 264
9.3.4　实现 cache 一致性 266
9.3.5　多级 cache、包含以及虚拟存储器 269
9.3.6　存储一致性 270
9.3.7　一致性存储器接口 273
9.3.8　结论 275
9.4　显式多线程处理器 275
9.4.1　单芯片多处理器 276
9.4.2　细粒度多线程 278
9.4.3　粗粒度多线程 278
9.4.4　同时多线程 280
9.5　隐式多线程处理器 285
9.5.1　化解控制相关 285

9.5.2 寄存器数据相关化解 288
9.5.3 存储器数据相关化解 289
9.5.4 结论 291
9.6 执行相同的线程 291
9.6.1 错误发现 292
9.6.2 预取 293
9.6.3 分支化解 294
9.6.4 结论 294
9.7 小结 294
9.8 习题 295

第1章　处理器设计

欢迎参与当代微处理器的设计。在短暂的30多年时间里，微处理器已经经历了显著的变化，其性能以每18个月翻一番的惊人速度提升。在过去的30多年时间里，计算机系统发生了许多革命与创新，其中微处理器功不可没。这些革命包括嵌入式微控制器，个人计算机，高级工作站，手持移动设备，应用程序和文件服务器，Internet上的Web服务，低功耗超级计算机，大规模计算集群系统。现在几乎每年都要出售超过1亿块微处理器，以满足移动设备、桌面系统及服务器市场的需求。包括嵌入式微处理器和微控制器在内，每年生产的总数超过10亿个微处理器单元。

微处理器是指令集处理器（ISP，Instruction Set Processor）。ISP执行预先定义指令集中的指令。微处理器的功能几乎完全取决于指令集，从而表明了它的执行能力。所有运行于微处理器上的程序都要基于指令集进行编码。这个预定义的指令集也叫指令集体系结构（ISA，Instruction Set Architecture）。ISA是软件与硬件之间的接口，或者是程序与处理器之间的接口。用处理器设计方法学的术语讲，ISA是设计的规范，而微处理器或ISP是设计的实现。与各种形式的工程设计一样，微处理器的设计过程天生就是一个创新的过程，有时需要进行微妙的折中处理，这需要非凡的直觉和敏锐的洞察力。

本书集中讨论当代微体系结构的超标量微处理器设计。本书比较系统地介绍了各种微体系结构技术及各种方案，并讲解一些基本的原则，希望能够培养出新的微体系结构设计人员，以致力于未来新一代微处理器的有效设计。

1.1　微处理器的发展史

第一块微处理器芯片Intel 4004于1971年诞生。4004是一个4位的处理器，大约有2300个晶体管，时钟频率刚刚超过100 KHz。4004主要应用于计算器。2001年是微处理器诞辰30周年纪念。高端微处理器包含有近1亿多只晶体管，时钟频率达到2 GHz，因此成为超级计算机系统和强劲的客户服务器系统的组成部分，这些系统已经充满了整个因特网。在很短的几年里，微处理器的主频将接近10 GHz并包含几亿只晶体管。

微处理器30的年发展历程体现了计算机工业中技术发展的不寻常的事实，参见表1.1。微处理器的发展与著名的摩尔（Moore）定律十分吻合，此定律是Gordon Moore在1965年发现的，即在单个芯片上的器件集成度将以每18个月到24个月的速度翻一番。在以前的30多年时间里，微处理器芯片上的晶体管数量增加了4个数量级。在同一时期，微处理器性能增加了5个数量级。在过去的20年时间里，微处理器性能每18个月翻一番，或每10年就成为原来的100倍。这样显著的发展速度是其他工业所不能比的。

表1.1　微处理器飞速发展的各个10年

	1970~1980	1980~1990	1990~2000	2000~2010
晶体管数量	2 K~100 K	100 K~1 M	1 M~100 M	100 M~2 B
时钟频率	0.1 MHz~3 MHz	3 MHz~30 MHz	30 MHz~1 GHz	1 GHz~15 GHz
指令数/时钟周期	0.1 IPC	0.1 IPC~0.9 IPC	0.9 IPC~1.9 IPC	1.9 IPC~2.9 IPC

在这30年的每一个10年中，微处理器在计算机工业最关键的技术中一直起着重要的作用。在第一个10年中，4位微处理器的问世很快导致8位微处理器的出现。这些数据宽度较小的微处理器内部都嵌入一个微控制器，并为洗衣机、电梯和喷气式发动机提供大量的嵌入式应用。同时8位微处理器也成为新型计算机平台，即个人计算机（Personal Computer）的核心部分，同时成为PC时代的先驱。

20世纪80年代的10年是32位微处理器体系结构与微体系结构等技术的发展时期。指令集设计问题成为大学与工业界研究的焦点。人们意识到指令集体系结构有助于硬件的有效实现，并能在编译器的优化过程中起到很好的杠杆作用。指令流水以及快速cache缓存成为标准的微体系结构技术。于是，基于32位微处理器的科研与工程工作站问世，它们具有强大的计算能力。这些工作站又依次成为研制下一代更高性能微处理器的更加强有力的工具。

在20世纪90年代，微处理器成为计算机中最强大、最受欢迎的组件。最快的微处理器时钟主频竟然已经超过了最快的超级计算机的主频。此时，个人计算机和工作站已经十分普遍，成为人们生产和通信必不可少的工具，但人们进一步渴望能够掌握更为强大的微体系结构技术，这些技术能够将微处理器的所有性能完全发挥出来。用于获取极高主频性能的深度流水技术和单周期多指令执行技术变得大受欢迎。这时，出现了指令的乱序执行和分支预测技术，它们可用来避免或减少流水线的停顿周期数。在微处理器时代的第三个10年快要结束的时候，几乎所有形式的计算平台，从个人手持设备、主流桌面系统、服务器到最强大的并行集群计算机都是基于微处理器而构建的。

现在我们正在步入微处理器时代的第四个10年，目前这种发展势头没有一点减缓的迹象。大多数技术都将按摩尔定律的速度持续至少10年到15年。到了2010年，我们可以预测，微处理器将会包含有10亿个晶体管、时钟频率将会超过10 GHz。我们完全可以相信，在许多领域仍然会有大量的创新出现。当前大家关注的焦点技术，即指令级并行将会扩展为包括线程级并行和存储级并行在内的技术。从历史的观点来看，体系结构应该隶属于一个系统的范畴，例如，多处理器和存储层次。现在，这些都将在一个单芯片上实现。这时，功耗将成为主要的性能障碍，如果要将发展速度保持在前30年的水平上，那么在所有的设计层次中都需要有新的解决方法，包括生产过程、电路设计、逻辑设计、微体系结构设计和软件运行时的环境。

本书主要介绍在微体系结构层次上微处理器设计的基本原则。我们将以综合的方式讲述前30年中开发研究的主要技术。本书力图将整个知识结构融入一个比较系统的框架中。对于那些比较复杂、难于解释的概念与技术，我们将进行提炼，使之变得更加直观，易于理解。同时还精选了最近出现的许多创新性技术。希望本书能够有助于产生一些优秀的新一代微处理器设计人才，他们将创造微处理器时代的第四个10年。

1.2 指令集处理器设计

本书的重点是设计指令集处理器，而指令集处理器的关键在于指令集体系结构（ISA），它定义了指令集处理器必须要实现的功能。ISA在指令集处理器的设计中起到了关键的作用。

1.2.1 数字系统设计

任何工程设计都是以规范开始的，其目的就是获得一种好的设计或实现方式。规范是一种行为描述，它指出需要什么或回答类似“它能做些什么”的问题；而实现是一个结构描述，它是关于设计结果的描述或回答“它是如何构建的”之类的问题。典型的设计过程包括两个基本任务：综合

和分析。综合试图找到基于规范的实现。分析将检查一个实现，以判断它是否或者如何来与规范保持一致。综合是一种更具创新性的工作，它需要寻找可能的解决方法，并进行不同的折中权衡以进行优化设计，最终获得最好的设计效果。它需要使用一些模拟工具来进行合法化检查及性能评估。典型的设计过程需要不断的多次分析与综合才能获得最终较为完善的设计。参见图 1.1。

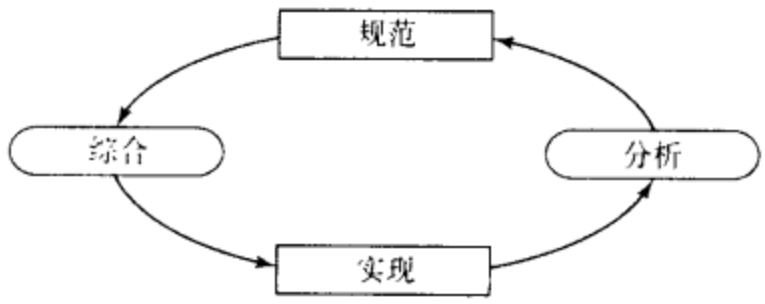


图 1.1　工程设计

在数字系统设计中，规范的制定非常严格，而设计的优化则依赖于强有力的软件工具。组合逻辑电路的规范将采取各种形式的布尔函数来定义输入输出变量之间的关系。实现一般都优化为两级与门和或门（AND-OR）的设计或者逻辑门的多级网络设计。优化力图减少设计中逻辑门的数量以及逻辑间的级数。对于时序电路设计，规范是以状态机的形式描述的，包括状态变量、输出以及下一状态功能的规范。优化则希望减少状态的数目以及相关组合逻辑电路的复杂性。使逻辑最小化与状态最小化的软件工具是必不可少的。逻辑和状态机模拟工具是用来辅助综合过程的。这些工具能够验证设计中的逻辑正确性，决定关键延迟路径以及状态机的最大时钟频率。

微处理器的设计是一个更加复杂的过程，难以直观描述。微处理器的设计规范就是指令集体系结构，它定义了微处理器必须执行的一整套指令集。实现就是实际的硬件设计，采用硬件描述语言（HDL，Hardware Description Language）进行描述。HDL 有很多基本模块，从逻辑门和触发器到更为复杂的模块，如译码器和多路复用器，并包含完整的功能模块，如加法器和乘法器。设计一般描述为这些基本模块的原理图或者互连结构。

设计现代高端微处理器的过程包括两个主要的步骤：微体系结构设计和逻辑设计。微体系结构设计包括为获得预期的性能而对关键技术进行的研究与确定。通常用一个性能模型作为评估这些技术有效性的分析工具。性能模型在时钟周期的粒度上准确地模拟了处理器的行为，能够计算执行一个测试程序（benchmark）所需要的时钟周期数目。微体系结构设计的最终结果是微处理器结构的高级描述。这种描述一般使用寄存器传输语言（RTL，Register Transfer Language）来定义处理器内部结构中所有主要的模块以及这些模块之间的互连。在进行逻辑设计时，通过加入实现细节来对 RTL 描述进行精化，最终生成实际硬件设计的 HDL 描述。RTL 和 HDL 描述可以使用相同的描述语言。例如，Verilog 就是这样一种语言。本书将主要讲述微体系结构设计。

1.2.2　体系结构、逻辑实现和物理实现

Blaauw 和 Brooks 所著的一本关于计算机体系结构的经典教科书中定义了三个基本抽象层次：体系结构、逻辑实现和物理实现。体系结构（architecture）规定了处理器的功能性行为。逻辑实现（implementation）是实现体系结构的逻辑结构和组织。物理实现（realization）是逻辑实现的物理结构和具体表现形式。

体系结构通常也称为指令集体系结构。它对指令集处理器的指令集合进行说明，并定义处理器的功能性行为。为了能被处理器执行，所有的软件都必须与指令集匹配，或者用该指令集进行编码。每个程序都被编译成这个指令集中的一个指令序列。IBM 360、DEC VAX、Motorola 68K、

PowerPC 和 Intel IA32 都是著名的体系结构。与体系结构相关的概念包括汇编语言、指令格式、地址模式和编程模型。这些概念都是ISA的一部分，直接与软件交互并且对编译器或程序员是可见的。

逻辑实现就是体系结构的具体设计，也称为微体系结构（microarchitecture）。某种体系结构在它的ISA生命周期内可以有许多逻辑实现。对于在这个ISA上编写的任何程序，同一体系结构的所有逻辑实现都可以执行。上面所列出的体系结构都有相应的逻辑实现，例如，IBM 360/91、VAX 11/780、Motorola 68040、PowerPC 604 和 Intel P6。与逻辑实现相关的一些概念包括流水线设计、cache存储器和分支预测器。逻辑实现或微体系结构的特征通常在硬件中实现，对软件是透明的。微处理器设计人员或微体系结构设计人员的工作就是研究并发展这些微体系结构的特征。

物理实现是设计的具体物理表现形式。对于微处理器而言，这个具体的物理表现形式通常是单芯片或多芯片的封装。对于一个逻辑实现，可以有许多不同的物理实现。这些物理实现可以按时钟频率、cache存储器容量、总线接口、结构技术及封装等方面的差异而具有不同的形式。与物理实现相关的概念包括圆片大小、物理组成因素、功耗、冷却和可靠性。这些都是芯片设计者和使用芯片的系统设计者所关心的内容。

本书主要讨论现代微处理器设计的逻辑实现。与体系结构和物理实现相关的问题也很重要。体系结构是逻辑实现的说明。与体系结构相关的概念对设计复杂性和逻辑实现的设计有着非常重要的影响。与物理实现相关的概念，例如圆片大小和功耗，也必须在设计的过程中加以考虑并作为设计目标的一部分。

1.2.3 指令集体系结构

指令集体系结构（ISA）作为软件与硬件或者程序与处理器之间的一种约定，它起着非常重要的作用。通过指令集体系结构，程序与机器能够相互独立发展。程序可以根据ISA的规定进行开发，而无需了解任何有关实际机器的实现细节。与此类似，机器也能够依据ISA进行设计，不需要关心任何有关程序运行的细节。所有为某一特定的ISA编写的程序应该能够运行于按照同一ISA所实现的任何一台机器上。

ISA也为软件的开发提供了便利。为特定ISA编写的程序能够运行于相同ISA的所有实现之上。给定一个ISA，在这个ISA的生命周期内，将会开发出许多实现，或者同时开发出具有不同性价比的多种实现。为这个ISA所编写的程序只需要开发一次，就可以运行于所有的实现之上。这种程序兼容性极大地减少了软件开发的费用，增加了软件使用的寿命。遗憾的是，由于这种特性，使得程序移植到一个新的ISA上非常困难。成功的ISA，或者是具有大量软件应用基础的ISA，容易保持更长的时间。IBM 360/370 与 Intel IA32 就是这样的例子。

ISA不仅是软件开发者或者编译器的参考，也是处理器设计者的规范。微处理器设计从ISA开始，直到产生符合规范的微体系结构。对于给定的ISA，每一个新型的微体系结构必须符合规范，以确保它能够完成ISA所规定的功能。保证所有已经存在的软件能够正确地运行于新型微体系结构上是至关重要的。

自从计算机问世以来，人们相继开发并使用了许多不同类型的ISA。它们的不同点集中在如何定义操作类型以及操作数上。ISA通常要定义一套称为汇编指令的指令，每一条指令都指定一种操作类型和一个或多个操作数。每个ISA所定义的汇编语言都是不同的。一个汇编语言程序由汇编指令序列组成。ISA主要通过每条指令的操作数数目进行区分，例如，2地址和3地址体系结构。一些早期的ISA使用一个累加器作为隐含操作数。在基于累加器的体系结构里，累加器作为隐含的源操作数和目的操作数。其他早期的ISA假定操作数存放在一个栈结构里，操作作用在栈顶的一个或者两个操作数上。大多数现代ISA都将操作数保存在一个多端口的寄存器文件中，所有的算术逻辑

运算都是基于这些保存在寄存器中的操作数进行的。一些特殊的指令，例如访存指令，则被设计为在寄存器文件与存储器之间移动操作数。一些传统的ISA允许操作数可以直接来自于寄存器文件和存储器。

由于人们一贯反对软件的重新编译和重新开发，所以ISA发展速度缓慢。一般来说，除非性能能够显著地增长，否则软件开发人员是不会付出一定的额外开销来重新编译已经存在的软件。虽然不断对旧有的ISA进行扩展能够适应新出现的应用，但全新的ISA要高一个档次。对于新出现的ISA，开发与其相配套的编译器和操作系统将需要花费10年以上的时间。ISA存在的时间越长，基于这个ISA的软件应用基础将越大，将来取代这个ISA的困难也就越大。在某些特定的应用领域可能会出现一些异常现象，即一个全新的、专用的ISA可能会极大地提高性能，例如提高10倍以上。

不像ISA的发展那样缓慢，微体系结构的发展非常迅速，每3~5年就有新的发展。在20世纪80年代，人们对于ISA设计的兴趣普遍比较浓厚，到处都是关于如何才能构造出最好的ISA的充满热情的讨论。但是，自从到了20世纪90年代，焦点转移到了实现具有革命性质的微体系结构技术上，这些技术可以应用于所有的、至少是绝大部分的ISA。在过去的10年里，只有很少一部分ISA能够占据微处理器领域，在未来的10年里很可能也将如此。另一方面，我们期望在未来的10年里，在这些ISA的基础上，能够出现不同的、创新的微体系结构。

1.2.4　动态静态界面

到目前为止，我们讨论了ISA所起的两个关键作用。首先，它提供了软件与硬件之间的约定，这将程序的发展与机器的发展相互独立开来。其次，ISA成为微处理器设计的规范。所有的实现必须满足这个规范并支持ISA规定的功能。除了这两个关键作用之外，每个ISA还有第三个作用。在ISA中还有一个内在的接口定义，区别哪些是在编译时静态完成的，哪些是在运行时动态完成的。Yale Patt将这个接口称为动态－静态接口（DSI，Dynamic-Static Interface），如图1.2所示。

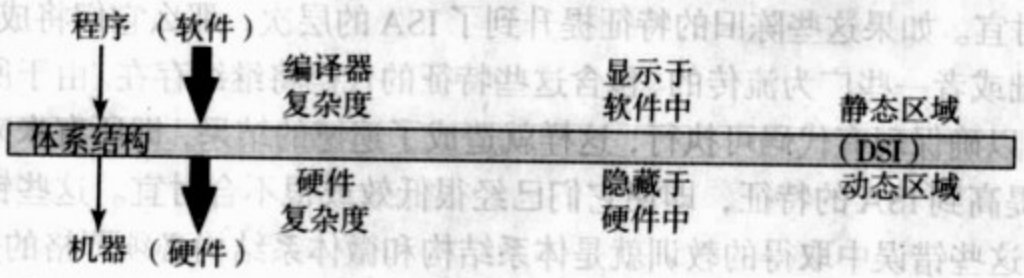


图1.2　动态静态界面

DSI是将ISA作为软件与硬件之间的约定所产生的直接结果。传统上，所有在编译时由软件和编译器静态完成的任务和优化，被认为是在DSI之上的。相反，所有在运行时由硬件动态完成的任务以及优化，则被认为是在DSI之下的。所有体系结构的特性都在ISA中规定，因此都处于静态区域。另一方面，微体系结构的所有实现特性都处于DSI之下并处于运行时的动态区域，通常这些特性对软件和编译器是完全透明的。前面已经说过，处于DSI之上的软件与处于DSI之下的微体系结构的发展是相互独立的。

ISA设计中的一个关键问题就是DSI的位置。DSI可以置于用高级语言编写的应用程序与底层机器的实际硬件之间的不同的抽象层次上。DSI的位置关系到哪些应该处于DSI之上，哪些应该处于DSI之下。例如，可以通过编译器优化那些处于DSI之上的内容并提高性能，也可以在微体系结构中优化那些处于DSI之下的内容来提高性能。在理想情况下，DSI的位置应该使得静态技术与动态技术能够很好地协同配合，在编译器的复杂性与硬件复杂性之间做出权衡并得出最好的组合，从而获得所需要的性能。

由于软件技术与硬件技术不断的发展，关于DSI的位置问题已经成为一个真正的挑战。

在ISA的设计历史中，人们提出了大量不同的DSI放置方式，其中的一些已经产生了成功的商品化ISA。关于DSI可能放置方式的概念性描述参见图1.3。该图并不是很严格，只是简单地阐述了DSI可以被放置在不同的级别。例如，Mike Flynn将DSI放到了很高的位置，所有的工作都在DSI之下完成。这样，用高级语言编写的程序可以通过直接执行语言的机器来执行。复杂指令集计算机（CISC）的ISA将DSI放置在传统的汇编语言这一层，或者是宏代码这一级。与此相对应，精简指令集计算机（RISC）的ISA降低了DSI的位置，期望通过运行于DSI之上的编译器来完成更多的优化工作。可以降低DSI的位置将原来CISC中的部分微体系结构特征提高到ISA中来考虑，这样做的目的是减小硬件复杂度，从而获得更快的机器速度。

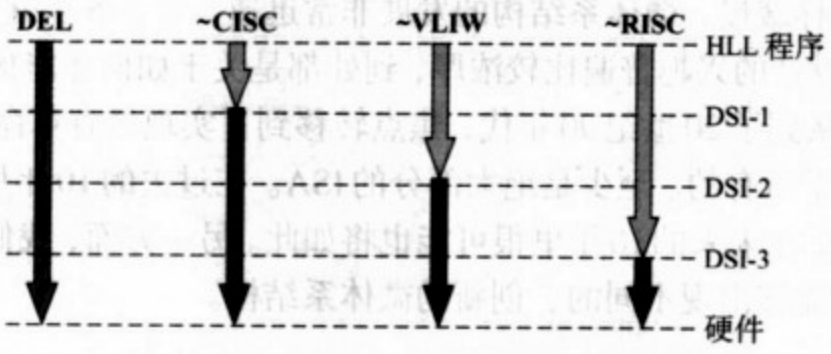


图1.3 在ISA设计中关于DSI放置的概念性阐释

DSI在将体系结构和实现分离。违反了这种分离原则可能会产生问题。ISA不断发展，同时不断加入许多新的特性，从而导致DSI的位置也在不断地移动。通过将以前的实现特征提高到体系结构的层次，从而以另一种方式将DSI降低，从而能有效地将某些原始的微体系结构特征暴露给软件。这有助于编译器的优化，减小硬件复杂度。但是，硬件技术变化得太快，处理器的逻辑实现必须适应并利用这些技术上的改变。随着实现类型以及技术的改变，一些陈旧的技术或微体系结构特征将变得低效、不合时宜。如果这些陈旧的特征提升到了ISA的层次，那么它们将成为ISA的一部分，那些软件应用基础或者一些广为流传的、包含这些特征的代码将继续存在。由于所有未来的实现必须支持整个ISA，以确保现有代码可执行，这样就造成了遗憾的结果，即所有未来的实现必须继续支持那些在早期提高到ISA的特征，即便它们已经很低效或很不合时宜。这些错误已经在真实的ISA中发生了。从这些错误中取得的教训就是体系结构和微体系结构必须严格的分离。理想化的情况是，体系结构或者ISA应该只包含表达功能或者软件算法语义所必需的特征，而不管那些用于优化程序性能的所有特征应该归入实现还是归入微体系结构领域。

本书的焦点不是ISA的设计，而是微体系结构技术，从而着重强调性能。ISA特征可以影响设计的结果以及为获取高性能而考虑的设计复杂性。但是，我们的观点是，在当代高端微处理器设计中，微体系结构将在微处理器性能中居于支配地位，而不是ISA。因此，本书的重点是获取高性能的微体系结构技术，同时还有许多其他很重要的设计目标，例如功耗，耗费以及可靠性。但是，在历史上，性能从来都是受到绝对的重视，并且关于提升性能方面的技术形成了很复杂的知识结构。本书试图描述的就是这些知识结构。

1.3 处理器性能法则

微处理器设计中最先考虑的当然是性能。每一代处理器的性能相对于上一代处理器都有重大飞跃。在最近几年里，如何减小功耗成为一个比较棘手的问题，从而成为一个潜在的重要设计目标。但是，对更高性能的追求仍然是第一位的，处理器的性能仍是最关键的设计目标。

1.3.1　处理器性能公式

在20世纪80年代，一些研究者独立发现并总结出了一个公式，可以比较清晰地定义处理器的性能，并能够清楚地描述那些影响着处理器性能的基本因素。该公式后来成为处理器性能的基本法则，如公式1.1所示。首先，处理器性能公式指出，处理器性能是依据执行一段特殊代码所需要的时间来衡量的（时间/程序，Time/Program）。其次，时间/程序或者执行时间（execution time）的度量可以用公式加以描述并包括三项：指令/程序（instructions/program），时钟周期数/指令（cycles/instruction），时间/时钟周期（time/cycle）。第一项指出特定的程序需要执行的动态指令的数目，这一项也可以称为指令数（instruction count）。第二项指出平均（在整个程序的执行范围内进行平均）执行每一条指令需要耗费多少个时钟周期，这一项一般由CPI（Cycles Per Instruction）来表示。第三项指示每个时钟周期所需要的时间，也就是机器的时钟周期时间。

$$\frac{1}{\text{性能}} = \frac{\text{时间}}{\text{程序}} = \frac{\text{指令}}{\text{程序}} \times \frac{\text{周期数}}{\text{指令}} \times \frac{\text{时间}}{\text{周期}} \qquad \text{（公式 1.1）}$$

程序的执行时间越短，性能越好。考察公式1.1，我们得出结论，可以通过减小公式三项中任意一项的数量来提高处理器性能。如果指令数可以减小，那么所需要执行的指令就会减少，因此执行时间也会减少。如果CPI减小，则平均每条指令就会耗费更少的时钟周期数。如果可以减小时钟周期时间，整个程序的执行时间显然随之减少。在该公式中，提高性能似乎非常简单。但是，实际并不是那么直接。公式中的这三项并不是相互独立的，它们之间有着复杂的联系。减小三项中的任何一项都可能潜在地增加了其他两项的数值，因此很难阐述清楚它们之间的关系。提升性能极富挑战性，包括微妙的折中方案和精巧的权衡。这使得处理器的设计变得更加吸引人，很多时候更像是艺术而非科学。下一节将详细阐述各种不同的提升性能的方法。

1.3.2　处理器性能优化

可以说，所有性能优化技术最终都会归结为减少处理器性能公式三项因素中的一项或多项。一些技术可以在保持其他两项不变的情况下，减少其中的某一项。例如，当编译器进行优化时，在目标代码中消除冗余及无用的代码，从而减少了指令数，但是这不会影响CPI和时钟周期时间。当快速电路技术或者更为先进的结构减少了信号传输延迟时，机器时钟周期时间可以减小，但是却不会影响指令数和CPI。在不计较代价的情况下，通常都希望使用这种类型的性能优化技术。

其他优化技术在减少某一项的同时，会增加另外一项或者两项的数值。对于这些技术，只有在减少项的作用大于增加项的情况下，才能获得性能的提升。我们可以通过减少处理器性能公式中的任意一项来考察这些技术。

有许多种方法可以减少指令数。首先，指令集可以包含更多的复杂指令，每个指令可以执行更多的动作。执行的指令总数可以显著地减少。例如，在RISC ISA上运行的程序所需要的指令数目可能是运行在CISC ISA上的两倍。虽然指令数减少了，但是执行部件的复杂性增加了，从而导致时钟周期时间增加。如果利用深度流水线来避免增加时钟周期时间，那么分支预测错误所产生的开销也会导致CPI变大。其次，特定的编译器优化可以使执行的指令减少。例如，循环展开可以减少循环结束指令的执行次数。但是，这将导致静态代码的规模增加，从而影响指令cache的命中率，而这也会增加CPI。另外一个类似的例子是某些函数调用。通过消除调用和返回，可以减少执行的指令数目，但是代码规模会显著增加。第三，现在越来越多的研究者提出通过微体系结构技术动态消除冗余计算。他们已经观察到在程序执行时，对同一数据集的重复计算频繁发生。因此，早期计算的结果可以缓存下来，不需重复计算而直接使用。这种计算重用技术能够减少指令数，但是会增

加硬件实现的复杂性，导致时钟周期时间的增加。我们可以看出，减少指令数会潜在地增加CPI和时钟周期时间。

减少CPI的愿望激发了许多体系结构和微体系结构技术。导致RISC的一个关键动机就是减少每条指令的复杂性，从而减少处理每条指令所需要的时钟周期数目。我们已经看到，这将引起指令数的增加。另一种减少CPI的关键技术是指令流水，流水线处理器可以重叠执行多条指令。相对于非流水设计，在时钟周期时间相同的前提下，流水设计可以显著地减少CPI。较短的流水线，即级数较少的流水线，将比深度流水线具有更少的CPI，但是时钟周期时间要长一些。cache存储器能减少平均存储器访问延迟（以时钟周期数计算），因此使用cache也能减少CPI。当发生条件分支时，处理器将不得不从非连续地址处取出新的指令，从而产生停顿周期。分支预测技术可以减少停顿周期数目，从而减少CPI。但是由于增加了流水线取指段的复杂性，因此采取分支预测会潜在地增加时钟周期时间。如果利用深度流水线维持相同的时钟周期时间，则会增加CPI。超标量处理器允许流水线的每一段同时处理多条指令。在每个时钟周期内执行多条指令，CPI将会显著减少。当然，每一级流水段的复杂度也增加了，导致时钟周期时间或者流水线深度也潜在地增加，因此CPI也会增加。

减少时钟周期时间的关键技术是流水线技术。流水线将处理一条指令的任务有效地划分成多个流水段。每个流水段的延迟（信号传播延迟）决定了机器的时钟周期时间。通过使用深度流水线，每个流水段的延迟，即时钟周期时间可以减少。在最近几年里，流水线技术已经成为保持高端微处理器时钟频率显著增长的主要技术。参见表1.1，在最近的10年里，性能提高都得益于时钟频率的增加。

现在有这样一种趋势，即通过深度流水线来增加时钟频率。当流水线深度增加时，CPI可能会以三种方式上升。首先，当流水线前端深度增加时，取指和执行之间的流水级数就会增加。这增加了分支预测错误时产生的开销，从而导致CPI上升。第二，如果流水线长到连基本ALU操作也需要多个时钟周期，那么即便通过结果定向硬件，在两条相关指令之间所必须的等待时间也需要多个时钟周期。第三，当时钟周期随着流水线深度而增加时，存储器的延迟（以时钟周期数目计）将会显著增加。这会增加存储器操作的平均延迟，因此增加了总体的CPI。最后，在流水线中有硬件和延迟的开销，这将会减小在性能上的收益。这种通过加深流水线而获得更高频率的技术已经使用了超过10年的时间。我们并不清楚在复杂性和功耗开销达到极限之前，这些技术还能利用多久。

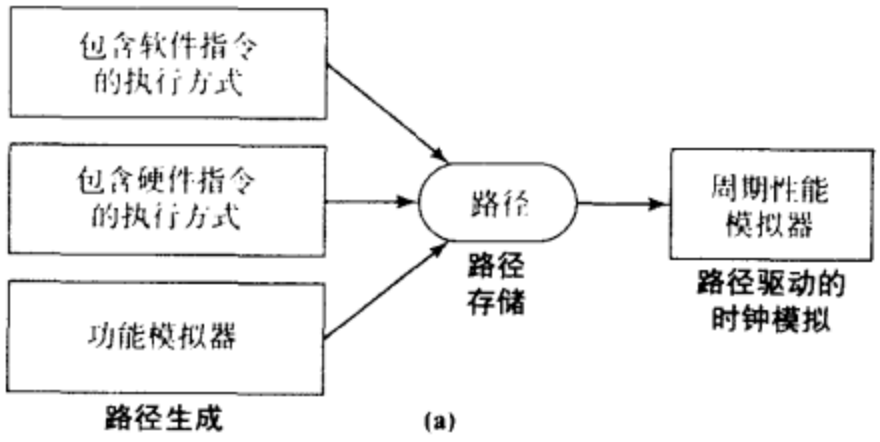
从以上讨论可以得出结论，提高性能并不是直截了当的事情，需要许多方面的权衡，包括一些微妙的问题。工业界中最有才能的体系结构设计人员和处理器设计者似乎都拥有对于采取更好的折中方法的某种直觉和洞察力。这将是本书的一个目标，或者说是一个梦想，即不仅仅是介绍超标量处理器设计的概念和技术，同时也要讲授那些超一流微体系结构设计人员的直觉与洞察力。

1.3.3 性能评价方法

在现代微处理器设计中，由于不可能对硬件建立原型，大多数设计者都使用仿真器进行性能模拟，从而保证设计过程中的功能正确性。现在使用两种比较典型的仿真器：功能仿真器和性能仿真器。功能仿真器模拟体系结构级（ISA）的机器，用于验证程序能否正确执行。实际上，功能仿真器是对一个程序的指令进行解释或者执行。性能仿真器模拟一个设计的微体系结构，用于测量执行一个程序所需要的时钟周期的数目。通常性能仿真器并不关心指令执行的语义正确性，而只关心指令执行的时间。

性能仿真器可以分为路径驱动（trace-driven）和执行（execution-driven）驱动。如图1.4所示，路径驱动的性能仿真器对预先生成的路径进行处理，从而得到这些路径上的指令执行的时钟周期数。

路径是指令执行的动态序列，可以通过三种不同的方法生成，参见图 1.4（a）。一种方法是当测试程序在某个实际系统上执行时，通过软件工具，在程序运行的过程中插入特殊的指令，插入指令将生成动态执行路径。另外一种方法是当程序在系统上执行时，通过硬件工具，插入特殊的硬件探针以监测系统总线，记录实际执行路径。软件工具可能会显著地增加代码规模和程序执行时间。硬件工具需要监测硬件，并且受限于监测硬件的缓冲容量。第三种路径生成方法使用功能仿真器来模拟程序的执行。模拟时，在仿真器中嵌入钩子，以记录动态执行路径。对于这三种方法，一旦路径生成，都可以存储起来，以便路径驱动性能仿真器重复使用。



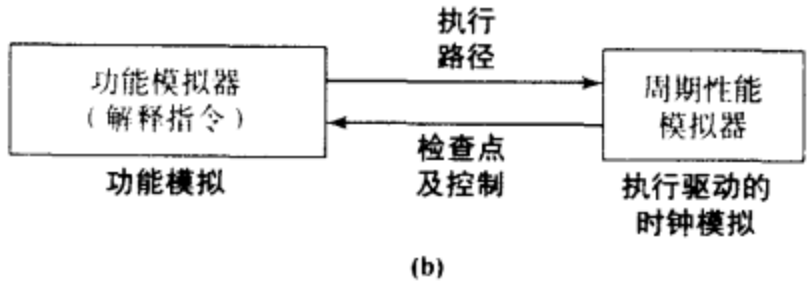


图 1.4　性能仿真器方法：（a）路径驱动模拟；（b）执行驱动模拟

执行驱动性能仿真器克服了路径驱动仿真器的一些缺陷，参见图 1.4（b）。执行驱动性能仿真器不是使用预生成的路径，而是与功能仿真器实现接口化，两个仿真器以串联方式同时工作。在模拟过程中，功能仿真器执行指令，将指令的相关信息传递到性能仿真器中。然后，性能仿真器记录这些指令的执行时间以及它们发出的动作。性能仿真器可以给功能仿真器发出指示，以建立检查点，记录下模拟状态，将来可以从这个检查点恢复现场。检查点机制可以模拟推断执行指令的过程，例如进行分支预测之后的指令。执行驱动仿真器能够模拟错误预测的指令，例如经过流水线的错误预测分支后的指令。在路径模拟中，预生成的路径只包括实际（非预测）执行的指令，路径驱动仿真器不能说明错误分支路径上的指令以及它们与其他（非预测）指令之间关于资源的潜在竞争。执行驱动仿真器也缓解了对存储长路径的需求。大多数较为先进的性能仿真器都使用执行驱动。

大多数先进的执行驱动性能仿真器都有功能仿真器的支持，可以执行全系统模拟，也就是说，模拟包括应用程序和操作系统指令、存储器层次，甚至包括输入/输出设备。

在性能仿真器中，随着所模拟的机器资源的数量和细节的不同，微体系结构模型的实际实现也很不一样。一些性能模型仅仅是一个时钟周期计数器，这些性能模型假设资源无限并考虑指令相关，然后简单地统计执行某条路径所需要的时钟周期总数。其他一些模型对机器的结构进行显式地模拟，包括所有组件模块。这些性能模型模拟指令在不同流水线段中的实际动作，包括每个时钟周期内有限机器资源的分配。虽然许多性能仿真器都声称“准确到时钟周期”，但它们在每个时钟周期中模拟并记录指令动作的方法可能大相径庭。

在微处理器的早期设计阶段，对于性能仿真器有很大的依赖性，验证性能仿真器的准确性非常困难。一般而言，性能模型或者仿真器是在设计的早期阶段开发的，帮助设计人员在不同微体系结构特征之间进行折中。在这一阶段，性能模型的验证没有任何参考可言。当设计有了进步，设计中的RTL模型有了发展，RTL模型能够作为性能模型准确性验证的参考。但是，RTL模型的模拟速度非常慢，因此只能使用非常短的路径。在整个设计过程中，为了确保性能模型能够及时反映出RTL模型中产生的所有改变，使性能模型和RTL模型协调一致的发展，必须要有一定的规则加以保证。性能模型的postsilicon验证也是十分重要的，这样这次设计可以成为下一代设计的良好起点。研究中所使用的大多数性能仿真器都是没有经过验证的。这些仿真器非常复杂，就像所有大型的软件一样，包含有许多难以消除的错误（bug）。所以在许多研究刊物上发表的、使用未验证的性能模型得出的许多测试数据，极有可能是错误的。

除了具有准确性验证的困难之外，另外一个与性能仿真器相关的问题就是模拟通常需要很长的时间。当前大多数性能评价方法都包含许多测试程序的模拟，这些程序包含上万亿条指令。在设计的早期阶段，性能仿真器广泛用于对众多折中方案进行评价，这需要在模型的模拟过程中使用许多不同的参数集合。对于执行驱动性能仿真器，由于相当详细地模拟了复杂机器的各个模块，所以具有一个4次方到5次方量级的减速因子是很平常的。从另一个方面看，为了模拟目标机器（被模型化的机器）中一个简单的机器时钟周期，将要在宿主机器中执行1万到10万个机器周期。一个较大集合的模拟有可能需要花费许多天才能结束，甚至需要使用许多仿真器。

1.4 指令级并行处理

指令级并行处理可以非正式地定义为多条指令的并行执行。传统的串行处理器每次只能执行一条指令。在下一条指令执行之前，前一条指令必须已经完成。在某种程度上，流水线处理器可以重叠执行多条指令，从而实现指令级并行处理。流水线有多少级，就有多少条指令可以同时执行。传统的串行（CISC）处理器每处理一条指令，平均需要约10个机器时钟周期，即CPI=10。而对于流水线（RISC）处理器，尽管完成每一条指令仍然需要那么多处理时间，但是通过多条指令的重叠执行，可以将平均CPI有效地降低到接近1，但前提是每个周期都有一条新的指令进入流水线。

标量流水线处理器仍然有一定的限制，那就是每个周期内最多只能发射一条指令。由于存在这种限制，CPI最多也只能降低到1。换句话说，标量处理器的吞吐率在最好情况下也只能达到每个时钟周期1条指令（IPC=1）。较好的指令级并行处理方式是在每个时钟周期取出多条指令，并在更宽的流水线中执行。20世纪80年代单芯片微处理器的设计目标是达到CPI=1，而20世纪90年代的设计目标则是将CPI降低到1以下，或是IPC大于1。IPC大于1的处理器称为超标量处理器。本节先对指令级并行技术进行总体概括，使读者能从标量流水线处理器自然过渡到超标量处理器。

1.4.1 从标量到超标量

标量处理器每个时钟周期至多只能取出一条指令并进行发射。超标量处理器则可以在每个时钟周期取出多条指令进行发射。这一小节主要讲述基本原理以及从标量处理器到超标量处理器的过渡。

1.4.1.1 处理器性能

在1.3.1节中，我们介绍了处理器性能的基本法则，如公式1.1所示。公式1.1是指令数、平均CPI和时钟周期时间的乘积，实际上代表了性能的倒数。我们可以改写这个公式，直接将性能表示

成指令数的倒数、平均IPC（IPC=1/CPI，即每个周期完成的指令数）与时钟频率的乘积，参见公式1.2。考察这个公式可知，可以通过增加IPC、提高频率或者减少指令数来提高性能。

$$\text{性能} = \frac{1}{\text{指令数}} \times \frac{\text{指令}}{\text{周期}} \times \frac{1}{\text{周期时间}} = \frac{IPC \times \text{频率}}{\text{指令数}} \qquad \text{（公式 1.2）}$$

指令数由三个因素共同决定：ISA、编译器和操作系统。ISA中每条指令的完成工作量对程序的指令总数有很大的影响。编译器的效率也对指令数有很大的影响。在程序的执行过程中，应用程序对操作系统的功能调用将增加执行的指令总数。

平均IPC反映了处理器达到的平均指令吞吐率，它是衡量微体系结构效率的一个关键参数。IPC的倒数CPI（每指令周期数）用于标志处理每条指令所需要的平均周期数。CPI的使用在标量流水线处理器时代相当普遍，因为不同形式的流水线停顿造成的性能开销都能很清楚地用CPI来表示。流水线处理器最终的性能目标就是将平均CPI降低到1。当我们进入超流水领域时，使用IPC显得更为方便。这时超流水处理器的性能目标就是获得大于1的平均IPC。本书中提到的大多数微体系结构技术都以提高IPC为目标。

生产工艺和电路技术对频率也有很大的影响。减少流水线中每一段的逻辑门的级数，同时增加流水线的级数也能够提高时钟频率。传统流水线的每一段有20级逻辑门，而当代大多数流水线只有10级甚至更少。为了在超标量设计中获得很高的IPC，流水线必须设计得很宽，以便每一段中可以同时处理多条指令。流水线的加宽增加了硬件的复杂度，增加了流水线各段之间的信号传播延迟。因此，在一个更宽的流水线中，为了维持相同的频率，流水线需要加深。这就需要在流水线究竟是要加宽还是要加深的选择中进行复杂的权衡。

1.4.1.2　并行处理器性能

当考虑在性能日益增长的处理器中进行指令并行处理时，很有必要重新思考一下经典的Amdahl定律。传统的超级计算机都是一些能够执行标量和向量计算的并行处理器。在标量计算中，只有一个处理器处于使用状态。在向量计算中，所有N个处理器都在对数组数据进行操作。由这种并行计算机所执行的操作如图1.5所示，其中N是机器中处理器的数量，h为机器进行标量运算的时间片。相反，$1-h$则是机器进行向量运算的时间片。

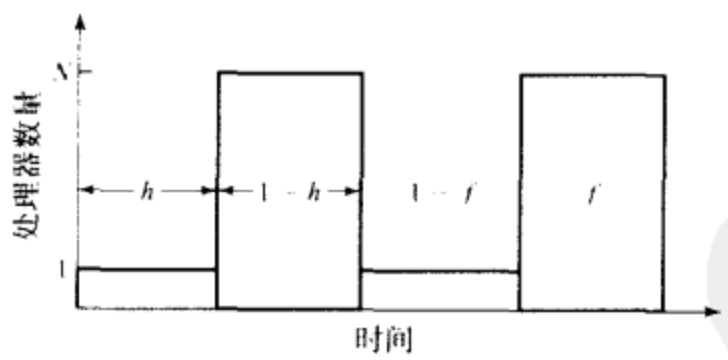


图1.5　传统超级计算机中的标量和向量处理

Amdahl定律中的一个公式说明，并行计算机的效率E是N个处理器总的利用率，或者N个处理器处于工作状态的时间。效率E可以表述为：

$$E = \frac{h + N \times (1-h)}{N} = \frac{h + N - Nh}{N} = 1 - h \times \left(1 - \frac{1}{N}\right) \qquad \text{（公式 1.3）}$$

当处理器数目N变得很大时，效率E接近于$1-h$，即机器在向量计算上所花费的时间。当N变大时，机器在向量计算上所花的时间越来越少，接近于0。因此，当N变得很大时，效率E接近于0。这意味着几乎所有的计算时间都用于标量计算，并且即使N继续增大，也不能使整体执行时间进一步减少。

Amdahl 定律的另外一个公式基于向量计算的工作量，或者是程序的向量化比例。如图 1.5 所示，f代表程序在向量计算模式中并行执行的时间片，而 1–f则代表程序必须串行执行的时间片。如果 T 是运行程序所需要的总时间，那么相关的加速比可以描述为：

$$S = \frac{1}{T} = \frac{1}{(1-f) + \frac{f}{N}} \qquad \text{（公式 1.4）}$$

这里，T 是执行串行部分所需要的时间 1–f 与执行并行部分的时间 f/N 的和。当 N 变大时，式中的第二项接近于 0，总的执行时间就决定于执行串行部分所需的时间。这通常称为串行瓶颈（sequential bottleneck）。也就是说，执行串行部分的时间，即执行标量计算所需的时间限制了通过开发并行性所能获得的性能提高。当 N 增加或者机器的并行性增加时，性能受程序串行部分的影响也将越大。

当处理器数目增加的时候，并行处理器的效率下降得非常快。而且，程序的向量化比例，即程序中能够并行执行的部分从 100% 慢慢下降时，效率的下降速度开始增加。类似地，当程序的向量化比例 f 开始从 100% 慢慢下降，整体加速比也会迅速下降。因此，整体性能的提高对程序向量化比例非常敏感。换句话说，当机器的并行性增加时，并行处理所带来的整体加速比将会更为强烈地依赖于程序的串行部分。

1.4.1.3　流水线处理器性能

Harold Stone 认为，并行处理器的性能模型可用于流水线处理器。流水线处理器典型的执行过程如图 1.6（a）所示。机器并行度参数 N 现在代表流水线的深度，即流水线的级数。执行过程分为三个阶段。首先是流水线填充阶段，在这个阶段，开始的 N 条指令进入流水线。第二个阶段是完全流水阶段，流水线完全充满，代表流水线的稳定阶段。这里有一个前提，即假设流水过程中没有中断，这代表了理想的执行过程。第三个阶段是流水线排空阶段，在这一阶段没有新的指令进入流水线，流水线继续处理停留在流水线中的指令。

为了方便建模，可以将流水线填充阶段所做的工作转移到排空阶段，并将图 1.6（a）修改为图 1.6（b）。整个工作量保持不变，即两个图中的面积是一样的。流水线为 N 级，所有 N 个段都在工作的时间片是 g，只有一个流水线工作的时间片是 1–g。实际上，1–g 可以看做是只有一条指令在流水线中移动的时间片，即这时流水线中没有重叠执行的指令。

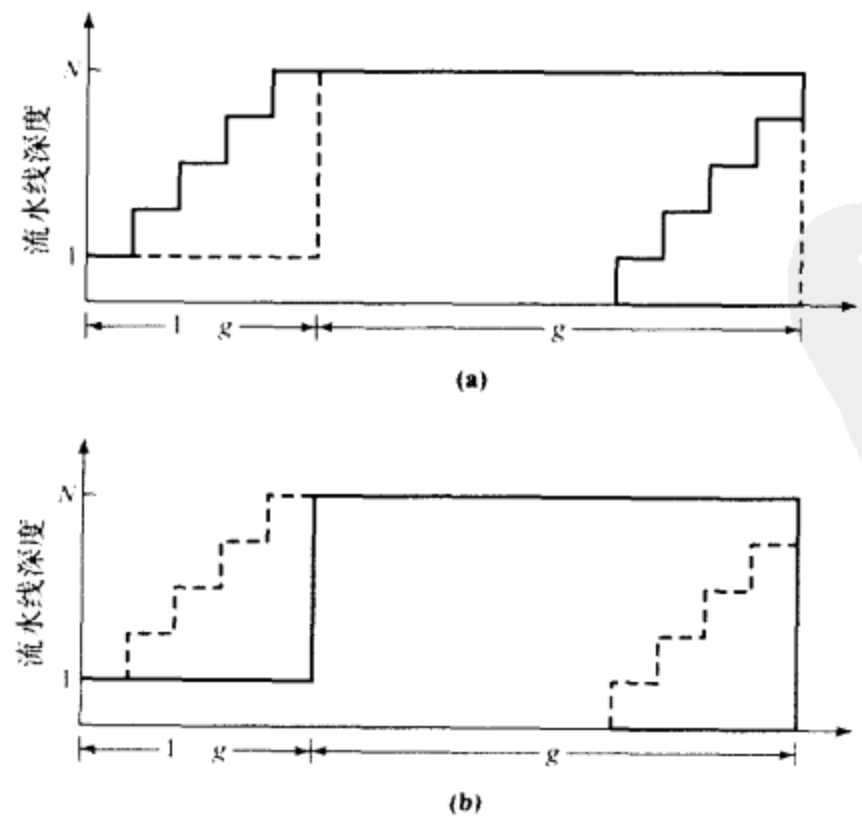


图 1.6　典型流水线执行过程：（a）实际情况；（b）模型

实际流水线的执行图与理想流水线的执行图不同，必须考虑停顿的周期，如图 1.7（a）所示。考虑停顿后，在整个执行过程中，流水线不能保持在完全流水阶段，这种稳定状态将被流水线的停顿打断。由于在流水线的完全流水阶段中存在中断，每一次停顿实际上都转化成了新的流水线排空段以及新的流水线填充段，如图 1.7（a）所示。可以将两个执行部分的填充阶段的工作转移到流水线的排空阶段，修改后的执行图如图 1.7（b）所示。

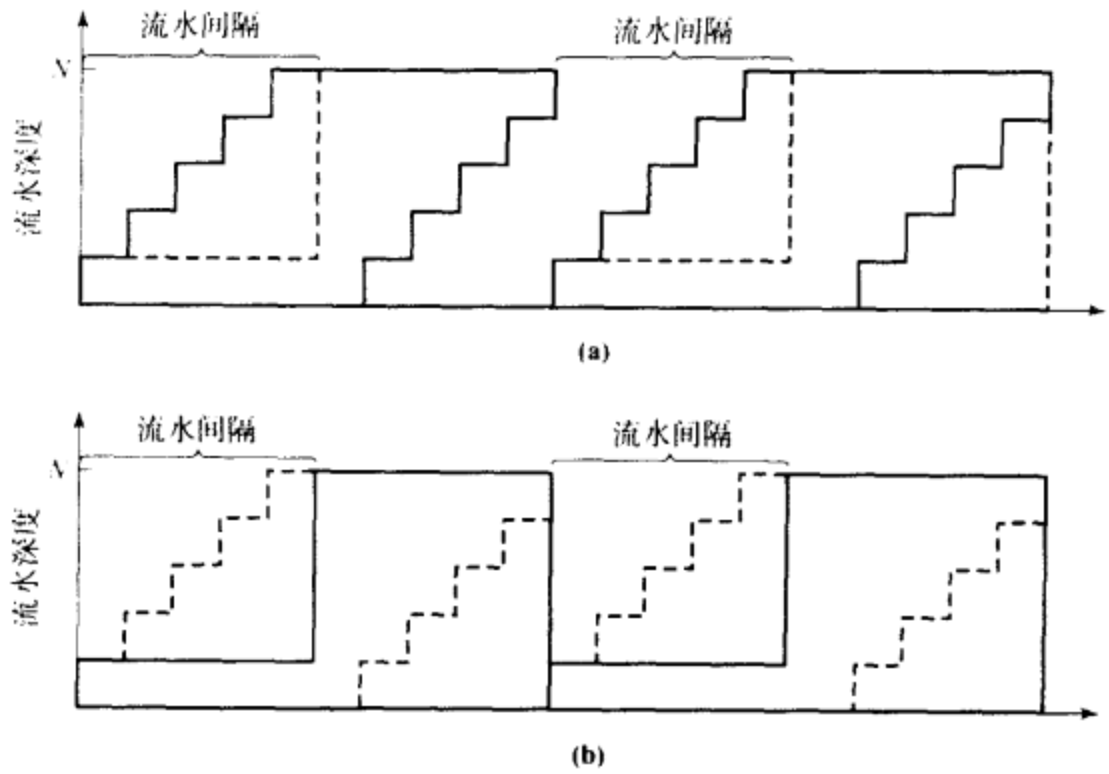


图 1.7　现实流水线执行过程：（a）实际情况；（b）模型

与执行图类似，我们可以借用并行处理器的性能模型并将它应用到流水线处理器中。这时 N 是指流水线的级数，或者是可能的最大加速比，而不是处理器的数量。参数 g 现在已经变成流水线完全充满的时间片，而参数 $1-g$ 则代表流水线停顿的时间片。加速比 S 可以表示如下：

$$S = \frac{1}{(1-g)+\frac{g}{N}} \qquad \text{（公式 1.5）}$$

注意，g 是指流水线完全充满的时间片并类似于 f，即并行处理器模型中程序串行部分的比例。因此，Amdahl 定律可以通过类比的方式应用到流水线处理器中。当 g 从 100% 稍微下降一点时，流水线处理器的加速比或者性能下降得很快。换句话说，通过流水线所能获得的实际性能提高将会由于一小部分停顿周期而大幅度下降。当流水线级数 N 增加时，停顿周期所占的时间对处理器采取流水化带来的实际加速比影响很大。和并行处理器中的串行瓶颈一样，停顿周期也是流水线处理器中的关键问题。实际上，停顿周期构成了流水线处理器中的串行瓶颈。

根据并行处理器中的 Amdahl 定律，公式 1.5 是一个简单的流水线处理器性能模型。假设在这个模型中，一旦流水线停顿，那么在流水线中将只存在一条指令，即处理器完全变成一个串行非流水的处理器。这表示当流水线停顿时，就不允许指令的重叠执行，这相当于将流水线暂停 N 个时钟周期，以使产生停顿的指令完全穿过流水线。但是，我们知道，通过对流水线进行很好的设计，例如使用定向路径，可以消除产生流水线停顿所产生的阻塞，这样所花费的开销就不是 N，大多数情况下小于 N。因此，可以对公式 1.5 的模型进行精化：

$$S = \frac{1}{\frac{g_1}{1}+\frac{g_2}{2}+\cdots+\frac{g_N}{N}} \qquad \text{（公式 1.6）}$$

公式1.6是公式1.5的一般化，它为分析流水线处理器性能提供了一个更为准确的模型。在这个模型中，g_i代表流水线中有i条指令的时间片，也就是说，g_i代表流水线停顿（$N-i$）个周期的时间片。当然，g_N是流水线完全充满的时间片。

在第2章中，我们通过将公式1.6应用于一个6级TYP流水线来说明流水线处理器的性能模型。注意，TYP流水线有1个周期的load指令开销以及4个周期的分支开销。根据第2章中提供的统计结果，load指令和分支指令的典型比例分别为25%和20%。假设TYP流水线的设计倾向于分支不成功，而分支成功的指令占分支指令的66.6%，那么只有66.6%的分支指令会导致分支开销。因此，只有13%的指令（分支）将会导致4个周期的开销，25%的指令（load指令）将会导致1个周期的开销。余下的指令（62%）将不会导致任何开销。TYP流水线的性能可以表示为公式1.7：

$$S_{TPY} = \frac{1}{\frac{0.13}{(6-4)} + \frac{0.25}{(6-1)} + \frac{0.62}{6}} = \frac{1}{\frac{0.13}{2} + \frac{0.25}{5} + \frac{0.62}{6}} = \frac{1}{0.22} = \mathbf{4.5} \qquad \text{（公式1.7）}$$

相对于串行处理器或者非流水线处理器，6级TYP流水线处理器的性能因子是4.5。注意，TYP是一个理论加速比为6的6级流水线。基于公式1.6的模型，实际加速比为4.5，如公式1.7所示，这可以看做是TYP流水线的“有效流水度数（effective degree of pipelining）”。实际上，6级TYP处理器就像一个只有4.5级的流水线。6和4.5之间的区别反映了潜在（峰值）的流水线并行度和可以获得（实际）的流水线并行度之间的区别。

1.4.1.4 超标量方案

现在，我们重新叙述一下Amdahl定律，将并行处理器的性能用公式1.8来表示：

$$S = \frac{1}{(1-f) + \frac{f}{N}} \qquad \text{（公式1.8）}$$

该模型给出了并行系统相对于非并行系统的性能加速比。机器并行度用参数N来表示，N指出机器中处理器的数量，反映了系统能够同时执行的最大任务数。而参数f则代表程序中可以并行执行的部分，反映了程序的并行度。这个公式的模型来自于传统的含有一个标量单元和一个向量单元的超级计算机。向量单元含有N个处理器，它们在同一时间启动N个任务来执行程序中的并行部分。而程序中的非并行部分则由标量单元以串行的方式执行。我们已经看到，在并行处理中，程序中的非并行部分对于整体性能有着很大的负面影响。

对于程序中的非向量化部分必须顺序执行则过于保守，也是不必要的。如果程序中的非向量化部分可以获得一定程度的、即便是很低的并行度，程序中串行瓶颈的影响也能显著地得到缓解。图1.8阐述了这个道理。该图来自于IBM的技术报告，是由Agerwala和Cocke于1987年共同完成的，它表示了在N值发生变化的情况下，将加速比作为f的函数，其中f为程序的向量化比例，而N是机器的最大并行度。例如当N=6时，加速比为：

$$S = \frac{1}{(1-f) + \frac{f}{(6)}} \qquad \text{（公式1.9）}$$

考查图1.8中公式1.9的曲线，我们可以看到，如果f为100%，加速比等于6，即完全向量化。当f从100%处开始下降时，加速比下降得非常快；当f变成0%，加速比则变成1，即不能获得加速比。随着N值的增加，加速比下降的速率也显著增加；当f接近0%时，无论N值为多大，所有的加速比都接近1。现在假设程序的非向量化部分可以获得最小为2的并行度。此时加速比变为：

$$S = \frac{1}{\frac{(1-f)}{2} + \frac{f}{6}} \qquad \text{（公式1.10）}$$

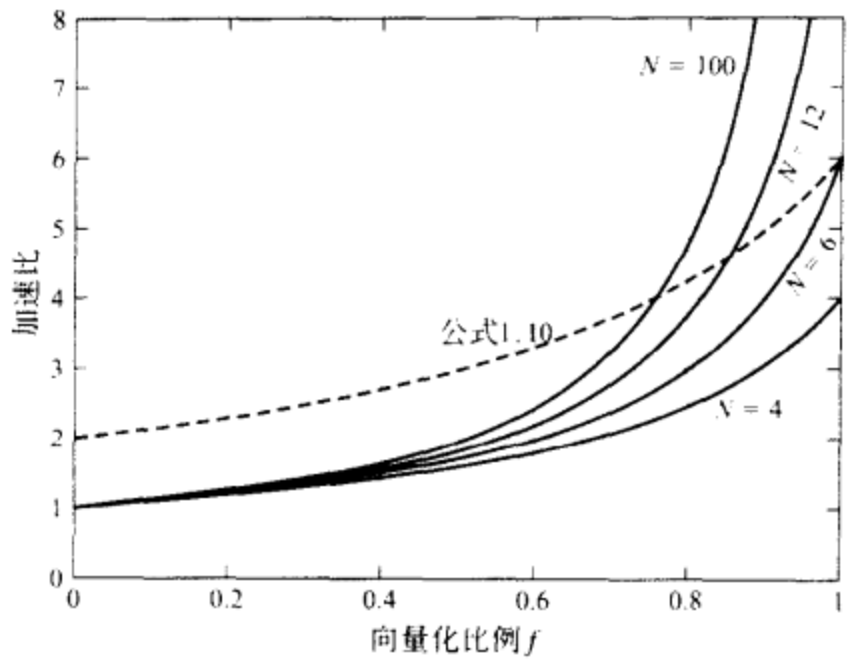


图 1.8　非向量化代码通过指令级并行消除串行瓶颈

考查图 1.8 中公式 1.10 的曲线，我们可以看到当 f 为 100% 时，它也以加速比为 6 的地方为起点，但是，当 f 开始从 100% 处下降时，它比公式 1.9 的曲线下降得要缓慢一些。实际上，当 f 接近 75% 时，该曲线将穿过 N 值为 100 时公式 1.8 的曲线。这意味着当 f 小于 75% 时，把最大并行度仅仅设定为 6，即 N=6，而非向量化部分的最小并行度设定为 2 的系统，要比把最大并行度设为 100 的系统更好。程序的向量化比例 f 是一个囊括了应用算法、编程语言、编译器和体系结构等因素在内的复杂函数。除了那些包含许多计算在内的科学应用之外，大多数通用计算的程序都没有很高的向量化比例。可以肯定地说，大多数通用程序的 f 值要低于 75%，并且对于许多程序，f 值要比 75% 低得多。

设计超标量处理器的一个主要动机就是开发可以获得一定（相对于向量来说较低）级别的并行度的通用处理器。目的是确保大部分的程序都能获得一定程度的指令级并行，以减轻顺序执行时瓶颈所产生的影响。当然，高度向量化的程序将会通过并行而持续获得较好的加速比。注意，即便 f 具有很大的值，公式 1.10 所对应的曲线也总是比公式 1.9 对应的曲线高，并且，在 f 值比较低的时候，还比其他 N 值大的曲线高。超标量处理器的目的就是在所有类型的应用程序中（包括在那些不能向量化的程序），获得指令级并行和相应的加速比。

1.4.2　指令级并行的极限

在公式 1.10 中，程序有 f 部分的并行度为 6，有 1–f 部分的并行度为 2。加速比 S 可以看做是整个程序可以获得的整体并行度。例如，如果 f 为 50% 并且峰值并行度 N 为 6，那么加速比或者整体并行度如下所示：

$$S = \frac{1}{\frac{(1-f)}{2} + \frac{f}{6}} = \frac{1}{\frac{0.5}{2} + \frac{0.5}{6}} = 3 \qquad \text{（公式 1.11）}$$

公式 1.11 的含义就是整个程序可以获得大小为 3 的整体并行度或者总的并行度。将这个结果应用到指令级并行，我们可以看出，公式 1.11 指出了在同一时间可以同时执行 3 条指令。对于传统的向量计算，可以同时操作的数目在很大程度上取决于向量或者数组的规模，或者是数据集的规模。对于通用而未经优化的程序，关键的问题是，指令级并行的整体并行度到底有多少潜力可挖？

指令级并行（ILP，Instruction-Level Parallelism）可以非正式地定义为整体并行度（由指令数衡量），可以通过多条指令的并行执行来获得。对于 ILP 可能的极限已经研究了将近 30 年。人们进

行了大量的专业研究，这些有关ILP极限的研究结果差异极大。下面是一些关于ILP极限方面的研究：

- Weiss and Smith [1984] 1.58
- Sohi and Vajapeyam [1987] 1.81
- Tjaden and Flynn [1970] 1.86
- Tjaden and Flynn [1973] 1.96
- Uht [1986] 2.0
- Smith et al. [1989] 2.0
- Jouppi and Wall [1988] 2.4
- Johnson [1991] 2.5
- Acosta et al. [1986] 2.79
- Wedig [1982] 3.0
- Butler et al. [1991] 5.8
- Melvin and Patt [1991] 6
- Wall [1991] 7
- Kuck et al. [1972] 8
- Riseman and Foster [1972] 51
- Nicolau and Fisher [1984] 90

当然，以上所列的只是一部分，但已经很清晰地表明了研究之间的差异与可能存在的矛盾。这些研究中的大多数都进行了不同程度的理想化约束。现实的问题是如何在实际设计中达到这样的ILP。这些研究得出的极限并不是随着年代的顺序而没有任何变化。在20世纪90年代的10年中，关于ILP极限的争论取代了20世纪80年代的RISC与CISC之间的争论。这场关于ILP极限的新争论仍然没有结束。

1.4.2.1 Flynn瓶颈

在早期的研究中，有一项是由Tjaden和Flynn于1970年在Stanford大学进行的，结论是大多数程序的ILP小于2。这个极限可以非正式地称做“Flynn瓶颈”。这项研究的重点在于基本块内可以开发的指令级并行度。由于穿越基本块边界时涉及到控制相关，这依赖于运行时的数据。这里，我们假设基本块必须顺序执行。由于大多数基本块的规模较小，所以典型的并行度小于2。这一结果（即Flynn的瓶颈）已经被其他几项研究加以证明。

Riseman和Foster于1972年进行的一项研究也证明了这一结果。但是，他们进行了相应的扩展性研究，即如果控制相关被化解，将能够获得多大的ILP？研究表明，如果一定数量的控制相关被化解，那么就可以获得一定程度的并行。如果可以化解的控制相关是无限的，那么ILP约为51。这一研究指出了控制相关对ILP的极限有着很大影响。

1.4.2.2 Fisher的乐观估计

Nicolau和Fisher于1984年在Yale大学也进行了一项研究。这项研究表明在许多程序中，几乎拥有无限的ILP。这项研究使用的测试程序更加数值化，一些并行性结果值来自于数组类型的数据并行性。他们的研究还假设了一台可以同时执行许多指令的理想化机器模型。当使用了一些理想化的假设后，原来由于Flynn瓶颈而在获取ILP方面造成的悲观现象又有了乐观的前景。我们将该ILP极限非正式地称做“Fisher的乐观估计”。

最初这种乐观的估计引起了大量的怀疑，后来的一系列事实证明了这项研究的正确性。首先，随着编译器不断提供更多的支持，一种称为 VLIW（超长指令字）处理器的机器模型发展了起来。随后，成立了一个公司（Multiflow 公司）来研发实际的 VLIW 机器，这导致了 Multiflow TRACE 计算机的出现。TRACE 计算机有强大的 VLIW 编译器的支持，这个编译器使用路径调度（trace scheduling，由 Josh Fisher 等人开发）来开发指令级并行。Multiflow 公司理所当然获得了成功，最后安装了超过 100 台机器。更为重要的是，产品化的 TRACE 计算机是第一个平均 IPC 大于 1 的通用单处理器。尽管 TRACE 机器实际获得的 ILP 远远小于 Nicolau 和 Fisher 于 1984 年早期公布的极限值，但是它证明了相对于以前的极限值 2，ILP 还是获得了显著的提高。

1.4.2.3　作用因素

关于ILP极限值的许多研究使用了不同的实验方法，并作了不同的假设。有三个因素对这些实验结果起到了一定的作用：使用的测试程序、假设的机器模型以及所使用的编译技术。每一项研究都采用了自己的一套测试程序集合，并且所选择的测试程序对结果影响极大。最近，SPEC（Standard Performance Evaluation Corporation）测试程序被广泛采用，大多数处理器和计算机系统制造商都提供了系统的SPEC等级。虽然在报告产品的SPEC等级时有着严格的规范（参见www.spec.org），但是研究者们在使用时仍然有很大的差异。另外还有明显的迹象表明，SPEC测试程序仅适合于运行科学和工程应用的工作站，而并不适应其他一些领域的应用，例如商业事务处理和嵌入式实时计算。

关于ILP值极限的第二个关键因素就是那些在研究过程中关于机器模型所做的不同假设。大多数研究都假设一个理想化的模型。例如，cache 存储器通常并没有被考虑进去，或者假设为仅具有一个时钟周期延迟和 100% 的命中率。一些模型假设拥有无限的寄存器文件，机器的规模无限大。通常所有操作和功能部件都假设有一个周期的延迟。其他研究采用更为实际的机器模型，这通常会得出更悲观的结果，但可能没有必要那么悲观。当然，在所使用的指令集体系结构中同样存在着大量的不一致。有些是假想的体系结构，其他的则使用现实中的体系结构。所使用的体系结构也会对实验结果产生强烈的影响。

最后，关于使用的编译器技术所做的假设也有很大差异。许多研究并不包括关于编译器的任何考虑，其他一些研究则假设编译器具有无限的能力。通常，这些研究都使用在实际机器中收集到的动态路径。基于这种路径的模拟结果并不仅仅与测试程序和所选择的体系结构相关，同时也与用于生成目标代码的编译器密切相关。编译器技术对于ILP极限的潜在作用是一个正在进行研究的领域。现在，具有完全功能的编译器和已经用于商业用途的实际编译器之间仍然存在着巨大的差距。人们期望在编译器领域能够出现更多的先进技术。

或许，迄今为止，从这些研究中所得出的最保险的结论就是，ILP的实际极限超出了现代机器可以获得的范围。在这方面，还有更多的、更好的研究空间。任何具体极限的假设都可能是不成熟的。随着更强大高效的微体系结构技术的发展以及更为强大的编译技术的出现，以前所做的假设将不得不加以改变，先前得出的极限也将会向上调整。

1.4.3　指令级并行的机器

相对于其他形式的粗粒度并行，包括多个程序段或者多个计算任务的同时处理，指令级并行可以称为一种“细粒度并行”。为开发 ILP 所设计的处理器称为 ILP 处理器，它是一种典型的在功能部件级具有并行资源的单处理器。Norm Jouppi 在 1989 年对 ILP 处理器进行了分类。ILP 处理器是根据一系列参数进行分类的。

- **操作延迟（OL）** 指一条指令产生结果后使用的机器时钟周期数。所使用的参考指令是能够代表指令集中大多数指令的简单指令。操作延迟就是指这种指令执行时所需的机器时钟周期数。
- **机器并行度（MP）** 指机器支持的可以同时执行的最大指令数目。实际上，我们可以把它看做在任何时候能够同时在流水线中运行的最大指令数。
- **发射延迟（IL）** 指发射两条连续的指令之间所需要的机器时钟周期数。参考指令仍然选用简单指令。在这里，发射是指一条新的指令初始化后进入流水线。
- **发射并行度（IP）** 指在每个时钟周期内可以发射的最大指令数。

在Jouppi的分类方法中，使用标量流水线处理器作为基准处理器。其分类方法还使用了一个通用的4级流水线加以说明。这4级分别是：

1. IF（取指）
2. DE（译码）
3. EX（执行）
4. WB（写回）

EX段为决定操作延迟提供了参考。作为基准的标量流水线处理器，参数定义为OL=1个时钟周期，IL=1个时钟周期。该基准处理器按照如图1.9所示的指令执行机制，每个时钟周期可以向流水线发射一条新指令，每条指令的执行需要一个时钟周期。相应的MP=k，即流水线的级数。在图1.9中，MP=4。IP为每周期一条指令。注意，所有的4个参数都是处理器的静态参数，不考虑与程序执行相关的动态行为。

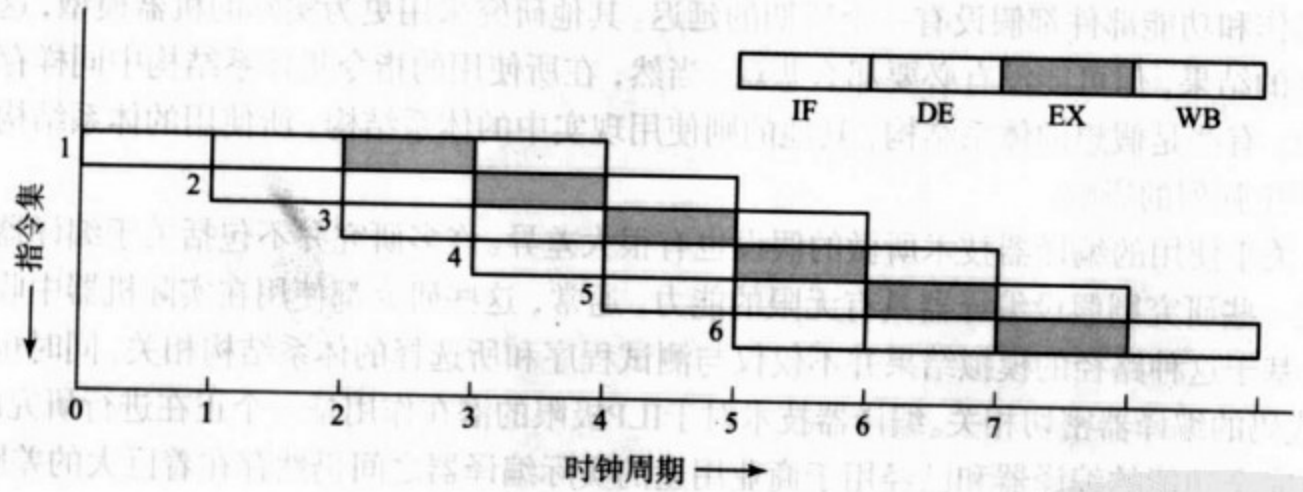


图1.9 基准标量流水线处理器的指令执行过程

当讨论ILP处理器的性能或者加速比时，将使用这个基准处理器作为参考。在本章前面部分我们曾经讨论过加速比，相对于不能将多条指令的处理重叠起来的串行非流水处理器，流水线处理器可以获得一定的加速比。这种形式的加速比则限制在标量处理器领域内的比较上，其重点是相对于（标量的）非流水线处理器，在（标量的）流水线处理器中可以获得吞吐率的提高。在后续章节中，将主要讨论ILP处理器，所涉及的加速比是指ILP处理器的性能与那些作为参考的标量流水线处理器之间的比较。

1.4.3.1 超流水处理器

超流水处理器是相对于基准处理器而定义的，它比基准处理器有着更高的流水度（degrees of pipelining）。在超流水处理器中，处理器的时钟周期比基准处理器短并定义为“次时钟周期”（minor

cycle）。超流水处理器的时钟周期是基准处理器的1/m，或者在一个基准处理器的时钟周期中有m个次时钟周期。超流水处理器的参数定义如下：OL=1 个时钟周期 =m 个次时钟周期，IL=1 个次时钟周期。换句话说，简单指令仍然需要一个基准时钟周期并等于 m 个次时钟周期，但是在执行的时候，处理器可以每过一个次时钟周期就发射一条新指令。因此，IP=1 条指令 / 次时钟周期等于 m 条指令 / 时钟周期，MP = $m \times k$。超流水处理器的指令执行过程参见图 1.10。

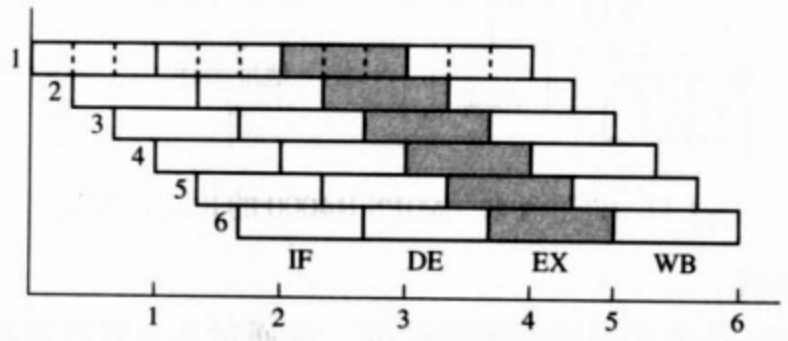


图 1.10　超流水处理器的指令执行过程

超流水处理器的流水度已经超过了简单指令的操作延迟所决定的流水度。实际上超流水处理器将执行段进一步分成多个流水段。一个“流水化程度不够”的处理器不能以指令的执行速度来发射指令。从另一个方面来说，超流水处理器发射指令的速度比执行的速度还要快。对于一个流水度为 m 的超流水处理器，可以在 m 个次时钟周期内执行一条简单指令，通过因子 m，它可以潜在地获得比基准处理器更好的性能。从技术上来说，传统的流水线处理器如果需要多个时钟周期来执行简单指令，那么应当归类为超流水处理器。例如，在 CDC 6600 和 CRAY-1 中，它们执行定点加法的延迟都是 3 个时钟周期，并且每个时钟周期都能够发射新的指令。因此，它们实际上是超流水处理器。

在某种程度上，超流水处理器的分类存在一些人为的因素，因为分类取决于基准时钟周期的选择以及简单操作的定义。超流水处理器的主要特点是某条指令的结果不能被后续的 m–1 条指令所利用。因此，超流水处理器可以简单地看做在定向路径上存在一些限制的深度流水线处理器。在标准的流水线处理器中，隐含的假设是定向路径的源和目的可以分别是任何流水段的输出和输入。如果超流水处理器被看做 $m \times k$ 段的深度流水线处理器，那么某些段的输出就不能看做是定向的源，某些段的输入就不能接收定向的数据。原因是一些需要多个次时钟周期和多个流水段完成的操作是原子操作，不能被数据定向中断。这是流水线处理器和超流水处理器之间最主要的区别。本书中除了本节，在其他地方都没有将超流水处理器作为与流水线处理器有区别的单独一类处理器来特殊对待。

64 位 MIPS R4000 处理器是第一个称为“超流水”的处理器，R4000 的流水线有 8 个物理段，如图 1.11 所示，物理时钟周期时间为 10 ns。然而芯片需要 50 MHz 的时钟输入，并且有一个片上时钟倍频器。因此，R4000 使用 20 ns 作为它的基准时钟周期。可以认为 R4000 的基准流水线为 4 段，基准时钟周期为 20 ns，因此 R4000 的超流水程度为 2，一个基准时钟周期包含两个次时钟周期。在 R4000 中，需要多周期的原子操作是 cache 访问操作。例如，前两个物理段（IF 和 IS）都要访问 I-cache，类似地，DF 和 DS 段都要访问 D-cache。这些都是不能被中断的操作，没有任何定向的数据能够占用 IF 和 IS 段之间的缓冲或者 DF 和 DS 段之间的缓冲。cache 访问在这里将作为一种“简单”操作并且进行了流水化，同时需要两个次时钟周期的操作延迟。整个流水线的发射延迟是一个次时钟周期，也就是说，每隔 10 ns 就可以发射一条新指令。因此，相对于 4 段基准流水线，R4000 可以潜在获得的加速比为 2。

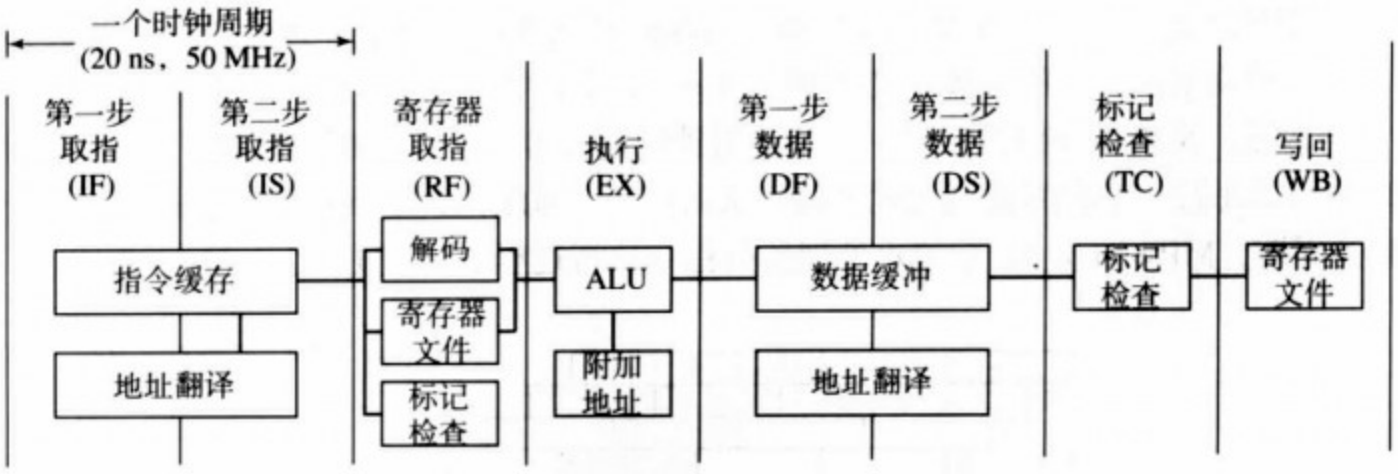


图 1.11 “超流水”MIPS R4000 的 8 段流水线

1.4.3.2 超标量处理器

超标量处理器是基准标量流水线处理器的扩展，它通过 3 个参数来描述：OL=1 个时钟周期，IL=1 个时钟周期，IP =n 条指令 / 时钟周期。处理器时钟周期与基准时钟周期相同，没有次时钟周期。简单的操作都在一个时钟周期内完成。在每个时钟周期内，可以发射多条指令。超标量的度数（superscalar degree）由发射并行度 n 决定，n 是每个时钟周期内可以发射的最大指令数。超标量处理器的指令执行过程如图 1.12 所示。与标量流水线处理器相比，度数为 n 的超标量处理器可以看做是有 n 条流水线或者 n 倍宽度的流水线，即在每个流水段中可以驻留 n 条指令，而不是一条。超标量处理器的 MP = $n \times k$。可以看出，具有相同度数的超流水处理器与超标量处理器具有相同的处理器并行度，可以获得相当级别的性能。

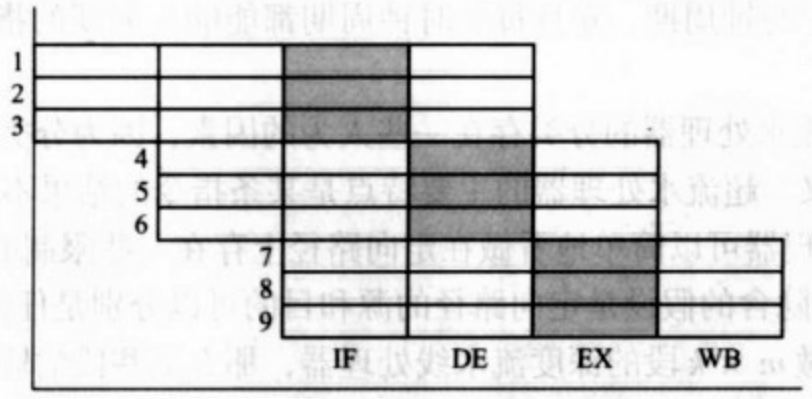


图 1.12 度数 n =3 的超标量处理器的指令处理策略

没有任何理由可以说明超标量处理器为什么不能超流水。如果每个次时钟周期发射 n 条指令，发射延迟将减小到基准时钟周期的 1/m，总的发射并行度或者吞吐率将是每个基准时钟周期 $n \times m$ 条指令。最终处理器的并行度将是 MP = $n \times m \times k$，这里的 n 是超标量的度数，m 是超流水的度数，而 k 是基准处理器的流水线度数。处理器并行度可以看成是 MP = $n \times (m \times k)$，表示超标量处理器有 $m \times k$ 个流水段。这样处理器可以等效地看做是超标量度数为 n、具有 $m \times k$ 段的深度流水线处理器，而没有必要冠以“超标量超流水”处理器这样一个冗长的名称。

1.4.3.3 超长指令字处理器

ILP 的另一类处理器是被 Josh Fisher 称做超长指令字（VLIW，Very Long Instruction Word）的处理器，它与超标量处理器有些类似。这两类处理器的实现意图以及性能目标十分相似，主要的区别在于动态静态界面（DSI）的位置，以及关于哪些通过硬件机制在运行时完成、哪些通过软件在编译时完成的划分。VLIW 处理器的指令执行过程如图 1.13 所示。

和超标量处理器不一样，VLIW 处理器不需要复制 IF 和 DE 段来同时对 n 条独立指令进行取指和译码。在超标量处理器中，究竟是哪 n 条指令将被发射到执行段是在运行时决定的。对于 VLIW

处理器，这样的指令发射决策是在编译时进行的，编译器决定哪 n 条指令将被同时发射到执行段，并将这 n 条指令作为一个超长指令字存放到程序存储器中。

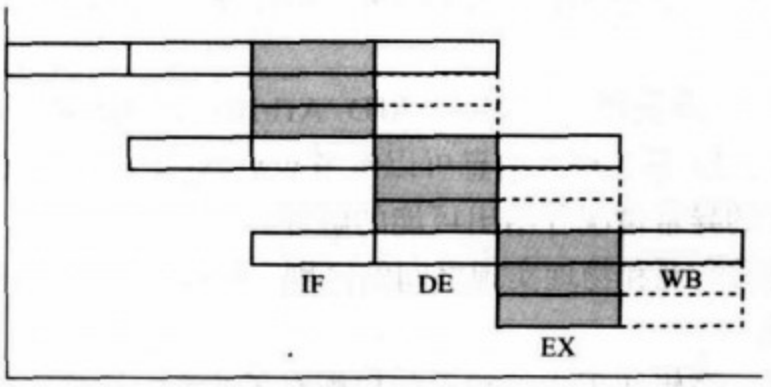


图 1.13　度数为 $n=3$ 的 VLIW 处理器的指令处理策略

超标量处理器和VLIW处理器代表了两种不同的方法，它们的最终目标是相同的，即通过指令级并行处理来提高处理器的性能。两种方法的发展道路及背景各不相同。有人提出这两种方法是相互促进的，应该将它们结合起来。本书主要集中在微体系结构的动态技术上，因此，我们不准备特别关注依赖于编译技术的 VLIW 的特点。

1.5　小结

微处理器对计算机工业产生了空前的冲击。在微处理器30多年的历史中发生的变化是显著的。微处理器现在已经进入了第四个10年。预测在未来的10年中，微处理器能够给我们带来什么影响引起了我们的极大兴趣。

尽管指令集体系结构设计在过去是一种时尚，但现在已经不再是人们特别感兴趣的问题。我们已经学习了许多关于如何设计一个一流的、可扩展的ISA的知识。但是，代码兼容性和软件应用基础对于ISA的寿命更为重要。本章已经详细地说明了任何ISA的不足都可以通过微体系结构技术来克服。并且，随着可移植字节码和动态实时（JIT，Just-In-Time）编译的出现，ISA 的含义以及动态静态界面的位置将会变得更加模糊。

在未来的10年里，微体系结构将会无处不在。当单个芯片的集成度接近10亿只晶体管时，许多传统的（微）体系结构特性，例如存储器子系统、多处理器、输入/输出子系统等，将会变成单片上的问题，因此将成为微体系结构问题。传统的系统级体系结构将成为芯片级设计的一部分。我们很有可能看到集成了多个处理器、cache存储器层次、存储器控制器（甚至可能是存储器）、输入/输出设备以及网络接口设备的芯片。

当接近甚至超过 10 GHz 的时钟速度时，需要重新考虑微体系结构设计的许多基本原则。一个简单的ALU操作可能需要多个时钟周期，一个复杂的分支预测器将需要近10个时钟周期，存储器延迟将达到1000个时钟周期以上，贯穿整个芯片的信号延迟可能有数十个时钟周期。我们现在认为非常先进的流水线技术可能成为相当基础的技术。

未来的微处理器将变成单芯片的计算系统，并且需要开发不同形式的并行机制。这些系统将会超越指令级并行并获得负载程序中的线程级并行（TLP，Thread-Level Parallelism）。或许最重要的将是获得存储级并行（MLP，Memory-Level Parallelism），以便能够处理多个同时发出的访存请求。当存储器延迟比 CPU 的时钟周期时间慢到三个数量级以上时，我们需要寻求更好的办法来扩展存储器带宽，以缓解存储器延迟对整体性能所造成的负面影响。这时主要的挑战将不再是数据操作，而是数据的移动。

1.6 习题

1. 利用因特网上的资源，分别列出在SPECINT2000、SPECFP2000和TPC-C中得分最高的前5个比较结果。
2. 比较两种不同的处理器系列（例如，AMD Athlon和HP PA-RISC）的主频与Graph SPECINT2000之间的关系，相应信息可以参考www.spec.org。对性能随着频率如何变化进行评论，指出其中的异常情况并给出可能的解释。
3. 解释体系结构、逻辑实现和物理实现之间的区别。并解释在公式1.1中，它们是如何与处理器性能联系起来的。
4. 随着硅工艺的发展，逻辑实现中的约束与权衡有了改变，这将影响到动态静态界面（DSI）的位置与定义。解释为什么体系结构刚开始发展的时候，添加一个分支延迟槽（如MIPS体系结构）是比较合理的，但现在却不是很有吸引力。
5. 在很多情况下，逻辑实现中的问题使人们对于指令集体系结构进行权衡时无法做出其他选择。至少举出一个历史上的实现约束，解释为什么CISC指令集在1970年是一个明智的选择。
6. 一个程序的运行时间是由程序的指令数目、每条指令所需的时钟周期数以及时钟频率的乘积决定的。假设在一个类似MIPS的RISC指令集的某个指令集合中，指令分布如下：store指令占15%，load指令占25%，分支指令占15%，整数运算占30%，整数移位占5%、整数乘法占5%。假设load指令需要两个时钟周期，分支需要4个时钟周期，ALU运算需要1个时钟周期，整数乘法需要10个时钟周期，试计算平均CPI。
7. 参数同第6题，假设有一个编译优化将乘法转变成一系列的移位和加法操作。对于第6题的指令集合，有50%的乘法可以转化成移位加法序列，平均长度为3条指令。假设频率固定，计算程序的平均指令数目、CPI以及整体性能加速比。
8. 类似Pentium 4的最新处理器一般不会实现单周期移位。给定第7题的条件，假设由于优化而产生的附加指令有 s =50%是移位指令，移位指令的执行需要4个时钟周期。重新计算程序的平均指令数目、CPI以及整体性能加速比。这个优化是否可取？
9. 给定第8题的假设，计算平衡率 s（附加指令中移位指令的百分比）。即找到相应的 s（如果存在），使得程序性能与没有进行优化（参见第6题）的时候是一样的。
10. 给定第8题的假设，并假设Pentium 4处理器的移位单元有两种实现可供选择：在2 GHz频率下，移位延迟为4个周期；或者在1.9 GHz频率下，移位延迟只有两个周期。假设流水线的其他部分均能运行在2 GHz下，并且2时钟周期延迟的移位单元将使整个处理器的频率降为1.9 GHz。哪种选择会提供更好的整体性能？
11. 使用Amdahl定律，计算向量化比例为85%的程序在分别有4、8、16和32个处理器的系统上运行时的加速比。
12. 通过阅读相关文献［Jouppi and Wall 1988］与［Wall 1991］，比较这两种对ILP极限的研究理论，解释为什么差异这么大。
13. 1995年，IBM AS/400系列计算机从CISC指令集转移到RISC指令集。由于使用了简化的指令集，时钟频率和CPI值得到了极大提高。但是，也是因为这个原因，程序的指令数也增长得很快。给定下列参数，计算这次转变所带来的性能增长。进一步计算平衡时钟频率、平衡CPI以及平衡代码扩展率，假设在计算一个因素时其他两个因素保持不变。

性能因素	AS/400 CISC（IMPI）实际	AS/400 RISC（PowerPC）实际	实际比率	平衡率
频率	50 MHz	125 MHz	2.5	?
CPI	7	3	0.43	?
程序指令数（动态计数）	1000	3300	3.3	?

第2章　流水线处理器

流水线技术是提高系统吞吐率的一项强大的实现技术，并且不需要大量重复设置硬件。20世纪60年代早期的一些高端机器中第一次采用了流水线技术。第一个采用指令流水线的机器是IBM 7030（又称做Stretch计算机）。后来的CDC 6600同时采用了流水线和多功能部件。

到了20世纪80年代，流水线技术成为RISC处理器设计方法中最基本的技术之一。RISC设计方法的大部分技术都直接或间接以高效流水线为目标。从此以后，流水线技术也被有效地应用到CISC处理器的设计中。Intel i486是IA32体系结构中的第一个流水线实现。Digital的VAX和Motorola的M68K的流水线版本在商业上也取得了成功。

流水线技术是当前指令集处理器设计中广泛应用的技术。这一章的重点将放在（标量）流水线处理器的设计。流水线处理器设计中的许多方法和技术，例如用于检测和化解相关的流水线互锁机制，都是超标量处理器设计的基本方法。

当前的趋势是朝着超深度流水线（very deep pipeline）的方向发展。流水线的深度已经从不到10增加到超过20。深度流水是获得高速时钟频率的必要条件，这是提高处理器性能的一个非常有效的方法。有迹象表明，这种趋势还将持续下去。

2.1　流水线基础

这一节讲述提出流水线的动机和设计的基本原则。在流水线的发展史上主要有两种流水线：算术流水线和指令流水线。本书的重点将是指令流水线，下面我们首先分析一个算术流水线的实例。算术流水线能更好地说明流水线设计原则中的一些理想假设。我们称这些理想假设为“流水线理想假设”（pipelining idealism）。正是为了解决指令流水线中理想假设与实际考虑之间的矛盾，才使得流水线处理器的设计变得非常有趣。

2.1.1　流水线设计

下面将介绍流水线设计的基本概念、提出的动机以及流水线技术的局限，还描述了Peter Kogge提出的一个理论模型，这个模型从硬件设计的角度描述了一个最佳流水线。

2.1.1.1　提出的动机

提出流水线最初的动机是，提高系统吞吐率的同时几乎不增加硬件的开销。系统吞吐率或者称为带宽，利用系统单位时间内完成的任务数目加以衡量，它刻画了系统的性能。对于一个系统，如果它在一段时间内完成一个任务，那么吞吐率P就是$1/D$，D是任务的延迟或者称为系统完成任务的延迟。如果要使用同一系统完成多项同样的任务，就可以使用流水线技术来提高系统的吞吐率。每项任务的延迟将保持不变或者只增加一点。

流水化意味着将系统分割成许多段，并且在段与段之间增加适当的缓冲。这些段以及段与段之间的缓冲就构成了流水线。系统原来的运算量被分割成k个分量，分别由流水线的k个段来处理。一旦前面的任务通过了流水线的第一段，新的任务就可以进入流水线。因此，每隔D/k个时间单位就可以启动新的任务，而不是每隔D个时间单位。其中，k是流水线的段数。同时，流水线

中 k 个分量的计算是重叠的。这里，假设以前的延迟 D 被均匀地分摊到流水线中 k 段的处理当中，并且增加的缓冲没有带来任何额外的延迟。如果总的任务数目很多，流水系统的吞吐率可能达到非流水系统吞吐率的 k 倍。只需要简单地在流水线的段之间增加一些缓冲，就能潜在地提高性能，这就是流水线设计思想主要的吸引力所在。图 2.1 是一个 k 级流水系统可能提高 k 倍吞吐率的示意图。

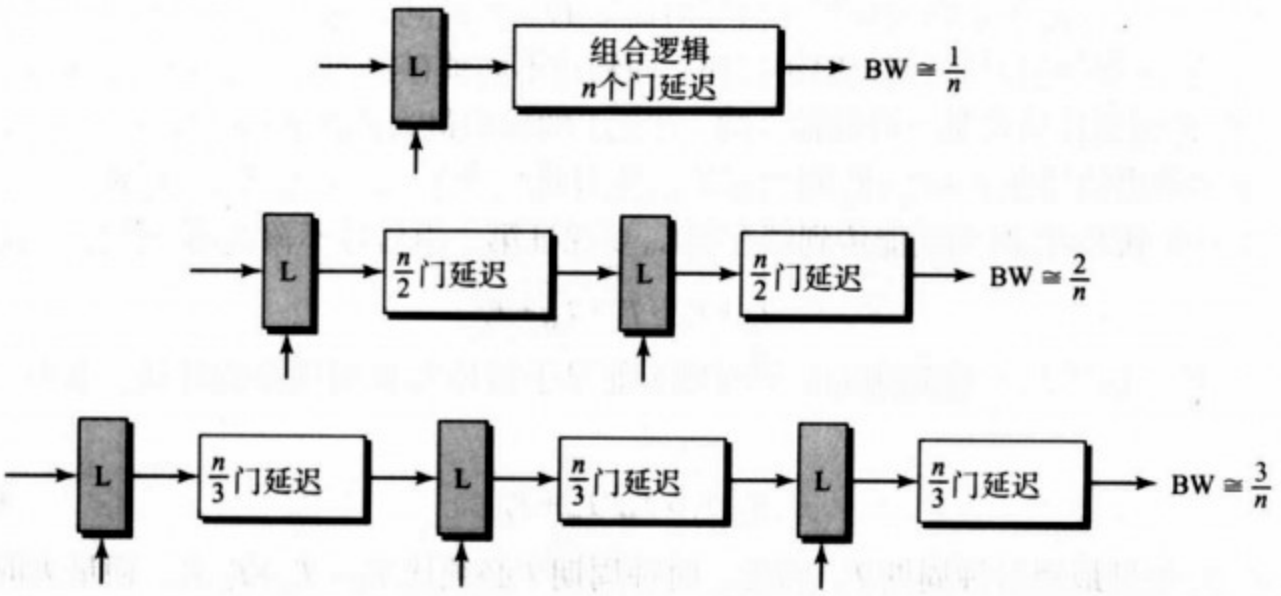


图 2.1　一个 k 级流水系统的吞吐率可能提高 k 倍

目前为止，假设段间缓冲不会带来任何额外的延迟，而这是不现实的。图 2.2（a）显示了 IBM 360/91 中的流水化乘法部件中使用的 Earle 锁存器，它用于缓存段间加法器的进位。在 Earle 锁存器中，当时钟 C=1 时，输出 Z 的值等于输入 D 的值。当时钟较低时，将进入锁存区，输出 Z 的值不随输入 D 的改变而改变。要保证正确的锁存，必须要有足够的锁存时间。中间的与门将保证锁存操作的正确完成，该与门的作用是消除了一个潜在的冲突，该冲突是由于多个信号同时发生改变而引起的。上下两个与门可能同时向相反的方向改变，如果没有中间的与门，或门的输出可能产生一个虚假的脉冲。Earle 锁存器可以防止产生这种错误信号。并且，Earle 锁存器可以集成到其他逻辑当中，这样就不会引入额外的延迟。图 2.2（b）显示了门锁存器是如何合并到一个组合逻辑电路的最后两级与门和或门中的，合并之后没有引入任何额外的延迟。图 2.2（b）所示的电路和图 2.2（a）中的电路逻辑功能完全相同，并且没有因为引入锁存器电路而带来额外的两级门延迟。但是，由于门的扇入增加了一个，可能导致总的门延迟略有增加。

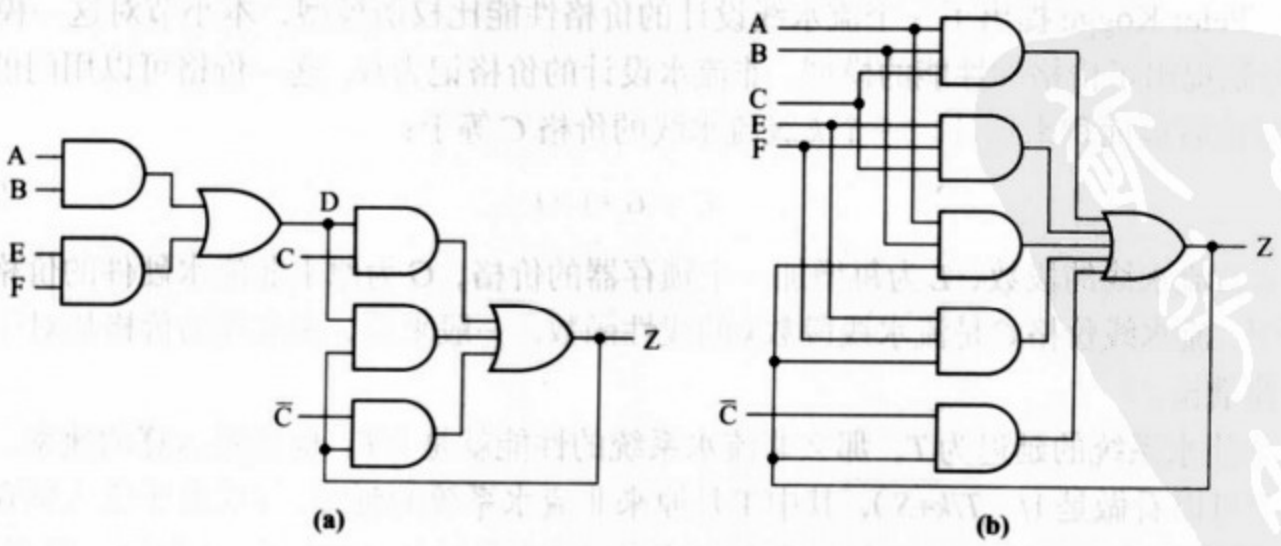


图 2.2　采用 Earle 锁存器不会增加门延迟：（a）跟在组合逻辑后的 Earle 锁存器；（b）采用在组合逻辑中的 Earle 锁存器

2.1.1.2 局限性

既然流水化设计所获得的性能与流水线的深度（流水线的段数）成正比，因此似乎最好的设计就是尽量增加流水线的段数。但是，由于时钟的限制，流水线划分的段数有一个物理局限。

流水线的每一段可以看做一个组合逻辑F加上一个锁存器L，信号通过F之后在L处锁存。定义T_M为通过F的最大传输延迟，即通过最长信号路径的延迟；T_m为通过F的最小传输延迟，即通过最短信号路径的延迟；T_L为正确建立时钟信号所需要的额外时间，包括必要的建立和保持时间，以保证正确的锁存，同时还包括可能的时钟扭曲，即到达不同锁存器的时钟沿的最大时间差。假设第一组信号X_1在T_1时刻作用在某一级的输入端，那么F的输出最迟将在T_1+T_m时刻有效，要保证L的正确锁存，F的输出信号在$T_1+T_m+T_L$时刻必须仍然有效。当第二组信号X_2在T_2时刻作用在F的输入端时，最快只要到T_2+T_m时刻就能传到锁存器L。要保证第二组信号不覆盖第一组信号，必须满足：

$$T_2 + T_m > T_1 + T_M + T_L \qquad \text{（公式2.1）}$$

该公式说明，信号X_2可能到达的最早时刻不能早于信号X_1正确锁存的时刻。该不等式可以重写为：

$$T_2 - T_1 > T_M - T_m + T_L \qquad \text{（公式2.2）}$$

其中T_2-T_1就是最短时钟周期T。因此，时钟周期T必须比$T_M - T_m + T_L$大，而最大时钟频率不能超过$1/T$。

基于以上分析，有两个因素限制了时钟频率，一个是组合逻辑的最长和最短传输延迟之差，即$T_M - T_m$；另一个是为正确建立时钟信号所需要的额外延迟，即T_L。如果所有信号的传输路径是一样长的，那么第一个因素就可以消除，而这可以通过延长最短路径来实现。因此，可以认为$T_M - T_m$接近0。第二个因素是为了锁存流水段的结果而引入的。锁存结果的延迟包括信号通过一个反馈回路的传播延迟，以及信号在回路中的建立时间。另一个引起T_L增大的因素是最坏情况下的时钟扭曲。时钟信号到达各个锁存器的时刻可能不尽相同，这是由于时钟信号的产生以及在整个系统内的分布造成的。在一个完全同步的系统当中，必须考虑最坏情况下的时钟扭曲。总的来说，一个同步系统的流水线深度取决于锁存需要的最短时间，以及时钟分布中一些不确定的延迟。

2.1.1.3 权衡

时钟限制最终决定了流水线的深度，除了这一限制以外，如果考虑价格，即流水线的费用开销，最佳设计通常也不会采用最大的流水线深度。在流水线系统的硬件设计中，必须考虑价格和性能的权衡。Peter Kogge提出了一个流水线设计的价格性能比权衡模型，本小节对这一模型进行了总结，并分别提出了价格和性能的模型。非流水设计的价格记为G，这一价格可以用门的数量、晶体管的数量或者硅面积来估计。一个k级流水线的价格C等于：

$$C = G + k \times L \qquad \text{（公式2.3）}$$

其中k为流水线的段数，L为每增加一个锁存器的价格，G为原来非流水硬件的价格。基于这一价格模型，流水线价格C是流水线段数k的线性函数。一般来说，流水线的价格相对于流水线的深度呈线性增长。

假设非流水系统的延迟为T，那么非流水系统的性能就是$1/T$，也就是运算的速率。而流水系统的性能P可以看做是$1/(T/k+S)$，其中T是原来非流水系统的延迟，S是由于引入锁存器而带来的延迟。假设原来的延迟T可以均匀地分布到流水线的k段当中，$(T/k+S)$是与每一段相关的延迟，因而就是流水线的时钟周期。因此，$1/(T/k+S)$就等于时钟频率，同时也是流水线的吞吐率。所以，流水线的性能如下所示：

$$P = \frac{1}{\left(\frac{T}{k} + S\right)} \quad \text{（公式2.4）}$$

注意，P 是 k 的非线性函数。

根据以上给出的价格和性能模型，价格性能比的表达式为：

$$\frac{C}{P} = \frac{G + k \times L}{\frac{1}{\left(\frac{T}{k} + S\right)}} \quad \text{（公式2.5）}$$

该表达式还可以记为：

$$\frac{C}{P} = LT + GS + LSk + \frac{GT}{k} \quad \text{（公式2.6）}$$

图 2.3 给出了两组 G, L, T 和 S 的采样值。

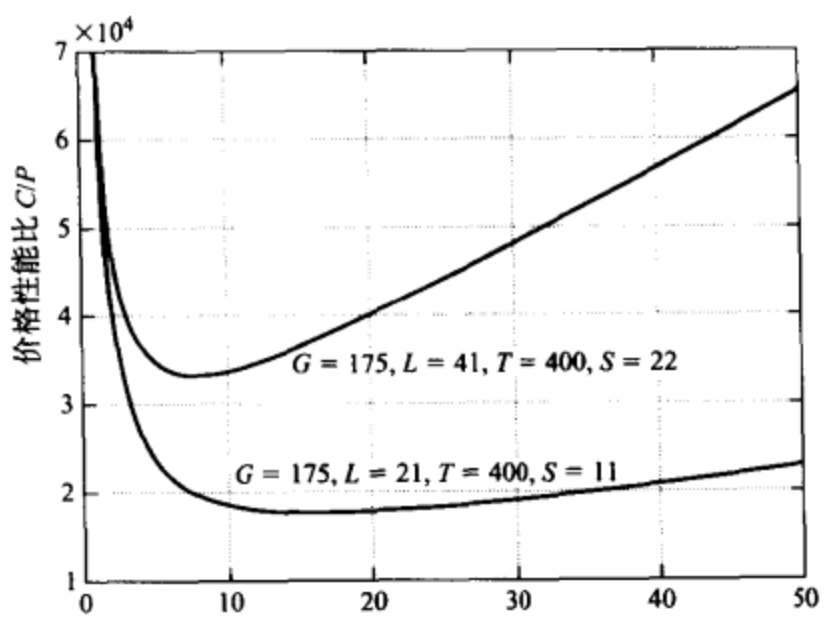


图 2.3　流水设计的价格性能比权衡模型

公式2.6表明价格性能比是 k 的函数。对该函数取 k 的一阶导数并令其为0，可以求得价格性能比取最小值时的 k 值，如公式 2.7 所示，同时这个 k 值也就是流水线的最佳深度。

$$k_{opt} = \sqrt{\frac{GT}{LS}} \quad \text{（公式2.7）}$$

上式给出了 k 的最佳值。一个 $k < k_{opt}$ 的流水线系统，可以称为“不够完全流水化”，即说明还可以进一步实现流水化并增加流水线的深度，这样在性能上获得的好处将超过价格的提高。另一方面，$k > k_{opt}$ 的流水线系统可以认为是“过度流水化”的，这时候增加流水线深度而提高的性能由于价格的增加而被抵消。上述权衡模型完全是基于硬件系统设计来考虑的，没有考虑流水线的动态行为，也没有考虑执行的程序。我们将在后续章节中讨论这些问题。

2.1.2　算术流水线的实例

流水线的类型主要分为两种：算术流水线和指令流水线。虽然本章的重点是指令流水线，但我们还是先来看一个算术流水线的实例。算术流水线能很清楚地说明流水线的作用，而且不涉及到指令流水线中的一些很复杂的问题。这些问题将在本章的后面几节中讨论。

2.1.2.1　浮点乘法

本小节用一个流水化的浮点乘法部件作为算术流水线的实例。这个有点“过时”的设计来自于 Shlomo Waser 和 Mike Flynn 所著的一本经典教材。虽然使用的是 1980 年的技术，但是它仍然不失

为一个能很好地说明算术流水线的实例。该设计采用64位的浮点格式，指数e（8位）采用“加-128（excess-128）”表示方式，尾数m（57位，包含隐藏位）采用带隐藏位的“符号-大小（sign-magnitude）”表示方式。

采用的浮点乘法算法如下：

1. 检查是否有操作数为0。如果有，结果立刻被置为0。
2. 将两个首数（指数的实际位模式）相加，并校正为“加-128”的形式，即$e_1+(e_2-128)$。
3. 对两个尾数m_1和m_2进行定点乘法运算。
4. 对尾数乘积进行规格化，包括将结果左移1位，指数减1（规格化之后，尾数就不会以0开始）。
5. 四舍五入。将第一个保护位加1（这一位位于尾数最低位的右边）。这是有效的四舍五入。如果尾数溢出，那么尾数必须右移1位并且指数加1，以保持规格化的结果。

图2.4给出了非流水设计的浮点乘法器的功能单元图，输入锁存器存放了两个要进行相乘的操作数。在下一个时钟周期，这两个操作数的乘积将存放到输出锁存器。

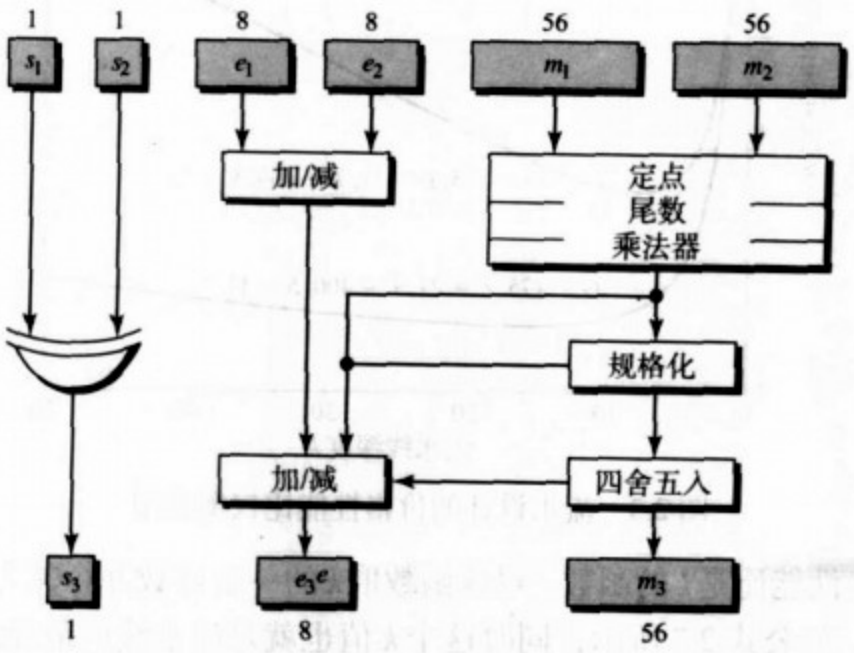


图2.4 非流水的浮点乘法器

定点尾数乘法器是其中最复杂的模块，它包括3个子模块，分别是部分积产生、部分积化简和最终化简。我们可以得到每个子模块的硬件复杂性（集成电路芯片的数目）和传输延迟（ns）。

- **部分积产生** 使用8 × 8的硬件乘法器可以同时产生多个部分积。要产生所有的部分积，需要34个8 × 8乘法器，其中的延迟为125 ns。
- **部分积化简** 所有的部分积产生之后，必须进行求差或者求和。一个称为（5, 5, 4）计数器的求和电路可以用于将两列5位化简成一个4位的和。一个（5, 5, 4）计数器可以用一个$1K \times 4$的ROM来实现，它的延迟是50 ns。要化简所有的部分积需要三级（5, 5, 4）计数器，所以一共需要72个$1K \times 4$的ROM，总的延迟是150 ns。
- **最终化简** 当所有的部分积被化简成两个部分积之后，可以使用一个快速先行进位（CLA）加法器来进行最终的化简并输出最后的结果。完成这一步工作需要16个4位CLA加法器和5个4位CLA单元。总共是21块集成电路芯片，总的延迟是55 ns。

尾数部分还需要两个模块：一个移位器来完成规格化（2块芯片，20 ns延迟），一个增量器来完成四舍五入（15块芯片，50 ns延迟）。指数部分有一个加减模块，需要4块芯片，而其延迟并不

重要，因为这个模块不在关键延迟路径上。输入输出锁存器分别需要17块和10块芯片。非流水设计的各个模块总的芯片数目和关键延迟时间如表2.1所示。

根据表2.1中的数据，非流水化的浮点乘法器需要175个芯片，时钟频率可以达到2.5 MHz，时钟周期是400 ns。就是说，非流水设计的吞吐率可以达到2.5 MFLOPS（每秒百万次浮点运算）。

表2.1　非流水浮点乘法器的芯片数目和各模块的关键延迟

模块	芯片数目	延迟
部分积产生	34	125 ns
部分积化简	72	150 ns
最终化简	21	55 ns
规格化	2	20 ns
四舍五入	15	50 ns
指数部分	4	---
输入门阀	17	---
输出门阀	10	---
合计	175	400 ns

2.1.2.2　流水化的浮点乘法器

非流水的浮点乘法器可以通过流水化来提高吞吐率。在这个例子中，假设子模块没有流水化，也就是说，划分流水段的粒度只到子模块这个层次。下面，将讨论关键延迟路径上的各个（子）模块的延迟，如表2.1的第3列所示。其中可以看到，部分积化简子模块的延迟最长，为150 ns，该延迟决定了流水段的延迟。关键路径上的5个（子）模块可以划分成3段，每一段的延迟分别是125 ns（部分积产生）、150 ns（部分积化简）和125 ns（最终化简，规格化以及四舍五入）。这个3级流水线的设计如图2.5所示。

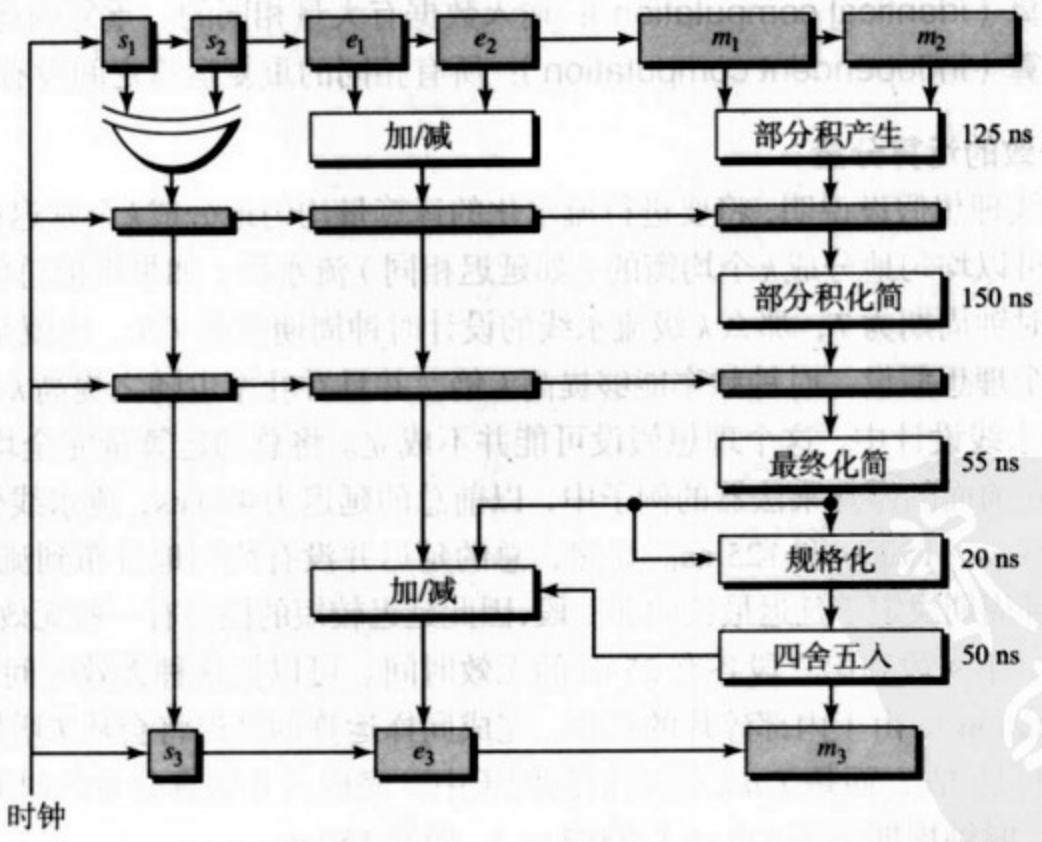


图2.5　流水化的浮点乘法器

在确定流水设计的实际时钟频率时，必须考虑时钟要求。假设流水线级之间使用边沿触发寄存器来缓存中间结果，那么必须在150 ns的级延迟上增加17 ns的时钟边沿到寄存器输出的延迟，以及5 ns的建立时间。这导致时钟周期最小为172 ns。因此，新的流水化设计中的时钟频率可以达

到5.8 MHz，而不是以前的2.5 MHz。吞吐率的绝对值提高了2.3。然而，需要注意的是，每次执行乘法运算的时间也有所增加，即从400 ns增加到516 ns。

流水化设计惟一需要增加的硬件是边沿触发寄存器，用于缓存流水线级之间的结果。在以前非流水化设计中175块芯片的基础上，还需要增加82块芯片。如果使用芯片数目作为硬件复杂度的衡量标准，流水化设计需要257块芯片，硬件复杂度提高了45%。然而，在性能上却有130%的提高。显然，这种3级流水线的浮点乘法器设计要优于以前非流水化的设计。

这个例子假设板级实现时采用了已有的部件，如果采用现在的芯片技术，整个设计可以很容易地做在一块芯片里。虽然浮点乘法器的板级设计可能已经过时，但我们的目的是通过这一个已经发布了的、带有真实延迟和硬件价格参数的设计，来简单说明流水线的作用。实际上，图2.3中上面一条曲线的参数就取自这个例子。

2.1.3 流水线理想假设

回想一下，k级流水线设计的目的是想要将吞吐率提高k倍，其原理如图2.1所示。然而，在前面的例子中，具有3级流水线的浮点乘法器的吞吐率只提高了2.3倍。吞吐率的提高不到3倍的主要原因是，只有在理想情况下，k级流水线的设计才能提高k倍吞吐率。理想情况主要基于3条假设，我们称之为“流水线理想假设”（pipelining idealism）。理解流水线理想假设是正确评估流水线设计的关键。实际流水线与流水线理想假设之间的矛盾使得流水线的设计更富有挑战性，而正是弥补实际和理想之间差距的各种解决方案，组成了流水线设计的主要技术。流水线理想假设包括：

1. **一致的运算分量**（uniform subcomputation）：要完成的整个运算量可以均匀地分成延迟一致的若干分量。
2. **重复的运算**（identical computation）：输入数据有大量相同的、重复的运算。
3. **独立的运算**（independent computation）：所有相同的重复运算之间没有相关性。

2.1.3.1 一致的运算分量

第一条流水线理想假设说明，将要进行流水化的运算量均匀地分成k个延迟相同的分量。这意味着以前的设计可以均匀地分成k个均衡的（如延迟相同）流水段。如果以前总的运算延迟，也就是非流水设计的时钟周期为T，那么k级流水线的设计时钟周期就是T/k，也就是流水线中每一段的延迟。基于这个理想假设，时钟频率能够提高k倍，并且吞吐率也随之提高k倍。

在实际的流水线设计中，这个理想假设可能并不成立。将总的运算量完全均匀地分成若干分量是不可能的。在前面的浮点乘法器的例子中，以前总的延迟为400 ns，流水线分为3段，每一段的延迟分别是125 ns、150 ns和125 ns。显然，总的延迟并没有均匀地分布到流水线的3段当中。由于流水线的时钟周期决定于延迟最长的那一段，因此延迟较短的段会有一些无效或者失效的时间。在我们的例子中，第一段和第三段各有25 ns的无效时间。可以把这种无效时间称为“内部碎片”（internal fragmentation）。由于内部碎片的存在，完成同样运算的延迟将会从T增加到T_f，并且流水线的时钟周期不再是T/k，而是T_f/k。在我们的例子中，完成三个运算分量的时间是450 ns，而不是以前的400 ns；时钟周期不是133 ns（400/3 ns），而是150 ns。

还有一个内在的假设，即流水段之间引入的缓冲不会带来额外的延迟，并且流水线的时钟信号也不会引入额外的延迟。同样，这一假设在实际当中也是不能成立的。在相应的例子中，为了保证时钟信号的正确建立，有22 ns的额外延迟，因此总的时钟周期是172 ns。而理想情况下的时钟周期是133 ns。172 ns和133 ns之间的差异导致了实际性能的提高达不到3倍。

第一条流水线理想假设主要包含了以下两种观点：在将以前的运算量分成若干分量时，没有引入无效时间；段间缓冲的引入及时钟没有带来额外的延迟。在芯片级的设计中，可以采用一些锁存器，比如Earle锁存器可以使流水线时钟产生的延迟尽可能地小。因此如何将总的运算量均匀地分到各个流水段中构成了流水线设计的第一个挑战。我们的目标是，尽可能使各个流水段保持均衡，以减少内部碎片。由于各个流水段不能完全均衡内部碎片是不满足第一条流水线理想假设的主要原因，这将成为一种开销，并导致k级流水线设计的吞吐率提高达不到理想的k倍。

2.1.3.2　重复的运算

第二条流水线理想假设说明，流水线不断地重复执行相同的运算。这意味着在多组输入数据中，有些运算是相同的并且这些运算要反复执行。每次执行时，流水线提供的分量执行顺序都是一样的。对于浮点乘法器的例子，将包含大量的浮点数对要进行相乘，每一对操作数都要通过相同的3级流水线。这一假设主要说明，每一次的重复运算都会使用流水线中的所有段。对于我们的例子这显然是成立的。

浮点乘法器满足这条假设，是因为该流水线只完成了一个功能，即浮点乘法。如果所设计的流水线要完成多个功能，这一假设就不满足了。例如，设计一条既可以计算加法又可以计算乘法的算术流水线。在多功能流水线中，单独一项功能可能并不需要所有的流水段，而可能由流水段的不同子集来完成不同功能。由于输入数据以同步方式通过流水线，有些数据可能不需要某些流水段，当它们通过这些段的时候，将什么也不执行。这种没有使用或者空闲的流水段带来了另一种形式的流水线失效，可以称之为“外部碎片”（external fragmentation）。与内部碎片类似，外部碎片也是一种流水线开销，应该尽量减小。浮点乘法器的例子则不存在外部碎片。

第三条流水线理想假设认为所有的流水段总是被充分利用。除了说明不存在外部碎片之外，这条理想假设还说明要处理的数据是大量的。第一个数据达到流水线的最后一段需要k个周期，这k个周期称为流水线的“填充时间”。最后一个数据进入流水线的第一段之后，还需要k个周期排空流水线。在填充和排空过程中，不是所有的流水段都处于工作状态。如果有大量的输入数据要进行处理，可以使流水线的填充和排空时间只占总时间的很小一部分。因此，所有流水段都可以认为总是处于工作状态。实际上，在前面浮点乘法器的例子中，5.8 MFLOPS的吞吐率就是基于这个假设计算出来的。

2.1.3.3　独立的运算

第三条流水线理想假设说明，流水线要处理的运算是彼此独立的。这意味着，所有同时驻留在流水线中的运算是不相关的，也就是说，任何两个运算之间不存在数据或者控制相关。这一假设保证流水线处于“流动”（streaming）的工作方式，即后面的运算不会因为存在相关性而等待前面运算的完成。对于浮点乘法器的例子中，这一假设是成立的。如果有多对操作数要进行相乘，一对操作数的相乘不会依赖另一个乘法的结果。这些操作数可以按照“流动”的方式进行处理。

这一假设对于一些流水线可能不成立。后面的运算可能需要前面运算的结果，而这两条运算可能同时驻留在流水线中。如果后面的运算已经进入到需要结果的流水段，而这时前面的运算还没有到达产生结果的流水段，那么后面的运算必须在那一段等待。这种等待称为“流水线停顿”（pipeline stall）。如果某一运算停顿在流水线的某一段，所有后续的运算可能都要被迫停顿。流水线停顿不可避免地导致某些段处于空闲状态，这可以看做是一种动态的外部碎片，因此会导致流水线吞吐率降低。如果相关是不可避免的，那么在设计流水线的时候，就要尽量减少流水线停顿的次数。

2.1.4 指令流水线

前面三条流水线是流水线设计的理想假设。在大多数情况下，算术流水线的实际情况与理想假设相差不大。然而，对于指令流水线，实际情况与理想假设之间的差距就要大一些。而连接这一差距的桥梁正是指令流水线设计的趣味性和挑战性所在。在设计流水线处理器时，这三条假设成为三个主要的挑战。

这里只是简单地介绍了这三个挑战，我们将在2.2节中讲解流水线处理器设计时进行深入的讨论。这三大挑战同时也提供了一条探索流水线处理器设计技术的思路。

2.1.4.1 指令流水线设计

在设计指令流水线时，三条流水线理想假设成为设计的目标。一条指令的处理过程就是要流水化地运算，它必须被划分成多个尽量一致的子运算，以获得尽量均衡的流水线。处理一条指令的时间延迟称为“指令周期”，每一段的时间延迟决定了“机器周期”。指令周期是一个逻辑概念，它是指指令的执行周期。一个程序包含多条指令，程序的执行相当于相应运算的反复执行。机器周期是一个物理的概念，它包含数字逻辑电路中存储单元的时钟等，同时它也是流水线的时钟周期。

可以把前面的浮点乘法器看做是一个非常简单的处理器，它只有一条指令，也就是浮点乘法。指令周期包含一条浮点乘法的执行，如图2.6（a）所示。根据功能单元的划分，这一运算可以很自然地划分为5个子运算。

1. 部分积产生（125 ns）
2. 部分积化简（150 ns）
3. 最终化简（55 ns）
4. 规格化（20 ns）
5. 四舍五入（50 ns）

按照流水化的目标，可以将最后三个子运算合并成一个子运算，这就得到了3级流水线，如图2.6（b）所示。将图2.6（a）中的指令周期映射到图2.6（b）中的3个机器周期，就得到一个3级流水线的设计。可以将指令周期称为体系结构（逻辑的）原语，它是由指令集体系结构定义的，而将机器周期称为机器（物理的）原语，它是在处理器的微体系结构中定义的。图2.6（b）的流水化设计是图2.6（a）定义的体系结构的一种实现。

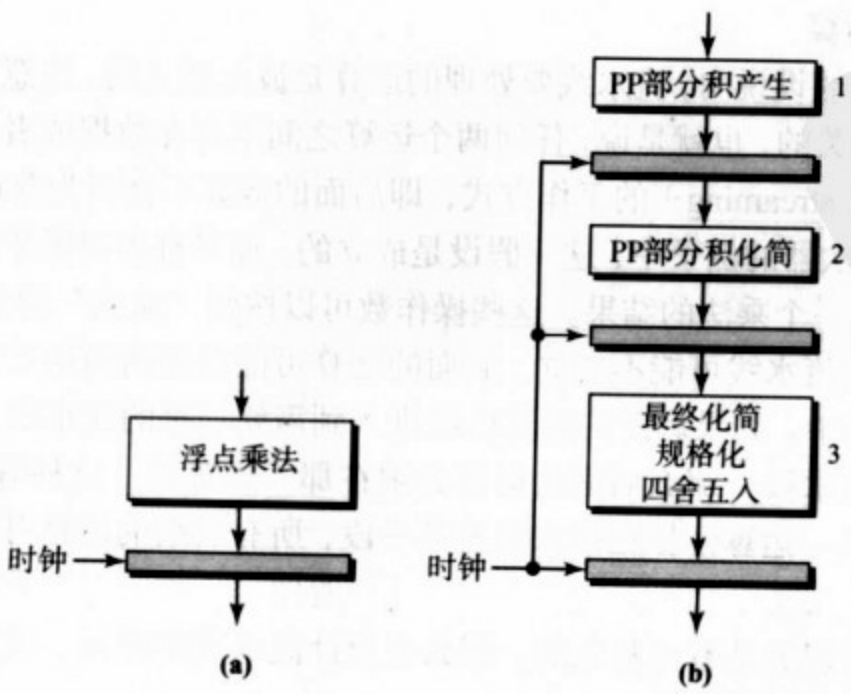


图2.6 指令周期与机器周期的简单说明

指令流水线设计的主要任务可以看做是将逻辑指令周期映射到物理机器周期。换句话说，就是将指令周期所对应的运算分成一个子运算的序列，由流水线的各段运行。要有效地进行这种分割或者映射，必须考虑三条流水线的理想假设。

一致的运算分量　我们将指令周期分解成多个机器周期的过程称为“量化流水段”（stage quantization）。在进行量化流水段时，要尽量减小内部碎片。如果在量化流水段时考虑不充分，就会引入内部碎片，从而影响流水线的利用率。第一条流水线理想假设带来了指令流水线设计的第一个挑战，称之为流水线的均衡。流水线越均衡，内部碎片越少。

重复的运算　与单一功能的算术流水线不一样的是，指令流水线天生就是多功能流水线，即它必须能够处理多种不同的指令类型。不同的指令类型需要的子运算序列稍微不同，所对应的硬件资源也有所不同。指令流水线设计的第二个挑战就是，连接或者整合不同指令类型对不同资源的需求。流水线应能够支持所有的指令类型，同时尽量减少每种指令类型的无效或者空闲的机器周期，也就是减少外部碎片。

独立的运算　算术流水线处理的是数组数据，它们一般都不相关。与算术流水线不同的是，指令流水线所处理的指令彼此不是完全独立的。因此，指令流水线必须要有一套内部机制，用来检测指令间发生的相关并保证指令相关不会导致错误。指令相关可能会导致流水线停顿，从而引起失效。回想一下，前面提到过，流水线停顿是一种动态形式的外部碎片，它会降低流水线的吞吐率。因此指令流水线设计的第三个挑战就是尽量减少流水线停顿。

2.1.4.2　指令集体系结构的影响

在正式讨论指令流水线设计的三个主要挑战之前，简单地考虑一下指令集体系结构（ISA，Instruction Set Architecture）对于指令流水线的影响，将很有启发意义。下面将依次考虑三条流水线理想假设。

一致的运算分量　使流水段保持均衡的第一个挑战说明，必须保证分量的延迟是一致的。考察一条指令处理过程中的所有运算分量，其中一定有一个分量的延迟最长，并且这个分量不容易再分解成细的运算分量。在流水线处理器设计中，这样的关键分量是对存储器的访问。由于处理器和存储器的速度差异，存储器访问将成为关键的运算分量。为了使指令流水线效率更高，应该对寻址模式进行优化，并且应该采用能够跟上处理器速度的快速 cache。

重复的运算　第二个挑战是将不同的指令类型对于资源的需求统一起来，这是RISC体系结构的主要目的之一。降低指令类型的复杂性和多样性可以降低整合不同指令类型任务的难度。复杂的寻址模式不仅需要额外的访存，而且增加了资源需求的多样性。将所有这些资源需求整合到一条指令流水线当中是一件相当困难的事情，并且所得到的流水线对于很多指令其效率很低。这些指令将会产生外部碎片，从而降低了流水线的利用率。对于一个RISC体系结构的流水线而言，资源整合要简单一些并且得到的流水线效率更高，外部碎片更少。

独立的运算　第三个挑战是尽量减少由于指令的相关而引起的流水线停顿，这可能是流水线处理器设计中最吸引人的地方。要正确地完成操作，指令流水线必须检测并化解指令相关。复杂的寻址模式，尤其是与存储器访问有关的模式，会因为存储器地址标识符的问题而导致相关检测非常困难。一般来说，寄存器相关比较容易检测，因为寄存器在指令中是明确指出来的。简单而且对称的指令格式便于进行译码和相关检测。相关的检测和化解可以由编译器静态完成，也可以在运行时由硬件动态完成。在编译和运行时分别进行怎样的处理决定于动态静态界面（DSI，Dynamic-Static Interface）的定义。DSI的定义将导致许多有趣而微妙的权衡。在进行权衡时，必须将编译器、（微）体系结构和微处理设计紧密结合起来。

2.2 流水线处理器设计

在设计指令流水线或者流水线处理器时，三条流水线理想假设体现了设计中的三个主要挑战。流水线处理器设计中的主要任务就是缩小理想假设与实际情况的差异。三条流水线理想假设对应的三个主要设计挑战是：

1. 一致的运算分量 =>保持流水段均衡
2. 重复的运算 =>统一指令类型
3. 独立的运算分量 =>尽量减少流水线停顿

这三个挑战将在后面的三个小节中依次讨论，它们为如何介绍指令流水线技术提供了一个很好的框架。所有关于流水线处理器的设计技术，都可以看做是为解决这三个挑战而进行的努力。

2.2.1 保持流水段均衡

在流水线处理器设计中，要流水化的运算就是每个指令周期所要进行的工作。一个典型的指令周期可以根据功能划分成以下 5 个基本运算分量：

1. 取指（IF，Instruction fetch）
2. 译码（ID，Instruction decode）
3. 取操作数（OF，Operand(s) fetch）
4. 执行（EX，Instruction execution）
5. 存储（OS，Operand store）

典型的指令周期从指令的取指开始，然后进行译码，以决定这条指令完成什么操作。一条指令通常带有一个或者多个操作数，这些操作数存放在寄存器或者存储器中，这取决于所使用的寻址模式。一旦所需要的操作数已准备好，就可以执行这条指令所规定的操作。将执行所产生的结果保存起来之后，指令周期就结束了。结果可以保存在寄存器中，也可以保存在存储器中，这同样也是由寻址模式决定的。下一条指令将重复整个运算分量序列。在这5个基本运算分量的执行过程中，会产生一些副作用。通常这些副作用会改变机器状态。之所以将这种改变称为“副作用”，是因为这些改变没有在指令定义中明确规定。所采用的ISA对5个基本运算分量的实现复杂性和延迟时间将产生巨大的影响。

2.2.1.1 量化流水段

分割指令周期的一种很自然的方法是按照5个基本运算分量来划分，即将每一个运算分量对应到一个流水段，从而得到一个5级指令流水线，如图2.7所示。我们称这条流水线为基本指令流水线，简称GNR（generic）流水线。在GNR流水线中，逻辑指令周期将映射到5个物理机器周期中。机器周期与指令周期之比为5，这个值反映了流水化的程度以及流水段的粒度。

量化流水段的目的是将指令周期分割到均衡的流水段中，以尽量减少内部碎片的数目。量化流水段可以从自然的功能划分开始，例如5个基本运算分量。多个延迟较小的分量可以合并成一个新的分量，以获得更均衡的流水线。例如，在图2.6（b）的流水线设计中，可以将浮点乘法运算的三个运算分量（最终化简，规格化和四舍五入）合并成一个分量。同理，典型指令周期的5个基本运算分量中的一些分量也可以合并起来，以获得更均衡的流水线。例如，如果某种指令集体系结构采用定长指令、简单寻址模式和正交的指令格式，那么IF和ID段相对于其他段的设计就比较简单，因此这两个分量可以合并成一个新的分量。用延迟时间来衡量，所得到的4个分量要更均衡一些。

基于这 4 个分量实现的 4 级指令流水线如图 2.8（a）所示。实际上，MIPS R2000/R3000 处理器的流水线就将 IF 和 ID 分量进行了合并。这一方法的要点是以延迟最长的分量为参考，尽量将小于这个延迟的分量合并起来，从而使总的延迟与参考延迟相差不多。这样做的结果是，增加了机器周期的粒度，降低了流水化程度。

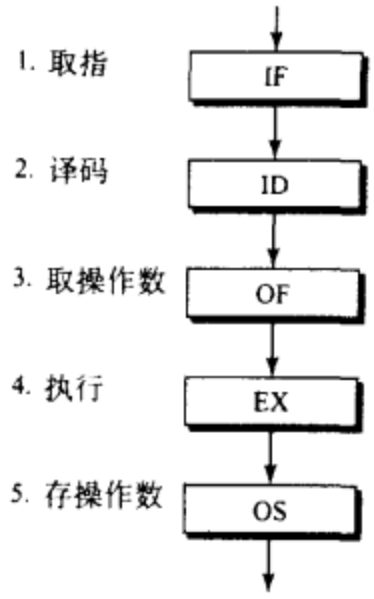


图 2.7　5 级 GENERIC（GNR）指令流水线

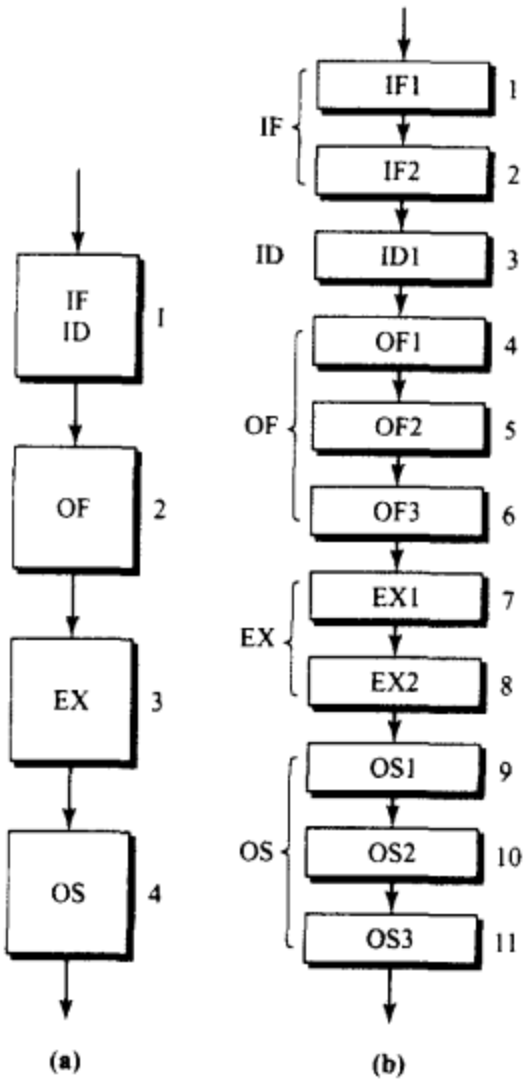


图 2.8　（a）一个 4 级指令流水线的例子；（b）一个 11 级指令流水线的例子

均衡流水线还有一种相反的方法，不是将具有较短延迟的分量合并起来，而是将一个延迟很长的运算分量分割成多个延迟较短的分量。该方法以延迟最短的分量为参考，将长延迟的运算分量分割成更小的分量，使得各个分量的延迟与参考分量的延迟差不多。这将导致机器周期的粒度更细、流水化的程度更高。例如，如果一种 ISA 采用了复杂寻址模式，OF 和 OS 分量都要访存，使得这两

个分量的延迟很长，那么可以进一步分成多个分量。另外，EX分量中要执行的一些操作可能相当复杂，同样可以进一步分成多个分量。图2.8（b）显示了一条11级的流水线。OF和OS分量都被细化到3个流水段中，而IF和EX分量则被细化到两个流水段中。ID分量则作为均衡流水线的参考。

这里提出了量化流水段的两个方法：将多个分量合并成一个分量以及将一个分量进一步分成多个分量。在进行指令流水线设计时，两个方法可以结合起来使用。在前面的讨论中已经看到，指令集体系结构对量化流水段有着重大的影响。不管在哪种情况下，量化流水段的目标都是尽量减少内部碎片的开销。例如，假设5个基本运算分量总的延迟为280 ns，图2.8（a）中的4级流水线设计和图2.8（b）中的11级流水线设计的机器周期分别是80 ns和30 ns。相应地，4级流水线总的延迟为320 ns（80 ns × 4），11级流水线总的延迟为330 ns（30 ns × 11）。而最初的总延迟为280 ns，这之间的差距就是因为存在内部碎片。因此，4级流水线的内部碎片是40 ns（320 ns~280 ns），11级流水线的内部碎片是50 ns（330 ns~280 ns）。可以认为4级流水线的效率要比11级流水线的效率更高一些，因为前者的内部碎片带来的开销小一些。当然，11级流水线的吞吐率是非流水设计的9.3（280 ns/30 ns）倍，而4级流水线的吞吐率是非流水设计的3.5倍（280 ns/80 ns）。从这两个设计中都可以看出，由于内部碎片的存在，使得11级流水线和4级流水线在性能的提高上都达不到理想情况下的11倍和4倍

2.2.1.2 硬件要求

在大多数实际工程中，设计目标并不是尽可能达到最好的性能，而是达到最好的性价比。因此，除了单纯地提高指令流水线的吞吐率（性能），还必须考虑硬件要求（价格）。一般来说，流水化程度越高，所需要的价格也就是硬件要求就越高。显然，在流水线段与段之间增加缓冲的同时也增加了价格。在2.1.1.3节中提出的模型中已经看到，流水化的程度存在一个临界点，超过这个临界点后，由于段间缓冲的开销增加，所获得的性能就打了折扣。在深度流水设计中，除了缓冲的开销，还有其他更重要的硬件要求。

关于如何评价一条指令流水线的硬件要求，首先要记住是，对于一个k级指令流水线，在最坏的情况下或者（从性能的角度看）最好的情况下，流水线中可能同时存在k条指令。在这种情况下，流水线的每一段都有一条指令，k条指令处于指令周期的不同阶段。因此，整个流水线必须要有足够的硬件来支持k条指令的同时处理。硬件要求包含三方面的内容：每一段进行控制和完成数据操作的逻辑；支持多个流水段同时访问的寄存器文件端口；支持多个流水段同时访存的存储器端口。

首先考察图2.8（a）中的4级指令流水线。假设在一个load/store型的体系结构中，一条典型的寄存器–寄存器指令在第1段中读出两个寄存器操作数，在第4段中将结果写回寄存器中。一条load指令在第2段从存储器中读出数据，一条store指令在第4段向存储器写回数据。将所有流水段的要求结合起来，表明寄存器文件需要两个读端口和1个写端口，数据存储器接口必须能够在一个机器周期中同时处理1次读请求和1次写请求。另外，第1段在每个周期中都要读指令存储器并进行取指。如果使用的是统一的（指令和数据）存储器，那么存储器在每个时钟周期必须能够支持2次读请求和1次写请求。

对于图2.8（b）的11级指令流水线，可以进行同样的分析。为了适应慢速的指令存储器，IF分量被进一步分割并细化成两个流水段，即IF1和IF2段。IF1进行取指的初始化，IF2完成取指。虽然取指要使用两个时钟周期，但由于实现了流水化，当第一条指令进入IF2中进行取指时，第二条指令可以在IF1中初始化。这表明指令存储器每个时钟周期必须能够同时支持两个请求，分别来自IF1和IF2。同样，OF和OS分量分别都被细化成3个流水段，这表明在任何时候最多可能有6条

指令正在访存。因此，数据存储器每个时钟周期必须同时支持6个彼此独立的、无冲突的访问，这就需要有一个6端口的数据存储器。而且，如果指令存储器和数据存储器使用的是同一个存储器，就需要有一个8端口的存储器。这样的多端口存储器实现起来非常昂贵，而比较经济的解决方案是，使用多体交叉存储器模拟真实的多端口存储器，但通常不能保证在任何时候同时访问都是无冲突的。

随着流水化程度也就是流水线深度的增加，流水线的硬件资源会显著地增加，其中最突出的是寄存器文件和存储器端口数目的增加，以获得更高的访问并行度。另外，为了适应漫长的存储器访问延迟，访存分量必须进行流水化。然而，在实现上，访存的流水化如果超过两个时钟周期就会变得相当复杂，因此必须对频繁的无冲突访存进行折中。

2.2.1.3　指令流水线实例

本节将介绍两个产品化的流水线处理器是如何对流水段进行量化的。MIPS R2000/R3000 RISC处理器采用了5级指令流水线，如图2.9（a）所示。MIPS体系结构是load/store型体系结构。IF和ID分量将合并到IF段，每个时钟周期要读一次存储器（I-cache）。OF分量在RD和MEM段都有操作。ALU指令只有寄存器操作数，取操作数在RD段完成并且要读两个寄存器。对于load指令，取操作数需要访问存储器（D-cache）并在MEM段完成。MEM段是流水线中惟一能访问D-cache的流水段。OS分量在MEM和WB段完成。store指令一定会访问D-cache，因此在MEM段完成，而ALU指令和load指令在WB段将结果写回寄存器文件。

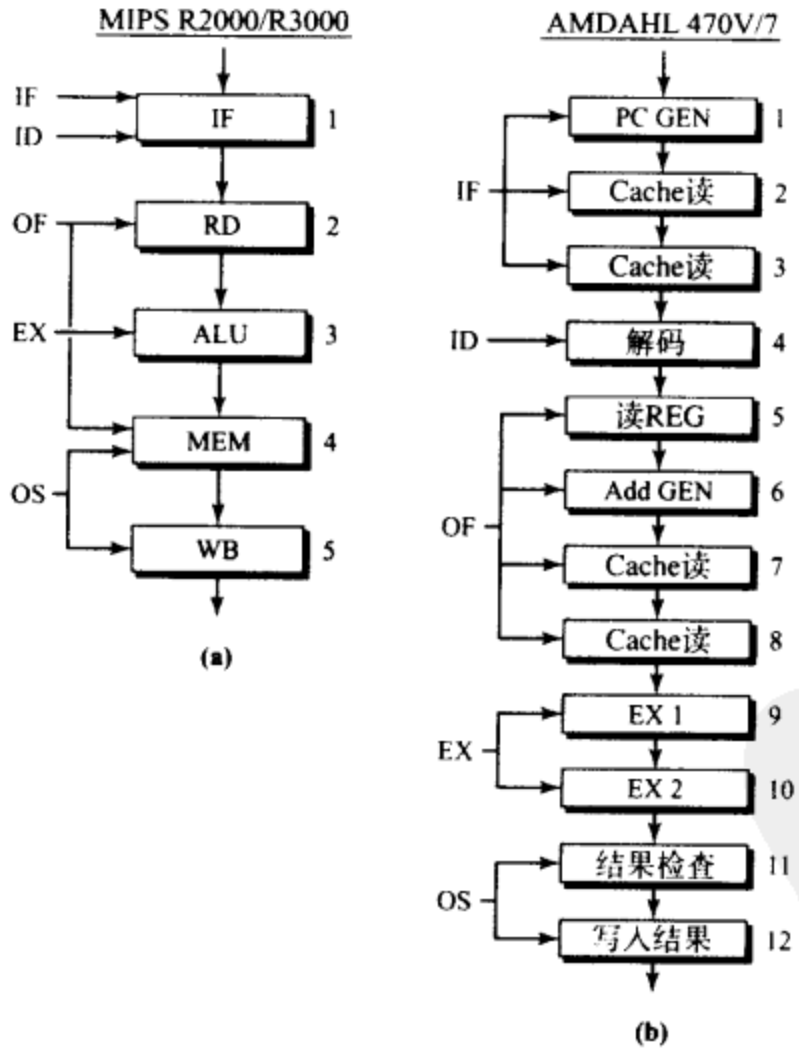


图2.9　两个商业处理器的指令流水线：（a）MIPS R2000/R3000的5级流水线；（b）AMDAHL 470V/7的12级流水线

MIPS处理器通常采用分离的指令和数据cache。每个时钟周期内，R2000/R3000必须同时支持一次I-cache读请求，一次D-cache读请求（load指令）或者写请求（store指令）。对I-cache的访问在IF段完成，对D-cache的访问在MEM段完成。注意由于采用了分离的cache配置，I-cache和

D-cache 都不需要多端口。否则，如果指令和数据都存放在一个 cache 里，就需要双端口来提供支持。寄存器文件每个时钟周期有两次读请求（RD 段）和一次写请求（WB 段），因此至少要有两个端口。

图 2.9(b)显示了 AMDAHL 470V/7 的 12 级指令流水线。IF 分量在前面 3 个流水段完成，由于体系结构要求支持复杂的寻址模式，OF 分量被细化成 4 个流水段。EX 和 OS 分量都被细化成两个流水段。12 级流水线的第 1 段用于计算下一条指令的地址；第 2 段完成 cache 访问的初始化；第 3 段将指令从 cache 中读到指令部件 I-unit；第 4 段进行指令译码；第 5 段读取两个通用寄存器，这些寄存器用做地址寄存器；第 6 段计算出操作数在存储器中的地址；第 7 段完成 cache 访问的初始化并准备读取存储器操作数；第 8 段将操作数从 cache 中读到 I-unit 中，同时也完成寄存器操作数的读取；第 9 段和第 10 段是执行部件 E-unit 中的两个运算段；第 11 段对计算结果进行错误检查；最终结果在第 12 段被存放到目的寄存器中。

这个 12 级的流水线在每个时钟周期必须支持两次寄存器读（第 5 段），1 次寄存器写（第 12 段），还有 4 次 cache 读请求（第 2 段、第 3 段、第 7 段、第 8 段）。该流水线处理器的存储器子系统显然要比 MIPS R2000/R3000 复杂得多。

目前流水线处理器的设计趋势是向着更高的流水程度和更深的流水线深度发展。这样得到的流水线级粒度更小，时钟频率更高。第一代 RISC 处理器的流水线通常只有 4 到 5 级，而现在超过 10 级是很平常的事情。还有一个趋势是采用多条级数不等的流水线，这是超标量处理器设计中的课题，我们将在第 3 章中讨论。

2.2.2 统一指令类型

第二条流水线理想假设则认为流水线反复进行同样的运算。对于大多数指令流水线，重复完全相同运算的理想假设并不成立。指令流水线重复处理的指令具有不同的指令类型。虽然指令周期不断重复，但每次可能处理的是不同类型的指令。不同的指令类型有不同的资源需求，并且各个运算分量的顺序也可能不一样。指令流水线要能满足不同的需求，必须提供所有指令类型所需要的所有运算分量的超集。单独一种指令类型可能不需要流水线中的所有流水段。对于单独一种指令类型，不需要的流水段将成为某种形式的失效或开销，在 2.1.3.2 节中，称这种失效或开销为流水线的外部碎片。统一指令类型的目的，也是针对第二条流水线理想假设的挑战，就是尽量减少所有指令类型的外部碎片。

2.2.2.1 指令类型的分类

要完成一次运算，计算机必须完成以下三个基本任务：

1. 算术操作
2. 数据移动
3. 指令定序

处理器在处理指令的过程中完成这三个基本任务。算术操作包括对指定的操作数执行算术和逻辑运算，这是整个运算中最突出的部分，所以经常和整个运算等同起来。一个处理器能够支持大量的算术操作类型。数据移动负责将操作数和结果在存储地址之间进行移动。通常存储器地址是分层次的并有专门的指令来进行数据移动。指令排序负责指令顺序的确定。通常一个程序中的运算包含很多条指令，通过执行一个指令序列来完成所需要的运算。这个指令序列，或者叫程序流，可以由指令自身来确定。

如何将这三个基本任务分派给 ISA 中的各种指令，是指令集设计中的关键问题。可以制定一条非常复杂的指令，完成所有的三个基本任务。在一个典型的、水平微码的（horizontally microcoded）

机器中，每一条微指令（microinstruction）都有一些域来指明这三个基本任务。在比较传统的指令集体系结构，如 CISC（Complex Instruction Set Computer）体系结构中，许多指令能完成一种以上的基本任务。

受 RISC 体系结构的影响，最近的大多数指令集体系结构都有一些共同的特征。这些体系结构包括 Hewlett-Packard 的 Precision 体系结构，IBM 的 Power 体系结构，IBM/Motorola 的 PowerPC 体系结构，还有 Digital 的 Alpha 体系结构。这些现代 ISA 倾向于采用定长指令、正交的指令格式、load/store 型体系结构和简单寻址模式。这些体系结构中的大多数都能与指令流水线很好地协调起来。在大多数情况下，本书的例子和说明都采用这种典型的 RISC 体系结构。

在一个典型的现代 RISC 体系结构中，指令集采用一种专门的指令类型来完成三个基本任务中的一个，也就是说，每一种指令类型只完成三个基本任务中的一个。根据三个基本任务，可以将指令分成三种类型：

1. **ALU 指令**　完成算术和逻辑运算
2. **load/store 指令**　完成数据在寄存器及存储器间的移动
3. **分支指令**　控制指令执行的顺序

ALU 指令所执行的算术逻辑操作严格地针对寄存器操作数进行，只有 load 和 store 指令能访问数据存储器。load/store 指令和分支指令都采用比较简单的寻址模式，通常只支持带偏移的寄存器间接寻址模式。对于分支指令，还支持 PC 相对寻址。在下面关于三种指令类型的详细规范中，还假设使用了指令 cache（I-cache）和数据 cache（D-cache）。

三种指令类型的语义可以根据指令类型所要完成的运算分量的序列来规定。这个语义规范可以从 5 个基本运算分量（2.2.1 节）着手，然后进一步的细化。最终，这些分量将指定用于硬件实现的寄存器传输顺序。为方便起见，可以将 ALU 指令进一步划分成整数和浮点指令。在表 2.2 中，我们对 ALU 指令的语义进行了规定。

表 2.2　ALU 指令类型规范

基本运算分量	ALU 指令类型	
	整数指令	浮点指令
IF	- 取指令（访问 I-cache）	- 取指令（访问 I-cache）
ID	- 解码指令	- 解码指令
OF	- 访问寄存器文件	- 访问 FP 寄存器文件
EX	- 执行 ALU 操作	- 执行 FP 操作
OS	- 回写寄存器文件	- 回写 FP 寄存器文件

在 load/store 体系结构中，load 和 store 指令是惟一能访问数据存储器的指令。load 指令将数据从存储器移动到寄存器；store 指令将数据从寄存器移动到存储器。load 和 store 指令的语义规范见表 2.3，其中假设只有一种寻址模式，即带偏移的寄存器间接寻址。这种寻址模式将一个寄存器的内容与定义在指令立即数域的偏移相加并得到有效地址。

比较表 2.2 和表 2.3 中的规范，可以看到，ALU 和 load/store 指令类型所需要的运算分量的序列有点类似，但不完全相同。ALU 指令不需要生成存储器地址。另一方面，load/store 指令要生成有效地址，但不需要明确的算术或者逻辑操作，仅仅将数据在寄存器和存储器之间移动。甚至 load 指令与 store 指令之间也有微小的区别。对于 load 指令，OF 基本分量细化为三个分量，包括读寄存器文件获得基地址、生成有效地址和访问存储器。而对于 store 指令，OS 基本分量由两个分量组成，包括生成有效地址和将寄存器操作数写入存储器。这里假设基地址和寄存器操作数都是由 OF 分量访问寄存器文件获得的

表 2.3 load/store 指令类型规范

基本运算分量	load/store 指令类型	
	load 指令	store 指令
IF	- 取指令（访问 I-cache）	- 取指令（访问 I-cache）
ID	- 解码指令	- 解码指令
OF	- 访问寄存器文件（基地址） - 生成有效地址（基地址 + 偏移量） - 访问（读）内存（访问 D-cache）	- 访问 FP 寄存器文件（寄存器操作数及基地址）
EX	-	-
OS	- 回写寄存器文件	- 生成有效地址（基地址 + 偏移量） - 访问（写）内存（访问 D-cache）

最后，表2.4给出了无条件跳转和条件分支指令的分量序列。分支指令采用了与load/store指令类似的寻址模式。另外，还支持PC 相对寻址模式。在这种寻址模式中，将当前程序计数器的内容与偏移量相加获得分支（或者跳转）的目标地址。通常这个偏移量可以是正值也可以是负值，表示向前分支或向后分支。

比较表2.2、表2.3和表2.4中三种主要指令类型的规范，我们可以看到，三种指令类型的初始化分量比较类似，然而后面的分量有些差异。例如，ALU 指令不访问数据存储器，因此不需要生成存储器地址。另一方面，load/store 指令和分支指令都需要有效地址生成分量。load/store 指令要访问数据存储器，而分支指令要提供目标指令的地址。我们还可以看到，对于条件分支指令，除了要生成有效地址以外，还必须对分支条件进行检查。可以简单地查看上一条指令产生的状态位，或者将寄存器操作数的算术运算作为分支指令执行的一部分，或者是检查一个特定寄存器的值。

表 2.4 branch 指令类型规范

基本运算分量	branch 指令类型	
	Jump（无条件）指令	条件 branch 指令
IF	- 取指令（访问 I-cache）	- 取指令（访问 I-cache）
ID	- 解码指令	- 解码指令
OF	- 访问寄存器文件（基地址） - 生成有效地址（基地址 + 偏移量）	- 访问寄存器文件（基地址） - 生成有效地址（基地址 + 偏移量）
EX	-	- 计算分支条件
OS	- 利用目标地址更新程序的计数器	- 若条件为真，利用目标地址更新程序的计数器

根据以上指令语义的规范，可以确定三种主要指令类型的资源需求。虽然这三种指令类型在取指和译码方面有一些共同的地方，但是指令类型之间还是存在差异。这些语义上的差异将导致不同的资源需求。

2.2.2.2 资源需求的整合

统一不同的指令类型主要是将不同的资源需求有效地整合到一条指令流水线中去，使得这条流水线适合于所有的指令类型。整合的目标是，尽量减少流水线所需要的资源总数，同时尽量提高流水线中所有资源的利用率。通常，资源的整合的过程包含以下三个步骤：

1. 分析每种指令类型的运算分量序列，确定相应的资源需求。
2. 找出指令类型之间的共性，合并相同的运算分量，以使用相同的流水段。
3. 如果在不违背指令语义的情况下存在一定的灵活性，可以改变或者重新排列分量的顺序，以便进一步的整合。

可以将这一过程应用到表2.2、表2.3和表2.4中规定的指令类型中，以便进一步的说明。为简单起见，没有考虑浮点指令和无条件跳转指令。ALU，load，store和分支指令类型的规范总结如图2.10所示。这4种指令类型所需要的4个运算分量序列取自表2.2，表2.3和表2.4，并总结于图2.10左侧的4列中。现在我们由上至下实施统一的过程，检查4个分量的序列和要求提供支持的硬件资源。这个过程产生了指令流水线各段的定义。

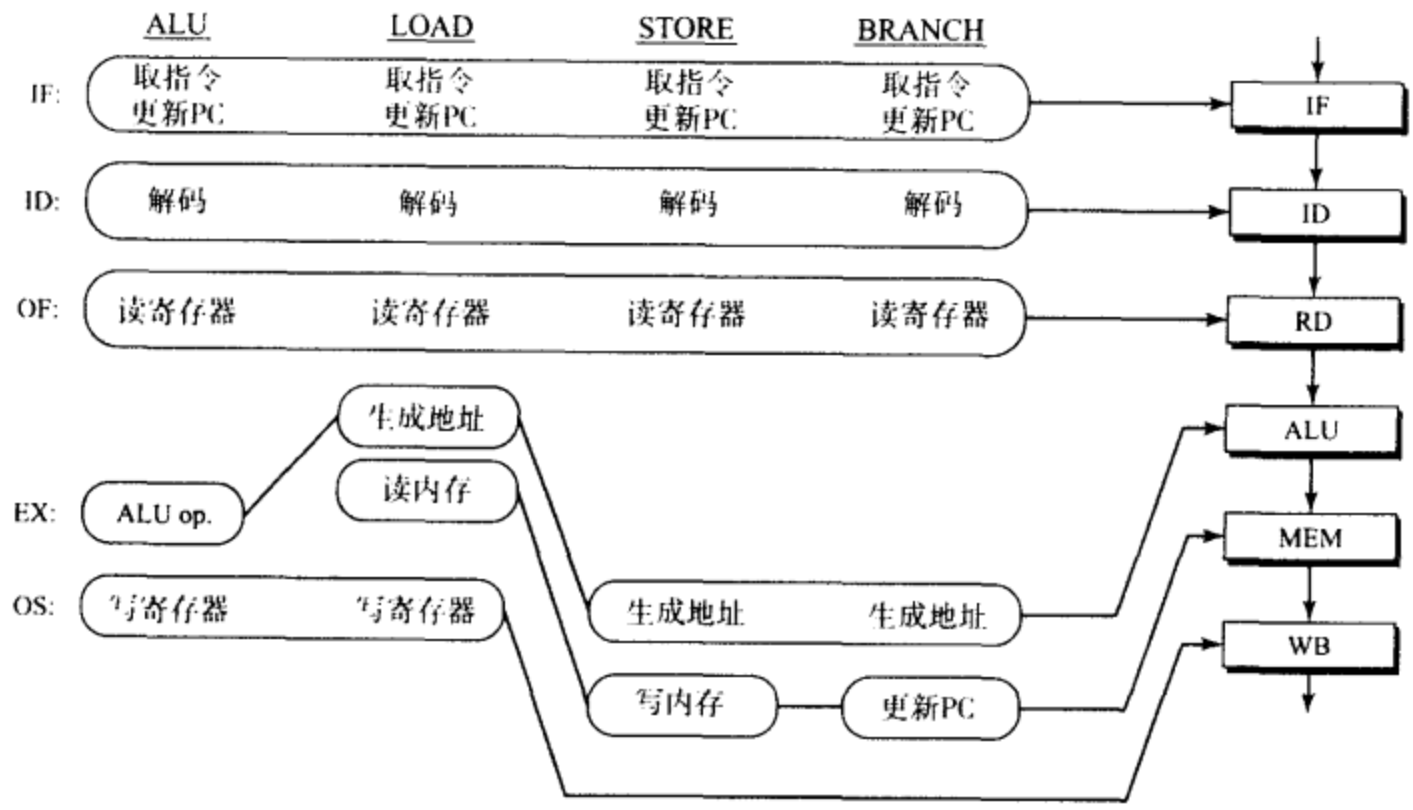


图2.10　ALU, load, store和分支指令类型的规范总结

所有4种指令类型都有相同的IF和ID分量，因此，4种指令类型的前两个运算分量可以很容易地合并起来，并用来定义流水线的前两段，即IF和ID段，从而完成取指和译码的工作。

所有4种指令类型的OF分量都要读寄存器文件。ALU指令访问两个寄存器操作数；load指令和分支指令访问一个寄存器并获得基地址；store指令访问一个寄存器并且获得操作数，访问另一个寄存器以获得基地址。在4种情况下都要访问一个或两个寄存器，这些相似的分量可以合并成流水线的第3段，称为RD段并从寄存器文件读取最多两个寄存器操作数。因此寄存器文件在每个时钟周期内必须支持两个独立的并行读操作。

ALU指令要求有一个ALU功能单元来完成必要的算术和逻辑运算。虽然load，store和分支指令不需要执行这样的操作，但是它们需要生成有效地址来访存。可以看出，地址生成可以由ALU功能单元来完成。因此，这些分量可以合并到流水线的第4段，称为ALU段并包含一个ALU功能单元，完成算术逻辑运算或者有效地址的生成。

load和store指令都需要访问数据存储器。因此，要为这个分量分配一段，即流水线的第5段，称为MEM段。

ALU和load指令都要向寄存器文件写回结果，这是它们的最后一个运算分量。ALU指令将寄存器操作的结果写入目标寄存器，load指令将访存的数据写入到目标寄存器。ALU指令不需要访存，因此，理论上向目标寄存器回写这一过程可以在ALU段之后立即进行。然而，为了与load指

令的寄存器回写分量统一起来，ALU指令的寄存器回写分量向后推迟了一段，在流水线的第6段进行，称为WB段。这将导致ALU指令在MEM段产生一个空闲周期。这是一种外部碎片并将给流水线带来一些效率损失。

对于条件分支指令，必须在更新程序计数器之前确定分支条件。因为ALU功能单元用于有效地址的生成，它就不能用来完成分支条件的判断。如果分支条件的判断只是检查一个寄存器的值，判断它是否为0、正数或负数，那么只需要一个简单的比较器。这个比较器要在读出参考寄存器的值以后加入流水线，而参考寄存器的值在RD段读出，所以最早能加入比较器的段是ALU段。因此，假设条件分支发生，最早能用分支目标地址更新程序计数器的段是MEM段，也就是在ALU段中计算出目标地址并确定分支条件之后。

通过对不同指令类型的资源需求进行整合，可以得到一条6级指令流水线，如图2.10右侧所示。这条指令流水线将作为一条典型（TYP）指令流水线，并在本章的后续部分用做说明的工具。除了ALU指令有一个空闲周期（MEM）外，store指令和分支指令也会带来一些外部碎片。store指令和分支指令都不需要向寄存器写回结果，因此在WB段就处于空闲状态。总的来说这条6级指令流水线的效率是比较高的。load指令用了全部6段流水线，其他三种指令类型使用了6段中的5段。

在将不同指令类型统一到一条指令流水线中的时候，将包含三个优化目标。第一是尽量减少所有指令类型所需要的资源总量，这有点类似于求取所有不同资源需求的最小公倍数。第二个目标是在不同的指令类型下，尽量提高所有流水段的利用率。换句话说，就是尽量减少每种指令类型带来的空闲周期，因为这将产生外部碎片，从而引起流水线效率的降低和吞吐率的降低。第三个目标是尽量减少每种指令类型的总延迟。因此，如果某种指令类型的空闲周期不可避免，但在如何安排空闲周期上具有灵活性，那么比较好的方法是将它放到流水线的末尾。这将使指令提前完成，从而减少指令总的延迟。

2.2.2.3 指令流水线的实现

在6级TYP指令流水线（图2.10）中，最多可能有6条不同的指令同时驻留在流水线中，分别处于一个流水段中。寄存器文件每个时钟周期要支持两次读请求（RD段中的指令）和1次写请求（WB段中的指令）。I-cache每个时钟周期要支持1次读请求。如果没有分支指令的中断，IF段将不断增加程序计数器的值并从I-cache中顺序取出下一条指令。D-cache每个时钟周期要支持1次读或者1次写。只有MEM段要访问D-cache，因此，任何时候流水线中只有一条指令能访问数据存储器。

图2.10中的流水线只是6级TYP指令流水线的逻辑表示，仅仅表示了流水线中6个段的时序。TYP指令流水线的实际物理结构如图2.11所示，这是TYP指令流水线的功能模块图。在该图中，段与段之间的缓冲进行了明确的标记。两段之间的逻辑缓冲可能包括分布在整个图中的多个物理缓冲。在逻辑上顺序通过6级流水线的路径只有一条，而实际上可能包括多条物理路径。每条指令在流水线中的处理将沿着这些物理路径进行。

图2.11中的TYP指令流水线的物理组织比实际看起来要复杂一些。为了便于读者理解，我们先考察流水线与寄存器文件以及存储器子系统的接口。假设采用分离的cache，即指令cache与数据cache分开，并且指令cache（I-cache）与数据cache（D-cache）都只有一个端口。与大多数标量流水线处理器类似，TYP流水线的存储器子系统接口简单而有效。IF段访问I-cache，MEM段访问D-cache，如图2.12所示。I-cache每个时钟周期能取一条指令，I-cache失效将导致流水线停顿。在流水线的MEM段，load（store）指令将读（写）D-cache。注意，这里假设访问D-cache和I-cache的延迟都在一个时钟周期以内。但是，随着cache变得更大，处理器逻辑的流水程度将会加深，cache保持一个时钟周期的延迟也更加困难。

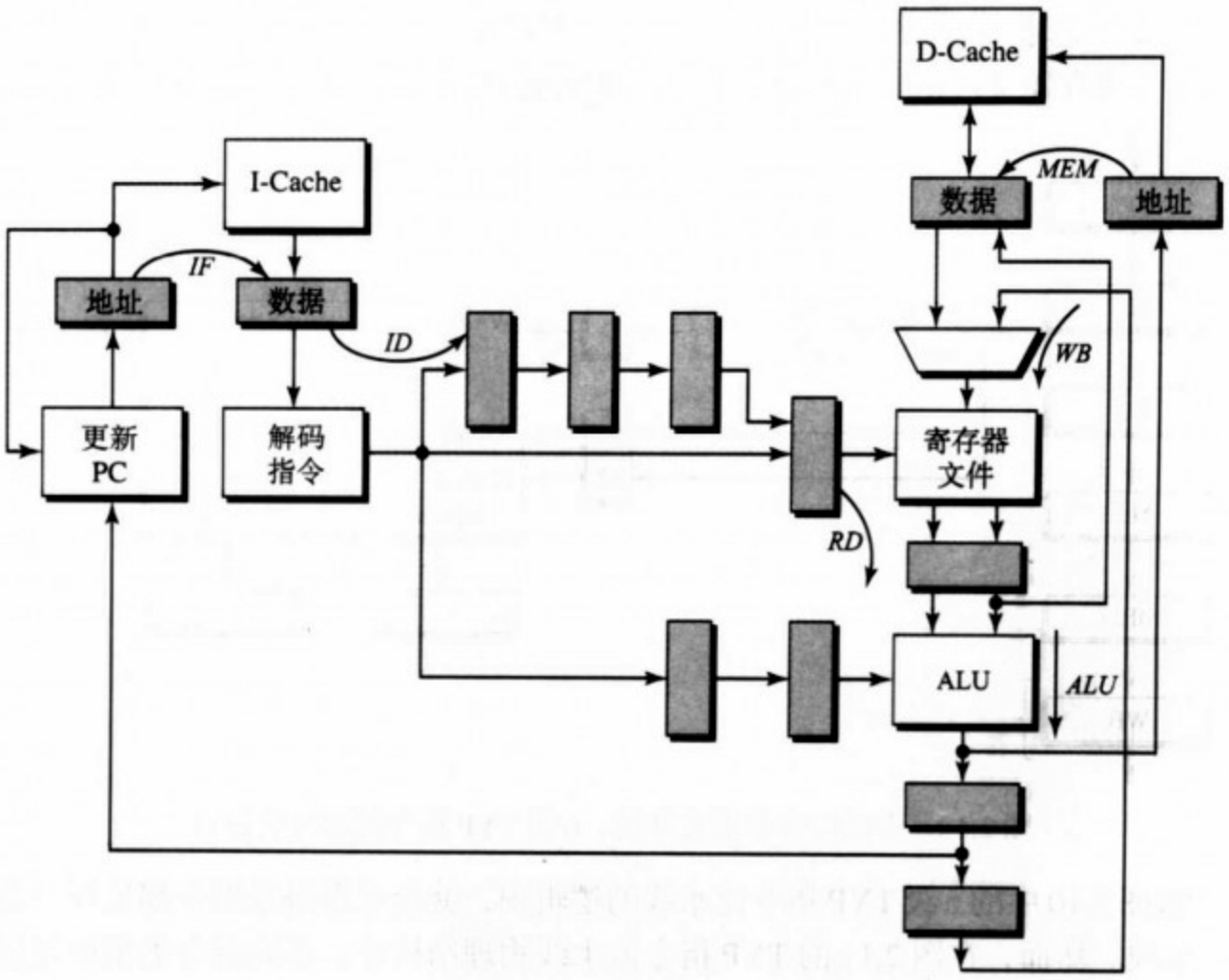


图2.11　6级TYP指令流水线的物理组织方式

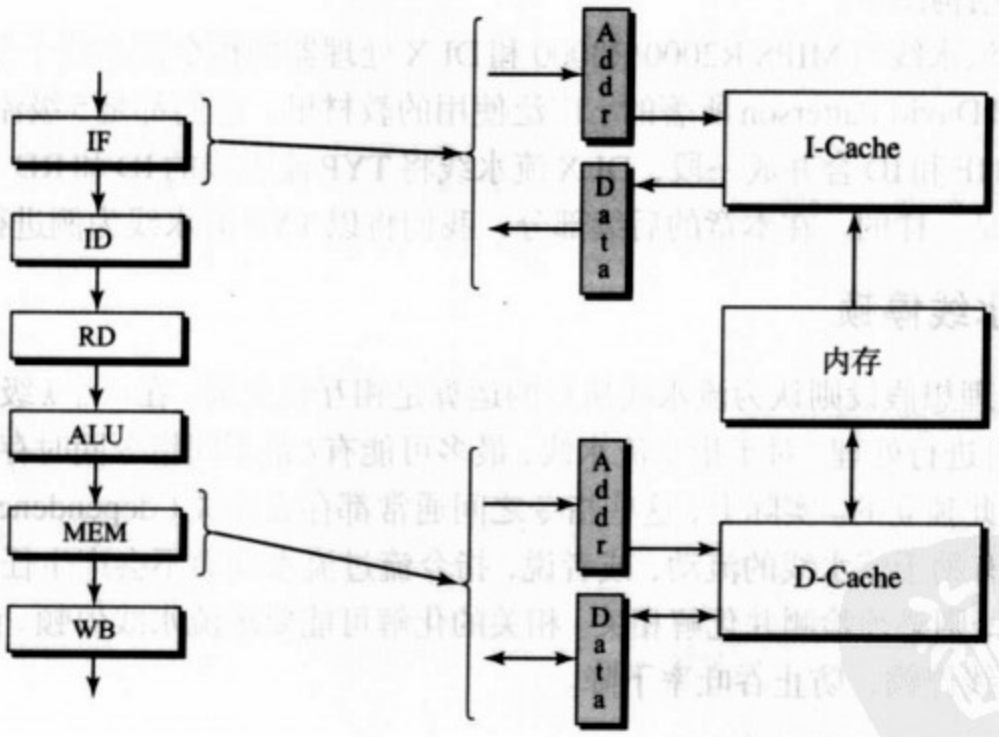


图2.12　内存子系统的、6级TYP指令流水线的接口

到多端口寄存器文件的接口如图2.13所示。只有RD和WB段访问寄存器文件。在每个时钟周期，寄存器文件要支持两次寄存器读（RD段）和一次寄存器写（WB段）。因此寄存器文件要有两个读端口，一个写端口，如图2.13所示。这个寄存器文件有三个地址端口，两个数据输出端口，一个数据输入端口，从而满足一个时钟周期内的两次读操作和一次写操作。WB段中正在执行寄存器写的指令比RD段中正在执行寄存器读的指令领先3个时钟周期，或者领先3条插入的指令。因此，在寄存器写地址端口处有三个附加的流水段缓冲，从而保证写操作目标地址到达寄存器文件写地址端口的时间，恰好和要写入的数据到达输入端口的时间相同。三端口的寄存器文件并不复杂，然而，

当端口数目超过三个时，硬件的复杂度会很快增加，特别是增加写端口的数目，更是如此。多达20个端口的多端口寄存器文件也是可行的，一些高端微处理器就采用了这样的设计。

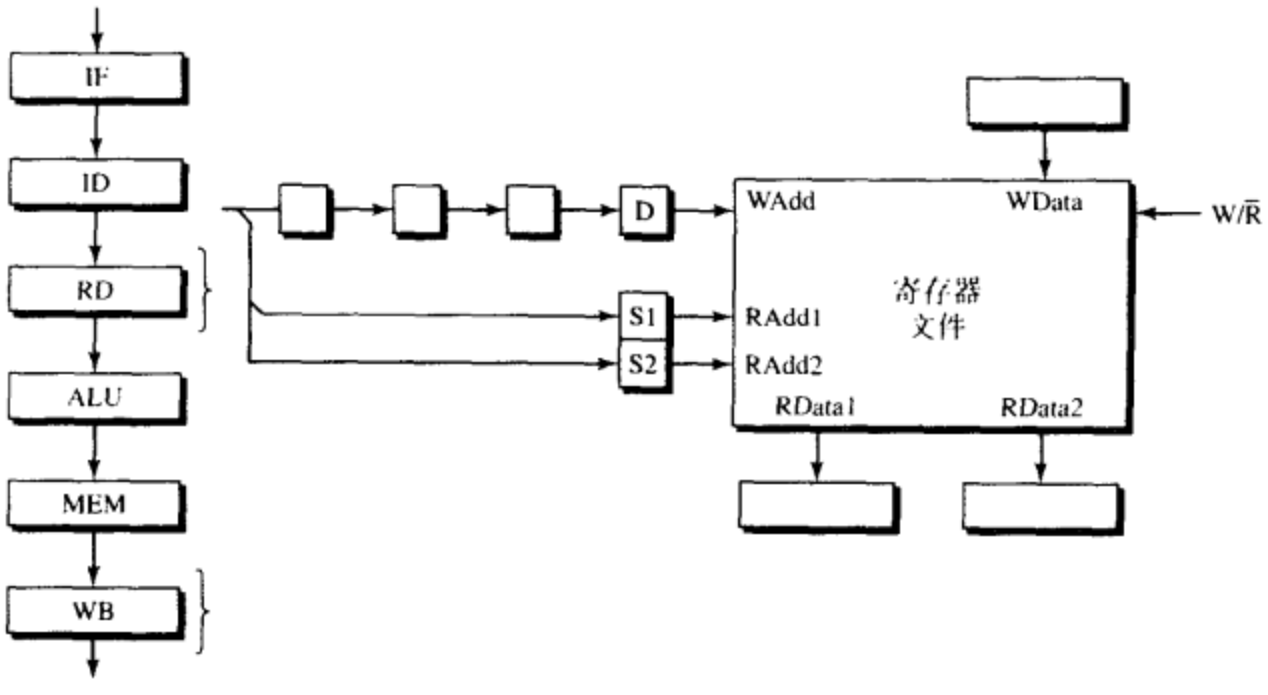


图2.13 多端口寄存器文件的6级TYP指令流水线的接口

如果考察图2.10中的6级TYP指令流水线的逻辑图，就会觉得每条指令都是以一条直线路径通过6级流水线。然而，在图2.11的TYP指令流水线物理结构中，不同指令类型所经过的物理路径是不一样的。图2.11中，一些路径进行了标识，表明它们属于哪一段流水线。有些流水段在物理上分布于整个物理结构图中。

6级TYP指令流水线与MIPS R2000/R3000和DLX处理器的指令流水线十分类似，DLX出现在John Hennessy和David Patterson所著的、广泛使用的教材里，它们都是5级流水线。MIPS流水线将TYP流水线的IF和ID合并成一段，DLX流水线将TYP流水线的ID和RD合并成一段。除此之外，其他4级都是一样的。在本章的后续部分，我们将以TYP流水线为例进行讲解。

2.2.3 减少流水线停顿

第二条流水线理想假设则认为流水线执行的运算是相互独立的。在一个k级流水线中，可能有k个不同的运算同时进行处理。对于指令流水线，最多可能有k条不同指令同时存在于流水线中。这些指令可能不是彼此独立的。实际上，这些指令之间通常都存在相关（dependence）。如果流水线中的指令不相关，将有助于流水线的流动，或者说，指令流过流水线而不会产生任何流水线停顿。如果指令间存在相关，则必须检测并化解相关。相关的化解可能要求流水线停顿。设计的挑战和目标就是尽量减少流水线停顿，防止吞吐率下降。

2.2.3.1 程序相关性和流水线相关

在ISA的抽象层次上，程序是汇编语言指令的序列。一条典型的指令可以定义为一个函数i：$T \leftarrow S1\ op\ S2$，其中指令i的定义域为$D(i)=\{S1,S2\}$，值域为$R(i)=\{T\}$，从定义域到值域的映射由操作op定义。给定两条指令i和j，指令j跟在指令i之后，那么如果下列三个条件有一个成立，就认为指令i和指令j之间存在数据相关，或者指令j依赖于i并记为$i\delta j$。

$$R(i) \cap D(j) \neq \varnothing \quad \text{（公式2.8）}$$

$$R(j) \cap D(i) \neq \varnothing \quad \text{（公式2.9）}$$

$$R(i) \cap R(j) \neq \varnothing \quad \text{（公式2.10）}$$

第一个条件表明，指令j所需要的操作数在指令i的值域里。这称为先写后读（RAW，read-after-write）相关或者真相关，记为$i\delta_d j$。真相关的含义是，指令j不能在指令i完成之前开始执行。第二个条件表明，指令i所需要的操作数在指令j的值域里，或者说指令j所修改的变量是指令i的操作数之一。这称为先读后写（WAR，write-after-read）相关或者叫反相关，记为$i\delta_a j$。如果存在反相关，那么指令j不能在指令i执行之前完成，否则，指令i获取的就是错误的操作数。第三个条件表明，指令i和指令j的值域中存在同一个变量，或者说它们会修改同一个变量。这称为写后写（WAW，write-after-write）相关或者输出相关，记为$i\delta_o j$。如果存在输出相关，那么指令j不能在指令i之前完成，否则，如果j后面的指令要使用j所修改的变量作为操作数，将取得错误的值。显然，读后读（RAR，read-after-read）相关，也就是指令i和指令j都访问同一个操作数的情况，不管访问的顺序如何，都不会造成任何危害。

以上定义域和值域的三种重叠方式带来了三种可能的数据相关，即真相关（RAW）、反相关（WAR）和输出相关（WAW）。因为在汇编代码里，指令的定义域和值域可能是存放在寄存器或者存储器中的变量，所以数据相关中共同的变量可能对应于一个寄存器或者存储器地址，我们称之为寄存器相关和存储器相关。这一章主要将重点放在寄存器相关上，图2.14对RAW、WAR和WAW的寄存器相关进行了说明。

除了数据相关以外，两条指令之间还存在控制相关。给定指令i和j，j跟在i的后面。如果指令j是否执行依赖于指令i的结果，则称指令j控制依赖于指令i，记为$i\delta_C j$。控制相关是程序的控制流结构引起的，条件分支将给指令的顺序带来不确定因素。条件分支之后的指令与分支指令存在控制相关。

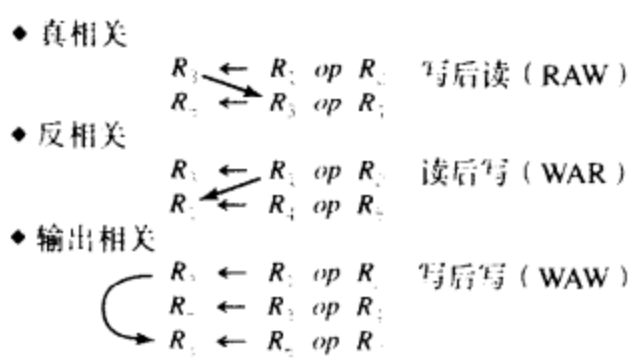


图 2.14　RAW, WAR 以及 WAW 数据相关

一个汇编语言程序由指令序列组成。这个程序的语义假设指令是顺序执行的，即相邻两条指令的执行是有顺序的。如果指令i+1跟在指令i后面，那么一般程序假设先执行指令i，然后再执行指令i+1。如果遵循这样的顺序执行，那么程序语义的正确性就得到了保证。更准确地说，因为一个指令周期包括多个运算分量的执行，指令i的所有运算分量要在指令i+1的任何一个运算分量开始之前执行完毕。这称之为程序的全序执行（total sequential execution），即指令序列的所有运算分量都是顺序执行的。

给定一个k级流水线的处理器，k条指令的处理在流水线中是重叠的。当指令i结束第一个分量并开始第二个分量的处理时，指令i+1将开始第一个分量的处理。一条特定指令的k个分量与其他指令的分量是重叠处理的。因此，全序执行并不成立。而全序执行是保证程序语义的充分条件，并不是必要条件，即全序执行是保证程序语义正确性的“过于充分”的规定。保证程序语义正确性的必要条件是不违背所有指令间的相关。换句话说，如果指令i和指令j存在相关性，并且指令j在i之后，那么指令i和j对于同一个变量的读写必须按照原来的顺序。在流水线处理器中，如果不注意，就会潜在地违背程序的相关性。对程序相关性的潜在违背称为流水线相关。为保证程序的正确执行，所有流水线相关必须被检测出来并进行化解。

2.2.3.2 流水线相关的确定

当所有指令类型都统一到一条指令流水线中，并且流水线的每一段的功能都定义好之后，就可以对流水段进行分析，找出所有可能出现的流水线相关。流水线相关与流水线的结构和指令间的相关都有关系。本章的重点将放在标量指令流水线。根据定义，标量指令流水线是由多个流水段按照线性方式组织起来的单流水线，指令按照程序定义的顺序进入指令流水线。除非碰到流水线停顿，否则指令会逐条地流过标量指令流水线。也就是说，每条指令在每个时钟周期内进入流水线的下一段。对于标量指令流水线，可以确定出由于数据相关而导致的流水线相关发生的必要条件，这些条件通常与流水线结构有关。

流水线相关是对程序相关性的潜在违背。流水线相关可以依照程序相关性的类型来分类。WAW相关是对输出相关的潜在违背，WAR相关是对反相关的潜在违背，RAW相关是对真相关的潜在违背。数据相关是两条针指令针对同一个变量的读写。要发生相关，流水线中至少要有两段所包含的指令能同时访问同一个变量。

图2.15说明了在流水线结构中发生WAW，WAR和RAW相关的必要条件。这些必要条件是由存储器和寄存器数据相关引起的（图中只说明了寄存器相关的情况）。要发生由输出相关$i\delta_o j$引起的WAW相关，至少要有两个流水段同时对一个变量执行写操作，如图2.15（a）所示。如果只有一个流水段能对这个变量执行写操作，就不会发生相关，因为对这个变量的两次写，实际上是所有的写，它们都会按照程序所定义的顺序来进行。图2.15（b）表明，如果发生WAR相关（反相关），流水线中必须要有这样的两段，不妨设前面的一段为x，后面的一段为y，x段要写一个变量，而y段要读这个变量。要违背反相关$i\delta_a j$，指令j必须在指令i执行读之前执行写，或者说指令j必须在指令i到达y段之前到达x段。如果该必要条件不满足，指令j就不可能在指令i完成读之前执行写。例如，如果只有一段流水线可以对同一个变量执行读写操作，那么所有对这个变量的访问都将按照原来的顺序进行，因此就不会发生WAR相关。当执行读的流水段在执行写的流水段之前时，前面的指令i一定可以在后面的指令j开始写之前完成读，因此这样的流水线就不存在WAR相关。实际上，图2.15所表示的必要条件，同时也是充分条件，可以看做是产生WAW，WAR和RAW流水线相关的特征条件。

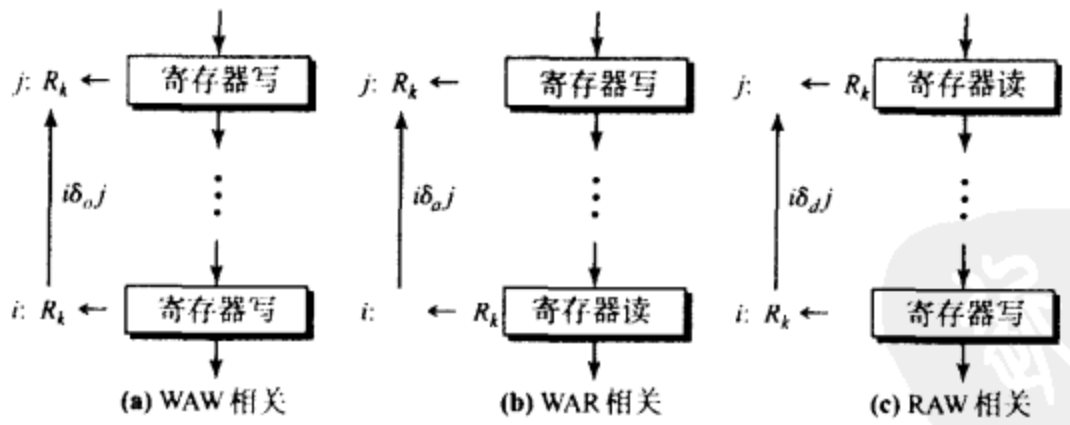


图2.15 产生WAW相关、WAR相关以及RAW相关时，流水线结构的必要条件

图2.15(c)表明，如果发生由真数据相关$i\delta_d j$导致的RAW相关，流水线中必定存在这样两段x和y，不妨设x段在y段之前，x段对一个变量执行读，而y段对这个变量执行写。对于这样的流水线结构，如果后面的指令j到达x段比前面的指令i到达y段早，就会违背真数据相关$i\delta_d j$。通过与前面WAR相关类似的分析可以看出，如果这个条件不满足，就不会发生RAW相关。例如，如果只有一个流水段执行所有的读写操作，那么就将产生全序执行并且不会有相关发生。如果执行读的段在流水线中比执行写的段靠后，那么也不会发生RAW相关，因为前面指令的所有写操作将在后面的指令读操作之前完成。

由于流水线相关是由于潜在地违背程序相关性而引起的，所以存在一种确定指令流水线中可能发生的流水线相关的系统方法，这种方法依次考察每一种相关类型。本章采用的方法将按照以下顺序考察程序的相关性。

1. **存储器数据相关**
   a. 输出相关
   b. 反相关
   c. 真数据相关
2. **寄存器数据相关**
   a.输出相关
   b.反相关
   c.真数据相关
3. **控制相关**

我们通过将这个方法应用于6级TYP指令流水线进行说明。首先，考虑存储器数据相关。存储器数据相关是指两条指令访问（读或写）存放在存储器中的同一个变量。对于load/store型体系结构，存储器数据相关只能在load/store指令之间发生。要确定存储器数据相关是否会导致流水线相关的发生，必须检查流水线对load/store指令的处理过程。假设采用分离的cache并且只有MEM段能访问D-cache。因此，所有load/store指令对存储器的访问都必须且只能在MEM段进行，因为流水线中只有这一段能访问数据存储器。根据图2.15所表述的必要条件，TYP流水线不可能发生由存储器数据相关引起的流水线相关。所有对存储器的访问是顺序进行的，对load/store指令的处理是按照全序执行的模式进行的。因此，对于TYP流水线，不存在由存储器数据相关引起的流水线相关。

现在考虑寄存器数据相关。要确定寄存器数据相关是否导致流水线相关的发生，必须确定流水线中所有能访问寄存器文件的流水段。在TYP流水线中，所有的寄存器读发生在RD段，所有的寄存器写发生在WB段。首先考察输出（WAW）相关并记为$i\delta_o j$，即指令i和后续指令j对相同的目标寄存器进行操作。要保证输出相关，指令i必须先写寄存器，然后指令j才能向这个寄存器执行写。在TYP流水线中，只有WB段能向寄存器文件执行写操作，因此，所有的寄存器写在WB段中是按照全序方式执行的。根据图2.15（a）的必要条件，TYP流水线不存在输出相关引起的流水线相关。

下面考察反相关（WAR）并记为$i\delta_a j$，即指令i所读的寄存器是后续指令j的目标寄存器。必须保证指令i在指令j写之前读这个寄存器。惟一能使反相关引发流水线相关的情况是，指令j能够在指令i读寄存器之前写这个寄存器。这对于TYP流水线是不可能的，因为所有的寄存器读都在RD段发生，并处于惟一能对寄存器进行写操作的WB段之前。因此，TYP流水线不存在图2.15（b）中所示的必要条件。所以，TYP流水线不存在反相关引起的流水线相关。

TYP流水线中惟一能引发流水线相关的寄存器数据相关是真数据相关。TYP流水线中存在图2.15（c）中所示的必要条件，因为执行寄存器读的RD段在执行写的WB段之前。真数据相关记为$i\delta_b j$，指的是指令i执行寄存器写而后续指令j读取同一个寄存器。如果指令j紧跟在指令i之后，那么当j到达RD段时，指令i将仍然处于ALU段。因此，直到指令i到达WB段，指令j才能读到正确的结果。要保证真数据相关性，必须阻止指令j进入RD段，直到指令i在WB段中执行完毕。真数据相关会导致RAW流水线相关，因为后续指令到达读寄存器的流水段的时间要先于前面的指令完成寄存器写的时间。

最后考虑控制相关。引起控制相关的操作是改变控制流的指令，即条件分支指令。条件分支指令的结果决定了取指的下一条指令是顺序的指令还是分支目标处的指令。条件分支指令后面可以有

两条候选的指令。在指令流水线中正常的操作下，取指段使用程序计数器的值获取下一条指令，然后增加程序计数器的值，使它指向顺序的下一条指令。取指段每个时钟周期都要重复这一步骤，以保持流水线一直充满指令。当取到一条条件分支指令时，当前的指令流就存在中断的可能。如果条件分支没有发生，那么取指段顺序取得的下一条指令就是正确的。然而，如果条件分支实际发生了，那么顺序取得的下一条指令就是不正确的。所以只有知道了分支的条件是否满足才能确定分支的方向。

控制相关可以看做是关于程序计数器的寄存器数据（RAW）相关。条件分支指令要写PC，而取下一条指令要读PC。如果分支发生，条件分支指令用目标指令的地址更新PC，否则，使用下一条指令的地址更新PC。在TYP流水线中，用目标指令的地址来更新PC是在MEM段完成的，然而使用PC的值来取下一条指令是在IF段。因此，IF段完成PC寄存器的读取，而后面的MEM段完成PC的写入。根据图2.15（c），IF和MEM段的时序满足发生RAW相关的必要条件。因此，TYP流水线存在控制相关，并且可以看做是关于PC寄存器的一种RAW相关。

2.2.3.3 化解流水线相关

给定了TYP流水线的结构，数据相关引起的惟一一种流水线相关是RAW相关。另外，还存在控制相关引起的流水线相关。这些相关都是前面一条指令i向一个寄存器（或者PC）写，而后面一条指令j读同一个寄存器。由于存在流水线相关，所以必须提供化解相关的机制，即保证不违背相应的数据相关性。关于TYP流水线的每一个RAW相关，必须保证读寄存器在写寄存器以后发生，这个寄存器又称为相关寄存器。

要化解RAW相关，就要阻止后续的指令j进入读相关寄存器的流水线段，直到前面的指令i通过了写相关寄存器的流水线段。实现的方法是将执行寄存器读的流水线段前面所有的段都停止掉，这样就阻止了指令j进入读相关寄存器的流水线段。在最坏情况下，指令j必须停止的时钟周期数等于流水线中引起相关的关键段之间的距离，即对相关寄存器执行读和写的那两段。对于TYP流水线的情况，如果前面的指令i是ALU或者load指令，关键段是写寄存器的WB段，对于所有的后续指令，关键段是读寄存器的都是RD段，这两段之间的距离是3，因此，最坏情况下的开销是3个时钟周期，如表2.5所示。最坏的情况是指令j紧跟在指令i后面的时候，即$j=i+1$。在这种情况下，指令j必须在ID段等待3个时钟周期，即3个时钟周期之后，当指令i离开WB段时，指令j才能允许进入RD段。如果后续的指令j不是紧跟在指令i后面，即i与j之间插入了其他指令，那么开销将小于3个周期。这里假设插入的指令与指令i不相关，这时时钟开销的实际数目等于$3-s$，其中s是插入的指令数目。例如，如果i与j之间有3条指令，那么就没有开销。在这种情况下，指令j正好在指令i离开WB段的时候进入RD段，这时不需要停顿来满足RAW相关。

表2.5 在TYP流水线中，RAW相关在最坏情况下产生的开销

前导指令类型（i）	ALU	load	Branch
后续指令类型：（j）	ALU, L/S, Br.	ALU, L/S, Br.	ALU, L/S, Br
相关寄存器	指令寄存器（Ri）	Int. Register（Ri）	PC
寄存器WRITE级（i）	WB（第6级）	WB（第6级）	MEM（第5级）
寄存器READ级（j）	RD（第3级）	RD（第3级）	IF（第1级）
RAW相关生成的段间距离或时钟周期	3个时钟周期	3个时钟周期	4个时钟周期

对于控制相关，前面的指令i是一条分支指令，它在MEM段更新PC寄存器。取下一条指令j需要在IF段读取PC的值，这两段之间的距离是4。因此，最坏情况下的开销是4个周期。当碰到条件分支指令时，将停止IF段从而停止进一步的取指，直到分支指令完成MEM段并且用分支目标地址更新了PC。这要求IF段停止4个周期。通过进一步分析可知，只有当分支实际发生的时候，

停止流水线才是必要的。如果最后分支没有发生，IF段完全可以继续取下一条指令。设计流水线时，可以将这一特性考虑进去，即当遇到条件分支指令时不停止取指令，流水线假设分支不会发生。在发生分支的情况下，在MEM段使用分支目标地址对PC进行更新，并将驻留在前面流水线段中的指令删除或者排空，获取的下一条指令是分支目标处的指令。如果采用这样的设计，那么只有当发生分支时才会有4个时钟周期的开销，否则就没有额外的开销。

与寄存器数据相关导致的RAW相关类似，控制相关产生的4个时钟周期是最坏情况下的开销。如果与指令i没有控制相关的指令可以插入到i和j之间，指令j是与i有控制相关的指令，那么实际开销的周期数会由于插入指令而减少，而减少的数目等于插入的指令数目。这就是“延迟分支”（delayed branche）的概念。这些本来作为开销的时钟周期，通过插入一些无论分支是否成功都要执行的指令，实际开销的时钟数目就变成$4-s$，s是指令i与j之间可以插入的没有控制相关的指令数目。如果采用延迟分支，即采用在延迟的时钟周期中插入指令的方法，那么实现假设不发生分支而继续顺序取指的技术就比较困难。原因是必须提供一套机制来区别填充的指令和时序的正常指令。在发生分支的情况下，填充的指令不需要删除，但是顺序的正常指令就必须删除，因为它们不应该执行。

2.2.3.4　通过定向路径减少开销

到目前为止，我们所讨论的惟一一种化解相关的机制是，停止执行发生相关的指令，并保证对相关寄存器的读写是按照正常的顺序进行的。在流水线的实现中，还有许多更加有效的技术，可以减少流水线相关引起的时钟开销数目。采用定向路径就是其中的一种技术。

在流水线相关中，前面的指令i与后面的指令j存在相关性。对于RAW相关，指令j需要指令i的结果。图2.16说明了当指令i是ALU指令或者load指令时的处理过程。如果指令i是ALU指令，那么指令j所需要的结果将在ALU段产生，并且当指令i结束ALU段时，结果就可用。换句话说，当指令i离开ALU段时，指令j所需要的操作数在ALU段的输出有效，并且不需要再等待两个时钟周期，直到i离开WB段。如果ALU段的输出能够通过一条物理的定向路径（forwarding path）到达ALU段的输入端，那么指令j就可以在指令i离开ALU段的时候进入ALU段。这时，指令j不需要在RD段读取寄存器文件并且访问与指令i相关的操作数，而是通过访问ALU段的输出端来获得操作数。当前面的指令i是ALU指令时，有了这条定向路径和相关的控制逻辑，最坏情况下的开销就降为0。甚至，如果后续相关的指令是第$i+1$条指令，也不需要停顿，因为指令$i+1$可以在指令i离开ALU段的时候进入ALU段，就和正常的顺序一样。

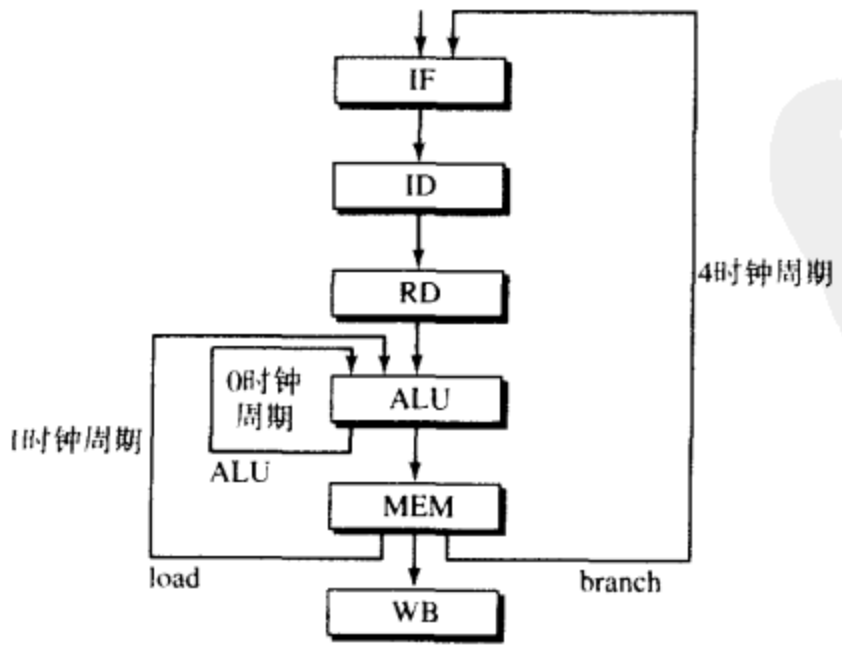


图2.16　当指令i是ALU指令或load指令时的处理过程

如果前面的指令 i 是load指令而不是ALU指令，同样可以采用类似的定向路径来减少开销的时钟数目。如图2.16所示，如果前面的指令是load指令，这条指令的结果，即要存放到寄存器中的存储器数据值，将在load指令结束MEM段时，在MEM段的输出端有效。同样，可以增加一条从MEM段输出到ALU段输入的定向路径，来满足后续指令的需求。后续指令可以在前面的load指令结束MEM段时进入ALU段。这个方法有效地将最坏情况下的开销从3个时钟周期降到只有一个时钟周期。在最坏情况下，相关的指令是第 i+1 条指令，即 $j = i$+1。在正常的处理过程中，当指令 i 处于ALU段时，指令 i+1 处于RD段。当指令 i 进入MEM段时，必须停止前面几段流水线，将指令 i+1 阻止在RD段。然而，在下一个时钟周期，当指令 i 离开MEM段时，通过从MEM段输出到ALU段输入的定向路径，指令 i+1 可以进入ALU段。在实际效果上，指令 i+1 只被阻止了一个时钟周期，因此最坏情况下的开销只有一个周期。采用了定向路径之后，对于RAW相关，最坏情况下的开销参见表2.6。

表2.6 在TYP流水线中，采用定向路径时RAW相关产生的开销

前导指令类型：（i）	ALU	load	Branch
后续指令类型（j）	ALU, L/S, Br.	ALU, L/S, Br.	ALU, L/S, Br.
相关寄存器	Int. Register（Ri）	Int. Register（Ri）	PC
寄存器WRITE级（i）	WB（第6级）	WB（第6级）	MEM（第5级）
寄存器READ级（j）	RD（第3级）	RD（第3级）	IF（第1级）
从输出端定向：	ALU, MEM, WB	MEM, WB	MEM, WB
定向至输入端：	ALU	ALU	IF
w/定向路径产生的开销	0时钟周期	1时钟周期	4时钟周期

ALU指令的RAW相关引起的开销称为ALU开销。同样，load指令的开销称为load开销。对于TYP流水线，加上定向路径之后，ALU开销为0个时钟周期。也就是说，当前面的指令为ALU指令时，没有引入任何开销。注意，定向路径的起始端是ALU段的输出端，这是指令 i 的结果最早有效的地方。定向路径的目的端是ALU段的输入端，这是指令 j 要求获得指令 i 结果最迟有效的地方。从结果最早有效处到最迟有效处的定向路径称为“关键定向路径”（critical forwarding path），它是减少这种指令类型引起的相关开销上所能做的最大努力。

除了关键定向路径以外，还需要其他的定向路径。例如，需要从MEM段和WB段输出到ALU段输入的定向路径。需要这两条定向路径是因为，发生相关的指令 j 可能是第 i+2 条或者第 i+3 条指令。如果 $j = i$+2，那么当指令 j 准备进入ALU段时，指令 i 正在离开MEM段。因此，指令 i 的结果这时还没有写入目标寄存器，而在MEM段的输出端已经有效，并且必须被送到ALU段的输入端，使得指令 j 可以在下一个周期进入ALU段。同样，如果 $j = i$+3，指令 i 的结果必须从WB段的输出端传送到ALU段的输入端。这时，虽然指令 i 已经完成向目标寄存器写入结果，但是指令 j 已经穿越了RD段并准备进入ALU段。当然，当 $j = i$+4 时，指令 j 正常的访问寄存器文件就能满足RAW相关，不需要使用任何定向路径。当 j 到达RD段时，i 已经完成了WB段。

如果前面的指令是load指令，指令 i 的结果最早有效的地方是MEM段的输出，最迟需要这个结果的地方是ALU段的输入。因此，load指令在前时，关键定向路径是从MEM段的输出到ALU段的输入。这是这种情况下最好的结果，这时一个周期的开销是不可避免的。同样，当指令 i 离开WB段而相关指令准备进入ALU段时，还需要另一条从WB段输出到ALU段输入的定向路径。

从表2.6可以看出，我们没有采用定向路径来减少分支指令带来的开销。如果前面的指令 i 是分支指令，对于TYP流水线采用的地址模式，结果最早有效的地方是MEM段的输出端。对于分支指令，分支目标的地址和分支条件在ALU段中产生，直到进入MEM段，才检查分支条件并将分支目标地址写入PC。因此，只有过了MEM段，才能使用PC获取分支目标处的指令。另一方面，

PC 必须在 IF 段的起始处有效，才能取得下一条指令。因此，最迟需要结果的地方是 IF 段的开始。这样，关键定向路径所能做出的最大努力，就是在MEM段用分支目标地址更新PC，然后在下一个周期开始时获取分支目标处的指令。如果分支条件的判断在ALU段中能尽早地进行，使得能在ALU段结束时更新 PC，那么分支的开销将从 4 个周期降为 3 个周期。

2.2.3.5　流水线互锁的实现

化解流水线相关的硬件机制称为流水线互锁机制，流水线互锁硬件要检测出所有的流水线相关，并保证所有的相关得到满足。流水互锁机制包括停止流水线的某一段以及数据在定向路径中的传输控制等。

由于引入了定向路径，标量流水线不再是一条简单的流水线序列，数据从第一段流向最后一段。定向路径提供了从后面段的输出到前面段的输入的反馈路径。例如，为了化解前面一条指令是ALU指令时出现的流水线相关，需要三条定向路径，如图2.17所示。这些定向路径称为ALU定向路径。当ALU指令 i 通过流水线的时候，后面可能有多条指令与指令 i 存在数据相关（RAW）。图2.17右侧说明了如何在三个连续的周期中满足多条相关的指令。在t1周期中，指令 i 通过定向路径a将结果传递给相关指令 i+1。在下一个时钟周期t2时，指令 i 通过定向路径b将结果传递给相关指令 i+2。如果指令 i+2 也需要指令 i+1 的结果，这个结果同样可以在这个周期中通过定向路径传递给 i+2。在t3周期中，指令 i 可以通过定向路径c将结果传递给指令 i+3。同样，如果指令 i+3 需要指令 i+2 或者 i+1 的结果，也可以分别通过路径a或者路径b来传递。

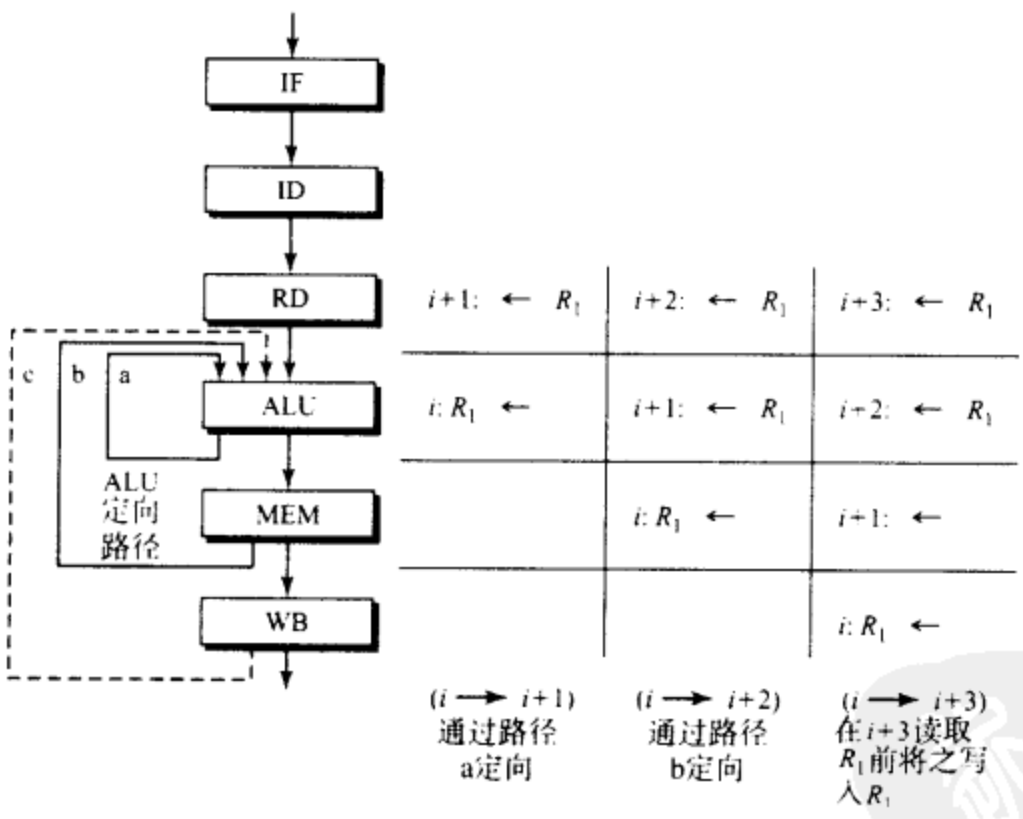


图 2.17　化解 ALU 指令出现的流水线相关

图2.17只是一个逻辑图，其物理实现如图2.18所示。注意，RAW相关的检测是通过使用一个比较器，比较连续两条指令的寄存器标识符来实现。图2.18中采用了4个5位（假设有32个寄存器）比较器。如果后续的指令 j 当前处于RD段并准备读取两个寄存器操作数，那么前两个比较器（右边）用于检测指令 j 与指令 j–1 之间可能存在的RAW相关，此时指令 j–1 处于ALU段。这两个比较器将比较指令 j 的两个源寄存器标识符和指令 j–1 的目的寄存器标识符。与此同时，另外两个比较器（右边）用于检测指令 j 与指令 j–2 之间可能存在的RAW相关，此时指令 j–2 处于MEM段。这两个比较器将比较指令 j 的两个源寄存器标识符和指令 j–2 的目的寄存器标识符。这4个比较器的输出将作为下一个周期的控制信号，如果检测到相关，就激活相应的定向路径。

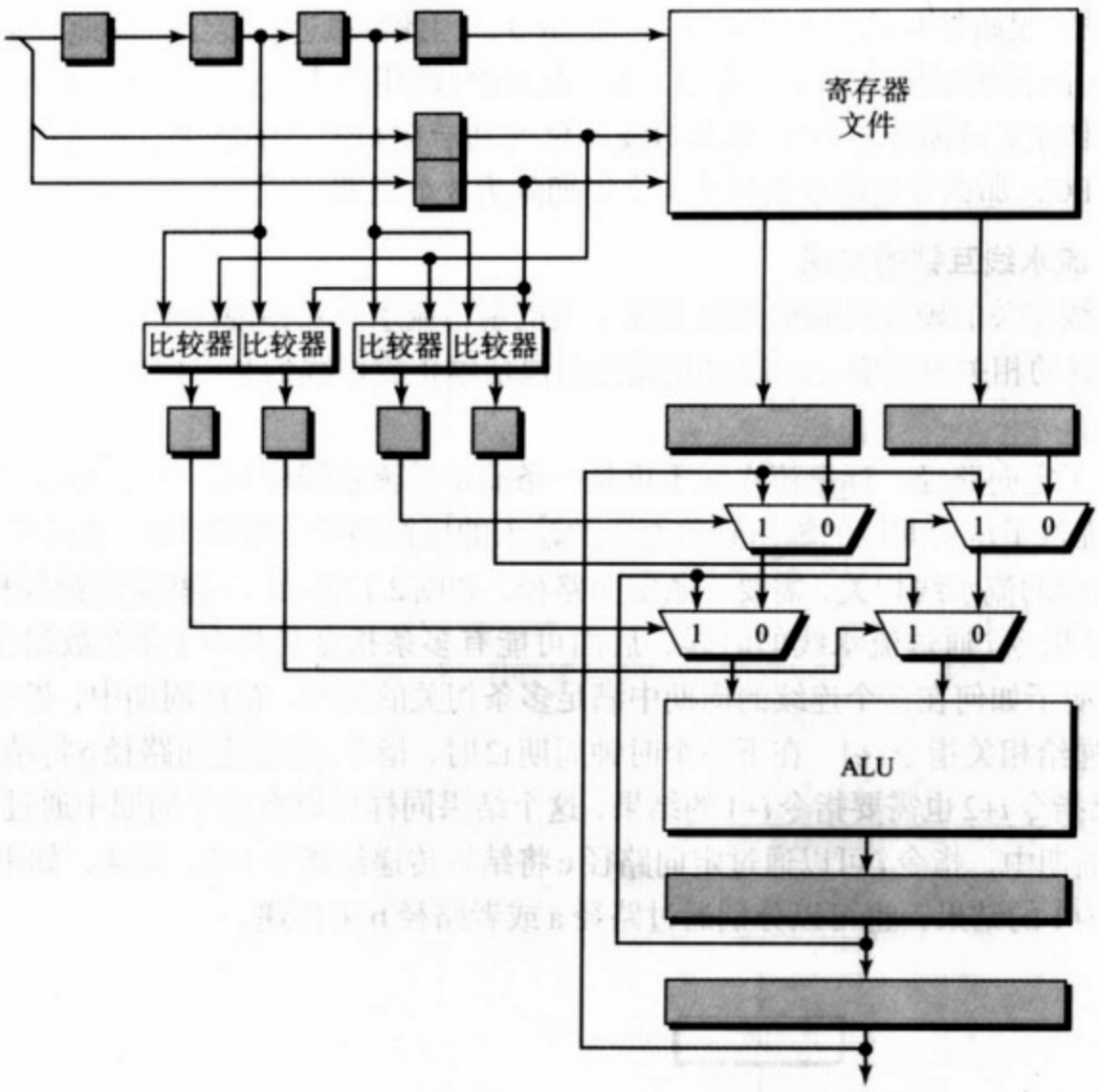


图 2.18 包含 ALU 指令的、RAW 相关的流水线互锁机制

如果检测到指令 j 与 j–1 之间存在RAW相关，第一组比较器的输出将激活定向路径a。同样，如果检测到指令 j 与 j–2 之间存在RAW相关，第二组比较器的输出将激活定向路径b。如果 j 与 j–1 和 j–2 都存在相关，两条路径可以同时激活。

图 2.17 中的路径c没有在图 2.18 中表示出来。原因是，如果在设计多端口寄存器文件的时候进行了适当考虑，那么这条定向路径是没有必要的。如果将三端口（两个读，一个写）的寄存器文件设计成先执行写，再执行两次读，那么第三条定向路径就是没有必要的。当指令 j 通过RD段时，将读到新的正确结果。换句话说，定向已经在寄存器文件内部完成，没有必要等待一个周期之后再读相关寄存器，或者将结果从WB段的输出定向到ALU段的输入。这是一种合理的设计，可以减少一个周期的开销或者一条定向路径。实际上MIPS R2000/R3000流水线就采用了这样的设计。

要减少load指令引起的流水线相关带来的开销，需要另外一套定向路径。图2.19显示了当流水线相关中的第一条指令是load指令时所需要的两条定向路径，这两条定向路径称为load定向路径。定向路径d将MEM段的输出传递到ALU段的输入，定向路径e将WB段的输出传递到ALU段的输入。当第一条指令 i 到达ALU段时，如果指令 i+1 与 i 相关，它必须在RD段等待一个周期。在下一个周期中，当指令 i 准备离开MEM段时，可以将结果通过路径d定向到ALU段，使得指令 i+1 可以进入ALU段。当指令 i+2 也与指令 i 相关时，同样的结果将通过路径e从WB段到达ALU段，这使得指令 i+2 能够进入ALU段，从而不需要任何等待。同样，如果寄存器文件先执行写后再执行读，路径e也是不必要的。例如，指令 i 在WB段执行寄存器写，而同时指令 i+2 在RD段读取指令 i 的结果。

既支持由ALU指令引起的流水线相关，又支持由load指令引起的流水线相关的所有定向路径的物理实现如图2.20所示。这里假设寄存器文件在每个时钟周期先执行写，后执行读，因此路径e

没有表示出来。注意，虽然分别位于图 2.17 和图 2.19 中的 ALU 定向路径 b 和 load 定向路径 d 都是从 MEM 段的输出到 ALU 段的输入，但它们实际上是两条不同的物理路径，如图 2.20 所示。这两条路径将进入第一对多路选择器并且两者之间只能选择一条，这取决于MEM段中的指令是ALU指令还是 load 指令。定向路径 b 起始于 MEM 段的缓冲，这个缓冲包含上一个周期 ALU 的输出；定向路径 d 也起始于 MEM 段的缓冲，但包含的是从 D-cache 获得的数据。

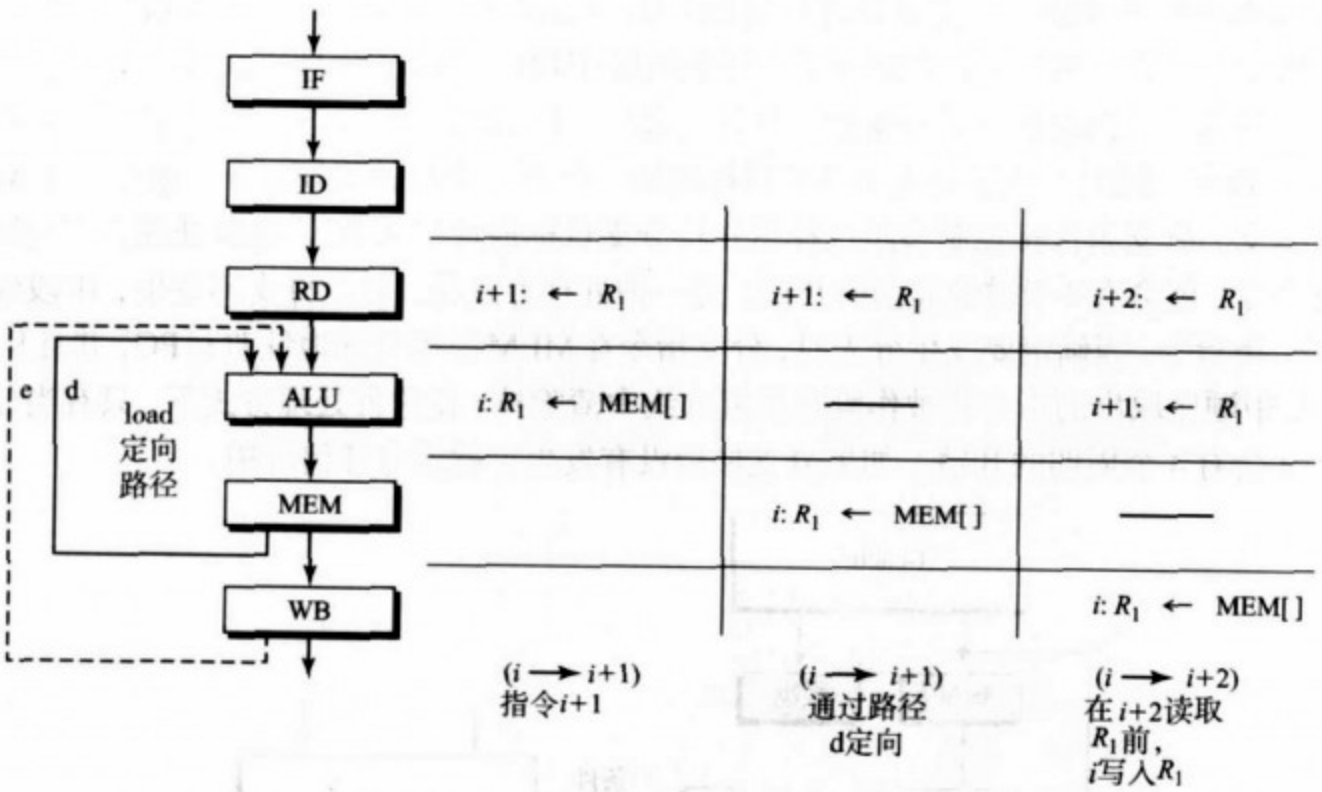


图 2.19　包含 load 指令的的、支持流水线相关的定向路径

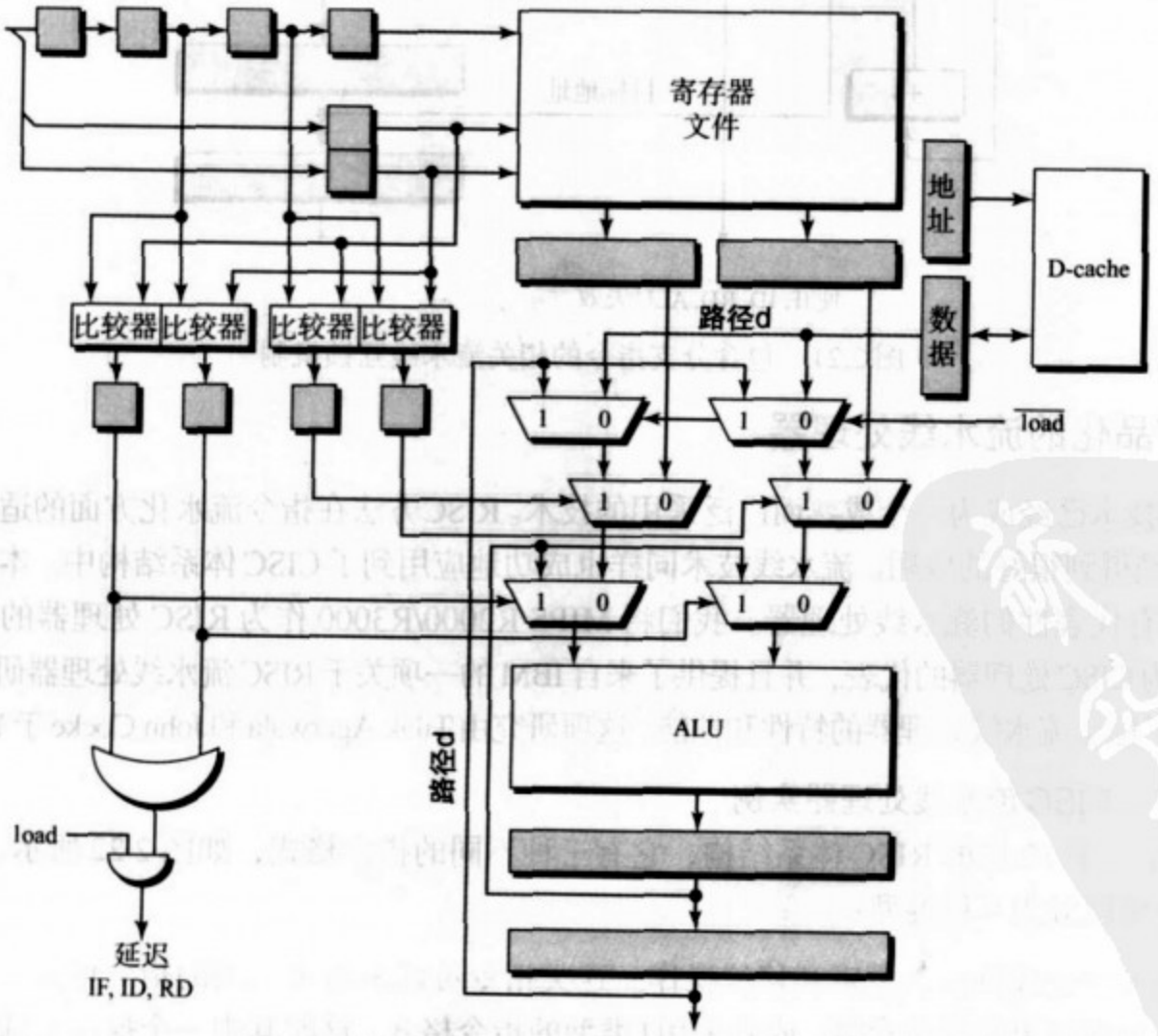


图 2.20　包含 ALU 和 load 指令的、RAW 相关的流水线互锁机制

不管是ALU指令还是load指令，都使用同样的两对比较器来检测寄存器相关。需要两对比较器是因为互锁硬件必须检测指令i与i+1以及指令i与i+2之间可能存在的相关。当寄存器文件设计为先执行写，后执行读时，指令i与i+3之间的相关会自动得到满足。如果检测到指令i与i+1之间存在相关，并且指令i是load指令时，那么第一对比较器的输出将和一个来自ID段的、指明当前指令i与i+1是load指令i的信号一起产生一个控制信号，并将流水线的前三段停止一个周期。

TYP流水线的互锁硬件还需要处理控制相关引起的流水线相关。处理分支指令引起的控制相关的互锁机制如图2.21所示。正常情况下，每个周期IF段访问I-cache进行取指，同时将PC加1并准备取下一条指令。当取出一条分支指令并译码之后，IF段将停止取指，直到分支指令通过ALU段。在ALU段中，同时产生了分支条件和目标地址。在下一个时钟周期，分支指令处于MEM段。如果发生分支，分支条件将控制多路选择器并将分支目标地址写入PC。这种处理方式说明，只要遇到分支指令，就会有4个时钟周期的开销。另一种处理方式是，假设分支不发生，IF段继续顺序地获取下一条指令。当确定要发生分支时，分支指令在MEM段用目标地址改写PC，并且从IF，ID，RD和ALU中顺序取出的指令将被作废并从流水线中清除掉。在这种处理方式下，只有当分支确实发生时，才会有4个周期的开销。如果分支最后没有发生，就没有任何开销。

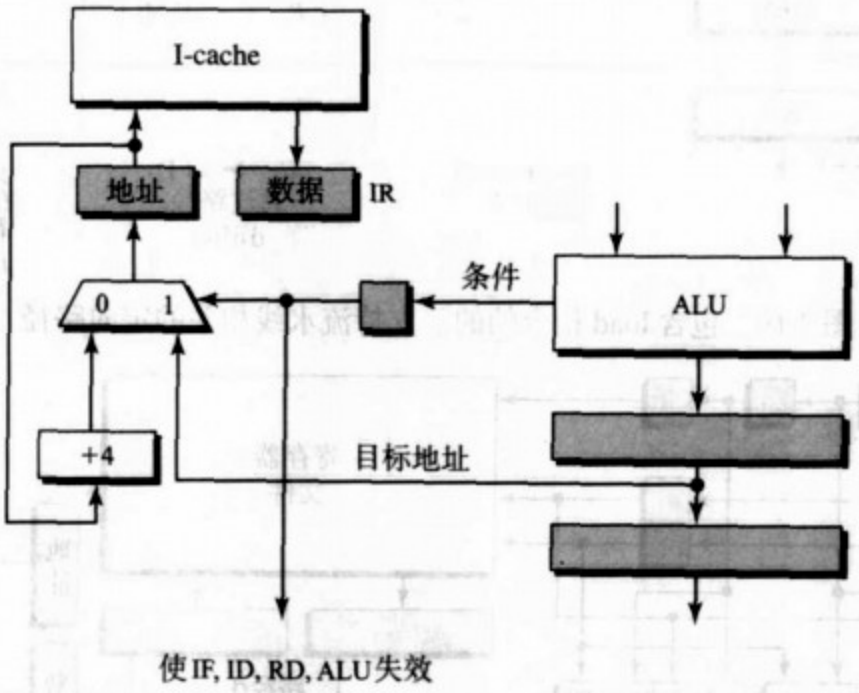


图2.21 包含分支指令的相关流水线互锁机制

2.2.4 产品化的流水线处理器

流水线技术已经成为一个成熟而广泛采用的技术。RISC方法在指令流水化方面的适应性相当重要，并且已经得到很好的应用。流水线技术同样也成功地应用到了CISC体系结构中。本小节将重点介绍两个具有代表性的流水线处理器。我们将MIPS R2000/R3000作为RISC处理器的代表，而将Intel i486作为CISC处理器的代表，并且提供了来自IBM的一项关于RISC流水线处理器研究的实验数据，它反映了标量流水线处理器的特性和性能，这项研究由Tilak Agerwala和John Cocke于1987年完成。

2.2.4.1 RISC流水线处理器实例

MIPS是一个32位的RISC体系结构，它有三种不同的指令格式，如图2.22所示。

其指令可以分为4种类型：

- 计算指令完成算术、逻辑和移位操作。这类指令可以采用R类型的指令格式，这时所有的操作数和结果都是寄存器；或者采用I类型的指令格式，这时其中一个操作数通过指令的立即数域指定。

- load/store指令在存储器和寄存器之间移动数据。这类指令采用I类型的指令格式。惟一的寻址模式是基址寄存器加上立即数域所指定的偏移量。
- 跳转和分支指令掌握程序的控制流。跳转是无条件的，使用J类型的指令格式，目标地址是26位target域和PC高4位所组成的绝对地址。分支是有条件的，使用I类型的指令格式，目标地址通过PC加上立即数域中16位的偏移量获得。
- 其他指令用于完成协处理器中的操作，以及其他一些特殊的系统功能。0号协处理器是系统控制协处理器，CP0指令完成存储器管理和异常处理。浮点指令作为协处理器指令实现，由一个特殊的浮点处理器来执行。

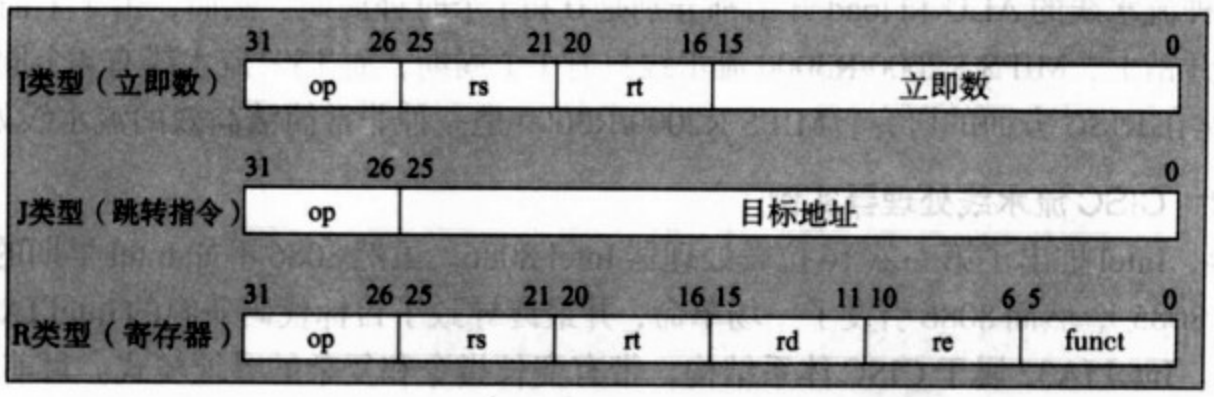


图 2.22　MIPS 指令集体系结构中的指令格式

MIPS R2000/R3000流水线是一个5级指令流水线，与TYP流水线很相似。然而，流水线的每一段又进一步分成两个独立的相位（phase）并表示为相位1（ϕ1）和相位2（ϕ2）。每一段和相应的相位所完成的功能参见表2.7。

表 2.7　MIPS R2000/R3000 5 级流水线的功能描述

级	相位	功能描述
1. IF	ϕ1	利用TLB转换虚指令地址
	ϕ2	利用物理地址访问I-cache
2. RD	ϕ1	从I-cache中返回地址并进行标记和奇偶性校验
	ϕ2	读取寄存器文件，若存在分支则生成目标地址
3. ALU	ϕ1	执行ALU操作，若存在分支则检查分支案件
	ϕ2	结束ALU操作，若存在load/store指令，则转换虚地址
4. MEM	ϕ1	访问D-cache
	ϕ2	从D-cache中返回数据并进行标记和奇偶校验
5. WB	ϕ1	写寄存器文件
	ϕ2	---

该5级流水线包含了一些有趣的特征。对I-cache的访问需要一个完整的时钟周期，但实际上是在IF段的ϕ2和RD段的ϕ1进行的。可以使用一个TLB来为I-cache和D-cache进行地址转换。为访问I-cache而进行的地址转换在IF段的ϕ1访问TLB，为访问D-cache而进行的地址转换在ALU段的ϕ2访问TLB，而D-cache的访问在MEM段进行。寄存器文件每个时钟周期首先执行写（WB段的ϕ1），然后执行读（RD段的ϕ2）。这条流水线需要一个三端口（两个读，一个写）的寄存器文件，一个单端口的I-cache和一个单端口的D-cache。

通过从ALU和MEM段输出到ALU段输入的定向路径，所有的ALU指令引起的相关都不会带来任何开销。通过从MEM段输出到ALU段输入的定向路径，load指令引起的相关在最坏情况下只有一个周期的开销。分支的开销也只有一个周期。R2000/R3000有如下几个特点，首先，分支指令只使用PC相对寻址模式。不像寄存器那样只能在RD段访问，PC在IF段之后也可以访问。

因此，分支目标地址可以在RD段计算出来，尽管需要附加一个加法器。第二个特点是，分支指令不会生成并存储任何显式的条件位。分支条件在ALU段的φ1处通过比较参考寄存器的内容而产生。通常，如果分支条件在ALU段（第3段）产生，取指在IF段（第1段）完成，分支开销将会是两个周期。然而，在这一特别的流水线设计中，I-cache的访问要到IF段的φ2处才开始。这样，由于分支条件在ALU段的φ1结束时已经产生，而直到IF段的φ2才开始访问I-cache，因此，在分支条件的控制下，RD段中计算出的分支目标地址可以在访问I-cache之前进入PC。所以分支指令只有一个周期的开销。

在减少流水线相关带来的开销方面，5级的MIPS R2000/R3000流水线要比6级的TYP流水线好一些。两种流水线的ALU和load开销都分别是0和1个时钟周期。然而，由于上面所介绍的特点，在分支开销上，MIPS R2000/R3000流水线只有1个周期，而TYP流水线有4个周期。得益于Stanford大学在RISC方面的研究，MIPS R2000/R3000是一种非常简洁高效的流水线处理器。

2.2.4.2 CISC流水线处理器实例

1978年，Intel推出了第一款16位微处理器Intel 8086。虽然8086不如Intel早期的8位微处理器（8080和8085），然而8086引发了一场革命，并最终导致了目标代码兼容的Intel IA32系列微处理器的出现。Intel IA32属于CISC体系结构，带有变长指令和复杂的寻址模式。目前，Intel IA32在销售量和应用软件上占有绝对的统治地位。1985年，Intel推出IA32系列的32位实现——Intel 386。IA32系列的第一个流水线实现是Intel 486并于1989年推出。最初的8086芯片只有不到3万个晶体管，而486芯片超过了100万个晶体管。486与之前所有的IA32系列微处理器目标代码兼容。在20世纪90年代早期，它成为最受欢迎的用于个人电脑的微处理器。

486采用了一个5级指令流水线，表2.8描述了每一段的功能。指令预取部件通过总线接口部件将16字节的指令块取到预取队列中。在取指段中，每条指令都从32字节的预取队列中取得。指令译码分成两段，硬连线的控制信号和微指令都在指令译码阶段产生；执行阶段完成ALU操作和cache访问；地址转换和有效地址生成都在指令译码阶段完成；存储器访问在执行阶段完成。因此，使用存储器读出的数据将不会引入任何开销，执行段的输出定向到它的输入。然而，如果一条指令产生了一个寄存器结果，而后面的指令要使用这个寄存器来进行地址生成，那么一个时钟周期的开销是不可避免的，因为地址生成是在指令译码阶段进行的。流水线的第5段完成寄存器写回。浮点操作由一个片上浮点部件完成并且需要多个时钟周期。

表2.8 Intel 486 5级流水线功能描述

级	功能描述
1. 取指 指令	从32位预取队列中取指 预取单元对该队列实现入队和清空操作
2. 解码-1 指令	将指令转换为控制信号以及微码级地址。 并初化地址及内存
3. 解码-2 指令	访问微码级内存 将微指令输出至执行单元
4. 执行	执行ALU以及内存访问操作
5. 寄存器 回写	将结果写回寄存器

通过5级指令流水线，许多IA32指令只需要一个周期就能完成。一些指令则需要使用微指令，因此仍然需要多个时钟周期。显然，通过指令流水化，486的性能获得了很大提高。对指令组合和常用指令的执行时间进行测试的结果显示，Intel 386的指令平均执行周期数（CPI）为4.9，而流水化的Intel 486能达到1.95，加速比约为2.5。而用术语表达就是，5级i486的有效流水程度为2.5。

显然，这其中包含了巨大的流水线开销，这主要是IA32指令集体系结构的复杂性和保持目标代码兼容性所造成的。尽管如此，对于一个CISC体系结构，该加速比已经相当可观。486充分体现了将CISC体系结构流水化的可行性。

2.2.4.3　标量流水线处理器的性能

Tilak Agerwala和John Cocke于1987年撰写了一篇关于IBM在流水线RISC处理器方面的研究报告，该报告对标量流水线RISC处理器的性能做了评价。本章引用了这篇报告中的一些重要论述。在他们的研究中，假设采用分离的I-cache和D-cache，I-cache每个时钟周期能提供一条指令并且只有load/store指令访问D-cache。两个cache的命中率都假设为100%，默认的延迟都是一个周期。以下是研究中要用到的统计信息：

1. **动态指令集合：**
   a. ALU指令：40%（寄存器－寄存器）
   b. load指令：25%
   c. store指令：15%
   d. 分支指令：20%
2. **动态分支指令集合：**
   a. 无条件分支：33.3%（总是发生跳转）
   b. 条件分支－发生跳转：33.3%
   c. 条件分支－不发生跳转：33.3%
3. **load调度：**
   a. 不能被调度：25%（没有填充延迟槽）
   b. 可以向后移动一条或两条指令：65%（填充2个延迟槽）
   c. 可以向后移动一条指令：10%（填充1个延迟槽）
4. **分支调度：**
   a. 无条件分支：100%可调度（填充1个延迟槽）
   b. 条件分支：50%可调度（填充1个延迟槽）

处理器的性能可以用每条指令的平均执行周期数来估计。标量流水线处理器的理想目标是达到CPI=1，这意味着流水线平均每个周期处理或者完成一条指令。IBM的研究对这一理想目标进行了量化。首先，假设ALU指令没有额外开销，load指令和分支指令有两个周期的额外开销（即需要3个周期完成）。给定上述动态指令集合，可以计算出由于load指令和分支指令而引入的CPI开销。

- load指令开销：$0.25 \times 2 = 0.5$ CPI
- 分支指令开销：$0.20 \times 0.66 \times 2 = 0.27$ CPI
- 总的CPI为：$1.0 + 0.5 + 0.27 = 1.77$ CPI

由于有25%的动态指令是load指令，并且假设load指令的开销为两个周期，CPI的开销就是0.5。假设流水线认为分支指令不发生或者偏向于不发生，那么只有66.6%的发生跳转的分支指令存在两个周期的开销。同时考虑load指令和分支指令的开销，总的CPI就是1.77。这与理想目标（CPI=1）相去甚远。

假设load指令可以使用一条定向路径绕过寄存器文件，其开销可以从两个周期减为一个周期。假设存在这样一条定向路径，CPI可以减为1.0+0.25+0.27=1.52。

此外，还可以使用编译器将其他指令调度到load和分支指令的延迟槽。根据上述统计信息，由于有65%的load指令可以向后移动一条或两条指令，10%的load指令可以向后移动一条指令，总共就有75%的load指令可以进行调度，因此其开销可以再减少一个周期（完全消除了额外的开销）。有33.3%的分支指令是无条件分支，通过调度可以将其开销从两个周期减为一个周期。由于流水线认为分支不发生，因此没有发生跳转的条件分支指令就没有额外的开销，这部分指令占分支指令的33.3%。剩下33.3%的分支指令是发生了跳转的条件分支指令，假设其中的50%可调度，即可以向后移动一条指令，因此发生了跳转的条件分支指令中有50%只有一个周期的开销，另外50%才有正常的两个周期的开销。新的CPI开销和总的CPI计算如下：

- load指令开销：0.25 × 0.25 × 1 = 0.0625 CPI
- 分支指令开销：0.20 × [0.33 × 1+0.33 × 0.5 × 1+0.33 × 0.5 × 2] = 0.167 CPI
- 总的CPI为：1.0+0.063+0.167 = **1.23** CPI

通过调度load指令和分支指令的延迟槽，load指令和分支指令引入的CPI开销大大减少。1.23的CPI已经很接近CPI=1的理想目标。分支指令带来的开销仍然很大，进一步减少这部分开销的方法之一，就是要考虑如何减少两个周期的分支开销。IBM的研究表明，如果不采用寄存器间接寻址模式，将有90%的分支可以通过PC相对寻址模式编码。如果采用PC相对寻址模式，生成分支目标地址就不需要访问寄存器文件。可以加入一个单独的加法器，这样目标地址的计算和寄存器的访问可以同时进行。因此，对于采用PC相对寻址模式的分支指令，分支的开销可以减少一个周期。33.3%的分支指令是无条件分支，这些指令是100%可调度的，因此，分支的开销只有一个周期。如果其中的90%可以采用PC相对寻址模式，从而消除了分支的开销，那么将只剩下10%的无条件分支会带来一个周期的开销。无条件分支相应的CPI开销就是0.20 × 0.33 × 0.10 × 1=0.0066 CPI。

采用PC相对寻址模式之后，取指段就不再默认分支不发生。不管分支是否发生，都可以用同一种方式处理条件分支。根据一个条件分支是否可以采用PC相对寻址模式和是否可以进行调度，一共有4种可能情况。这4种可能情况下的开销参见表2.9。

表2.9 考虑PC相对寻址以及利用延迟槽进行调度时，条件分支所产生的开销

PC相对寻址	可调度指令	分支指令产生的开销
Yes (90%)	Yes (50%)	0时钟周期
Yes (90%)	No (50%)	1时钟周期
No (10%)	Yes (50%)	1时钟周期
No (10%)	No (50%)	2时钟周期

66.6%的分支是条件分支，包括发生和不发生跳转的指令。通过表2.9中的各种情况可以得到条件分支产生的开销，即0.20 × 0.66 × {[0.9 × 0.5 × 1]+[0.1 × 0.5 × 1]+[0.1 × 0.5 × 2]}=0.079 CPI。无条件分支和条件分支所引入的开销总和为0.0066+0.079=0.0856 CPI。加上前面load指令的开销，总的开销及CPI如下所示：

- load指令开销：0.0625 CPI
- 分支指令开销：0.0856 CPI
- 总的CPI为：1.0+0.0625+0.0856=**1.149** CPI

通过一系列的改进，CPI由1.77降到了1.15，这与理想目标CPI=1相当接近。从另一个角度来看，CPI=1代表理想的指令流水线，其中每一个时钟周期都有一条新的指令进入流水线。这只有当所有的指令都是独立的指令这一条件得到满足时，这一目标（第二条流水线处于理想状态）才能实

现。而实际的程序中则存在指令相关。CPI=1.15表明，在能够处理指令相关的实际指令流水线的设计中，只引入了15%的开销或者失效，这充分反映了指令流水线的效率。

2.3　深流水线处理器

流水线技术是提高处理器性能的一种很有效的方法。同时，一些强有力的因素推动我们采用深流水线。深流水线增加了流水段的数目，同时减少了每一段中门级逻辑的数目。深流水线的主要好处是能够缩短机器周期时间，从而提高时钟频率。20世纪80年代的大多数流水线微处理器只有4级到6级流水线，而现代高端微处理器的时钟频率则达到了GHz级，流水线深度超过20级。流水线不仅仅越来越深，而且越来越宽，例如超标量处理器。由于流水线越来越宽，流水线每一段的复杂性也相应增加，这样每一段的延迟也会增加。要保持同样的时钟频率，较宽的流水线就要做得更深。

流水线的深度存在一个下限。对于深流水线，化解流水线相关的开销更大。图2.23说明了当流水线变得更宽更深时，ALU、load和分支等相关开销的变化。比较图中一条浅流水线和一条深流水线，我们可以看到，ALU的开销从0个周期增加到1个周期，load开销从1个周期增加到4个周期。最重要的是，分支开销从3个周期一下上升到11个周期。随着流水线开销的增加，平均CPI也相应增加。这样，由深流水线中高时钟频率所获得的潜在性能就会被CPI的增加抵消。要保证深流水线的总体性能有所提高，频率的增加必须超过CPI的增加。

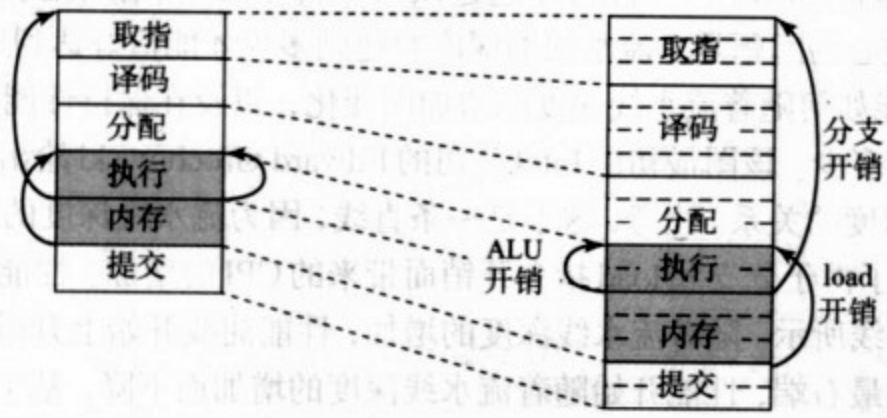


图2.23　通过增加流水线深度而降低ALU指令、load指令及分支指令的开销

为了降低深流水线中由于分支开销产生的负面影响，存在着两种相应的方法，如图2.24所示。在三种流水线开销中，分支开销最大，因为它跨越了前端所有的流水段。对于一次错误预测的分支，前端流水线中所有的指令都要被清除掉。减少分支开销的第一个方法是，减少前端流水线的段数。例如，带有变长指令格式的CISC体系结构需要很复杂的指令译码逻辑，这样就需要较长的流水线。通过使用RISC体系结构，可以减少译码的复杂性，这样前端流水线的段数就比较少。另一个例子是，在将指令装载到I-cache之前使用预译码逻辑对指令进行预译码。经过预译码的指令所需要的译码逻辑较少，因此所需要的流水线段数也较少。

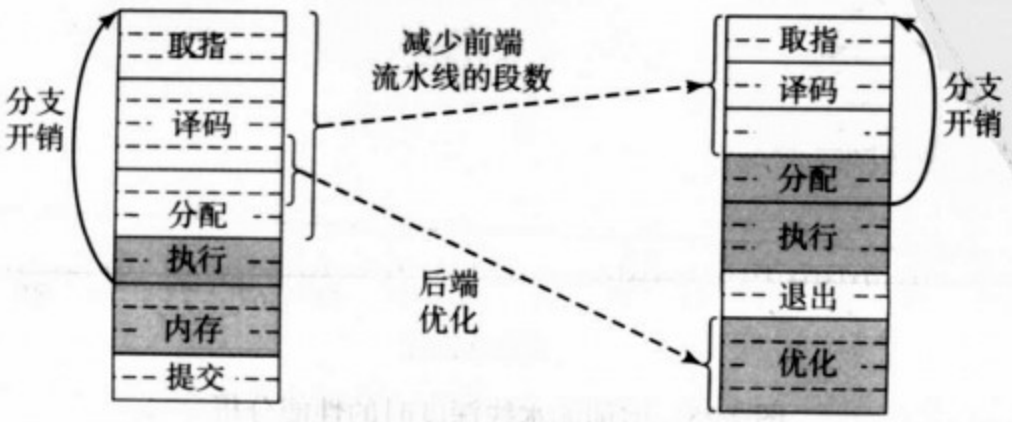


图2.24　降低深流水线中分支开销产生的负面影响

第二种方法是将前端流水线的一部分移至后端，这样流水线的前端保持较少的流水段，因此分支开销也要小一些。该方法是当前研究的一个热点。当指令序列反复在流水线上执行时，前端流水线对同样的指令重复执行同样的工作，即取指、译码以及分配。一些人建议，可以将这些工作的结果缓存起来重复利用，而不需要重新执行。例如，将已经译码的指令存放到一个特殊的cache中，如果以后需要读取同样的指令，可以通过访问这个cache完成，这样就可以跳过译码段。除了只是缓存已经译码的指令以外，可以对这些指令实现进一步的优化，使得前端流水线的段数进一步减少。以上的缓存和优化过程都可以作用在流水线的后端，而不影响前端的深度和相应的分支开销。为了获得提高时钟频率所带来的好处，必须将流水线的开销控制在一定的范围内。

在设计深流水线时，存在各种不同形式的权衡。在2.1节中曾提到，相对于非流水设计，一个k级流水线的吞吐率可能会提高k倍。当将价格考虑进去时，就存在价格和性能的权衡。该权衡决定了k的最佳值不能为任意大小。图2.3对此进行了说明。这类权衡涉及的是实现流水线的硬件代价，它表明流水线的深度存在一个极限，如果超过了这个极限，那么在硬件上的开销相对于所获得的性能提高就不太划算。

基于深流水线与CPI之间的分析，还存在另外一种形式的权衡，就是对提高时钟频率和增加CPI的权衡。根据处理器性能的Iron定律，性能决定于时钟频率和平均IPC的乘积，或者频率与CPI之比。当流水线的深度增加时，频率提高的同时CPI也在增加。只有在性能上出现净增长，增加流水线深度才是可取的。因此存在一个临界点，超过该临界点时，继续增加流水线的深度对提高性能的帮助不大。我们所关心的问题是，流水线的深度要达到多少才能恰好达到这个点？

图2.25反映了性能如何随着流水线深度的增加而变化，以及在保持性能得到有效提高的情况下，流水线深度能达到多少。该图最初由Intel公司的Edward Grochowski绘出。图中最陡峭的曲线说明了频率与流水线深度的关系。注意，这不是一条直线，因为流水线深度的增加会开来一些额外的开销。CPI曲线显示了由于分支和load指令开销而带来的CPI的增加。性能通过频率与CPI之比来计算，如中间那条曲线所示。随着流水线深度的增加，性能曲线开始上升得非常快，然后逐渐变得平缓。在这条曲线的最右端，性能开始随着流水线深度的增加而下降。基于以上分析，我们可以认为，在保证流水线能够获得性能提高的前提下，流水线的深度能达到57级！

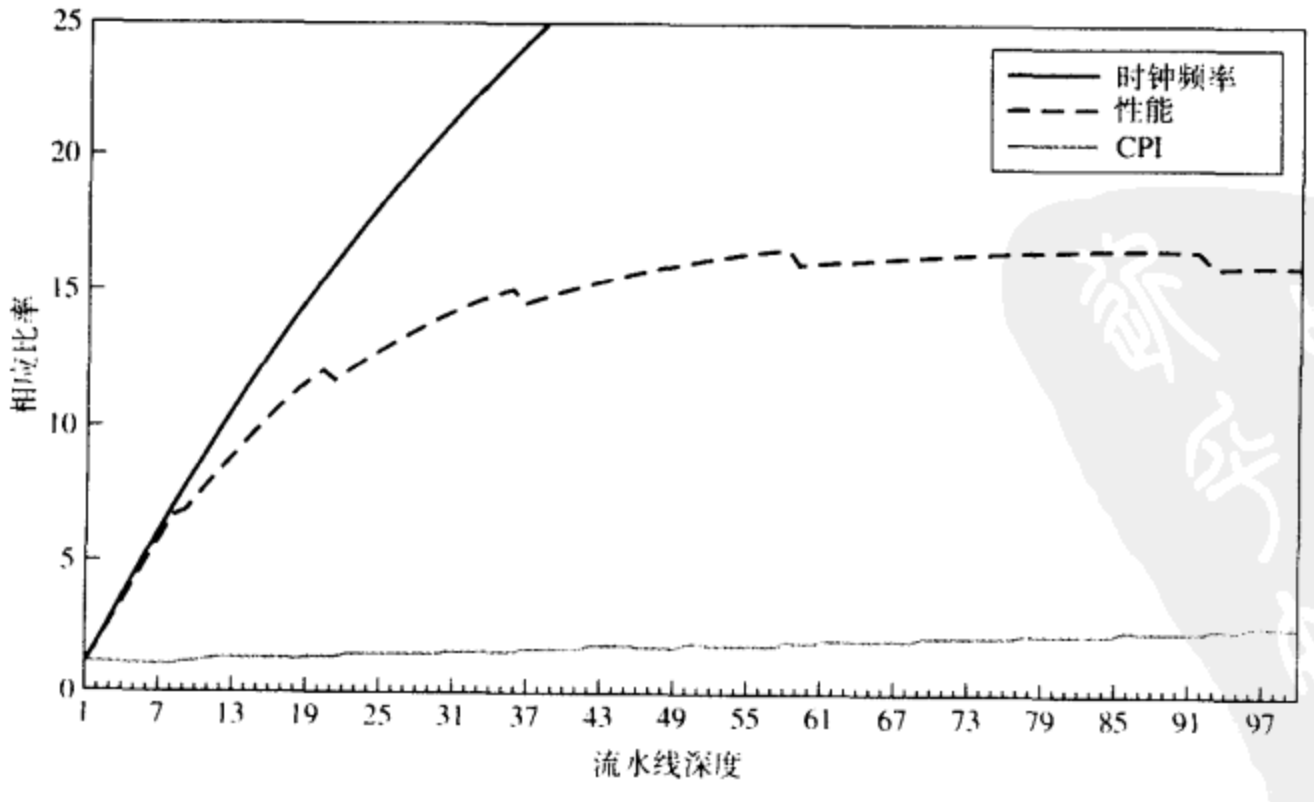


图2.25 增加流水线深度时的性能分析

人们从未停止过对更高时钟频率的追求。在这个10年内很有可能达到10 GHz，要想进一步提高时钟频率，就需要更深的流水线。

2.4　小结

流水线技术是一项可以应用于任何ISA的微体系结构技术。RISC体系结构的特性确实能够使流水线设计更加容易，也更加有效。然而，流水线技术同样适合于CISC体系结构。流水线技术已经被证明是提高处理器性能的一项十分强大的技术，并且在流水线深度方面仍然有很多上升空间。不难想像，以后将会有更深的流水线出现。

流水线处理器性能的主要障碍是指令相关引起的流水线停顿。其中，控制相关引起的分支开销是最主要的部分。动态分支预测能够减少这一部分的开销，只有当分支预测失误时才会带来分支开销。当分支预测正确时，流水线没有停顿。然而，当检测到预测失误时，必须清空流水线。

随着流水线深度的增加，分支开销也会增大并成为主要的挑战。策略之一是通过减少流水线前端的段数来减少分支开销，流水线前端指的是从取指段到完成分支指令的相应段的距离。另一种策略是增加动态分支预测算法的准确性，从而降低预测失误的频率，因此引入分支开销的频率也得到了降低。本章我们没有讨论动态分支预测，这是一个很重要的课题并将在第4章中加以讨论。

流水线处理器设计改变了CPU设计的传统观念。传统的设计观念将处理器设计分成数据通路设计和控制通路设计，数据通路设计着重于ALU和其他功能单元以及寄存器访问的设计，而控制通路设计将重点放在译码状态机的设计和控制信号序列的产生上，这些控制信号用来正确地操纵数据通路。这样的设计观念已经不太恰当。对于流水线处理器，这种划分已不是很明显。指令在译码段进行译码，经过译码的指令，包括相应的控制信号，被传递到流水线的下一段并由后面的各段使用。流水线的每一段只用到译码之后的指令的某一个域以及相应的控制信号，基本上不需要集中地控制。流水线处理器使用的是一种分布控制，通过控制信号在流水线中的传递来实现。传统的、通过控制状态的先后顺序来处理指令的方式，现在已被穿越流水线各段的方式取代。从根本上说，不仅仅是数据通路被流水化，控制通路也被流水化了。此外，传统的数据通路和控制通路现在已经被合并到同一条流水线中。

2.5　习题

1. 公式2.4反映了理想流水线的性能与流水线深度的关系，这看起来很像Amdahl定律。描述这两个公式中项与项之间的关系并给出一个直观的解释，为什么这两个公式如此相似？
2. 使用参数G、T、L和S，通过公式2.7可以计算出价格性能比最优的流水线深度k_{opt}，计算2.1节中流水化浮点乘法器的k_{opt}，用芯片数量作为价格参数（芯片数目G=175，每个集成的锁存器上的芯片数目为L=82/2=41），用给出的延迟作为T和S参数（T=400 ns、S=22 ns），计算出的k_{opt}与书中建议的流水线设计有何不同？
3. 讨论并找出两个原因，为什么公式2.4只能大概估计流水线的潜在加速比。
4. 考虑给TYP指令集及其流水线增加一条装载立即数的指令，这条指令从指令字中提取出16位的立即数值，然后带符号扩展成32位并存放到指令字中指定的目标寄存器中。由于立即数提取和符号扩展不需要ALU，有人建议这样的指令可以在ID段将结果写回寄存器。用图2.15描述的相关检测算法，分析这样的设计会带来何种相关。
5. 如果不考虑流水线互锁硬件（在第6题中讨论），第4题中经过变化的流水线还需要什么额外的资源？讨论这些资源以及它们的价格。

6. 考虑第4题中经过变化的流水线，重新绘制图2.18中的流水线互锁硬件，使之能够正确地处理装载立即数指令。
7. 考虑给TYP指令集及其流水线增加字节宽度的ALU指令，这些指令的语义与已经有的单字宽度的ALU指令相同，只是源和目的操作数都只有一个字节的宽度。字节宽度的操作数存放在与单字宽的操作数相同的寄存器中，占用其中的低字节部分，并且寄存器写只能影响低字节部分（即高字节部分保持不变）。重新绘制图2.18中的RAW流水线互锁检测硬件，使之能够正确地处理新增加的ALU指令。
8. 考虑给TYP流水线增加一条带有索引寻址模式的store指令，与已经存在的“寄存器+立即数”寻址模式不同的是，这种store指令通过两个源寄存器的和来计算有效地址，即指令stx r3, r4, r5执行r3←MEM[r4+r5]。讨论在TYP流水线中要支持这样的指令需要什么硬件资源，讨论这种指令的优点和缺点。
9. 考虑增加一条load-update指令，这种load指令的有效地址通过“基址寄存器+立即数”来计算，然后将所得到的结果写回基址寄存器。即指令lwu r3, 8(r4)执行r3 ← MEM[r4+8]；r4 ← r4+8。讨论在TYP流水线中支持这样的指令需要什么硬件资源。
10. 考虑第9题中经过变化的流水线，重新绘制图2.20中的流水线互锁硬件，使之能够正确处理load-update指令。
11. 给定2.2.4.3节中描述的IBM的研究数据，计算增加一个第0级的数据cache后的CPI，该第0级数据cache能够在一个周期内提供数据，命中率为75%。第0级和第1级cache并行工作，这样当第0级cache失效时，第1级cache能够在下一个周期返回数据并产生一个load延迟槽。假设在load延迟槽可填充和不可填充的情况下，第0级cache的命中率的分布是一致的。
12. 给定第11题的假设，如果第1级cache是顺序访问的，即在第0级cache失效之后访问，将产生两个load延迟槽，而不是一个。计算这种情况下的CPI。
13. 在带有一个数据cache的TYP流水线中，load指令在检查标识数组以确定是否命中的同时访问数据数组，读取相应存储地址中的数据。而store就要麻烦一些，因为在覆盖数据之前必须先检查标识，否则，如果cache失效，就会导致错误的结果。设计一种解决方案，使之不需要向cache发送两次请求，换句话说，使流水线遇到load指令时不会停顿。参考图2.15，考虑是否有新的RAW，WAR或者WAW相关。
14. 表2.7所示的MIPS流水线采用了两相时钟机制，充分利用了共享TLB（用于虚实地址转换），因为取指在第1阶段访问TLB，读取数据在第2阶段访问TLB。然而，当处理条件分支时，分支目标地址和分支失败的后续地址都要在第1阶段进行虚地址转换，也就是在ALU段的第1阶段并和分支条件的检查同时进行，这样才能够保证在第2阶段取指，而无须考虑分支目标还是后续地址。这样似乎需要一个双端口的TLB。从体系结构上找出一种解决方案，避免使用双端口TLB。

第3章　超标量结构

我们已经看到，流水线技术能够有效地提高处理器的性能，但是在第2章中提到的标量流水线存在着很大的局限性。现实生活中的应用对于计算机性能的需求是无止境的，因此如何克服标量流水线的局限性就成为迫切需要解决的问题。超标量流水线技术打破了这些局限性。

超标量机器的本质特点是可以同时发射多条指令，从而克服了单指令流水线每周期只能发射一条指令的限制，并通过配置多个功能部件，增强了并行执行程序的能力，提高了吞吐率。超标量处理器的另一个主要特点是“乱序执行”（out-of-order execution），即不按照程序给定的原始指令顺序执行。程序中指令的先后顺序意味着指令之间的某种优先性，而实际上这种优先性往往是不存在的。因此，在处理器中引入“乱序执行”来克服不必要的优先性限制，不必重写源程序就可实现更大的并行性。本章和接下来的几章将会系统地介绍超标量处理器设计的相关知识。本章主要讨论超标量流水线的结构，在第4章中将讨论在指令执行过程中超标量处理器需要解决的若干问题，而在第5章和第6章中则分析两个商品化的超标量处理器IBM PowerPC 620和Intel P6。

3.1　标量流水线的局限性

标量处理器只有一条指令流水线（假设有k段），所有的指令都要流经相同的流水段。同一时刻一个流水段内最多只能有一条指令，而整条流水线内的所有指令都前后紧接、步调一致地向前流动。指令一般是在一个流水段中停留一个时钟周期，在下一个周期流入到下一个流水段，除非流水线发生了停顿。严格的标量流水线有以下三种局限性，在后面几节将会详细地讨论：

1. 标量流水线最大的吞吐率不会超过每周期一条指令。
2. 将不同类型的指令放在一条流水线中处理，效率低下。
3. 为了保证流水线的步调一致而插入的停顿使流水线产生很多气泡。

3.1.1　标量流水线吞吐率的上限

正如在第1章中提到的，根据公式3.1，通过提高IPC和主频，或者降低总的指令数量均可以提高处理器的性能。

$$\text{性能} = \frac{1}{\text{指令数}} \times \frac{\text{指令}}{\text{周期}} \times \frac{1}{\text{周期时间}} = \frac{IPC \times \text{频率}}{\text{指令数}} \qquad \text{（公式3.1）}$$

采用深流水线可以提高主频。深流水通过减少每段的逻辑门级数降低时钟周期，从而提高主频。然而，深流水是以增加流水线的硬件开销为代价的。同时，在解决指令相关时增加了损失的时钟周期数，潜在地降低了处理器的CPI（平均每条指令执行的周期数），主频提高所带来的好处被抵消了。

不考虑流水线的深度，标量流水线在每个时钟周期最多只能启动一条指令执行，因此标量流水线的IPC（平均每周期完成的指令数）不会超过1。为了提高吞吐率，需要使IPC大于1，特别是当深流水线不再经济的时候，很有必要设计一种能够在同一周期启动多条指令执行的方案，因此需要

增加流水线的宽度以适应多条指令同时位于同一流水段的情况。我们称这种流水线为并行流水线（parallel pipeline）。

3.1.2 低效的统一流水线

我们曾经对流水线做过这样的假设：重复进入流水线的计算在流水线中的处理过程是一样的。但是对于指令流水线，情况就不同了。不同类型的指令需要进行不同的子运算操作，把它们集中在一条流水线中完成，实现起来比较困难而且低效。在第2章中，TYP流水线统一处理不同类型的指令，虽然前几段（IF、ID和RD段）指令的操作是一样的，但是在执行阶段（ALU和MEM段）则互不相同。实际上，正是由于浮点指令同其他指令很难统一，在TYP中才省略了浮点指令的实现。在RISC中，浮点指令就曾因带有CISC的复杂特性而被认为违反了RISC的设计原则。

某些特殊类型的指令很难统一到单一流水线中，比如浮点指令和部分定点指令（乘法、除法等指令），因为它们的执行需要多个时钟周期，执行延迟不定或较长的指令很难和延迟为一个周期的指令统一在一条流水线中。鉴于CPU和存储器之间的速度差距越来越大，访存指令的延迟（以机器周期的数目计算）也会越来越大。除了执行延迟上的不一致，不同类型指令所需的硬件资源也不尽相同。随着对硬件性能需求的不断提高，不同类型的指令将采用不同的专用执行部件，这使得流水线的执行阶段更难统一。

所以，对于未来的高性能微处理器，强制所有指令在一条流水线中执行几乎是不可能的。并行流水线为了解决这一局限性，在流水线的执行阶段为不同类型的指令提供了不同的执行部件（子流水线），我们称这种流水线为多配置流水线（diversified pipelines）。

3.1.3 严格流水线导致的性能损失

标量流水线是严格的（rigid），所有指令在其中流动时必须前后紧接，步调一致。指令按照程序给定的顺序流入流水线，如果没有流水线停顿，所有指令将同步流动并且不会打乱指令的原始顺序。一旦某条指令由于和前导指令之间产生了相关性而导致停顿，那么该指令保持在当前段不动，而流水线中所有的前导指令可以继续流动。根据标量流水线的严格性，如果发生相关的指令的当前流水段为第i段，那么所有前面的流水段，即保存着后续指令的段1、2、…、i–1，都必须停顿，直到第i段指令所需的操作数通过定向得到之后，所有i段流水线的停顿才能解除。消除了指令之间的相关性之后，停顿的流水段就可以再次同步地向前流动了。因此，在严格的标量流水线中，某段的停顿将导致它之前所有的段停顿。

标量流水线中的停顿将向后传递并导致了不必要的流水线气泡，从而使某些流水段空闲。虽然第i段的指令由于和前导指令的相关而停顿，但是后续指令中某些指令可能和前导指令不存在任何相关。比如这条指令在第i–1段，它的停顿就是不必要的。按照程序的语义，这条指令不必在第i–1段等待。如果该指令可以绕过发生停顿的第i段（通过旁路逻辑），它就可以流入到后续流水段继续执行，这样可以有效地减少一个周期的停顿开销，如图3.1所示。如果多条指令可以绕过停顿的流水段，就可以减少多个周期的开销，从而使得流水线表面上看起来一直在执行有效指令。理论上，所有的停顿开销都可以被消除。这种允许后续指令绕过停顿指令的技术称为指令的乱序执行（out of order execution）。严格的标量流水线不支持指令的乱序执行，为了解决指令之间的相关，必须插入不必要的停顿周期。支持指令乱序执行的并行流水线称为动态流水线（dynamic pipeline）。

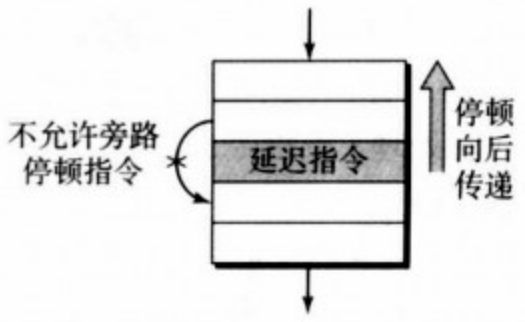


图 3.1　严格流水线中，停顿向后传递导致不必要的停顿周期

3.2　从标量流水线到超标量流水线

超标量流水线可以看做是从标量流水线衍化而来，它消除了前面提到的标量流水线的限制。超标量流水线属于并行流水线，因为它每个时钟周期可以启动多条指令执行。同时，它又属于多配置流水线，因为在执行阶段提供了多个不同的功能部件。此外，为了在没有编译器干预的情况下实现高性能，它还可以采用动态流水线技术。下面我们将详细讨论超标量流水线的这三个特点。

3.2.1　并行流水线

机器的并行度可以用任一时刻能够同时处理的最大指令数来衡量。一条 k 级的标量流水线中可以同时有 k 条指令在处理，因此相对于非流水的机器而言，它可以获得 k 倍的加速比。而这一加速比同样可以通过并行使用 k 台非流水机器来获得。两种实现如图 3.2（b）和图 3.2（c）所示，可以分别看做是机器的时间并行和空间并行。程度相同的时间并行和空间并行理论上可以获得相同的加速比，但显然时间并行比空间并行要节省硬件资源，因为空间并行要重复配置多个处理单元。并行流水线是时间并行和空间并行的结合体，是获得指令高吞吐率的有效途径，如图 3.2（d）所示。

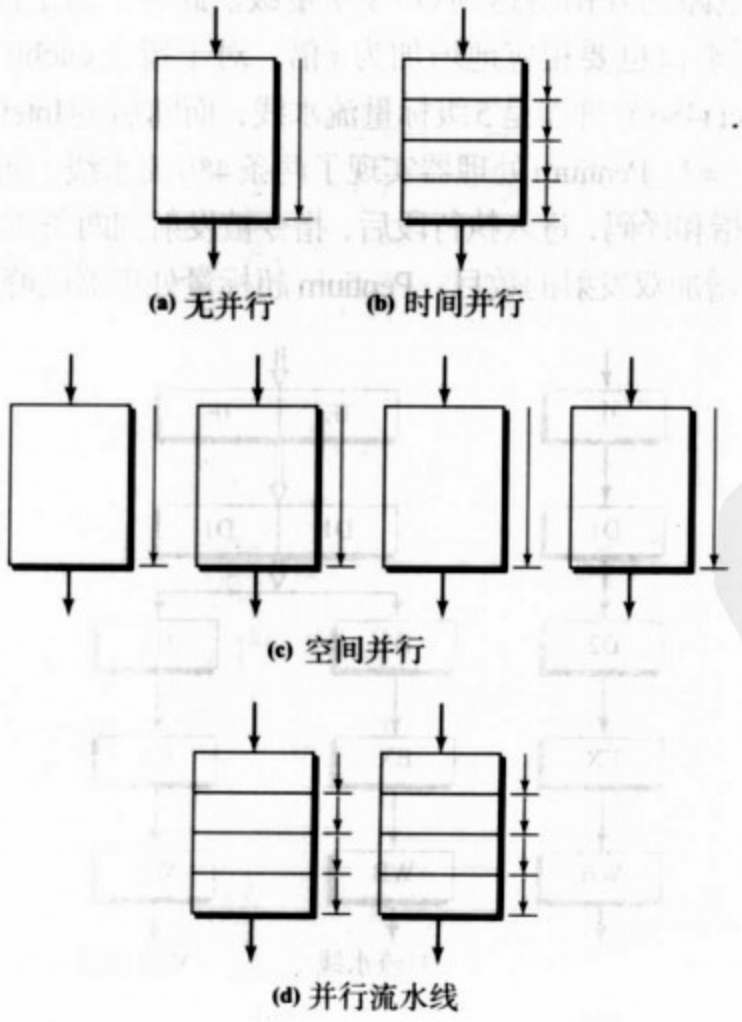


图 3.2　机器并行性：（a）无并行（无流水）；（b）时间并行性（流水线）；（c）空间并行性（多部件）；（d）时间并行和空间并行融合

标量流水线的加速比一般针对非流水机器，它取决于流水线的深度。而并行流水线的加速比，或者说超标量流水线的加速比往往相对于标量流水线来计算，它取决于并行流水线的宽度。宽度为 s 的并行流水线的每一段都可以同时处理 s 条指令，其相对于同一深度的标量流水线的加速比为 s。图 3.3 显示了一个 $s=3$ 的并行流水线。

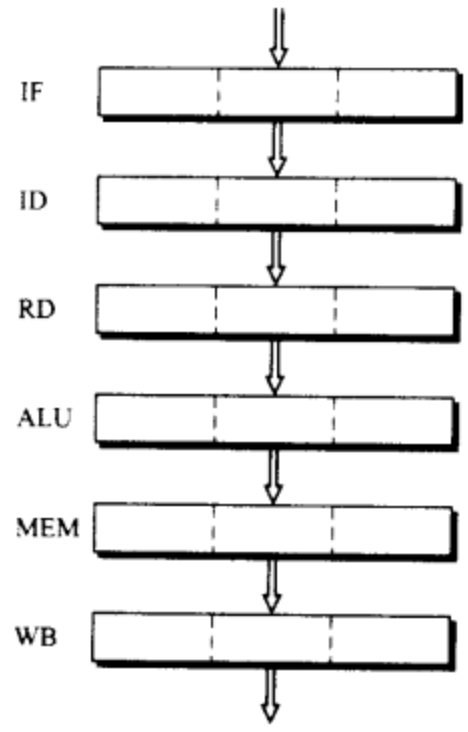


图 3.3 s=3 的并行流水线

实现并行流水线需要增加大量的硬件资源。由于任一流水段每周期都可以处理 s 条指令，所以每一段逻辑的复杂度也相应地增加到 s 倍。在最坏的情况下，例如在相邻两段的指令缓冲区之间采用 $s \times s$ 的交叉开关互连，电路的开销将达到 $O(s^2)$ 量级。此外，为了使 s 条指令能同时访问寄存器文件，寄存器文件的读写端口也要相应地增加为 s 倍。对于指令 cache 和数据 cache 也是一样的。

如第 2 章中提到的，Intel i486 处理器是 5 级标量流水线，而随后的 Intel Pentium 处理器则是一个超标量流水线的处理器，宽度 $s=2$。Pentium 处理器实现了两条 486 流水线，如图 3.4 所示。在流水线的前两段可以同时对多条指令取指和译码，进入执行段后，指令被发射到两条流水线中，即 U 流水线和 V 流水线，这样做的目的是尽量增加双发射的数目。Pentium 超标量处理器的峰值速度是每周期两条指令。

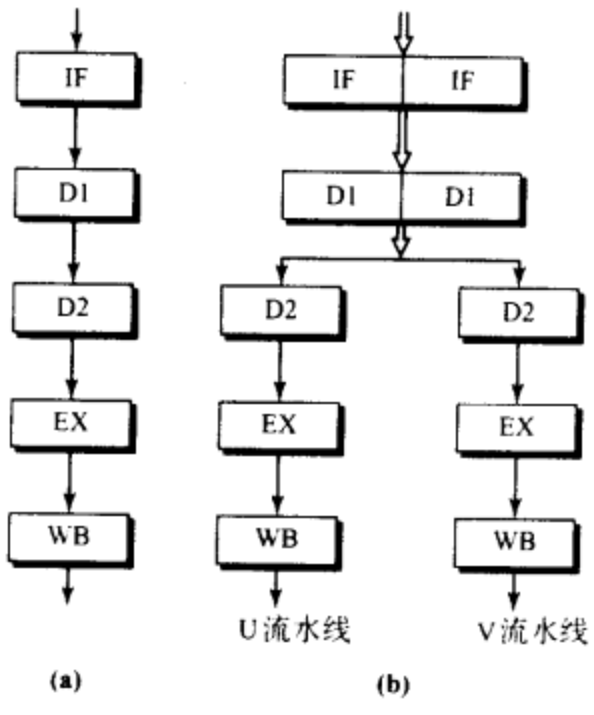


图 3.4 （a）5 级 i486 标量流水线；（b）s=2 的 5 级 Pentium 并行流水线

同 i486 的标量流水线相比，Pentium 的并行流水线需要增加大量的硬件资源。5 级流水线的宽度加倍，从而两条执行流水线每段每周期可以处理两条指令，这都需要大量的硬件支持。执行段可以处理 ALU 操作或访问数据 cache 的操作，因此对于同时处理两条 ALU 指令的情况，必须增加额外的寄存器端口。如果同时执行的是两条 load/store 指令，数据 cache 也要配置双端口，而双端口 cache 实现的代价是很高的，因此 Pentium 的数据 cache 是用单端口 8 路交叉的 cache 来实现的，可以同时处理 U 和 V 流水线中两条访问不同存储体的 load/store 指令。如果一个存储体发生冲突，即两条 load/store 指令访问同一个存储体，处理器会串行化这两个访问。

3.2.2　多配置流水线

不同类型的指令执行所需的硬件资源是截然不同的。标量流水线中将执行不同类型指令的硬件集中在一条流水线中实现，因此效率很低。既然每种类型的指令只用到执行部件的一个子集，没有必要流过执行部件的所有段，所以在不必要的流水段中，这些指令只有一个空操作而已。所有指令的延迟是相等的，都等于流水线的长度。这种实现给后续指令带来了不必要的停顿，并且需要额外的定向电路。

单一流水线造成的低效性在并行流水线中得到了解决。并行流水线在执行段配置了多个功能部件并形成了多条流水线。但对于宽度为 s 流水线，并非是 s 条同样的流水线。在执行段，流水线的类型是不同的。如图 3.5 所示，对于 4 条不同类型的执行流水线，其功能部件和深度也各不相同，RD 段负责将指令按照类型分派到各条流水线。

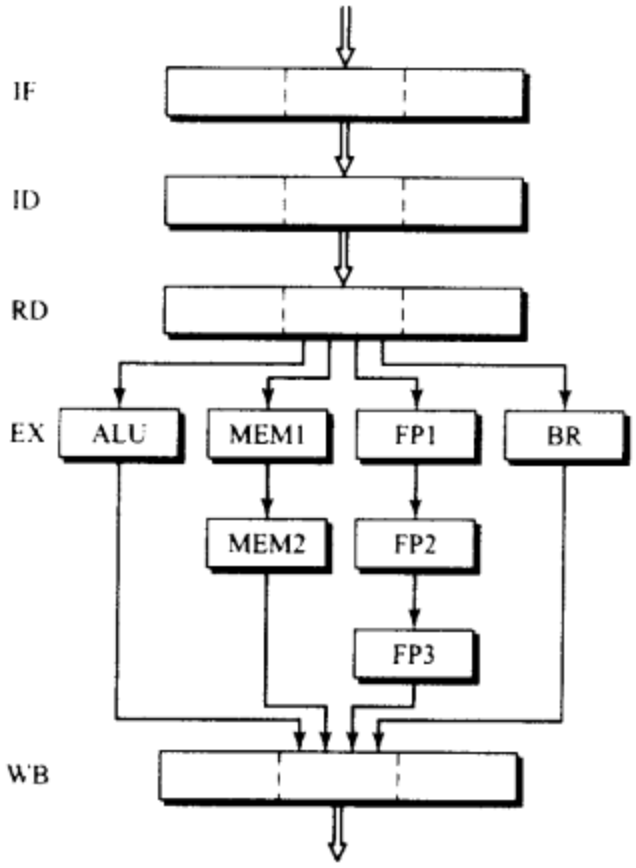


图 3.5　4 条执行流水线的多配置并行流水线

多配置流水线有很多优点。每条流水线只执行一种指令，因此可以根据其特点进行专门的优化设计并采用高效的硬件实现。每条指令只需执行必要的延迟并流过该子流水线所有的段。这种实现比起配置s条同样的标量流水线要高效得多。如果在分派之前就可以解决所有不同类型指令间的相关，那么一旦指令进入了各自的流水线，就不会因为同其他流水线的指令存在相关而导致停顿，这意味着把相关控制的复杂度分布在了各条流水线中。

多配置并行流水线的设计有一些需要注意的地方。理想情况下，功能部件的数目应该和程序能够达到的最大并行度匹配，而类型分配应该遵循程序中指令出现的动态特征。在大多数第一代超标量处理器中，只是简单地在标量流水线之外增加了一条处理浮点指令的浮点流水线。随着超标量流水线从双发射发展到4发射，一般的处理器在执行段都配置了4个功能部件：整数、浮点、load/store和分支部件。在一些最新的设计中还配置了多个整数部件，其中一些用来处理长延迟的整数操作，如乘法、除法以及其他一些图形图像和信号处理应用的操作。

同流水线技术一样，通过配置多种多样的功能部件来设计高性能CPU的技术也早就出现了。CDC 6600集成了流水线技术和多部件技术，它配置了10个不同的功能部件，如图3.6所示。10个功能部件读取24个操作寄存器中的数据，这些寄存器包括8个地址寄存器（18位）、8个索引寄存器（18位）和8个浮点寄存器（60位）。10个部件功能相互独立，包括1个定点加部件（18位）、一个浮点加部件（60位）、两个乘法部件（60位）、一个除法部件（60位）、一个移位部件（60位）、一个布尔计算部件（60位）、两个增量（increment）部件和一个分支部件。CDC 6600微处理器采用了流水线结构，在执行段前还有两级译码段，但功能部件都是非流水的并存在不同的执行延迟，如定点加法部件的延迟为3个时钟周期，而浮点乘法（除法）则需要10（29）个时钟周期。CDC 6600 CPU的设计目标仅仅是保持指令发射频率为每机器周期1条指令。

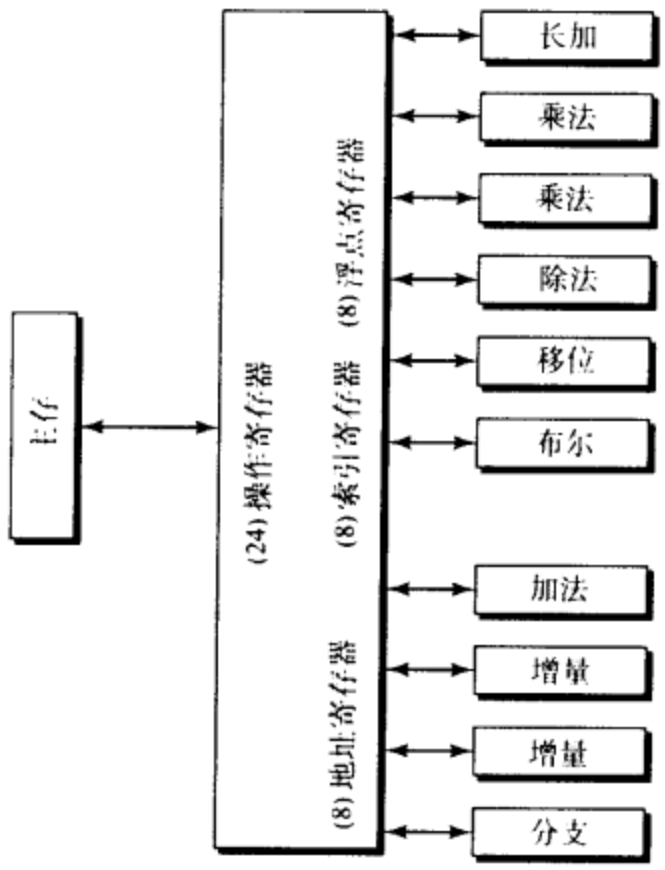


图3.6　10个功能部件的CDC 6600 CPU

另外，还有一种超标量微处理器也采用了和CDC 6600相同的多配置功能部件的策略。在IBM和苹果公司合作推出PowerPC之前，Motorola设计了一个十分简洁的超标量微处理器，这就是88110。有趣的是，88110也是采用10个功能部件，如图3.7所示，它包括两个整数部件、一个位操作部件、一个浮点加部件、一个乘法部件、一个除法部件、两个图形处理部件、一个load/store部件和一个指令顺序控制／分支部件。大多数部件是单周期延迟，只有除法部件是多周期延迟。除了除法部件以外，其余都是流水线设计。在功能部件的数目及类型分配方面，88110是超标量设计的一个典范。

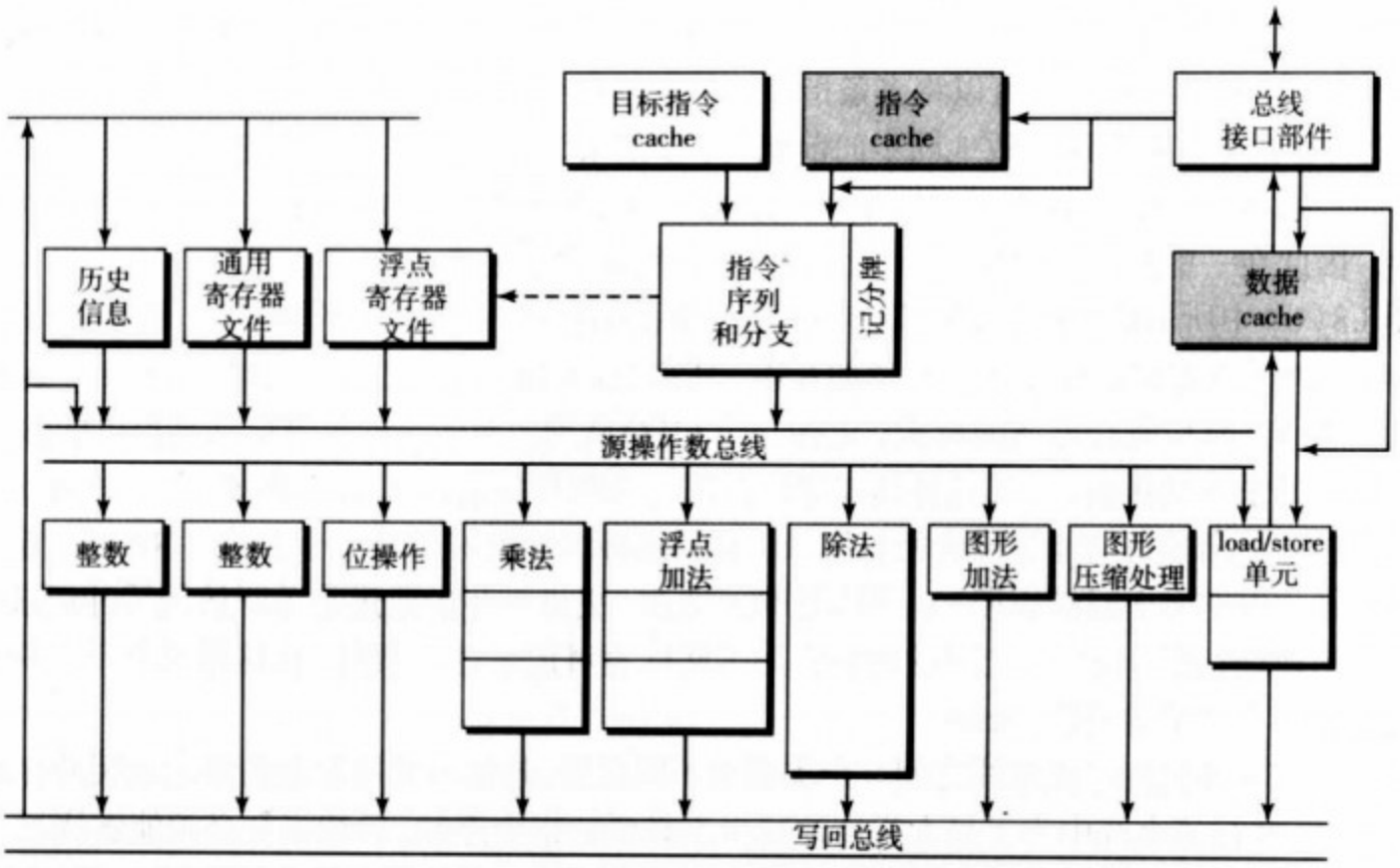


图 3.7　Motorola 88110 超标量微处理器

3.2.3　动态流水线

在流水线设计中，段间的缓冲是必需的。在严格的标量流水线中，相邻段之间（段 i 和段 i+1）往往使用单记录缓冲，如图 3.8（a）所示。缓冲保存所有刚刚从第 i 段流出的控制信息和数据，并在下一周期送入第 i+1 段。单记录缓冲控制起来十分简单：在每个周期开始时，缓冲中当前的内容将输入到第 i+1 段，并在结束时锁存第 i 段的输出结果。一般情况下，缓冲每周期都刷新，除非当前保存的指令必须被保持而禁止流入下一段，这时缓冲时钟无效，指令停顿在缓冲内。显然，在严格标量流水线中，第 i 个缓冲一旦停顿，所有第 i 段以前的流水段都必须停顿。因此，在没有停顿发生时，每条指令在一个缓冲中仅保留一个周期，然后就流入下一级缓冲。所有指令都按照程序给定的原始顺序依次流过各级缓冲。

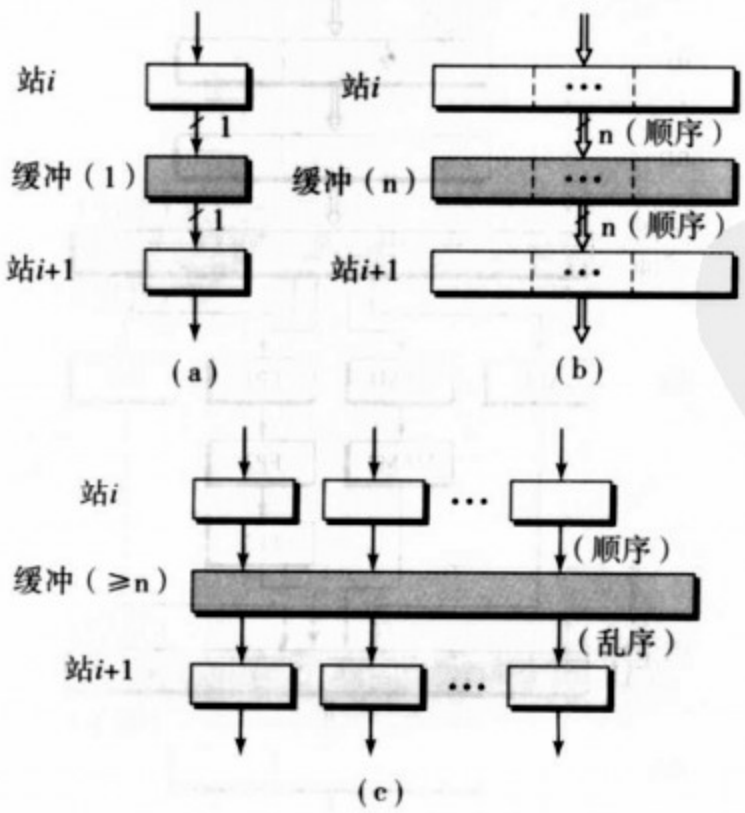


图 3.8　站间缓冲：（a）单记录缓冲；（b）多记录缓冲；（c）再定序多记录缓冲

并行流水线相邻段间则使用多记录缓冲，如图3.8（b）所示。多记录缓冲可以看做是单记录缓冲的简单扩展，每个时钟周期可以将多条指令的信息锁存到缓冲中，而在下一周期送入下一级流水段。如果所有指令需要步调一致地同步向前流动，多记录缓冲的控制就和单记录缓冲一样简单，所有记录同步地刷新或停顿即可。但是，在并行流水线中，这种控制方式会导致某些指令产生不必要的停顿，因此在一些高效的设计中，往往采用更为复杂的多记录缓冲。

图3.8（b）中所示的多记录缓冲只有一个统一的读端口和一个统一的写端口，各条记录之间没有互连。一种改进方案是在记录之间增加互连，使得数据可以在各记录之间移动。比如，这些记录可以组织成线性移位寄存器链的形式，就像一个FIFO队列。另外一种改进措施是提供能够独立访问缓冲各条记录项的机制，使其具备显式寻址各条记录项和控制各条记录项读写端口的能力。如果缓冲每个输入输出端口均能够寻址所有记录，那么这种缓冲就好像是一个多端口RAM，指令可以在一条记录中保留多个时钟周期，也可以更新或修改。更进一步的改进是关联访问缓冲记录：即不按照传统的地址索引，而是将记录的内容作为关联标签对记录进行索引。在这种机制下，多记录缓冲类似于一个小容量的关联cache。

超标量流水线同标量流水线之间一个关键的不同点是，超标量使用复杂的多记录缓冲存放正在执行的指令。并行流水线中为了最大程度地减少不必要的指令停顿，后续指令必须能够绕过停顿的前导指令，而这将会导致指令的原始顺序在执行时发生改变。通过指令的乱序执行，可以克服指令流的相关限制，从而开发潜在的并行性，即指令所需的操作数一旦可用，指令就开始执行。支持乱序执行的并行流水线称为动态流水线。动态流水线通过使用复杂的多记录缓冲来实现乱序执行，并允许指令以可变的顺序进入和离开缓冲。图3.8（c）中给出了一个再定序多记录缓冲的示意图。

图3.9给出了一个发射宽度s=3的动态多配置并行流水线，其执行部分包括4个流水化功能部件，前后各有一个再定序多记录缓冲。第一个缓冲称为分派缓冲，它按照程序原始顺序载入已经译码的指令，然后按不同的顺序分派到各个功能部件，即指令进入分派缓冲是顺序的，流出后是乱序的。此外，流水线中各功能部件的执行延迟不同，属于多配置流水线。

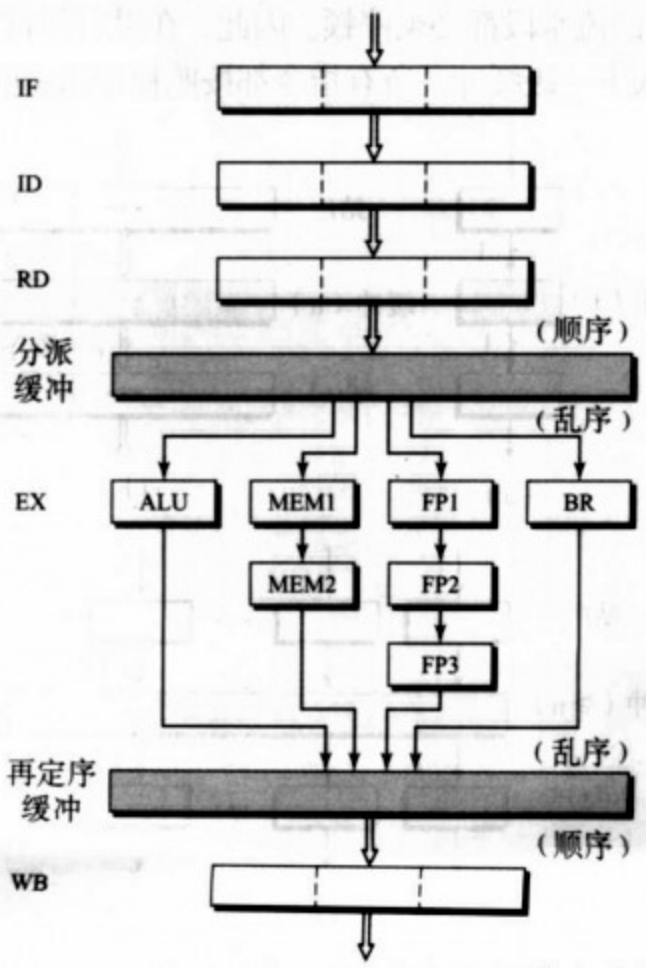


图3.9　宽度s=3的动态流水线

由于指令以乱序流入功能部件，并且各功能部件的延迟也不相等，因此指令的结束也是乱序的。为了保证能够按照程序原始顺序正确地处理异常，指令的提交（即对机器状态的修改）必须是顺序的。在指令乱序结束的情况下，要实现顺序提交，必须在执行段的末尾再添加一个再定序缓冲。该缓冲有时又称为完成缓冲，存放已经执行结束的指令，并按程序的原始顺序将指令提交给写回段并修改机器状态。动态流水线结构通过乱序执行可以获得最短的程序执行时间，并且按程序的原始顺序提交指令并修改机器状态，从而实现了精确中断。

3.3　超标量流水线综述

本节将讨论超标量流水线设计中的几个关键问题并着重讨论结构设计。有关微处理器动态执行中的技术问题以及指令的执行优化将在第4章讨论。本章的重点是微处理器的结构设计，而第4章的重点是微处理器和程序之间的相互影响。

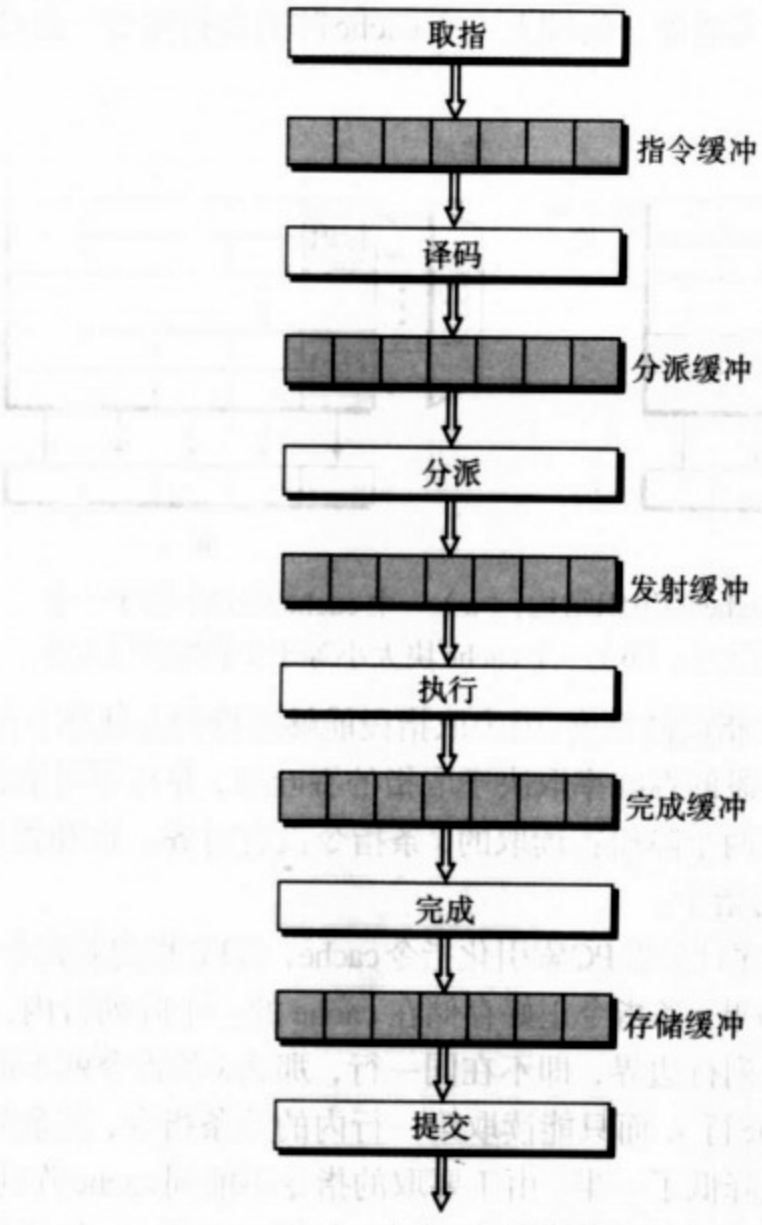


图3.10　6站TEM超标量流水线

在第2章中，我们曾使用6级TYP流水线为例讨论了标量流水线的设计，同样，本章以图3.10所示的TEM流水线为模板来讨论超标量流水线结构的设计。同标量流水线相比，超标量流水线的实现更加复杂和多样化。TEM流水线并不是哪种典型超标量流水线的具体实现，它的6级流水线仅仅是一个逻辑上的流水线，它同物理上的实现不是一一对应的。但使用TEM提供的超标量流水线框架足以讨论一般的超标量微处理器结构所需要处理的6个主要任务。

TEM的6级流水线是取指（Fetch）、译码（Decode）、分派（Dispatch）、执行（Execution）、完成（Complete）和提交（Retire）。执行段包括多个不同类型的流水化功能部件，而且各个功能部件的延迟互不相同。分派段负责将不同类型的指令发射到相应的功能部件中。由于指令在执行段是乱序的，因此完成段必须将指令重排序，以保证按照程序的原始顺序更新机器状态。此外，流水段之间使用多记录缓冲隔离，缓冲的复杂度取决于它们在流水线中的位置和功能。接下来的几节我们将依次讨论6级流水段的功能和设计上需要考虑的问题。

3.3.1　取指

与标量流水线不同，由于超标量流水线是并行流水线，因此每周期可以从指令cache中取出多条指令。如果流水线宽度为s，那么取指段每拍就可以从指令cache中取出s条指令，这意味着指令cache在物理组织上必须足够宽，cache每行要能够存储s条指令来满足流水线的一次取操作。在后面的讨论中，我们总是假设访问指令cache的延迟为1，取指宽度和cache行的大小相等。在这种cache结构中，一个cache行的大小可能等于物理上一个cache阵列的行宽度，也可能等于几个阵列行宽度，如图3.11所示。

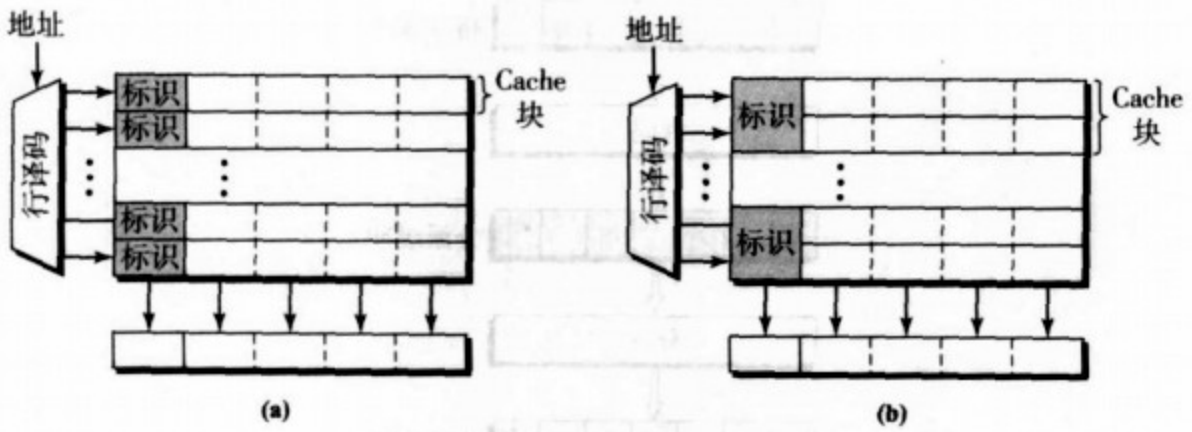


图3.11　指令cache的组织结构：（a）一个cache块大小等于一个物理行大小；（b）一个cache块大小等于2个物理行大小

取指段的设计目标就是增大取指段的宽度，因为取指段能够维持的吞吐率直接影响到整个超标量流水线的吞吐率，后续几个流水段的吞吐率取决于取指的吞吐率，并且不可能超过它。要实现每周期s条指令的取指吞吐率主要有两个障碍：读取的s条指令没有对齐，这和指令cache的物理组织有关；控制流改变了取指集中的指令。

在每个时钟周期，取指段用程序计数器PC索引化指令cache，将PC指向的指令连同后续的s-1条指令（共s条指令）一同取出。如果s条指令正好存储在cache的一个阵列行内，将该行一次取出即可；但是如果s条指令跨越了阵列行边界，即不在同一行，那么s条指令就不能在一个周期内读出（假设每周期只能读取一个cache行），而只能读取第一行内的几条指令，其余的指令要等待下一周期。在后一种情况下，取指带宽降低了一半。由于要取的指令不能同cache阵列行边界对齐，s条指令的读取需要两个时钟周期，从而导致取指带宽的减小，如图3.12所示。如果物理上阵列的一行只对应一个cache行，如图3.11（a）所示，跨越阵列行的边界就等于跨越cache行边界，这将会引起额外的一些问题。当要取的指令组跨越两个cache行时，如果第二个cache行不在cache中，则即使第一行存在于cache中，也会引发cache失效。即使两个cache行都在指令cache中，一个周期内访问多个cache行也会产生问题。

对于对齐问题有两种解决方案。一种方法就是编译时使用的静态技术：编译器了解指令cache的组织结构，比如寻址方案以及行大小。利用这些信息，可以将指令合理地放置在存储器中，从而保证要取的指令组同物理阵列行对齐。例如，每条作为分支目标的指令都放在存储器中一行开始的

位置，从而提高了读取的 s 条指令始于行首的概率。这种技术在实际中已经加以应用并证明是有效的，但有一个问题就是当目标代码调入一个不同组织结构的指令 cache 中时，可能会发生不对齐的情况。另外一个问题是静态代码占据了大范围的地址，这间接导致了 cache 的命中率下降。

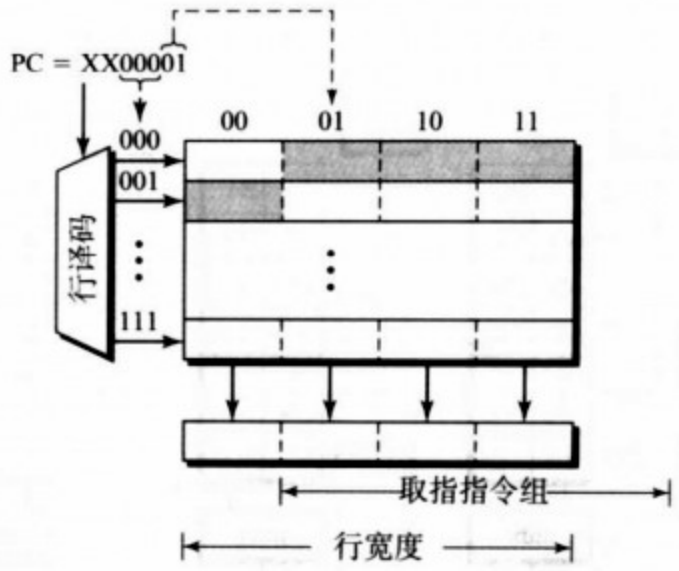


图 3.12　指令 cache 的行边界和取指指令组没有对齐

第二种方案是执行时由硬件协助解决，即使指令组跨越了阵列行边界（不是cache行地界），对齐硬件也能保证一个周期内取出 s 条指令。IBM RS/6000 提供了这种硬件支持，下面简要介绍一下它的设计方案。

RS/6000 的指令 cache 是两路组相联，cache 行大小为 16 条指令（64 个字节）。cache 的每个阵列行存储 4 个关联组（每组有两条指令），因此每个 cache 行跨越 4 个物理阵列行。如图 3.13 所示，指令 cache 阵列由 4 个独立的子阵列组成（编号为 0, 1, 2, 3），4 个子阵列可以并行访问，每次 cache 访问可以从每个子阵列中取出一条指令。一个关联组中的两条指令（A 和 B）哪一条被取出取决于相应的标识和指令地址的匹配。在 4 个子阵列中，指令地址是交错分配的。

如果 PC 恰好指向第一个子阵列，即子阵列 0，那么连续的 4 条指令可以在 4 个子阵列中同时取出。这 4 条指令在 cache 中位于同一阵列行，4 个子阵列的访问使用同一个行地址。但是，如果PC指向的不是第一个子阵列，假设为第二个子阵列，4条连续的指令就跨越了两个阵列行。RS/6000 会检测到没有指向阵列 0 的指令地址，然后自动地将访问越界指令的行地址加 1，该功能是由每个子阵列的“T 逻辑”来实现的。例如，PC 指向阵列 2，2 号和 3 号阵列的访问可以用给定的地址，而“T 逻辑”会检测到这种情况并对访问 0 号和 1 号阵列的行地址自动加 1。因此，从子阵列 0 和 1 取出的指令实际上来自 cache 阵列的下一行。

所以，只要待取的指令组不跨越cache行边界，不管起始地址指向第几个子阵列，每周期总能取出4条连续的指令。如果待取的指令组跨越了cache行边界，则在当前周期只有放置在第一个cache行的指令可以取到。RS/6000 的 cache 行容量为 16 条指令，因此指令序列的起始地址有 16 种可能，则取指带宽的平均值为 $(13/16) \times 4+(1/16) \times 3+(1/16) \times 2+(1/16) \times 1 = 3.625$。

虽然待取的指令组可以从任意一个子阵列开始，但只有子阵列 0、1 和 2 需要“T 逻辑”，3 号子阵列的指令总是属于当前阵列行地址指向的指令组，并且不需要“T 逻辑”。RS/6000 的指令缓冲网络具有旋转功能，可以把取出的指令旋转后按照程序的原始顺序送出去。指令 cache 这种复杂的设计，保证了在待取指令组不能和阵列行对齐的情况下依然可以取得很高的取指带宽。显然，这种实现很耗费硬件资源，但由于 RS/6000 是在多个芯片上实现的，所以依然是可行的。

除了对齐问题以外，另一种限制取指带宽的因素是指令序列中控制流的改变。如果指令序列中某条指令是条件分支指令，在分支出现跳转的情况下，它后面的指令都将作废，取指带宽也将大大

减小。问题的根源在于指令间的控制相关以及条件分支的处理技术，我们将在第4章中详细地讨论控制相关的处理以及分支指令的处理。

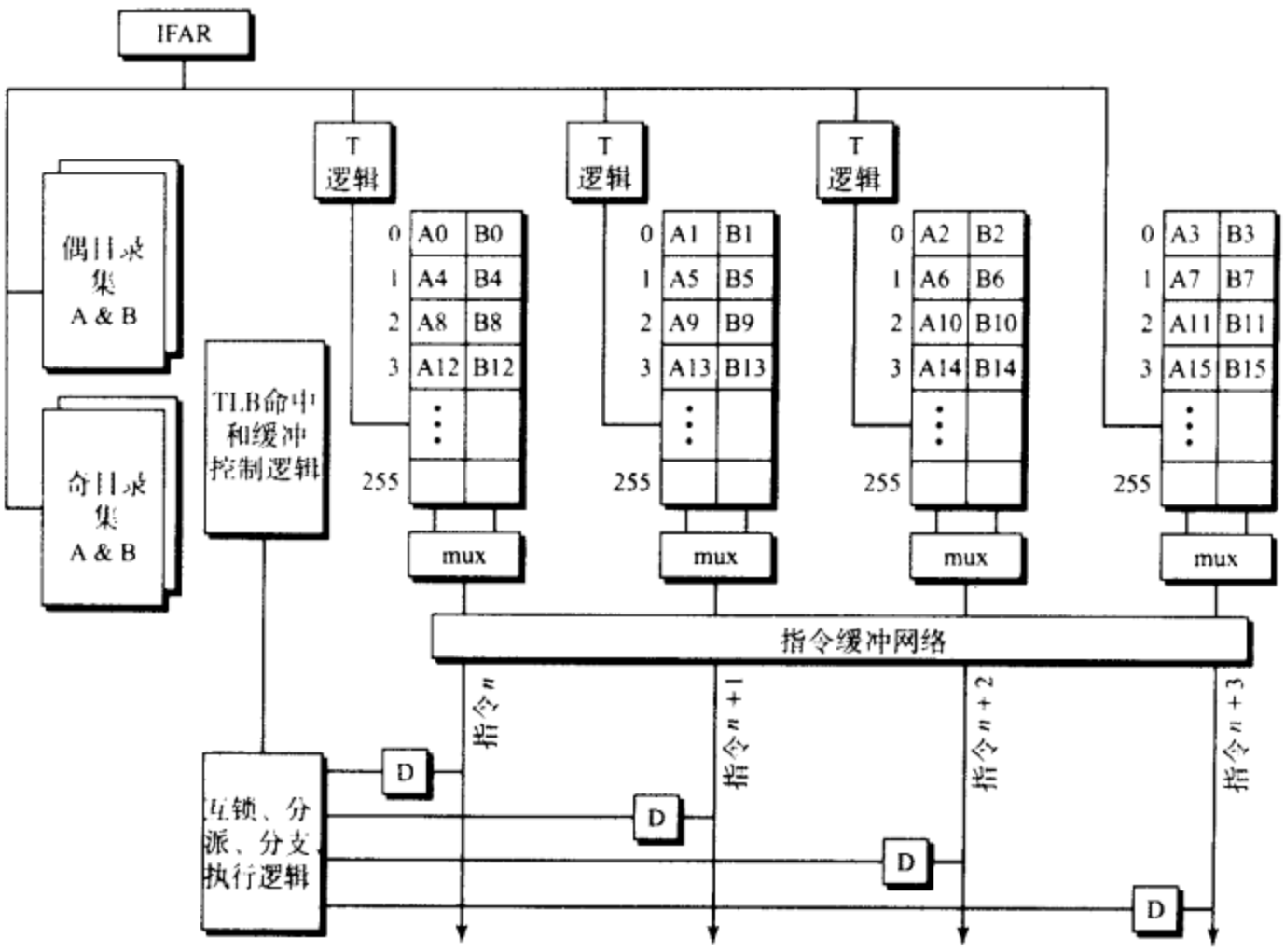


图 3.13　RS/6000 中两路组关联指令 cache 的自动对齐结构

3.3.2　指令译码

指令译码段对从cache取出但还没有分派到功能部件的指令进行译码，这主要包括指令的识别、指令类型的确定以及指令间相关性的检测。指令译码的复杂度受两个因素的影响：ISA（指令集体系结构）和并行流水线的宽度。对于典型的RISC指令集，指令是定长的且格式简单，因此译码操作比较简洁，无需确定每条指令的起止字节。相对较少的指令格式和寻址方式使得指令类型的辨别相对容易，只需对很少的几个字段进行译码（如操作码）就可以确定指令类型和格式，其他字段的意义也可以很快确定。因此，使用RISC指令集可以简化指令译码操作。

对于RISC标量流水线，译码操作十分简单，译码段一般和读寄存器段综合起来实现。而并行RISC流水线中则需要同时译码多条指令，确定指令间的相关性并调度不相关的指令执行。此外，为了提高取指段的效率，必须尽快在译码段确定其中分支指令的分支方向，以反馈给取指段从而控制正确的指令进行取指。这些操作连同寄存器的读取使RISC并行流水线的译码段逻辑变得很复杂。相关性的判断需要大量的比较器，寄存器需要配置多个读端口，操作数的传递也需要多条总线的支持。因此，译码段将成为整条超标量流水线中的关键路径。

CISC并行流水线中的译码操作则变得异常复杂，一般需要多级流水段实现。CISC并行流水线中每条指令的提取和类型鉴别不再简单，Intel Pentium和AMD K5都使用了两级流水段来译码IA32指令。在更深的Intel Pentium Pro流水线中，则使用了5个时钟周期访问指令cache和对IA32指令译码。CISC指令集中指令长度的不定导致了译码周期的不定，一条指令的长度确定以后才能进行下一条指令的译码，因此多条指令并行译码不易实现。在最坏的情况下，由于一条指令可能在所取

指令序列的任意一个字节开始，因此需要设置大量的译码器同时译码这些“可能”的指令。这种实现非常复杂而且效率很低。

CISC并行流水线的译码还有另外一个难点，即译码器必须将指令转换为内部更低一级的操作，才可以被硬件直接执行。这些内部操作和RISC指令相似，可以看做一串微指令。AMD K5中的内部操作称为RISC操作或ROP，Intel P6中则称为微操作或μop，每条IA32指令都要转换为一条或多条ROP（μop）。按照Intel的说法，平均每条IA32指令可以转换成1.5条到2.0条μop。在CISC并行流水线中，指令译码段和完成段之间执行的指令都是这种微操作。本书为了方便起见，沿用了Intel的术语，称这些内部操作为μop。

下面，以Intel Pentium Pro的指令译码为例来介绍CISC并行流水线的译码操作，如图3.14所示为P6的取指译码单元。每个时钟周期从指令cache读出16字节的指令送入指令队列，三个并行译码器同时开始对队列中的指令进行译码。第一个译码器可以对所有IA32的指令进行译码，而其他两项功能则受到限制，只能对诸如寄存器－寄存器类型的简单IA32指令译码。

译码器将IA32指令转换为内部的3地址μop，μop都是load/store模式的指令。每条具有复杂寻址方式的IA32指令都会转换成多个μop。第一个通用译码器每周期可以译码一条IA32指令，最多生成4个μop；另外两个受限的译码器分别只能译码一条简单的IA32指令，生成1个μop。每周期至少有一条IA32指令通用译码器译码并生成一个或多个μop。最好的情况下，另外两个译码器也能译出两条简单指令，跟随前面译出的指令一起流入下一段。因此，理想情况下，三个并行的译码器可以同时生成6条μop。对于需要转换成多于4个μop的复杂IA32指令，当其到达译码队列头部时，通用译码器调用μop序列发生器并提供微程序代码，实际上是预先编写好的μop序列，这些μop往往需要两个或多个周期。译出的μop送入再定序缓冲（ROB）中保存，并等待下一周期分派到功能部件。ROB有40条记录项，因此可以保存40条μop。

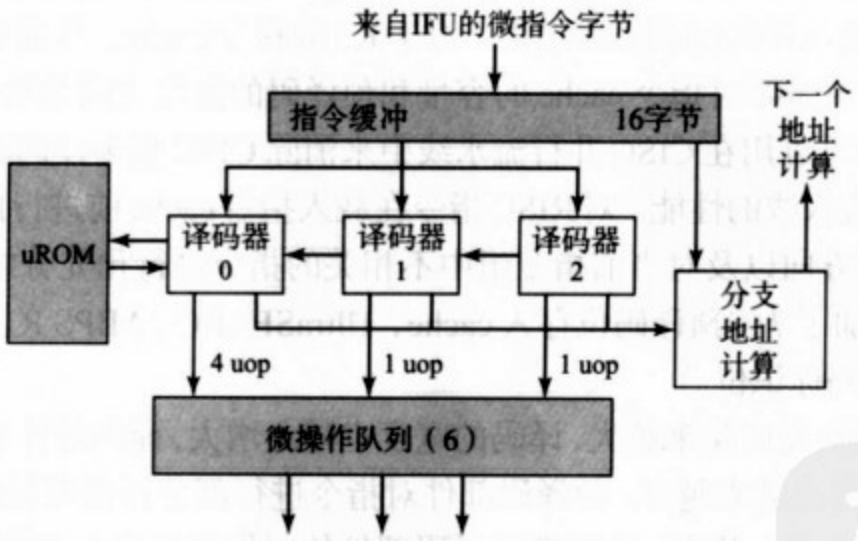


图 3.14　Intel P6超标量流水线中的取指/译码部件

超标量微处理器，特别是CISC并行流水线和多发射流水线，其指令译码部件十分复杂，往往需要几个流水段来实现。译码段站数的增加，将导致分支开销（以时钟数目计算）相应增加。因此，不提倡一味地增加并行流水线译码段的深度。为了解决复杂性，业界提出并实现了一种称为预译码（predecoding）的技术。

预译码技术将指令译码的部分工作转移到指令cache进行。当cache失效并需要从存储器调入一个cache块时，在内存与cache之间可以增加一个硬件将cache块中的指令进行预译码。译码的信息以预译码标识位的形式连同指令一起存在于指令cache中，在指令译码时可以简化操作。因此，指令的部分信息只需在载入cache时译码一次，而不必每次从cache中取出时重新译码。通过在指令载入cache时增加部分硬件资源，可以大大简化流水线中的译码逻辑。

AMD K5是CISC超标量流水线设计，它的译码过程就采用了预译码技术，IA32指令从存储器中取出之后在载入cache之前被预译码。一次总线操作可以从存储器取出8个指令字节，预译码部件对这些字节进行预译码，每个指令字节附加5位标识位，保存着一条IA32指令的起止位置、需要译出的μop数目以及操作码和前缀的位置等信息。附加位和所属指令字节一起存放在指令cache中，因此初始的cache块大小由128位（16字节）增加到208位，如图3.15所示。每次cache访问，16个指令字节连同80位附加信息一起取出。在译码段，预译码信息大大简化了译码段的工作量，4个同样的译码器可以对多条IA32指令进行译码，每周期最多可译出4条μop。

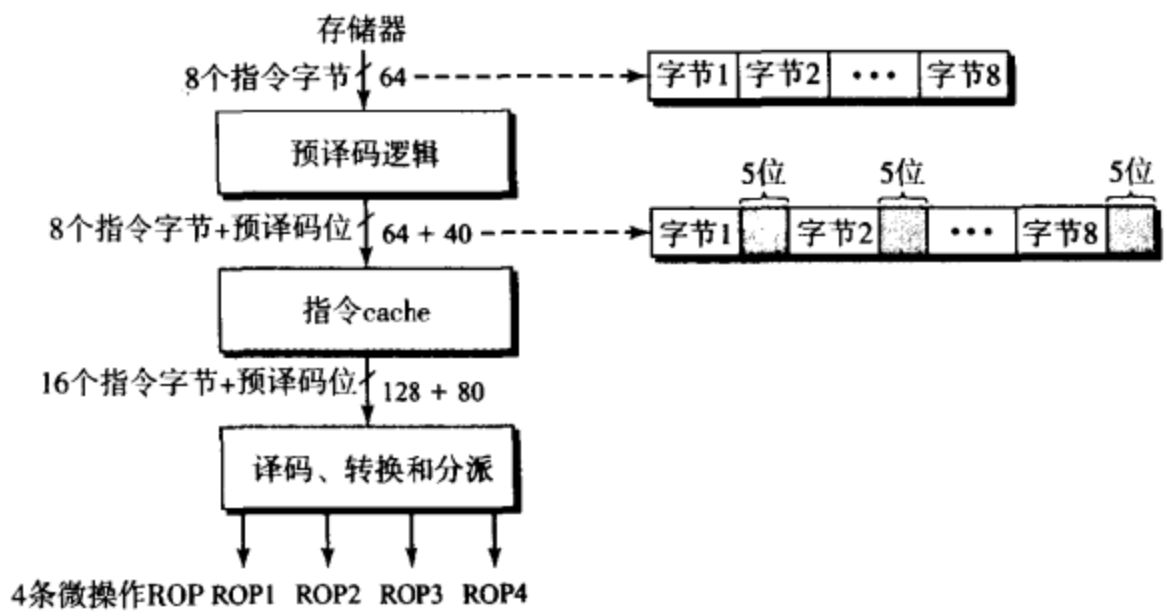


图3.15 AMD K5的预译码机制

预译码对处理器的设计主要有两种影响。一种是由于在存储器到指令cache的通路上增加了预译码操作而增加了指令cache的失效开销。不过当cache失效率很小时影响不大。另一个开销是指令cache的大小由于存储预译码标识位而增大，对于K5的指令cache，其容量大约增加了50%。因此，是否使用预译码技术需要对指令cache的容量和预译码的程度之间采取一种折中方案。

预译码技术不单单可以用在CISC并行流水线中来消除CISC指令译码时的时间瓶颈，它还可以用来改善RISC并行流水线的性能。对RISC指令在载入指令cache前进行预译码，译码信息可用来确定分支指令的分支方向以及对当前指令组中不相关的指令进行预先分组。例如，在PowerPC 620中对每条指令字附加了7位预译码位存入cache，UltraSPARC、MIPS R10000和HP PA-8000也使用了4位到5位预译码标识位。

随着超标量流水线的宽度越来越大，译码的宽度也随之增大，译码部件越来越成为性能上的瓶颈，因此预译码技术用得也越来越多。预译码部件对指令进行部分译码并保存这些信息，从而有效地减轻了译码部件的工作量。其实，可以把预译码部件的工作看做是在存储器和指令cache之间对指令进行了一种转化，因此可以将预译码部件的功能加以扩展，在不同ISA的目标代码之间实时地进行转化操作，这种技术也正在为人们所关注。

3.3.3 指令分派

指令在超标量流水线中必须经过分派才能开始执行。标量流水线中，所有类型的指令均在同一条流水线中执行，因此不存在分派阶段。超标量属于多配置流水线，其执行段使用了很多不同类型的执行部件，不同类型的指令要在不同部件中执行。指令的类型在译码段一旦确定，指令就必须被发射到相应的部件执行，这就是指令分派所做的工作。

虽然超标量流水线是并行流水线，但指令的取指和译码都是集中在一起处理的，即所有指令受同一个控制器控制。虽然每周期取出多条指令，但所有指令都是同时取自同一个指令cache，然后

存入同一个指令缓冲。指令的译码同样也是集中处理。在CISC指令集中，由于要从取来的CISC指令字节中提取单条指令，所有指令字也必须集中处理。即便是对于RISC指令，译码器也必须确定指令间的相关性，因此同样需要集中译码。

另一方面，在多配置流水线中，只要指令间的相关性得到解决，各功能部件就能并行独立地执行各自的指令。因此，在超标量流水线中，在译码与执行之间，指令要由集中控制转为分布处理，这一工作理所当然地应由指令分派来完成，如图 3.16 所示。

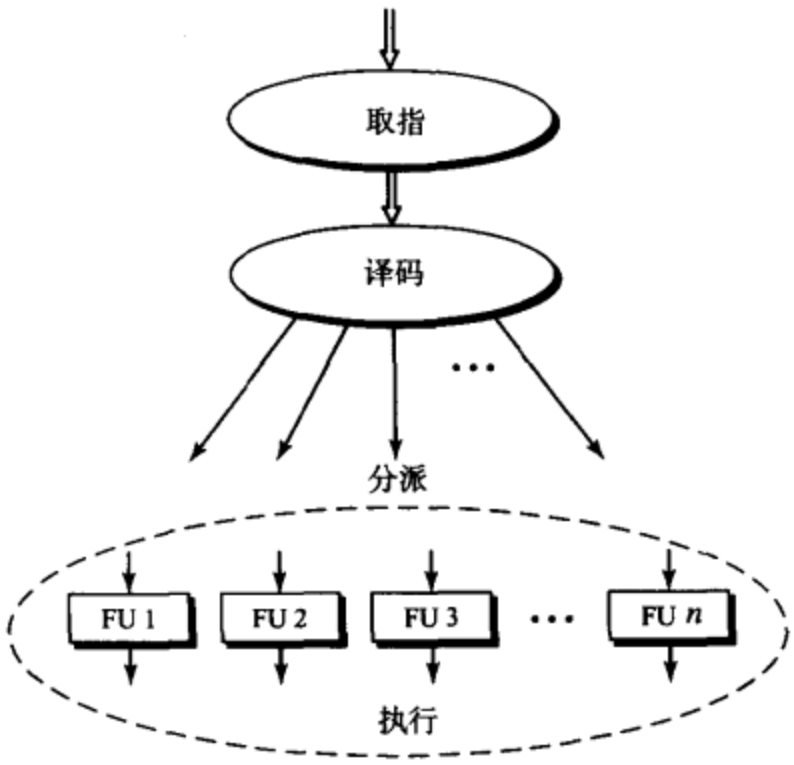


图 3.16　超标量流水线中分派的地位

在指令译码与执行之间，还需要对指令进行临时的缓冲，因为在执行之前所有指令都必须保证操作数是可用的。在译码阶段从寄存器文件中读取操作数，但是在超标量流水线中，由于前导指令还没有完成对寄存器的更新，所以当前指令的某些操作数可能没有准备好。这种情况下，最简单的解决方法就是停止译码段的继续执行，直到所有操作数就绪，但是这种方法对译码段的吞吐率的影响很大，因此是不可取的。更好的解决方法是，不能立即执行的指令把已经就绪的操作数读出并放入一个单独的缓冲中，并在此等待仍没有就绪的操作数。所有操作数就绪以后，从缓冲中读出指令并发射到功能部件执行。沿用Tomasulo算法中的术语，可以称这个临时的指令缓冲为保留站（reservation station）。保留站将指令译码和执行加以分离，从而平衡了译码段以及执行段吞吐率的频繁变化，消除了译码段不必要的停顿，同时也为执行部件输送足够的指令。

指令分派段的保留站有两种实现方式。如果在分派段前端使用一个统一的缓冲，则称为集中保留站（centralized reservation station）；如果在分派段后端使用多个缓冲，则称为分布保留站（distributed reservation station）。图 3.17 和图 3.18 中给出了这两种实现。

Intel Pentium Pro 中使用的是集中保留站，一个保留站有多个记录项连往各个功能部件。指令从集中保留站中分派后，直接送往功能部件执行。在PowerPC 620中则使用了分布保留站，每个功能部件的输入端设有自己专门的保留站。指令从分派缓冲读出后，将按类型分派到各个专用保留站，并在此等待操作数然后再流入功能部件执行。当然，以上两种实现方式只是极端的选择，它们也可以结合起来使用。MIPS R10000 中就使用了这两种方案的混合，称为分组保留站（clustered reservation station）。保留站以及功能部件都按指令类型或数据类型进行了分组，每个分组保留站被多个功能部件共享使用，指令被分派到多个分组保留站中等待执行。

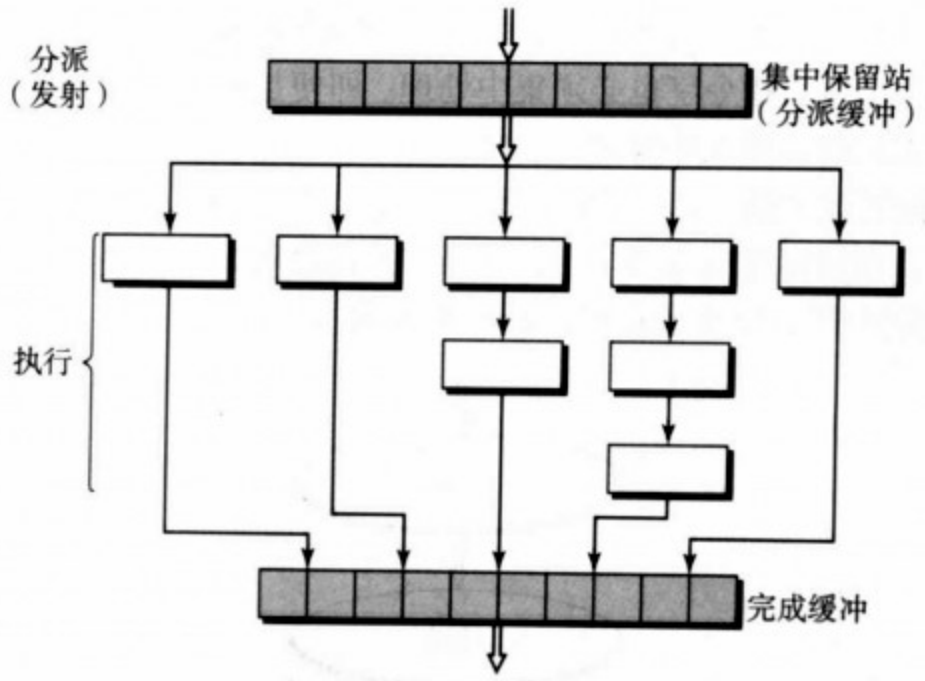


图 3.17 集中保留站

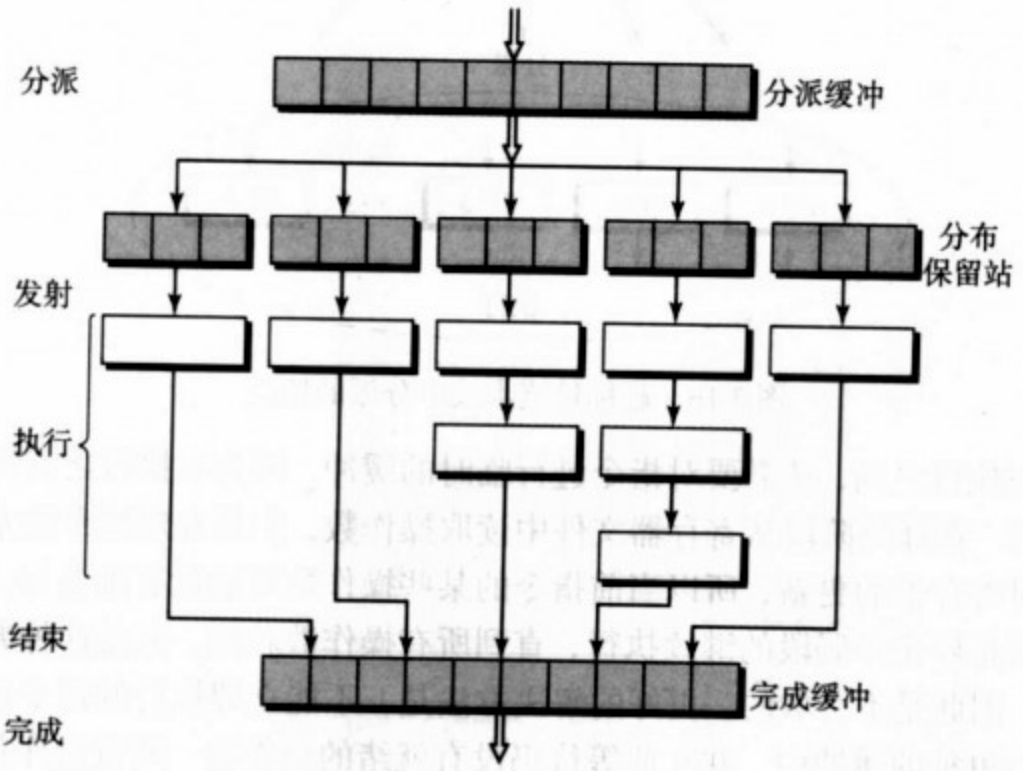


图 3.18 分布保留站

设计保留站需要全面地考虑各种因素。虽然集中保留站可以被各种类型的指令共用，因此，总的使用率较高，但是硬件设计非常复杂，需要集中控制并且要设置多个端口来满足多个部件的同时访问。分布保留站只需一个端口和少数几条记录项，硬件简单，但每个保留站空闲时不能被其他功能部件使用，所有保留站总的利用率较低。当一种类型的保留站中各记录项都被占用时，所有这种类型的后续指令都必须停顿。

鉴于保留站的实现有多种选择，因此有必要说明一下本书使用的一些术语。“分派”是指译码后指令同相应类型的功能部件之间的“连接”；而“发射”是指指令在功能部件中执行的初始化。在分布保留站中，这两个事件不是同时的。指令首先从集中的译码/分派段间缓冲分派到各个专用保留站并等待操作数，当所需操作数全部就绪后，再从保留站发射到功能部件执行。而在集中保留站中，所有类型的指令被统一地保存在集中保留站中，直到它们的操作数就绪才被直接分派到各个功能部件加以执行。因此，在使用集中保留站的微处理器中，指令和功能部件的连接同执行的初始化是同时完成的，即分派和发射是不可分的，两个术语可以互换，如图 3.17 所示。

3.3.4　指令执行

指令执行段是超标量流水线的核心。当前超标量设计的趋势是向着大规模并行和多条流水线的方向发展，即配置了越来越多、越来越专用化的功能部件。功能部件的专用化趋势使得功能部件在设计时可以进行更加有针对性的优化，对特定类型的指令执行效率更高。早期的标量流水线只有一个功能部件，除了浮点指令由单独的浮点协处理器处理外，所有其他指令都在这个功能部件上执行。以 TYP 流水线为例，该功能部件由 2 级流水段组成 ALU 和 MEM。多数第一代的超标量处理器是并行流水线并配置两个功能部件：一个功能部件执行整数指令，另一个功能部件执行浮点指令。早期的超标量处理器将浮点执行部件整合在一条指令流水线中，没有使用协处理器。

目前的超标量处理器一般都有多个整数部件，甚至多个浮点部件，这是两种最基本的部件类型。有的部件实现得非常复杂，但效率很高，每周期可以执行多条指令，甚至一些指令的操作数可以多于两个。图 3.19（a）显示了 TI SuperSPARC 的整数执行部件，它采用了级联的 ALU 部件。3 个 ALU 操作单元跨越了 2 级流水线，每周期可以处理两个整数操作。如果两个操作不相关，就可以在第一段使用 ALU0 并和 ALU2 同时执行。如果第二条指令和第一条指令存在相关，那么第一条指令在第一段的 ALU2 中执行，第二条指令在第二段的 ALUC 中执行。使用这样一种功能部件在更多的时钟周期内可以实现双发射。

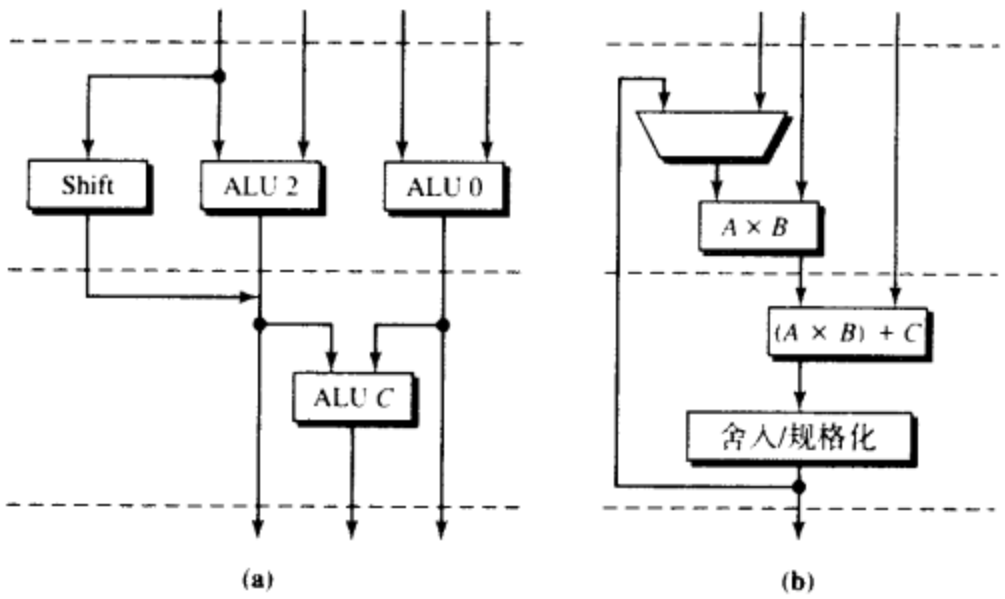


图 3.19　（a）TI SuperSPARC 中的整数部件；（b）IBM RS/6000 中的浮点部件

IBM RS/6000 中的浮点部件是使用一个两级流水的 MAF（multiply-add-fused）乘 – 加部件来实现的。MAF 有三个输入（A, B, C）并执行乘 – 加操作（$A \times B$）$+C$，如图 3.19（b）所示。配置 MAF 部件主要是因为经常需要使用浮点乘法计算点积 $D = (A \times B) + C$。如果编译器能够把大量的乘法和加法指令合并成 MAF 指令，仅设置一个 MAF 部件就可以使浮点指令的吞吐率达到每周期两条指令。浮点乘法指令实际上是通过 MAF 部件执行（$A \times B$）+0 来实现的，而加法指令则通过 MAF 部件执行（$A \times 1$）$+C$ 来实现。由于 MAF 部件是流水实现的，所以即使不执行 MAF 指令，在 MAF 上执行一般指令的吞吐率也可以达到每周期 1 条指令。

整数部件除了执行整数 ALU 指令以外，还用来计算存储器地址、处理分支以及 load/store 指令。然而，在最近的一些设计中，出现了单独的分支部件以及 load/store 部件。分支部件主要是修改 PC，load/store 则直接和数据 cache 相连。其他一些专用部件诸如图形图像处理部件也相继出现，如 Motorola 88110 中专门配置了一个位处理部件和两个像素处理部件。对于许多信号处理和多媒体应用，常用的数据类型是字节，多数情况下，4 个字节是打包在一个 32 位字中同时处理的，因此需要设置专门的 32 位功能部件来提高吞吐率。TriMedia VLIW 处理器面向信号处理和多媒体应用，因

此设置了这种功能的专用部件。如TriMedia-1处理器可以在一个周期内处理完一条quadavg指令。quadavg指令是将4个舍入平均数相加，这在MPEG解码中经常用到，如下式所示：

$$quadavg = \frac{(a+e+1)}{2} + \frac{(b+f+1)}{2} + \frac{(c+g+1)}{2} + \frac{(d+h+1)}{2} \qquad (公式3.2)$$

8个变量是8个字节的操作数，a、b、c和d存成一个32位数值，e、f、g和h存成另一个32位数值。专用功能部件以这两个数值为操作数并在一个周期内计算出quadavg的结果。而如果8个单字节数据单独计算，则需要大量的加法和除法指令。随着多媒体应用的发展，专门处理特殊数据类型的专用功能部件将会越来越多。

在超标量流水线中，各种功能部件数量的比例配置是一个有趣的问题，一般应由应用领域来决定。如果使用第2章中典型程序的统计结果：40%ALU指令、20%分支指令以及40% load/store指令，比例是4∶2∶4，那么每配置4个ALU部件，就要相应地配置两个分支部件和4个load/store部件。当前许多超标量处理器都配置了4个或更多的ALU部件（包括整数ALU和浮点ALU），多数处理器配置一个分支部件，但可以对条件分支推测执行。一般处理器都只有一个load/store部件，但在某些限制条件满足的情况下，每周期可以处理两条load/store指令。显然，这种配置方案略失平衡，load/store部件的配置太少，原因是设计能够同时被多个load/store部件访问的数据cache是一件十分困难的工作，多端口数据cache的电路设计十分复杂，而且会导致访存速度严重降低。

在许多设计中可以使用多个存储器体来模拟多端口存储器。存储器被分为几个体，每个体每周期可以执行一个读或写操作。如果两个load/store指令的有效地址恰好不指向同一个体，两条指令就可以同时从两个体中取数；而如果有效地址指向同一个体，则发生体冲突，两条指令必须串行访问存储器。多体数据cache往往用来模拟多端口数据cache，Intel Pentium处理器就使用8体数据cache模拟一个2端口数据cache。真正的多端口存储器能够实现无冲突的同时访问，而且读端口往往比写端口多。多个读端口可以通过设置多个存储器备份来实现，而所有存储器的写操作都要广播到每个备份，从而使每个备份的内容都保持一致。每个备份又可以设置几个读端口，则总的读端口等于各个备份的读端口之和。例如，4个读端口、两个写端口的存储器可以用两个存储器模块的备份来实现，每个模块有两个读端口和两个写端口。因此，实现多于两个的load/store部件对于超标量流水线设计来说是一个挑战。

指令执行段资源的并行性是由空间并行性和时间并行性共同决定的。设置多个功能部件是一种空间并行性的体现方式，而同样的并行程度可以通过将功能部件流水化来实现，这是时间并行性的一种体现方式。例如，在当前一些设计中，数据cache不使用双端口cache实现，而是将数据cache分为两级流水，也可以同时处理两条load/store指令。目前，为了降低周期时间，提高时钟频率，一般是采用深度流水。空间并行性在实现上增加了硬件复杂度，因此需要更先进的硅生产线。时间并行性虽然可以提高硬件的使用效率，但是增大了指令的总延迟，并且潜在地增加了指令相关时的停顿开销。

在实际的超标量流水线设计中，功能部件的数量往往大于并行流水线的宽度。超标量流水线的宽度由每周期取指、译码或完成的指令数量来确定，但是由于每周期指令序列的组成是动态变化的，因此每次发射的指令类型都不同，不一定完全和功能部件类型匹配。同时，由于不同功能部件的专用性，为了避免结构相关给处理器造成停顿，功能部件的数目必须大于流水线的宽度，才不会使执行段成为流水线的瓶颈。而且，在编译程序优化时往往会对指令进行调度，因此应尽量使程序中指令类型的变化趋于平缓，保证与功能部件类型更好地匹配。当然，不同的应用程序在指令类型的变化上表现出不同的特性，编译器也只能通过对局部指令进行调整来提高性能。目前，已经有人基于SPEC测试程序对功能部件的数量和类型分配完成了相应的评估工作。

如果配置了太多的功能部件，除了部件本身的复杂性外，还会带来额外的硬件复杂性。各个功能部件的输出与输入之间的定向通路、复合的总线结构以及总线控制和仲裁逻辑都使硬件变得异常

复杂。完全的crossbar互连网络由于成本太高而不经常使用，操作数在功能部件之间的传递会导致另一种形式的结构相关，而且这种互连也增加了流水线执行段的延迟。为了支持数据定向，保留站必须能够监听总线，查看是否有所需数据与标识匹配，一旦总线广播该数据，就将其锁存起来。如果用 n 表示功能单元的数目，执行段的硬件复杂度一般是以 n^2 的速率增加的。

3.3.5　指令的完成和提交

指令完成意味着指令执行结束并修改了机器状态。指令结束是指指令离开功能部件并进入完成缓冲。接下来，指令离开完成缓冲，整条指令完成。指令执行结束时，其结果只保存在非体系结构缓冲中，而当其完成时，其结果已经写入了体系结构寄存器。对于要修改存储器的指令，从完成到更新存储器还有一段时间。例如，对于store指令，当其离开完成缓冲并存入store缓冲中时指令完成，并在store缓冲中等待空闲的总线周期写入数据cache；当该指令离开store缓冲，将数据写入数据cache后，指令才算提交。本书中，“完成”是指更新机器状态，而“提交”是指更新存储器状态。对于不更新存储器的指令，完成的同时也就提交了指令。因此，在分布保留站中，指令经过了以下阶段：取指、译码、分派、发射、执行、结束、完成和提交，其中发射和结束分别指指令开始执行和指令结束执行。某些超标量处理器提供商在使用这些术语时存在着细微的不同，“发射”和“分派”、“完成”和“提交”经常混用，有时“完成”用来表示执行结束，而“提交”表示更新机器状态并且没有固定的标准。

程序在执行过程中，中断（interrupt）和异常（exception）经常会打断程序的执行。动态流水线的超标量微处理器由于指令存在乱序执行，所以必须能够正确地对中断和异常进行处理。中断一般是来自于外部环境诸如I/O设备和操作系统，并在程序执行期间异步发生。中断发生时，程序必须挂起，然后交给操作系统处理中断。一种实现方案是停止取指段读取新指令，完成当前流水线中指令的执行并记录机器当前状态。操作系统处理完中断后，将恢复机器状态，程序继续执行。

异常是由程序自身指令的执行而引起的。指令的一些算术操作可能导致异常，如除0操作和浮点的上溢、下溢等。发生异常时，计算的结果是无效的并且操作系统介入处理，将异常写入日志文件。在基于页管理的虚存系统中，指令在访问存储器时发生的页故障也能引起异常。此时，必须耗费上千个时钟周期将新页从辅存调入主存，因此当前程序通常被挂起并切换到一个新的程序（在多线程环境中）。当页故障处理完毕后，原来的程序再重新开始执行。

指令发生异常时应该能够把当前机器的体系结构状态保存起来，异常处理完毕后再恢复指令的执行。机器这种能够以单条指令的粒度挂起和重启程序的能力称为精确异常。精确异常能够在异常指令执行前设置检查点（checkpoint）并保存机器状态，重新执行时则恢复保存的机器状态。为了支持精确异常，超标量处理器必须保存机器状态，恢复时使程序在相同的现场下以原始顺序继续执行。异常发生时，机器状态必须反映一个条件，即异常指令前的所有指令都已完成，而后续指令都没有完成。对于支持精确异常的动态流水线，即使指令存在乱序执行，机器状态的保存也要满足顺序性。

在动态流水线中，指令将顺序取指并译码，但是以乱序的方式执行，也就是在进入保留站时是顺序的，离开保留站时是乱序的，结束时也相应地是乱序的。但是为了支持精确异常，指令完成时应是顺序的，即对机器状态的更新是顺序的，因此在并行流水线的完成段需要设置一个再定序缓冲（reorder buffer）。指令执行结束后，先乱序地存入再定序缓冲，然后再按照程序原始顺序离开再定序缓冲，这时指令才完成对体系结构状态的修改。如图3.20所示，保留站和再定序缓冲之间的部分是流水线的乱序部分，即执行段。同时，图3.20也对指令的各个处理阶段用本书中的术语进行了注解。

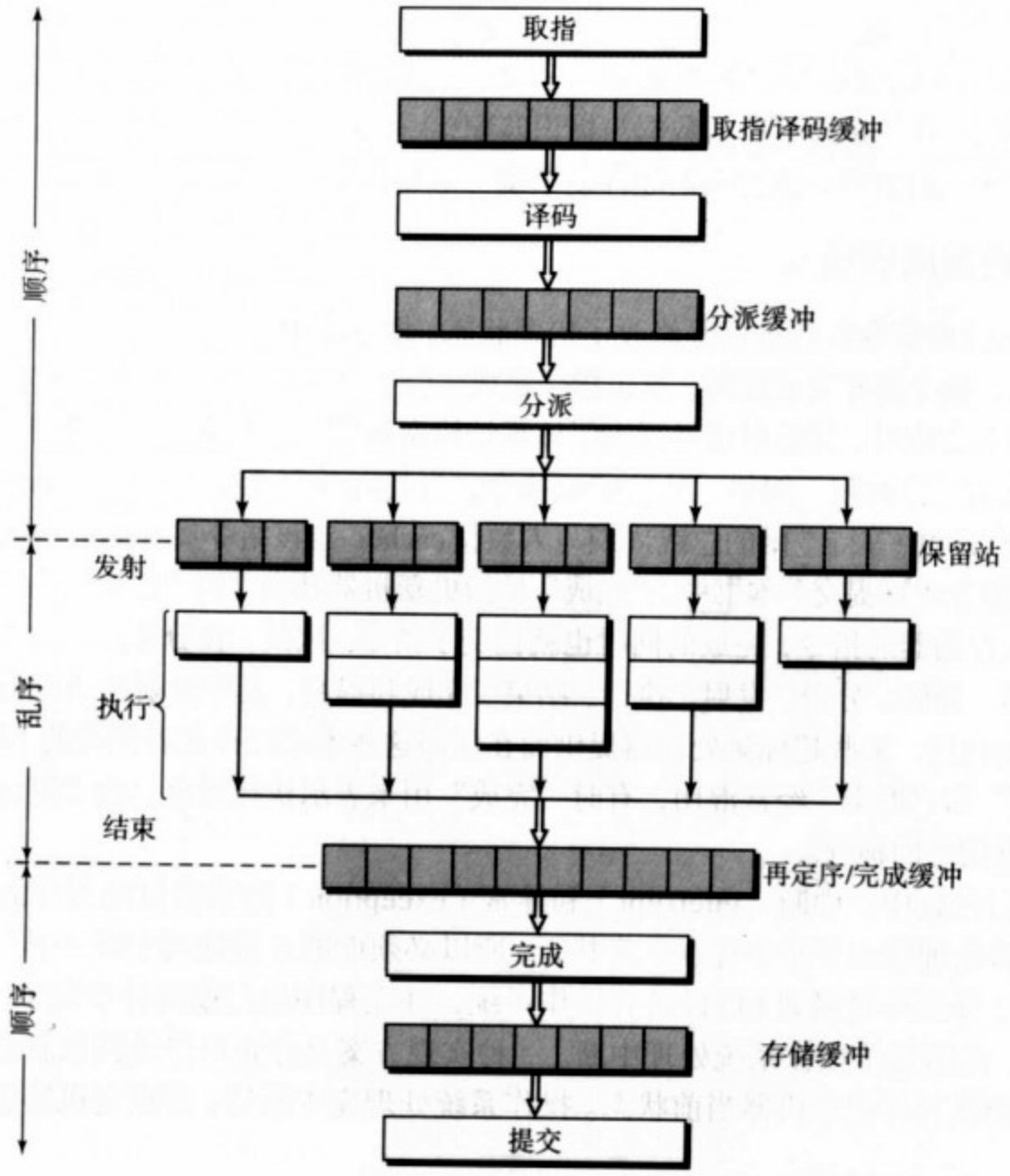


图 3.20 动态流水线中的保留站和再定序缓冲

指令完成段用再定序缓冲来处理精确中断。发生异常时，当前指令在再定序缓冲中标识出异常指令。完成段按顺序检查每条指令是否可以完成，当发现一条带标记的指令时，则不允许其完成，但所有之前的指令都可以完成，同时，应标记该条指令为检查点并保存机器状态。机器状态包括所有体系结构寄存器以及程序指针。流水线中的其他指令即使已经执行结束，也必须使之失效。异常处理完毕后，需要恢复检查点的机器状态并取出发生异常的那条指令，从而使程序重新开始执行。

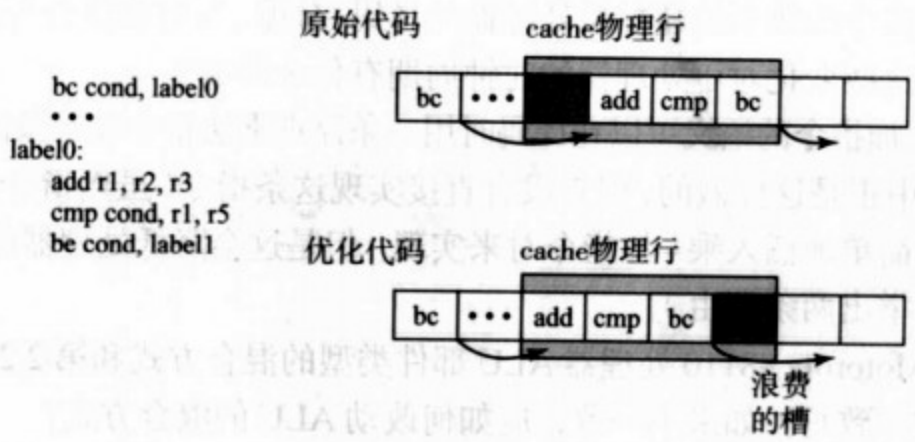
3.4 小结

图 3.20 给出了一个当前超标量流水线的基本结构，前端是顺序执行，执行段是乱序执行，但后端是顺序提交。前端和后端流水段每周期都能处理多条指令。由于等待操作数就绪，指令可能在保留站中停留多个时钟周期，一旦操作数就绪，指令就从保留站发射到相应部件加以执行。执行结束后，指令进入再定序缓冲（完成缓冲），然后按照程序原始顺序完成并提交。再定序缓冲一般是按照指令原始顺序组织的循环队列。

本章主要介绍的是超标量流水线的组织结构以及各个流水段相关的问题。到此为止，我们已经讨论了超标量流水线的静态结构，在第4章中将讨论超标量流水线的动态行为。本章从一个较高的层次阐述了超标量流水线的组织结构并省略了实现上的细节，主要是为从标量流水线到超标量流水线实现一种过渡，以便于在下一章中对超标量的细节技术展开讨论。

3.5　习题

1. 标量流水线中是否可以实现乱序执行？如果可以，简述其实现方法并给出一段代码说明这种设计在性能上如何优于顺序标量流水线。
2. 超标量流水线需要对单条并行流水线包括cache端口在内的资源进行复制。而实际上，在双发射的超标量流水线中，数据cache要设置两个端口，而指令cache可以只设置一个，试解释其原因。同时，试分析单端口指令cache相对于双端口指令cache为什么性能要差。
3. 在3.3.1节中曾提到，编译器生成目标代码时，可以将分支目标指令放置在一个cache阵列行的起始位置，提高一周期内取出多条指令的可能性。然而，两条分支指令之间的指令数目是确定的，这种方法仅仅是把浪费的取指槽从分支前移到了分支后，如下图所示。例如，移动label0处的代码来满足cache阵列行的对齐并不能改善取指效率，因为损失的取指槽从cache阵列行的开始移到了末尾。试分析取指块大小和分支间动态间距的关系，以及它们之间的相互影响有多大。评价一下这种基于静态编译调度的对齐策略在各种情况下的性能。



4. 图3.13所示的自动对齐取指硬件中，仍然不能达到最大取指带宽（每周期4条指令）。设计一种结构，使它总能够每周期读取4条指令。
5. 在3.3.2节中曾提到，在指令调入cache前添加预译码位可以简化流水线译码段的工作。另外一种扩展方式是将指令译码后的形式存入译码指令cache。在Pentium Pro中，将取来的指令动态地转换成简单的类RISC指令序列，再送到流水线执行。如果为这种将复杂指令转换为类RISC简单指令的机器设置译码指令cache，试分析一下其中的不利因素。
6. 集中保留站相对于分布保留站最主要的优势是什么？
7. 顺序流水线处理器中，流水线锁存器用来保存执行部件计算出的结果，直到写回段时才写入寄存器文件。在乱序流水线处理器中，重命名寄存器的作用和上述锁存器相同。给定一个4发射、乱序执行的TYP流水线，为了避免由于寄存器不够用而产生的停顿，计算最少需要多少个重命名寄存器？如果出于提高频率的考虑，在指令分派和执行段之间、执行和提交段之间各增加5级流水线，结果又是什么？
8. 多体交叉cache对于解决每周期执行多条load/store指令是一种有效的方法。分别画出2路交叉数据cache和两个load/store部件之间数据的通路图，以及8路交叉数据cache和4个load/store部件之间的数据通路图。分析一下交叉多体cache的优缺点。
9. Pentium 4处理器整数算术部件的执行速度是其他处理器的两倍，因为它将整数加法器分成两级来实现，在第一级计算低16位，第二级计算高16位。同时也使ALU的延迟从1增加到2。然而，如果两条算术计算指令是相关的，在第一条指令发射后，下一个时钟就可发射第二条指令。这是因为，第二条指令的低位计算只和第一条指令的低位计算相关。画出此ALU部件的流水线简图，并画出在这种优化措施下需要的旁路逻辑。

10. 给定在第9题中提到的ALU部件，前导指令为算术指令，与前导指令相关的后续指令如下所示：算术指令、逻辑指令（与/或/异或）、左移和右移。试分析其等待延迟。
11. 为什么第9题中提到的按位分割流水线的方法不适用于相关的浮点指令流水线？
12. 假设一个4发射的超标量处理器，每周期从再定序缓冲中提交4条指令，在这一段中需要检测哪些数据相关？试画出相关的检测硬件图。
13. 虽然4发射的超标量处理器的吞吐率一般只是比1略大，为什么还要在提交段设置宽度为4的提交硬件？
14. 多数近期出现的通用指令集都加入了多媒体扩展，从而支持小型数组的类向量操作。例如，IA32扩展了MMX和SSE。一条多媒体指令可以并行载入8个8位操作数到一个64位寄存器中，算术指令以SIMD（单指令流，多数据流）的方式对8个操作数进行相同的操作。简述为了支持这种指令，典型超标量处理器的取指、译码、分派、发射、执行和提交逻辑应做哪些相应的修改。
15. PowerPC指令集提供了一条浮点乘-加指令，将前两个操作数相乘的积加上第三个操作数。说明一下这种指令给典型的超标量处理器的译码、分派、发射和执行等流水段带来了哪些实现上的困难。这些变化对于处理器的时钟周期有什么影响？
16. 上题中浮点乘-加指令的语义可以在译码时用一条浮点乘法指令和一条浮点加法指令来实现。MIPS R10000中正是这样做的，但它没有直接实现这条指令（虽然指令集中存在这条指令），而是在译码时简单地插入乘-加指令对来实现。但是这会降低处理器的IPC，试说明其中的原因（至少列举出两条理由）。
17. 在图3.7中，Motorola 88110处理器ALU部件类型的混合方式和第2.2.4.3节中IBM指令类型的混合方式一致吗？如果不一致，应如何改动ALU的混合方式？

第4章　超标量技术

在第3章中曾重点讨论了超标量流水线的结构设计，解决了指令在处理时遇到的一般问题。本章的重点是超标量处理器执行指令时的动态行为，以及针对某些特殊指令的处理技术。超标量流水线设计的最终性能目标是得到最大的指令吞吐率。如果将指令的处理过程看做三种指令或者数据的流动过程，即指令流、寄存器数据流和存储器数据流，问题的描述和解决就会变得相对简单。这一概念最早出现在Mike Johnson于1991年编写的教材“Superscalar Microprocessor Design”中，总的性能目标就是使三条流通路的容量最大。当然，三条通路之间并不是互相独立的，它们之间的相互作用十分复杂。本章就以这三条通路为线索讨论超标量微体系结构技术。

三条流通路大致上对应三种主要指令的处理，即分支指令、ALU指令和load/store指令。因此，要增大三条流通路的吞吐率就必须减少分支指令、ALU指令和load/store指令的开销。

1. **指令流**：分支指令处理。
2. **寄存器数据流**：ALU指令处理。
3. **存储器数据流**：load/store指令处理。

下面就以三种流通路为框架，详细介绍当前超标量处理器中提高性能时所用的微体系结构技术。

4.1　指令流技术

首先介绍指令流技术，因为指令流技术一般用在超标量流水线的前端，即取指段和译码段。流水线前端的吞吐率是整条流水线吞吐率的上限。对今天的流水线处理器而言，传统的、将处理器分为控制通路和数据通路的分发已经不适用了。不过，流水线前端和分支执行部件仍然可以看做是传统流水线中的控制通路，其基本功能是程序语义的控制。所有指令流技术的实现目标都是在程序流控制语义的要求下为超标量流水线提供尽可能多的指令执行。

4.1.1　程序控制流和控制相关

程序的控制流语义可以用控制流图（CFG, Control Flow Graph）来表示，相应的节点代表基本块，边代表控制流在基本块之间的转换。图4.1（a）是一个4个基本块（虚线框中的部分）的CFG，每个基本块包括多条指令（椭圆部分），有向边由条件分支指令（菱形部分）产生并代表基本块间的控制流。程序在执行时完成CFG中节点和边的动态转换，实际转换的路径由程序执行时的分支指令和分支的条件动态地加以决定，分支的条件取决于运行时的数据。

CFG中基本块以及其中的指令必须顺序地保存在存储器中，因此偏序关系的基本块必须在存储器中以全序方式存储。在将CFG中映射为连续的存储位置时，必须加入无条件分支指令，如图4.1（b）所示。将CFG映射到线性连续空间中可以很方便地在程序顺序执行时控制指令流动，但必须加入额外的无条件分支指令，如图4.1（b）所示。一旦程序执行时，条件分支和无条件分支指令产生了程序的跳转，将导致指令取指段的停顿，从而降低了取指的带宽。子程序的调用和返回指令也同样会导致取指段停顿。

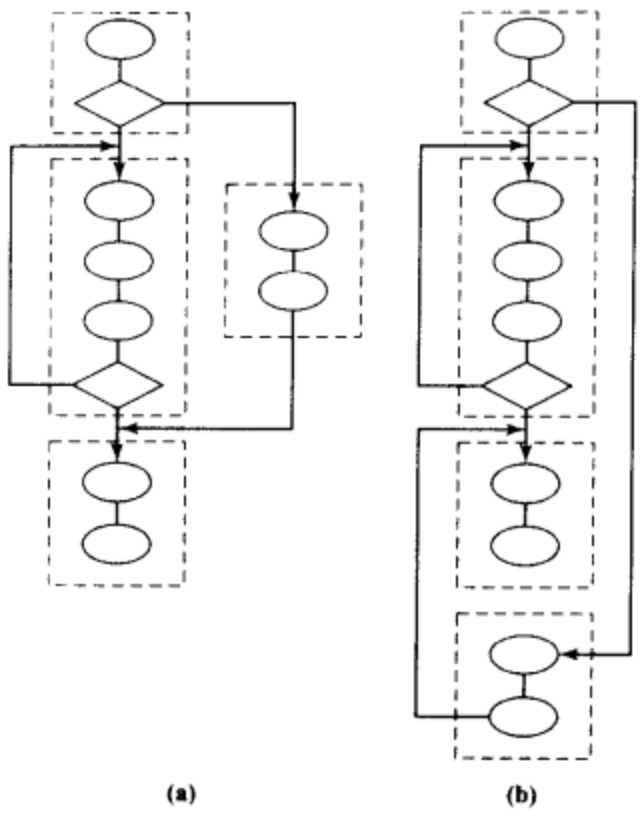


图 4.1 程序控制流：（a）控制流图（CFG）（b）将 CFG 映射到顺序存储器

4.1.2 分支造成的性能损失

流水线机器只有在流水模式（streaming mode）下才能达到最大的吞吐率。在取指段，流水模式是指从程序存储器中的连续位置顺序地取指。每当程序的控制流背离了顺序的方向，流水模式就会被打破。对于无条件分支指令，直到分支地址确定以后，下一条指令才能取出；而对于条件分支，处理器必须等待分支条件的结果，若分支跳转，则继续等待分支地址产生，否则顺序读取下一条指令。图4.2显示了分支指令对流水模式的影响，分支指令由分支部件来执行。对于条件分支指令，只有当其在分支部件中处理完毕并且条件和分支目标地址均确定之后，取指段才能正确地读取下一条指令。

在图 4.2 所示的结构中，处理条件分支的延时将导致取指段有 3 个周期的停顿，3 个周期对应着分支指令的译码、分派、执行三个流水段。3 个停顿的周期在标量流水线中只有 3 个空的指令槽，但在超标量流水线中还要乘以流水线的宽度，例如，4 发射的超标量流水线总的开销是 12 个“气泡”。在第 3 章中也曾提到，这种流水线停顿，即 Amdahl 定律中的串行瓶颈（sequential bottleneck），使得处理器的实际性能相对于峰值性能大为减少。

条件分支指令的停顿开销实际上可以划分为：生成目标地址引起的开销和条件判定引起的开销。图4.3中的流水线开销是由生成分支目标地址引起的，实际开销还取决于分支指令的寻址模式。对于 PC 相对寻址方式，分支目标地址在取指段就可以确定，只有 1 个周期的开销；对于寄存器间接寻址方式，分支指令必须在译码段访问寄存器才能得到分支目标地址，所以开销是两个周期；而对于带偏移的寄存器间接寻址方式而言，指令不但要读取寄存器，还要在执行段进行偏移量和寄存器内容的加法，所以开销是3个周期。无条件分支指令的停顿只由分支目标地址的生成开销来确定，条件分支指令除此之外还要考虑分支条件的判定结果。条件判定的方式不同，造成的开销也就不同。图4.4给出了两种不同的开销。若使用条件码寄存器，并假设在分派段读取条件码，造成的开销是两个时钟周期；若指令集体系结构（ISA）包含使用通用寄存器比较的结果来产生分支条件，那么执行两个寄存器之间的 ALU 操作要增加额外的 1 个周期，即共需 3 个周期的停顿。因此，处理条件分支指令时，分支目标地址的生成和条件判定都有可能成为时间开销上的关键路径。例如，即使寻址方式使用 PC 相对寻址，但条件判定却需要访问条件码寄存器，那么该分支指令依然会导致两个周期的开销，而并非目标地址生成所用的 1 个周期。

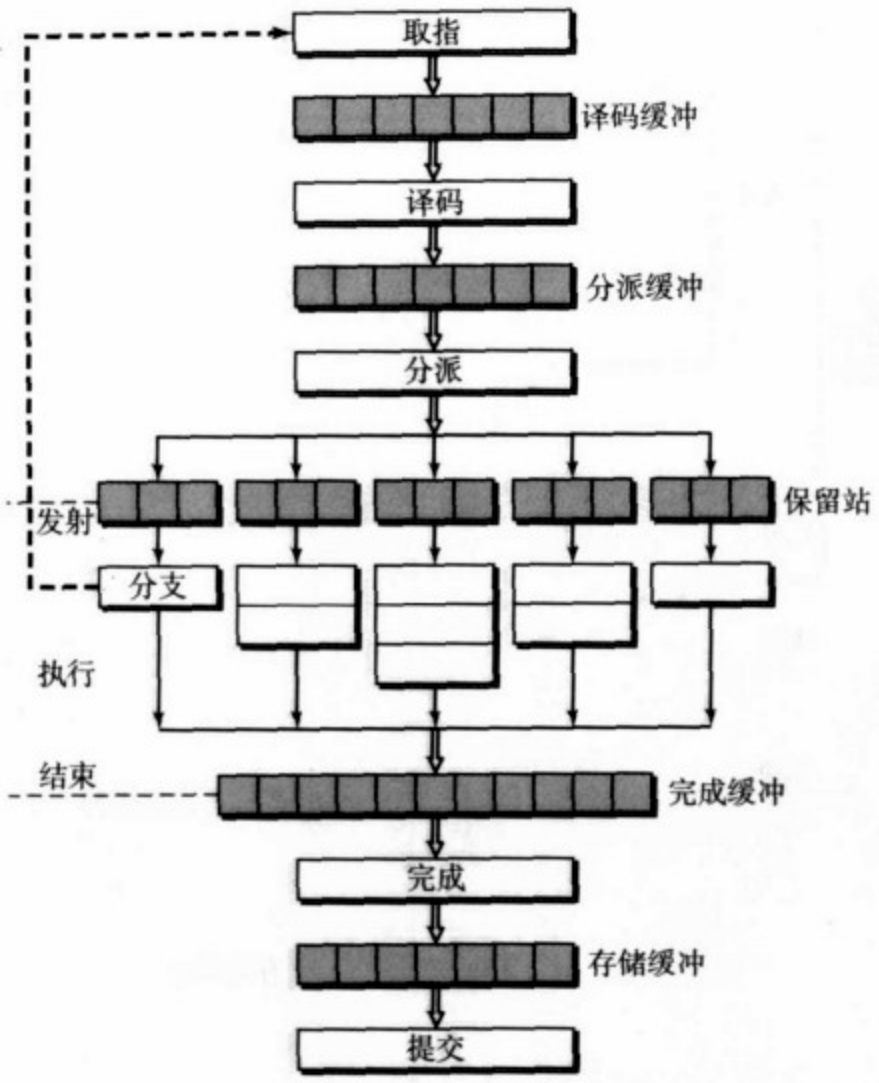


图 4.2　分支指令对顺序控制流的影响

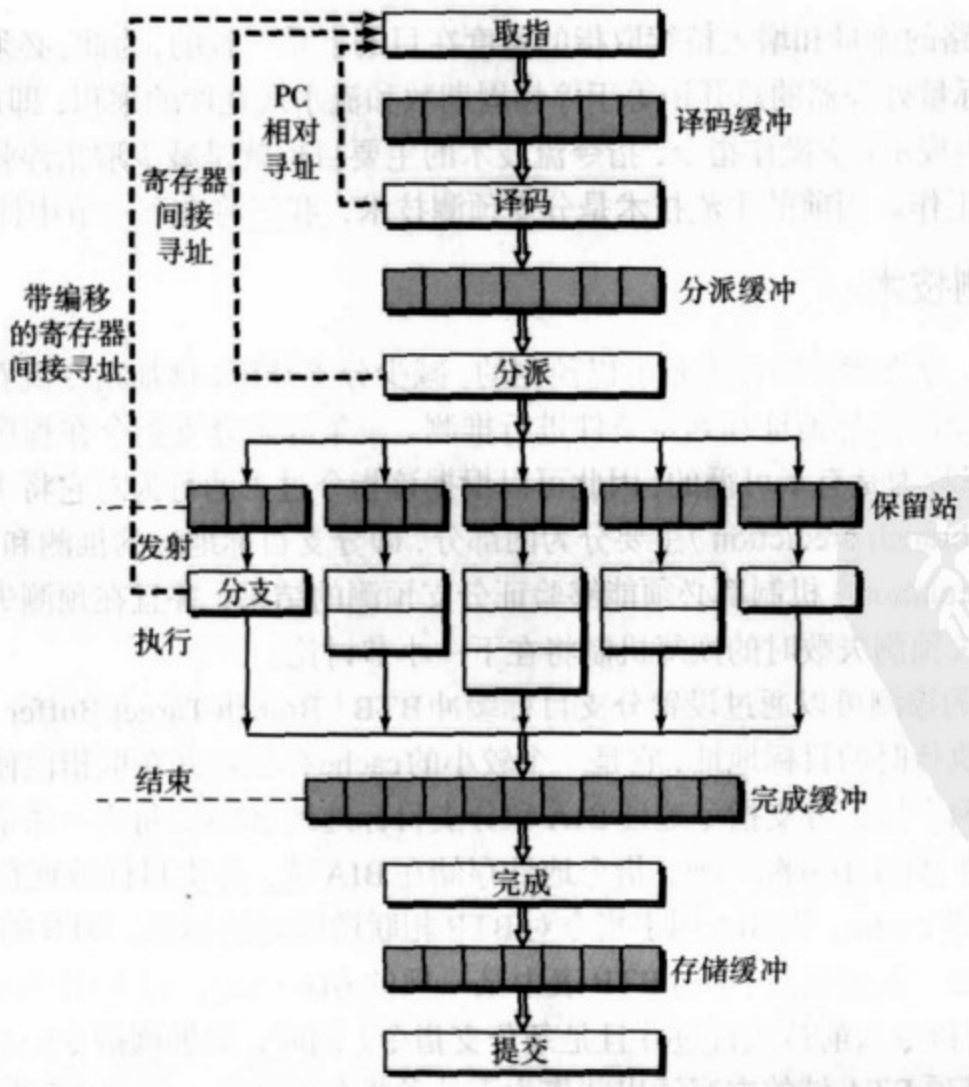


图 4.3　分支目标地址的影响

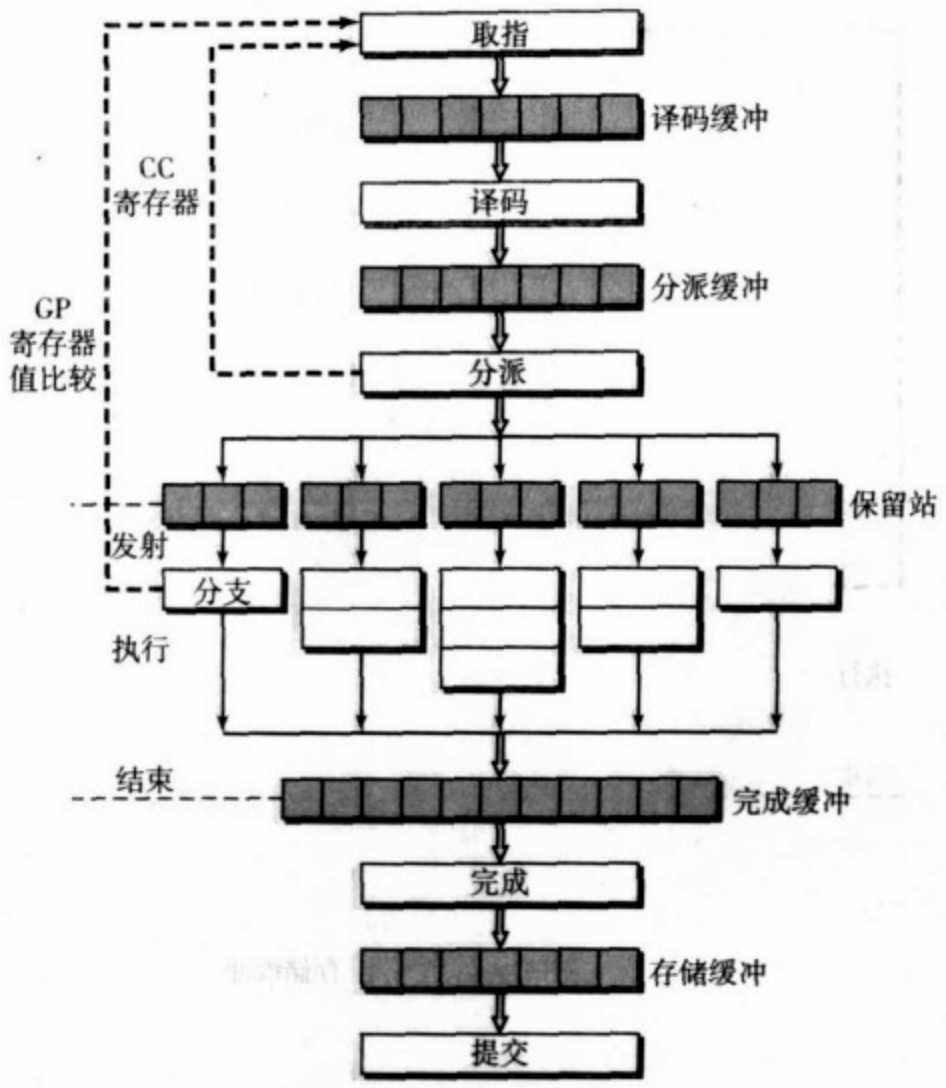


图 4.4 分支条件处理的影响

增加指令流通路的流量和增大持续取指的宽度在目的上是一致的，为此，必须减少取指段的停顿周期数。鉴于超标量处理器的总开销等于停顿周期数和流水线宽度的乘积，即n发射流水线每停顿一个周期，就要生成n条空操作指令，指令流技术的主要目标就是减少取指停顿或者利用这些停顿周期进行有用的工作。当前的主流技术是分支预测技术，我们将在下一节中详细介绍。

4.1.3 分支预测技术

实验研究表明，分支指令的行为是可以预测的。减少分支开销，增加指令流吞吐率的一种主要方法就是对分支指令的目标地址和判定条件进行推测。一条静态分支指令在程序运行时经常会被重复执行，它的动态行为是有章可循的，因此可以根据该指令过去的行为对它将来的行为进行有效地预测。分支预测（branch prediction）主要分为两部分，即分支目标地址的推测和分支条件的推测。任意一种推测（speculation）机制都必须能够验证分支预测的结果，并且在预测失败时能够恢复正确的执行方式。有关预测失败时的恢复机制将在下一小节讨论。

分支目标地址的推测可以通过设置分支目标缓冲BTB（Branch Target Buffer）来实现。BTB用来保存前几次分支执行时的目标地址，它是一个较小的cache存储器并在取指段使用PC访问。BTB的每条记录项包含两个域：分支指令地址BIA和分支目标地址BTA。每当一条静态分支指令第一次执行时，就为其在BTB中分配一项，指令地址存储在BIA域，分支目标地址存储在BTA域。如果BTB表是全相联的cache，则BIA用于指令对BTB相联访问时的标识。BTB的访问和指令cache的访问是同时进行的。如果当前PC与BTB表中某一项的BIA匹配，即BTB命中，意味着即将从指令cache中取出的指令以前被执行过并且是条分支指令。同时，如果该指令的预测结果为发生跳转（taken），则将该项BTA域的内容读出并作为下一条指令的地址，如图4.5所示。

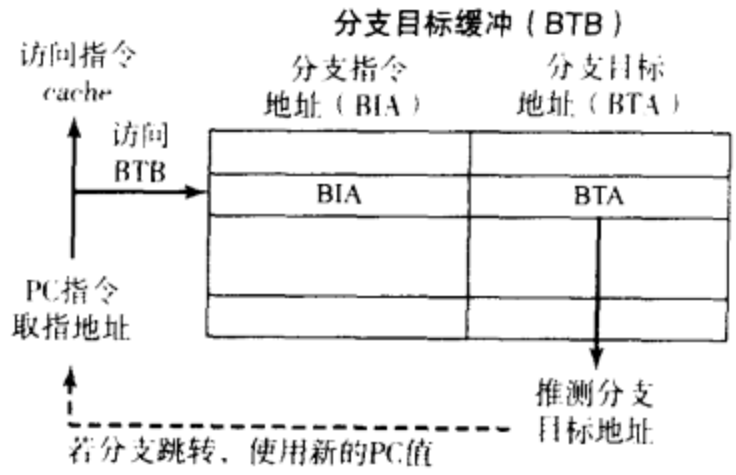


图4.5　使用分支目标缓冲进行分支目标的推测

使用分支指令的地址访问BTB表以及检索分支目标地址的工作都是在取指段执行的，因此，如果预测结果为跳转，推测生成的目标地址在下一周期就可以被取指段使用。同时，若分支指令执行的最终结果也证明了预测正确，则该分支指令在取指段没有造成任何的停顿，分支开销为0。虽然有了分支预测部件，分支指令还是要取出并送往流水线后端执行，以便能够确定预测是否正确。最终执行得到的目标地址和条件判定结果要和推测结果进行比较，如果一致，则表示预测正确；否则预测失败，同时需要恢复执行。此外，最终结果还要用来更新BTB表中BTA的内容。

分支方向的推测有多种方法，其中最简单的方法是设置取指硬件总是预测为不跳转。当遇到分支指令时，在不知道分支方向之前取指段总是顺序地取下一条指令，从而不导致任何停顿。这种方法实现简单，但效果不是很好。例如，很多分支指令用做循环结束时的指令，除了退出循环的那一次不跳转，其他时候都是要跳转的。另一种方向预测技术需要软件协同，同时要对ISA进行修改。例如，在分支指令编码的格式中增加1位，编译器通过设置这一位来通知硬件是预测跳转还是不跳转。编译器可以针对分支指令的类型以及历史信息（profile information）为这一位设置最合理的值，不同的指令可以进行不同的预测。Motorola M88110就采用了静态的软件预测技术。但是，这种预测只是一种静态预测，对于多变的执行环境采用同一种预测结果显然不尽合理。另一种较好的动态预测技术是基于分支目标地址偏移的预测。首先确定分支指令地址和分支目标地址的相对偏移，如果偏移为正值，则预测为不跳转；若为负值，则极有可能是结束循环的分支指令，则预测为跳转。基于偏移的分支预测技术在创新的IBM RS/6000中首先被使用，其他一些处理器中也曾采用。在当前超标量处理器中普遍采用的分支推测技术是基于分支执行历史信息的预测。

采用基于历史信息的分支预测策略对分支方向进行预测时，跳转还是不跳转取决于先前已经发生的分支方向。假设静态分支的历史执行信息能够对该指令将来执行的分支方向提供有用的信息，就可以根据这些历史信息对分支进行预测，但在设计时还需确定需要跟踪多少历史信息以及对各种历史信息进行何种预测。某些基于历史信息的分支方向预测算法可以用有限状态机（FSM）来描述，如图4.6所示。n位的状态变量标识该分支指令最近n次执行的方向信息，每个状态是用“跳转”、“不跳转”的序列所表示的历史信息，输出逻辑根据FSM当前的状态预测方向。一般情况下，输出结果取决于该分支指令前n次执行的结果。最终当分支指令执行完毕时，实际的分支结果再输入到FSM并触发FSM进行状态转换。状态转换逻辑比较简单，它是由n位状态变量链接成的移位寄存器，用来记录前n次分支执行的方向。

图4.7（a）是一个两位分支预测器FSM的示意图，使用两位历史位记录最近两次的分支执行方向，这两位就是FSM的状态变量并包含4种状态：NN, NT, TT和TN，代表前两次分支发生的方向。NN是初始状态，输出是T（跳转）还是N（不跳转）取决于两位预测FSM在当前状态下的输出结果。分支执行完毕后，实际结果输入到FSM，从而完成状态转换、更新历史信息并等待下一次的预测。

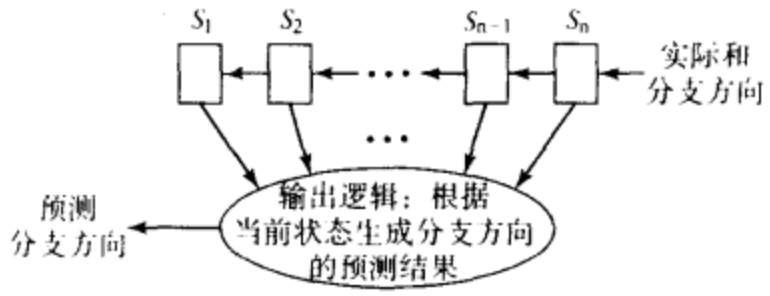


图 4.6　基于历史信息的、分支方向预测符的 FSM 模型

图 4.7（a）中的预测算法偏向于预测分支跳转（T），4 个状态中 3 个状态的输出都是 T，只要两个变量中有一个是T，就预测下一次执行为T。只有当连续两次结果都为N后（即状态变量为NN），才预测为N。这只是预测算法中的一种，还有许多种其他的设计策略，研究人员都对它们的设计进行了评估。

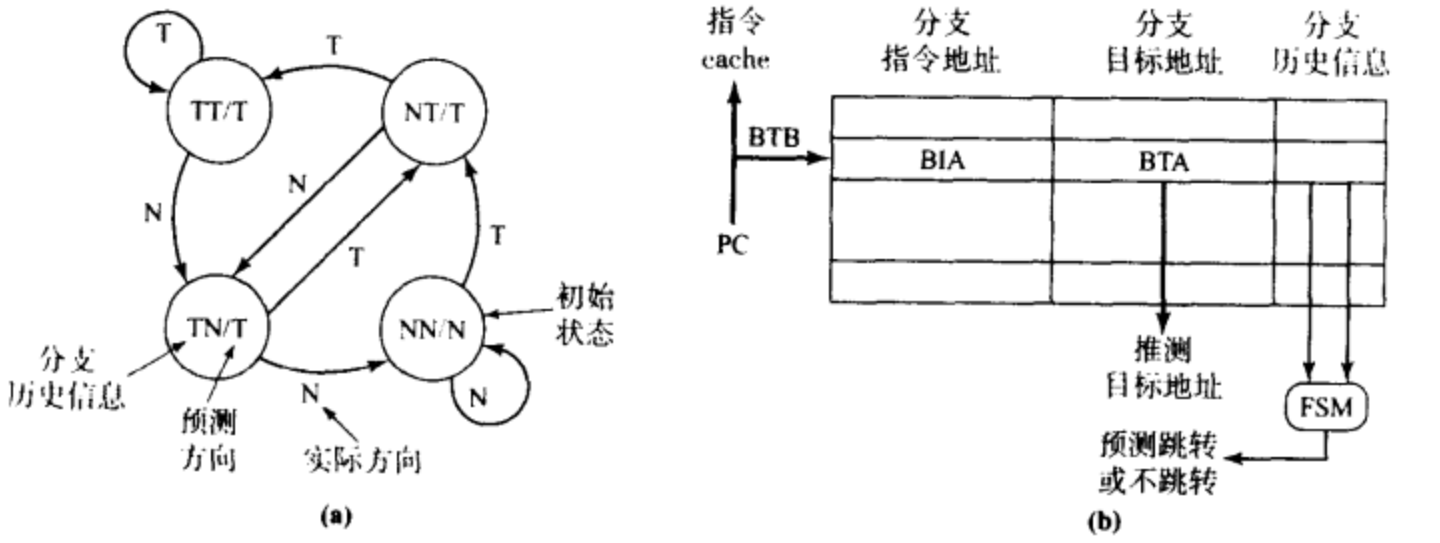


图 4.7　基于历史信息的分支预测：（a）两位分支预测算法；（b）带有分支历史信息位的分支目标缓冲

BTB 可以在每条记录中增加一个历史信息位域来支持基于历史信息的分支方向预测，位域的宽度取决于要跟踪的历史位的长度。每当PC 地址在BTB 中命中时，除了取出其预测目标地址外，还要检索历史信息位。历史信息位作为分支预测 FSM 的状态变量，FSM 的输出逻辑根据当前状态生成 1 位的预测结果，即分支预测方向。如果预测为跳转，则将预测得到的目标地址送入 PC，作为下一周期指令取指的地址。若预测最终证明是正确的，则该分支在取指段没有导致任何停顿。

1984 年，Lee 和 Smith 对分支预测进行了深入的实验性研究，他们使用了 6 种不同类型共计 26 个程序并作为测试程序对 3 种不同机器（IBM 370，DEC PDP-11 和 CDC 6400）进行测试。研究表明，在这些测试程序中，67.6% 的分支发生了跳转，而 32.4% 没有发生跳转，即分支跳转的概率与不跳转的概率之比约为 2∶1；使用基于操作码类型的静态分支预测策略，6 种程序的预测准确率从 55% 到 80% 不等；而使用 1 位历史信息的动态分支预测策略所得到的预测准确率为 79.7% 到 96.5%；用两位历史信息位预测得到的预测准确率为83.4%到97.5%；继续增加历史信息位数可以将准确率继续提升，但超过 4 位的预测准确率提升就非常微弱了。此外，他们还实现了具有 128 个相联组的 4 路组相联BTB，BTB 平均命中率为86.5%。综合预测准确率和BTB命中率，分支预测的平均效率约为80%。

另外，1992 年Ravi Nair在IBM对分支预测算法也进行了十分全面的研究，他使用的是RS/6000 和SPEC测试程序集。分支预测的目标就是将分支指令的执行同其他指令的处理重叠起来，以实现 0 延时分支或分支隐藏（branch folding），即把分支从指令执行的关键延时路径上隐藏起来。这次研究对最佳两位预测器进行了穷尽的搜索，两位预测 FSM 共有 2^{20} 种可能的结构。Nair 剔除了其中一些意义不大的 FSM，将搜索空间缩小到 5248 种。然后他进行了大量的模拟并对每一个测试程序寻找其最佳的两位预测器。图 4.8 给出了 SPEC 测试程序及其最佳预测准确率和相应的最佳预测器。

测试程序	最佳	"计数器"
spice2g6	97.2	97.0
doduc	94.3	94.3
gcc	89.1	89.1
espresso	89.1	89.1
li	87.1	86.8
eqntott	87.9	87.2

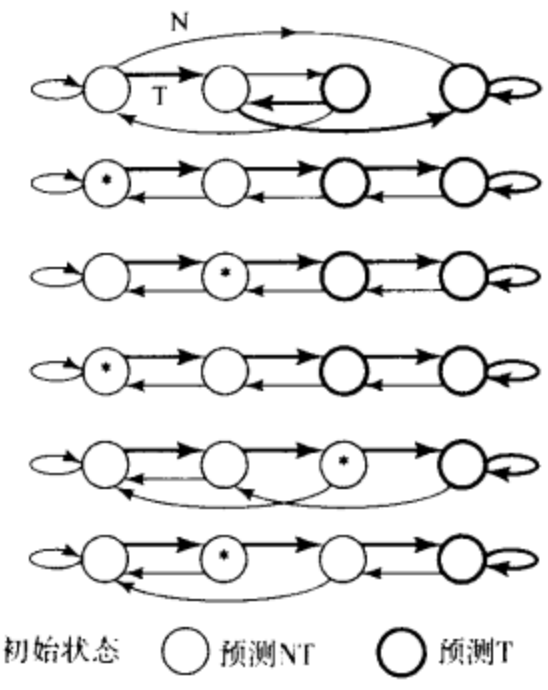


图4.8　Nair研究最佳两位分支预测器所用的6个SPEC测试程序

图中，粗体圆圈状态是指分支预测为T的状态，非粗体圆圈是预测为N的状态。粗体边线是当分支实际为T时状态转换的方向，非粗体边线为分支实际为N时状态转换的方向。标有星号的状态为初始状态。这6个测试程序最佳预测器的预测准确率为87.1%到97.2%。其中，doduc、gcc和espresso的最佳预测器是相同的（排除gcc的初始状态不同），都是一个两位上下饱和计数器。从左到右将4个状态记为0、1、2、3并代表两位计数器的4个值。每当分支结果为T时，计数器加1；否则减1。在较低的两个状态中，预测分支指令为N，而较高的两个状态预测输出为T。图4.8还给出了将两位饱和计数器预测策略应用于所有6类测试程序时的预测准确率，可以看出，对spice2g6，li和eqntott的预测准确率仅比它们相应的最佳预测器的预测准确率稍低一些，因此，两位饱和计数器预测策略对所有的测试程序来说都是一个不错的选择。实际上，在Jim Smith发明了该算法后，它已经在实际机器和实验机器上得到了广泛的应用。

Nair同样也进行了有关计数器预测FSM效率的研究。对于1位计数器预测，即用分支上一次执行的结果来预测当前的分支方向，预测正确率为82.5%到96.2%。在图4.8中已经看到，两位计数器预测正确率为86.8%到97.0%。而使用3位计数器时，预测正确率的提升就很小了，仅为88.3%到97.0%。因此，对于基于历史信息的分支预测策略，采用两位饱和计数器FSM是一种很好的选择。研究中同时使用了直接映射的分支历史信息表，但某些程序，如gcc有超过7000条的条件分支指令，即使历史信息表有1024个条目的大小，它对如此多的分支指令来说仍是有限的，由此而导致的别名冲突将会增加分支的开销。

4.1.4　分支预测失败的恢复

分支预测归根到底只是一种推测技术，而任何的推测技术都需要提供验证推测正确性的机制。动态分支预测可以看做是由两个相互影响的模块组成，前面的模块负责在流水线的前端推测分支方向和目标地址，后面的模块负责在流水线的后端验证推测的结果。同时，一旦前端预测失败，后端还要负责进行恢复，参见图4.9。

分支推测包括预测分支方向和从预测的路径上取指执行。而在预测路径的指令流中，同样也存在分支指令，对这些分支指令也要进行同样的分支预测，因此在第一条推测的分支指令处理完毕之前可能对多条分支进行了预测。图4.9（a）中所示的是对连续3条分支进行了预测，第一条和第三条预测为跳转，第二条预测为不跳转。这时，来自三个推测基本块的指令共存于处理器中，因此

必须能够鉴别它们，为此每一个推测基本块中的指令都要用标识加以标记。图4.9（a）所示的例子中，来自三个不同推测基本块的指令使用三个不同的标识，标识后的指令为推测指令，标识的值将区别所属的基本块。推测指令在流水线中流动时，标识也随之一起流动。进行推测时，分支指令的地址（或者下一条指令的地址）必须被保存起来，供失败恢复时使用。

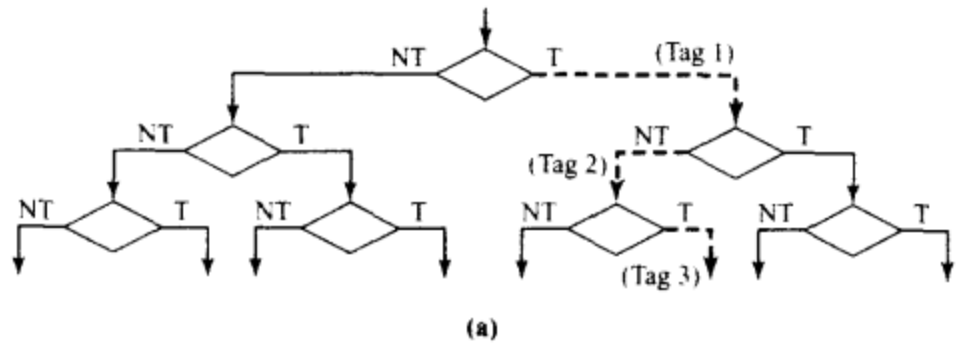


图4.9 分支预测的两部分：（a）分支推测；（b）分支验证/恢复

分支验证在分支指令执行完毕、正确方向已经确定后进行，前端预测结果的正确与否就在此时判定。如果结果证明预测正确，就可以释放推测标识，所有相关指令都变为非推测的并允许提交；如果预测失败，则需要执行两个操作：结束当前指令流并从一个新的、正确的地址取指。指令流的改变是通过修改PC来实现的，如果当时预测是不跳转，那么恢复时PC应该修改为计算后的分支目标地址；如果当时预测是跳转，那么PC的值应该修改为分支指令紧接的下一条指令的地址，当该指令被预测为跳转的时候，该地址可以从缓存的指令地址处得到。PC一旦改变，处理器就开始从一条新的路径取指，分支预测也重新开始。结束当前不正确的指令流时，要使用推测标识：所有与预测失败的分支指令具有相同标识的指令都必须清除，所有驻留在译码分派缓冲以及保留站中的指令则全部失效，再定序缓冲中的这类指令占用的空间也必须释放。图4.9（b）中显示了当3条分支指令中的第2条预测失误时的验证和恢复任务，第二条分支指令预测失败，但第一条预测正确，因此标识1所标记的指令都变为非推测的并且可以提交；而标识2和标识3所标记的指令必须失效且释放所占用的再定序缓冲记录项。从正确路径取指后，分支预测可以重新开始，标识1重新用来标记第一个推测基本块。另外，在分支验证时，相关的BTB记录也要进行相应的更新。

下面用PowerPC 604来说明在一个实际的超标量处理器中动态分支预测技术的实现。PowerPC 604是4发射的超标量流水线，每个时钟周期可以读取、译码和分派4条指令。它没有采用一个统一的BTB，而是采用了两个分离的缓冲支持分支预测，称为分支目标地址缓冲（BTAC）和分支历史信息表（BHT），如图4.10所示。BTAC是一个64记录的全相联cache，存储分支目标地址，而BHT是一个512项记录的直接映射表，存储分支指令的历史信息位。下面将会解释这样设计的原因。

BTAC和BHT都是在取指段使用当前指令地址PC访问，但BTAC访问延时是单周期的，而BHT是双周期的。如果BTAC命中说明当前的取指序列中存在分支指令，则预测分支将会跳转并读出分支目标地址中用于下一周期的取指指令。由于PowerPC 604每周期读取4条指令，所以每次的取指序列中有可能包含多条分支指令。因此，通过取指地址索引到的BTAC分支目标地址是取指

序列中第一条被预测为跳转的分支指令的目标地址。在第二个周期中，即译码段，根据从BHT中得到的历史信息，将预测同一条分支指令的方向。如果该结果同BTAC的预测相同（跳转），则保持指令流不变；如果结果不同（不跳转，顺序执行），则BTAC的预测结果失效，从分支指令的下一条指令重新取指，即BHT的预测可以使BTAC的预测失效。但是正如我们所期望的，这两级的预测往往是相同的，在某些情况下，BHT将会修正BTAC错误的预测。然而，BHT也有可能会错误地修改BTAC本来正确的预测，只是这种可能性比较小。当分支结果产生后，BHT将被更新，并且根据更新内容修改BTAC的内容，如果下次要将该指令预测为跳转，则可以在BTAC中加入一项；如果下次预测为不跳转，则删除BTAC中的一项。

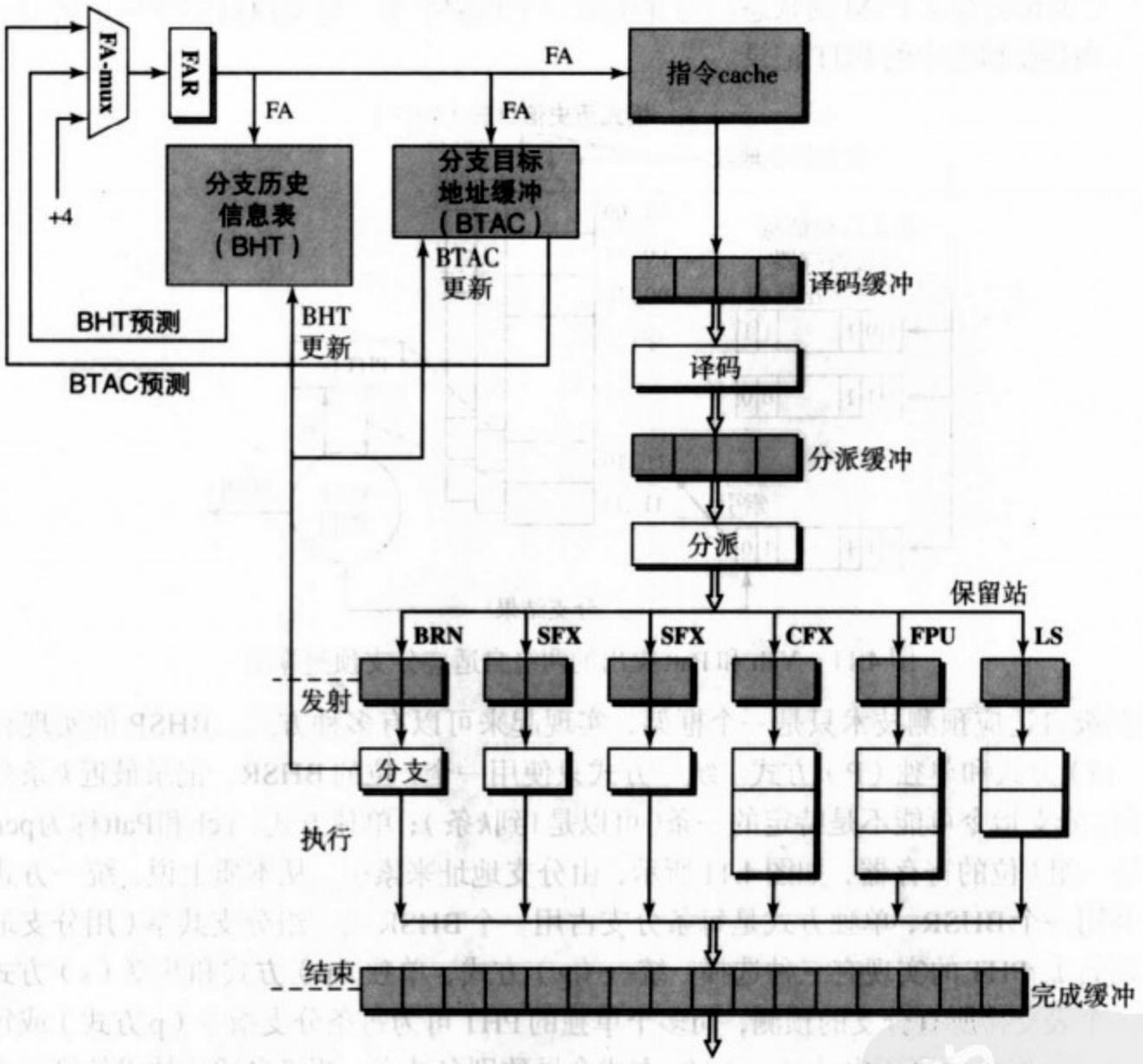


图4.10　PowerPC 604超标量微处理器中的分支预测硬件

在PowerPC 604中，分支部件的保留站包含4项，因此可以推测4条分支指令，即处理器中可以同时存在4条推测分支。为了区分4个推测基本块，只需两位的标识即可。分支执行完毕后对分支预测进行验证，推测执行的指令或者提交或者失效，相应的再定序缓冲记录也要释放，这些都需要使用两位标识。

4.1.5　先进的分支预测技术

前面讨论的动态分支预测技术存在许多局限性。对某条分支指令的预测仅仅是基于该静态分支有限的历史信息，实际的预测算法没有考虑分支执行的上下文信息。例如，算法没有使用到达当前分支指令的控制流的跳转信息，而且没有动态地根据上下文内容灵活地改变算法。研究表明，某

此分支的行为同在它之前执行的分支指令的行为有着很大的关联，因此，更精确的分支预测算法应当考虑其他关联分支指令的历史行为，从而动态地调整分支策略。

在1992年，Yeh和Patt提出了两级自适应分支预测技术，预测精度可以达到95%以上。两级自适应技术的核心是一个高度灵活的预测算法，可以随着上下文的变化调整自身内容。在前面的方案中，分支地址仅索引一个分支历史信息表，每个地址只有一条记录，而在两级自适应算法中使用一个分支历史信息表集合，称为模式历史信息表（PHT）。如图4.11所示，每个分支地址将索引一些记录的集合，然后根据动态分支上下文选择其中一条记录，上下文信息是由分支历史信息移位寄存器（BHSR）来保存的，称为一个模式（pattern）。可以使用BHSR检索PHT并选择一个相关记录，其内容作为预测算法FSM的状态信息并生成一个预测结果。分支执行完毕后，利用实际分支的结果更新BHSR和选中的PHT记录。

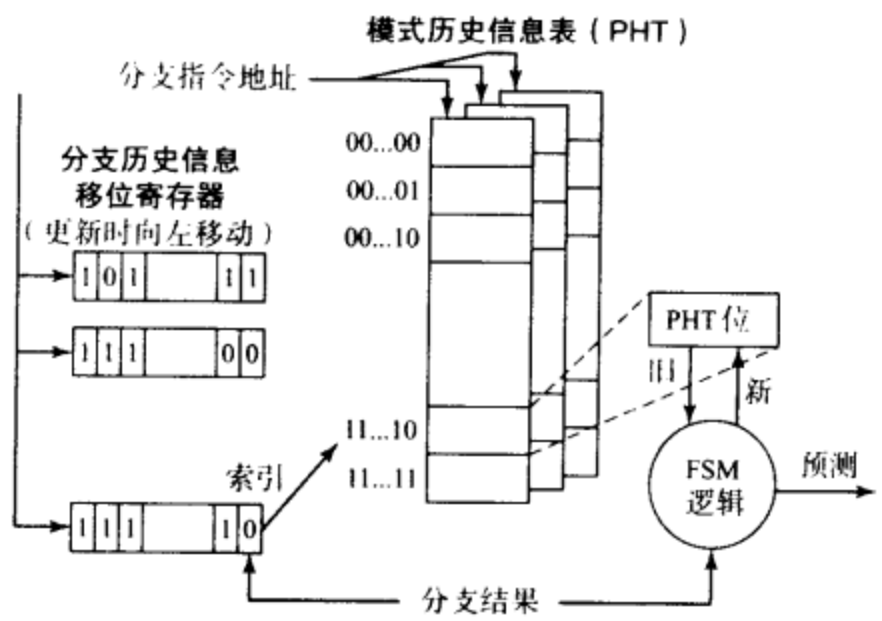


图4.11 Yeh和Patt提出的两级自适应分支预测算法

上述两级自适应预测技术只是一个框架，实现起来可以有多种方式。BHSR的实现有两种选择：统一（G）方式和单独（P）方式。统一方式只使用一个k位的BHSR，记录最近k条分支指令执行的方向，分支指令可能不是特定的一条（可以是1到k条）；单独方式（Yeh和Patt称为perbranch）的BHSR是一组k位的寄存器，如图4.11所示，由分支地址来索引。从本质上说，统一方式是所有分支指令共用一个BHSR，单独方式是每条分支占用一个BHSR或一组分支共享（用分支地址索引时会存在别名）。PHT的实现有三种选择：统一（g）方式、单独（p）方式和共享（s）方式。统一PHT利用一个表支持所有分支的预测，而多个单独的PHT可为每条分支指令（p方式）或每个分支指令组（s方式）提供预测历史信息，不过s方式会导致别名冲突。两级自适应技术的第三个设计参数是预测算法的实现。预测算法可以使用基于历史信息的FSM，Yeh和Patt称之为自适应算法（A）。

两级自适应分支预测技术的所有可能的实现可以按上述三个设计参数进行分类。每一种给定的实现都可以用三个字母的符号来表示，如GAs表示使用统一BHSR、自适应预测算法和共享PHT的实现方式。Yeh和Patt给出了三种特定的实现，在他们给定的测试程序下预测准确率可以达到97%，如下所示：

- GAg：一个BHSR，每个18位；1个PHT表，大小为$2^{18} \times 2$位。
- PAg：512 × 4个BHSR，每个12位；1个PHT，大小为$2^{12} \times 2$位。
- PAs：512 × 4个BHSR，每个6位；512个PHT，每个大小$2^{6} \times 2$位。

所有三种实现方案都使用两位自适应FSM预测器（A）。第一种实现使用了18位的统一BHSR和2^{18}个记录项的统一PHT，由BHSR来索引；第二种实现使用4路组相联（512个组）BHSR，每

个组长度为12位，以及包含2^{12}个记录项的统一PHT；第三种实现使用了和第二种方式同样多的BHSR，但每个组的长度为6位，以及512个PHT，每一项包含了2^6条记录项。所有的512组BHSR和512个PHT都用分支地址中的9位来索引，其余的地址位用于BHSR的相联访问标识。512个PHT是直接映像的，因此别名不可避免，即多个分支地址映射到同一个PHT。从实验得出的数据来看，别名对预测准确率的影响很小。传统的预测技术一般只能达到90%左右的预测精度，而两级自适应预测技术却能够得到大于95%的预测精度，因此在许多处理器中都使用了这种技术，如Intel Pentium Pro和AMD/NexGen Nx686。

在Yeh和Patt之后，McFarling在1993年、Young等人在1995年、Gloy等人在1996年都对两级自适应预测进行了深入的研究，现在一般称为关联分支预测器（correlated branch predictor）。图4.12是一个使用统一BHSR和共享PHT的关联分支预测器。两位饱和计数器作为预测FSM；BHSR记录了最近k条分支指令的执行方向，并保存动态控制流的上下文信息；PHT可以看做是一个二维的阵列表，其中包含了2^j列、2^k行以及两位预测符。如果分支地址有n位，其中j位用来索引PHT选择2^j列中的一列。由于$j<n$，可能会有两个不同的分支地址选中PHT的同一列，即别名，所以这种PHT称为共享PHT。BHSR的k位模式用来索引已选中的PHT列，并从2^k条记录项中选取一项。选中的记录项内容是两位的历史信息，用来完成基于历史信息的分支预测。传统的分支历史信息表相当于仅有一行的PHT，由j位分支地址索引，即图4.12虚线框中的部分。

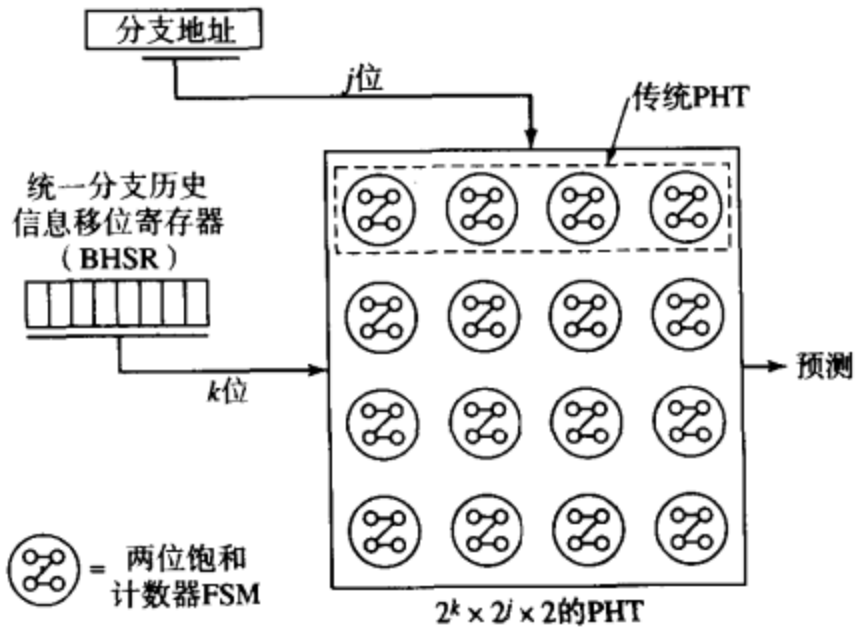


图4.12　使用统一BHSR和共享PHT（GAs）的关联分支预测器

图4.13是使用单独BHSR和共享PHT的关联分支预测器。同GAs相同，PAs也是用j位分支地址选择（PHT）2^j列中的一列，但同时还需要i位地址（可能和访问PHT的j位地址有所重叠）访问从而选中2^i组BHSR中的一组。因此，每个BHSR同一个分支地址或一组分支地址相关联（有别名），即用多个BHSR记录多组分支指令的控制流上下文信息，而不是使用一个BHSR记录所有分支指令的上下文信息，这增加了跟踪和利用不同指令间关联性的灵活程度，每一个BHSR跟踪最近k条同属一个分支指令组的指令方向信息。GAs和PAs方案的PHT大小均为$2^k \times 2^j \times 2$位，GAs的BHSR只有k位，而PAs则有2^i个k位BHSR。

1993年，Scott McFarling提出了一个非常高效的关联分支预测器，称为gshare。通过将分支地址的j位和统一BHSR的k位进行杂凑（hash）计算（相应位异或），得出的结果为$\max\{k, j\}$、索引$2^{\max(k,j)} \times 2$位的PHT表，并选中一个两位分支预测符进行分支预测，如图4.14所示。Gshare只使用了一个统一的k位BHSR和一个很小PHT，就可获得同其他关联分支预测器相当的预测精度。在DEC Alpha 21264超标量微处理器中就使用了这种技术。

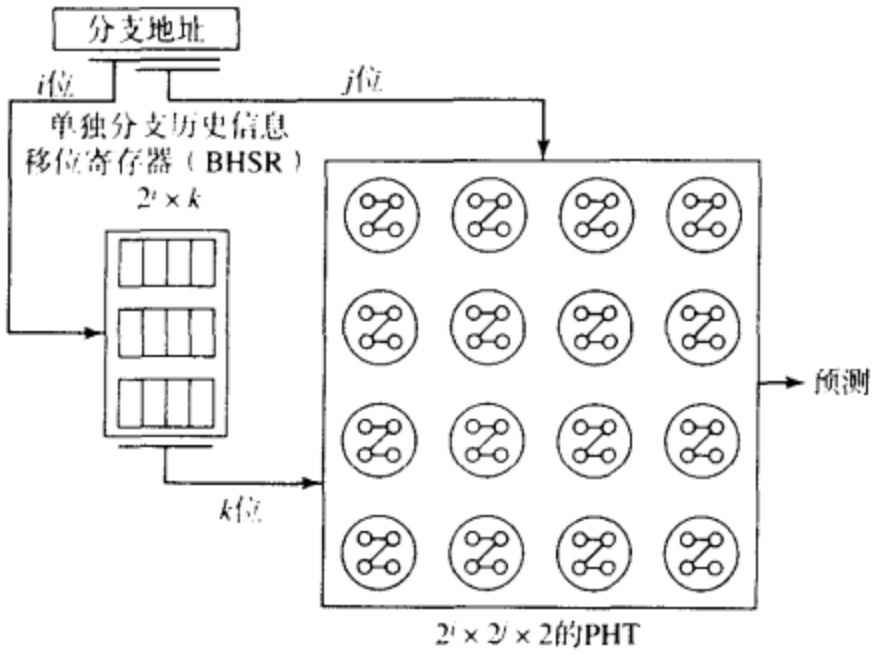


图 4.13 使用单独 BHSR 和共享 PHT（PAs）的关联分支预测器

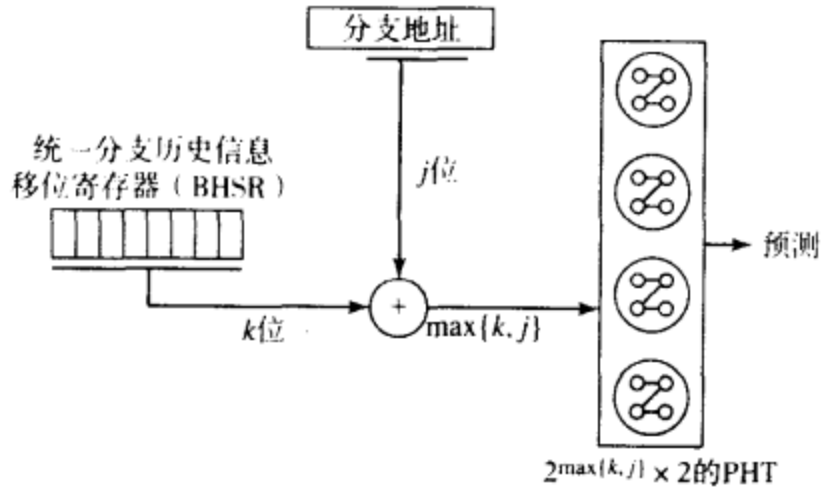


图 4.14 McFarling 的 gshare 关联分支预测器

4.1.6 其他指令流技术

指令流技术的主要目标就是在每个机器周期内为处理器内核提供尽可能多的有用指令，两个主要的挑战是条件分支和跳转分支的处理。对于一个多发射的超标量处理器，为提高条件分支处理的吞吐率，处理器每周期必须能够预测多个条件分支指令的方向和目标。例如，在每次取指4条指令时，可能所有4条指令都是条件分支。通常，处理器要用4条指令的地址索引一个4端口的BTB表，查找所有4条指令的历史信息和目标地址，因此这个预测器会十分复杂，必须根据所有历史信息位做出一个全局的预测并对取指进行推测。每周期处理多个分支的多分支预测技术已经由Conte等人（1995年）和Rotenberg等人（1996年）提出。多分支预测技术同样要求较高的分支预测准确率。统一分支历史信息表同单独历史信息表结合使用可以达到很高的预测准确率。对于那些没有很强倾向性的分支或分支序列，预测是很困难的，为此 Gus Uht 在 1995 年提出了 DEE（Dynamic Eager Execution）技术。DEE技术使用多个PC同时从多个地址取指，即从取指段开始同时存在了多条控制流，直到分支执行完毕再把错误的路径动态地删除（失效指令）。

跳转分支是提供足够有用指令的另一大障碍。在多发射机器中，取指部件每周期要能够准确地处理多条跳转分支指令，包括预测分支方向和目标地址，以及取指、对齐、合并多条分支的目标等一系列工作。Eric Rotenberg 在 1996 年提出了 trace cache 技术来解决这一难题，并且 Intel P4 处理器已经成功地实现了trace技术。trace cache是一种基于历史信息的取指机制，它将指令的动态执行轨迹信息存储在一个cache中，通过指令地址和分支结果加以访问。轨迹信息是根据动态的分支

指令行为收集起来的，包括多个不连续的基本块。每当指令地址命中trace cache，接下来的指令流就从trace cache中取指，而不是从指令cache中取指。由于trace cache中的动态指令序列可能包含多个跳转分支指令，且它们是顺序保存的，所以取指段不再需要从多个目标取指，也不再需要多端口的指令cache以及复杂的合并对齐逻辑。trace cache可以看做是按程序先前的执行轨迹对其基本块进行了重排序。当不连续的基本块第一次在支配路径上执行时，结束提交段对其进行合并对齐处理，并生成一个trace存入trace cache中。一旦trace cache被唤醒，后续的取指操作就不再来自指令cache而是来自于trace cache。trace cache中重新排序的基本块接近于动态执行的顺序，并很少在trace cache不连续的位置取指，因此，跳转分支指令的吞吐率得到了显著的提高。

4.2　寄存器数据流技术

寄存器数据流技术主要是指ALU（寄存器－寄存器型）指令在处理器执行部件中的有效执行。可以这样理解，ALU指令负责完成程序定义的“实际”工作，而控制流和load/store指令则分别负责提供所需的指令和数据。理想情况下，分支和load/store指令都是“无用”指令，不应当占用执行时间；计算的延时仅仅取决于ALU指令的处理，高效地处理这些指令是获得高性能的关键。

假定存在一个load/store体系结构，ALU指令的操作都是针对寄存器数据的，操作码指定一个二进制的操作，两个源寄存器保存源操作数，一个目的寄存器指示保存结果的寄存器。$R_i \leftarrow F_n(R_j, R_k)$定义了一个ALU指令，它的执行需要：$F_n$功能部件；$R_j$和$R_k$源操作寄存器$R_i$目的寄存器。如果$F_n$不可用，则存在结构相关；如果$R_j$和$R_k$不可用，则存在真实数据相关；如果$R_i$不可用，则存在反相关或输出相关。

4.2.1　寄存器重用和假数据相关

假数据相关、反相关和输出相关是由于寄存器重用（register reuse）引起的。如果保存操作数时不重用寄存器，就不会导致假数据相关。寄存器重用也称为寄存器回收（register recycling）并包含两种不同的形式：静态和动态。静态寄存器重用是一种编译器的优化。编译的过程主要完成两个工作：代码生成和寄存器分配。代码生成的任务是生成面向机器的指令，生成器一般都假设有无限多的寄存器符号可用来保存中间数据，每个寄存器符号保存一个值且只写一次，生成的代码称为单一赋值代码。然而，特定ISA体系结构的寄存器数目是有限的，寄存器分配工具必须将无限的寄存器符号资源映射到有限的、固定的体系结构寄存器上，并尽可能地保证中间数据驻留在寄存器中，同时避免存储器与寄存器之间频繁的数据移动，这都是通过寄存器重用来实现的。当寄存器中的旧值已保存且不再使用时，该寄存器就可以写入一个新值，寄存器重用使得一个寄存器可以保存多个值。

写寄存器称为寄存器的定义，读寄存器称为寄存器的使用。每个寄存器的定义可能对应多个寄存器的使用。定义和最后一次使用之间的持续时间称为该值的活跃区间（live range）。在生存范围的最后一次使用完毕后，该寄存器就可以被赋值为另一个值，同时开始另一个活跃区间。寄存器分配工具一般都尽量把不重叠的活跃区间映射到同一个体系结构寄存器，以达到最大程度的寄存器重用。单一赋值代码中，寄存器符号和值是一一对应的，经过寄存器分配，每个体系结构寄存器成为了一个变量并可以接受多个赋值，值和寄存器的一一对应关系也就不存在了。

如果指令是顺序执行的，且在寄存器的先前定义之前或先前定义的最后一次使用之前不允许对其重新定义，那么共享同一个寄存器的多个活跃区间在执行时永远不会重叠，所以寄存器的重用不会带来任何问题。若程序在执行时按原始顺序进行，值与寄存器之间的一一对应关系可以隐式地得到保护。然而在超标量机器中，特别是由于指令可以乱序执行，寄存器的读写操作并不按照程序的

原始顺序，因此值与寄存器间的一一对应关系被打破。为了保证语义正确性，所有的反相关和输出相关必须被检测并消除。只要假数据相关消除了，处理器仍旧可以以乱序读写寄存器。

寄存器重用的动态形式是指循环中的指令重复执行时导致的重用。较先进的超标量机器支持多条指令同时处理，对于执行一个相对较小的循环，可能会有多个循环迭代同时处于流水线中，即存在来自不同循环迭代的寄存器定义指令的多个副本，这就是寄存器重用的动态形式。因此，必须检测并消除来自多个循环迭代的动态指令之间的反相关和输出相关，以保证程序正确地执行语义。

消除反相关和输出相关的一个方法是终止导致相关的指令，直到前导指令访问相关寄存器完毕。如果两条指令之间发生的是反相关（WAR），后面的指令（更新寄存器的指令）必须终止，直到在前的指令完成寄存器的读取；如果发生的是输出相关（WAW），后面的指令（更新寄存器的指令）必须等待直到前面的指令完成寄存器的更新操作，这些不必要的停顿会导致性能急剧地下降。假数据相关是由寄存器的重用而产生的，并不是程序语义所固有的。

4.2.2 寄存器重命名技术

处理假数据相关的一种更有效的方法是为同一体系结构寄存器中的多个定义动态分配不同的名字，称为寄存器重命名（register rename）。寄存器重命名需要硬件的支持并可以执行时撤销寄存器重用，恢复所有当前指令的值与寄存器之间的一一对应关系。通过重命名，程序在执行过程中重新生成了单一赋值代码，从而消除了反相关和输出相关，使得本来具有假相关的指令可以并行地执行了。

实现寄存器重命名的一般方法是在体系结构寄存器文件（ARF）之外配置一个独立的重命名寄存器文件（RRF）。RRF可以通过直接复制ARF加以实现，RRF只是ARF的一个副本，一个寄存器可以重命名一次，但在这种实现方法中，RRF的使用效率不高。当前许多设计是用入口比ARF少的寄存器堆来实现RRF，RRF中的寄存器可以重命名ARF中的任何一个寄存器。这种重命名策略相当灵活、方便、高效，但需要一个映射表来保存对应关系。如图4.15所示。

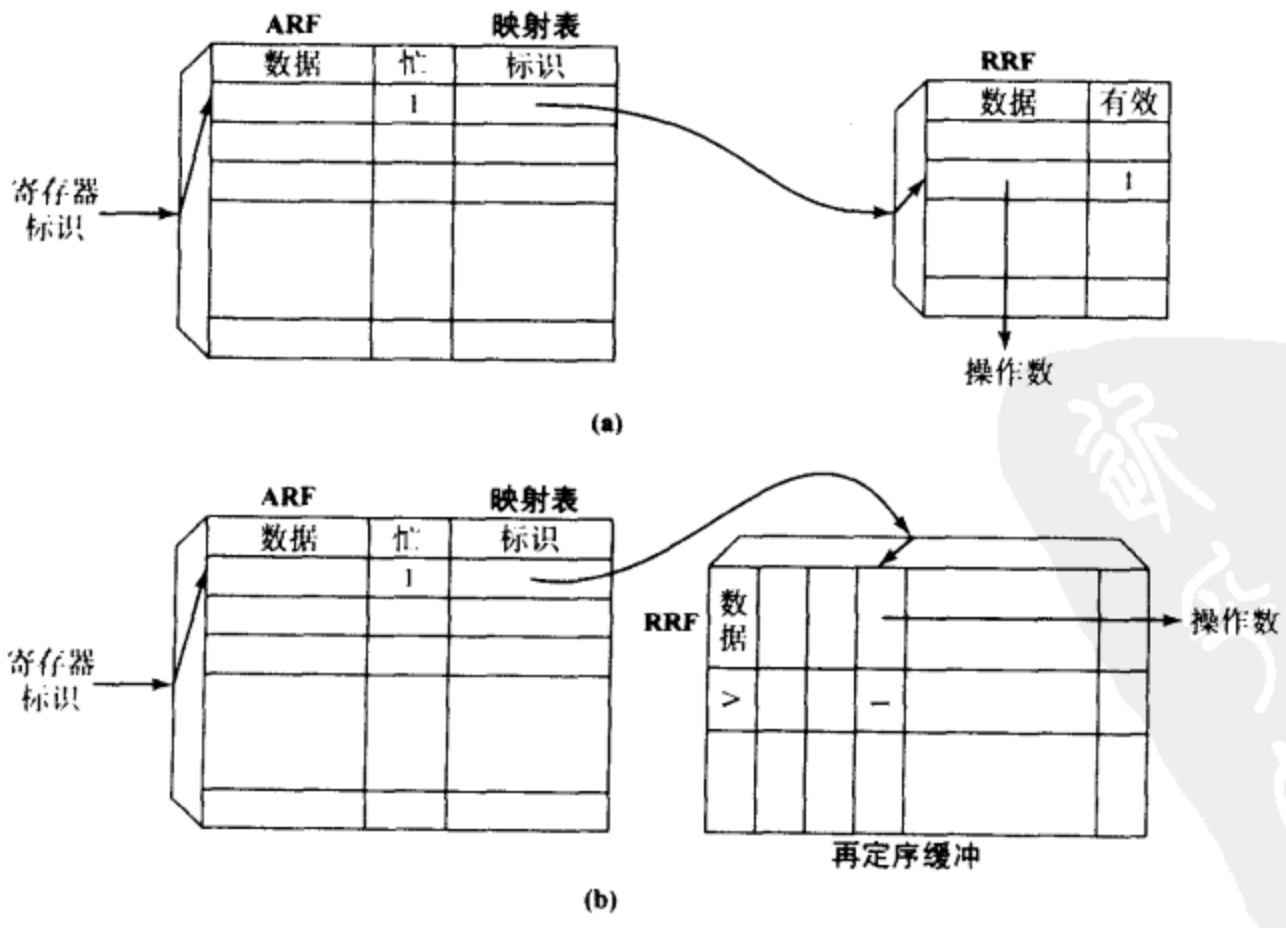


图4.15 重命名寄存器文件（RRF）的实现方式：（a）独立结构；（b）同再定序缓冲结合

当使用单独的RRF实现寄存器重命名时，RRF的放置有两种选择：一种是用同ARF一样的独立结构实现RRF，如图4.15（a）所示；另一种是将RRF和再定序缓冲结合实现，如图4.15（b）所示。两种方式都需要在ARF中加入一个忙/闲位和一个映射表。如果ARF选中的某寄存器已经置为“忙”，即该寄存器已经被重命名，则读取映射表中指向RRF的指针或标识。对于前一种实现，标识定义了一个重命名寄存器，用来索引RRF；而后一种实现的标识则定义了再定序缓冲的一个入口，用来索引再定序缓冲。

只从图4.15来看，两种实现似乎没有什么区别，其实它们之间存在着细微但本质上的区别。如果RRF作为再定序缓冲的一部分实现，再定序缓冲的每条记录项包含一个额外的域作为重命名寄存器，因此对每条正在执行的指令都分配了一个重命名寄存器，这是一种基于最坏情况的设计。由于并不是每条指令都进行寄存器定义，比如分支指令就不更新任何体系结构寄存器，因此这种设计十分浪费。另一方面，再定序缓冲集成提供了RRF从功能部件接收数据的端口，并在指令完成段将并更新ARF。如果RRF用单独的寄存器堆实现，则需要提供额外的端口从功能部件接收数据以及更新ARF。具体设计中选择哪种RRF需要进行利弊权衡，在实际的机器中两种方式都有体现。下面我们将主要针对独立RRF进行重命名技术的深入探讨。

寄存器重命名完成三项任务：读取源寄存器；分配目的寄存器；更新寄存器。源寄存器的读取一般在译码段（或分派段）完成，目的是读取源操作数。指令译码时，源寄存器标识用来索引多端口ARF读取操作数。每个操作数的读取可能遇到三种情况。第一，如果寄存器不忙（没有置“忙/闲”位），即没有未完成的写操作要更新该寄存器，其中的值是最新的，则读出操作数。第二，如果寄存器忙，即存在一个未完成的写操作要更新此寄存器，其中的值不是最新的，则访问对应的映射表并索取重命名标识，然后用该标识访问RRF。此时又分为两种情况：在访问RRF时，如果读到的记录有效位为1，即该寄存器结果已经得出，尚在等待提交，此时表明操作数已经就绪，可以从RRF中读出并使用；如果有效位为0，即寄存器更新指令仍未执行完毕，重命名寄存器中的值不能使用，此时可以将重命名寄存器的标识从映射表定向到保留站，当操作数就绪时再根据这个标识读取。源操作数读取的三种情况如图4.16所示。

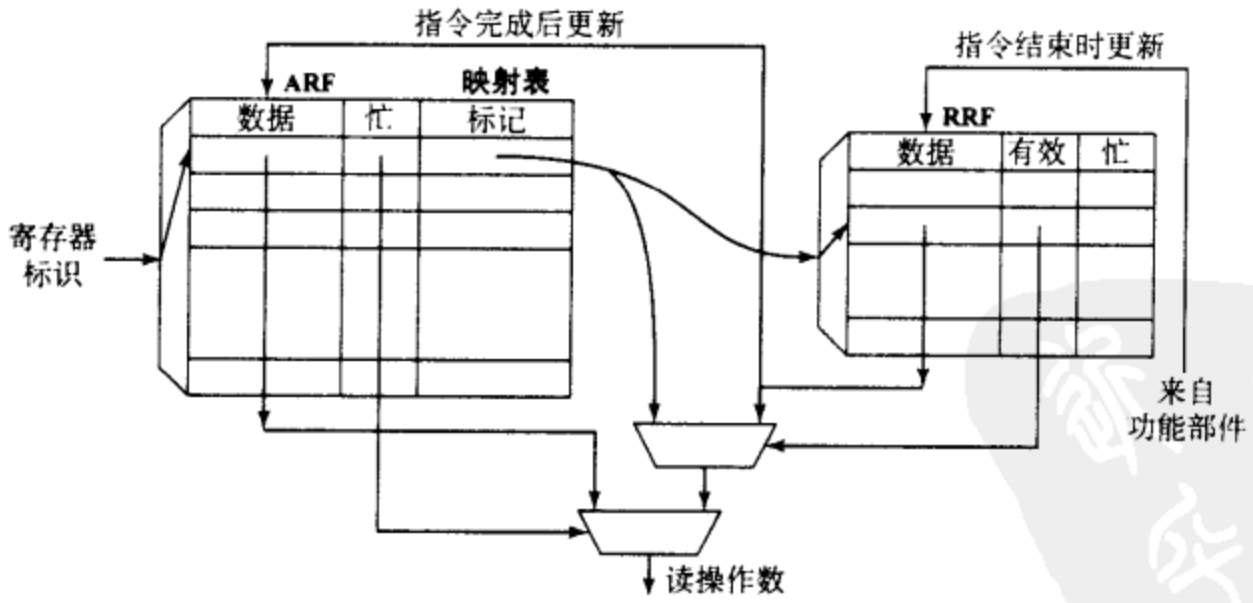


图4.16　寄存器重命名的任务：读取源操作数、分配目的寄存器和寄存器更新

分配目的寄存器同样在指令译码段（或分派段）完成，这一过程分为三个子任务：置“忙/闲”位、分配标识和更新映射表。指令译码时，目的寄存器标识用来索引ARF，选中的体系结构寄存器置为“忙”，从而变为“未决”状态（pending）。然后为该寄存器分配一个未使用（由“忙/闲”标识是否使用）的重命名寄存器，并将该重命名寄存器置为“忙”。其在RRF中的索引将作为标识写入ARF对应的映射表，供后继相关指令读取操作数时使用。

寄存器更新则在流水线的后端完成，它不是译码/分派段重命名行为的一部分，但对RRF的操作产生直接的影响。寄存器更新分为两个步骤，如图4.16所示。寄存器更新指令执行完毕后，结果将写入目的标识指定的RRF相应记录项；指令完成后，结果再从RRF复制到ARF。因此，寄存器的更新包括RRF的修改和ARF的修改，如果更新指令位于再定序缓冲的头部，这两步就可以在连续两个时钟内完成，否则若再定序缓冲中该指令前还有其他未完成的指令，这两步之间则会隔开若干个时钟。一旦重命名寄存器完成从RRF到ARF的复制，它就被置为“闲”，可以重新为其他的体系结构寄存器提供重命名。

目前，我们一直假设寄存器重命名是用两个独立的寄存器文件（ARF和RRF）来实现的，其实这是没有必要的。体系结构寄存器和重命名寄存器可以集合成一个寄存器文件加以实现，其容量为ARF和RRF的记录数目之和。集中的寄存器文件不固定区分哪些是体系结构寄存器，哪些是重命名寄存器，每个寄存器既可以指定为体系结构寄存器使用，又可以指定为重命名寄存器使用。不像独立的ARF/RRF实现那样，指令完成时必须将RRF的结果复制到ARF，集中寄存器文件的实现只需将寄存器的名称从重命名寄存器改为体系结构寄存器即可，这节约了RRF和ARF之间的互连传输。集中寄存器文件最主要的缺点就是提高了硬件复杂度，其次就是现场切换要保存机器状态时，包含机器体系结构状态的寄存器子集必须在状态开始保存之前被显式地标出。

在IBM RS/6000设计中的浮点部件使用了集中寄存器文件的方法，如图4.17所示，它使用40个物理寄存器来支持ISA定义的32个体系结构寄存器。根据映射表的内容确定40个物理寄存器中的哪32个为体系结构寄存器。映射表有32条记录，用5位的体系结构寄存器标识索引，每条记录返回一个6位标识，指示哪一个物理寄存器是该体系结构寄存器对应的重命名寄存器。

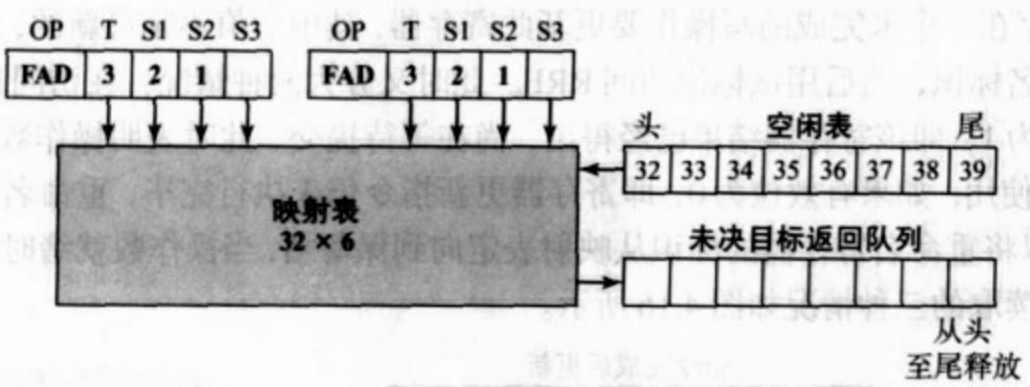


图4.17 IBM RS/6000中浮点部件（FPU）的寄存器重命名策略

RS/6000的FPU是流水化的功能部件。重命名段在译码段之前，包括一个映射表、两个循环队列以及相应的控制逻辑。第一个队列称为空闲表（FL），包含当前可用于重命名的物理寄存器。第二个队列称为未决目标返回队列（PTRQ），包含了那些已经用于对体系结构寄存器重命名的物理寄存器，而这些体系结构寄存器在随后的指令中又被重新命名。一旦PTRQ中物理寄存器的最后一次使用开始，该寄存器就可以返回到FL表中了。重命名段每周期可以处理两条指令，而且由于三个源操作数、一个目的操作数的FMA指令（乘-加指令）可能同时出现，映射表必须提供8个端口并支持多达8个寄存器标识的同时转换。初始化时，映射表置为特定的对应关系，即体系结构寄存器i映射到物理寄存器i（i=0, 1, 2, …, 31），物理寄存器32到39链入FL，而PTRQ保持为空。

指令流过重命名段时，可以用体系结构寄存器标识索引映射表得到物理寄存器标识。8端口的映射表有32条记录项并用5位体系结构寄存器标识索引，每条记录包含一个6位标识，指出体系结构寄存器映射的物理寄存器。映射表的内容是体系结构寄存器同物理寄存器之间最新的映射关系，它确定了当前有哪些物理寄存器代表体系结构寄存器。

在RS/6000的FPU设计中，只有load指令才能触发新的重命名操作，这防止了由于消除假数据相关、等待load指令执行而导致的流水线停顿。每当load指令流过重命名段，目的寄存器将标

识索引映射表，选中的记录内容则压入到PTRQ队列，同时，FL中的下一个物理寄存器载入映射表。这种实现方式有效地将目的寄存器重新定义到一个不同的物理寄存器，所有以该体系结构寄存器为源操作数的后继指令都将使用新的物理寄存器标识作为源寄存器。在重命名段（即译码段和执行段）之后，FPU只使用物理寄存器标识，所有真相关的消除也都是使用物理寄存器标识。

映射表技术是最有效、最全面的寄存器重命名技术，每个物理寄存器都可以用来对任何体系结构寄存器进行再定义。可以使用复杂的硬件逻辑实现对8端口映射表和两个循环队列的控制，同时由于很难确定哪条指令是最后使用寄存器的指令，将PTRQ中的寄存器返回到FL也是一件麻烦的事。这种映射表同基于独立重命名寄存器的实现技术不同，在指令完成后，不需要进行重命名寄存器向体系结构寄存器的内容复制。另一方面，在发生中断和现场切换时，必须根据映射表的内容及时确定出包含机器当前体系结构状态的物理寄存器。

寄存器的重用导致了寄存器数据之间存在反相关和输出相关，为避免停顿，当前多数超标量微处理器都实现了一定形式的寄存器重命名。寄存器重命名一般都在指令译码时完成，而且由于要同时实现多条指令的多个寄存器标识的重命名，其操作十分复杂，尤其对于发射宽度较大的超标量机器。此外，还有可能在同一个取指指令组中对某一寄存器进行多次再定义。因此，在不影响机器时钟周期的前提下，实现多发射超标量机器中的重命名机制是一件很困难的事。为了消除由于寄存器数据假相关导致的串行化限制，提高机器性能，有必要开发一种动态的重命名技术。

4.2.3　数据相关和数据流极限

两条指令间的写后读相关（RAW）称为数据真相关，因为它们之间是生产者（producer）和消费者（consumer）的关系。位置在后的指令为消费者，位置在前的指令为生产者，生产者产生消费者所需的操作数。真相关强制相关指令之间必须串行化，生产者必须在消费者执行之前完成。真相关来自程序的语义信息，可用数据流图和数据相关图（DDG）来描述。

图4.18是一个实现FFT（快速傅立叶变换）的代码段，两条高级语言描述的语句编译后成为16条汇编指令，其中包含load和store指令。浮点数组变量保存在存储器中，在执行前必须先进行载入，计算完毕后，结果再存回存储器。整数寄存器*ri*保存数据的索引地址，浮点寄存器*fj*保存中间数据。描述浮点寄存器读写操作的DFG如图4.19所示。

```
w[i+k].ip = z[i].rp + z[m+i].rp;
w[i+j].rp = e[k+1].rp * (z[i].rp - z[m+i].rp) - e[k+1].ip * (z[i].ip - z[m+i].ip);
```

(a)

```
i1:  f2 ← load,4(r2)
i2:  f0 ← load,4(r5)
i3:  f0 ← fadd,f2,f0
i4:  4(r6) ← store,f0
i5:  f14 ← load,8(r7)
i6:  f6 ← load,0(r2)
i7:  f5 ← load,0(r3)
i8:  f5 ← fsub,f6,f5
i9:  f4 ← fmul,f14,f5
i10:  f15 ← load,12(r7)
i11:  f7 ← load,4(r2)
i12:  f8 ← load,4(r3)
i13:  f8 ← fsub,f7,f8
i14:  f8 ← fmul,f15,f8
i15:  f8 ← fsub,f4,f8
i16:  0(r8) ← store,f8
```

(b)

图4.18　FFT代码段：（a）原始描述；（b）编译后的汇编指令

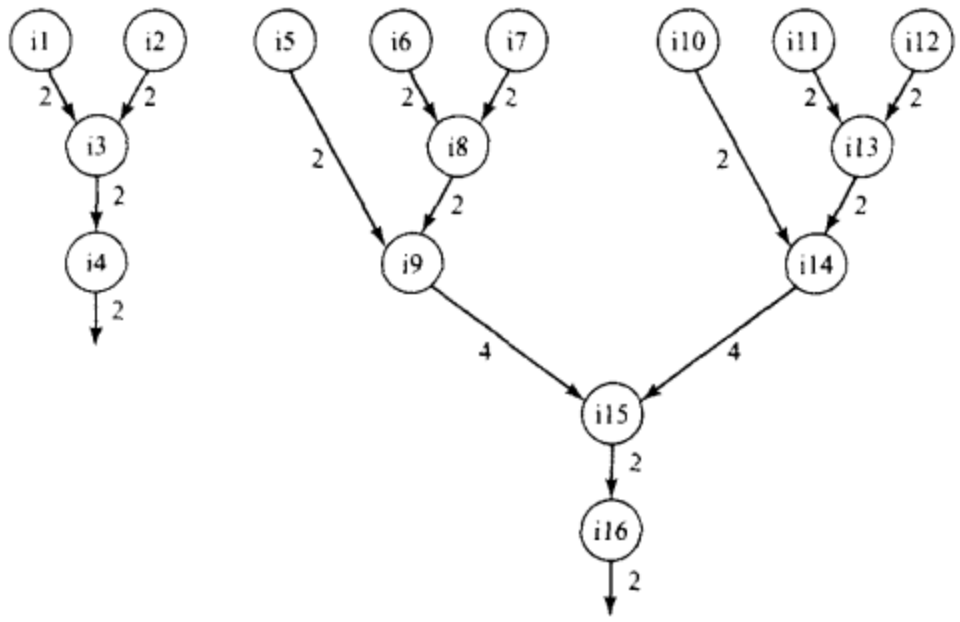


图 4.19　图 4.18（b）代码段的数据流图

图中的每个节点代表图4.18（b）中的一条指令，两条指令之间的有向边代表它们之间存在真相关。DFG 中相关寄存器可以同相关边一起指定，相应延时也可以标出。图 4.19 中，每条边都标出了生产者指令的执行延时，其中 load、store、加法和减法指令延时为 2 周期，乘法指令为 4 周期。

相关边的延时可以累加，总的累计延时最长的相关链就是 DFG 的关键路径。即使机器有足够的资源，该代码段也不能再比关键路径执行得快，这就是所谓程序执行的数据流极限，即能够达到的最高性能。图 4.19 中的数据流极限为 12 个时钟周期，是由程序中的真相关决定的。传统的数据流执行模型保证了每条指令当其所有操作数就绪后马上开始执行。当前所有的寄存器数据流技术本质上都是为了逼近数据流极限这一目标而设计的。

4.2.4　经典的 Tomasulo 算法

IBM 360/91 的浮点部件设计实现了目前人们所熟知的 Tomasulo 算法，并为现代超标量处理器的设计打下了基础。当前大多数的寄存器数据流技术的特点在Tomasulo算法中都能找到它们的影子，因此我们有必要进行深入的研究。首先将介绍 IBM 360 浮点部件最初的设计方案，然后详细描述修改后的 IBM 360/91 FPU 中 Tomasulo 算法的实现，最后用一个例子阐述其操作过程及其产生的效率。

IBM 360 浮点部件的最初设计如图 4.20 所示。FPU 包含两个功能单元：一个浮点加法单元和一个浮点乘除单元。同时包含三个寄存器文件：浮点寄存器（FLR）、浮点缓冲（FLB）和存储缓冲（SDB）。FLR 有 4 个寄存器并作为体系结构浮点寄存器。存储器-存储器和存储器-寄存器寻址方式的浮点指令都进行了预处理，地址计算和存储访问都在 FPU 外部进行。当数据从存储器读出后，被放入 6 个 FLB 寄存器中的一个。同样，如果指令的目的操作数是存储器，结果就被放入 3 个 SDB 寄存器中的一个，然后由另外一个独立部件访问 SDB，将结果存入存储器。通过 FLB 和 SDB 两个额外的寄存器文件处理寄存器-存储器和存储器-存储器型指令，FPU则在功能上就是一个寄存器-寄存器型机器。

IBM 360 的指令单元（IU）对所有指令进行译码，并将所有浮点指令顺序地送往浮点操作栈（FLOS）。在 FPU 中，浮点指令进一步译码并从 FLOS 顺序地发送至两个浮点功能单元中去。两个功能单元都不是流水化的，从而产生多周期的延时：加法部件是两个周期，乘除部件对乘法和除法分别是 3 个周期和 12 个周期的延时。

在 20 世纪 60 年代中期，IBM 开始着手开发后来称为 360 系列机 91 模型的处理器。其设计的目标是能够并发执行多条浮点指令，使流水线达到每周期完成一条指令的吞吐率。由于 360 系列机的 ISA 寻址方式极其复杂，且执行单元是多周期延时，因此设计难度很大。最终的结果修改了 360/91 中的浮点单元 FPU，其中使用了 Tomasulo 算法，如图 4.21 所示。

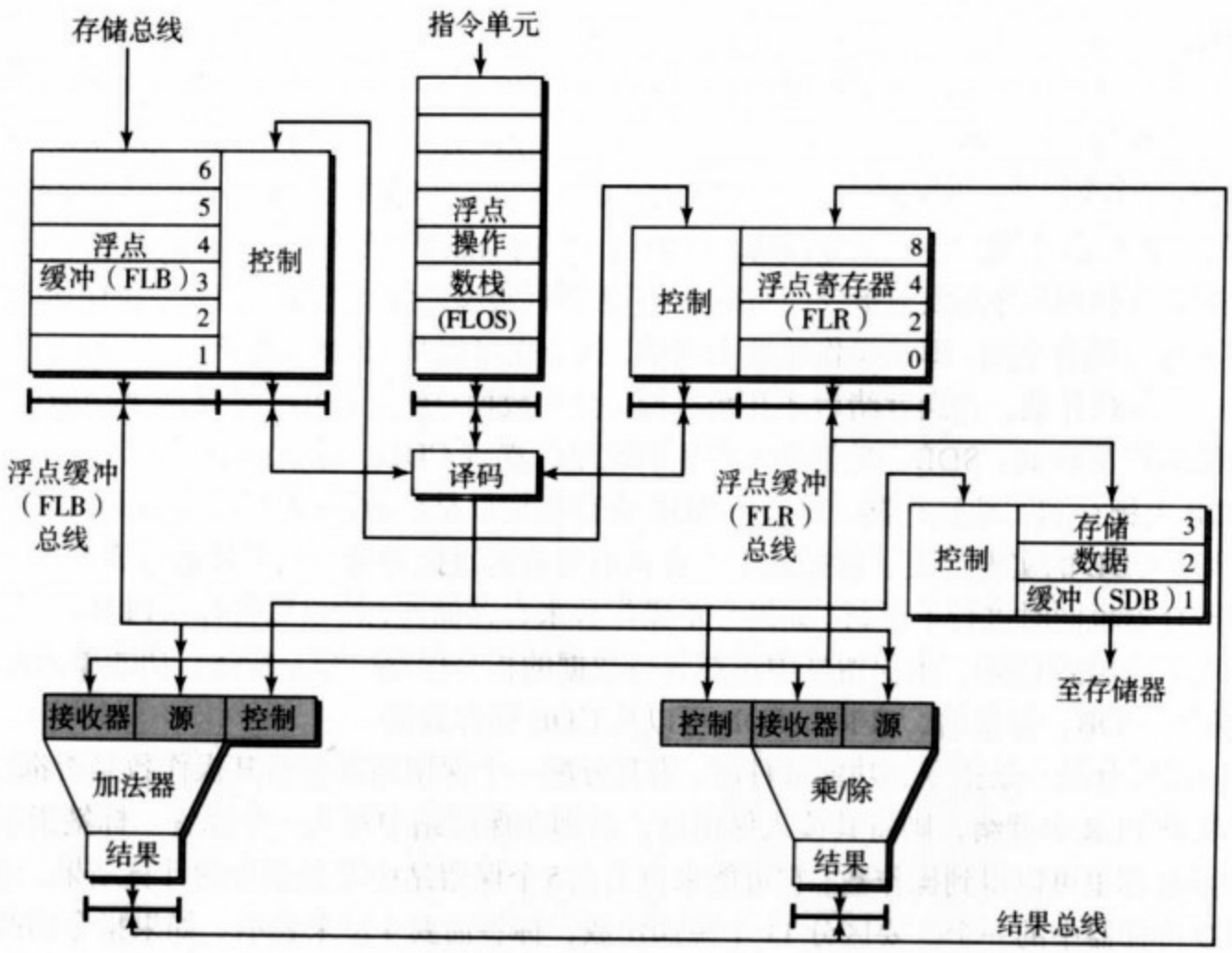


图 4.20　IBM 360 浮点部件的最初设计

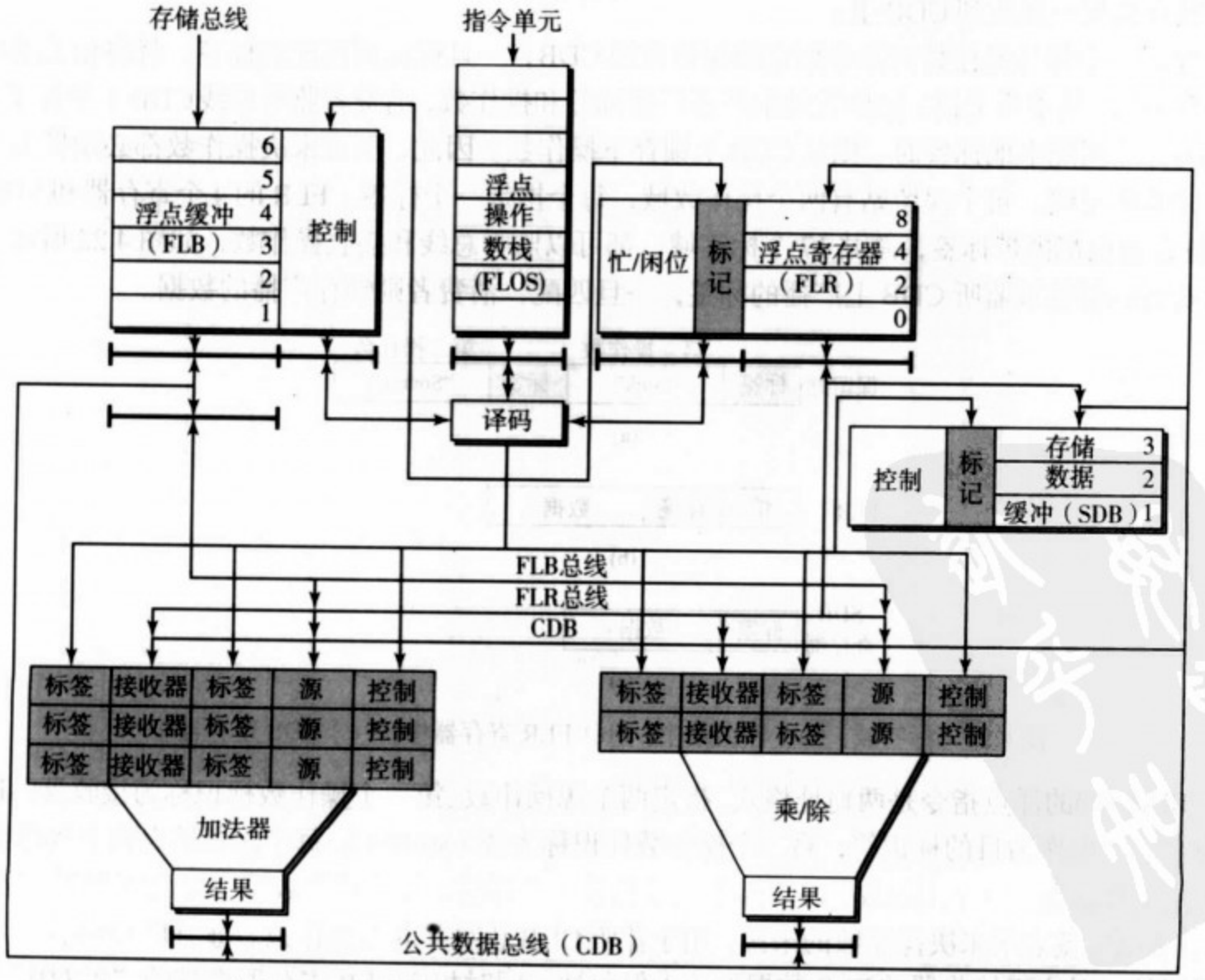


图 4.21　IBM 360/91 使用 Tomasulo 算法后的浮点部件设计

Tomasulo算法在初始FPU的基础上加入了三种新的机制：保留站、公共数据总线和寄存器标签。在FPU的初始设计中，每个功能单元在输入端只有一个缓冲单元，保存当前正在执行的指令。如果功能单元忙，且FLOS的下一条指令需要同一个功能单元，则停止发射。为了消除结构相关的瓶颈，在功能单元的输入端配置多个缓冲，即保留站。加法单元有3个保留站，乘除单元有两个保留站，保留站可以看做是虚拟的功能单元，只要有空闲的保留站，FLOS就可以向功能单元发射指令，即使功能单元当时正在执行其他指令。FLOS顺序地发射指令，因而避免了因不同类型的浮点指令之间需要排序所带来的停顿。

只要保留站有空闲，即使操作数还未就绪，指令也可以从FLOS发射到功能单元，然后指令在保留站中等待操作数，直到就绪后才开始执行。公共数据总线（CDB）连接两个功能单元的输出、所有保留站以及FLR、SDB。功能单元产生的结果广播到CDB，保留站中需要该数据的指令将数据从CDB上锁存到本地锁存器。FLR和SDB寄存器如果是该数据的目的寄存器，也要从CDB锁存该结果。CDB的实现方便了数据从生产者到消费者的直接传递，而无需通过寄存器中转，指令的目的寄存器也同时进行了更新。如果一个操作数来自存储器，读出后将存入FLB寄存器，即FLB数据也可以输出到CDB，由保留站中正在等待数据的指令保存。综上所述，功能单元和FLB可以输出数据到CDB，保留站、FLR和SDB可以从CDB锁存数据。

当FLOS分派一条指令到功能部件时，为其分配一个保留站并查看其操作数是否都已就绪。如果操作数在FLR中就绪，则将其读入保留站，否则在保留站中写入一个标签。标签指示没有就绪的操作数在哪里可以得到操作数，它可能来自当前5个保留站中某条指令的计算结果，也可能来自6个FLB寄存器中的一个。要区分11个源操作数，标签需要4位来表示。如果指令的两个操作数域有一个是标签而不是具体数据，就表示该指令正在等待操作数。一旦操作数计算出来，生产者就将标签和数据一起放到CDB上。

保留站中等待操作数的指令用它的标签监视CDB，一旦探测到匹配的标签，就将相关的操作数锁存下来。从本质上讲，操作数的生产者广播标签和操作数，消费者监听总线CDB上的标签，当广播的标签匹配本地标签时，则从CDB上锁存下操作数。因此，所有未决操作数都必须携带一个标签并监听总线。每个保留站有两个操作数域，每个携带一个标签。FLR的4个寄存器和SDB的3个寄存器也都携带标签，总共17个标签域，都可以监听总线和接收操作数，如图4.22所示。每个消费者的标签域监听CDB上广播的标签，一旦匹配，消费者则锁存广播的数据。

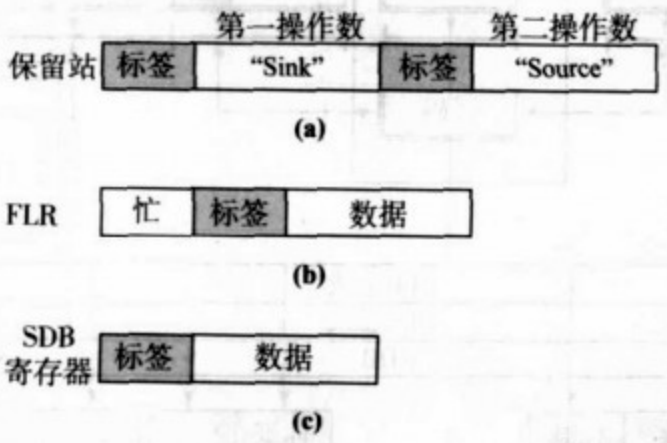


图4.22 标签域：（a）保留站中；（b）FLR寄存器中；（c）SDB寄存器中

IBM 360的浮点指令是两地址格式，指定两个源操作数。第一个操作数标识称为接收器（sink），因为它同时也作为目的标识符；第二个操作数标识称为源（source）。每个保留站有两个操作数域，一个用做接收器，一个用做源。每个操作数域携带一个标签，如果操作数域中是真实数据，标签位为0；否则标签表示未决操作数的来源，用于监听CDB总线并取得操作数。每当FLOS将指令分派到保留站，对应于接收器的FLR数据将读入保留站，同时相应FLR寄存器携带的“忙/闲”位为

忙，说明该寄存器存在一个未决的写操作；在标签域中写入这个保留站的标识，即指出该数据最终将从哪里得到。接下来如果某条后继指令需要该操作数作为源，当其分派到保留站后，则只将FLR中的标签域（又称伪操作数）写入保留站相应的标签域。若其“忙/闲”位为忙，即FLR值不是最新的，其标签域指示的是真实数据将从哪里得到。除了保留站和FLR寄存器，SDB寄存器也可能是未决操作数的目的寄存器并同样需要标签域来指示。

下面用一个例子来说明一下Tomasulo算法的执行过程。为了简便起见，在某些地方使用了不同于IBM 360/91的设计。首先，不使用IBM 360/91的两地址指令格式，而是使用三地址指令格式来避免冲突，例子中的指令序列都是寄存器－寄存器型指令。为了减少时钟数目，FLOS每周期分派（按程序顺序）两条指令。假设指令在分派到保留站的同一时钟周期就可以开始执行，执行的延时加法均为2拍、乘法均为3拍，并允许指令将结果在最后一个执行周期定向到相关指令，且相关指令在下一周期开始执行。标签域1、标签域2和标签域3标识了加法部件的3个保留站，标签域4和标签域5标识了乘除部件的两个保留站，即保留站的ID。下面显示了4条寄存器－寄存器类型的指令序列：

```
w:  R4 ← R0+R8
x:  R2 ← R0*R4
y:  R4 ← R4+R8
z:  R8 ← R4*R2
```

图4.23显示了前3个周期的执行情况。在第1个周期内，指令w和x分别分派到（按顺序）保留站1和4，w和x的目的寄存器分别是R4和R2（即FLR的4号和2号寄存器），两个寄存器都置为忙。因为指令w对应保留站为1号，R4的标签域就写入1，表示保留站1中的指令将修改R4，同样R2的标签域写入4。指令w的两个源操作数都已准备好，因此可以立即执行，指令x则需要w的结果（R4）作为第二操作数。因此，当x分派到保留站4后，第二操作数的标签域则写入1，表示保留站1中的指令将生成该操作数。

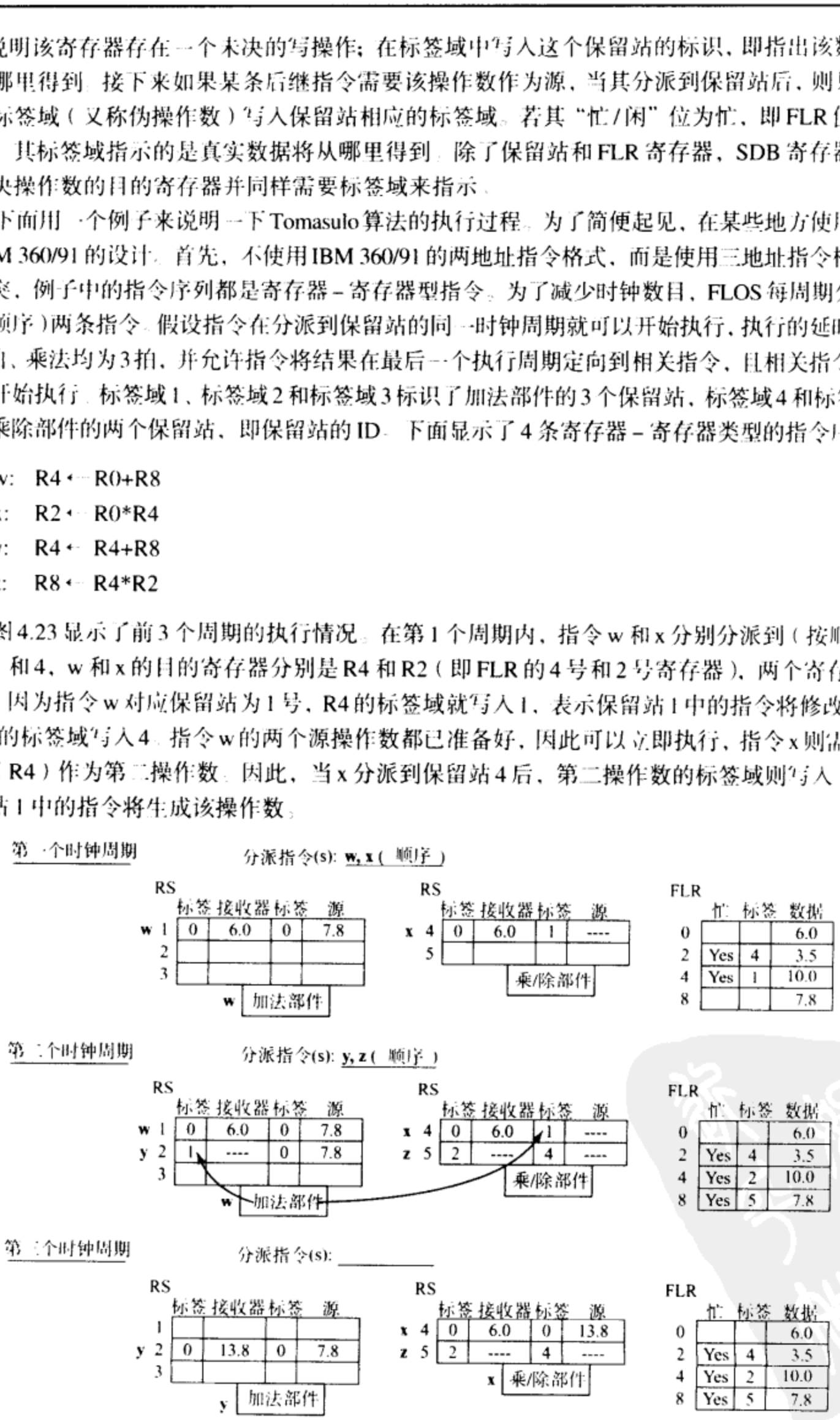


图4.23　Tomasulo算法执行示例（1）

在第2个周期中，指令y和z按顺序分别分派到保留站2和5。指令y的第一操作数是指令w的计算结果，因此在它分派到保留站2后，第一操作数的标签域写入1。同样，指令z分派到保留站5后，第一操作数和第二操作数的标签域分别写入2和4，表示保留站2和保留站4将生成这两个操作数。R4同时又是指令y的目标寄存器，所以R4的标签域更新为2，表示保留站2（即指令y）负责R4的更新。R4的“忙/闲”位仍保持“忙”，R8在指令z分派到保留站5后置为忙，标签域同时置为5。在第2个周期末尾，指令w执行完毕并将ID（保留站号1）和结果广播在CDB上，所有标签值为1的相应项都将引发一个标签匹配并将广播的结果锁存下来。保留站2的第一个标签域（指令y）和保留站4的第二个标签域（指令x）都会触发匹配，指令w的结果就被定向到相关指令x和y。

在第3个时钟周期中，指令y开始在加法单元中执行，指令x则开始在乘除单元中执行。指令y在第4个周期执行完毕并将结果和ID广播在CDB上，保留站5的第一个标签域（指令z）和R4的标签域将触发匹配并将y的结果锁存。指令x在第5个周期执行完毕并将结果和ID广播在CDB上，保留站5的第二个标签域（指令z）和R2的标签域将触发匹配并将结果锁存。在第6个周期中，指令z开始执行并在第8个周期结束。

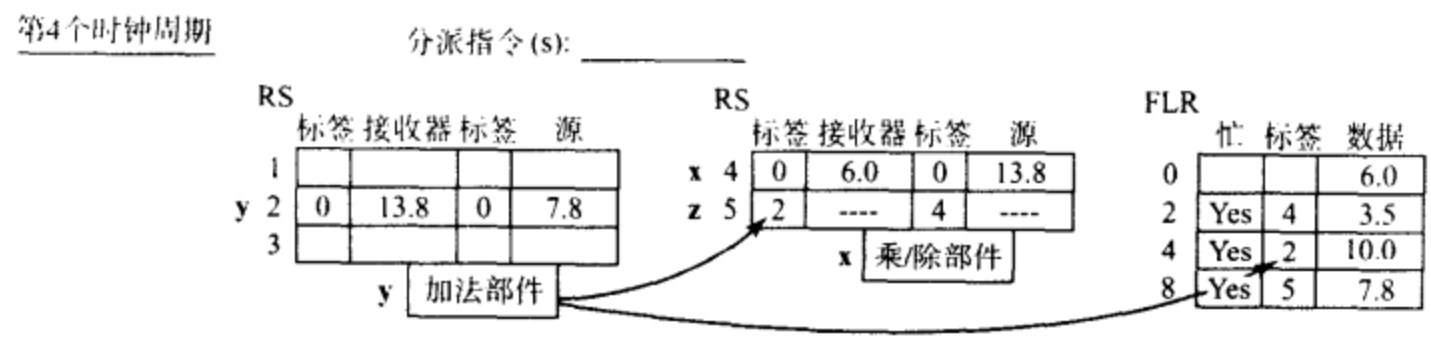


图4.24　Tomasulo算法执行示例（2）

图4.25（a）所示为上例4条指令的数据流图，4条粗体线代表4个真相关，另外3条为反相关和输出相关。指令按程序指定的顺序分派，分派时就将操作数读入保留站，从而解决了反相关，因此后续指令不会覆盖前导指令要读取的寄存器值。如果该操作数是未决的，就写入一个标签代替。操作数就绪后，指令通过标签匹配获得该操作数。

指令分派后，目的寄存器的标签域写入保留站ID。后续分派的指令如果有相同的目的寄存器，此寄存器的标签则更新为新指令的保留站ID，即寄存器的标签域总是保存着最新修改指令的保留站ID。如果有多条正在执行的指令目的寄存器相同，则只有最后的那条指令更新寄存器，从而隐式地解决了输出相关，只要后继指令修改了寄存器的标签，前导指令的写操作将会失效。但是这带

来了新的问题：不能支持精确异常，因为寄存器文件没有随处理器状态一起变化，即寄存器可能错过了一些中间状态。例如在图 4.23 中，第 2 周期指令 w 应该更新 R4，而指令 y 的目的寄存器和它相同，指令 y 分派时，R4 的标签域从 1（指令 w）更新为 2（指令 y）。在第 2 周期的结尾，指令 w 的广播 ID 为 1，因此 R4 不会触发标签匹配，也就不会锁存指令 w 的结果。最终 R4 只会被指令 y 修改，而如果指令x触发了一个异常，精确异常就会被打破，因为寄存器文件没有随着处理器的时序状态一同变化。

Tomasulo 算法通过寄存器重命名解决了反相关和输出相关。FLR 寄存器的每一次定义都触发一次寄存器的重命名，并用更新该寄存器的指令所在的保留站 ID 作为其标签，因而有效地消除了假相关，避免了流水线停顿，数据流极限就仅仅取决于真相关了。图 4.25（b）是仅包含真相关的数据流图。在图 4.25（a）中，如果所有 4 条指令顺序执行并消除所有数据相关（包括假相关），总的执行延时为 10 个时钟周期（假定加法延时为两个周期，乘法延时为 3 个周期）。如果仅考虑真相关，关键路径则包括指令 w、x 和 z，执行延时仅为 8 个时钟周期。因此该指令序列的数据流极限为 8 个周期，Tomasulo 算法达到了这一极限。

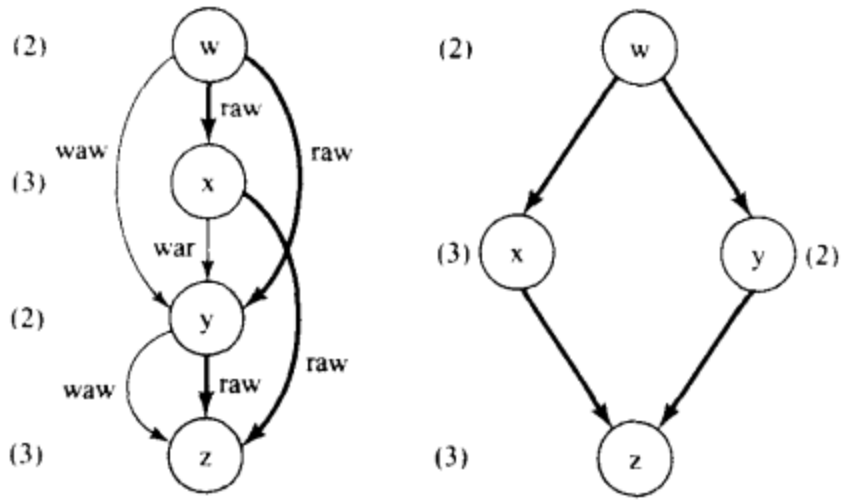


图 4.25　指令序列的数据流图：（a）所有数据相关；（b）真实数据相关

4.2.5　动态执行内核

当前大多数代表最新技术水平的超标量微处理器都嵌入了一个乱序的执行内核，流水线的前端顺序地取指、译码和分派，后端也顺序地完成和提交。乱序的执行内核（也称为动态执行内核）就像一个精简的 Tomasulo 算法，可以看做是一个嵌入式数据流（或微数据流）引擎，使指令在执行时尽量接近数据流极限。一个动态执行内核的操作过程可以描述为三个阶段：指令分派、指令执行和指令完成，如图 4.26 所示。

指令分派阶段包括目标寄存器的重命名、保留站和再定序缓冲的分配，以及将指令从分派缓冲写入保留站这三个过程。为了叙述方便，假设寄存器重命名在指令分派段完成，所有体系结构寄存器的再定义都被映射到重命名寄存器，后继的寄存器使用指令都对相应的重命名寄存器操作。这种做法对所有生产者 – 消费者的关系进行了区分，从而消除了寄存器假相关。

然后，分派缓冲中的指令按类型分派到相应保留站。假设使用的是分布式保留站，保留站指的是连接到功能单元的指令缓冲，而保留站记录指的是指令缓冲中的记录项。分配保留站记录的同时，还要为分派指令分配再定序缓冲记录，再定序缓冲记录按指令顺序进行分配。

通常，必须具备空闲的重命名寄存器、空闲的保留站记录和空闲的再定序缓冲记录，指令才能够分派，如果任何一个条件不满足，则停止分派。指令从分派缓冲到保留站记录的分派过程需要通过一个复杂的路由网络，如果路由网络的连通性低于全互连的 crossbar（在真实的设计中往往是这种情况），那么由于资源冲突也要停止分派。

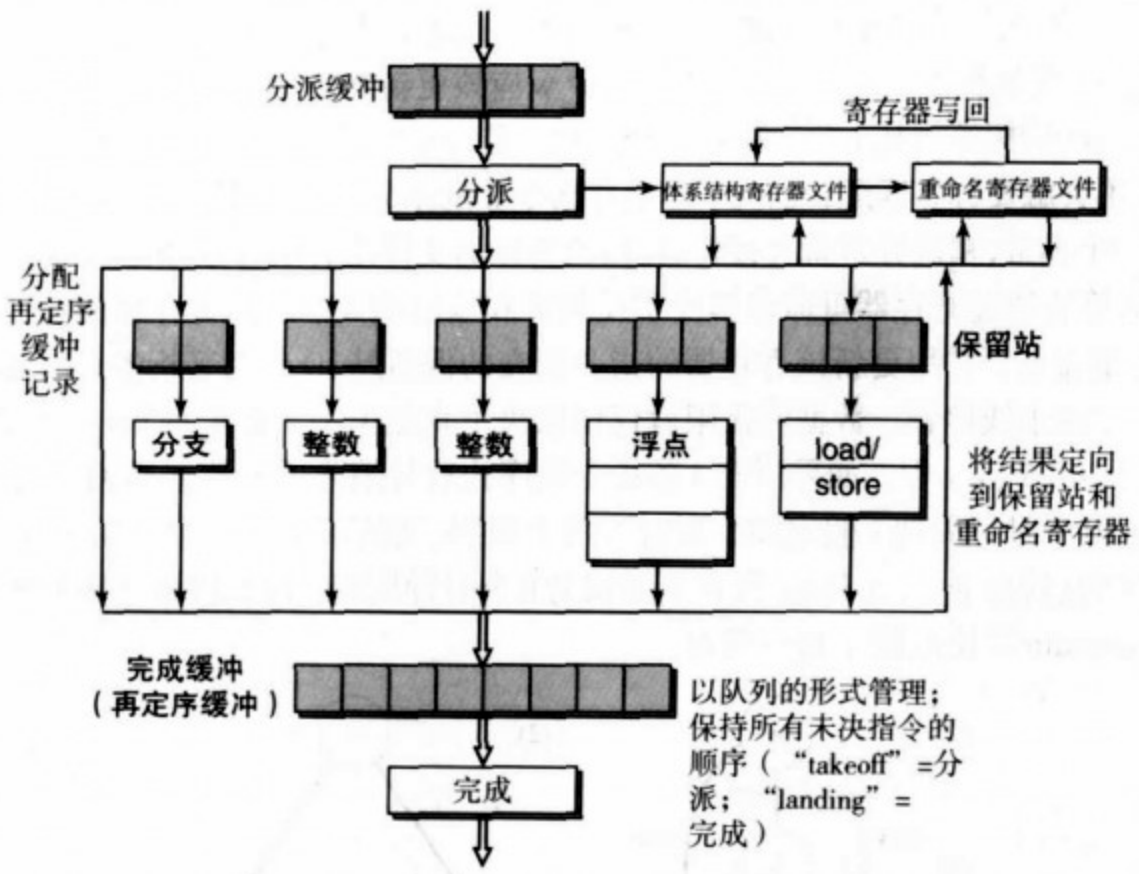


图 4.26 动态执行的微数据流引擎

指令执行阶段包括就绪指令的发射、发射指令的执行和执行结果的定向这三个过程。每个保留站负责保存就绪指令并调度它们执行。指令分派到保留站后，它的操作数可能还没有完全就绪，因此指令必须在保留站中等待，同时监听总线并检查是否存在匹配的标签。探测到标签匹配后，表明操作数已就绪，总线上广播的操作数就被锁存到保留站中。如果指令所有操作数都已得到，则变为就绪状态，即能够被发射到相应的功能单元加以执行了。如果某个时钟周期有多条指令就绪，则需要调度算法（一般使用“先来先服务”策略）选择其中一条发射。若每个保留站只连接了一个功能单元（分布式保留站就是如此），则每周期每个保留站只能发射一条指令。

指令一旦流入功能单元，就开始执行。功能单元的执行延迟各不相同，有的是单周期延时，有的则是固定多周期延时，某些功能单元的延时甚至根据操作数的值和操作类型的不同而变化。通常，即使功能单元的延时是多周期的，一旦指令开始在流水化的功能单元中执行就不会在中间再发生停顿，因为所有的数据相关都在发射之间解决掉了，因此不会再有资源上的冲突。

指令执行结束后，需要将目的标签（即目标重命名寄存器标识）和结果发布到定向总线上。所有在保留站中等待该结果的相关指令都开始标签匹配并将结果锁存下来。通过这种方式，无需将中间结果更新到相关寄存器中，就可以在相关指令之间传递数据。与此同时，RRF也使用广播的标签作为索引将广播的结果写入到选中的RRF项中。

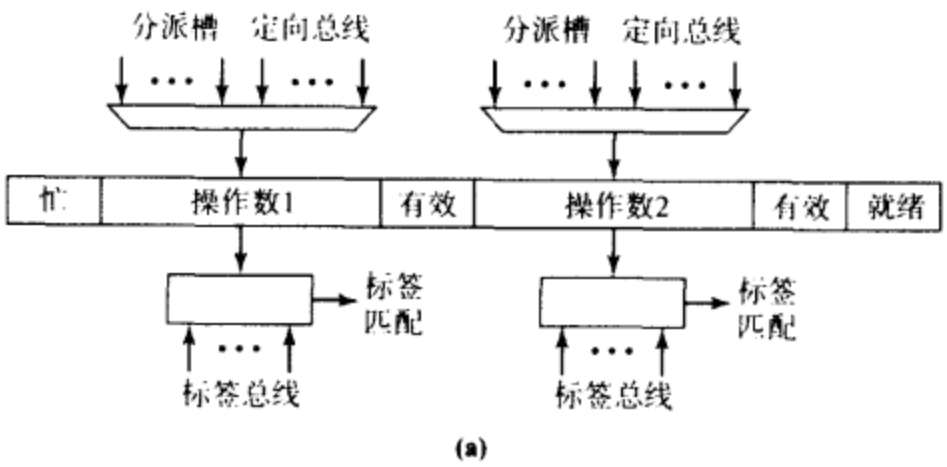
一般情况下，保留站中的记录在指令发射后就进行回收并供后续指令分派使用，从而尽可能地避免由于保留站饱和导致的停顿。而某些指令由于执行时会产生异常，将来某个周期可能要重新调度执行，所以这些保留站直到指令执行完毕且确定无异常后才进行回收。例如，load指令可能会导致数据cache失效，需要耗费多个周期处理，这时不需要停顿功能单元，而只需要等待失效处理完毕，再重新从保留站发射load指令即可。

在上述的动态执行内核中，不需要进行相关寄存器的读写操作，就可以满足生产者-消费者的关系。相关操作数直接从生产者指令定向到消费者指令，使延时达到最小。假设指令在得到其最后一个操作数的同一周期就可以发射，且没有其他指令竞争功能单元，那么它可以立即开始执行。因此，如果有足够的资源消除结构相关带来的停顿，动态执行内核将可以达到数据流极限。

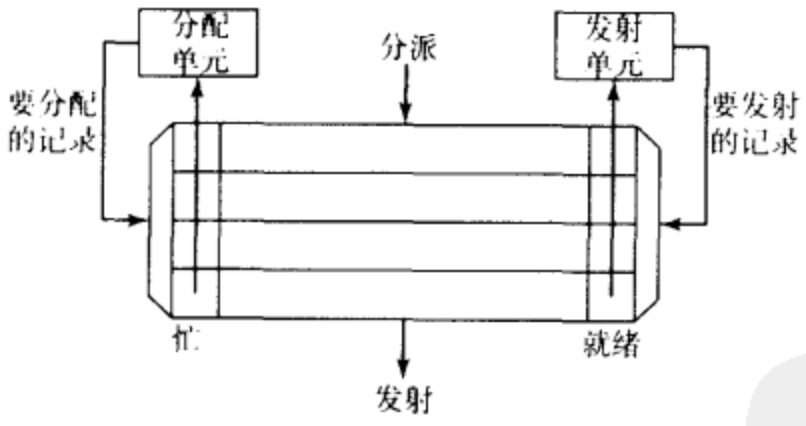
4.2.6　保留站和再定序缓冲

除了功能单元、动态执行内核的关键部件还有保留站和再定序缓冲，它们的操作限定了动态执行内核的功能。本节将讨论保留站和再定序缓冲实现的有关问题，主要阐述它们的组织结构和行为，重点是保留站和再定序缓冲记录的装载和卸载。

与保留站有关的操作有三个：分派、等待和发射。典型的保留站如图4.27（b）所示。每条记录有多个域，如图4.27（a）所示，其中一位是“忙/闲”位，指示该记录是否已被分配，还有一位“就绪”位，指示其中的指令是否可以发射。分派是指将指令从分派缓冲装载到保留站记录中，一般包含下面三步：选择空闲保留站记录、读入操作数或标签、设置该记录项为忙。分配单元根据“忙/闲”位来选择空闲记录并由分配单元完成。分配单元检测所有记录项的“忙/闲”位，选择一个空闲的记录进行分配，这可以采用一个优先级编码器实现。一旦分配了保留站记录，指令的操作数（或标签）就被读入到该记录。每条记录有两个操作数域，并且每个操作数携带一个有效位。如果操作数域中是真实数据，则将有效位置1；如果操作数中是标签，即未决的操作数，则有效位置0并等待数据定向。此外，一旦分配了记录，该记录必须置为忙。



(a)



(b)

图4.27　保留站机制：（a）保留站记录；（b）分派和发射

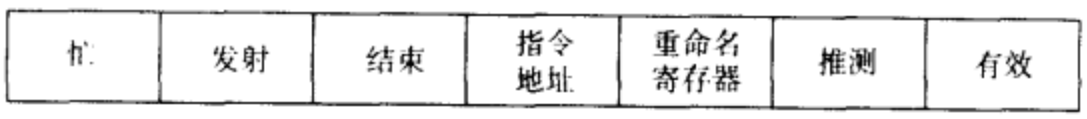
包含操作数未决的指令必须在保留站中等待，同时不间断地监听标签总线。当发生标签匹配时，操作数域将定向过来的结果锁存并将有效位置为1。指令的两个操作数都有效后，置“就绪”位为1，表示该指令可以被发射执行了，这一过程通常称为指令唤醒（instruction wake up）。

指令发射是指从保留站中选择一条就绪指令送往功能单元加以执行，也称为指令选择（instruction select）。指令是否就绪由“就绪”位指示，发射部件根据调度算法选择相应的指令，如图4.27（b）所示。调度算法可以是按程序中的指令顺序选择，也可以是按照等待时间的长短选择。一般情况下，指令发射到功能单元后，它占用的保留站记录就被回收（通过将“忙/闲”位复位）。

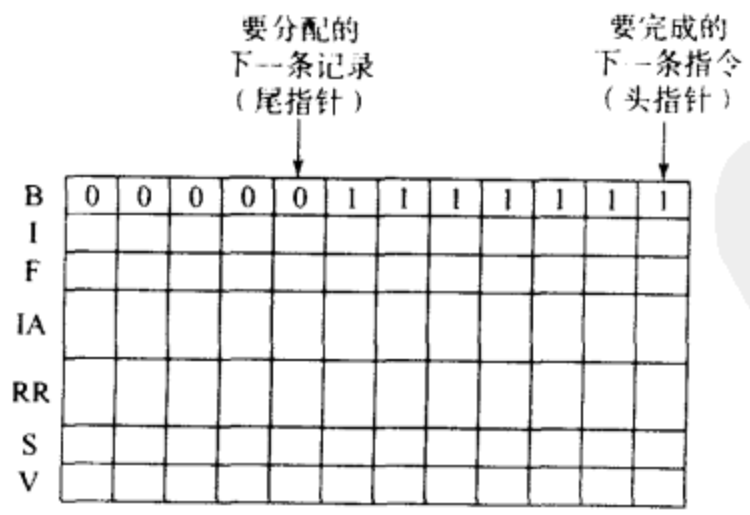
较大的保留站实现起来非常复杂。输入端的源会有多个，包括所有分派槽和定向总线，如图4.27（a）所示，因而使得数据路由网络异常复杂。指令等待操作数时，保留站的操作数域必须不间断地监听总线上的标签，这相当于在所有包含多个键值的保留站记录中检索目标。如果记录较少，则这种方法是可行的，而如果记录数目增加，复杂性则急剧增加。这部分硬件通常称为唤醒逻辑。记录数目增加的同时，还会影响到发射单元和调度算法，这部分硬件通常称为选择逻辑。在任意给定的时钟周期内，可能有多条指令就绪，选择逻辑必须选出最适合的指令发射。在超标量机器中，一个保留站支持多条指令同时发射，选择逻辑必须从所有就绪指令中选出最适当的指令集合进行发射。

再定序缓冲中保存的内容是所有已经分派但尚未完成的指令，包括保留站中等待操作数的指令、在功能单元中执行的指令和那些已经执行完毕但等待顺序提交的指令。再定序缓冲中指令的状态可以使用几位信息来标识，每条指令有下面几种状态：等待执行状态、执行状态和执行完成状态，状态位在指令从一个状态转换到另一个状态时进行更新。另外还需附加一位来指示指令是否为推测指令（在预测路径上）。如果推测可以跨越多条分支，还需附加几位指示该指令属于哪一个推测基本块。分支执行完毕后，推测指令变为正确指令（分支预测成功）或失效（分支预测失败），只有执行完毕的正确指令才能被提交，标记为无效的指令不进行提交。图4.28（a）显示了再定序缓冲记录中的各个域，同时包括了重命名寄存器域。

再定序缓冲组织成一个循环队列并包含头指针和尾指针，如图4.28（b）所示。在指令分派时，如果分配了一个再定序缓冲记录，则尾指针递增，每周期可以分配的记录数目取决于指令分派的宽度。指令从队列的头部提交，每次提交的指令数量取决于可以提交的指令数目以及提交带宽，而提交带宽又取决于另外一个路由网络的容量和寄存器写回可用的端口数。最为关键的是体系结构寄存器文件写端口的数目，这些端口从重命名寄存器（或者是再定序缓冲，当它们被当做重命名寄存器的时候）向体系结构寄存器传递数据。指令完成提交后，重命名寄存器和再定序缓冲记录均被释放，再定序缓冲的头指针也进行相应的更新。在某种程度上，再定序缓冲可以看做是动态执行内核的控制中心，因为所有正在执行的指令状态都在再定序缓冲中存在相应的记录。



(a)



(b)

图4.28 （a）再定序缓冲记录；（b）再定序缓冲结构

可以将保留站和再定序缓冲合为一体来管理所有正在执行的指令，称为指令窗口（instruction window）。由于在分派时必须同时为指令分配一条保留站记录和一条再定序缓冲记录，因此可以将这两条

记录合并为指令窗口的一条记录，即指令分派到指令窗口。指令窗口的记录监听标签总线并接收定向的操作数，操作数都就绪后，指令从指令窗口发射到功能单元执行，最终也在指令窗口中完成提交。指令窗口的大小决定了机器中可以有多少条指令在同时执行，同时也决定了机器所能达到的指令级并行程度。

4.2.7　动态指令调度器

动态指令调度器是动态执行内核的核心所在，包括指令窗口和相关的指令唤醒、选择逻辑。当前动态指令调度器有两种设计方案比较流行：数据捕获和无数据捕获（data capture）。

图 4.29（a）所示为一个带数据捕获的调度器。当分派指令时，已经就绪的操作数从寄存器文件（体系结构寄存器或物理寄存器）读入指令窗口，即数据捕获；对于还未就绪的操作数，则将标签写入指令窗口，在数据从功能单元定向过来时，根据标签锁存操作数。结果都是通过定向的方式送往指令窗口中等待数据的指令。数据定向和指令唤醒是以Tomasulo算法的方式结合在一起的。另外还需一条定向路径来更新寄存器文件，使得后继的相关指令可以从寄存器文件获取源操作数。

最近有些微处理器采用另外一种调度器设计策略，即不带数据捕获，如图 4.29（b）所示。寄存器的读操作是在调度器之后进行，也就是指令发射到功能单元的时候。指令分派时，不将操作数写入指令窗口，而只将操作数标签（指针）载入指令窗口，然后调度器进行标签匹配并唤醒指令。不过，功能部件的结果只是定向到寄存器文件，所有就绪指令发射时将从寄存器文件直接读取操作数，然后开始执行。数据定向和指令唤醒是分开的，指令唤醒通过将标签定向到调度器来实现。在无数据捕获的调度器中，指令窗口尺寸可以明显减小并且数据定向路径的宽度也可以减小。

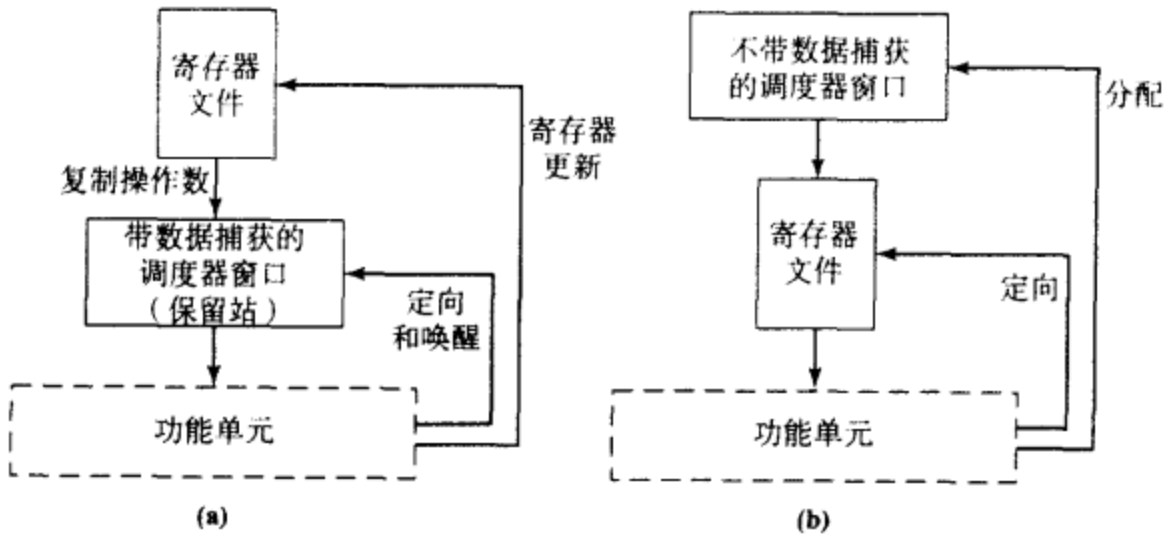


图 4.29　动态指令调度器设计：（a）带数据捕获；（b）无数据捕获

寄存器重命名和指令调度之间存在着密切的关系。如前所述，动态寄存器重命名的一个目的就是消除寄存器重用引起的假相关，另一个目的是建立两条相关指令之间的生产者-消费者关系。数据真相关由生产者和消费者指令中相同的重命名寄存器标识符来确定，重命名寄存器标识符在数据定向时还作为标签来使用。在无数据捕获的调度器中，寄存器标识符用来访问寄存器文件、检索源操作数；目的寄存器标识符则用做标签唤醒的相关指令。在带数据捕获的调度器中，数据定向和指令唤醒使用的标签不是真正的寄存器标识符，标签主要用来确定相关指令间的生产者-消费者关系并可以随意赋值。比如，Tomasulo算法中使用保留站ID作为标签进行相关指令的结果定向以及体系结构寄存器的更新。另外，显式的寄存器重命名不包括物理寄存器。

4.2.8　其他寄存器数据流技术

许多年来，数据流极限被看做是一个绝对的理论极限并体现了终极性能目标。30 多年来，对于数据流体系结构和数据流机进行了大量的研究工作，数据流极限假设数据真相关是绝对的、不可

逾越的。有趣的是，在20世纪60年代末期至70年代早期，对于控制流机器也进行了相同的假设，即控制相关是绝对的，当遇到条件分支时，由于对真实路径的不确定，执行下一条指令之前除了等待分支执行完以外没有其他的选择。但从那时起，分支预测领域的技术取得了巨大的进步，条件分支和相应的控制相关不再是不可逾越的障碍，相反可以通过对分支方向和分支目的地址的推测技术加以克服。推测技术的基础是分支指令的结果具有很强的可预测性。直到1995年，研究人员才开始对数据真相关的绝对性提出了质疑。

在1996年，有几篇论文相继提出了值预测（value prediction）的概念。第一篇论文是由Lipasti，Wilkerson和Shen撰写，着重讨论了载入数据的预测。研究发现，同一条静态load指令载入的值往往是可预测的。第二篇论文基于同样的思想，着重于ALU指令结果的预测。研究表明，许多指令的计算结果也是可预测的。值局部性（value locality）的概念是指某些指令往往重复地计算出相同的一组（一个）结果。通过追踪这些指令的计算结果，未来的计算值可以根据历史信息进行预测。这些研究使得近些年来发表了大量的论文并对值预测器的设计进行了探讨。最近的一项研究表明，一种混合值预测器可以达到80%以上的预测精度，而结合了值预测的理想设计方案可以使IPC在SPEC测试下提高8.6%到23%。

通过在取指段进行值预测，如果指令的结果预测准确，后继的相关指令可以使用该推测值开始执行，而不必等待前导指令的译码和执行操作，因此有效地消除了两条指令间数据真相关的串行化限制。从某种意义上说，如果预测准确，数据流图中的相关边就被有效地删除了，因此值预测提供了超越数据流极限的潜力。当然在流水线的后端还要进行的预测验证工作将成为指令执行吞吐率新的限制。如果预测失败率较小且失败开销也很小（0周期或者1周期），且验证延时比平均执行延时小，值预测就可以有效地提高机器性能。显然，值预测高效的实现方案是保证它能有效提高性能的关键。

另一种最近经常提及的技术思想是动态指令重用（dynamic instruction reuse）。同值局部性相似，对真实程序的研究表明，同一指令序列在执行时往往使用相同的一组输入数据，因此机器经常进行重复的工作。动态指令重用技术试图追踪冗余计算，当探测到冗余计算时，先前的计算结果直接被使用而不进行计算操作。这种技术不是推测性的，而无需进行验证工作。值预测技术可以看做是消除数据流图中的某些相关边，而动态指令重用技术则是从数据流图中删除包括节点和边在内的子图。较早的一项研究表明，当冗余计算消除技术用于功能性语言书写的程序中时，将可以获得很大的性能提高，最新的研究也证明了这一点。这一领域的研究当前比较热门，近期有可能会出现一些突破性的研究成果。

在第8章中将更加详细地讨论这些先进的寄存器数据流技术。

4.3 存储器数据流技术

存储器指令负责在存储器和寄存器文件之间传递数据，从而支持ALU指令的执行。ALU指令的寄存器操作数必须首先从存储器载入进来才能使用。寄存器的数目是有限的，程序执行中用到的操作数不可能都保存在寄存器文件中，因此，编译器设置换出代码（spill code）并把某些操作数临时地保存到存储器中，需要时再将其载入，换出代码使用store和load指令实现。通常，编译器只是将标量数据放置在寄存器中，对于数据和链表等复杂的数据结构，由于它们占用的空间较大，从而超出了寄存器文件的容量，因此需要将其放置在存储器中。对这些数据结构进行操作时，首先要执行一批load和store指令。因此，对load/store指令进行有效地处理可以减少数据在存储器和寄存器间移动的开销。

由于存储器指令的执行延时一般很长，load/store指令的处理以及冗余存储器数据流则变成了整个机器性能的瓶颈。内存地址的计算和内存访问增加了load/store指令的执行延时，同时，为支

持虚存，虚拟地址需要转换为物理地址才能进行存储器访问，这也增加了访存延时。cache技术可以有效地降低访存延时，此外，也已经开发了各种各样的其他技术来克服总的访存延时，从而提高load/store指令的存吐率。

4.3.1　存储器访问指令

存储器数据流指令的执行有以下三个步骤：生成存储器地址、存储器地址转换和数据访问。首先将阐述这三步的操作，然后讨论load/store指令在超标量流水线中的处理。

寄存器文件和存储器都是指令集体系结构定义存储数据的结构。在ISA中，存储器定义成随机访问、容量为2^n的存储集合，即每个存储器位置都是由一个n位的地址加以访问且访问延时相同。同体系结构寄存器文件一样，存储器是软件指令可见的；与寄存器文件不同的是，存储器的地址往往不作为指令格式的一部分显式保存，而是由指令指定一个寄存器和一个偏移量并通过计算生成，因此地址生成包括寄存器访问以及偏移量的计算。

对于实现了虚存的系统，除了地址生成，还需要进行地址转换操作。体系结构存储器组成了程序的虚存空间，每个程序都将其看做私有地址空间。机器中实现的物理存储器组成了物理地址空间，物理地址空间比虚拟地址空间要小，而且要被多个程序共享。虚拟存储器是一种将程序的虚拟地址空间映射到机器物理地址空间的机制。在这种机制下，虚拟存储器能够支持程序在比物理地址空间大得多的虚拟地址空间下运行，并且通过将多个虚拟地址空间映射到同一个物理地址空间来支持多程序执行。映射机制就是关于如何将有效地址（虚拟地址）转换成可以访问物理存储器的物理地址。映射机制通常用映射表实现，地址转换就是进行查表操作。

第三步是存储器访问。对于load指令，可以将数据从存储器某个位置读出，然后写入寄存器；对于store指令，可以将寄存器的值写入存储器某一位置。对于load和store指令，前两步操作，即地址生成和地址转换是相同的，而第三步则由超标量流水线执行，其方式有所不同。

在图4.30中，这三步操作由流水线的三个段完成。第一段执行有效地址的生成。假设load和store指令都采用寄存器间接寻址加一个偏移量的模式。对于load指令，地址寄存器操作数一旦就绪，它就发射到实现了流水化的功能单元中执行并在第一段生成有效地址。对于store指令，则必须等待，直到地址寄存器和数据寄存器操作数就绪后才能发射。

第一段生成有效地址之后，第二段将虚拟地址转换成为物理地址，一般通过访问快表TLB（Translation Lookaside Buffer）来完成。TLB是硬件控制的表结构，保存着虚拟地址到物理地址的映射。TLB本质上是存储器中页表的cache（第4.3.2节中将对TLB和页表进行更多的介绍）。如果当前被转换的虚拟地址所在页的映射情况不在TLB中，即发生了TLB失效。如果该映射在页表中，则可以通过访问存储器中的页表获取该映射，一旦失效的映射被载入TLB，转换过程也就结束了。还有可能映射甚至不在页表中，即相关的页还没有进行映射，也不在存储器中，这种情况将导致页故障（page fault）并需要访问磁盘存储器检索失效的页。页故障将触发程序异常，同时需要挂起当前程序。

在流水线第二段成功地完成了地址转换后，load指令将在第三段开始访问数据存储器。在该周期的末尾，数据从数据存储器中检索出来并写入重命名寄存器或者再定序缓冲，到此load指令执行完成。体系结构寄存器的更新直到load指令从再定序缓冲完成后再进行。这里，假设数据存储器在第三段的访问时间为一个周期，如果使用数据cache，这种假设是合理的（在4.3.2节中对cache进行了详细的描述）。但是使用数据cache时，要载入的数据可能不在数据cache中，即数据cache失效，需要从存储器中将数据调入cache。cache失效时必须停顿load/store流水线。

store指令的处理在某些方面和load指令不同。store指令在流水线第二段进行了地址转换后就认为已经完成了执行，要存储的寄存器数据保存在再定序缓冲中，store指令提交时再将数据写入存

储器。“延迟写”的目的就是防止存储器被错误地更新，因为store指令可能由于发生异常或分支预测失败等原因而失效。load指令只是读存储器，其失效不会对存储器状态产生不必要的影响。

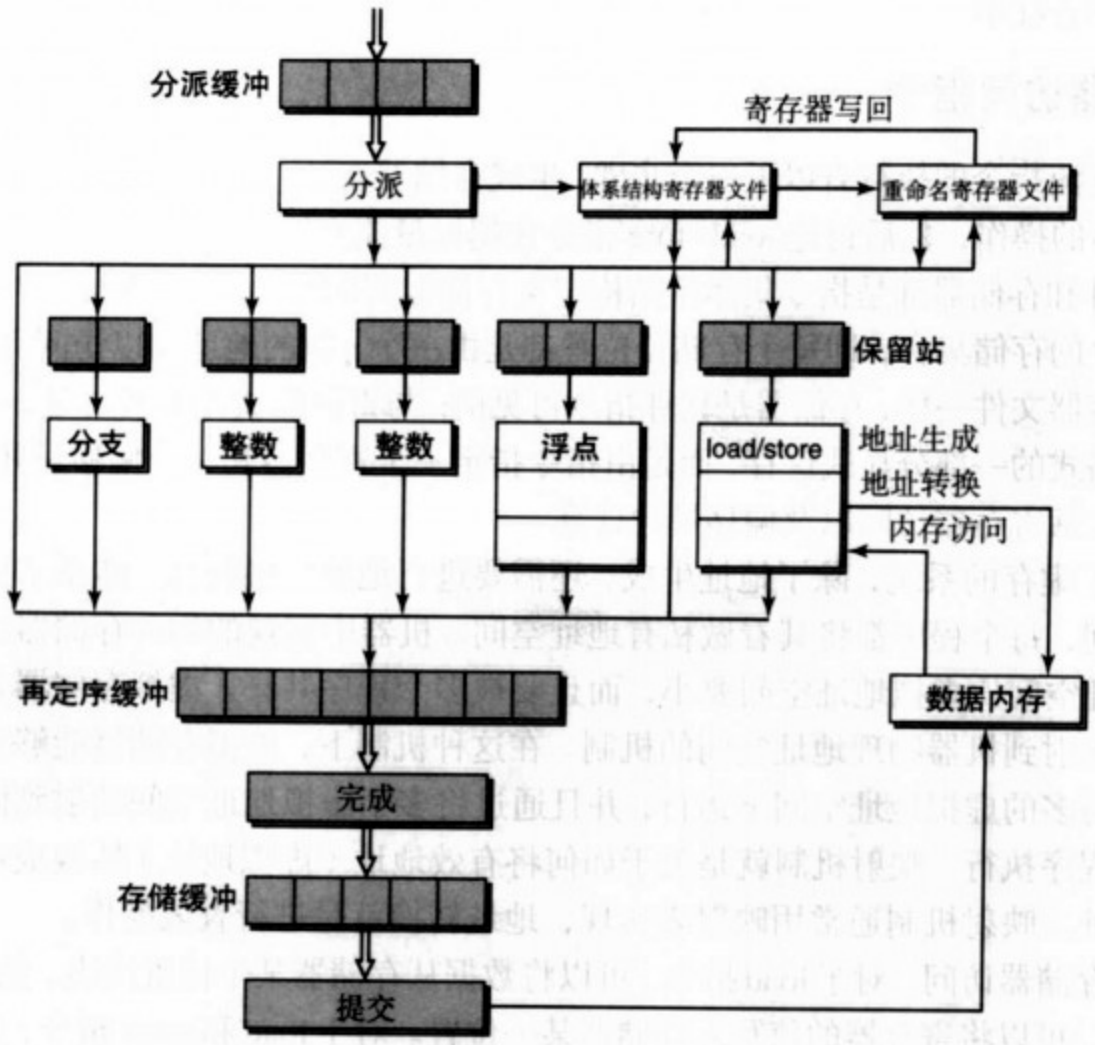


图 4.30 load/store 指令的处理

store指令一般不在完成时更新存储器，而是将数据保存到存储缓冲中。存储缓冲是一个FIFO缓冲，保存已经完成了的store指令，每条store指令在存储总线空闲时进行存储器的更新。存储缓冲的作用就是使store在存储器不忙时提交，从而优先load指令的操作。这里“完成”是指CPU状态的更新，而“提交”是指存储器状态的更新。由于使用了存储缓冲，store可以已经完成但尚未提交。当程序发生异常时，异常指令的后续指令有可能由于乱序执行而提前执行结束，因此在异常程序挂起之前必须从再定序缓冲中使这些指令失效，同时排空存储缓冲，即提交存储缓冲中的store指令。

假设地址转换和存储器访问都可以在一个周期内完成，这一般需要TLB和cache的支持。下一节将对cache存储系统和虚存系统的实现进行简要的回顾，熟悉存储器层次结构的读者可以跳过这一节。

4.3.2 存储器层次结构

在讨论特定的存储器数据流技术之前，我们先来从实现的角度回顾一下cache系统和虚存系统的基本知识。本小节主要介绍4个主题：存储器访问机制、cache实现、TLB实现和cache与TLB的交互。

访问多端口存储器一般有两种基本方法：通过地址索引和通过标签相联查找。如图4.31（a）显示了地址索引存储器，通过译码器将n位地址进行译码并选择2^n项中的一项，进行读写操作。地址与数据之间的映射是固定的，虽不灵活但实现简单。相反，相联存储器通过一个键值（key）查找存储器并选择某一项，如图4.31（b）所示。存储器的每一项都有一个标签域和一个比较器，从而比较标签内容和键值，发生匹配时，则选中该项。使用这种相联查找方式可以灵活地将数据保存

在存储器的任一位置，但是实现起来较为复杂。地址索引存储器和相联查找存储器的一个折中方案是组相联存储器，即使用索引又使用相联查找，如图4.31（c）所示。地址用来索引一组记录项，而每组中的多条记录则通过关键值查找定位。这种数据放置方式具有一定的灵活性，而又不至于像全相联存储器那么复杂。

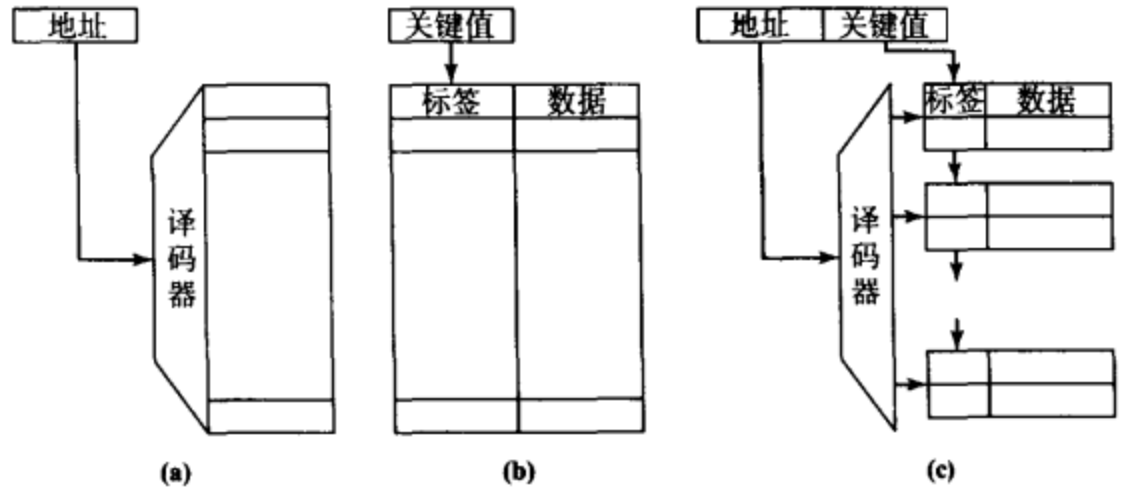


图4.31　存储器访问机制：（a）地址索引存储器；（b）（全）相联存储器；（c）组相联存储器

存储器层次组织结构是为了使整个存储子系统达到高容量、低延时的目标。较低一级的存储器一般容量大、速度慢，而高一级的存储器则容量较小、速度快。利用存储器访问的局部性，层次存储结构是一种在不增加成本和复杂性的前提下，可以提供与大容量、高速度存储器性能相当的一种方法。存储器属于较低一级的存储器，cache系统则属于较高一级的存储器。

存储器一般实现为大容量的地址索引存储器，cache则可以实现为图4.31所示三种方案的任意一种。若cache用索引存储器实现，则称其为直接映射cache（参见图4.32）。由于直接映射cache相对于存储器容量较小，因而需要的地址位数较少，译码器也较小，所以只需译码存储器地址的少数几位。因此，许多存储器地址将会映射到直接映射cache的同一地址。为保证选中的项包含正确的数据，剩余的（没有参加译码的）地址位用来确认选中的项。因此，除了数据域以外，每项还有一个标签域，用于存储没有参加译码的地址位。当cache的一项被选中后，读出它的标签域并同访问地址的剩余位进行匹配，从而确定该记录项是否包含着要访问的正确数据。

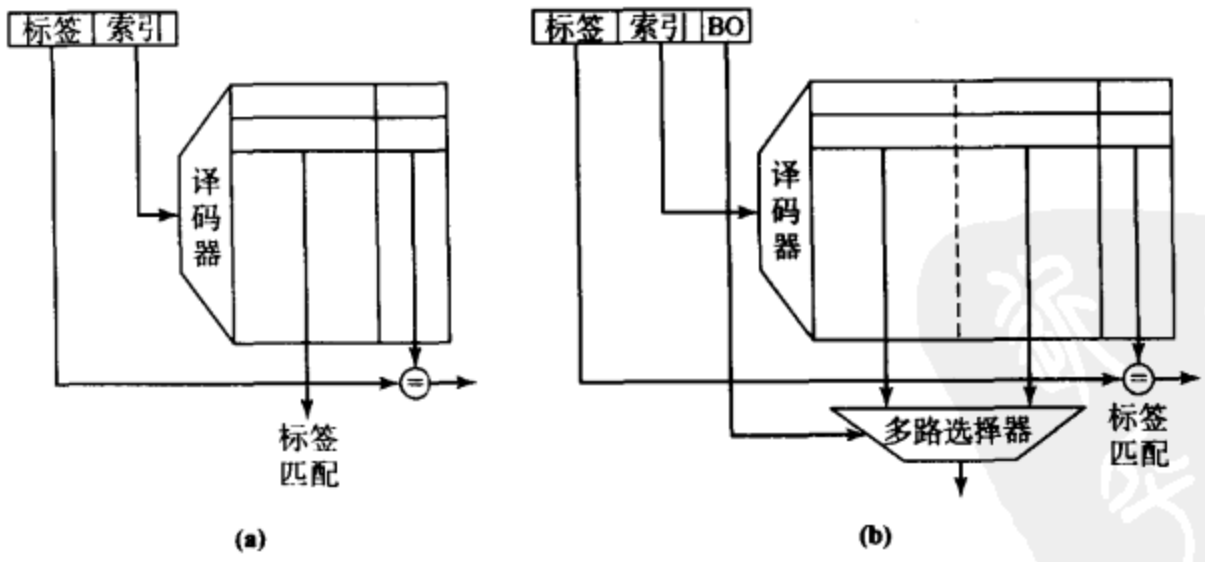


图4.32　直接映射cache：（a）每块单个字；（b）每块多个字

图4.32（a）所示为一个直接映射cache，每项（或块）为一个字。为了利用空间局部性，cache的块大小可以包含多个字，如图4.32（b）所示。若cache块包含多个字，地址的某几位将用来选择命中项中的特定字。因此，访存地址分为三部分：用来选择某一项的索引位、用来选择块内某字的块内位移和用来进行标签匹配的标签位。

cache 也可以用全相联或组相联存储器来实现，如图 4.33 和图 4.34 所示。全相联 cache 在数据放置上具有最大的灵活性，除了块内位移外，访存地址的其他位都用做相联查找的标签域。全相联 cache 最高效地利用了 cache 的所有项，但是实现复杂。组相联 cache 体现了组内部放置数据的灵活性。索引位选中某一组，然后标签位再选中组内某项，最后由块内位移选择块内的某字。访存地址三部分的大小在划分时要经过仔细考虑。

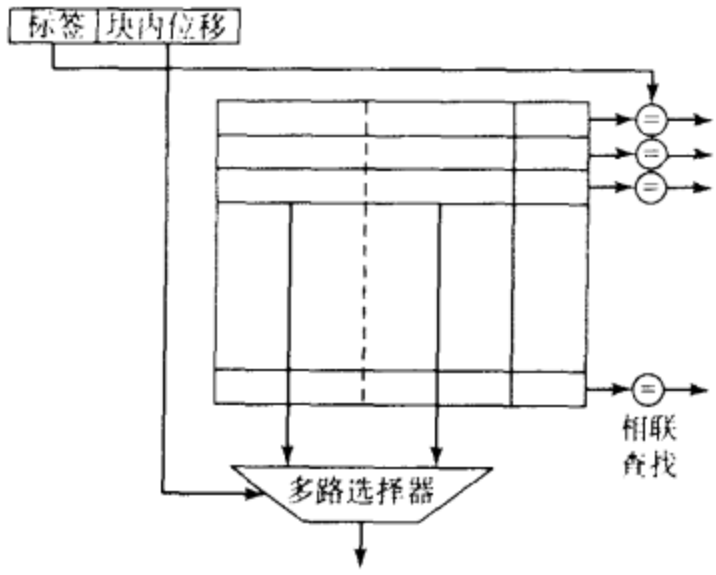


图 4.33　全相联 cache

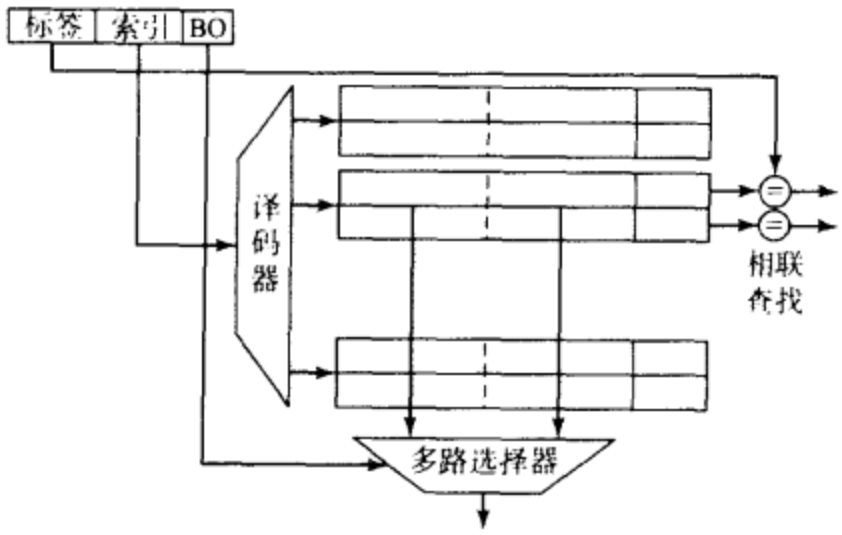


图 4.34　组相联 cache

虚存系统最本质的特征就是虚拟地址空间到物理地址空间的映射，即将虚拟地址转换为物理地址，处理器不能直接使用其生成的虚拟地址进行存储器访问，而是要首先将其转换为物理地址，再使用物理地址访问物理存储器，如图 4.35 所示。

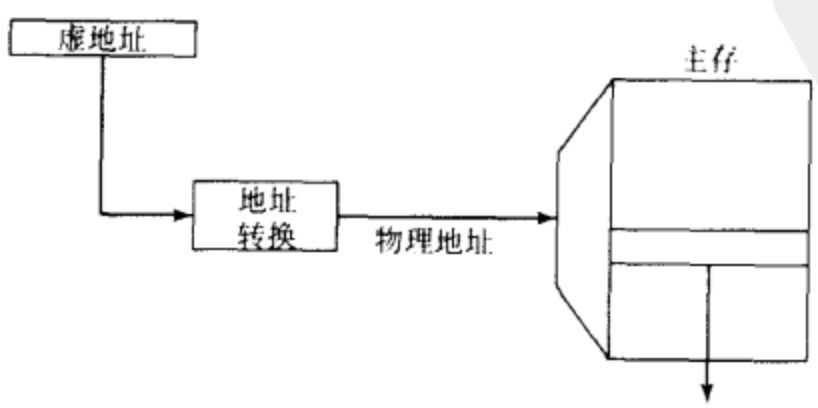


图 4.35　虚地址到物理地址的转换

地址转换可以使用转换存储器完成。用虚拟地址索引转换存储器，检索出的数据就可以作为物理地址进行存储器访问，即对应于虚拟地址的物理地址存放在转换存储器相应的记录中。图4.36显示了使用转换存储器进行字地址转换，即将虚拟地址空间中字的虚拟地址映射到物理地址空间中的字物理地址。

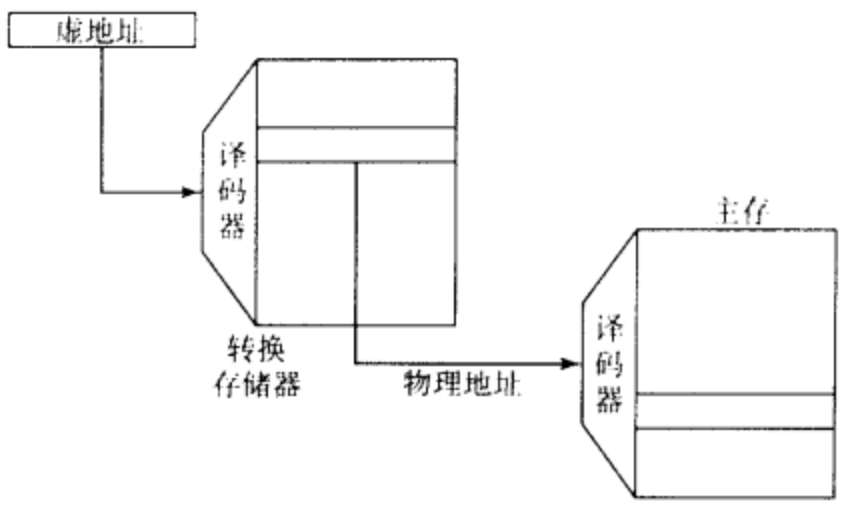


图4.36　使用转换存储器进行字虚地址到字物理地址的转换

图4.36所示的转换存储器存在两个缺点。首先，字地址的转换需要转换存储器与存储器的记录数目相同，这将使物理存储器的大小加倍。转换可以在更大的粒度上进行。存储器中的多个字（一般是2的幂次方）可以组合成页（page），只有页的地址需要转换。在页的内部，字可以直接使用虚拟地址的低位（即页内偏移）进行选择，而无需转换。如图4.37所示，在使用页的系统中，转换存储器称为页表（page table）。

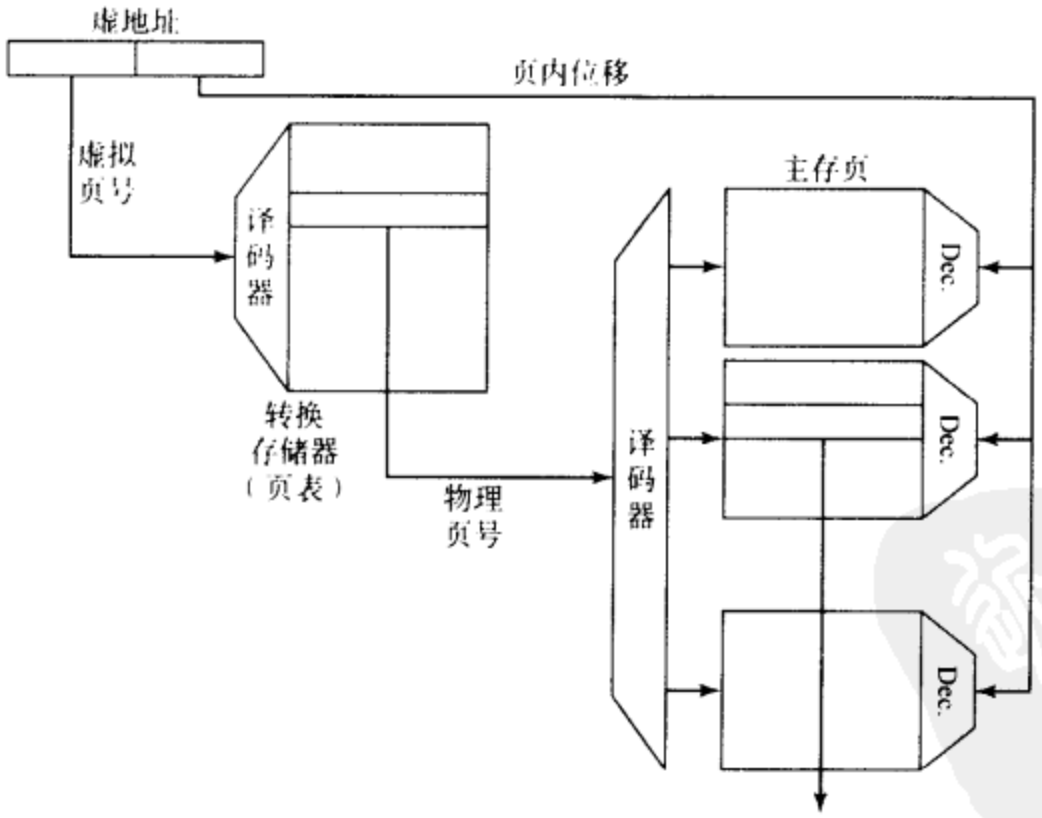


图4.37　使用转换存储器进行页虚拟地址到页物理地址的转换

第二个缺点是每条访存指令都需要进行两次访存操作，第一次访存是访问页表获取物理页号，第二次访存是使用转换后的物理地址（物理页号加上页内偏移）访问物理存储器。在实际的实现过程中，页表一般都存放在存储器中（由操作系统分配地址），因此每条访存指令都需进行两次物理存储器的顺序访问，这将成为严重的性能瓶颈，解决方法是使用快速cache存储器存放页表。

快表（TLB）就是存储页表的快速cache存储器。和其他cache系统一样，TLB也有图4.31所示的三种访问策略。直接映射TLB容量小、速度快，如图4.38所示，虚拟页号划分为TLB索引和标签两部分。虚拟页号转换为物理页号，然后和页内偏移连接在一起就构成了物理地址。

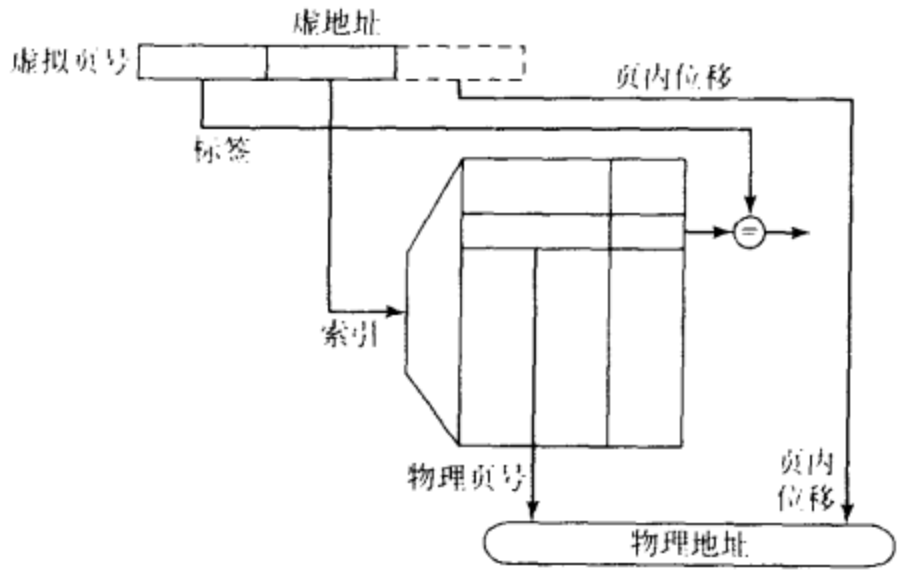


图4.38　直接映射TLB

为保证TLB使用的灵活性和效率，TLB也可以使用相联存储器实现。图4.39所示为组相联和全相联的TLB。对于组相联TLB，虚拟地址划分为三个域：索引、标签和页内位移。页内位移的大小由页大小决定，而页大小则由体系结构和操作系统决定。剩余的地址位，即索引和标签则组成虚拟页号。在全相联TLB中没有索引位，标签位将组成虚拟页号。

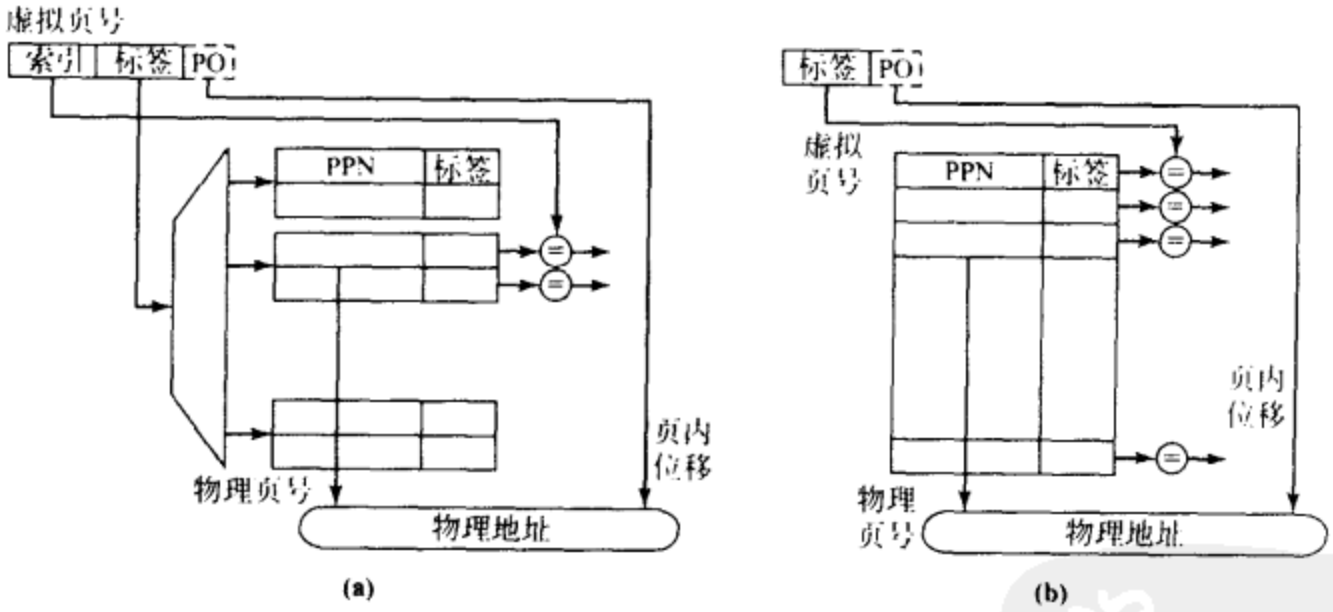


图4.39　相联TLB：（a）组相联TLB；（b）全相联TLB

将一部分页表缓存在TLB中可以加速地址转换，但也有可能发生TLB失效，因为并不是所有的虚拟页号－物理页号映射可以同时存储在TLB中。访问TLB时，如果失效则必须从存储器的页表中将映射关系调入到TLB中，这将导致流水线停顿多个时钟。TLB失效还有可能导致页故障，即虚拟页号－物理页号的映射也不在页表中。发生页故障意味着该页不在存储器中，必须从辅助存储器中读取。处理页故障需要进行系统调用并访问磁盘存储器，这或许需要几百个、几千个时钟周期。因此，程序发生页故障时，一般从执行状态进入挂起状态，直到操作系统将页故障处理完毕。

数据cache用来缓冲存储器中的部分数据，TLB用来缓冲页表部分中的数据，数据cache和TLB之间的交互如图4.40所示。结合图4.30的load/store部件流水线，图4.40中的n位虚拟地址即第一

段生成的有效地址。虚拟地址包括虚拟页号（v位）和页内偏移（g位）。如果TLB是组相联cache，v位的页号则再被分为k位索引和（$v-k$）位标签。load/store部件流水线的第二段对应于使用虚拟页号进行TLB访问。假设没有TLB失效，就可以读出物理页号（p位）并与g位的页内偏移合并生成m位的物理地址，即$m=p+g$（m不必和n相等）。在流水线第三段，m位的物理地址用来访问数据cache。m位物理地址的解析取决于具体数据cache的设计，如果数据cache块包含多个字，则低b位为块内位移，它用来选择块中的一个字；块的命中由剩余的（$m-b$）位地址决定。如果数据cache是组相联的，则剩余的（$m-b$）位地址又分为t位的标签和i位的索引，i的值由cache总容量和相联度决定，即共有2^i组。如果没有cache失效，在第三段的末尾（假设数据cache访问周期为1）就可以得到读出的数据（假设是在执行load指令）。

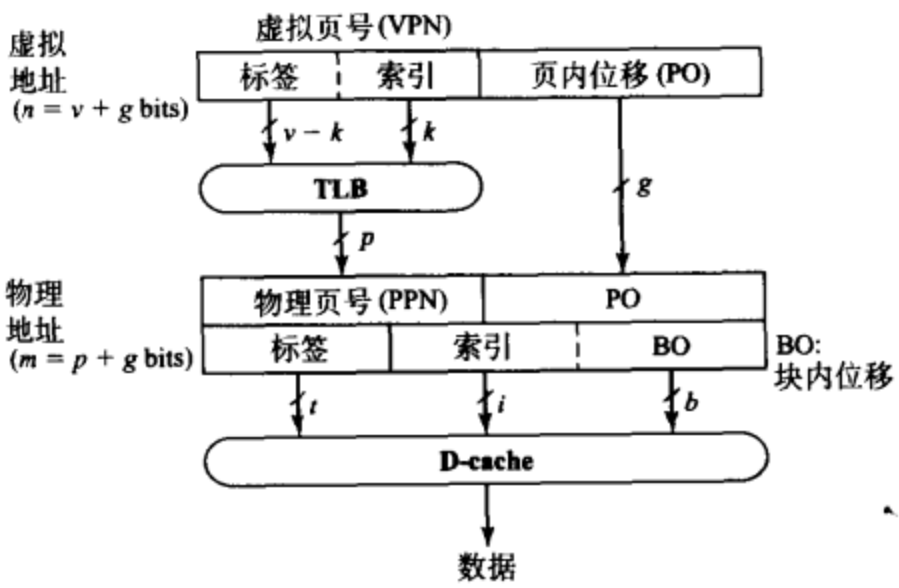


图 4.40　TLB 和数据 cache 之间的交互

图4.40所示的组织结构存在一个缺点，即TLB的访问必须在访问数据cache之前进行。TLB和数据cache访问的串行化导致总的延时是两次访问的和，这就是为什么在图4.30中将地址转换和存储器访问放在两个流水段的原因。此问题的一种解决方法就是使用虚拟索引访问数据cache，即允许TLB的访问和数据cache的访问并行进行，如图4.41所示。

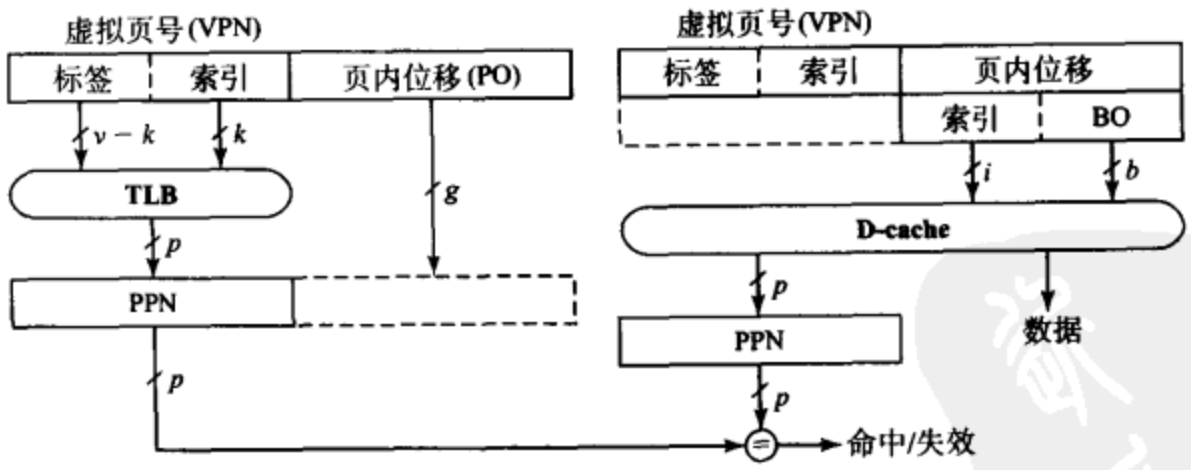


图 4.41　虚拟索引数据 cache

虚拟索引访问数据cache最直接的实现方法就是仅使用页内偏移位访问数据cache。既然页内偏移不需要进行地址转换，所以可以直接使用。g位的页内位移可以用做访问数据cache的块内位移（b位）和索引（i位）。为简单起见，假设数据cache是直接映射，容量为2^i，块大小为2^b个字。每项的标签域不保存虚拟页号，而是包含了转换以后的物理页号并在数据cache的块填充时进行保存。在页内偏移访问数据cache的同时，其余的虚拟地址位（虚拟页号）进行TLB的访问。假设TLB

访问时间和数据 cache 的访问时间是相当的，在物理页号从 TLB 中读出的时候，数据 cache 的标签（也包含物理页号）也读取了出来。可以将两个 p 位的物理页号进行比较，从而确定数据 cache 是否命中（页号匹配）。通过虚拟索引访问，可以实现地址转换和数据 cache 访问并行操作，从而降低了执行延时。

4.3.3 存储器访问的排序

如果两条 load/store 指令指向同一个内存地址，则它们之间存在存储器数据相关，即存在别名（aliasing）或冲突。load 指令进行存储器的读操作，store 指令进行存储器的写操作，同寄存器相关一样，load 指令和 store 指令之间存在着读后写（RAW）、写后读（WAR）和写后写（WAW）相关。store（load）指令后的一条访问同一存储器地址的 load（store）指令，将导致 RAW（WAR）存储器数据相关。两条对同一地址访问的 store 指令导致 WAW 相关。为保证程序语义的正确性，这些存储器数据相关必须保持。

保持存储器数据相关最直接的方法就是按程序的顺序执行 load/store 指令。完全的指令顺序执行固然足以保持存储器数据相关，但不是必要的，保守的策略会给程序的执行性能带来限制。如图 4.42 所示，DAXPY 是此代码段的名称，它将一个数组乘以一个系数，然后加上另一个数组。DAXPY（Double Precision A Times X plus Y）是 LINPAC 程序的核心，它在数值计算程序中很常见。注意，此循环的所有循环迭代之间是数据不相关的，因此可以并行执行。然而，如果强制所有 load/store 指令必须按顺序执行，那么对于第二次循环的第一条 load 指令，将会直到第一次循环的 store 指令执行完毕后才能开始执行。这一限制使该循环的所有迭代必须串行化执行。

```
Y(i)= A * X(i) + Y(i)

    F0 ← LD,a
    R4 ← ADDI,Rx,#512      ;last address

Loop:
    F2 ← LD,0(Rx)          ; load X(i)
    F2 ← MULTD,F0,F2       ; A*X(i)
    F4 ← LD,0(Ry)          ; load Y(i)
    F4 ← ADDD,F2,F4        ; A*X(i) + Y(i)
    0(Ry) ← SD,F4          ; store into Y(i)
    Rx ← ADDI,Rx,#8        ; inc. index to X
    Ry ← ADDI,Ry,#8        ; inc. index to Y
    R20 ← SUB,R4,Rx        ; compute bound
    BNZ,R20,Loop           ; check if done
```

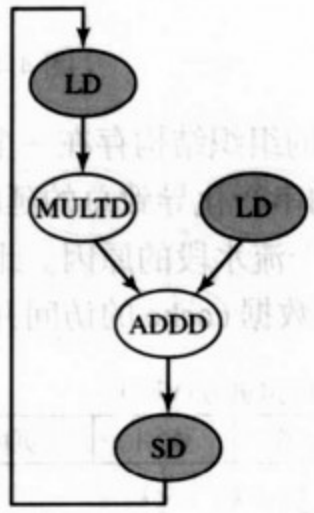


图 4.42 DAXPY 示例

如果允许load/store指令在不破坏存储器数据相关的前提下乱序执行，就可以获得性能的提高。还是以DAXPY循环为例，图4.42中描述了该循环体中核心指令的真实数据相关，这些数据相关都是存在于同一循环迭代的指令之间的，在多个循环迭代之间不存在数据相关。和循环相关的分支指令预测准确度很高，因此，下一循环迭代指令的读取过程很快。在后续循环迭代中，相同指令的体系结构寄存器通过寄存器重命名机制进行了动态重命名，因此，多个循环迭代之间不会因为寄存器的动态重用而导致寄存器相关。可见，如果 load/store 指令允许乱序执行，即后续循环迭代的 load 指令可以在前一循环迭代的 store 指令执行前开始执行。通过多个循环迭代的重叠执行，循环的执行性能可以获得极大提高。

存储器模型对load/store指令的乱序执行附加了一定的限制条件。首先，为了方便异常恢复，必须顺序地保存存储器的状态，即存储器的状态必须按照load/store指令顺序执行的状态进行记录。其次，大多数共享内存的多处理器系统使用顺序一致性存储模型，要求每个处理器对共享内存的访问

按照程序的顺序完成。这两个限制条件都要求 store 指令按照程序的顺序执行，至少对存储器的更新是按照程序的顺序进行的。如果要求 store 指令按程序的顺序执行，WAW 和 WAR 存储器的数据相关都被隐式地保持了，因此可以忽略该问题。因此，只有 RAW 存储器的数据相关需要考虑。

4.3.4　载入旁路和载入定向

将 load 指令乱序执行是提高性能的一种主要措施。在 DAXPY 一例中可以看到，load 指令一般位于相关链的开始处，提前执行 load 指令可以方便其他相关指令的提前执行。对于存储器数据相关性来说，load 指令执行的是对存储器的读操作以及对目的寄存器的写操作，因此，load 指令是寄存器定义（DEF）指令，而后续指令一般为与其相关的寄存器使用（USE）指令。我们的目标就是，不破坏存储器数据的 RAW 相关，使存储器能够按照内存顺序一致性模型进行更新，尽可能早地执行 load 指令，甚至可以将其提前到前导 store 指令之前。

有两种乱序执行 load 指令的技术：载入旁路和载入定向。如图 4.43（a）所示，载入旁路是指当载入地址和前导 store 指令不存在别名，即 load 和 store 之间不存在存储器数据相关时，将在后的 load 指令置于前导 store 指令之前执行。如果 load 和 store 存在别名，即存在 RAW 相关，就使用载入定向将 store 的数据直接传递给 load 指令，load 无需再访问存储器，如图 4.43（b）所示。两种情况都实现了 load 指令的提前执行。

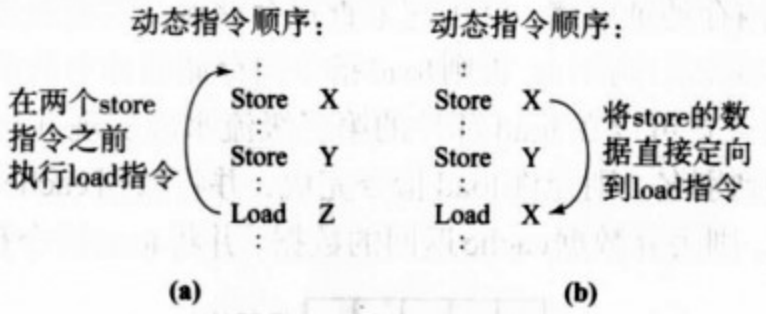


图 4.43　load 指令的提前执行：（a）载入旁路；（b）载入定向

在讨论实现载入旁路和载入定向需要解决的问题之前，首先介绍一下执行内核中负责load/store 指令处理的那部分结构。如图 4.44 所示，下面对载入旁路和载入定向的讨论都将基于该系统结构，其中有 1 个 store 部件（2 级流水）和 1 个 load 部件（3 级流水）共享一个保留站，并假设 load 和 store 指令是从该共享保留站中顺序发射的。store 部件还设有一个存储缓冲，load 部件和 store 部件以及存储缓冲都能访问数据 cache。

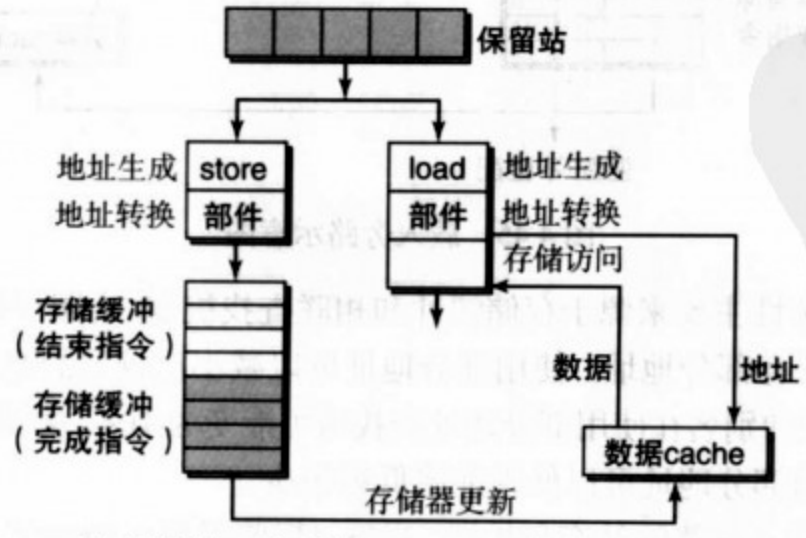


图 4.44　load/store 处理机制：独立的 load、store 部件共享一个保留站并顺序流出

在图 4.44 所示的结构中，store 指令在执行时可能处于几种状态。当 store 指令分派到保留站时，将为其在再定序缓冲中分配一个记录。指令一直保存在保留站中，直到所有操作数就绪，然后发射

到执行内核加以执行。一旦内存地址生成并完成转换，就认为store指令已经执行结束并将其放入存储缓冲的结束指令部分（同时更新再定序缓冲）。存储缓冲是一个队列结构，由两部分组成：结束指令（finished）和完成指令（completed）。结束指令是已经执行结束但尚未更新体系结构状态的指令；完成指令是已经更新完体系结构状态、等待更新存储器的指令。两部分的定义可以通过写缓冲的队列指针或给写缓冲的每一项增加状态位来实现。结束指令有可能是推测指令（speculative），若出现推测错误，则需要从存储缓冲中清除相应指令。当store指令在再定序缓冲中执行完毕，指令从“结束”状态更新为“完成”状态，这可以通过改变存储缓冲指针或刷新状态位实现；当完成的store指令离开存储缓冲并更新了存储器，就完成了指令提交。因此，从存储器的角度来看，store指令直到提交完毕才算真正的执行结束。若此时发生了异常，则需要排空存储缓冲中的完成指令，以便能够保证顺序地更新存储器。可见，在分派和提交之间，store指令可能处于以下三种状态：发射（在执行部件中）、结束（在存储缓冲的结束指令区）和完成（在存储缓冲的完成指令区）。

实现载入旁路的一个关键问题是检测可能存在的别名，即存储器数据相关性。如果load在前导store写存储器之前读取该数据，就认为该load指令旁路了store指令，因此在load提前执行之前，必须首先确定那些已发射但尚未提交的store指令是否同该load存在别名。假设load/store指令是从保留站中顺序发射的，所有这样的store指令都应该在存储缓冲中，包括结束指令和完成指令，别名检查就可以使用存储缓冲完成。为此，可以在存储缓冲的每一项记录上增加一个内存地址标签域。一旦load指令访问的内存地址就绪，就用它来查找存储缓冲的地址域，如果发生匹配，则证明别名存在，因此load指令不能乱序执行；否则load指令同存储缓冲中的前导store指令不存在相关，可以提前执行。相联匹配查找可以在load部件的第三级流水段实现并与cache访问同时进行，如图4.45所示。若没有检测到别名，则允许load指令完成，并将数据cache返回的数据写入重命名目标寄存器；若检测到别名，则丢弃数据cache返回的数据，并将load指令存回保留站等待以后发射。

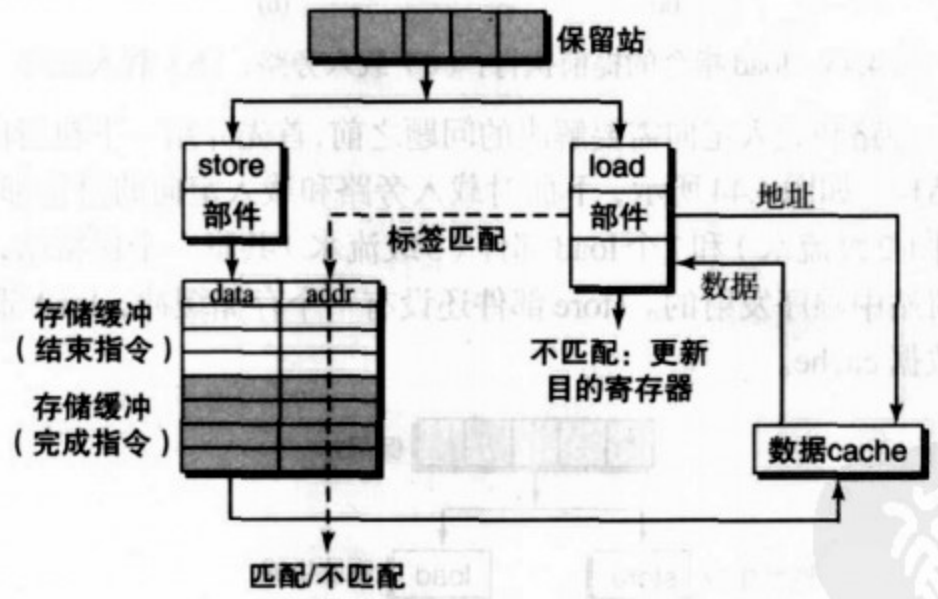


图4.45 载入旁路示意图

载入旁路的实现复杂性主要来源于存储缓冲和相联查找机制。为减少复杂性，可以将地址标签域中保存的全地址改为保存部分地址。使用部分地址可以减小比较器的宽度，但查找结果不理想，使用全地址查找时不存在的别名在使用部分地址查找时可能变为别名，从而丧失了载入旁路的可能。一般情况下，使用足够的部分地址可以使性能降低得很少。

载入定向技术是对载入旁路的补充和扩展。当load想要提前到store之前执行时，若检测到load和一个前导store指令存在别名，就可以将该store指令所保存的数据直接定向到该load指令。载入定向同样也需要对存储缓冲进行相联查找，当检测到别名时，不是将load指令存回保留站并等待以

后发射，而是将匹配的存储缓冲记录中store指令的数据定向到load指令的重命名目标寄存器。这种技术不但允许load指令提前执行，还省去了对数据cache的访问，从而减轻了数据cache总线带宽上的负担。

要支持载入定向，必然要增加存储缓冲的复杂度，如图4.46所示。首先，必须使用全地址进行相联查找。载入旁路如果使用部分地址查找机制，则带来的仅仅是性能的降低；而对于载入定向，所带来将是语义的错误。其次，存储缓冲中有可能有多条store指令和当前load指令存在别名，因此需要一个判断逻辑来确定哪一条store指令是最近的，即在定向之前需要额外的优先编码逻辑来定位同load相关的、最近的store指令。第三，存储缓冲需要增加一个读端口。在加入载入定向之前，存储缓冲有1个写端口供store部件写入和1个读端口供数据cache读取。加入了定向技术后，就需要再给load部件提供一个新的读端口，否则载入定向和数据cache更新就会产生端口冲突。

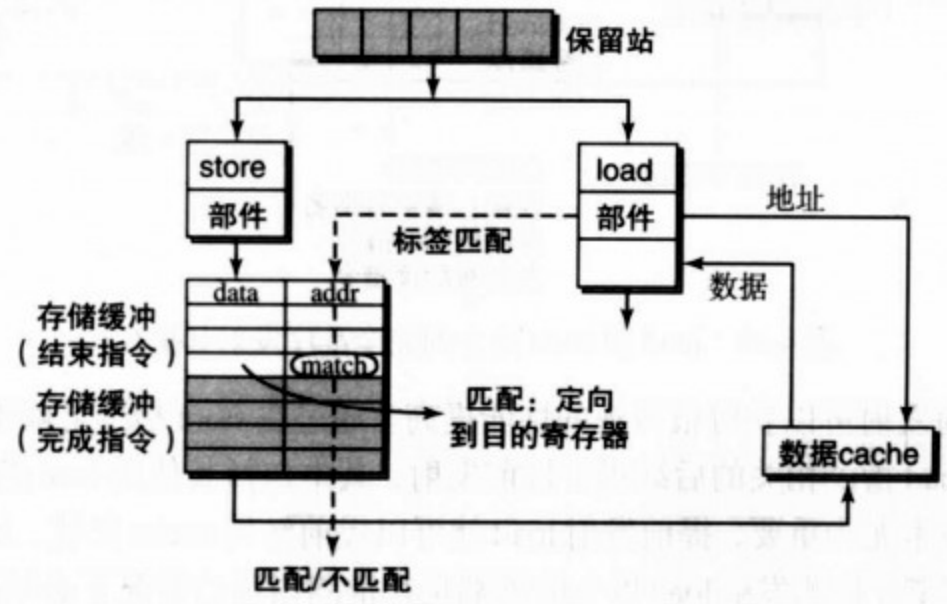


图4.46　载入定向示意图

载入旁路和载入定向可以带来很大的性能提高，Mike Johnson的研究表明，载入旁路可以带来11%到19%的性能提高，而载入定向可以再产生1%到4%的性能提高。

以上都是假设load指令和store指令共享同一个保留站，并按程序原始顺序分别发射到load部件和store部件，从而保证在load指令执行时，所有前导store指令都已经保存在存储缓冲中，因此只需在存储缓冲中进行相联查找并简化了存储器相关性检测。然而，这种假设限制了load指令的乱序执行。load指令在就绪后，即便前导store指令和它不存在别名，也限制了load指令的发射。因此允许load和store从load/store保留站中乱序发射，可以获得更大程度的乱序执行并使load指令的执行更加提前，尤其对于那些位于关键相关链首的load指令，它们的提前执行可以克服性能上的瓶颈。

要实现load和store乱序发射，必须首先解决一个新的问题。若load指令乱序发射，有可能在它之前的store指令仍在保留站中或仍在执行流水线中，并且还没有进入存储缓冲。因此，仅对存储缓冲中的store进行相联查找不能确定load指令同所有前导store指令之间的别名关系，这一时刻，可能前导store指令的访存地址还在保留站中或者还在流水线中进行计算。

一种解决方案是假定与不在存储缓冲中的store指令没有别名冲突，继续执行load指令，以后再进行验证，这样load指令就可以乱序发射并推测执行。如果load同所有存储缓冲中的store指令不存在别名，load指令可以结束（finish）。如图4.47所示，结束load指令缓冲（finished load buffer）和存储缓冲的管理相同，load指令在结束执行之后、完成之前存入结束load指令缓冲中。每当一条store指令完成时，就与结束load指令缓冲中的load指令进行别名检测。若无别名冲突，store指令允许完成；若检测到别名，即有一条已经结束执行的后续load指令同当前store指令相关，该load

指令及其后续指令的执行结果必须失效并且重新发射，甚至重新取指。这需要复杂的硬件支持并且导致了性能上的损失。

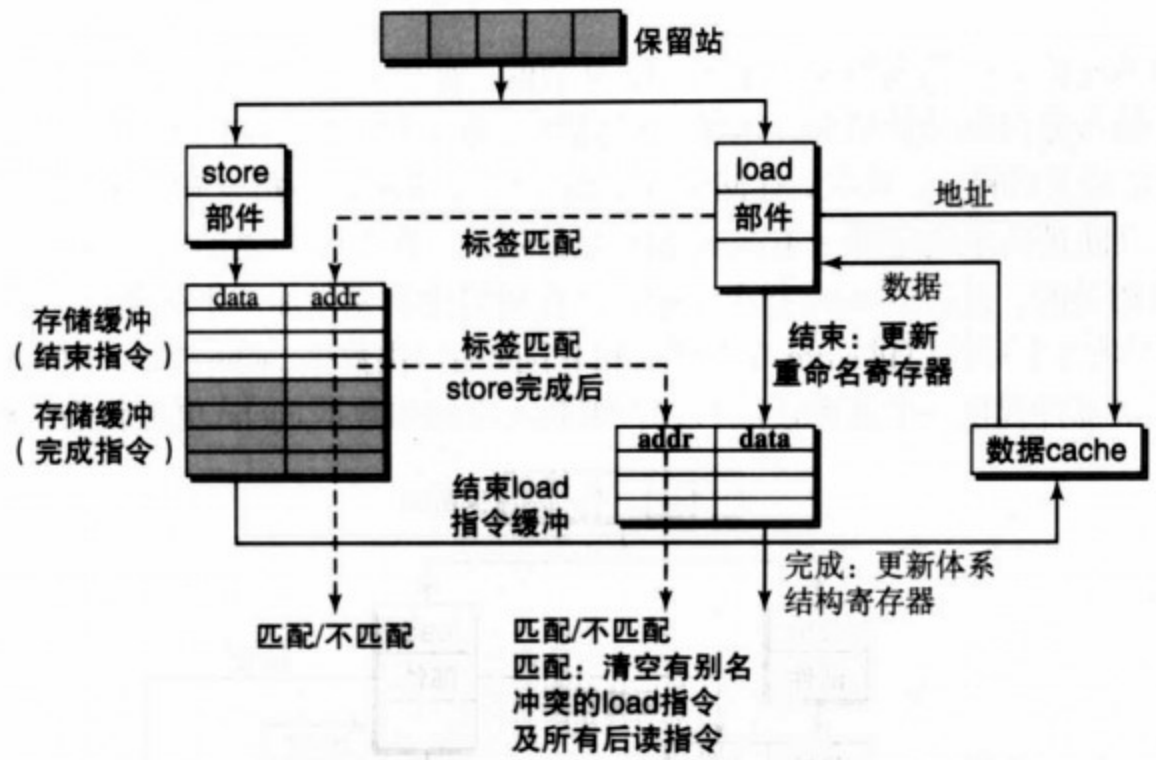


图 4.47　load 和 store 指令的完全乱序发射和执行

load 指令的提前发射可以获得很可观的性能提高。通过在 store 指令之前推测性地发射 load 指令，可以使许多与 load 指令相关的后续指令提前发射，其中包括其他的 load 指令。在 cache 失效频繁的情况下，这种技术尤为重要，提前发射 load 就可以提前触发 cache 失效，从而屏蔽掉部分甚至全部失效所产生的开销。推测发射load指令的弊端是在推测错误的情况下必须进行恢复。减少恢复开销的一种方法是进行别名（相关性）预测。在许多典型的程序中，load 指令与前导 store 指令之间的相关关系都是可预测的。存储器相关性预测符用来对 load 指令与其前导 store 指令是否存在别名进行预测，从而确定是否能够推测发射该load指令。为了能够确实地提高性能，必须明智地选择是否需要推测发射 load 指令。

4.3.5　其他存储器数据流技术

除了载入旁路和载入定向以外，还有一些其他的存储器数据流技术，这些技术都有一定的针对性，比如增加存储器带宽或降低存储器延时。随着超标量处理器的处理宽度越来越大，需要更大的存储器带宽来支持每周期多条 load/store 指令的访问。同时，处理器同存储器之间速度的差异也越来越大，访存的延时，尤其当 cache 失效时的延迟，将成为影响机器性能的一个主要瓶颈。

增加存储器带宽的一个途径是使用多端口数据 cache，在执行内核中配置多个 load/store 部件。在 4.3.4 节中只配置了 1 个 store 部件和 1 个 load 部件以及 1 个单端口数据 cache。load 部件优先访问数据 cache，store 指令在存储缓冲中排队等候，在存储器总线不忙时，存储缓冲获得访问权并将其中的 store 指令提交到存储器。总的数据存储器带宽为每周期 1 条 load/store 指令，这是一个很大的限制，尤其在多条 load 指令突发（burst）时。减缓这一瓶颈的一种方法是提供两个 load 部件和一个双端口数据 cache，如图 4.48 所示。双端口 cache 每周期能够同时处理两个 cache 访问，使存储器的带宽加倍，但这会导致硬件复杂度提高，成本加倍。减少硬件成本的一种办法是用交叉多体存储器实现数据 cache，使两个对不同存储体的同时访问可以在同一周期内进行。如果需要访问同一个体，则产生体冲突，因而两个访问必须被串行化。经验表明，8 体的 cache 组织结构能够将体冲突的频率降低到可以接受的水平。

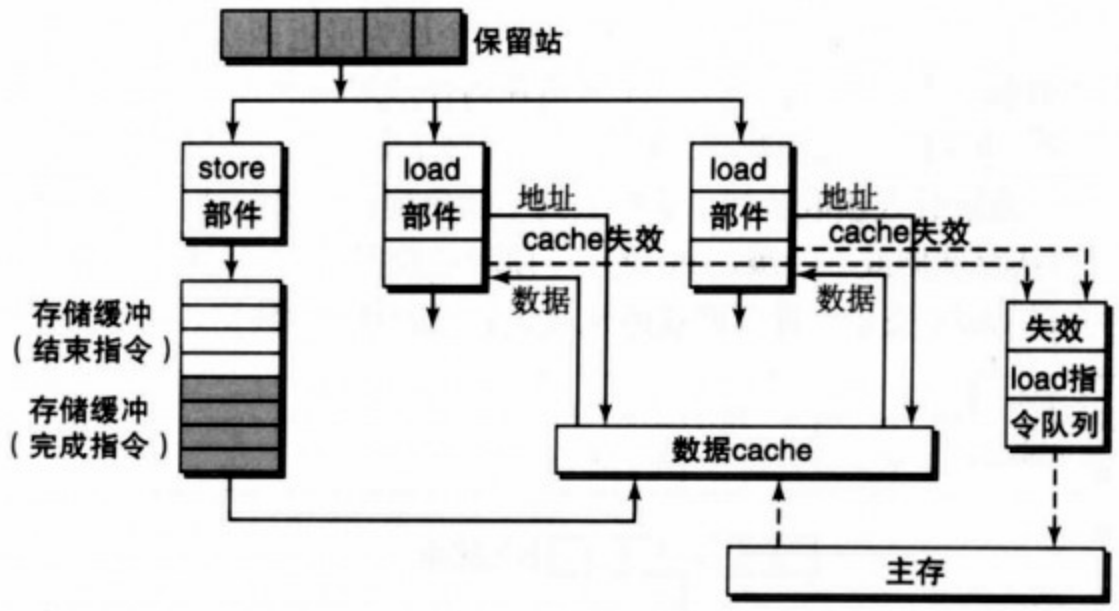


图 4.48　双端口非阻塞 cache

降低存储器延时最常用的方法是使用 cache，现在 cache 的使用已经很普遍。由于处理器速度和存储器速度的差距在变大，一般需要配置多级 cache，多数高性能超标量处理器都至少有两级 cache。第一级 cache（L1）速度和处理器速度相近，访问延时为一个到几个时钟周期，而且一般单独地设置数据 cache 和指令 cache；第二级 cache（L2）数据和指令统一放置（可以在片内，也可以在片外），可以同 L1 cache 并行访问或串行访问（在 L1 失效时）。当前一些新的设计中使用了三级 cache（L3），将来在设计中集成大容量的片内 L3cache 将是很常见的事情。除了使用 cache 及多级 cache 技术，还有两种技术可以降低存储器延时，即非阻塞 cache 和预取 cache。

非阻塞 cache 通过降低 cache 失效开销来降低存储器的访问延时。在一般的 cache 结构中，当 load 指令引起 cache 失效时，将停顿 load 部件流水线，禁止所有 load 指令的发射，直到 cache 失效处理完毕。这种基于停顿的策略非常保守，使得不会引发 cache 失效的后继 load 指令也不能发射。非阻塞 cache 克服了这一缺点，将引发 cache 失效的 load 指令放入失效 load 指令队列（missed load queue），并允许后续 load 指令的访问。如图 4.48 所示，失效 load 指令在处理失效过程中将保存在失效 load 指令队列中，当所需 cache 块从存储器取出后，失效 load 指令弹出队列并结束执行。

本质上，失效 load 指令同后续不相关指令的处理重叠，从而隐藏了 cache 失效开销。如果后续指令同失效的 load 指令相关，则它们仍然要停止发射。非阻塞 cache 所能隐藏的失效周期数取决于失效 load 后不相关的指令条数。失效 load 指令队列包含多条记录，因而可以处理多条 load 同时失效的情况。多条失效 load 指令的 cache 失效开销可以重叠，从而总的失效开销较少。

实现非阻塞 cache 必须考虑很多的问题。load 指令的失效可能出现集中突发的情况，因此支持多条失效 load 以及重叠处理的能力很重要，非阻塞 cache 与存储器或低一级 cache 的接口必须支持多个访问的重叠或流水处理。cache 失效所引发的 cache 写同存储缓冲中 store 指令提交时的 cache 写存在冲突。非阻塞 cache 所带来的另一个复杂问题是推测 load 指令。失效的 load 可能位于一条推测的路径（预测路径）上，而这种推测（分支预测）又有可能是错误的。若该路径最终证明是错误的，是否需要处理 cache 的失效呢？在使用了分支预测技术的机器中，有许多处于错误路径上的 load 指令，处理它们导致的 cache 失效将耗费大量的存储器带宽。研究表明，非阻塞 cache 可以降低 15% 的失效开销。

降低或隐藏 cache 失效开销的另一种方法是使用预取 cache。预取 cache 对将来可能发生的失效进行预测，提前引发失效并将失效开销同失效 load 指令之前的指令处理重叠起来。图 4.49 显示了一个预取数据 cache，它需要两个额外的结构来实现：内存地址预测表和预取队列。内存地址预测表保存先前执行的 load 指令信息并包含三个域：第一个域为 load 指令地址，作为标签来定位某条

记录；第二个域为load指令先前访问的数据地址；第三个域为最近两次访问的数据地址的差值。使用分支预测所生成的取指地址与该表的第一个域值在内存地址预测表中进行相联查找。如果发生匹配，即命中表中某项，同时，将先前数据地址（第二个域）加上第三个域中的差值生成一个预测访存地址，然后将预测地址载入预取队列。预取队列中的每项都用来前瞻性地访问数据cache，如果发生cache失效，则从存储器或下一级cache访问。预取对数据cache的访问仅仅是一个“探测”，即引发那些可能存在的cache失效，并不真正地从数据cache中检索数据。

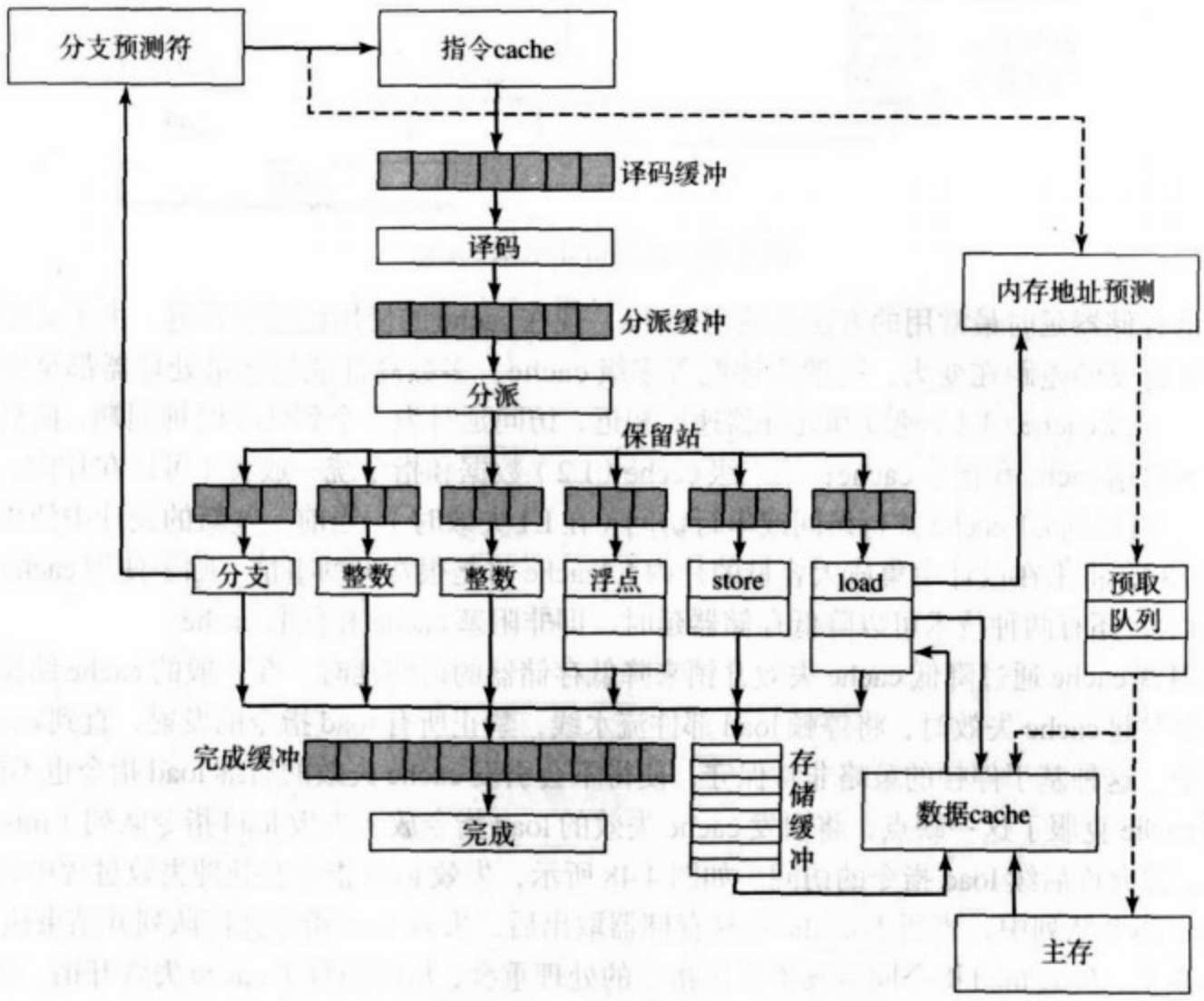


图 4.49 预取数据 cache

预取cache的目的就是预测将会发生的cache失效，从而尽早地引发它们，将调入cache块的时间同失效load之前的指令处理时间重叠起来，最终隐藏cache失效开销。当已预测的失效load指令执行时，数据已经在cache中了，因此不会再引发cache失效，也就不会有任何失效开销。预取的实际效率取决于多种因素。预取的距离，即失效提前触发的距离，必须能够完全隐藏失效开销，这就是为什么使用预测指令地址访问内存地址预测表的原因，即为了尽可能早地完成数据预取，但预取效率会受分支预测效率的限制。此外，错误预测路径上的预取将给数据cache中的数据带来破坏，因而可以为内存地址预测表的每一项增加一个状态位或信赖位来缓解这种预取危害。另一个问题是预取进行得过早而导致有用的cache块被替换，从而造成不必要的失效。还有一个因素就是内存地址预测算法的设计。基于差值的载入地址预测对于数组中load指令的预取很有效，而对于链表之类的数据结构就显得很低效，因此，内存地址的预取需要开发更加高效的预测算法。

为了扩大存储器数据流，load指令必须尽早、尽快地执行，store指令则相对容易，因为研究数据表明，store出现的概率小于load，而且它们一般不在影响性能的关键路径上。为了加速load指令的执行，必须降低load处理延时，load指令总的处理延时包括4部分：（1）流水线前端包括

取指、译码和分派的延时；（2）保留站中解决数据相关的延时；（3）执行流水线中地址生成和转换的延时；（4）cache或内存的数据访问延时。非阻塞cache和预取cache都是针对第4部分延时所进行的优化。由于存储器速度的原因，第4部分延时十分关键。为了获得较高的时钟频率，超标量处理器的流水线变得越来越深，因此如果按时钟周期数计算延时，前三部分的延时同样占有很大的比重，有很多推测技术可以降低这几部分的延时，如载入地址预测、载入值预测和存储器相关性预测。

最近的载入地址预测技术就是为了降低前三部分延时而提出的。为了处理第一部分延时，使用了同内存地址预测表相似的载入预测表，该表用预测指令的地址进行索引，一旦命中则表明在下一取指序列中存在load指令。因此，load指令的预测在取指段进行并且无需译码和分派。预测表的每一项记录都包含预测的有效地址，在取指段就可以读出，不必再经过保留站等待基址寄存器的值和执行流水线的地址生成。因此，对数据cache的访问在下一周期就可以开始，一般在译码段的末尾就能从cache读到数据。这种形式的载入地址预测技术如果预测准确且cache命中，则可以有效地将前三部分的延时降低到两个时钟周期，即取指和译码两段。

虽然载入地址预测的硬件结构同预取过程非常相似，但这两种机制仍存在很大的区别。载入地址预测虽然是推测性的但确实执行了load指令，而预取主要是将要用的数据调入cache，并非实际执行该load指令。使用载入地址预测，同该load指令相关的后续指令由于操作数已就绪，因此也可以提前执行。因为载入地址预测是一种推测技术，因此必须经过验证且一旦预测失败，必须进行恢复。验证的过程是将实际load指令按正常方式取指、运行，并将推测的结果和正常运行的结果比较。如果结果匹配，则推测结果将变为非推测结果，并且所有推测执行的相关指令都变为非推测的；如果结果不同，则使用正确结果，所有相关指令重新执行。若载入地址的预测机制准确度较高，则很少出现预测错误且错误开销很小，因此就可以提高总的性能。

比载入地址预测更进一步的预测方式是载入值预测。载入地址预测是预测load指令的数据有效地址，而载入值预测是对要载入的数据值进行预测。扩展载入预测表，使其不但包含载入地址，还包含载入的数据，就可以实现载入值预测。研究表明，许多load指令的载入数据都是可预测的，如许多load指令载入的是同上次载入一样的数据。因此，将静态load指令载入的数据保存在载入预测表中，用来对下一次该指令的执行进行值预测。载入预测表可以在取指段进行访问，从而获得load指令的预测值，在下一时钟周期中相关指令可以直接使用。如果预测正确，值预测将会大大降低load指令处理的延时。同样，值预测也需要验证，同时，预测错误时则需要恢复。

除了载入地址预测和值预测以外，另一种load指令推测执行技术是存储器相关性预测。在4.3.4节中所讲的载入旁路和载入定向都需要进行存储器相关性检测。对于载入旁路，必须确定load指令同被旁路的store指令之间不存在别名；对于载入定向，则必须确定最近的、存在别名冲突的store指令。如果所涉及的load/store指令太多，相关检测逻辑会很复杂，有可能占用整整一级流水站，因此有必要消除此类延时。实验数据表明，存储器相关性关系是可以预测的。在load/store指令执行时跟踪存储器相关性信息，并在该load/store指令重新执行时利用这些信息对存储器相关性进行预测，则方便了载入旁路和载入定向的提前执行。同其他推测技术一样，验证和错误时的恢复机制都是必需的

4.4　小结

在本章中我们系统、清晰地介绍了现代超标量微处理器设计中的基本微体系结构技术，同时没有向读者灌输太多的数据和图表，而将重点放在通用技术而非特定商业产品的特征上。在第5章和

第6章中将详细介绍两种实际的产品。在第5章中将介绍IBM/Motorola PowerPC 620并详细列举大量的性能数据。虽然PowerPC 620在商业上不是很成功，但它是早期大部分“乱序”微处理器设计的代表。在第6章中将介绍Intel P6微体系结构，即IA32指令集第一个“乱序”体系结构的实现，同时它在商业上也获得了巨大的成功。高效、艺术的P6微体系结构是Intel连续几代产品的基础，如在Pentium Pro、Pentium II和Pentium III的设计中都能找到最初P6体系结构的影子。

超标量微处理器设计是一门快速发展着的“艺术”，它集合了研究者们的聪明才智和设计者们非凡的技术。在第7章中我们将对这门“艺术”的发展历史进行简要的回顾，从中可以总结出有益的经验和教训。超标量微体系结构技术的主体知识框架仍在不断地扩展，同时，来自各研究机构和传统“宏体系结构”领域的创新思想不断涌现，为未来的超标量微体系结构设计寻求着新的发展途径，在第8章和第9章将介绍其中一些新的思想。我们希望能够在该书中为读者增加更多的高级技术专题。

4.5 习题

第1到6题：

下面的代码同时扫描两个数组A和B，对每一个元素，将两个值中的较大者保存到数组C的对应位置。三个数组长度均为N。

指令集：

add	rd, rs, rt	rd ← rs+rt
addi	rd, rs, imm	rd ← rs+imm
lw	rd, offset(base)	rd ← MEM[offset+base] (offset=imm, base=reg)
sw	rs, offset(base)	MEM[offset+base]← rs (offset=imm, base=reg)
bge	rs, rt, address	if(rs >= rt) PC ← address
blt	rs, rt, address	if(rs < rt) PC ← address
b	address	PC ← address

注：r0恒为0

测试代码：

指令序号 #	标号		汇编程序		
	main				
1		addi	r2,	r0,	A
2		addi	r3,	r0,	B
3		addi	r4,	r0,	C
4		addi	r5,	r0,	N
5		add	r10,	r0,	r0
6		bge	r10,	r5,	end
	loop:				
7		lw	r20,	0(r2)	
8		lw	r21,	0(r3)	
9		bge	r20,	r21,	T1
10		sw	r21,	0(r4)	
11		b	T2		

```
        T1:
12                  sw      r20,    0(r4)
        T2:
13                  addi    r10,    r10,    1
14                  addi    r2,     r2,     4
15                  addi    r3,     r3,     4
16                  addi    r4,     r4,     4
17                  blt     r10,    r5,     loop
        end:
```

1. 划分该段测试程序的基本块并用指令序号在下表中标出。

基本块	1	2	3	4	5	6	7	8	9
指令									

2. 画出测试程序的控制流图。
3. 生成指令执行轨迹（即执行的基本块序列）。用下面的数组作为输入运行程序，记录每个基本块执行的次数。

```
N=5;
A[ ]={8,3,2,5,9};
B[ ]={4,9,8,5,1};
```

4. 用上面生成的数据填写下面两张表。

表 1：指令比例

	静态		动态	
类型	数目	%	数目	%
ALU				
Load/Store				
Branch				

注：将非条件分支看做跳转分支指令。

表2：基本块/分支数据

	静态	动态
平均基本块大小（指令数）		
跳转分支数		
不跳转分支数		

5. 根据上面收集的分支信息，重新安排基本块并翻转分支条件，从而使跳转分支最少并且将执行概率大的路径集中在一起。写出新的程序代码。
6. 根据新的程序代码，重新统计上面两张表中的数据。

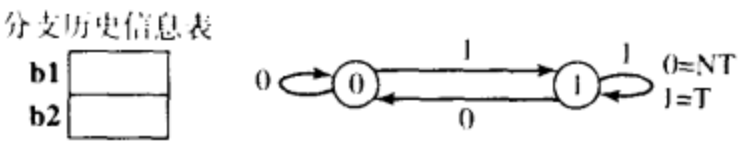
第 7 到 13 题：

对于下面这段代码（第 5 题的部分循环体）：

```
if(x is even) then                          ←(branch b1)
     increment a                             ←(b1 taken)
if(x is a multiple of 10) then              ←(branch b2)
     increment b                             ←(b2 taken)
```

假设用 9 次循环执行下面的输入序列，x = 8, 9, 10, 11, 12, 20, 29, 30, 31。

注：假设每个动态分支在下一个动态分支执行之前都更新预测符记录项（即没有更新延时）。



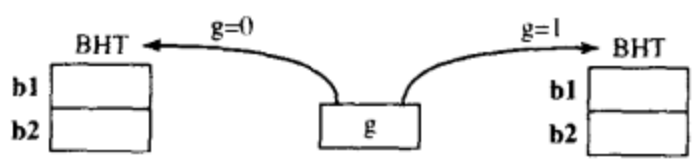
7. 假设使用 1 位（历史信息位）状态机（参见上图）作为预测算法并对该循环中的两个分支指令进行预测。填写每次循环两条分支指令的预测结果和实际执行结果。假设预测符的初始状态为 0，即 NT。

	8	9	10	11	12	20	29	30	31
b1 预测：									
b1 实际：									
b2 预测：									
b2 实际：									

8. b1 和 b2 的预测精度如何？

9. 总的预测精度如何？

10. 假设使用两级分支预测，除了 1 位预测符之外，还有 1 位全局寄存器（g），寄存器 g 保存上一次分支执行的方向（可能和当前分支不是同一分支），用来索引两个独立的 1 位分支历史信息表，如下图所示：



根据 g 的值选择两个 BHT 之一并进行正常的 1 位预测。填写 9 次循环中每条分支指令的预测结果和实际执行结果。假设初始状态 g=0，即 NT。对每一次预测，根据当前 g 的值，两个 BHT 中只有一个被选中和更新，所以下表有些位置为空。

注：假设每个动态分支在下一个动态分支执行之前都更新预测符记录项（即没有更新延时）。

	8	9	10	11	12	20	29	30	31
For g=0									
b1 预测：									
b1 实际：									
b2 预测：									
b2 实际：									
For g=1									
b1 预测：									
b1 实际：									

b2 预测：____ ____ ____ ____ ____ ____ ____ ____ ____

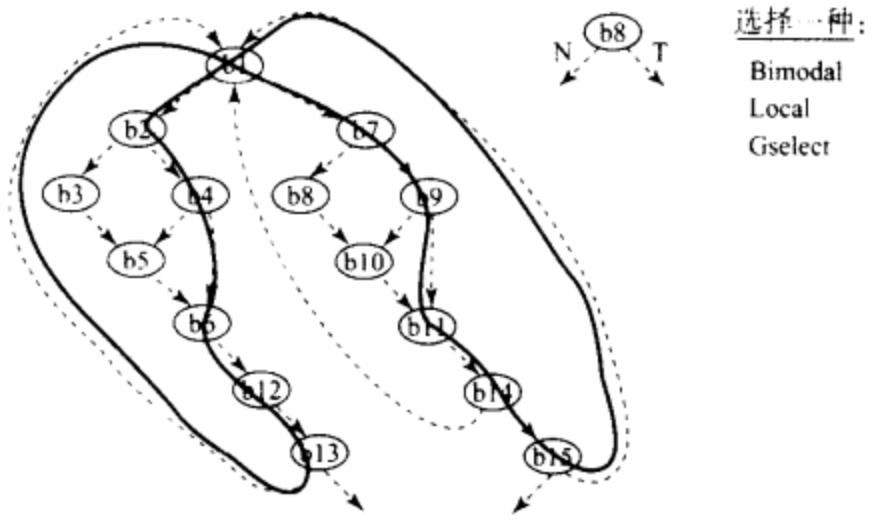
b2 实际：____ ____ ____ ____ ____ ____ ____ ____ ____

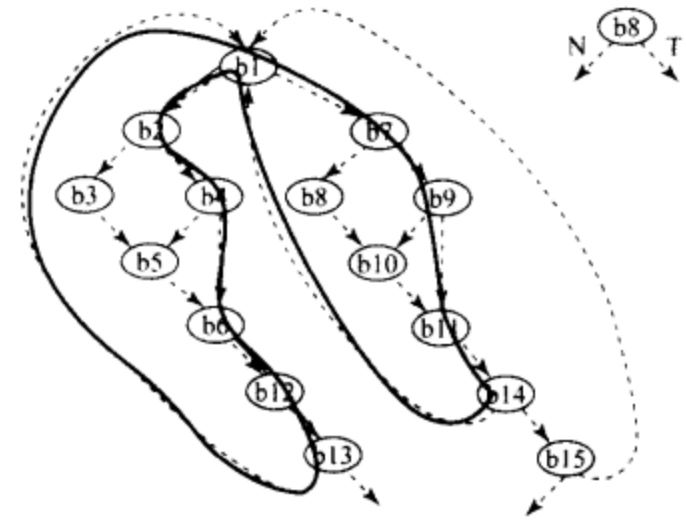
11. b1 和 b2 的预测精度如何？

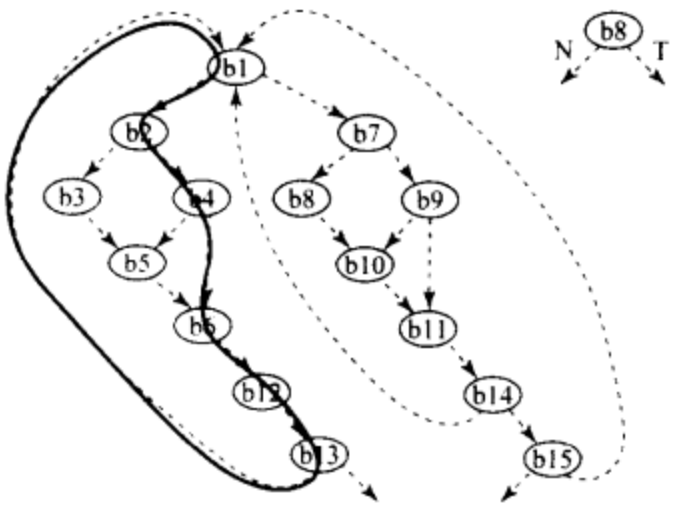
12. 总的预测精度如何？

13. g=0 时，b2 的预测精度如何？为什么？

14. 下图为一个简单程序的控制流图，包含了3条不同的执行轨迹。对每种轨迹指出哪一种分支预测符（bimodal、local 或 Gselect）可以最好地预测分支的行为。对一种特定的轨迹，可能不止一种预测符可以达到同样的预测精度。假设每种轨迹执行多次，CFG 的每个节点代表一个条件分支。local、global 和 Gselect 预测符的分支历史信息寄存器限制为 4 位长度。







15. 分支预测在分支指令执行完毕后才能解决。像 PowerPC 604 这样早期的乱序处理器在解决分支时只是简单地强制分支必须顺序执行。讨论为什么这种方法可以简化分支重定向逻辑，并详细解释若要支持乱序分支应怎样修改微体系结构。
16. 描述一种情况，使乱序分支能够带来性能上的提高，并进行实验来验证这种假设，或者修改一个时序仿真器来管理该实验。
17. 给定图 4.17 所示的 RS/6000 浮点重命名方案，写出图 4.42 中 DAXPY 循环执行两次后的映射表、空闲表和未决目标返回队列的内容。
18. 用Tomasulo算法模拟以下代码片段的执行，给出每个时钟周期内保留站的内容、寄存器忙闲位和标签（保留站 ID）以及数据域的内容（如下表），并指出每周期各个功能单元中正在执行的指令。每当公共数据总线进行数据定向时，画出数据的生产者和消费者并用箭头连接。

```
i:    R4 ← R0+R8
j:    R2 ← R0*R4
k:    R4 ← R4+R8
l:    R8 ← R4*R2
```

假设存在两套分派逻辑和两条CDB。加法延时为2，乘法延时为3，指令在分派的同一周期就可以开始执行并且所有相关性都满足。

周期#: _____

	标签	接收位	标签	源
1				
2				
3				

加法器

	标签	接收位	标签	源
4				
5				

乘/除

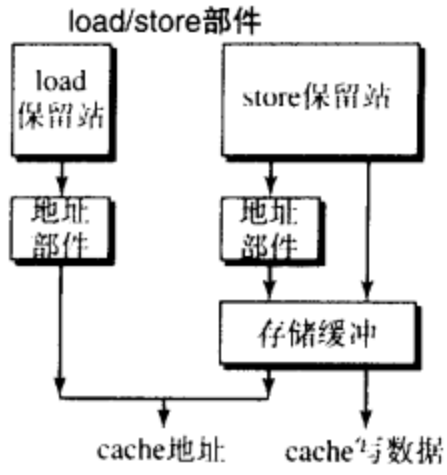
	忙	标签	数据
0			6.0
2			3.5
4			10.0
8			7.8

分派指令(S): _____

19. 确定第 18 题中的代码执行是否达到了数据流极限。为什么？
20. 在本章中曾提到，载入旁路是一种增大存储器数据流的技术，通过载入旁路，load指令可以跳过 store 指令提前执行。store 指令一旦生成地址，就可以完成并写入存储缓冲等待提交。后继的 load 指令如果和这些存储缓冲中的 store 指令没有地址别名就可以旁路它们。
本题将模拟载入旁路（没有载入定向）。给定一个load/store指令序列及它们的地址，每条指令左边的数字代表该指令分派到保留站的时刻并且马上就可以开始执行，每条 store 指令右边还有一个数字，代表其准备提交，即离开存储缓冲开始写内存的时刻。
假设：
(1) 所有用于地址计算的操作数在分派时都已就绪。
(2) 一条 load 和 store 指令的地址计算均需要一个时钟周期。
(3) 一条 load 或 store 指令访问 cache 需要一个时钟周期。
(4) 保留站记录在地址计算完成后回收。
(5) 存储缓冲记录在 cache 访问时回收。
(6) store 指令能够在提交就绪后的一个周期内访问 cache。
(7) 指令数需要发射到保留站。
(8) cache 命中率 100%。

例：

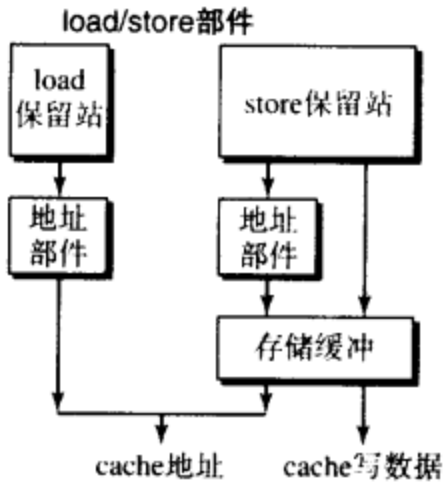
分派	指令	提交
1	load A	
1	load B	
1	store C	5



周期	load保留站				store保留站				存储缓冲				cache 地址	cache 写数据
1	Ld A	Ld B			St C									
2	Ld B								St C				Ld A	
3									St C				Ld B	
4									St C					
5									St C					
6													St C	data

代码：

分派	指令	提交
1	store A	6
2	load B	
3	load A	
4	store D	10
4	load E	
4	load A	
5	load D	



周期	load保留站				store保留站				存储缓冲				cache 地址	cache 写数据
1														
2														
3														
4														
5														
6														
7														
8														
9														
10														
11														
12														
13														
14														
15														

21. 试比较载入定向和载入旁路。

22. 对于上面的指令序列，载入定向能否提高性能？为什么？

第23题到第25题：

非阻塞cache的设计目的就是增加处理器的并行度，从而将cache失效处理同其他的处理和失效重叠。在下面的习题中，假设将下列负载在一个处理器上执行，cache块大小为64字节，总线带宽8字节，设t_{miss}为5个时钟，$t_{transfer}$为16字节/时钟，因此若为阻塞cache，一次失效的开销为$t_{miss}+(64/16)\times t_{transfer}=5+4\times 1=9$个时钟。负载如下：

```
for(i=1;i<10000;++i)
        a+=A[ i ]+B[ i ];
```

用RISC汇编语言书写如下，r3指向A[0]，r4执行B[0]：

```
        li r2, 9999             # load iteration count into r2
loop:   lfdu   r5, 8(r3)        # load A[ i ], incr, pointer in r3
        lfdu   r6, 8(r4)        # load B[ i ], incr. pointer in r4
        add    r7, r7, r5       # add A[ i ] to a
        add    r7, r7, r6       # add B[ i ] to a
        bdnz   r2, loop         # decrement r2, branch if not zero
```

下面是一个该循环体的时序图，假设每个数组都不能命中cache，cache是简单的阻塞cache（m表示miss latency，t表示transfer latency，A表示load from A，B表示load from B，a表示add，b表示branch）：

	周期										1										2									
	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9
数组A		m	m	m	m	m	t	t	t	t																				
数组B																														
执行	A										B										a	a	b	A	B	a	a	b	A	B

在下面的习题中，假设每条指令执行周期为1，cache失效时所有后续指令将停顿直到失效解决（同上表所示），而且两个数组开始均不在cache中，失效时再读取。

23. 假设cache是带关键字定向的阻塞cache（即所需字在传递后直接定向至寄存器），并只能处理单独一个失效。试填写下表。

	周期										1										2									
	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9
数组A		m	m	m	m	m																								
数组B																														
执行	A																													

	3										4										5									
	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9
数组A																														
数组B																														
执行																														

24. 若cache为非阻塞cache，试填写下表。

	周期										1										2									
	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9
数组A		m	m	m	m	m																								
数组B																														
执行	A																													

	3										4										5									
	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9
数组A																														
数组B																														
执行																														

25. 若 cache 块大小不是 64 字节，而是 32 字节，试填写下表。

	周期										1										2									
	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9
数组A		m	m	m	m	m																								
数组B																														
执行	A																													

	3										4										5									
	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9
数组A																														
数组B																														
执行																														

第5章　PowerPC 620

PowerPC系列微处理器中包括64位的PowerPC 620。620是使用真正乱序执行、分支预测技术、分布式多入口保留站技术和寄存器动态重命名技术的第一款64位微处理器，同时包含了6个流水化的执行单元和一个提交缓冲，因此能够保证精确中断。620所采用的这些技术，大部分都没有在其他微处理器中实现过，因此引起了研究学者和设计人员的兴趣。本章将介绍使用VMW（Visualization-based Microarchitecture Workbench）生成的620性能仿真器对620微处理器微体系结构进行指令级，或者说时序级的性能评测。

我们还将介绍IBM POWER3和POWER4的设计，尽管它们在开发指令级并行上都很成功，但我们着重介绍它们和PowerPC 620的区别，这将有利于强调在处理器实现上的趋势：通过cache层次设计来增加存储带宽、实现更准确的分支预测以及更多的执行资源和更深的流水线。

5.1　简介

PowerPC体系结构是IBM、Motorola和Apple公司PowerPC联盟的产物，它基于POWER体系结构，利用并行指令执行并且能够适应技术的进步。PowerPC联盟已经发布了几款芯片。第一款芯片是PowerPC 601，该芯片是从POWER体系结构到PowerPC体系结构的过渡；第二款芯片PowerPC 603是一款低功耗芯片；接下来设计的PowerPC 604芯片则为一款更为先进的芯片，主要是针对桌面系统；第四款芯片就是64位的PowerPC 620。

最近，Motorola和IBM都进行了独立的通用PowerPC兼容产品的开发，Motorola着重于为Apple开发32位的桌面系统芯片，而IBM则着重于开发用于UNIX（AIX）和商业（OS/400）系统的服务器部分。Motorola最近的32位设计是PowerPC G3和PowerPC G4，它们都是从PowerPC 603衍生而来，其特点是流水线短，执行资源有限，成本非常低。IBM的服务器包括顺序执行多线程的Star系列（Northstar, Pulsar, S-Star）和乱序执行的POWER3和POWER4。此外，Motorola和IBM已经为嵌入式市场开发了各种PowerPC核。本章的重点在于PowerPC 620以及它在高端市场的后继产品POWER3和POWER4。

PowerPC体系结构包含32个通用寄存器（GPR）和32个浮点寄存器（FPR）。条件寄存器可以作为一个32位的寄存器进行寻址（CR），也可以作为一个寄存器文件，包括4个8位域（CRF）或者一个单独的32位域。此外还包括一个计数寄存器（CTR）和一个链接寄存器（LR），主要用于分支指令，还包括一个整数异常寄存器（XER）和一个浮点状态控制寄存器（FPSCR），分别用来记录相应类型指令的异常状态。PowerPC的指令集是典型的RISC指令集，增加了浮点乘加指令（FMA）、支持更新有效地址寻址方式的访存指令、设置和操作分支状态寄存器位的指令。

620是宽度为4的超标量处理器，使用了先进的分支预测技术和分派策略，能够尽可能早地取指并将指令分派到各执行单元。620有6个并行的执行单元：两个单周期整数单元，1个多周期整数单元，1个浮点单元（3级），1个load/store单元（2级）和1个分支单元。620使用分布式保留站和寄存器重命名来实现乱序执行，参见图5.1。

620处理指令包含5个主要的阶段：取指（fetch）、分派（dispatch）、执行（execute）、完成（complete）和写回（writeback）段，某些段间使用缓冲来保存指令执行过程中的动态变量。这些缓

冲包括指令缓冲、保留站和完成缓冲。流水段和相应缓冲如图 5.2 所示，执行段的某些单元实际上是多级的流水线。

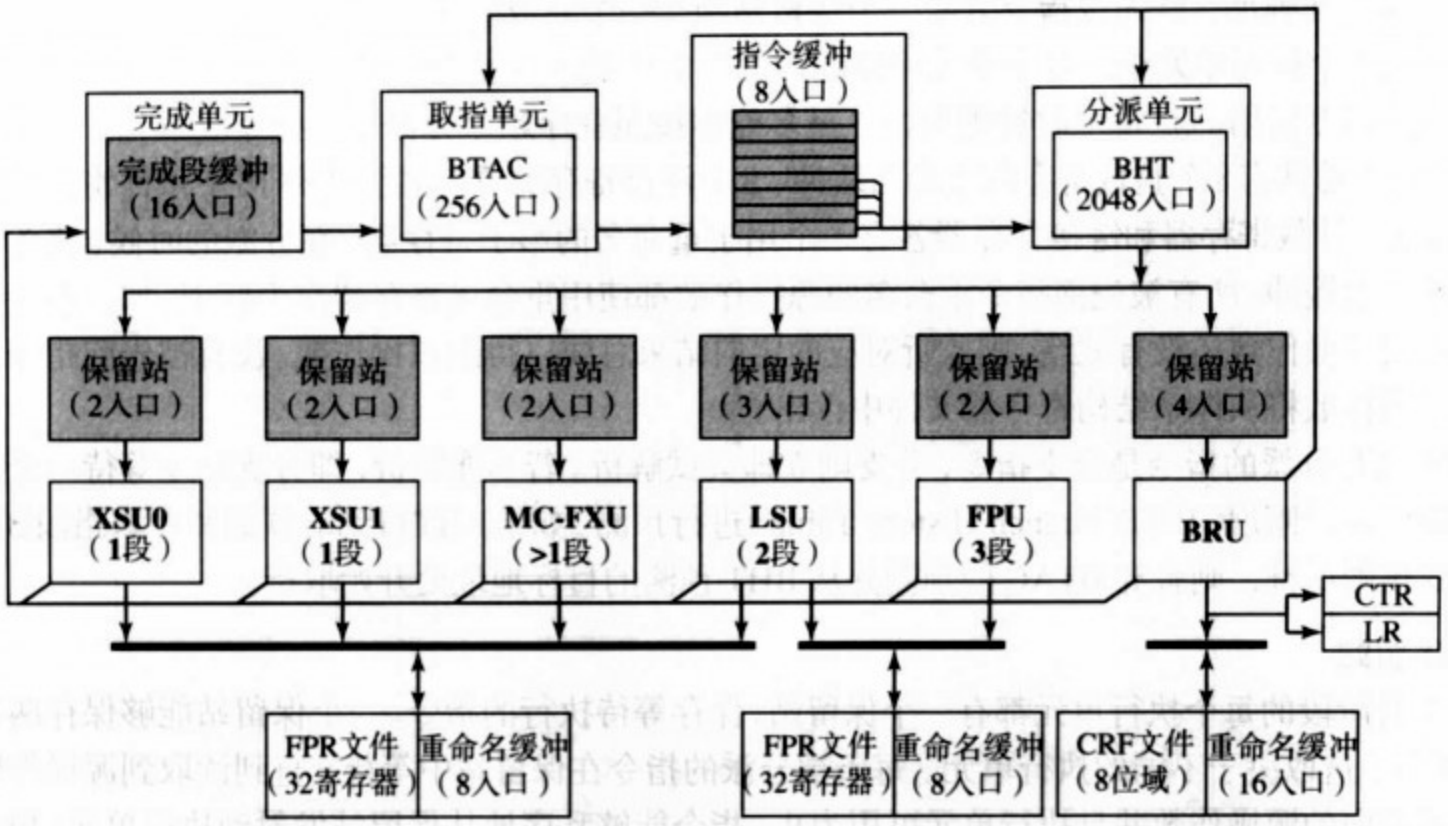


图 5.1　PowerPC 620 微处理器的时钟图

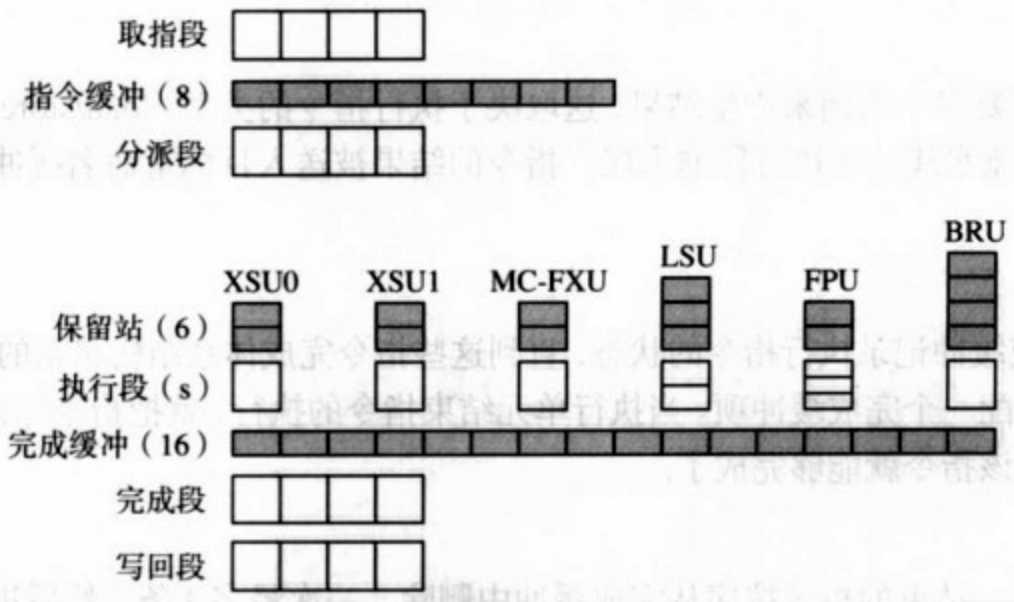


图 5.2　PowerPC 620 微处理器的指令流水

取指段

取指部件访问指令 cache，每个时钟周期从指令 cache 中取出 4 条指令送入指令缓冲。当遇到 cache 块尾或者分支指令跳转的时候，在一个周期内就无法取得 4 条有效指令。分支预测时，由于已经从预测的执行路径得到了指令，因此预测错误会浪费时钟周期。在取指段，可以使用BTAC（Branch Target Address cache）进行初级的分支预测，从而得到下一个时钟周期取指的目标地址。

指令缓冲

指令缓冲在取指段和分派段之间用以保存指令。如果分派段跟不上取指部件的速度，指令缓冲则保存这些指令直到分派单元能够处理。一次最多可以缓冲 8 条指令并以两条指令为单位进行缓冲和移动，从而简化了逻辑指令。

分派段

分派单元对指令缓冲中的指令进行译码，检查这些指令是否能够分派到保留站。如果一条指令的分派条件满足，就为该指令分配一个保留站项、一个完成缓冲项和一个目标操作数重命名缓冲项。在6个执行单元中。每个单元每周期最多只能接受一条指令。某些不常见的串行化约束也可以停止指令发射。在每个时钟周期内，至多能够按照程序顺序分派4条指令。

620中总共有8个整数寄存器重命名缓冲、8个浮点寄存器重命名缓冲和16个条件寄存器域重命名缓冲。计数寄存器和链接寄存器各有一个用于重命名的影子寄存器。在分派的时候，将为指令分配适当的缓冲。所有被先前指令重命名的源操作数都使用重命名寄存器作为标记。如果在指令分派的时候源操作数还没有就绪，则监听对应的定向结果总线以得到该操作数。没有被先前指令重命名的源操作数将从体系结构寄存器文件中读出。

如果要分派的指令是分支指令，分支则立即尝试解析。若不能解析，即分支指令等待一个未就绪的操作数，则使用BHT(Branch History Table)进行预测。如果BHT得到的预测和在取指段BTAC得到的预测不符，则舍弃BTAC的预测并从BHT预测的目标地址处开始取指。

保留站

执行阶段的每个执行单元都有一个保留站，保存等待执行的指令。一个保留站能够保存两条到4条指令，这取决于不同的执行单元。每条被分派的指令在保留站中等待，直到读取到源操作数或者得到定向的源操作数并且执行单元可用为止。指令能够乱序地从保留站发射到执行单元(FPU和BRU除外)。

执行段

该段可能需要多个周期来产生结果，这取决于执行指令的类型。load/store单元是2级流水线，浮点单元是3级流水线。在执行段的末尾，指令的结果被送入目标重命名缓冲并且定向到等待的指令。

完成缓冲

16项的完成缓冲记录执行指令的状态，直到这些指令完成体系结构状态的更新。在分派阶段，将为每条指令分配一个完成缓冲项。当执行单元结束指令的执行，就把指令标记为结束状态。一旦指令执行结束，该指令就能够完成了。

完成段

在完成阶段，结束的指令按序从完成缓冲中删除，一次最多4条，然后进入写回阶段。如果体系结构寄存器文件没有足够的写端口，每个时钟周期完成的指令会更少。通过将结束指令保存在完成缓冲中直到写回，620保证体系结构寄存器文件按最近完成的指令更新状态，从而在乱序执行的情况下保证了精确中断。

写回段

在这一阶段，写回逻辑将提交在前一个周期完成的指令，并将结果从重命名缓冲写入体系结构寄存器。

5.2 实验框架

PowerPC 620性能的模拟是使用CMU开发的VMW框架来实现的。PowerPC 620的5个机器描述是基于620设计小组提供并且定期进行更新的设计文档生成的。设计小组的一名成员仔细检查了文档的解释，如此反复若干个周期直到620设计结束。

指令和输入数据是在现有的PowerPC 601微处理器上使用软件工具生成的，主要是几个SPEC 92的测试程序（4 个整数程序和 3 个浮点程序）。这些测试程序和它们的动态指令组合参见表 5.2。大多数的整数测试程序都有类似的指令组合，li 和其他的测试程序相比，多周期指令较多。大部分指令都将数据移入或者移出专用寄存器。浮点测试程序就有很大的不同，Hydro2d 中有很多非流水的浮点指令，如浮点除法，在 620 上需要执行 18 个周期。

性能模拟使用了 trace 驱动方式，trace 驱动的模拟对于可变延迟的指令，如整数乘/除和浮点除法不够精确。对于这些指令，假设为最小延迟，这些操作的频率和延迟的变化程度都很低。而且，执行的路径只包括被真正执行的指令。在模拟运行当中，没有包括因为预测失败而丢弃的推测指令。指令 cache 和数据 cache 的动作都包括在模拟当中，cache 大小是 32 KB 并且为 8 路组相联。数据 cache 是 2 路交错编址。cache 失效开销是 8 个时钟周期并且假设 L2 cache 是“完美”的（命中率 100%）。

表 5.1 给出了每种测试程序模拟的指令总数和消耗的 620 时钟周期的总数。在 620 上运行每个测试程序得到的 IPC（平均每周期执行的指令数）也在表中列出，IPC 的值反映了 620 体系结构开发指令级并行的能力，详细的分析见下文。

表 5.1　测试程序的性能比较

测试程序	动态指令	执行周期	IPC
compress	6 884 247	6 062 494	1.14
eqntott	3 147 233	2 188 331	1.44
espresso	4 615 085	3 412 653	1.35
li	3 376 415	3 399 293	0.99
alvinn	4 861 138	2 744 098	1.77
hydro2d	4 114 602	4 293 230	0.96
tomcatv	6 858 619	6 494 912	1.06

表 5.2　测试程序集的动态指令组合

	指令组合	整数测试程序 (SPECint 92)				浮点数测试程序 (SPECfp 92)		
		compress	eqntott	espresso	li	alvinn	hydro2d	tomcatv
整数	运算（单周期）	42.73%	48.79%	48.30%	29.54%	37.50%	26.25%	19.93%
	运算（多周期）	0.89%	1.26%	1.25%	5.14%	0.29%	1.19%	0.05%
	load	25.39%	23.21%	24.34%	28.48%	0.25%	0.46%	0.31%
	store	16.49%	6.26%	8.29%	18.60%	0.20%	0.19%	0.29%
浮点数	运算（流水线）	0.00%	0.00%	0.00%	0.00%	12.27%	26.99%	37.82%
	运算（非流水线）	0.00%	0.00%	0.00%	0.00%	0.08%	1.87%	0.70%
	load	0.00%	0.00%	0.00%	0.01%	26.85%	22.53%	27.84%
	store	0.00%	0.00%	0.00%	0.01%	12.02%	7.74%	9.09%
分支	无条件分支	1.90%	1.87%	1.52%	3.26%	0.15%	0.10%	0.01%
	条件分支	12.15%	17.43%	15.26%	12.01%	10.37%	12.50%	3.92%
	计数寄存器中的条件分支	0.00%	0.44%	0.10%	0.39%	0.00%	0.16%	0.05%
	链接寄存器中的条件分支	4.44%	0.74%	0.94%	2.55%	0.03%	0.01%	0.00%

5.3　取指

假设指令缓冲未满，620取指部件每个时钟周期可以取4条指令，如果取指部件在顺序取指之前等待分支指令的解析，或者是简单地预测分支不跳转而顺序取指，机器执行的速度都会因分支指令的跳转而受到很大的影响　因此，精确的分支预测算法对于保证超标量处理器的使用效率很重要

5.3.1 分支预测

620的分支预测分两个阶段进行。第一级预测在取指段进行，在取指的时候遇到分支指令时，使用BTAC对目标地址进行初步预测。第二级预测则更为精确，在分派单元使用BHT进行预测。BHT包含分支历史信息，根据两位分支历史位来进行分支预测。

在分派阶段，620试图立即根据可用信息解析分支。如果分支是无条件分支，或者条件寄存器的相关位已经就绪，就不需要进行分支预测，分支则立即被执行。另外一方面，如果源状态寄存器位还不可用，也就是生成该状态位的指令还没有结束，就使用BHT进行分支预测。BHT每一项包括两位历史信息，在分派阶段用来预测分支是成功还是失败。当分支指令解析之后，将使用实际分支的方向来更新BHT。BHT是大小为2048项的直接映射表，和BTAC不同，BTAC是相联cache。如果两个更新BHT的分支指令地址之间刚好相差2048的整数倍，则它们的预测会彼此相互影响。

620在分派阶段就可以解决或者预测分支，但即使这样，在新的分支目标确定之前还是存在一个时钟周期延时，因此620在取指阶段基于当前指令的地址进行了一个初级的预测。如果某一个地址命中BTAC，保存在BTAC中的目标地址将被用来作为下一个周期的取指地址。BTAC比BHT要小，只有256项并且是2路组相联。BTAC只保存那些预测为跳转的分支目标地址，预测为不跳转的分支指令不保存在BTAC中。只有无条件分支和PC相对的分支指令才会使用BTAC，计数寄存器或者链接寄存器寻址的分支指令目标地址是不可预测的，因而不保存在BTAC中，一般这种指令在取指段总被BTAC预测为不跳转。此外还用一个链接寄存器栈保存过程的返回地址并用来预测条件返回指令。仿真器上没有实现链接寄存器栈。

BTAC预测有4种可能的情况出现：BTAC失效，分支没有跳转（预测正确）；BTAC失效，分支发生跳转（预测错误）；BTAC命中，分支发生跳转（预测正确）；BTAC命中，分支没有跳转（预测错误）。BTAC命中一个跳转的分支、但提供错误目标地址的情况不会出现，只有PC相对分支会在BTAC中命中，因此必然使用同样的目标地址。每条分支指令都进行两次分支预测，一次在取指段由BTAC进行，一次在分派段由BHT进行。如果BHT预测和BTAC的预测不符，则使用BHT的预测并舍弃BTAC的预测。如果预测相符并且是正确的，所有推测取得的指令都是有用的并且没有产生任何损失。

将BHT和BTAC各种可能的预测和结果综合起来，总共有6种可能的输出。总的来说，BHT和BTAC进行的预测在很大程度上是相关的，BTAC预测失误而由BHT的预测并得到纠正的情况则很少见，而BTAC预测正确却被BHT的错误预测所覆盖的情况就更少了，参见表5.3。BTAC对分支所进行的早期预测不使用分支历史信息。在BTAC中，命中即表示预测分支为跳转，没有命中则表示预测分支为不跳转。BHT的预测是基于分支历史信息的，而且更为精确，但是如果它的预测和BTAC的预测不一致，将会导致一个时钟周期的开销。BHT跟踪分支执行的历史并且更新BTAC中的项，这就是两个预测之间所谓的相关性。

表5.3总结了测试程序分支预测的统计数据。整数测试程序的BTAC预测精度为75%到84%，浮点测试程序的预测精度在88%到94%之间。由BTAC做出正确预测的分支，如果也被BHT正确预测，则不会导致分支开销。总的分支预测正确率由BHT来决定。对于整数测试程序，大约有17%到29%的分支指令在到达分派段时就被解决。对于浮点测试程序，这个值是17%到45%。整数测试程序的预测错误率为8.7%到11.4%，浮点测试程序为0.9%到5.8%。现在的分支预测机制对于浮点测试程序效果很好，整数测试程序的预测则还有改进的余地。

表 5.3　取指指令

分支处理			*compress*	*eqntott*	*espresso*	*li*	*alvinn*	*hydro2d*	*tomcatv*
分支判定	无符号		40.35%	31.84%	40.05%	33.09%	6.38%	17.51%	6.12%
	包含符号		59.65%	68.16%	59.95%	66.91%	93.62%	82.49%	93.88%
BTAC 预测	正确		84.10%	82.64%	81.99%	74.70%	94.49%	88.31%	93.31%
	错误		15.90%	17.36%	18.01%	25.30%	5.51%	11.69%	6.69%
BHT 预测	已判定		19.71%	18.30%	17.09%	28.83%	17.49%	26.18%	45.39%
	正确		68.86%	72.16%	72.27%	62.45%	81.58%	68.00%	52.56%
	错误		11.43%	9.54%	10.64%	8.72%	0.92%	5.82%	2.05%
BTAC 预测错误，BHT 预测正确		0.01%	0.79%	1.13%	7.78%	0.07%	0.19%	0.00%	
BTAC 预测正确，BHT 预测错误		0.00%	0.12%	0.37%	0.26%	0.00%	0.08%	0.00%	
分支预测准确率		88.57%	90.46%	89.36%	91.28%	99.07%	94.18%	97.95%	

5.3.2　取指和推测

分支预测的主要目的是保证较高的取指带宽，从而使得超标量机器的剩余部分能够始终处于忙态。预测失败则会导致取指周期的浪费，从而减少有效的取指带宽。另外一个使得取指带宽减少的原因是指令cache失效。这两个负面因素对于取指带宽的影响参见表5.4。对于整数测试程序，因为预测失败而导致的取指周期浪费的比例为6.7%到11.8%并占有相当大的份额；而对于所有测试程序，指令cache失效引起的取指周期浪费不到1%。

表 5.4　0带宽取指周期

测试程序	预测失败	I-cache 失效
compress	6.65%	0.01%
eqntott	11.78%	0.08%
espresso	10.84%	0.52%
li	8.92%	0.09%
alvinn	0.39%	0.02%
hydro2d	5.24%	0.12%
tomcatv	0.68%	0.01%

分支预测是一种推测技术。当推测有效地进行时，通过避免控制相关带来的限制，可以有效地增加机器的性能。620最多可以连续推测4条分支指令，当遇到第5条分支指令的时候则在分派段停顿。推测指令可以执行到流水线后端直到分支被解析。此时，如果推测证明是错误的，则推测指令都将失效。推测指令可能在分支指令解决之前已经结束执行并到达提交阶段，但它们直到分支指令解析后才会被提交。

表5.5给出了旁路分支指令出现的比率，它反映了指令推测执行的程度。旁路分支指令的平均数量由每个时钟周期正确预测的分支数目决定。若一条分支指令确定预测失败，则该指令后面的推测指令不进行模拟。对于整数测试程序，在34%到51%的时钟周期中，620在执行一个或者多个分支后推测指令。对于浮点测试程序，推测的程度则较低。预测失败的频率（参见表5.4）与旁路分支的平均数（参见表5.5）和预测精确度（参见表5.3）有一定的关联。

表 5.5 旁路分支的分布及其平均数量

	旁路指令的数量					
测试程序	0	1	2	3	4	平均值
compress	66.42%	27.38%	5.40%	0.78%	0.02%	0.41
eqntott	48.96%	28.27%	20.93%	1.82%	0.02%	0.76
espresso	53.39%	29.98%	11.97%	4.63%	0.03%	0.68
li	63.48%	25.67%	7.75%	2.66%	0.45%	0.51
alvinn	83.92%	15.95%	0.13%	0.00%	0.00%	0.16
hydro2d	68.79%	16.90%	10.32%	3.32%	0.67%	0.50
tomcatv	92.07%	2.30%	3.68%	1.95%	0.00%	0.16

5.4 指令分派

分派段的主要任务是将指令从指令缓冲分派到保留站中。PowerPC 620 使用顺序分派策略。

5.4.1 指令缓冲

指令缓冲位于取指段和分派段之间，共8项。取指段负责填满指令缓冲，分派段检查指令缓冲的前4项并且将这些指令分配到保留站中。指令分派之后，指令缓冲中的剩余指令以两条指令为单位移入空闲项中。

图5.3（a）反映了指令缓冲的利用率，记录了指令缓冲中不同指令数量出现的频率。指令缓冲将取指段和分派段分离开来，从而缓和取指和指令分派并行性之间的时序变化和区别。对于指令缓冲中有0条指令的概率，浮点测试程序比整数测试程序要低很多，这个概率和表5.4中所示的预测失败概率有很大的关系。另一方面，指令缓冲的饱和会引起取指停顿。

5.4.2 分派停顿

620分派指令时需要并行检查所有能够引起分派停顿的条件，这些条件在下面的讨论中会详细地列出。在模拟的时候，每次按所列顺序对以下列表中的一个条件进行检查，一旦遇到某个分派停顿条件满足，后面的条件无需再进行检查，将该条件作为导致停顿的条件即可。

串行化限制

某些指令会引起单指令串行化（single-instruction serialization）。所有前面分派的指令必须在串行化指令开始执行之前完成，串行化指令后面的指令必须在串行化指令执行完毕之后才能够被分派。这一限制条件对于性能有很坏的影响，但非常少见。

等待 mtspr 的分支

某些类型的分支指令在分派阶段访问计数寄存器。写入特殊功能寄存器的指令（mtspr）如果正在写计数寄存器，则会引起后继相关的分支指令推迟分派，直到mtspr指令结束执行。这种条件也很少见。

寄存器读端口饱和

通用寄存器文件包含 7 个读端口，浮点寄存器文件则包含 4 个读端口。某些时候，会出现请求一个读端口但是所有的读端口都被占用的情况，即读端口饱和。状态寄存器域由于有足够的读端口，所以饱和的情况永远不会发生。

保留站饱和

指令被分派后将保存在相应执行单元的保留站中，直到发射流出。每个执行单元有一个保留站，每个保留站中有多个项，相应的数量因执行单元而异。当指令可以分派到保留站中但是保留站中却没有空闲项的时候，就发生保留站饱和。

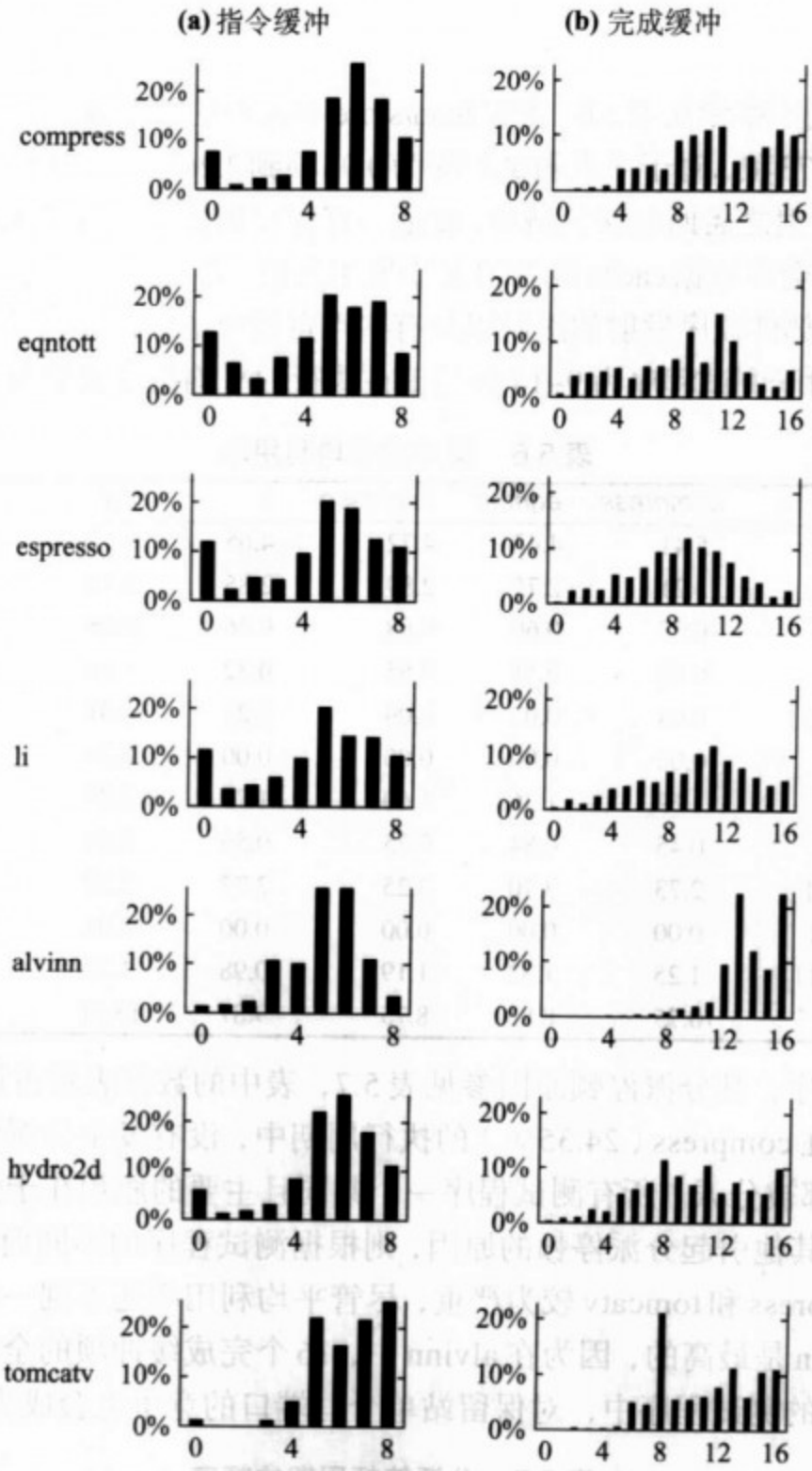


图 5.3　指令缓冲以及完成缓冲的利用率

重命名缓冲饱和

当每条指令被分派的时候，它的目标寄存器被重命名到相应的重命名缓冲文件中。620有3个重命名缓冲文件，分别属于通用寄存器、浮点寄存器和条件寄存器域。通用寄存器和浮点寄存器都有8项重命名缓冲，条件寄存器域则有16项重命名缓冲。

完成缓冲饱和

完成缓冲项也在分派阶段分配并且直到指令执行结束才回收。620有16个完成缓冲项，即最多有16条指令可以同时执行。任何试图分派多于16条指令同时执行的请求都会引起停顿。图5.3（b）给出了各种测试程序完成缓冲的利用率。

多于一条分派到同一单元的指令

一个保留站有多个项，但即使保留站中还有多个空闲项，每个保留站每个时钟周期至多也只能接收一条指令。本质上来说，这个约束是由于保留站只有一个写端口造成的。

5.4.3 分派效率

所有缓冲的平均利用率参见表5.6。考察load/store单元中的三个保留站项，它对于整数测试程序平均使用1.36项到1.73项，对于浮点测试程序为0.98项到2.26项。与其他的执行单元不同，load/store单元不会在指令发射之后回收保留站项，而是一直保存到指令执行结束，一般在指令发射之后的两个周期，因为可能会在数据cache或者TLB中发生失效。浮点单元保留站项的利用率比整数单元的利用率要高。浮点部件顺序发射的限制以及有些浮点指令的非流水特性妨碍了指令的发射。完成缓冲的平均利用率对于测试程序为9~14并与正在执行的平均指令条数有关。

表5.6 缓冲的平均利用率

缓冲	*compress*	*eqntott*	*espresso*	*li*	*alvinn*	*hydro2d*	*tomcatv*
指令缓冲（8）	5.41	4.43	4.72	4.65	5.13	5.44	6.42
分派缓冲（4）	3.21	2.75	2.89	2.85	3.40	3.10	3.53
XSU0 RS入口（2）	0.37	0.66	0.68	0.36	0.48	0.23	0.11
XSU1 RS入口（2）	0.42	0.51	0.65	0.32	0.24	0.17	0.10
MC-FXU RS入口（2）	0.04	0.07	0.09	0.28	0.01	0.10	0.00
FPU RS入口（2）	0.00	0.00	0.00	0.00	0.70	1.04	0.89
LSU RS入口（3）	1.69	1.36	1.60	1.73	2.26	0.98	1.23
BRU RS入口（4）	0.45	0.84	0.75	0.59	0.19	0.54	0.17
GPR重命名缓冲（8）	2.73	3.70	3.25	2.77	3.79	1.83	1.97
FPR重命名缓冲（8）	0.00	0.00	0.00	0.00	5.03	2.85	3.23
CR重命名缓冲（16）	1.25	1.32	1.19	0.98	1.27	1.20	0.42
完成缓冲（16）	10.75	8.83	8.75	9.87	13.91	10.10	11.16

对于所有测试程序，其分派停顿原因参见表5.7，表中的数据表示占该测试程序执行周期总数的百分比。例如，在compress（24.35%）的执行周期中，没有发生分派停顿，即分派缓冲中所有的指令（前4项）都被分派。所有测试程序一个共同且主要的瓶颈在于保留站的饱和，尤其是load/store部件。对于其他引起分派停顿的原因，则根据测试程序的不同而不同。重命名缓冲饱和造成的停顿对于compress和tomcatv较为严重，尽管平均利用率还不到一半。完成缓冲饱和导致停顿的概率对于alvinn是最高的，因为在alvinn中，16个完成缓冲项的全部利用概率最高，参见表5.39（b）。在很多的测试程序中，对保留站单个写端口的竞争也会成为一个主要的瓶颈。

表5.7 分派停顿周期的频率

分派停顿	*compress*	*eqntott*	*espresso*	*li*	*alvinn*	*hydro2d*	*tomcatv*
串行化	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%
移至特定寄存器产生的限制	0.00%	4.49%	0.94%	3.44%	0.00%	0.95%	0.08%
读端口饱和	0.26%	0.00%	0.02%	0.00%	0.32%	2.23%	6.73%
保留站饱和	36.07%	22.36%	31.50%	34.40%	22.81%	42.70%	36.51%
重命名缓冲饱和	24.06%	7.60%	13.93%	17.26%	1.36%	16.98%	34.13%
完成缓冲饱和	5.54%	3.64%	2.02%	4.27%	21.12%	7.80%	9.03%
同一单元的另一种情况	9.72%	20.51%	18.31%	10.57%	24.30%	12.01%	7.17%
非分派停顿	24.35%	41.40%	33.28%	30.06%	30.09%	17.33%	6.35%

图5.4（a）给出了分派并行性的分布（每个时钟周期分派的指令数）。每个时钟周期分派的指令数从0到4。这个分布给出了每个时钟周期分配n条指令的频率（对于整个执行过程），其中$n = 0, 1, 2, 3, 4$。在所有测试程序中，有超过一半的执行周期并在相应的周期中至少分派了1条指令。

5.5　指令执行

620 在执行阶段的宽度将会显著增加。取指、分派、完成和写回阶段都是 4 指令宽度，即每个时钟周期可以处理最多 4 条指令，执行阶段则有 6 个执行单元，可以发射并且处理 6 条指令。而且，同其他段不同，在执行段将以乱序执行指令以达到最大的指令吞吐率。

5.5.1　发射停顿

指令一旦分派到保留站，必须等待源操作数就绪才可以开始执行。此外还有一些其他的限制。下面列出了全部可能阻碍发射的因素：

禁止乱序执行

指令从保留站流出时，一般情况下允许乱序执行，但是也会出现这样的情况，保留站中的某条指令不能超越前导指令的执行。例如，在分支部件和浮点部件中，指令必须顺序执行。

串行化限制

读写非重命名寄存器（如XER）的指令，或者以非重命名方式读写重命名寄存器（例如load/store multiple 指令）的指令，以及修改或者同步机器状态的指令（如 eieio 指令，强制 I/O 的按序执行），必须等待前面的指令执行完后才能够执行。这些指令停顿在保留站中直到它们的串行化要求被满足。

等待源操作数

保留站的主要任务是保存指令，直到它的源操作数可用。如果某条指令的源操作数不可用，该指令就在保留站中等待直到源操作数定向给它。

等待执行单元

有时，两条或者更多的指令将在同一时钟周期准备就绪。在这种情况下，第一条指令会被发射，第二条指令必须等待。当指令在MC-FXU（一个非流水部件）中执行，或者当浮点除法指令将 FPU 置于非流水的模式下时，也属于这种情况。

这 4 种停顿类型出现的频率参见表 5.8。表中列出的数据包括除分支单元外的所有执行单元，顺序执行的限制只适用于浮点单元。通常，发射串行化将导致停顿的数目正比于测试程序指令组合中多周期整数指令的数目。大多数的多周期指令会访问专用寄存器或者将整个条件寄存器作为一个非重命名单元进行访问，这就需要串行化机制。load/store 单元的大多数发射停顿是由于等待执行单元而引起的，就绪的load/store指令数量往往超过load/store单元所能处理的能力。对于所有测试程序，在执行的大部分周期里不会遇到发射停顿。

表 5.8　停顿类型出现的频率

停顿类型	*compress*	*eqntott*	*espresso*	*li*	*alvinn*	*hydro2d*	*tomcatv*
禁止乱序执行	0.00%	0.00%	0.00%	0.00%	0.72%	11.03%	1.53%
串行化	1.69%	1.81%	3.21%	10.81%	0.03%	4.47%	0.01%
等待源操作数	21.97%	29.30%	37.79%	32.03%	17.74%	22.71%	3.52%
等待执行单元	13.67%	3.28%	7.06%	11.01%	2.81%	1.50%	1.30%
非停顿类型	62.67%	65.61%	51.94%	46.15%	78.70%	60.29%	93.64%

5.5.2　并行执行

在本小节中将讨论指令从发射到执行、完成的过程。图 5.4（b）给出了并行发射的分布状态（每个时钟周期发射的指令条数）。每个时钟周期能够发射的最大指令数为6，也就是执行单元的数

目。虽然发射并行度和分派并行度应该有相似的平均值，但发射更加不集中化并且包含更少的限制，因而可以得到一个更加稳定的发射效率。在大多数周期中，发射指令的数目和总的IPC相近，而分派并行度在分布上则有较大的极端性。

下面研究一下结束并行度，参见图5.4（c），它与发射并行度很相似，因为指令发射之后在一定的时钟周期之后就会结束执行。但是对于eqntott, alvinn 和 hydro2d 这些测试程序，发射的并行度分布和结束的并行度分布就不尽相同，因为程序中有数量较多的load/store指令和浮点指令，这些指令和整数指令在发射之后执行所需的周期不同，发射并行度的分布有所平移。在整数测试程序中，指令的执行延迟则比较一致，并在发射并行度分布和结束并行度分布之间存在很大的相似性。

5.5.3 执行延迟

研究一条指令从分派到执行结束的平均延迟是很有必要的。如果所有的发射条件都满足并且执行单元可用，指令可以从分派缓冲分派到保留站中，然后在下一个时钟周期发射到执行单元执行。这是最好的情况。通常，指令必须在保留站中等待，总的执行延迟包括保留站中的等待时间和执行单元的实际延迟。表5.9给出了6个执行单元在执行测试程序时总的平均执行延迟。

表5.9 6个执行单元的平均执行延迟

执行单元	*compress*	*eqntott*	*espresso*	*li*	*alvinn*	*hydro2d*	*tomcatv*
XSU0（1个段）	1.53	1.62	1.89	2.28	1.05	1.48	1.01
XSU1（1个段）	1.72	1.73	2.23	2.39	1.13	1.78	1.03
MC-FXU（>1个段）	4.35	4.82	6.18	5.64	3.48	9.61	1.64
FPU（3个段）	—a	—a	—a	—a	5.29	6.74	4.45
LSU（2个段）	3.56	2.35	2.87	3.22	2.39	2.92	2.75
BRU（>=1个段）	2.71	2.86	3.11	3.28	1.04	4.42	4.14

5.6 指令完成

指令一旦结束执行，就进入完成缓冲并等待按序完成操作和写回操作。完成缓冲功能就像一个再定序缓冲，将在执行段乱序执行的指令进行重新排序，从而得到按指令顺序完成的顺序。

5.6.1 完成并行度

所有测试程序的完成并行度分布参见图5.4（d）。和分派一样，每个时钟周期可以完成最多4条指令。每个并行度分布都可以得到平均值。实际上，对于每种测试程序，平均完成并行度应该和平均分派、发射、结束并行度相同，并且都等于该测试程序总的IPC。虽然指令允许乱序结束，但是指令完成必须按序进行，这意味着有时指令必须在完成缓冲中等待一条执行很慢的指令结束，但此后就可以一次提交4条指令。在某些情况下，完成缓冲将会饱和，从而导致分派段的停顿，参见图5.3（b）。

指令执行延迟较为一致的整数测试程序一般每个时钟周期提交1条指令。hydro2d在大部分的时钟周期内不提交任何指令，因为必须等待浮点除法指令结束。一般情况下，指令不能完成是因为它本身或者它前面的指令还没有结束，但也会有其他的原因。620有4个整数写回端口和两个浮点写回端口，因此很少出现整数写回端口不够的情况，但是，浮点写回端口不够的情况则时有发生。

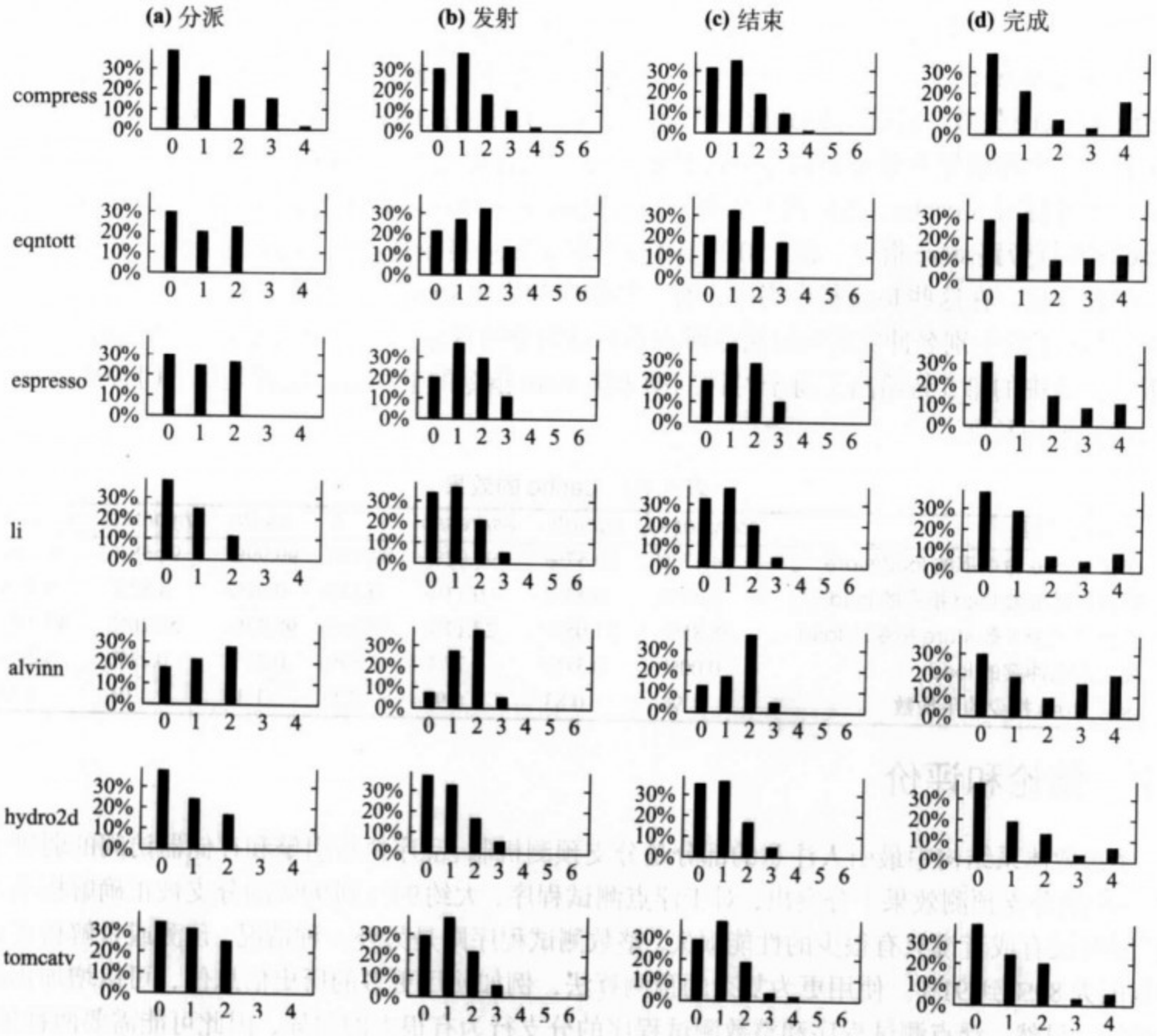


图 5.4　4 种指令的完成并行度分布

5.6.2　cache 的影响

数据 cache 的行为对于 CPU 的性能有着直接的影响。cache 失效会导致执行段和完成段额外的停顿。620 中数据 cache 是 2 体交叉的组织结构，每个体都有地址端口。load 指令或者 store 指令可以使用任何一个端口。cache 可以同时处理一个 load 操作和一个 store 操作。一条 load 指令和一条 store 指令如果访问不同的体，则可以在一个时钟周期内进行访问。对于读所访问的端口，cache 是非阻塞的。当发生读失效并进行 cache 块调入的时候，后继的 load 指令可以继续访问 cache。如果这次访问命中 cache，该指令则不受前面失效的影响而继续执行；否则，指令将返回保留站并保存起来。load/store 部件的保留站有多个项且指令可以乱序发射，因此可以在前面 3 条 load 指令访问 cache 都失效的情况下，处理一条在 cache 中命中的 load 指令。

存储器的顺序一致性模型要求所有的 store 指令必须以非推测地顺序加以执行。然而，如果所有的访存指令都以串行方式执行，性能则会受到很大影响。620 按序执行所有在完成段之后访问 cache（使用物理地址）的 store 指令，但允许在执行段访问 cache（使用虚地址）的 load 指令旁路 store 指令。当 store 指令完成之后，会检查 store 的地址和旁路的 load 指令（已经结束）地址是否存在别名冲突。如果发现别名，下一条 load 指令检查提交时将清空机器并重新对该 load 指令取指。未决 store 指令到相关的 load 指令之间没有数据定向。存储器访问弱排序可以消除某些不必要的停顿。大多数 load 指令是相关链的起始点，因而尽早执行 load 指令可以使得其他指令的执行提前。

表5.10总结了非阻塞cache的效果和load/store指令弱排序的效果。表的第1行给出了所有测试程序的数据cache命中率，命中率从94.2%到99.9%。鉴于cache的非阻塞特性，如果前面的load指令导致了cache失效，后面的load指令若命中就可以旁路前面的load指令。表中的第2行显示了后续load指令旁路前导失效load指令所占总load指令的百分比。当store指令完成后，将进入提交存储队列中等待写入cache，然后离开队列。当未决store指令在队列中等待时，可能会有load指令访问cache并且旁路store指令。表5.10的第3行列出了load指令在访问cache时旁路了至少1条store指令的百分比。在这些load指令当中，有一些指令和未决store指令的地址存在别名冲突，表中的第4行给出了发生别名冲突的load指令所占总load指令的百分比。大部分测试程序的别名问题都不是很多。表中的第5行给出了每个时钟周期未决store指令的平均数，也就是每个时钟周期提交存储队列的平均长度。

表5.10 cache的效果

cache的效果	*compress*	*eqntott*	*espresso*	*li*	*alvinn*	*hydro2d*	*tomcatv*
包含cache命中率的load/store	94.17%	99.57%	99.92%	99.74%	99.99%	94.58%	96.24%
旁路前导失效load指令的load	8.45%	0.53%	0.11%	0.14%	0.01%	4.82%	5.45%
旁路了至少1条store指令的load	58.85%	21.05%	27.17%	48.49%	98.33%	58.26%	43.23%
发生别名冲突的load	0.00%	0.31%	0.77%	2.59%	0.27%	0.21%	0.29%
未决store指令的平均数	1.96	0.83	0.97	2.11	1.30	1.01	1.38

5.7 结论和评价

620微体系结构中最引人注意的部分是分支预测机制、乱序执行引擎和存储器访问的弱排序模型。620的分支预测效果十分突出，对于浮点测试程序，大约94%到99%的分支被正确解析或者预测，同时没有或者说只有很少的性能损失。整数测试程序则是另外一种情况，预测或者解析正确率的范围为89%到91%。使用更为复杂的预测算法，例如使用更多的历史信息位，可以增加预测的正确性。显然，浮点测试程序和整数测试程序的分支行为有很大的差异，因此可能需要两种独立、不同的分支预测机制来分别解决两种预测。

虽然必须支持精确中断，620的乱序执行引擎还是开发了相应的指令级并行度，IPC的范围对于整数测试程序为0.99到1.44，对于浮点测试程序为0.96到1.77。620讨论的一个热点是load/store部件，load/store部件保留站的项数还需要增加，或者增加load/store部件的数目。尽管设计一个有多个load/store部件的系统难度很大，我们还是不能忽视load/store部件这个620中显而易见的瓶颈。620中有3个整数单元但只有1个浮点单元，这也是一个瓶颈。整数测试程序很少由于等待整数单元而停顿，但是浮点测试程序会因等待浮点单元而停顿。每周期只能向每个保留站分派一条指令也是分派停顿的一个原因，这使得乱序执行得不到充足的指令。选择620采用的分布式保留站还是Intel P6中的集中式保留站是一个有益的折中方案。前一种方法因为使用简单的单端口保留站使得硬件简单；后一种方法中不同的指令类型之间则可以共享多个端口和保留站。

在现代宽超标量处理器中，实现存储器访问弱排序的模型对于取得高性能是很有必要的。620允许load指令旁路store和其他的load指令，但是并不支持从未决store指令到相关load指令之间的数据定向。620直到store指令完成时才检查和load指令的别名冲突，这种以store为中心的方法使定向变得十分困难，当遇到别名时，需要从相关的load指令开始清空机器。620实现了两体交错的数据cache，在没有体冲突的时候，同一个时钟周期内可以处理一个load指令和一个store指令。根据动态指令混合比例的标准规则，在当前超标量微处理器中，处理load/store指令时有很大的不平衡性，增加load/store指令的吞吐率是现在最严峻的挑战。随着未来超标量处理器执行宽度的增加、

时钟频率更高，存储器的瓶颈问题将会更加严重。而且商业应用，例如事务处理（没有被SPEC 测试程序包含在内）也将对存储器和芯片 I/O 这两个瓶颈施加的压力更大。

这里有必要介绍一下与620同时代的、由其他公司设计的超标量微处理器，并研究一下不同的微处理器系列是怎样发展的，参见图5.5。PowerPC 微处理器和 Alpha AXP 微处理器代表了超标量处理器获得高性能的两种不同的解决方案，分别为“智慧型”（ brainiacs ）和“速度型”（ speed demon ）PowerPC体系结构不通过过度地增加时钟频率来获得最高的IPC，Alpha AXP微处理器则通过实现最高的时钟频率来得到一个可以接受的IPC。当然，PowerPC将来的芯片时钟频率同样会更快，将来的 Alpha AXP 芯片也会有更高的IPC。问题的关键在于，哪个因素应该更优先，IPC还是时钟频率？哪种方法是达到下一级性能的捷径？尽管从现在各种版本的 PowerPC 和 Alpha AXP 21164 微处理器看来，“速度型”略有胜出，但对于未来的趋势还是没有确定的答案。从某种角度上看，这两种方案的竞争更像是十几年前的 CISC 和 RISC 之争。

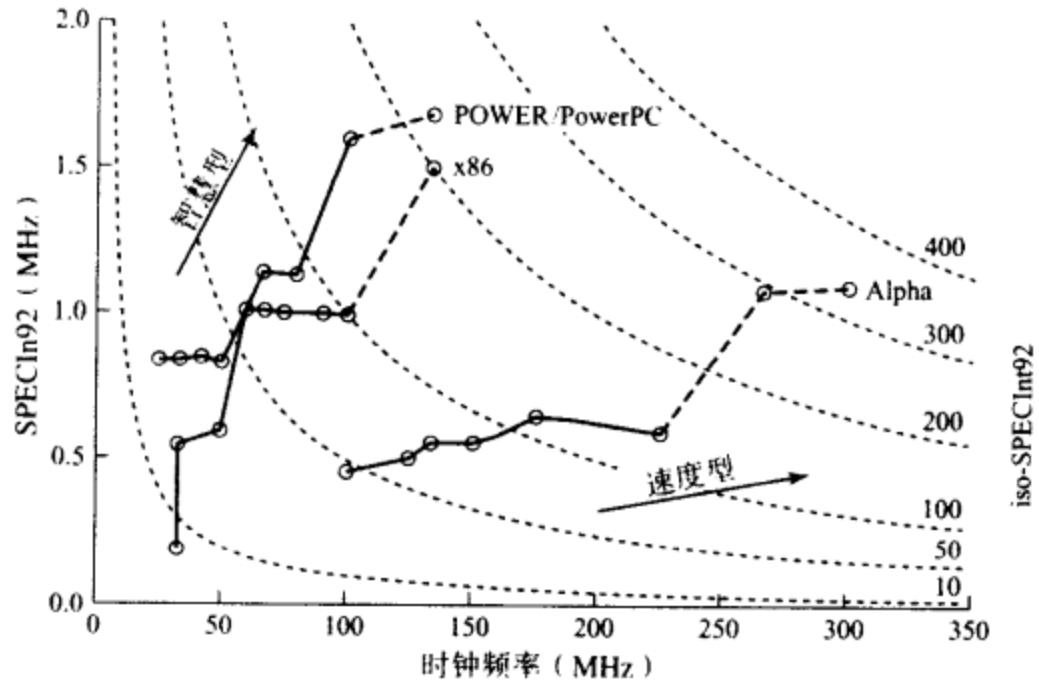


图 5.5　超标量微处理器的演化史

Intel 发布的 P6 展示了另外一种有趣的情况。为了得到较高的 IPC，P6 在微体系结构设计上所做的工作和620相当。但另一方面，P6又和21164有某种类似，都采用了比620更深的超流水技术。P6代表第3种，或者说是一种混合的方法来实现高性能。图5.5展示的是20世纪90年代中期的一个大概情况，但从那以后时钟频率已经大大提高了。现在我们已经不再使用SPECint92 测试程序，而是使用 SPECint2000，目前的时钟频率是若干 GHz。

5.8　IBM POWER3 和 POWER4

PowerPC 620 是 PowerPC 体系结构中第一次高端64位实现，主要面向服务器和高性能工作站市场。然而，由于设计实现上的巨大困难，该项目推迟了几年才完成，因此仅仅只能用在 Group Bull 开发的少数服务器系统上。与此同时，IBM组织独立的设计小组开发了和PowerPC 620有本质区别的 POWER2 微处理器以及 Star 系列服务器来打开服务器市场。

但1998年发布的POWER3处理器则在很大程度上受到了PowerPC 620设计的影响，并重用了620的流水线的结构和它的很多功能单元。表5.11总结了620和POWER3处理器之间一些关键的不同点。设计优化和若干年来半导体工艺的进步使得处理器的频率几乎为原来的3倍，尽管采用了类似的流水线结构，然而其性能还是有了很大的提高。

表 5.11 PowerPC 620、IBM POWER3 与 POWER4 之间的比较

属性	620	POWER3	POWER4
频率	172 MHz	450 MHz	1.3 GHz
流水线长度	5+	5+	15+
分支预测	Bimodal BHT/BTAC	Same as 620	3 × 16 K 组合
取指 / 发射 / 完成的带宽	4/6/4	4/8/4	4/8/5
重命名 / 物理寄器	8 Int, 8 FP	16 Int, 24 FP	80 Int, 72 FP
正在执行的指令	16	32	Up to 200
浮点单元	1	2	2
load/store 单元	1	2	2
指令 cache	32 KB 8-way SA	32 KB 128-way SA	64 KB DM
数据 cache	32 KB 8-way SA	64 KB 128-way SA	32 KB 2-way SA
L2/L3 尺寸	4 MB	16 MB	~1.5 MB/32 MB
L2 带宽	1 GB/s	6.4 GB/s	100+ GB/s
store 队列入口	6 × 8B	16 × 8B	12 × 64B
MSHR	I : 1/D : 1	I : 2/D : 4	I : 2/D : 8
硬件预取	None	4 streams	8 streams

POWER3 通过增加了指令执行的宽度和存储器的带宽，解决了 PowerPC 620 的一些缺点。尽管流水线的前端和后端保持同样的宽度，POWER3 将峰值发射宽度增加为每个时钟周期 8 条指令，并提供两个 load/store 部件和两个全流水的浮点部件。同时，完成缓冲增加为 32 项，有效窗口的大小变为原来的 2 倍并且整数重命名寄存器的数目加倍，浮点重命名寄存器的数目变为 3 倍。另外，存储器带宽进一步增加，同时使用了创新的 128 路组相联 cache 设计，将标签匹配硬件嵌入到两个 L1 cache 的标签阵列中，这极大地降低了失效率并且数据 cache 的容量也实现加倍。L2 cache 是片外 cache，因此容量也增加了很多。通过使用一个硬件预取引擎大大地减少了存储器访问延迟，该引擎可以检测4个独立的访问流并将它们从存储器中预取出来。这种预取机制对于访问模式较为规律、可预测性好的浮点负载效果很好。通过给指令 cache 提供两个失效状态处理寄存器（MSHR），可以给数据 cache 提供 4 个 MSHR，可以支持多个单独的 cache 失效。

PowerPC 家族的下一款高性能处理器是 POWER4 处理器，于 2001 年推出。对于这种全新的设计方案，其关键属性如表 5.11 所示。IBM 又一次将处理器主频提高了 3 倍，这次是通过更深的流水线结合更为先进的工艺技术实现的（例如，减少特征尺寸、铜互联和 SOI 技术）。POWER4 的流水线如图 5.6 所示，最好情况下的单周期整数 ALU 指令流水线也扩展到了 15 级。为了使这条流水线有足够的指令执行，POWER4 实现了一种组合的分支预测器，并使用 16 K 项的选择表在 16 K 项的 bimodal 预测符和 16 K 项的 gshare 预测符之间进行选择。3 个表中的每一项都只有 1 位，而不是 2 位的加减计数器。研究表明，16 K 的 1 位项的性能要好于 8 K 的 2 位项。对于 POWER4 所面向的服务器工作负载，分支预测符的容量失效比两位计数器所提供的充分信息更重要。

POWER4 和 POWER3 在执行宽度上是一样的，但是 POWER4 提供了更多的重命名寄存器（单个物理寄存器文件的形式），流水线中最多可同时存在 200 条指令。与 POWER3 一样，存储器带宽和延迟将被着重考虑并设置了多个 load/store 部件，最多支持 8 个单独的 cache 失效；同时还实现了高带宽的片上 L2 cache 以及大容量的片外 L3 cache，并作为一个整体来提高总的性能。POWER4 将两个完整的处理器核封装在一起并共享 L2 cache，从而形成一个单芯片多处理器结构。关于这方面的细节请参考第 9 章。

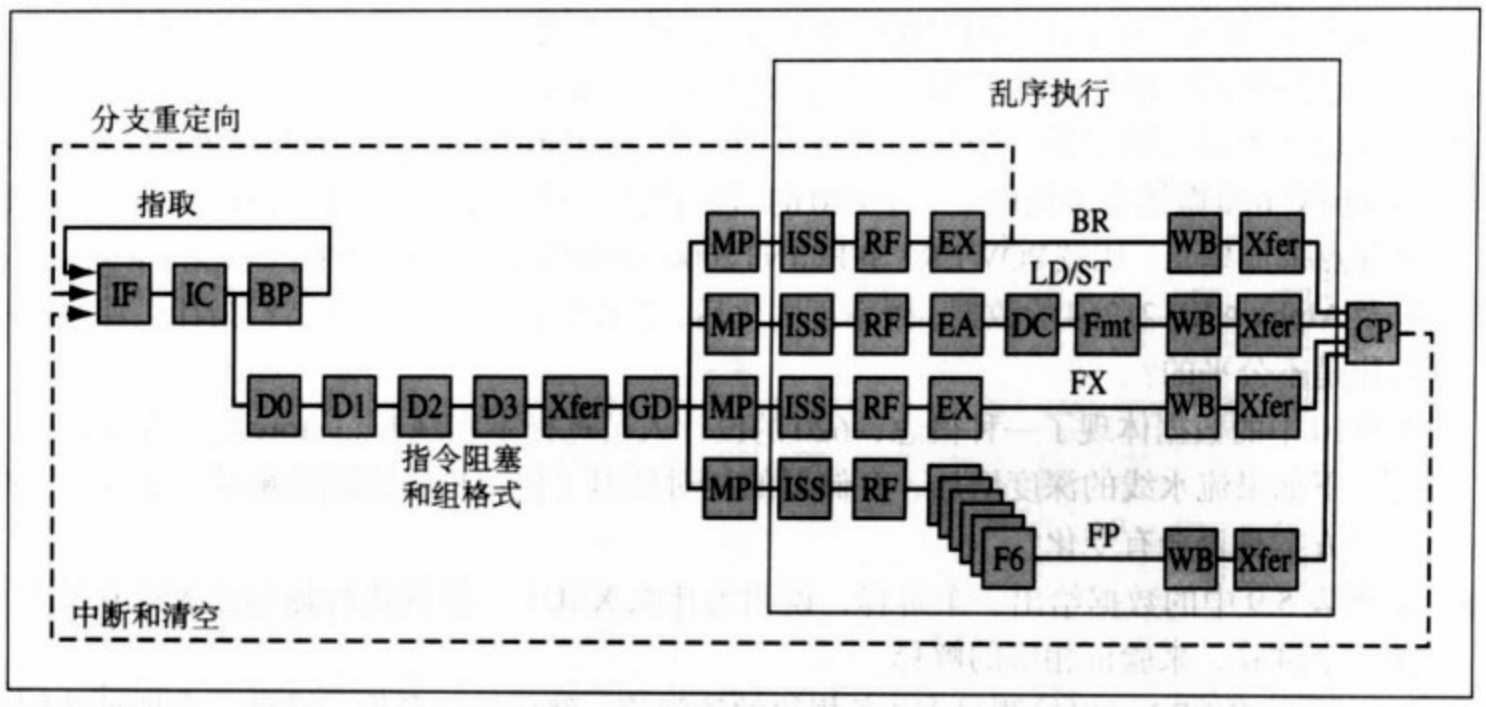


图 5.6　POWER4 流水线结构

5.9　小结

PowerPC 620是一款很有研究价值的第一代乱序执行超标量处理器，其特点是流水线短并支持指令级并行性以及存储器访问的弱排序，这对于其他类似的微处理器（例如，HP PA-8000 和 MIPS R10000）都是很关键的技术。这些技术在 POWER3 中有所发展并通过扩展执行资源来支持更大的并行性，同时解决了存储器带宽和延迟的瓶颈问题。最近的 POWER4 设计方案说明了现代处理器为了克服存储器带宽和延迟，不惜一切代价使用了片上或者片外的cache存储层次、流预取硬件以及大指令窗口等技术。同时，POWER4 说明了通过设计更深的流水线来加快时钟频率的趋势。在这种情况下，只有设计更为精确的分支预测算法才能保证流水线时刻都有足够的指令执行。

5.10　习题

1. 假设使用第 2 章的 IBM 指令组合，考虑 PowerPC 620 的完成缓冲和整数重命名缓冲的设计是否平衡。假设 load 指令和整数 ALU 指令需要一个整数重命名缓冲，其他指令则不需要。如果 620 设计不平衡，那么应该设置多少个重命名缓冲?
2. 假设使用表5.2中的指令组合，哪些测试程序将会受限于重命名缓冲？即哪些测试程序在用完完成缓冲之前先用完重命名缓冲？相反的情况下又是哪些测试程序？
3. 假设分派和提交的带宽一定，整数 ARF需要多少个读端口和写端口才能保证峰值吞吐率？假设使用表5.2中的指令组合，计算每个测试程序所需要的平均端口数目。解释为什么不设置平均情况下的端口数目。假设给定读端口和写端口的数目，分派受端口限制的可能性有多大？提交受端口限制的可能性多大？
4. 假设分派和提交带宽一定，整数重命名缓冲需要多少个读端口和写端口？假设使用表5.2中的指令组合，计算每个测试程序需要的平均端口数。解释为什么不设置端口数为平均情况下的端口数目。
5. 比较 PowerPC 620 BTAC 的设计与 Alpha 21264 的“next-line/next-set”预测符的设计，后者的描述参见 Alpha 21264 微处理器硬件参考手册（可以从 www.compaq.com 获得）。
6. 如果将 620 的设计更换为近期的设计方案并使用更深的流水线（例如，类似于 Pentium4 的 20 级流水线），表 5.5 中的结果将会怎样?

7. 根据表5.7判断，对于PowerPC 620的保留站，其配置是不够的。如果将保留站的数目加倍，对于这些测试程序会得到多大的性能提高？验证给出的答案。
8. 620设计中最为明显的一个瓶颈就是只用一个load/store部件。IBM的POWER3是基于PowerPC 620微体系结构的、一个620的后继设计，同时加入了第2个load/store部件和第2个浮点乘加单元。比较POWER3 SPECINT2000和SPECFP2000的数据（参见www.spec.org）以及Alpha AXP 21264的数据，频率正则化后，哪种处理器的分数更高？为什么使用频率正则化是不公平的？
9. 表5.10中的数据体现了一种概念，620的设计人员选择不实现load/store定向是正确的。讨论一下如果流水线的深度增加、存储器的相对延迟（使用机器周期数衡量）增加，这样的折中方案是否会有变化？
10. 根据表5.9中的数据给出一个解释，说明为什么XSU1一般的执行延迟比XSU0要大。设计一个试验，来验证相应的解释。
11. IBM的POWER3可以检测最多4条规律的访问流，然后进行预取。设计一个地址访问序列使其能够使用4个访问流。
12. IBM的POWER4可以检测最多8条规律的访问流，然后进行预取。设计一个地址访问序列使其能够使用8个访问流。
13. POWER3和POWER4的流预取是在处理器核之外进行的，使用在L1 cache中失效的cache块物理地址进行预取。解释为什么较大的虚拟存储器页能够提高预取策略的效率。
14. 假设一个程序顺序地扫描1 GB的数组，每次读取1 GB数组中以8字节对齐的浮点双精度数据，并假设预取引擎会在连续3个cache块都在L1 cache中失效后启动并将第4块预取进来。假设页大小为4 K，使用POWER3和POWER4中cache块的大小，计算该程序在没有预取的情况下总的失效率、在使用物理地址预取或虚拟地址预取情况下总的失效率。假设对每个8字节的字都有1次访问，报告每次L1数据cache访问的失效率。

第6章　Intel P6微体系结构

1990年，Intel开始研究新的IA32微体系结构，即P6。1995年P6的产品问世后，命名为Pentium Pro™处理器，在工作站和服务器系统上大受欢迎。Pentium II处理器是P6内核在桌面系统上的延伸，发布于1997年5月，Pentium II在基本的P6内核中加入了MMX指令集。1998年发布了基于P6的Pentium III处理器并扩展了MMX指令集和SSE指令集。在本章中P6指的是内核，而具体的产品则用产品名字来表示。

P6是一个与IA32兼容的高性能处理器体系结构，具有超流水动态执行的特征。它是一个第3级的超标量结构，围绕微数据流执行内核并使用乱序执行和推测执行技术。P6包含非阻塞cache和基于事务的监测总线。本章将描述P6的各个组件以及它们之间的组合方式。

6.1　简介

P6微体系结构的基本结构如图6.1所示，主要由三个部分组成：前端、中间部分和后端，前端必须按序执行，中间部分允许乱序执行，后端进行指令提交（retirement）处理，同时也必须按序执行。为了与IA32兼容，处理器在执行程序代码时必须遵循一定的规则。但是为了获得更高的性能，必须对一些限制进行放松，例如程序的执行应该严格按照程序自身所规定的顺序。执行时必须检测真数据相关，但除此之外，只需要保障某些访存顺序限制和IA32的精确故障语义。

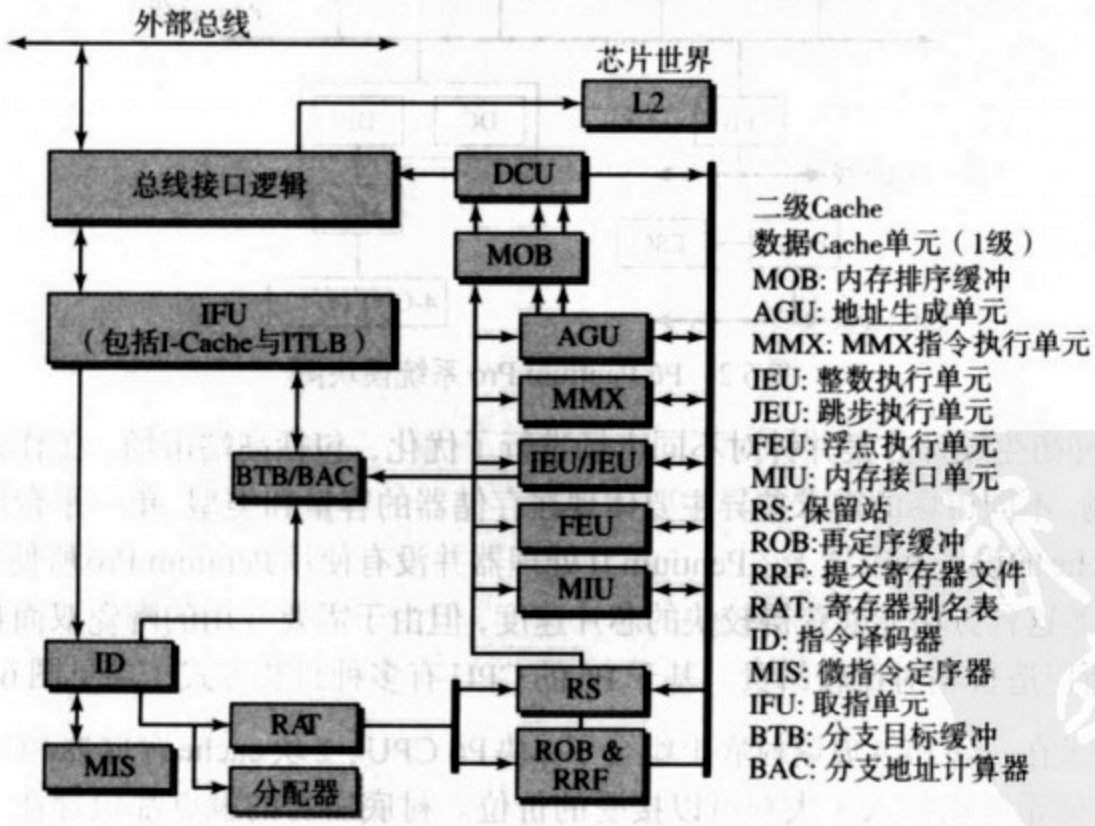


图6.1　P6微体系结构模块图

为了保证精确故障语义，处理器必须要确保对一些异步事件（如中断）和一些同步事件（如故障和自陷）的处理要完全按照i486系统的处理方式进行[①]，因此需要顺序提交，即将指令结果强行

① 分支和故障都使用相同的机制进行状态恢复。但是出于性能的考虑，分支恢复要尽早清除并重新启动流水线前端页故障是采用推断处理的，处理浮点故障时，只有当机器确定发生故障的指令处于特定的路径上时才进行相应的处理。

按照原始程序的顺序进行确认，再更新机器状态。通过流水线两端的按序执行机制，除了真数据相关和机器资源的限制以外，指令的实际执行可以不受任何其他约束。我们将在本章的后续部分探讨这三个部分的设计细节。

P6微体系结构包含许多新颖的方面。例如，在常规设计中，处理器应该有一个中央控制单元，监测和控制整个流水线。这个控制单元能够“理解”所有通过流水线的指令状态，同时还控制计算过程中所有状态的改变。但是P6微体系结构却故意避开了这一点，它没有中央控制单元。为了尽量简化处理器中的硬件，P6微体系结构将IA32指令转变成更简单的原子操作并称之为“微操作”（μop）。微体系结构改变机器状态的惟一方法就是执行这些微操作。

相对于同时代与之竞争的设计，P6微体系结构具有很深的流水线，由几段较短的流水线通过队列连接而成。这种方法提供了很高的时钟频率，深度流水的负面影响可以通过一系列的方法来解决，如采用更为先进的分支预测技术、高带宽快速访问的L2 cache以及更高的时钟频率（在给定半导体工艺的情况下）。

P6微体系结构有一个基于推断的乱序执行内核。任何基于推断的执行都可能产生错误，所以必须提供检测及恢复的方法。为了保证推断执行及错误恢复机制没有设计错误，恢复机制设计得很简单，简单的机制能得到更好的验证。因此，机器的事件处理（故障，自陷，中断，断点）使用相同的协议和机制。

P6的前端总线设计来自于Intel Pentium系列处理器，但进行了相应的改变。P6的前端总线主要是面向事务的并针对高性能多处理器系统设计的。图6.2显示了4个Pentium Pro处理器是如何连接到单条总线上的。P6提供给主存的芯片组包括数据通路（DP）、数据控制器（DC）以及存储器接口控制（MIC）芯片。P6提供了工业标准的PCI总线来支持I/O，以及一个连接早期EISA标准总线的桥

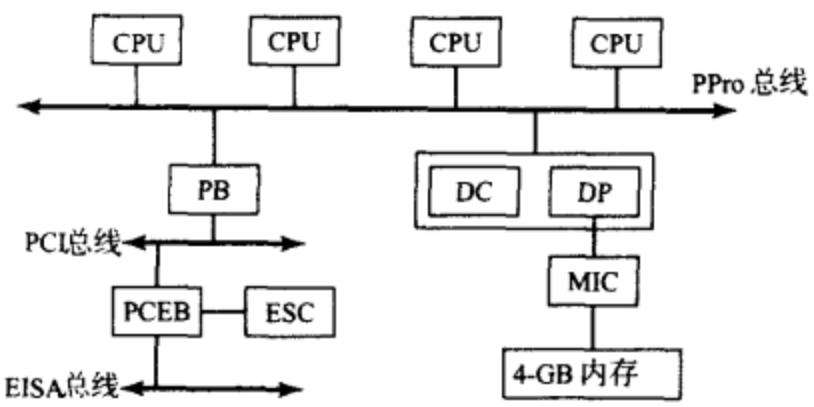


图 6.2 P6 Pentium Pro 系统模块图

P6包含各种衍生设计方案并针对不同市场进行了优化，包括高端市场、工作站市场、服务器市场和移动市场。不同市场的需求差异主要体现在存储器的容量和类型、单一平台所能支持的CPU数量以及L2 cache的设计和位置上。Pentium II处理器并没有使用Pentium Pro所使用的“两芯片封装”的方法，虽然这种方法能够获得较快的芯片速度，但由于需要专用的陶瓷双面封装而且制造步骤比较特殊，所以造价相对比较昂贵。基于P6的CPU有多种封装方式（参见图6.3）：

- Slot 1通过在一块FR4玻璃衬底上组合了1块P6 CPU、2块cache存储器和1个标识存储器，将P6微体系结构带入了大众可以接受的价位。衬底所有的触点都包含在一个边沿连接器（edge connector）中，在衬底封装的另外一侧为散热片。Slot 1的L2 cache的时钟频率为CPU频率的一半。
- Slot 2的物理尺寸比Slot 1大并允许4个SRAM来组成很大的cache，以供高性能工作站或服务器系统使用。Slot 2的盒式设计非常高效，因此尽管在L2总线上的请求数目有所增加，仍然能以CPU的时钟频率访问大容量的L2 cache。

- 随着硅处理技术的发展，1998 年 Intel 对 Celeron 处理器重新使用 PGA 封装，CPU 自身包含 L2 cache，消除了对 Pentium Pro 的“两芯片封装”和 Slot 1/Slot 2 的需求。

图 6.3　P6 产品封装

6.1.1　P6 微体系结构基础

在接下来的几节中，我们将讨论 P6 微体系结构中不同组件的操作。但是在此之前，有必要先从较高层次上考虑一下处理器的整体组成。

考察 P6 微体系结构的一种有效方法就是将它作为数据流处理器来看待，该数据流处理器包含一个“主动”的前端，但受到实现和代码兼容性的约束。设计一个实现指令级并行的微体系结构并不困难，加入多个执行单元也非难事，但真正的难点在于，如何能够充分利用这些执行单元。

对于这一问题，P6 的解决方法是：

- 通过在流水线的前端（指令 cache、译码器和寄存器重命名所在的部分）进行深度推断获得开发的并行性。
- 提供足够的临时存储单元，以便于中间结果可以暂时保存。
- 允许已经就绪的指令跳过其他没有就绪的指令执行（在乱序执行部分）。
- 提供足够的存储带宽，使之不至于成为瓶颈。

当推断执行沿着正确的路径前进时，前端产生的微操作将会顺利地进入保留站（RS, Reservation Station），在它们所需要的操作数据准备完毕时开始执行（其顺序一般与程序中的顺序不一样），并在再定序缓冲（ROB，Reorder Buffer）中分配一个位置，然后顺序提交。

微操作带有调度、分派、执行和提交时所需要的信息。微操作包含两个源操作数、一个目的操作数以及一个操作类型域。操作数使用逻辑地址，逻辑地址通过寄存器别名表（RAT, Register Alias Table）被重命名到 ROB 中的物理寄存器上。

当发生错误的推断之后[①]，乱序执行的内核将运行一个非常简单的协议。此协议确保内核清除已经知道发生了错误的推测状态，并保留所有还不知道是否出错的工作。同时，该协议要求前端放弃正在进行的工作，而从正确的分支地址处重新启动执行。

存储器操作是一类特殊的微操作。由于 IA32[②]指令集体系结构的寄存器非常少，IA32 程序必须频繁地访问存储器。这就意味着程序中的相关链（一组相关操作）通常是以一个存储器读操作开始的，这也就意味着预取机制非常重要（如果不这样做，基于推测执行的处理器的其他部分都将因为等待 load 操作的结果而被“饿死”）。但是并不是所有的 load 操作都能够推测。当一个 I/O 映射到存储器的系统中时，load 操作是不可恢复的。在 6.6 节中将讨论一些特殊的例子。store 操作不能进行推测，因为如果发生了 store 推测错误，没有任何办法恢复以前的数据。但是，出于性能上的考

① 例如，在第一次遇到一个分支时，分支预测器并不知道这里有一个分支，更不知道分支将会执行到什么地方。

② 在本章中，IA32 指标准的 32 位 Intel 体系结构，很多处理器采用了这种体系结构，如 Intel 486，Pentium，Pentium II，Pentium III，Pentium 4 处理器。

虑，store可以利用存储缓冲（SB，Store Buffer）进行载入定向，在数据真正写入cache或存储器之前，允许执行相关的load操作（以及它们的后续操作）。这与写回cache非常类似，可以在数据还没有写回到存储器之前从写缓冲中直接读取。

根据IA32编码的语义，从基于推断的乱序执行内核到机器状态的信息更新必须严格加以控制。我们将这种控制称为“提交”（retirement）。实际上，提交以前的所有动作都可以撤销，指令提交是一个不可恢复的确认动作。P6微体系结构每个时钟周期可以提交3个微操作，因此可以对3条IA32指令进行确认。如果IA32指令可以分解为多于3个微操作，提交过程必须确认原子性是否得到满足。

6.2 流水线

P6微体系结构的各个组件将在本章下一节介绍。在讨论各个部分之前，我们先来研究一下它们是如何结合在一起的。图6.4所示的流水线有助于我们深入理解其中的各个部分。

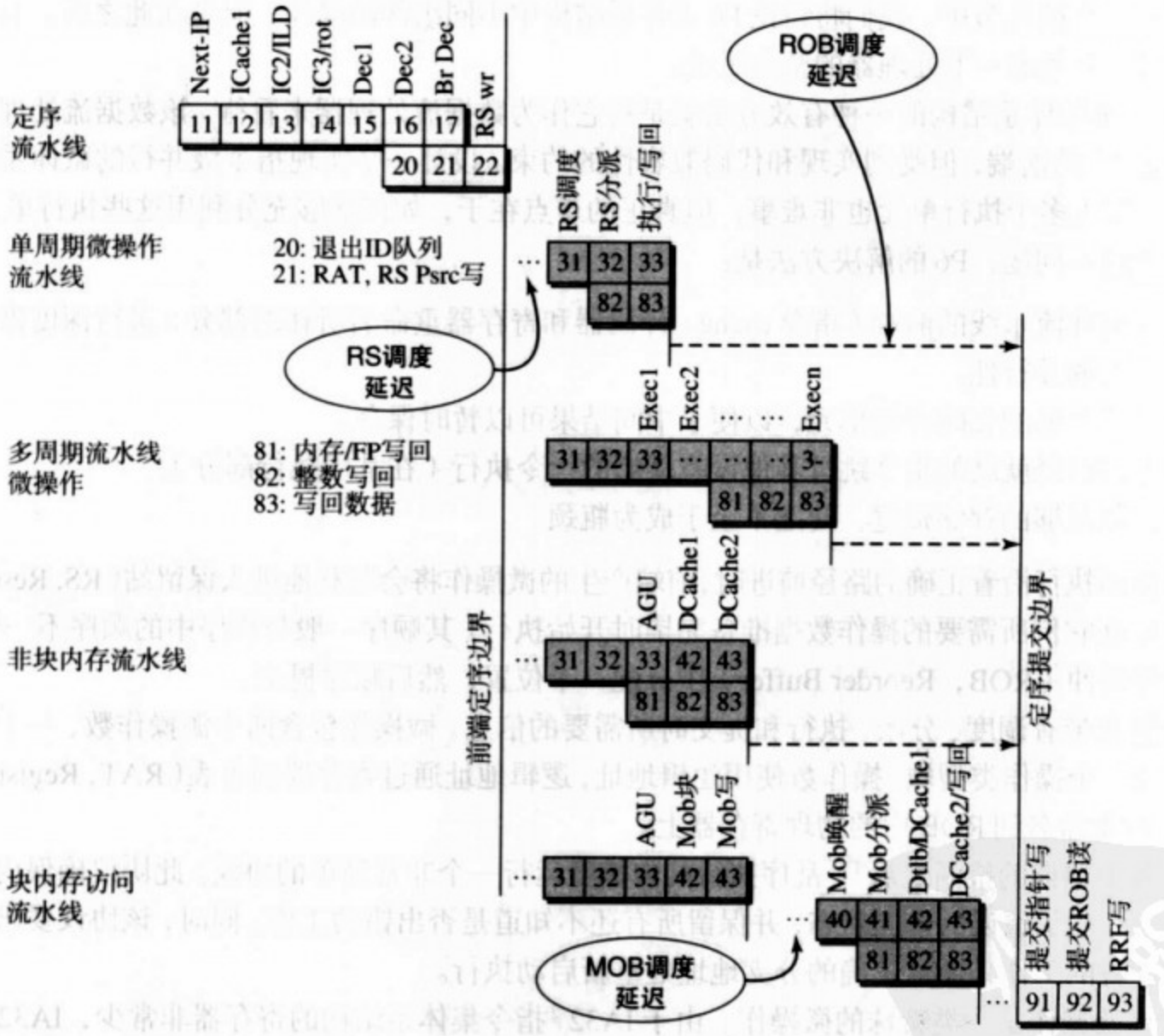


图6.4 P6流水线

关于图6.4，首先要注意的是其中有多条单独的流水线，而不是一条流水线，这是刻意安排的，它反映了P6微体系结构的设计哲学。

流水线分为三个部分：首先是按序执行的前端，其次是乱序执行的内核，最后是按序提交段。这些流水线单元在操作上相互独立，其原因将会在以后的章节中详细叙述。例如，当从错误推测的分支中进行恢复时，处理器的前端将立刻清空所有虚假的信息，这些信息均是由于错误推测而造成的，并且前端将从正确的分支目标处重新取出新的指令，此时内核仍然在对先前所取出的指令进行处理（直到遇见错误推测的指令）。

将流水线的整体分割成若干独立的部分同样十分重要，当各种各样的队列被填满时，将需要队列的填充者暂停并且直到队列排空。这时对于整体而言，仅有必要的部分才暂停，而不是全局的停顿。

6.2.1　按序执行的前端流水线

分支目标缓冲（BTB，Branch Target Buffer）使用按序前端的第一段（流水段11[1]）产生指令cache的索引，以访问那些正确的指令集合。但是，处理器总是在推测执行的，这种推测有可能发生错误。如果出现错误，那么处理器的某些地方将会对这种错误进行纠正，同时启动错误推测的恢复序列。

按序前端的第二段，即流水段12，从流水段11的BTB产生的地址处初始化Icache取指操作。第三段，即流水段13，继续访问指令cache。第四段，即流水段14，指令cache将完成取指并将新取出的cache块传递到指令译码器（ID），此后译码器开始译码。

ID使用流水段15和流水段16来对齐指令字节，确认最多3条IA32指令的边界并将这些指令分割成相关的微操作序列。

流水段17是ID的一部分，能够检测出刚被译码的指令中是否存在分支。在一定的条件下（例如，一个未预测的无条件分支），ID会发现BTB没有预测的分支（可能是因为BTB以前从来没有碰到过这条分支指令），这时可以清空流水线前端并从分支目标处重新取指，而不必等到多个时钟周期之后分支指令试图提交的时候再处理。

流水段17与流水段21是同名的[2]，流水段21是重命名段。寄存器别名表（RAT）将微操作的目的、源操作数重命名到再定序缓冲中的物理寄存器集合上。在流水段22，RAT将微操作（一次3个，因为P6微体系结构是一个3发射超标量）传递到乱序执行的内核。流水段22对按序部分到乱序部分的转变进行标记。被标记过的微操作将写入保留站（等待并且直到能够在合适的执行部件中执行）和再定序缓冲（排队并按照程序的顺序将所做的改变提交给机器状态）。

6.2.2　乱序执行的内核流水线

一旦按序执行的前端将微操作集合（最多可达3个）写入保留站，这些微操作就可能被执行。在决定哪些已经就绪的微操作可以执行时，保留站（RS）将重点考虑以下几个因素：微操作所有的操作数必须就绪；微操作所需的执行部件（EU）必须空闲；在EU完成微操作的时钟周期中，写回总线必须就绪；RS认为（不管什么原因），对于整体性能，没有任何微操作比这一微操作更重要。注意，这是处理器的乱序执行部分，RS不知道也不关心程序的初始顺序，它只是检测数据的相关性，并尽力通过一定的措施来提高系统的整体性能。

这意味着，当一个微操作进入RS之后，可能需要等待几十甚至上百个时钟周期。参见图6.4中标记了保留站调度延迟的地方。如果刚从错误推测的分支中恢复过来，并且进入RS的新指令流不再是推测的，那么调度延迟可能为0。这时，不将微操作写入RS，只是允许RS在两个时钟周期之后“发现”它们的数据已经准备完毕。此时从按序部分发射出来的第一个微操作不依赖任何推测状态，因为在错误分支预测处不可能有更多的预测状态。

正常情况下，微操作写入RS后，一直要等到RS发现它们的数据已经就绪（以及上面列出的限制条件均已满足）。RS需要花费两个时钟周期将这些微操作分派到执行单元，即流水段31到流水段33。

流水段33执行简单的单周期微操作，如逻辑操作或者简单的算术运算操作。更复杂操作，如整数乘法或者浮点操作将需要更多的时钟周期数。

1　为什么第一段命名为11而不是1——这可能是任意选择的

2　为什么有些流水段有多个名称？因为流水段是相互独立的，有时某个流水段的一部分和其他流水段的一部分连接在一起，有时又和另外一个流水段的一部分连接在一起。

单周期操作在流水段33的结尾处将结果置于写回总线上。写回总线是共享资源并由RS管理，所以当分派一个给定的微操作时，RS必须确保在一段时间以后的某个周期内，写回总线是就绪的。写回总线的调度是在流水段82进行的，而写回是在83段进行的（与流水段33同义）。

存储器操作有些复杂。所有的存储器操作首先必须产生有效地址，在IA32中通常是由段基址、偏移、基址和索引组合而成的。产生存储器地址的操作在流水段33中的AGU里执行。在接下来的两个时钟周期里将访问数据cache（Dcache），即流水段42和流水段43。如果cache命中，访问的数据将被返回到RS并成为微操作的源数据。

如果Dcache失效，机器将访问L2 cache。如果仍然失效，则load微操作将被暂停（再次进行下一次尝试是没有意义的，因为必须在存储器中取出数据，这是一个很慢的操作）。存储器定序缓冲（MOB，Memory Ordering Buffer）维护正在进行的存储器操作列表，保持load微操作的暂停状态，直到cache块的内容被新数据填充。其他与被暂停微操作无关的微操作序列可以继续访问cache。MOB使用流水段40来确定并“唤醒”暂停的load微操作。流水段41向Dcache重新发出load操作，并且（如上所述）在访问cache块时，Dcache将使用流水段42和流水段43。图6.4标示出了MOB的调度延迟。

6.2.3 指令提交流水线

指令提交将推测状态变为持久的、不能恢复的机器状态。例如，基于推测的乱序执行的内核可能会执行这样一个微操作：首先将0xFA作为指令结果写到ROB第14项的适当区域内，然后，如果在中间过程中没有发现错误预测的分支，那么当这一微操作成为处理器中停留时间最长的微操作时，将轮到它进行提交操作。在这个时候，这一微操作最初的意图（例如，将0xFA写到EAX寄存器中）才得以实现，它将ROB中第14项的数据0xFA写到提交寄存器文件（RRF，Retirement Register File）的EAX寄存器中。

这一看似简单的想法背后包含着一些复杂的因素。首先，ROB所进行的指令提交不应该仅仅被看做是一个微操作序列，而应该看做一系列的IA32指令。因为从体系结构的观点来看，仅对指令的一部分进行提交是不合法的，所以要么组成IA32指令的所有微操作都进行提交，要么什么也不做，这种原子性的规则要求对体系结构状态进行的部分修改对处理器以外的世界是透明的。所以，ROB必须要做的一部分工作就是检测给定IA32指令的边界，确定它们是否严格地遵守原子性规则。ROB通过检查指令译码器（ID）留在微操作中的一些标志来达到这样的目的：一些微操作被标记为IA32指令的第一个微操作，一些被标记为最后的一个微操作（显然，有一些根本没有标志，这表明了它们是IA32微操作序列的中间部分）。

当对组成IA32指令的一系列微操作进行提交时，不能处理外部的事件，所有外部事件必须等待，就像上一代IA32结构中所进行的处理一样（例如，i486™和Pentium™处理器）。但是在两个IA32指令之间，处理器必须能够捕获中断、断点、自陷和处理错误，等等。再定序缓冲保证这些事件只可能在适当时间发生，并且这些待处理的事件将按IA32指令集规定的优先级顺序进行处理。

处理器必须能够终止某一微码流的执行，切换到另一条微码流上执行一些微操作，然后返回到前面一条微码流并从断点处恢复执行。在这种情况下，当指令发生自陷的时候，可以认为指令只是执行了一部分。指令已经执行的部分不能被丢弃或重新开始，以免妨碍程序的继续执行。这种情况发生在进行TLB更新以及某些浮点辅助运算的时候。

再定序缓冲以微操作循环列表的方式实现，包括一个提交指针和一个新表项指针。再定序缓冲在流水段22中将微操作执行的结果写到表项中的某个区域；在流水段82读取ROB中微操作结果，并在流水段93提交而成为体系结构状态。

6.3　按序执行的前端

流水线前端主要的任务是保持执行部分始终进行有效的工作。在每一个时钟周期内，前端进行一次新的预测，选择一个认为最有可能的Icache地址并按该地址读取一组新的指令，最后将推测的cache块送给译码器进行译码。这种推测当然有可能是错误的，如果有错误，前端将重新定向到正确的地方。当出现分支预测错误时，将导致性能损失。微体系结构设计中的关键问题就是如何确保分支预测能够尽可能准确，并且在发现分支预测错误时，能够尽量减少恢复时间。

译码器最多将3条IA32指令转化成相应的微操作（如果IA32指令非常复杂，则是一条微操作流）并将它们放入队列。寄存器重命名部件为这些微操作的源和目的操作数分派新的物理寄存器标识，然后这些微操作被发射到处理器的乱序执行内核。

如图6.4中的流水线框图所示，P6微体系结构中按序执行前端的运行与处理器的其他部分无关。当在乱序内核中（在跳转执行部件JEU中）检测到错误预测的分支时，内核继续提交错误预测操作之前的微操作，但是要清空错误预测分支之后的所有微操作，参见图6.5。按序执行前端的清空操作完成后，在正确的分支目标代码处（由JEU指示）重新取出指令并进行译码。为了简化按序执行的前端与乱序内核之间的传输，来自于正确分支目标代码处的微操作将严格与乱序内核中原有的微操作进行隔离，直到乱序内核清空为止。通过统计发现，在新的FE2微操作通过按序执行的前端后，乱序内核通常已经清空了。

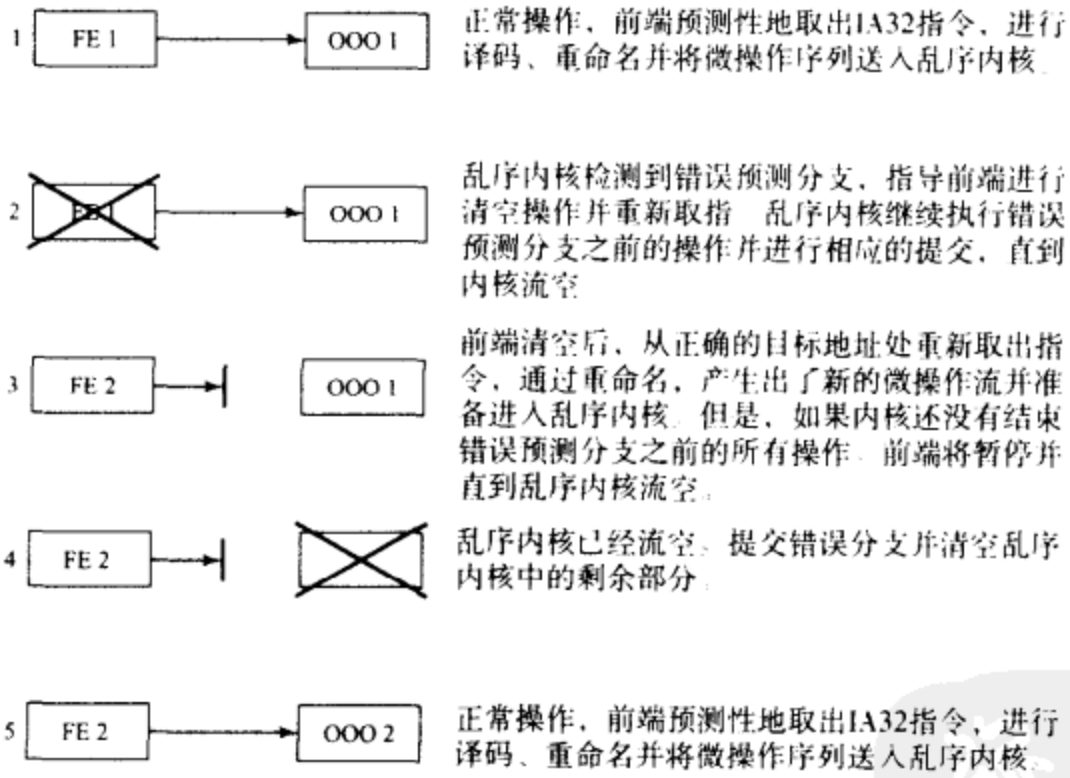


图 6.5　分支错误预测的恢复

6.3.1　指令缓存与 ITLB

片上指令cache将完成最常见的功能，即将最近使用的指令缓存起来。图6.7展示了具有4个流水段的取指部件（IFU）。在第一个流水段中（段11），IFU选择下一次cache访问的地址。这一个地址是从到达IFU的许多取指请求中选择出来的，这些请求来自于BTB和分支地址计算部件（BAC，Branch Address Calculator）。IFU选择具有最高优先级的请求并在第二个流水段（流水段12）开始执行请求。在第二个流水段中，IFU用前一段选择出的取指地址来访问众多的cache和缓冲。这其中包括指令cache和指令流缓冲。如果其中任何一个cache或缓冲命中，则读出指令并转到第三个流水段。如果所有cache和缓冲都失效，则向外部总线逻辑（EBL，External Bus Logic）发送请求，启动一个外部取指请求

流水段12还要使用相同的取指地址对其他两个缓冲器进行访问：IFU中的ITLB和分支目标缓冲（BTB）。访问ITLB将获得物理地址和存储类型，访问BTB将获得分支预测目标。BTB完成一次访问需要花费两个时钟周期。在第三个流水段（段13），IFU对来自前一个流水段（段12）的指令进行标记，其目的是决定指令的边界。分支预测的附加标记则在流水段13的末端由BTB附加。最后，在第四个流水段（段14），如果指令缓冲为空，则指令及其标记都被写入指令缓冲并有选择地进入ID

流水段11为流水段12选择的取指地址是一个线性地址，而不是虚拟地址或物理地址。实际上，IFU并不知道虚拟地址以及所有的分段。这样在P6流水线中，IFU就可以忽略段的边界，而将和段边界相关的违反检测留给IFU后面的部件，但IFU需要处理存储器的分页。当关闭分页模式后，流水段11所选择的线性取指地址与物理地址相同，并直接用来在流水段12中对所有的cache和缓冲进行索引。但是，当启用分页模式后，线性地址必须由ITLB翻译成物理地址。从虚拟地址到线性地址，最后到物理地址的处理序列如图6.6所示。

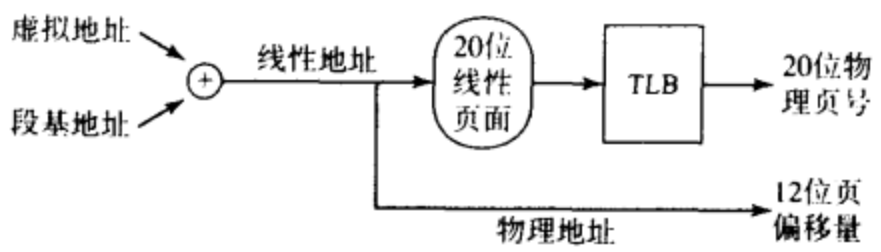


图6.6 虚拟地址、线性地址和物理地址的转换

前端需要使用物理地址（实际上是在物理地址上匹配的某些标识位，这些位没有转换）进行访问的单元有指令cache、指令流缓冲以及指令牺牲cache。分支目标缓冲则通过线性取指地址加以访问。带有BTB的按序执行前端的模块图如图6.7所示。

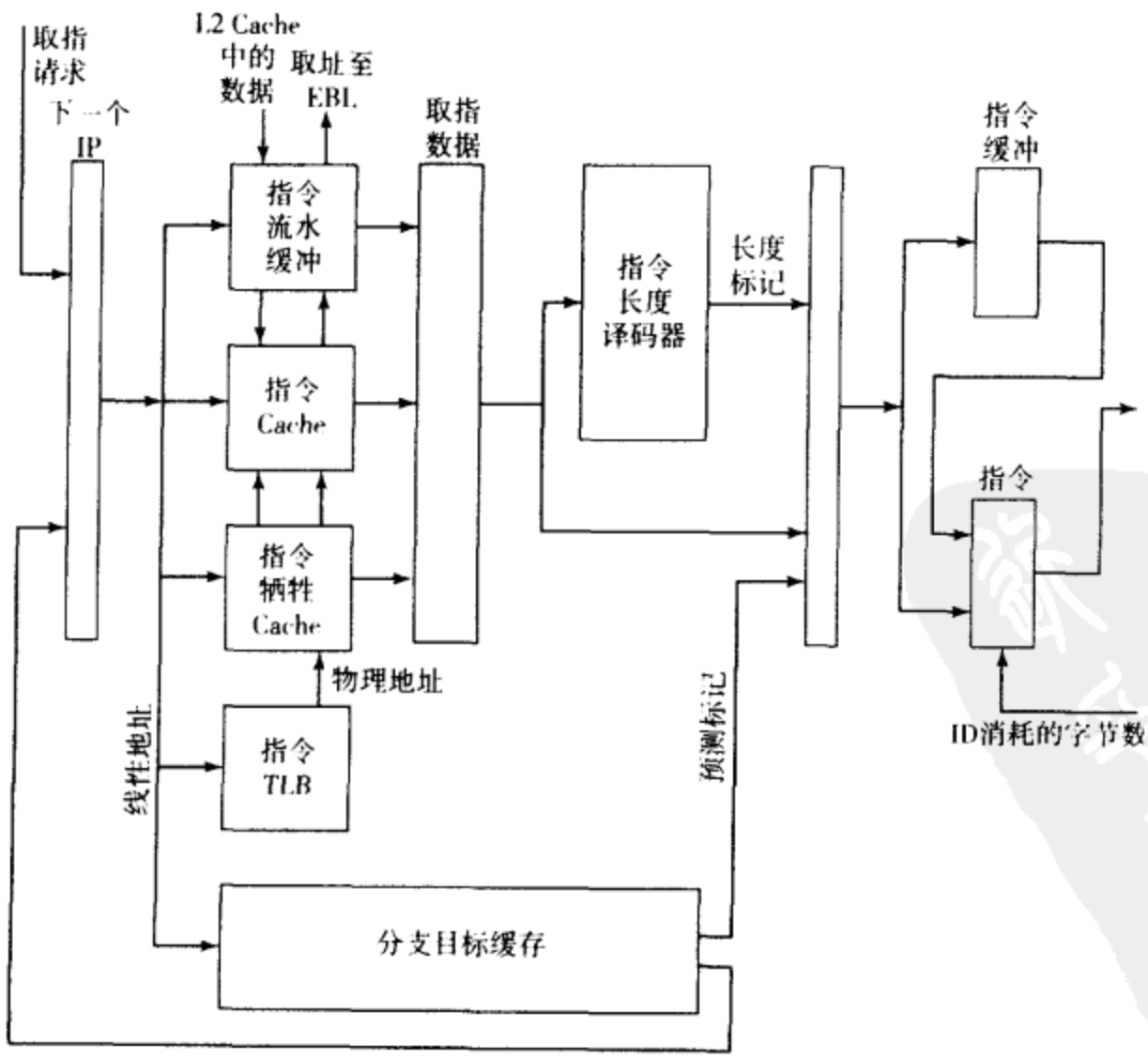


图6.7 前端流水段

6.3.2　分支预测

分支目标缓冲有两个主要的功能：预测分支路径和分支目标地址。BTB 在指令流水线中必须尽可能早地开始工作，以防止处理器执行错误的程序流。在类似于 P6 的基于推测的处理器中，执行错误的程序流只是一个性能上的问题，而不是正确性上的问题。机器总是能够保证正确执行，惟一的问题是速度。

分支决策（对发生或不发生跳转的判断）只能等到跳转执行部件（JEU）对分支进行解析后才能知道（流水段 33）。如果处理器一直等到将分支解析出来后再开始取指操作，则浪费了许多时钟周期。为了避免这一延迟，BTB 对分支结果进行预测，同时 IFU 提前进行取指（流水段 12）。这一预测有可能是错误的，但处理器能够检测并进行恢复。BTB进行的所有预测都将被后面的分支地址计算部件（BAC，流水段 17）或者 JEU（BAC，流水段 33）加以验证。

BTB 使用所取指令的起始线性地址进行预测并产生目标地址。这些信息（分支情况和目标地址）都发往 IFU，如果预测为发生分支，下一次取指则需要重新定向。只有在 JEU 对分支进行解析之后，才更新或者分配 BTB 的一个分支表项。有时分支表项的更新操作来得太迟，当指令流中又出现这一分支时将不会产生什么作用。为了克服这一更新延迟的问题，在 BTB 进行预测时（流水段 13），分支也在进行推测性的更新（在单独维护 BTB 状态中）。

6.3.2.1　分支预测算法

P6 BTB 中的动态分支预测使用 Yeh 和 Patt 提出的两级自适应学习算法，使用两级分支历史信息来进行预测。第一级是分支的历史，第二级是关于分支历史具体模式的分支行为预测。对于每一个分支，BTB 保持 N 位“真实”的分支历史（即关于以前 N 次分支的方向）。这一历史记录称做分支历史寄存器（BHR，Branch History Register）。BHR 中的模式可以编成包含 2^N 个表项的状态表，称为模式表（PT，Pattern Table）并使用给定模式的状态预测下一次碰到这一分支时的动作。模式表中的状态使用 Lee 和 Smith 提出的饱和上下计数器来更新。

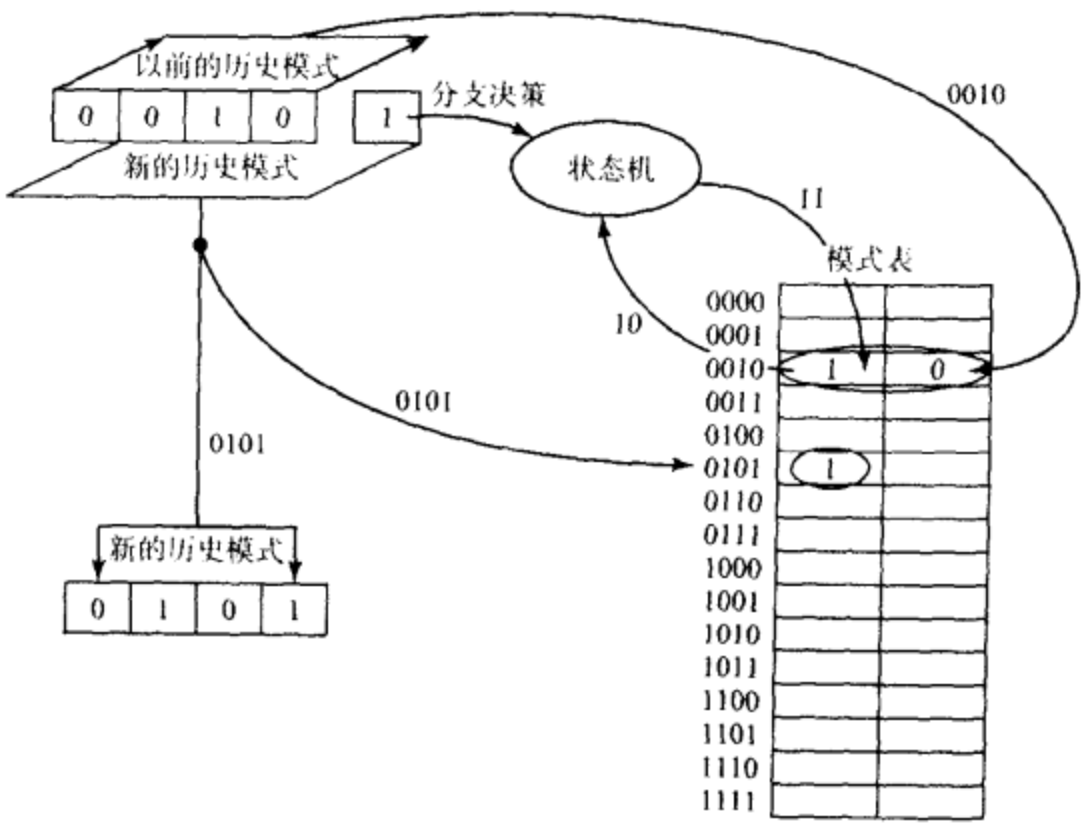
BTB 中每组使用 4 位半局部（semilocal）模式表，即每个表项包含 4 位的历史信息，每一组的所有表项使用相同的模式表（4 个分支为一组并共享一个模式表）。这种方式与 10 位全局模式表的性能是等价的，但是硬件复杂度更小，芯片的面积更小。BHR 包含一个基于推测的副本并在流水段 13 进行更新，而真正的 BHR 将在流水段 83 通过分支解析进行更新。模式表的更新只针对条件分支，即当条件分支在跳转执行部件计算后才进行更新。

为了对分支进行预测，可以使用以前分支的历史模式作为模式表的索引。模式表中状态的最高位指示预测的分支方向。以前的历史模式的陈旧状态通过 Lee 和 Smith 的状态机来更新。

关于算法如何工作的例子如图 6.8 所示。其中需要更新的历史模式表项为 0010，分支决策为跳转。下一次遇到同一个分支时，将使用新的历史模式 0101（最右边添加 1 并左移一位）来索引模式表，并获得新的分支决策 1（状态中的最高位，1 表示发生分支）。现在要对上一个历史模式 0010 的状态进行更新（因为这一历史模式的“未来”状态已经发生了变化）。使用 0010 来索引模式表并获得陈旧状态 10，并将 10 和当前的分支决策 1 一起送到状态机，从而得到新状态 11 并写回模式表。

BTB 还维护一个 16 项的返回站缓冲，用于进行调用返回预测。

由于电路速度的原因，BTB 的访问需要两个时钟周期。这将导致预测为跳转的分支需要在前端插入一个时钟周期的“气泡”。正常情况下，在指令译码器中的双缓冲及其输出队列消除了大多数气泡。BTB 的结构如图 6.9 所示。



两个处理并行发生：

1. 使用新的历史模式访问模式表，以获得新的分支决策。这一分支决策将在下一段写入 BTB
2. 使用以前的历史模式访问模式表，以获得要更新的状态。然后将更新的状态写回模式表

图 6.8　Yeh 算法

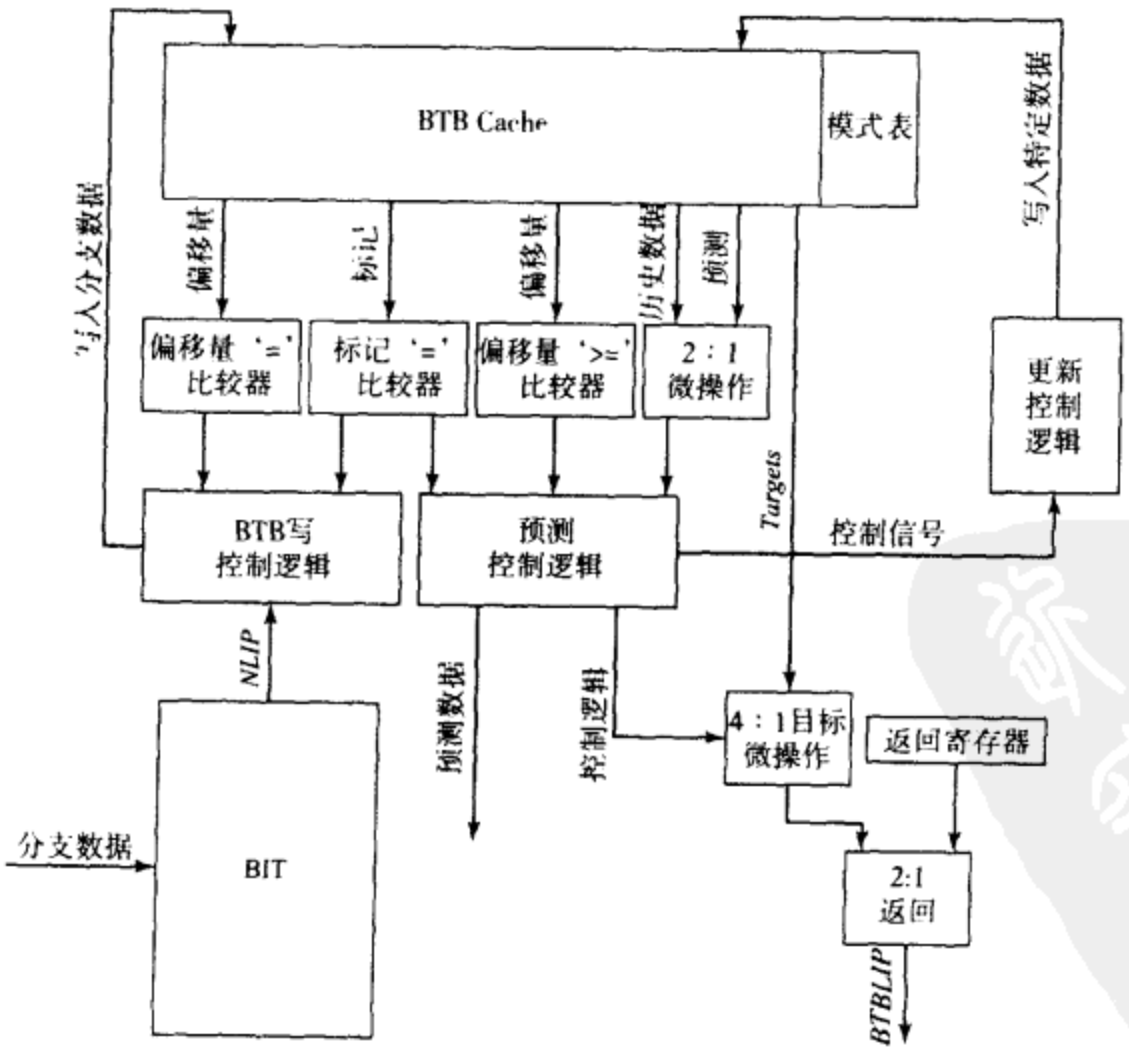


图 6.9　简化后的 BTB 模块图

6.3.3　指令译码器（ID）

ID 的第一段是指令控制块（ISB，Instruction Steering Block），它负责锁存来自 IFU 的指令字节，按顺序分成单独的指令并发往 3 个译码器中的一个。ISB 能很快检测出每个时钟周期有多少指令正在译码，并能够快速确定指令缓冲是否空闲。如果空闲，则能接收并锁存更多的来自 IFU 的指令（参见图 6.10）。

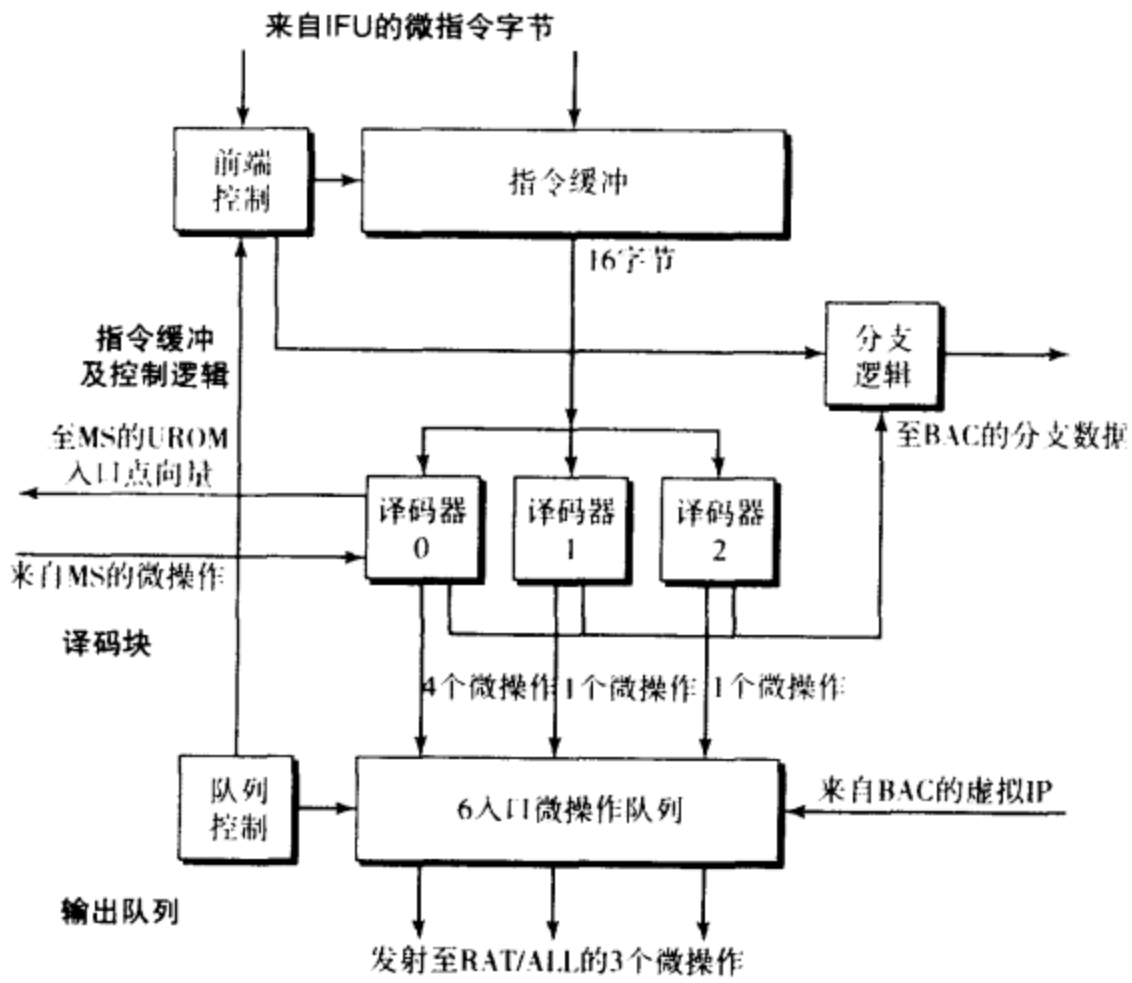


图 6.10　ID 模块图

作为 ID 的“前端”，ISB 逻辑还执行许多其他功能，同时检测并产生预测分支的正确序列。另外，它还为生成的微操作产生有效位并检测 ID 中的停顿条件。

然后，指令缓冲一次从 IFU 中取出 16 字节，这些数据都已经对齐，从而能够确保缓冲中的第一个字节是一条完整指令的第一个字节。平均指令长度是 2.7 字节到 3.1 字节，这意味着每次将 5~6 个完整指令取到缓冲中。如果满足以下任何一个条件，就可以将新的一批指令取到缓冲中：

- 由于分支预测错误而导致处理器前端重新启动。
- 当前缓冲中的所有完整指令都完成译码。
- 一个 BTB 预测为跳转的分支指令完成译码。

在一个时钟周期中，将 3 条正确对齐的指令分派到 3 个译码器中是比较复杂的，这是因为 IA32 指令具有不同的长度。甚至判断一条指令的长度都不是很容易，因为需要等指令的第一个字节译码之后才能理解后面字节的含义。由于分派 3 条不同长度的指令过程是串行的，因此如果能提前知道每一条指令的边界位置，这将有助于指令的分派。这项工作由 IFU 中的指令长度译码器（ILD，Instruction Length Decoder）来完成，从而完成预译码功能。ILD 扫描指令流的字节以确定指令边界，并标记出每一条指令的第一个操作码和最后一个字节。另外，IFU 还要标记出 BTB 所做的分支预测和代码断点。

每次可能有1条到16条指令被取到指令缓冲中，前三条指令分别被分派到三个译码器。如果指令缓冲不包含三条完整的指令，那么就将尽可能多的指令分派到译码器。控制逻辑使用“第一操作码标记部件”来将指令对齐并控制指令的分派。

由于指令缓冲中最多可能包含16条指令，将它们全部译码可能需要花费好几个时钟周期。在某个时钟周期分派的三条指令的起始字节可能处于缓冲中的任何位置。硬件将这三条指令对齐并分派到三个译码器。

尽管在一个时钟周期内能够有三条指令发往译码器，但有可能所有三条指令都不能完成译码。如果某条指令不能完成译码，那么相应的译码器会被清空，所有由这条指令产生的微操作将全部失效。缓冲中所有的指令完成译码需要多个时钟周期。下列情况将导致微操作失效并在下一个时钟周期将相应的指令重新分派到另外一个译码器：

- 如果在译码器0处检测到一条复杂指令并需要MS UROM的帮助，那么这条指令之后的指令产生的微操作全部失效。当MS生成“流”[1]中剩余部分的序列后，才对后续的指令译码。
- 如果一条指令被分派到一个功能受限的译码器（无法对这条指令进行译码），那么这条指令和所有后续指令将在下一个时钟周期被重新分派到其他译码器。这条指令及后续指令产生的微操作被全部失效。
- 如果遇到分支，那么后续指令产生的微操作将失效，因为每个时钟周期只能对一条分支指令进行译码。

注意，可以同时译码的指令数目与ID可以发射的微操作数目没有直接关系，因为译码器队列可以将微操作储存以后再进行发射。

6.3.3.1 复杂指令

复杂指令就是那些需要MS访问UROM从而产生微操作序列的指令。只有译码器0可以处理这些指令。有两种调用MS微码的方法：

- 对于很长的流，译码器0最多产生前4个微操作。MS生成剩下的微操作序列。
- 对于低性能指令，译码器0不会产生任何微操作并将控制转交给MS。

译码器1和译码器2不能对复杂指令进行译码，这是一种设计上的权衡，它综合考虑了实现中硬件的开销和对IA32代码动态执行情况的统计。复杂指令将在后续的时钟周期里被重新分派给编号更小的译码器，直到到达译码器0。MS从译码器0接收UROM的入口向量，并开始对微操作序列排序，直至微码流的结尾。

6.3.3.2 译码器分支预测

当指令缓冲从IFU取指完毕后，ID就会检查预测字节标记，以确定缓冲的完整指令集合中是否有预测为跳转的分支（由BTB动态预测）。ID会在分支指令的最后一个字节处找到预测的分支结果。如果在指令缓冲中发现预测为跳转的分支，ID会通知IFU已经“获取”预测分支。IFU就可以从分支目标处取出16字节的指令块并放到旋转器的输入缓冲。旋转器将分支目标指令对齐，使之成为下一条被取到ID指令缓冲的指令。ID可能会对预测分支指令立即译码，否则会浪费好几个时钟周期（因为要对分支指令之前的所有指令进行译码）。在分支指令完成译码之后，ID将于下一时钟周期锁存分支目标处的指令。

---

[1] “流”是指由微码ROM所发出的微操作序列。经常使用这样的微操作序列来表达IA32指令。

静态分支预测（没有使用任何有关运行时的信息）是由分支地址计算部件（BAC）完成的。如果BAC决定分支发生，它将给IFU提供目的IP并重新开始取指。当然，ID一定不能发射分支指令后的任何微操作，直到它完成分支目标处指令的译码。BAC将在两种情形下进行静态预测：发现一个BTB没有进行预测的无条件分支，或者发现一个目的地址方向是“向后”的条件分支（说明这是一个循环的跳转）。

6.3.4　寄存器别名表

寄存器别名表（RAT，Register Alias Table）提供整数寄存器、浮点寄存器以及标志并对寄存器重命名提供支持，相对于指令集显式提供的寄存器数量，它提供的寄存器集合更大。当把微操作提交给RAT时，微操作的逻辑源和目的操作数都被映射到相应的物理ROB地址上，也就是相应数据所处的位置。对于每个新的微操作，分派部件将用新的物理目的地址去更新映射数组。

考察图6.11和图6.12。在每个时钟周期里，RAT必须检查对应于每个微操作源操作数的物理ROB地址。这些物理地址成为微操作整体状态的一部分，并从此伴随微操作以后的处理。微操作（通过目的寄存器）对任何机器状态的修改也会通过分派部件提供的信息进行重命名，得到的物理目的地址也成为微操作整体状态的一部分并将写回到RAT，供后面以该物理地址为源地址的微操作使用。对于每个微操作，物理目的地址是独一无二的，因此可以用它作为微操作通过乱序执行内核时的标识。所有对微操作的检查和引用均使用“PDst”（物理目的地址）作为它的名字。

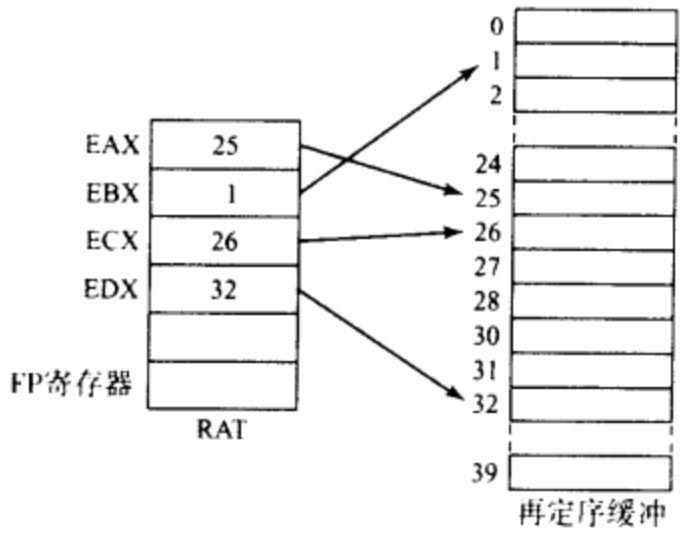


图 6.11　基本 RAT 寄存器重命名

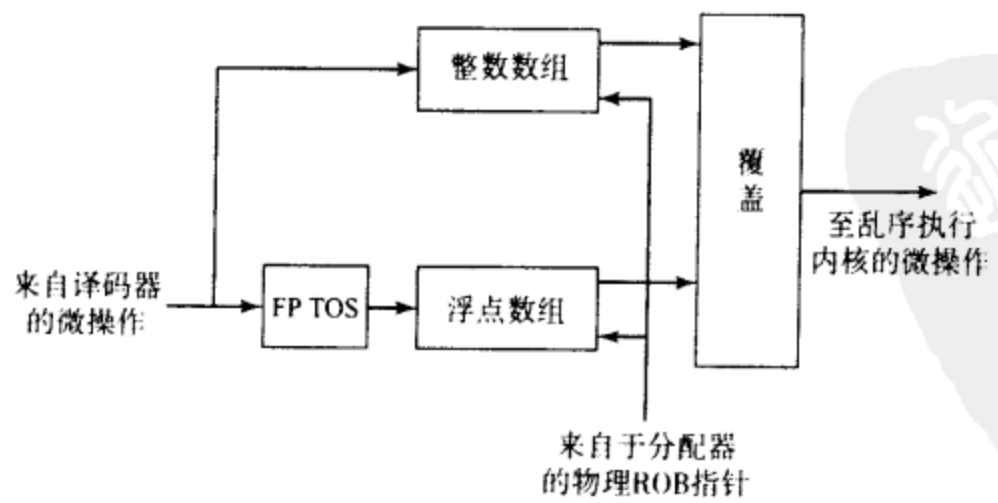


图 6.12　RAT 模块图

由于P6实现了一种超标量设计方案，在一个时钟周期内将有多个微操作进行重命名。如果在三个微操作中存在真相关链，如下所示：

```
μop0:  ADD EAX, EBX ;  src 1=ebx, src 2=eax, dst=eax
μop1:  ADD EAX, ECX ;
μop2:  ADD EAX, EDX ;
```

那么RAT逻辑必须在执行时提供重命名的源地址，而不是像处理前后两个时钟周期间的相关那样，仅仅查找目标地址。旁路逻辑将直接提供μop1的第二个操作数eax，而不必等待μop0的目的寄存器eax的值写回RAT，然后再作为μop1的第二个操作数读出。

RAT中的状态是推测性质的，因为RAT要持续地更新数组项（每个微操作目的地址一项）。当发生不可避免的分支预测失误时，RAT必须清除收集的虚假状态，并恢复到对下一组微操作可用的逻辑物理的映射。P6的分支预测失误恢复机制在将乱序执行内核所有错误预测状态清除之前，RAT不会进行新的重命名。这很有必要，因为寄存器引用会驻留在提交寄存器文件（RRF，Retirement Register File）中，直到出现新的推测微操作。因此，对预测失误的分支进行恢复时，RAT所要做的全部工作就是将所有的整数指针都恢复到它们在RRF中的副本。

6.3.4.1 RAT实现细节

IA32体系结构允许对通用整数寄存器（如EAX，AX，AH，AL等）进行部分的读取与写入，因此在寄存器重命名实现上就会产生问题。当部分宽度的写操作后面跟有一个更长宽度的读操作时，更长宽度的读操作所需要的数据必须能够得到前面多个写操作针对寄存器不同部分的更新。

对于这一问题，P6的解决方法是由RAT记录整数数组的每一项（对应一个微操作的目的寄存器）所使用的寄存器宽度，为数组的每一项增加一个2位的宽度域。这个2位的宽度域通过编码能够区分出三种宽度的寄存器写操作，包括32位、16位和8位。RAT使用寄存器的宽度信息来决定当前的读操作是否需要一个比前面写入的值更“宽”的寄存器值。如果需要，RAT必须生成一个部分写停顿。

另外一种情况是对8位寄存器的独立使用，这通常发生在16位的代码中。如果对一个整数寄存器的三种宽度的访问都采用同一个别名，那么寄存器8位子集的独立使用将会导致大量的假相关。例如，对于下面的微操作序列：

```
μop0:  MOV  AL, #DATA1
μop1:  MOV  AH, #DATA2
μop2:  ADD  AL, #DATA3
μop3:  ADD  AH, #DATA4
```

μop0和μop1将独立的数据移入AL和AH，μop2和μop3将AL和AH作为加法操作的源操作数。如果对于A寄存器只有一个别名，那么μop1指向AH的指针将改写为μop0指向AL的指针（都是指向A的别名）。当μop2试图读取AL时，RAT将得不到正确的指针，因而不得不停顿并等待μop1的提交。然后，μop3的源操作数AH将会丢失，因为μop2对AL进行了写操作。实际上，CPU的操作将变成串行并且性能受到损失。

为了防止这种情况，RAT包含了两个整数寄存器体，分别称为Low Bank和High Bank。对于32位和16位的访问，数据只能从Low Bank中读取，但是数据将同时写入两个体中。对于8位的访问，根据访问高位字节还是访问低位字节，从而对相应的体进行读写操作。因此，高位字节或低位字节寄存器使用的是不同的重命名表项，它们可以相互独立地重新命名。注意，High Bank只有4个数组表项，因为在指令集规范中规定，包含4个整数的寄存器（即EBP，ESP，EDI，ESI）不能进行8位宽度的访问。

RAT 的物理源地址（PSrc）指向 ROB 阵列中数据所处的真正位置。直到产生数据的微操作执行完毕并将结果写到某条写回总线上，数据才真正出现在 ROB 中。在 PSrc 完成写回以前，ROB 对应表项中的值没有意义。

RAT 的每一个表项都有一个 RRF 位，用来选择 RRF 或者 ROB 两个地址空间中的一个。如果 RRF 置为 1，表明数据放在寄存器文件中，物理地址被设置为 RRF 中的相应表项。如果 RRF 置为 0，表明数据放在 ROB 中，物理地址将指向 ROB 中相应的位置。6 位的物理地址域可以访问 ROB 的任何表项。如果 RRF 被置为 1，物理地址则指向寄存器文件，物理地址域包含的是指向 RRF 寄存器的指针。总线的设计使得 RRF 可以按 ROB 的方式获取数据。

6.3.4.2　基本 RAT 操作

对逻辑源地址（LSrc）进行重命名时，ID 所发射的 3 条微操作的 6 个逻辑源地址作为访问 RAT 整数数组的索引。RAT 数组中的每一个表项都有 6 个读端口，并允许 6 个 LSrc 读取数组中任何一个表项。

读取完成之后，必须用分派部件产生的新的物理目的地址（PDst）对数组进行更新。由于可能存在同一周期内的目的地址相关，因此可以使用一种带优先级的写机制来保证 PDst 正确写入每个数组表项。

PDst 写机制的优先级按以下方式给出：

最高：当前 μop2 的物理目的地址
　　当前 μop1 的物理目的地址
　　当前 μop0 的物理目的地址
最低：任何提交的微操作的物理目的地址。

指令提交是指从 ROB 中删除已经完成的微操作，经过确认后成为持久的机器状态。ROB 通知 RAT，（从现在开始）在再定序缓冲中将不会再找到已提交微操作的目的地址，而必须到寄存器文件（RRF）中去获取。如果 RAT 数组中存在与已提交的 PDst 匹配的表项（或多个表项），则需重新设置，以指向 RRF。

提交机制要求 RAT 将每个表项的 PSrc 与当前周期提交的 3 条微操作的目的地址进行匹配。对于所有的匹配项，相应的数组表项将被重新设置并指向 RRF。提交在 PDst 写机制中的优先级最低，在逻辑上，提交应该在任何新的微操作写 PDst 之前进行。因此，如果任何微操作写 PDst 和提交的重置操作同时发生，PDst 的写回操作应该最后进行。

由于 Intel 体系结构的浮点（FP）寄存器栈结构，重新设置浮点寄存器重命名的机制将更加复杂，因此需要提供特殊硬件从 FP 寄存器的地址中去掉栈顶（TOS）偏移。另外，还要维持一个浮点寄存器别名表（RfRAT，Retirement FP RAT），其中包含浮点寄存器的非推测别名信息。RfRAT 只在微操作提交时进行更新。RfRAT 的每一个表项的宽度是 4 位：1 位是提交有效位，另外 3 位是 RRF 指针。另外，RfRAT 还维护自己的非推测 TOS 指针。之所以增加 RfRAT 是因为只有通过 RfRAT，处理器才能从存在 FXCH 指令时的分支预测失败和其他事件中恢复过来。

FXCH 指令将浮点 TOS 寄存器表项与栈中任意一个表项（包括 TOS 自己）进行交换。FXCH 本来可以用三条 MOV 微操作来实现，其中要使用一个临时寄存器。但是面向 Pentium™ 处理器优化的浮点代码广泛使用 FXCH 来为 Pentium 的双执行部件组织数据。如果使用三条微操作来实现 FXCH，P6 处理器执行面向 Pentium™ 处理器优化的浮点代码时性能会有很大影响，因此将 FXCH 实现为一条单独的微操作。

P6处理器在处理FXCH操作时，仅仅交换RAT中浮点部分（fRAT）指向的两个要交换的源寄存器指针。因此在fRAT中需要额外的写端口，但避免了在RRF的两个寄存器之间执行超过80位数据的交换操作。另外，由于指针交换操作不需要通过执行部件，所以一旦ROB接收到FXCH，就将FXCH标志为“已完成”。因此，FXCH不会占用RS资源，而在0周期内完成。

在发生分支预测失误以前，fRAT可能推测并交换了任意数目的表项。发生预测失误后，所有从这个分支上发射的指令将被停止。过一段时间，ROB会触发一个信号，表示所有的微操作（包括分支）都已经提交。这意味着CPU中的所有数组已经重置，微体系结构状态必须恢复到分支预测失误时的机器状态。关键是要能够正确地消除FXCH推测所产生的影响。fRAT表项不能像RAT的整数部分那样简单地设置为常量，因为可能有任意多个FXCH已经提交，fRAT必须永远记住提交的FXCH结果。这正是设置RfRAT的目的：当前端必须清空时，应该“知道”将FP表项重置成什么值。

6.3.4.3 整数提交覆盖

当一个正在提交的微操作的PDst在RAT中仍然被引用时，在提交过程中，引用该PDst的RAT表项要把引用改为指向寄存器文件，即微操作的提交必须优先于对RAT的读取。在硬件实现上，这一操作是使用旁路在表读取之后执行的。这时，从表中读取的数据将被最新微操作提交的信息覆盖。

针对当前时钟周期内所有合法的提交指针，整数提交覆盖机制需要对整数数组的PSrc表项进行一些相关的匹配搜索。对于所有搜索到的匹配表项，表项内容将被重新设置以指向RRF。

提交覆盖操作是必需的，否则在RAT中读取到的PSrc将不再指向正确的数据。当前时钟周期提交的ROB数组表项不能再被任何新的微操作引用（因为数据已经出现在RRF中）。

6.3.4.4 新的PDst覆盖

微操作的逻辑源寄存器将作为访问RAT多端口整数数组的索引，数组则输出物理源寄存器，这些源寄存器将遵循提交覆盖规则。同时，RAT会接收到最新分配的物理目的（PDst）。来自于ID的逻辑源寄存器和目的寄存器会进行优先级比较，经过选择器并输出整数数组的PSrc或者输出分配器的PDst，作为最终重命名操作后的物理源寄存器。注意，源0永远不会被覆盖，因为在它之前没有任何本周期中的微操作与之相关。RAT覆盖的硬件模块图如图6.13所示。

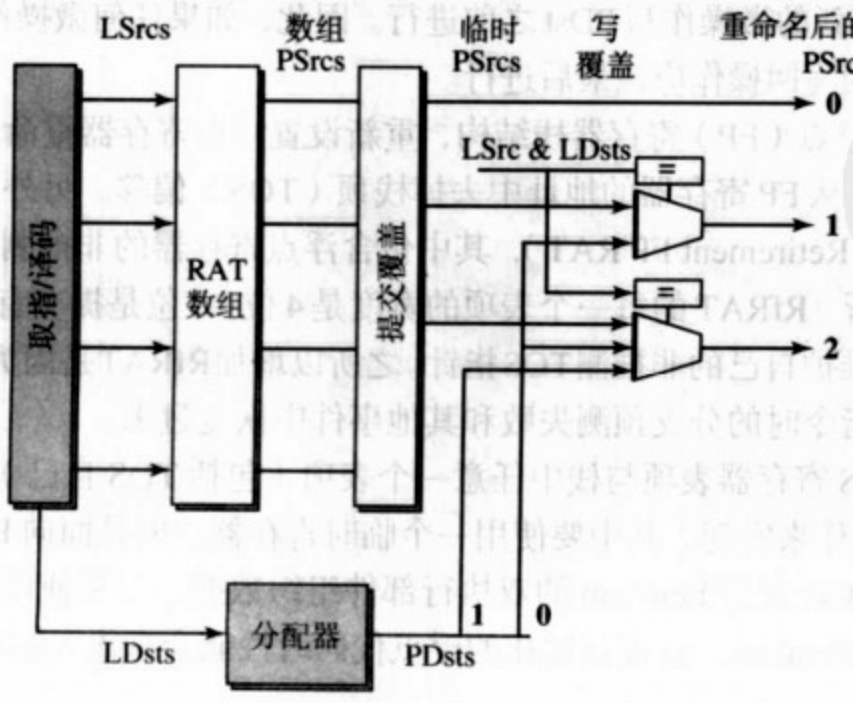


图6.13 RAT最新PDst覆盖

考虑下列微操作处理：

μop0：r1 + r3 -> r3
μop1：r3 + r2 -> r3
μop2：r3 + r4 -> r5

注意到μop1 的源操作数依赖于μop0 的目的操作数。这就意味着μop1 需要的数据不会在 RAT 所指向的寄存器中找到，而是出现在分配器所提供的新位置上。RAT 中的 PSrc 信息已经被μop0 的 PDst 刷新，必须在重命名操作的物理源寄存器输出到 RS 和 ROB 之前被修改。同样，μop2 的源操作数引用了μop0 和μop1 都要写回的寄存器。最新的 PDst 覆盖控制必须指示出，μop1（不是μop0）的 PDst 是用于修改 μop2 源引用的最终正确指针。

微操作序列可能是整数与浮点的混合操作。尽管可以用两个单独的控制块分别执行整数与浮点覆盖，但是通过逻辑寄存器名字的比较已经足够区分这两类微操作。很自然，只有相同类型的源引用和目的引用能够进行覆盖（例如，一个浮点目的引用不能覆盖一个整数源引用）。因此，浮点覆盖中的差异完全可以由整数机制独立处理。

浮点覆盖的要求与整数覆盖相同。微操作的提交与并行发射逻辑会防止在那些并发微操作读取数组之前，用最新的信息更新数组。因此，从 RAT 数组读取的 PSrc 信息一定经过了提交覆盖逻辑和最新 PDst 覆盖逻辑的覆盖。

总体上，浮点提交覆盖与整数提交覆盖一样，但是有一点不同，覆盖到 PSrc 上的值不是由逻辑寄存器源的名字来决定。更确切地说，提交逻辑目的寄存器将读取 RfRAT 以获取重新更改过的值。根据这次数组读取中 CAM 所匹配的提交微操作，提交覆盖控制必须在三个更新过的 RfRAT 复位值中选择一个。这些复位值肯定被任何并发的提交 FXCH 修改过。

6.3.4.5　RAT 停顿

RAT 能以两种方式停顿：内部停顿和外部停顿。如果由于局部寄存器写、标志的错误匹配或者其他微体系结构上的原因，而不能完整地处理当前微操作的集合，RAT 则产生一个内部停顿。因为 RS 或者 ROB 表的溢出，分配器不能处理完所有的微操作。对于 RAT 来说，这是一个外部停顿。

局部写停顿

当一个局部宽度的写操作（例如，AX，AL，AH）后面跟有更宽的读取操作（例如，EAX）时，RAT 必须停顿，直到前一个对相关寄存器进行操作的局部宽度写操作提交为止。这时，RRF 中该寄存器的所有部分都会重新组合，对于指令所需要的寄存器访问，将重新给出一个单独的 PSrc。

RAT 通过为每个寄存器别名维护大小信息（8 位、16 位或 32 位）来执行这一功能。为了处理 8 位寄存器的独立使用，在整数数组中，对于 EAX，EBX，ECX，EDX 中的每个寄存器都有两个表项和别名（H 和 L）。根据 Intel 体系结构规范所述，其他的大寄存器不能进行局部写操作。当 16 位或者 32 位的写操作发生时，两个表项都会更新内容。当 8 位写操作发生时，只有相应的表项（H 或者 L，不是同时）被更新。

因此，当一个逻辑源寄存器指向某个表项时，微操作中指定的大小信息将会与数组中读出来的大小信息进行比较。如果操作需要的规模比可以获得的规模（数组中读取出来的信息）大，RAT 将停顿指令译码器和分配器。同时，RAT 将清除导致停顿的微操作（以及所有后续的微操作）中的“有效位”（valid bit），并且一直持续到局部写提交为止，这称为有序流水段。这里，后续微操作不允许越过停顿的微操作提前执行。

不匹配（Mismatch）停顿

对标志进行读写是一种经常发生的操作，因此它们对于性能的影响比较关键，它们的重命名与寄存器重命名一样。对于标志，有两个别名表项：一个是算术标志表项，一个是浮点条件代码标志

表项，与其他整数数组表项的维护方式相同。当一个微操作需要写标志时，对应于微操作的PDst将写到相应的标志表项（也称目的寄存器表项）。当下一个微操作使用此标志作为源操作数时，则读取正确的标志表项并寻找标志所在的PDst。

对于通用重命名机制，由ID发射的每一个微操作都有相关的标志信息，并以掩码的形式告诉RAT哪些标志是微操作需要处理的，哪些标志需要作为微操作的输入。当以前尚未提交的微操作还没有处理完所有的标志，而这些标志又必须作为最新微操作的输入时，RAT需要停顿按序执行的机器并通知ID段和分配器，没有新的微操作能够进入RAT，因为当前微操作中的一个或多个操作不能发射，直到以前的某个写标志操作提交为止。

6.3.5 分配器

在每一个时钟周期，分配器假设它必须分配三个再定序缓冲、保留站、载入缓冲（LB）表项以及两个存储缓冲（SB）表项。分配器将产生指向这些表项的指针，对来自于ID单元的微操作进行译码并判定到底需要多少资源表项，以及将在哪一个RS分派端口进行分派。

基于微操作的译码和有效位的设置情况，分配器将查看所需要的资源是否已经满足。如果不满足，则插入一个停顿并冻结微操作的发射，直到以前的微操作提交并释放了足够的可用资源。

分配器的第一步是对ID发送的微操作进行译码。某些微操作需要一个LB表项或者SB表项，所有的微操作都需要一个ROB表项。

6.3.5.1 ROB分配

ROB表项地址是分配器分派的物理目的地址或者PDst。PDst用于直接定位ROB表项。如果ROB表项已满，分配器必须提前插入一个停顿信号以阻止对合法ROB表项的覆盖。

分配器将ROB缓冲作为一个循环队列来看待。换句话说，表项地址的分配是从0到最高地址，然后返回到0并重新分配。P6使用3-or-none分配策略：在每个周期，必须要有三个ROB表项可用，否则分配器暂停。ROB分配与微操作的类型无关，甚至并不依赖于微操作的有效性。3-or-none策略将有利于简化分配逻辑。

6.3.5.2 MOB分配

所有微操作都有一个load缓冲ID和一个store缓冲ID（即MOB ID或者MBID）与它们绑定在一起。load微操作将获得一个最新分配的LB地址和最后一次分配过的SB地址。非load微操作（存储或者其他微操作）将获得MBID（LBID=0）以及上一次store所分配的SBID（或存储着色）。

如同ROB一样，LB和SB都被看做循环缓冲。但是，它们在分配策略上有一些区别。由于并不是每一个微操作都需要一个LB表项或SB表项，因此，使用3-or-none策略（对于SB使用2-or-none策略）时，一旦LB或SB少于3个表项就停顿的做法将会对性能造成很大的影响。所以我们采用all-or-none策略，只有在所有合法的MOB微操作都能分配的时候，停顿才不会发生。

关于MOB分配的另一个重要部分就是包含Senior store的表项处理。这些存储操作已经由CPU提交或者已经经过确认，但是仍然在等待写往存储器的实际操作（现在处于写缓存）。这些存储缓冲表项不能取消分配，直到写往存储器的实际操作被真正地执行为止。

6.3.5.3 RS分配

分配器也产生写使能位并被RS直接用作表项的使能项。如果RS已满，必须尽早给出一个停顿指示，以防止合法的RS表项被覆盖。实际上，如果RS已满，使能位将全部清除，因此，将没

有任何表项可以进行写操作。如果RS没有满，但是由于操作的源寄存器发生冲突而导致停顿，RS将会废除在相应时钟周期内写到任何RS表项的数据（即写入数据，但是却被标记为无效）。

RS分配器与ROB和MOB的循环缓冲模型不同。由于RS是乱序分派微操作（一旦它们的数据准备就绪就开始分派），其空闲表项典型的分布方式是分散在已经使用或者已经分配的表项之间，所以不能使用正常的循环缓冲模型。取而代之的是一种位图方法，每一个RS表项对应着RS分配池中的一位数据。通过这种方法，这些表项可以在分配池中以任何顺序被移动或替代。RS的搜索方式是从地址0开始扫描，直到发现三个空闲的表项为止。

某些微操作可以分派到多个端口，将给定的微操作提交到一定端口的过程称做绑定。将微操作绑定到RS功能部件的接口是在分配时进行的。分配器执行负载平衡算法并且知道在RS中每个接口上有多少微操作在等待执行。该算法仅用于那些可以在多个EU上执行的微操作。这称做就绪微操作到执行接口的、基于负载平衡的静态绑定。

6.4　乱序执行的内核

6.4.1　保留站

保留站（RS）是微操作在执行之前等待分派的地方，直到它们的操作数都已就绪并且相应的执行部件可用为止。在每个时钟周期，RS将决定执行部件的可用性及源数据的合法性，同时，执行乱序调度，将微操作分派到执行部件并控制数据旁路到RS数组和执行部件。RS的所有表项都是一样的，可以保存任何类型的微操作。

RS有20个表项，可以从3个端口（P6微体系结构是超标量结构）写入表项的控制信息（微操作，表项合法性等）。这一信息来自于分配器和RAT。表项的数据部分可以来自于6个端口（3个ROB和3个执行部件写回）。CAM将控制合法写回数据到微操作源数据域的传递以及执行部件（EU）接口的数据定向（旁路）。CAM、EU仲裁和控制信息将用来决定每个表项的数据合法性和EU可用性（生成就绪指示位）。调度逻辑根据微操作是否准备就绪来调度多达5个微操作。微操作读取分派的表项并进入执行部件。

在流水段31，RS决定哪个表项已经或将要在流水段32进行分派。这时，有必要知道数据和执行资源（EU/AGU单元）的可用性，就绪信息将发送给调度器。

6.4.1.1　调度

调度器的基本功能就是对来自于RS的微操作进行调度，最多可以调度5个微操作。RS包含5个调度器，每一个调度器对应着所有的执行部件接口。图6.14显示了功能部件与相应RS端口的映射关系。

RS使用一个优先级指针来指示调度器应该在何处开始对20个表项进行扫描。优先级指针根据伪FIFO算法进行变动，减少陈旧表项的影响并提高性能。

6.4.1.2　分派

每个时钟周期内，RS至多可以分派5个微操作，包括两个EU，两个AGU以及一个存储数据（STD）接口。图6.14显示了执行部件到RS端口的连接。在指令分派之前，对于等待执行的微操作，由RS判定是否所需的资源已全部可用，然后调度相应的表项。RS向调度功能部件分派所有的微操作信息。一旦微操作已经分派到某个功能部件，并且没有因为cache失效而执行取消操作，那么表项就可以分配给另外一个新的微操作。在每个时钟周期内，回收指针通知分配器有关RS中所有20个表项的可用性信息。

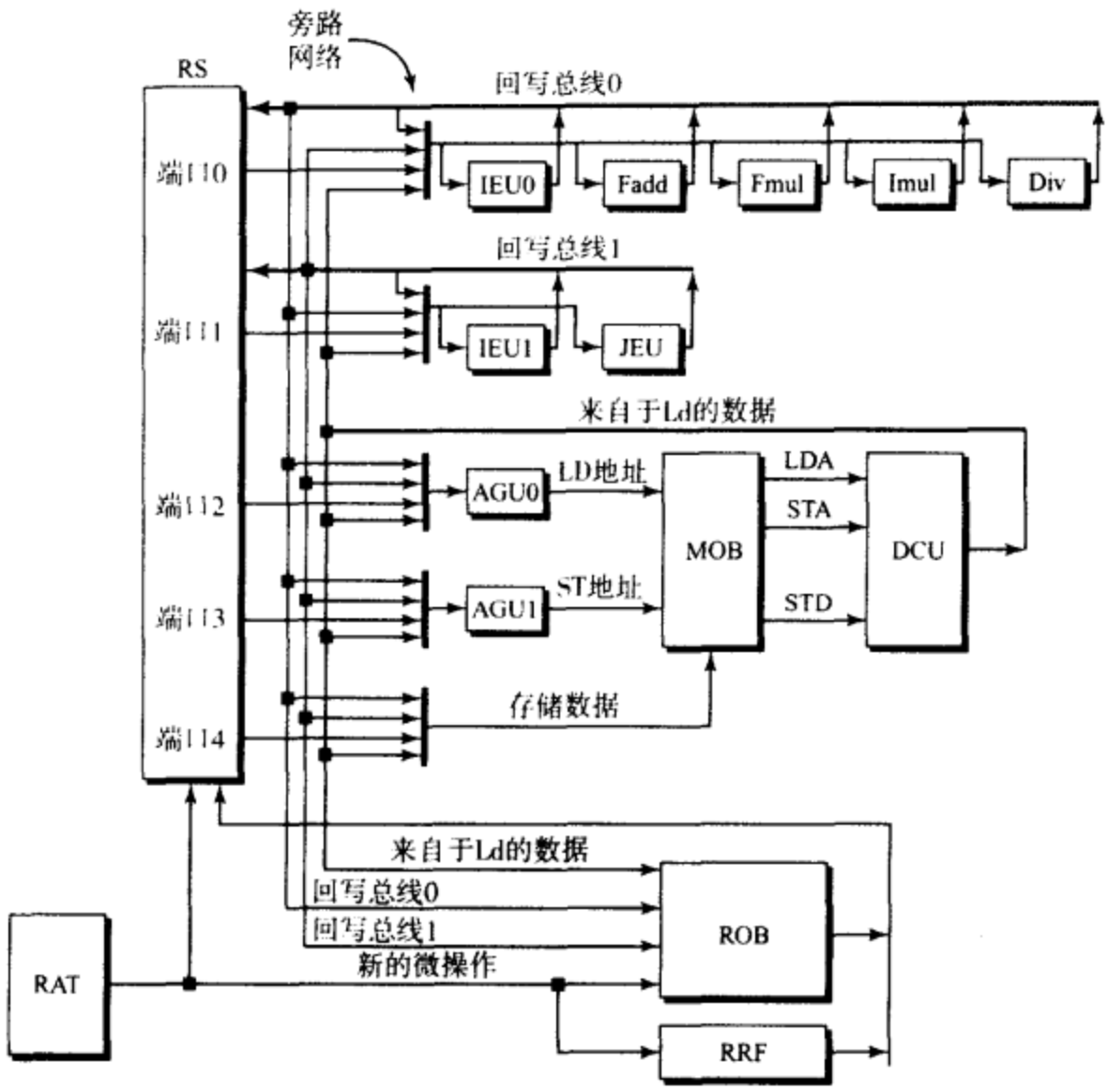


图 6.14　执行部件数据路径

6.4.1.3　数据写回

在初始化 RS 表项的时候，源数据有可能是非法的。微操作必须保存在 RS 中，直到所有的源数据变为合法。CAM（内容相联存储器）用于将写回物理目的标识（PDst）与所存的物理源（PSrc）进行比较。当发生匹配时，CAM 将插入相应的写使能，同时将所需要的写回数据直接发送到数组表项中合适的物理源中。

6.4.1.4　取消

由于cache失效或者其他可能存在的资源冲突，取消操作用来约束微操作的调度、分派或执行。所有取消的微操作将在后面某一时刻被重新调度，除非乱序内核被重新启动。

数据回写有时候也会失效，例如当存储器检测到cache失效的时候，如果分派的微操作与回写数据相关，则需要被取消并在以后某一时刻重新调度。由于 RS 流水线调度时假设 cache 访问会命中，并基于这一假设调度相关的微操作，因此存在发生这种情况的可能性。

6.5　指令提交

6.5.1　再定序缓冲

P6微体系结构存在三个基本因素导致使用再定序缓冲（ROB）：推测执行、寄存器重命名以及乱序执行。在某些方面，ROB 与按序执行机器中的寄存器文件类似，但通过附加的功能来支持推测的执行和寄存器重命名的提交

ROB 将执行单元（EU）的结果最终确认为体系结构状态之前，会将中间结果缓存起来并且支持推测的执行。同时，允许大多数微引擎以一个最大的速度来取指并执行。但是有一个前提，即假设分支已经进行了正确的预测并且没有任何异常发生。如果分支预测错误，或者在执行指令时有异常发生，微引擎可以通过丢弃存储在ROB中的结果来进行简单的恢复，微引擎也可以通过检查ROB中已被确认的状态来重新启动正确的指令。ROB 的一个关键功能就是控制微操作的提交与完成。

对EU结果的缓冲存储也用于执行寄存器的重命名。EU将结果只写入ROB中重新命名后的寄存器中。ROB 中的提交逻辑将根据每一个重命名映射的内容对体系结构寄存器进行更新。引用体系结构寄存器的微操作将获得体系结构寄存器的内容，或者是重命名寄存器的内容。由于P6微体系结构是超标量结构，因此，在同一时钟周期里，使用相同体系结构寄存器的不同微操作可能会访问不同的物理寄存器。

ROB 支持乱序执行，允许EU完成微操作并将结果写回，且不关心其他同步执行的微操作。因此，一旦微操作进入到执行部件，它就会以乱序方式完成。ROB 提交逻辑在提交的时候对体系结构状态进行更新，将已完成的微操作通过重新排序，还原成由指令译码器最初发射出来的原始顺序。

ROB 在处理器流水线的三个独立部分处于活跃状态（参见图 6.4）：重命名和寄存器读取流水段、执行/写回流水段以及提交流水段。

与P6中其他部件相对应的ROB位置如图6.1所示。ROB与分配器（ALL）、寄存器别名表（RAT）部件相邻。分配器管理着ROB物理寄存器（ROB数组表项），支持推测执行和寄存器重命名。ROB中体系结构寄存器的实际重命名由 RAT 管理。分配器和 RAT 都在 P6 流水线的按序部分中产生作用。因此，重命名和寄存器读取（或 ROB 读取）功能的执行顺序与程序流的顺序是一样的。

ROB 与保留站（RS）、EU 在机器的乱序部分相连，实际上属于松散耦合。在寄存器读取流水段时，从ROB中读出的数据包含微操作的源操作数，然后保存在RS中，直到微操作被分派到某个执行部件。EU通过5个写回端口（三个完全写回端口，两个用于STD和STA局部写回端口）将微操作的结果写回到ROB。相对于指令译码器发射出来的微操作，写回的结果处于乱序状态。由于来自于EU的结果是推断产生的，由EU检测出的任何异常都有可能是或不是“真实”的异常，这样的异常将写入微操作的特殊域内。如果证明微操作确实是错误推断执行的，那么异常就不是“真实”的，需要将连同微操作的剩余部分一起清除。否则，ROB 将检测在微操作提交期间的异常条件，并在将微操作的结果状态确认为系统状态之前调用正确的异常处理。

ROB提交逻辑与微指令序列器（MS）、存储器定序缓冲（MOB）之间具有重要的接口。ROB/MS接口允许ROB向MS通知异常，强迫微指令序列器跳转到特定的异常处理微码中。同时，ROB必须强迫控制流改变，因为相对于程序流来说，EU提供的事件是乱序的。ROB/MOB接口允许MOB在存储微操作的机器状态得到确认时，提交存储微操作的存储器状态。

6.5.1.1　流水线中的 ROB 存储

ROB 在 P6 流水线的按序和乱序部分都将产生作用。在流水段 21 和流水段 22，ROB 处于按序部分，在流水段21处将分配保存推断微操作结果的再定序缓冲表项。分配器和提交逻辑将再定序缓冲作为循环缓冲进行管理。如果有未被分配的再定序缓冲表项，分配器将把它们分配给发射出来的微操作。所分配的表项将通知给RAT并允许RAT更新它的重命名表或别名表。每个微操作所使用的表项地址（PDst）也将写入RS。机器的乱序部分用PDst来识别处于执行过程中的微操作，实际上就是ROB中的槽号。当ROB中的表项分配后，微操作中某些域的信息将写到表项的特定区域。这一信息可以在分配时写入，也可以在EU写回结果时写入。为了减小RS表项的宽度，也为了减少传播给EU和存储器子系统的信息量，对于译码时严格确定的微操作，提交微操作所需的任何信息都将在分配时写入ROB。

在流水段22，紧随在ROB表项分配后，微操作将从ROB读取所需的源操作数。物理源地址（PSrc's）由RAT通过别名表的更新进行传递，其中流水段21执行别名表的更新。源操作数可以处于以下三个位置：已经确认的体系结构状态（存在于提交寄存器文件），再定序缓冲或者是写回总线。RRF包含体系结构状态和微代码的可见状态。后续指令对RRF状态的引用称为微代码和宏代码可见状态。从RRF中读取的源操作数总是合法的并已经准备就绪，同时，等待执行部件的使用。从ROB中读取的源操作数有可能是非法的，这取决于源操作数的读取操作与前一个微操作的写回动作之间的时序关系，因为写回操作将对表项内容进行更新。对于特定的微操作，如果由ROB传送的源操作数是非法的，为了捕获（或通过EU的旁路直接获取）合法的源操作数，RS将进行等待，直到EU对某个PDst进行写回，且该PDst与源操作数的物理源地址相匹配

在流水段83，EU将目的数据及相关的所有事件信息写回到为微操作分配的表项中（事件指异常，中断，微代码辅助，等等）。因为微操作是从RS乱序发射出来的，所以重命名和寄存器读取流水段与写回流水段之间就形成了松耦合的关系。写回总线使用的仲裁由EU与RS决定。ROB是每个写回总线的终点，根据EU提供的PDst将存储总线上所有即将写回的数据写入。

ROB提交逻辑在流水段92和流水段93对宏代码和微代码的可见状态进行确认。由于写回操作相对于程序或微代码是乱序的，写回流水段与提交流水段之间就形成了松耦合的关系。总之，提交操作有效地对执行部件乱序完成的微操作进行再定序，将它们变成按序完成的微操作。提交操作占用两个时钟周期，但是提交段是流水化的。如果在再定序缓冲中分配了表项，提交逻辑将试图释放表项或是对它们进行提交。由于在流水线前面的流水段中，微操作是以连续的FIFO顺序分配的，所以在释放表项时，提交逻辑将再定序缓冲看成FIFO。这样可以确保提交操作最终会遵循原始程序的顺序，更新体系结构状态。

ROB包含所有的P6宏代码和微代码状态，它们的修改并没有遵循机器的串行顺序。串行化将流过机器乱序部分的微操作数目限制为1，这有效地保证了它们能够按序执行。这一状态大部分是从再定序缓冲中的推断状态直接更新而来，但扩展指令指针（EIP）寄存器是个例外。对于ROB中的每次更新，EIP需要数量很多的硬件，因为在每个时钟周期内可能提交的微操作数量将在0~3之间变化。

ROB作为具有独立端口的多端口寄存器文件加以实现，这些端口用于在分配时向微操作的某些域内填写提交所用信息、EU写回、RS读取ROB中的源，以及提交逻辑对推断结果数据的读取。

ROB包含40个表项。每个表项的宽度为157位。分配器和提交逻辑将其作为FIFO进行管理，源数据读取和目的数据写回操作将再定序缓冲看成寄存器文件。

RRF包含宏代码和微代码的可见状态。并不是所有的处理器状态都在RRF中，但是任何有可能被重命名的状态都在这里。表6.1给出了RRF中的寄存器列表。

在每个时钟周期内，提交逻辑都要为提交操作生成地址，同时计算提交操作的合法信号，指出哪些结果能够合法地写回数据的表项并可以进行提交操作。

IP计算模块将产生体系结构指令指针以及其他一些宏指令和微指令指针。通过所有可以提交的宏指令长度以及任何由跳转执行部件产生的分支目标地址生成宏指令指针。

当ROB确定处理器在沿着分支的错误路径执行时，该路径上的任何操作都不能提交。ROB在这种需要提交的第一个操作之前插入一个“清除”信号来达到这样的目的，之后机器对所有推测执行的操作进行排空。当ROB提交一个错误操作时，在流水段93和流水段94将对机器的按序部分和乱序部分进行清空。

表 6.1　RRF 中的寄存器

数量	寄存器名	尺寸	描述
8	i486 通用寄存器	32 位	EAX, ECX, EDX, EBX, EBP, ESP, ESI, EDI
8	i486 FP 栈寄存器	86 位	FST（0~7）
12	通用微码临时寄存器	86 位	存储整数值和浮点值
4	整数微码临时寄存器	32 位	存储整数值
1	EFLAGS	32 位	i486 系统标识寄存器
1	算术标识	8 位	i486 重命名标识
2	FCC	4 位	FP 条件码
1	EIP	32 位	指令指针
1	FIP	32 位	FP 指令指针
1	事件 UIP	12 位	事件报告微指令
2	FSW	16 位	FP 状态字

6.5.1.2　事件检测

事件包括错误、自陷、辅助和中断。再定序缓冲中的每一个表项都有一个事件信息域，执行部件将对这一区域进行更新。在提交的时候，提交逻辑将查看本次三个提交操作的信息域。事件信息域将显示提交逻辑是否有异常、是否发生错误、自陷或是需要辅助（assist）。中断由中断部件直接发出。万一出现了错误预测的分支，跳转部件将对事件信息域进行标记。

如果检测到事件，ROB将清除所有微操作，强迫MS跳转到微码事件处理器中。事件记录将保存起来，以使于微码处理器能够对结果进行正确修正或是调用正确的宏代码处理。宏指令和微指令指针同样予以保存，这使得程序在事件处理结束后进行恢复。

6.6　存储子系统

存储器定序缓冲（MOB）是P6存储器子系统的一部分。MOB 与处理器的乱序引擎相连，包含两个主要的缓冲：载入缓冲（LB）和存储地址缓冲（SAB）。这两个缓冲都是循环队列，每一个表项都单独代表一个载入微操作和存储微操作。为了有效处理存储操作，SAB 与存储器接口部件（MIU）的存储数据缓冲（SDB）、Dcache 的物理地址缓冲（PAB）之间协调运作。SAB、SDB 和 PAB 可以合起来看成一个缓冲，即存储缓冲（SB）。

LB 包含 16 个表项，可以保存 16 个载入操作。LB 将那些由保留站分派出的、但仍没有完成的载入操作排成队列。当冲突消除后，将重新分派队列中的操作。针对那些已完成的载入操作，LB 通过监视外部的写操作来保持正确的处理器载入顺序。处理器向推断读取的存储器地址发出的写操作将迫使乱序引擎清除并重新启动载入操作（载入操作将成为一个新的微操作）。

SB 包含 12 个表项，可以保存 12 个存储操作。SB 将所有的存储操作在它们被分派到存储器之前排成队列。当OOO引擎发出信号，通知这些存储操作将不再处于推断状态时，相关存储操作将以原始程序中的顺序进行分派。SAB用存储地址去检查载入操作并查看是否有冲突，使载入操作与已经执行完毕但仍处于 SB 中的存储操作保持一致。

当向保留站发射一个载入或者存储操作时，分配器将分配MOB资源。载入操作被译码成一个微操作，而存储操作将译码成两个微操作：存储数据（STD）和存储地址（STA）。在分配时，操作将以它在 LB 或 SB 中最终的地址（表项）进行标记，所有的标志称为 MOB ID（MBID）。可以将存储分成两个截然不同的微操作，这样可以在产生存储地址和数据时尽可能地实现其中的并行性。

MOB 接收来自于保留站的 LD 和 STA 操作。RS 提供微码，而地址产生部件（AGU）计算并提供访问所需的线性地址。Dcache 立即执行这些操作，或者稍后由 MOB 分派。无论哪种情况，它们

都将写入MOB数组的某一项中。在存储器的操作中，DTLB将线性地址翻译成物理地址，或者转换为页失效信号并由页失效处理器（PMH）进行处理。MOB将对线性地址及数据规模进行许多检查，以决定操作是继续还是阻塞。

在载入时，DCU应该将数据返回给内核，MOB将地址和状态位并行写入LB并通知操作已经完成。在STA操作的情况下，MOB将有效位写入SAB数组及再定序缓冲，即存储操作的地址部分已经完成。存储操作的数据部分由SDB执行。当数据已经接收并且已经写入缓冲时，SDB将通知ROB和SAB。MOB保留存储信息，直到ROB指示存储操作已经正常提交、体系结构状态得到确认为止。然后，它将操作从MOB分派到DCU并提交为存储系统状态。一旦提交完成，MOB将通知释放SAB资源，以便于分配器可以重新使用。存储器子系统将以程序最初的原始顺序执行存储操作。

6.6.1 存储器访问顺序

微操作寄存器操作数的相关性可以根据原始程序指令中寄存器的访问进行显式跟踪，但存储器操作具有隐含的相关性，载入操作与任何之前的、与其具有相同地址的存储操作都具有相关性。这些操作在MOB中进行的是推测执行，包括存储和载入，因此存储器访问将有可能返回陈旧数据并产生错误结果。

为了维护载入和存储之间的自身一致性，P6使用了一个概念，称为存储着色（store coloring）。每个载入操作都用一个存储缓冲ID（SBID）来标志，代表与载入相关联的地址，用来与执行序列中所有的存储操作进行比较。当存储器子系统执行载入操作时，MOB将使用SBID作为起始点，通过与缓冲中所有以前的存储操作进行比较来分析载入操作，但忽略后面的存储操作。

存储着色用于维护同一个处理器中load和store的有序一致性。在多处理器系统中也会出现类似的问题。如果载入是乱序执行的，则有可能使其他处理器的存储操作也表现为乱序，这是由于后续的载入超越了先前的载入而提前执行。后续的载入将会读取本该是先前载入应读取的陈旧数据，而先前的载入却读取了可能已经被其他处理器所改写的更新数据。如果对状态进行了确认，这些载入将违反处理器的顺序性规则（processor ordering）。为了避免这种情况，LB检查（监视）写往总线的所有数据。如果其他处理器所写的地址是推断读取的，那么推断完成的载入以及后续的操作将被清除，并重新执行以获得正确数据。

6.6.2 存储器load操作

load操作将从分配器和寄存器分配表（RAT）发射到RS。分配器向每一个发射到RS的load操作分派一个新的载入缓冲ID（LBID），同时也给load操作赋值一个存储色（store color），即以前分配过的最后一个存储操作的SBID。载入操作将在RS中处于等待状态，直到所需的操作数据就绪为止。一旦操作数就绪，RS在端口2上将load操作分派给AGU和LB。假设没有任何其他的分派在此端口上等待，LB将旁路此操作并由存储器子系统立即执行。AGU产生线性地址，此地址由DTLB、MOB和DCU使用。DTLB将地址翻译成物理地址的同时，DCU使用地址的低12位进行最初始的数据查找。同样，SAB使用低12位地址和存储色SBID来检查以前存储操作中可能存在的潜在冲突地址（“以前”是指程序中原始的先后顺序）。假设有一个DTLB页命中并且没有SAB冲突，DCU将使用物理地址进行最终的标志匹配，同时返回正确数据（假设没有失效和阻塞）。载入操作结束后，RS、ROB和MOB将标记它们的完成状态。

如果SAB检查到存在地址匹配，SAB将促使SDB通过定向路径传递SDB数据，而忽略DCU数据。如果存在SAB冲突，但是没有地址匹配（冲突），这时载入将会阻塞并写回到LB。它将一直处于等待状态，直到具有冲突的存储操作离开存储缓冲。

6.6.3 基本存储器存储操作

store操作分为两种微操作：存储数据（STD）和存储地址（STA）。由于一个存储操作是由这两个微操作组合而成，分配器只能在STD发射到RS后分配存储缓冲表项。SAB、SDB、PAB将使用相同的地址，即所分配的存储缓冲表项。当存储操作的源数据准备就绪，RS将4号端口上的STD分派到MOB并写入SDB。当STA地址的源数据准备完毕，RS将3号端口上的STA分派到AGU和SAB。AGU将产生出DTLB可以翻译的线性地址并写入SAB。假设DTLB发生页命中，物理地址将写入PAB。这时STA操作结束，MOB和ROB将更新它们的完成状态。

假设没有故障或者错误预测的分支发生，ROB将提交STD和STA。通过监测这次提交，SAB将存储操作（STD/STA对）标志为确认状态或者高级（senior）状态。一旦是高级状态，MOB在分派这些操作时将把操作码、SBID以及低12位地址发送到DCU。DCU和MIU使用SBID来访问PAB中的物理地址并将数据存储在SDB中，从而完成最终的存储操作。

6.6.4 延期存储器操作

一般情况下，我们希望大多数存储器操作都能够在RS分派后三个周期内完成（只比ALU操作多两个时钟周期）。但是，存储操作并不都像针对L1 cache的转换和操作那样可以预测，一些操作不是完全可以预测的。在这种情况下，存储器操作需要其他资源辅助处理，比如，DCU可能将一个未决的cache失效填充在缓冲中，而此时缓冲可能并不可用，该操作则必须要延期到资源可用时才能处理。

MOB LB使用通用机制来阻塞载入操作，直到接收到一个唤醒信号。与每个LB表项相关联的阻塞信息包含两个域：一个阻塞代码（阻塞类型）和阻塞标识符。阻塞代码标识阻塞的源信息（地址阻塞，PMH资源阻塞等）。阻塞标识符是与阻塞代码相关联的具体资源的ID。当收到一个唤醒信号时，所有与阻塞代码和标识符相匹配的延期存储器操作将被标识为“可以分派”。然后LB以一种十分类似于RS分派的方式对这些就绪操作进行调度和分派。

MOB SB使用一种约束机制来阻塞STA存储器操作。STA操作将一直被阻塞，直到ROB提交指针指示STA微操作已经是机器中等待最久的非提交操作。这一操作的分派将在DCU写提交发生的时候与STA的分派同时进行。对存储机制进行这种简化是因为STA很少阻塞。

6.6.5 页故障

对于所有存储器的载入和存储地址微操作，DTLB将存储线性地址翻译成物理地址。DTLB通过查询自身的缓冲数组进行地址翻译，这些缓冲数组保存了正在处理的页物理地址。DTLB也将物理地址的页属性缓冲起来，用来检查页保护故障和其他的相关异常。

DTLB所存储的地址只是所有可能的存储器页的一个子集。如果地址查询失败，DTLB向页失效处理单元（PMH）发出一个失效信号。PMH执行页表查找操作（page walk），从物理存储器中的页表中取出物理地址。然后PMH从片上存储类型范围寄存器（MTRR，Memory Type Range Register）中为物理地址查询有效的存储器类型，向DTLB提供物理地址和有效的存储器类型并存储在缓冲数组中（通常在处理器启动的时候配置这些MTRR）。

最后，DTLB为不同类型的故障，包括页故障、辅助、DCU结构故障检测和写回操作。这些对于数据和指令页都是必需的。DTLB也检查I/O以及数据的断点自陷，向DCU写回（对于存储地址微操作）或传递（对于载入和I/O操作）结果，而DCU负责为ROB写回提供数据。

6.7 小结

本章所描述的设计来自于本书作者的想法，但是也反映了设计者、微码编写者、验证人员和性能分析人员的智慧结晶。在经济大潮的驱使下，商业支配着Intel的设计方案，但我们力图在任何时候都遵循这样的原则：做出性能最好的选择并对这些选择进行量化分析。乱序、推断执行、超流水、超标量、微数据流、寄存器重命名以及无黏合多处理设计就是这种努力的结果。Intel在2002年生产出了10亿多片基于P6的微处理器，本章中所描述的许多基本观点会在下一代Pentium 4处理器中继续使用。

参考文献

Colwell, Robert P., and Randy Steck, "A 0.6μm BiCMOS Microprocessor with Dynamic Execution," *Proceedings of the International Solid State Circuits Conference*, San Francisco, CA 1995.

Papworth, David B., "Tuning the Pentium Pro Microarchitecture," *IEEE Micro*, August 1996.

Yeh, T-Y., and Y.N. Patt, "Two-Level Adaptive Branch Prediction," *The 24th ACM/IEEE International Symposium and Workshop on Microarchitecture*, November 1991.

Lee, J., and A.J. Smith, "Branch Predictions and Branch Target Buffer Design," *IEEE Computer*, January 1984.

6.8 习题

1. PowerPC 620并没有实现load/store的定向，而Pentium Pro则实现了这一方案。解释一下为什么这两种设计都有可能做出了正确的设计权衡。
2. P6在分支预测错误时以一种粗粒度的方式进行恢复，如图6.5所示。为什么说这种方案简化了再定序缓冲（ROB）和寄存器别名表（RAT）的分支错误预测恢复逻辑？
3. AMD的Athlon（K7）处理器采取了一些不同的方法将IA32的微指令动态地翻译成实际执行的机器指令。例如，在Pentium Pro上，只有一个存储器操作数的ALU指令（例如add eax, [eax]）将翻译成两个微操作：一个写往临时寄存器的load指令以及一个寄存器到寄存器的加法指令。相反，Athlon简单地将原始指令作为宏操作分派到发送队列，但是它将从队列中发送两次：一次作为load操作，一次作为ALU操作。指出至少两个这种指令集翻译所带来的微体系结构方面的好处。
4. P6的两级分支预测器具有推测和非推测两个分支历史寄存器。描述每个分支寄存器是在何时以及怎样进行更新的，并提供一些有关这一设计方案合理性的证据。
5. P6动态分支预测器由一个静态的预测器支持，它可以预测一些由于某些原因而不能被动态预测器预测的分支指令。静态预测发生在流水段17（参见图6.4）。有一种情况需要使用静态预测器，当BTB报告标识匹配错误时，说明这一特定分支没有分支历史信息。假设静态分支预测被证明是正确的，一种可能的优化方法是不在BTB中填充该分支（是静态可预测的），因为它有可能替换其他需要动态预测的分支。指出至少一种原因，说明这不是一种好方法。
6. 在P6发展的早期设计阶段，有两种不同的ROB：一种为整数ROB，另一种为浮点*ROB*。为了缩减圆片的面积，在P6的发展过程中，将它们组合到了一起。解释一下相对于统一的ROB，单独的I/F ROB有什么优点和缺点。

7. 从时序图中可以看出，P6的提交处理需要3个时钟周期。假设已知有一种方法可以实现ROB提交只需 2 个时钟周期。这种方法能获得实质性的性能提升吗？试给出相应的解释。
8. 在 6.3.4.2 节中，曾描述了当条件标志被正在运行的微操作局部写入时，发生的错误匹配停顿。给出一种可以防止在重命名过程中发生这种停顿的解决方法。
9. 在 6.3.5.3 节中，描述了 Pentium Pro 的 RS 分配策略。基于这一描述，P6 是集中式 RS 设计还是分布式 RS 设计？
10. 如果P6微体系结构必须支持一个包含判断谓词的指令集，那么对于寄存器重命名过程将会出现什么样的影响？
11. 如本章所述，P6微体系结构将存储操作分割成STA和STD操作来处理地址产生和数据移动。从微体系结构实现的观点来看，这样有什么意义？
12. 根据习题 11，如果存储没有被分割，则是否存在性能的提高（以每周期指令数统计）？试给出相应的解释。
13. 在 P6 微体系结构中，为避免将存储操作分割成单独的 STA 和 STD 操作，应该进行哪些改动？这样做可能对时钟周期产生什么样的影响？
14. AMD 最近宣布了 IA32 体系结构上的 x86-64 扩展，并加入了对 64 位寄存器以及 64 位寻址的支持。查看这些扩展（更多的信息可以在 www.amd.com 上获得），并大致列出在 P6 体系结构上为了适应这些附加指令需要做出的改动。

第7章　超标量处理器概览

超标量处理器将多指令发射技术应用于顺序执行的指令级体系结构中。使用指令级并行技术（ILP，instruction-level parallelism），可以不按照程序的顺序进行取指、译码、发射、执行和提交多条指令，这一过程对于外部是透明的。我们将多发射作为一种实现技术而不是体系结构技术，这样可以将超标量设计与VLIW的设计区分开来，而且这也是某种特定的指令集在各种实现上保持代码兼容性的基础。本章将回顾超标量设计的历史，同时研究一些重要的设计并对当前各种超标量处理器的体系结构及实现进行比较

7.1　超标量微处理器的发展

这一节将回顾超标量设计的历史，从IBM的Stretch及其后续设计（超标量ACS）开始，然后沿着超标量技术的发展道路一直讨论到当前的处理器。

7.1.1　单处理器并行的早期发展：IBM Stretch

我们现在称之为超标量指令发射的技术，最早的研究和实现开始于IBM的Stretch和由它直接衍生出来的IBM的机器。因为它的实现采用了积极的实现技术（如预译码、乱序执行、推测执行、分支预测失败恢复和精确中断），另外它还是IBM RS/6000的前继设计，所以有必要研究一下Stretch，它的另一个名字是IBM 7030。

Stretch的设计开始于1955年，那时IBM刚失去了一份为加州大学放射试验室（University of California Radiation Laboratory, Livermore Lab）设计高性能的十进制计算机系统的合同。IBM的竞争对手Univac（当时主要的计算机制造者）赢得了那份合同，并承诺在29个月内完成设计，这就是Livermore Automatic Research Computer（LARC）。当时IBM的标书中设计的机器速度比要求的速度快4倍到5倍，但需要350万美元，而不是要求的250万美元。

在接下来的一年中，IBM又积极地竞标Los Alamos科学实验室的一台二进制计算机，其设计要求是“速度至少要比当前的任何机器快100倍”，并于1956年赢得了合同，这就是Stretch。交货日期则定在1960年。Stephen Dunwell是项目负责人，他选择的主要设计人员为Gerrit Blaauw, Fred Brooks, John Cocke和Harwood Kolsky。其中，Blaauw和Brooks研究指令集设计的思想，这对于他们以后设计IBM S/360时很有帮助。Cocke和Kolsky建立了一个关键的仿真器，可以用来帮助小组研究各种结构的参数。Erich Bloch，后来成为IBM的首席科学家，在1958年被任命为工程经理，在他的领导下，于同年实现了原型单元并在1959年实现了一个工程模型。

设计小组选择了5个测试程序来进行模拟以确定机器的参数：一个流体动力学的网孔问题，一个Monte Carlo中的子扩散编码，第二个中子扩散编码的内层循环，一个多项式估值程序以及矩阵转置程序的内循环。Stretch有几条针对这种类型的科学计算指令，例如计数分支指令（branch on count），乘加指令（开始称为累加乘指令，后来称做FMA乘加指令），这些指令对于30多年后的RS/6000的性能有着重要的影响

Stretch的指令流经两个处理部件：一个是索引（indexing）和指令部件，它完成取指、预译码和部分地执行指令流；另一个是算术部件，它执行剩余的指令。Stretch也根据这种组织方式来划分

它的寄存器：索引和指令部件有16个64位的索引寄存器，算术部件的寄存器包括一组64位累加器和其他寄存器。按照功能部件的不同划分，不同的寄存器组也是RS/6000设计的一个特征。

Stretch的索引和指令部件（参见图7.1）将64位的存储字取到两个字的指令缓冲中。指令的长度可以是32位的，也可以是64位的，这样最多可以缓冲4条指令。索引和指令部件直接执行索引指令，为算术运算指令计算有效地址（比如，将索引寄存器内容和地址域相加）并开始读取存储器操作数。该部件本身就已经流水化并将指令执行和译码并行起来。取指逻辑有一个有趣的特征，即在所有的指令编码上增加预译码位。该操作每次处理一个字，这样两个半字的指令可以并行执行预译码。

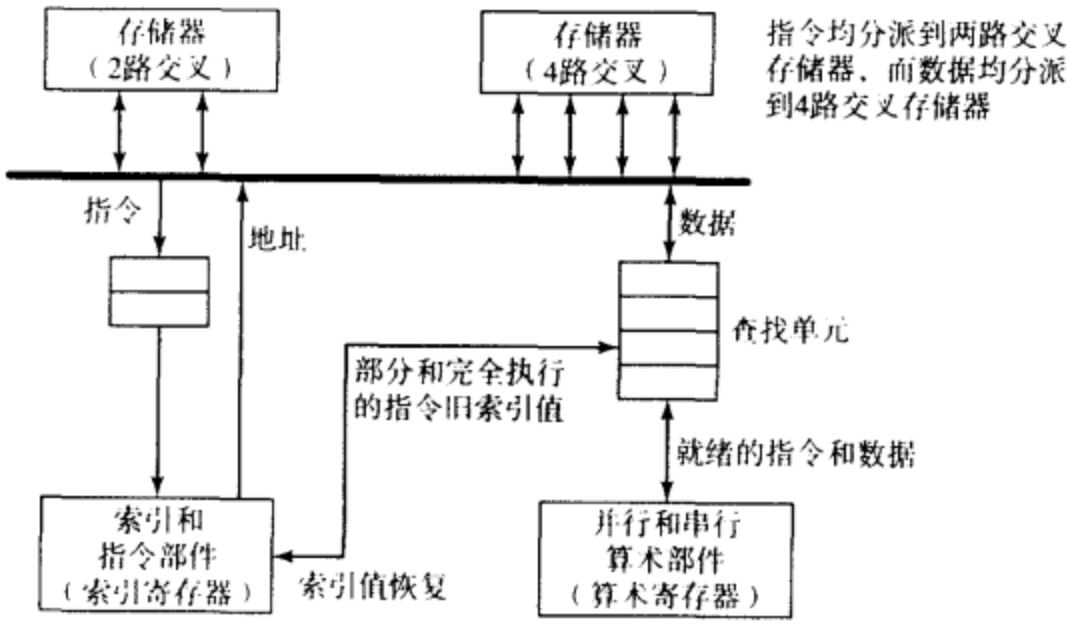


图 7.1　IBM Stretch 模块图

无条件分支和条件分支依赖于索引寄存器，例如根据指令计数判断循环结束的分支，可以完全在索引和指令部件中执行（RS/6000的分支部件）。依赖算术寄存器的条件分支总被预测为失败，并且分支失败的路径被推测执行。

所有的指令，包括完全执行和部分执行的（当时称为准备）指令，被送入一种新的缓冲结构中：查找单元，现在一般称为虚拟存储器，但是可以将它看做是由一个完成缓冲和一个历史缓冲组成的结构。对于一个完全执行了的索引指令（indexing instruction）和它的指令地址、任何已经被修改的索引寄存器中修改前的值，它们都可以放到4级查找单元中的某一级。保存过去值的历史信息，使得在分支预测失误或者中断的时候可以根据查找单元中保存的历史信息恢复索引寄存器的内容。部分执行的算术指令和它的指令地址也可以置于一个查找单元的某个级别中，同时，该指令等待存储器操作数就绪。一些复杂的指令被分离成独立的若干部分，从而在查找单元中可能需要占用多个级别。

只要算术指令的查找级别成为最高的，并且其存储器操作数已经就绪，该指令就可以被算术部件执行。和分支预测失误时的处理一样，使用查找单元保存的历史信息可以保证精确的算术异常。store指令也是当其查找级别变为最高的时候开始执行。当store指令位于查找单元的时候，通过检查查找单元中store指令后继的load指令的访存地址来实现写操作的重定向。如果读存储器的地址与写存储器的地址相符，load指令将被取消，store指令的值直接送入为load指令保留的缓冲中（当时称之为短路）。在查找单元中，每次只允许有一个未完成的store操作。而且，因为可能会对指令进行修改，store指令的访存地址要和查找单元每个级别中的指令地址进行比较。

Stretch的实现使用了169 100个晶体管和96 K的磁心存储器。索引部件和查找单元的时钟周期为300 ns（大于开始的估计值100 ns），域长度可变的部件和并行算术部件的时钟周期达到了600 ns。一条路径上的逻辑数量最多为23级并且其连接长度大约为15英尺[1]。并行算术部件每1.5 ns进行

[1] 1英尺 = 0.3048米

一次浮点加法，每 2.4 ns 进行一次浮点乘法。处理部件的功耗为 21 KW。CPU 不带存储器实现的面积为 30 平方英尺（长 5 英尺，宽 6 英尺）。

由于时钟周期的变化，Stretch 并没有达到设计初的性能要求。估计结果表明，其性能是 704 性能的60倍到100倍。在1960年，策划人员将Stretch的产品7030定价为1350万美元并提供给Livermore。估计它的性能为 7090 的 8 倍，而 7090 的性能则是 704 的 8 倍，这个估计主要是根据算术运算的时间得出的。

1961 年，当 Stretch 开始使用后，测试程序显示它的性能只有 704 的 4 倍。这一差别主要是由于 store 指令的延迟和对成功的算术分支指令预测失误所需要的恢复时间造成的，这两种情况都会使算术部件停顿。尽管 Stretch 是世界上最快的计算机（并且一直保持到 1964 年），但这种性能差异确实使 IBM 十分尴尬。1961 年 5 月，Tom Watson 在协商下将 7030 的价格降为 778 万美元，并且立即停止了其产品的销售。

尽管 Stretch 后来证明比预计的要慢并且比计划推迟一年推出，它还是给 IBM 在晶体管处理器的设计上带来了巨大的进步（IBM 此后 13 个月内推出了流行的 7090 系列），IBM 在计算机组织的概念上也有了巨大的进步（多道程序，存储保护，通用的中断，8 位字节和其他由 Stretch 首创的、后来用于 S/360 的概念）。Stretch 使用了大量的单处理器并行技术，包括乱序执行，推测执行，分支预测失误恢复和精确中断。它也是第一台使用交错存储器的机器，同时也是第一台缓冲 store 指令的值并给后继的 load 指令提供定向的机器。Stretch 为 John Cocke 以及其他继续研究多条指令并行译码的设计人员提供了很好的训练机会。

7.1.2 第一个超标量设计：IBM 高级计算机系统

1961 年，IBM 启动了两个高性能的项目，试图超过 Stretch 的性能。项目 X，也就是后来于 1964 年发布、1967 年正式推出的 IBM S/360 Model 91，其目标性能是 Stretch 的 10 倍到 30 倍。初始时钟周期的设计目标是 50 ns，Model 91 实际达到的时钟周期是 60 ns。Model 91 的浮点部件采用了 Robert Tomasulo 提出的算法，可以乱序执行指令并以此而闻名。

第二个项目，也就是项目 Y，其目标是设计一台速度 100 倍于 Stretch 的机器。项目 Y 的工作于 1961 年在 IBM 的 Watson 研究中心展开。由于该中心对 Stretch 的性能过于苛刻的要求，项目 Y 的工作曾一度停滞并推迟到 1963 年底。此时，发布了时钟周期为 100 ns 的 CDC 6600 超级计算机（该机器将标量指令发射和乱序指令执行结合在一起）。项目 Y 随后交给了 Jak Bertram 的计算机与编程实验小组；John Cocke, Brian Randell 和 Herb Schorr 开始负责电路工艺的定义、指令集和编译技术。1964 年底，6600 开始销售，另外，随着具有 25 ns 时钟周期的 6800（后来经过重新设计，命名为 7600）的发布，也给项目 Y 增加了紧迫感。Watson 决定设计一台更加先进的机器。在 1965 年 5 月，在 California 的 Menlo Park 建立了一个超级计算机实验室并由 Max Paley 和 Jack Bertram 领导。Herb Schorr 负责体系结构小组，Bob Domenico 负责电路小组，Fran Allen 负责编译小组，Russ Robelen 负责工程小组。体系结构小组的成员包括：Dick Arnold, Lynn Conway, Phil Dauber, Charlie Freiman, Don Rozenberg 和 Ed Sussenguth。电路小组的成员包括 Fred Buelow 和 John Zasio。工程小组包括 Bruce Beebe, Robert Blosk, Leo Hasbrouck, Dick Holleran 和 Merle Homan。1966 年，John Cocke 来到 California 并开始着手于编译器的工作。这项设计称为 ACS-1（Advanced Computer System 1）。

ACS-1 时钟周期的设计目标是 10 ns，性能是 7090 的 1000 倍（不再和未得到赏识的 *Strech* 进行比较了，这可能是公司的内部政策）。为了实现这一目标，ACS-1 的每个流水线段设计为 6 个门级逻辑并且电路的集成度很高。ACS-1 的设计目标包括优化的编译器，新的操作系统，流水化 I/O

通道，以及多线程磁盘等。ACS-1 计划于 1968 年推出，预期的客户是 Livermore 和 Los Alamos。但是在 1965 年底，其发布时间推迟到 1970 年以后。

ACS-1的处理器采用的译码方式是多条指令并行译码完成并且分派到两个保留站中，其中一个保留站提供乱序发射，参见图 7.2。1971 年，Schorr 在一篇关于 ACS-1 的论文中写道："多指令译码是该项目尝试的一种新的功能"。1994 年，Cocke 在一次采访中谈到，他是受 IBM 内部报告的启发而产生了多指令译码的想法。该报告是 Gene Amdahl 在 20 世纪 60 年代早期完成的。Amdahl 认为每个时钟周期只译码一条指令是对性能的主要限制。Cocke对报告中提出的各种基本限制进行了实验，认为多指令译码是可行的。

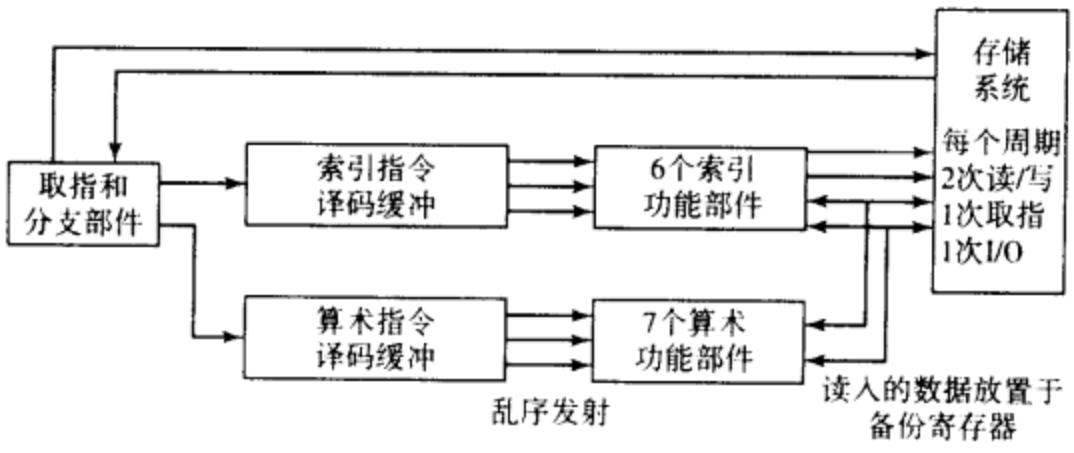


图 7.2　IBM ACS 模块图

尽管 Cocke 已经对多指令发射的方法提出了一些早期的建议，但到 1965 年底 Lynn Conway 才提出了设计中使用的通用化的动态指令调度方法。她描述了一个竞争栈（contender stack）并使用源和目的调度矩阵（scheduling matrix）和忙态向量（busy vector）来进行指令调度。当遇到条件分支的时候，就停止指令译码以及填写这些矩阵的操作。当分支解析出来之后，则恢复这些操作。为了给条件分支提供发射的优先级，还使用了逆序来扫描调度矩阵。

除了多指令译码和动态指令调度，ACS-1 在技术上、体系结构上、实现上和编译技术上提供了很多当今超标量处理器所使用的技术[Schorr，1971；Cocke，1988；Sussenguth，1990]。

另外，其技术上的创新还包括以下几点：

- ACS-1 处理器的面积计划为 1 平方米并以阵列的方式置于圆片上。每块芯片有几百个 I/O 引脚，使用焊接连线连接不同的芯片。连线使用压焊技术焊接在引脚上，任何超过 1.2 英寸[1]的连线都采用同轴的方式。Hardy 描述说："这看起来就像是芯片挂在蜘蛛网上"。每台机器都配有压焊设备并一起发售，可以将机器和示波器的采样端焊接在一起来进行诊断和修复。

ACS-1 体系结构上的创新包括：

- 类似于CDC 6600、寄存器 – 寄存器指令格式需要三个寄存器标识符。总共有 31 个 24 位的索引寄存器和31个48位的算术寄存器（单精度浮点数是48位的格式，双精度的浮点数是96位）。
- 包含 31 个备份寄存器（backup register），每一个寄存器都和对应的算术寄存器配对。这就给循环的多个迭代重叠执行提供了寄存器重命名机制，从而当 load 指令或者写回操作与前面操作的结果之间存在相关时，可以写入备份寄存器。

设计人员认识到，在分支上可能会损失一半甚至更多的性能，因而在减少分支的数目上采用了几种积极的技术：

[1] 1 英寸 = 2.54 cm

- Ed Sussenguth 和 Herb Schorr 将条件分支的处理动作分为三个独立的部分：分支目标地址计算，分支方向的判断以及 PC 的更新。ACS 将前面的两个动作组合为准备分支（prepare-to-branch）指令，使用 exit 指令来进行最后一个动作。这样就允许插入可变数目的分支延迟槽（称为对分支进行预测）。更为重要的是，这样可以实现多路分支。也就是说，可以执行多条准备分支指令、建立起多个分支的条件以及相关的目标地址，并且只有其中之一可以被 exit 指令使用。这样就只需要对指令计数器（PC）进行一次非顺序的更新。
- 设置了一组共 24 个条件码寄存器，允许提前计算分支的条件，也允许一条准备分支指令指定的包括任意两个条件码的逻辑表达式。这种方式在概念上和 IBM 的 POWER（RS/6000）和 PowerPC 体系结构中的设置 8 个独立的条件码相似。
- 为了处理偏移量较小的前向条件分支指令，在每种指令格式中加入了 1 位条件位。一种特殊形式的准备分支指令被用来作为条件 skip 指令。在条件解析出来的时候，如果 skip 指令中的条件为真，所有标记为条件执行的指令将从指令队列中移出。如果解析出来的条件为假，所有被标记的指令则去掉标记并许执行。skip 指令代替了一种常见的条件分支，在指令计数器中没有非顺序的变化。

ACS-1 实现上的创新包括以下几点：

- 索引操作包含 6 个功能部件：比较部件，移位部件，加法部件，分支地址加法和逻辑部件以及两个有效地址加法器。算术操作包含 7 个功能部件：比较部件，移位部件，逻辑部件，加法部件，除法和整数乘法部件，浮点乘法部件和一个附加的浮点加法器。
- 每个时钟周期可以发射 7 条指令：3 条索引操作（其中的两条可以为 load/store 指令），3 条算术操作和 1 条分支指令。每个时钟周期内，对一个 8 项的 load/store 索引指令缓冲进行译码，并且每个周期可以按序发射至多 3 条指令。每个时钟周期内，还对一个 8 项的算术指令缓冲进行译码，以寻找 3 条就绪的指令并且乱序发射这些指令。load 指令被送入索引和算术队列以保证指令的顺序。
- 与 Stretch 一样，设计人员希望 ACS-1 处理器索引部分的速度快于算术部分，这样 load 指令就可以尽早地开始，从而隐藏存储器访问的延迟。1982 年，这种通用技术由 Jim Smith 命名为分离的访问执行结构（decoupled access-execute architecture），参见 7.1.4 小节。在 Smith 的实现方案中，索引部件和算术部件是通过 FIFO 缓冲连接，而不是 ACS-1 中使用的备份寄存器。
- 使用了动态分支预测（1 位历史位）将指令预取到译码器中，但是不进行推断执行。研究表明，在恢复预测失误时会引起性能问题。Ed Sussenguth 建议使用一个包含 12 项的目标指令 cache，cache 的每一项包含 8 条指令，用于提供目标地址，从而消除成功分支的 4 个周期的开销。
- 在任何时候，最多可能有 50 条指令处于执行的某个阶段，所以中断和异常的开销很大。这样，大多数的外部中断都由硬件转化为对应的中断处理例程的分支指令，然后将这些分支指令插入到指令流中，这些分支指令带有特殊标记，从而保证前面发射的指令执行完毕（这种方式称为软中断）。算术异常存在两种处理模式：多发射或者顺序发射。在多发射的情况下，中断不能精确定位。S/360 Model 91 使用了同样的方法，类似的技术也用于 IBM POWER（RS/6000）
- 存储器流水化并且是 16 路交错的。

- 1965 年，IBM 在设计中引入了 cache 存储器，从而导致了 S/360 Model 85 的发布。ACS-1 采用了 cache 存储器，使用了一个 64 K 字的统一指令和数据 cache。ACS-1 所采用的 cache 为两路组相联，块大小为 32 字并使用 LRU 替换策略，每个时钟周期可以取 1 块共 8 条 24 位的指令。cache 命中需要 5 个周期，cache 失效需要 40 个周期。I/O 操作则通过 cache 进行。
- 使用一个 store 缓冲，为 load 定向提供地址冲突互锁机制。

最后，ACS-1 编译技术上的创新包括：

- 与机器并行开发了一个优化的编译器，优化策略包括指令调度，寄存器分配和全局代码移动，设计人员为 Fran Allen 和 John Cocke［Allen,1981］。模拟表明，对于一些实例，编译器能够产生出比手工优化效果更好的代码。Fran Allen 称赞这项工作为程序分析和机器相关或机器无关的优化提供了基础。

整个设计需要 240 000 个晶体管：其中 50% 用于浮点，24% 用于索引，剩余的 26% 用于指令排序。在一个集成电路圆片上，每个电路的功耗大约为 30 mW。处理器整个功耗大约在 7 kW 以上。

设计和编译小组都很注重对 6 个测试内核的研究，其中一个是双精度浮点内积。Cocke 估计机器对于这种类型的线性代数代码可以达到每个时钟周期完成 5 条到 6 条指令。Schorr［1971］和 Sussenguth[1990]包含了在 IBM 7090, CDC 6600, S/360 model 91 和 ACS-1 上运行简单循环（拉格朗日流体力学计算）和复杂循环（牛顿扩散）的性能比较结果，这些比较的结果参见表 7.1。

表 7.1　ACS-1 性能比较

	7090	CDC 6600	S/360 M91	ACS-1
LHC 的相对性能	1	50	110	2500
ND 上的相对性能	1	21	72	1608
ND 上保持的 IPC	0.26	0.27	0.4	1.8
ND 上的性能限制	机器的时序特性	取指	分支	算术

此外，还给出了几种机器的性能分析比较，分析的方法根据每种机器的电路数目和电路速度将相对性能标准化，其结果称为相对体系结构因子（relative architectural factor），但是结果还要受到存储器访问时间的影响（是否有 cache 存储器）。按照这种分析方法并且使用 7090 作为标准化的基准，Stretch 的因子是 1.2，CDC 6600 和 Model 91 的因子都是 1.1，Model 195（带 cache）的因子为 1.7，ACS-1 的因子是 5.2。

ACS-1 与 IBM 的其他项目存在着竞争。20 世纪 60 年代末，ACS-1 的设计和 IBM 的 S/360 体系结构并不兼容，这导致了它在公司内部失去了支持。1965 年，Gene Amdahl 加入了 IBM 公司并且来到 California 担当 Paley 的顾问，他重新设计了 ACS-1，以提供对 S/360 的兼容性。在 1968 年初，这一计划被管理层接受。然而，这一决定使项目处于一种混乱的状态，设计小组中大约有一半人离开了。

为了保持体系结构的兼容性，Amdahl 的 ACS-360 必须抛弃具有革新意义的分支机制，而且必须提供强排序的存储模型和精确的中断。ACS-360 的一个创新之处是 SIOF（Start I/O fast release），这样，高性能的处理器不会因为 I/O 通道的初始化而变慢。S/370 的体系结构也包含这一特征。但是，Amdahl 不得不在一个新的测试程序上进行测试，该测试程序包含了一个商业组件，所得到的 IPC 小于 1。1968 年，在仿真器中加入了第二个指令计数器和第二组寄存器，从而使 ACS-360 能够支持多线程。指令使用额外的红、蓝位标记指令流和寄存器组。正如设计人员所期望的那样，功能部件的利用率因为有了更多的不相关的指令而得到了提高。

1969年，ACS的电路设计还存在问题。另外，由于基于cache的S/360 Model 85的成功，以及国民经济的发展速度放慢、公司内部东、西海岸之间紧张的关系，导致ACS-360项目被取消［Pugh等人，1991］。Amdahl很快离开了IBM并经营了自己的公司。进一步的工作于20世纪90年代开始并在超标量的IBM S/370上进行。除了25年之后发布的ES/9000 Model 520以外，其间并没有发布相关的设计信息。

7.1.3 指令级并行研究

在20世纪70年代早期，发表了两项重要的、关于多指令译码和发射的研究成果：一项是Gary Tjaden和Michael Flynn[1970]发表的，另外一项是Gaxton Foster和Ed Riseman［Riseman and Foster, 1972］发表的。在1966年离开IBM以前，Flynn是IBM S/360 Model 91的项目经理，他曾经参观过ACS小组。他记得自己对多指令译码思想的印象非常深刻，后来他和他的学生Tjaden研究了7094中互锁和控制的潜在问题。Flynn也发表了关于多指令译码的第一个公开文献，也就是后来成为经典的、关于SISD/SIMD/MIMD的论文［Flynn, 1966］。

当Tjaden和Flynn专注于IBM 7094的多发射译码逻辑时，Riseman和Foster研究了CDC 3600的程序中分支的效果。两个小组都公布了它们的报告：在测试程序中可以获得的并行性很小（分别是每个时钟周期1.86条指令和1.72条指令）。然而，Riseman和Foster发现，如果知道哪条路径被执行，就能减少分支数目，从而增加并行性。

这些论文的效果是负面的，它们打击了对于细粒度、单程序并行的研究热情。直到20世纪80年代，Josh Fisher和Bob Rau的VLIW设计［Fisher, 1983; Rau等人, 1982］以及Tilak Agerwala和John Cocke的超标量设计，使得设计者确信了通用的多指令译码及发射的可行性，从而又出现了众多的设计成就。

7.1.4 DAE的副产品：第一个多指令译码的实现

20世纪80年代初期，Jim Smith在Wisconsin大学提出了访存和执行分离的体系结构（decoupled access-execute architecture）［Smith, 1982; Smith和Kaminski, 1982; Smith, 1984; Smith等人, 1986］。在1982年ISCA的论文中，他称Stretch是第一台访存和执行分离的机器，并允许load指令尽可能早地开始。Smith的扩展设计中包括load指令和store指令使用的队列，这种队列对体系结构是可见的。计算指令可以访问寄存器或载入数据队列。在他的思想指导下，设计并开发了双发射的Astronautics ZS-1（参见图7.3），参见［Smith, 1986; Smith等人, 1987］。

ZS-1从存储器中取出64位字放入指令分解器（splitter）中。指令的长度可以是32位的，也可以是64位的。所以指令分解器每个时钟周期最多可以取得两条指令。分支指令是64位的，它完全在指令分解器中执行并且可以从指令序列中删除。未解析的条件分支将使指令分解器停顿。访存（A）指令放入一个有4项的A指令队列，执行（X）指令放入一个有24项的X指令队列。指令发射在这些指令队列中进行。发射要求不存在相关和冲突，并由指令决定操作数是来自寄存器还是来自载入数据队列。访存处理器（Access processor）包括3个执行部件：整数ALU，移位和整数乘除部件；执行处理器（Execute processor）包括4个执行部件：逻辑，浮点加法器，浮点乘法器和倒数估算部件。每个时钟周期每个处理器最多可以发射1条指令。

在1982年的ISCA论文中，Smith还引用了CSPI MAP 200阵列处理器（array processor）作为访存和执行分离的一个实例［Cohler and storer, 1981］。MAP 200由访存和执行分离的处理器组成，两个处理器由FIFO队列耦合起来，但是每个处理器有自己的程序存储器，并由程序员来负责两个处理器之间的协作。也许这一设计启发了阵列处理器的设计者Glen Culler，他在1986年推出了双发射的DAE机器Culler-7［Lichtenstein, 1986］。

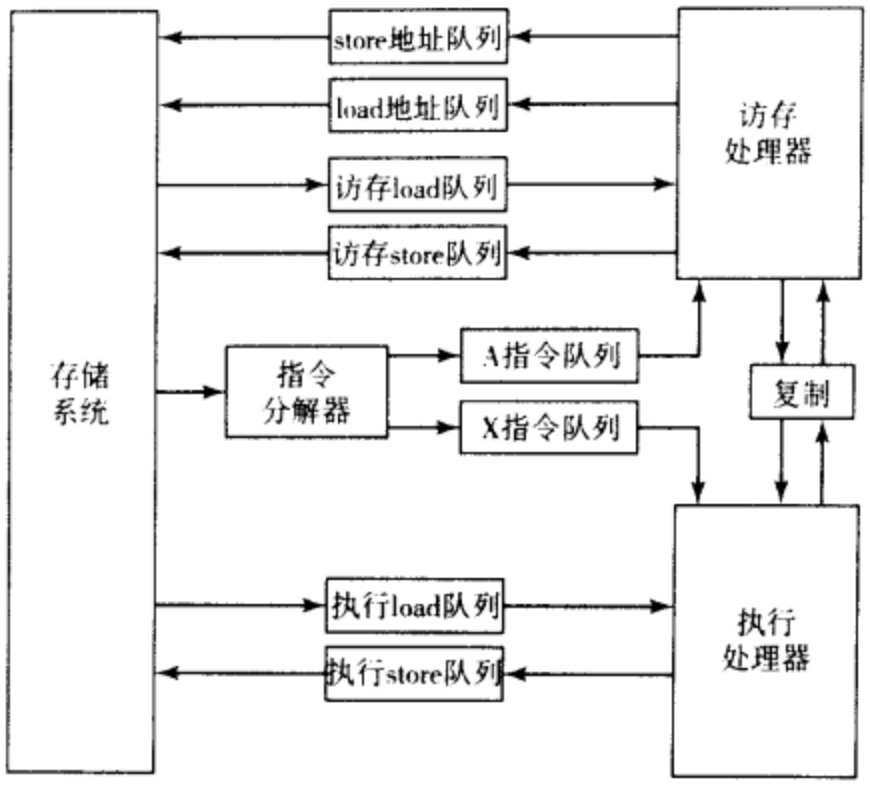


图 7.3　Astronautics ZS-1 模块图

Culler-7 是一个多处理器结构，包含一个基于 M68010 的核心处理器和 4 个用户处理器。每个用户处理器都是一个 A 机器和一个 X 机器的组合，A 机器用来控制程序的顺序和数据存储器的寻址和访问，X 机器用于浮点计算并且可以和 A 机器并行运行。A 机器和 X 机器使用一个 4 项的输入 FIFO 缓冲并和一个单项的输出缓冲耦合在一起。程序存储器包含 X 指令的序列，有时和 A 指令配对，有时后面跟有 1 条 A 指令。X 指令用于查找微码控制例程，这些例程是平行的微指令序列，用来指定浮点加法器、乘法器、两个 4 K 项的便笺式存储器以及各种寄存器和总线的操作。单精度的浮点操作是单周期的，双精度的浮点操作需要两个周期。用户编码的微码例程也可以放在控制微码的存储器中。如果存在 X 和 A 指令对，则它们将被一起取指、译码和执行。常见的顺序是一条 X 指令启动一个微码例程，后面跟有一系列 A 指令以提供必要的存储器访问。第一对必须被一起取指和执行，后面 A 指令的取指、执行和多周期的 X 指令的执行重叠。X 机器和 A 机器之间的输入输出缓冲是互锁的，并由程序员和编译器来负责防止死锁（例如，忽略下一条 X 指令需要的 A 指令）。

ZS-1 和 Culler-7 是第一批多指令并行取指、译码和发射的商用处理器。这种访存和执行指令的双发射方式在后面的设计中还会出现很多次（虽然不一定有 FIFO 缓冲）。在这种双发射方式中，一个整数部件负责整数和浮点的访存操作，并可以和浮点部件中的浮点计算指令并行发射和执行。

7.1.5　IBM 的 Cheetah, Panther 和 America

20 世纪 80 年代初，在 John Cocke 的要求和支持下，IBM 研究中心的 Tilak Agerwala 启动了一个双发射的项目，称为 Cheetah。该设计采用了 ACS 的思想，例如备份寄存器，以及来自 IBM 801 RISC 实验机器的思想。IBM 801 RISC 也是 John Cocke 的项目（1974 年 ~1978 年）。RS/6000 中的三个逻辑部件，即分支、定点（整数）和浮点部件，是在 Cheetah 中第一次采用的。1986 年，Cheetah 小组成员 Pradip Bose 在 FJCC 上发表的一篇关于编译器的研究论文中，介绍了一些双发射的机器，例如 Astronautics ZS-1 和 IBM 的设计。

从 1983 年到 1984 年，Agerwala 在几所大学邀请的谈话中首次公开使用了他为 ACS 和 Cheetah 及类似机器创造的名词：超标量（superscalar）。该名词有助于描述多指令译码机器的潜在性能，特

别是区别于向量处理器。这些谈话（有些可以从录像带上得到）以及IBM的一份相关技术报告，重新燃起了人们对多指令译码研究的兴趣。随着 Jouppi 和 Wall 的论文 "Available ILP"，以及 Smith, Johnson 和 Horowitz 的论文 "Limits on Multiple Instruction Issue" 在 ASPLOS-III［1989］上的发表，超标量和 VLIW 已经成为研究的热点。

Cheetah/Panther 的设计在 1985 年到 1986 年得到了进一步的发展，并产生了 4 路发射的设计方案，即 America。Greg Grohoski 在［Special issue, IBM Journal of Research and Development, 1990］中对其进行了描述。设计小组包括负责人 Grohoski, Marc Auslander, Al Chang, Marty Hopkins, Peter Markstein, Vicky Markstein, Mark Mergen, Bob Montoye 以及 Dan Prener。在设计中，一个通用的寄存器重命名器件取代了备份寄存器，一种更为积极的分支隐藏（branch-folding）方法取代了 Cheetah 的延迟分支方法。1986 年，IBM Austin 开发实验室采用了 America 的设计方案，并将它完善为 RS/6000 体系结构（也称为 RIOS 和 POWER1）。

7.1.6 分离的微体系结构

20 世纪 80 年代中期，加州大学伯克利分校的 Yale Patt 和他的学生 Wen-Mei Hwu, Steve Melvin 和 Mike Shebanow 建议将 IBM S/360 Model 91 的 Tomasulo 浮点部件通用化，他们称之为受限数据流（restricted data flow）。其关键思想是，一条顺序指令流可以动态地转换成半数据流图，从而以数据流的方式执行。指令流译码的结果保存在译码指令 cache（DIC, Decoded Instruction cache）中，该缓冲将指令译码部分和指令执行部分分开。

在 HPS（High-Performance substrate）项目中，Patt 和他的学生认为无论目标指令集有多复杂，半数据流图的节点保存在 DIC 中能简化操作和微指令。他们将这一观点实践于 VAX ISA 中，发现每条 VAX 指令平均需要 4 条微指令，一个受限数据流的实现可以将 VAX 指令流的 CPI 从 6 减少到 2［Patt 等人, 1986; Wilson 等人, 1987］。这种将 CISC 指令流翻译成动态调度的、类似 RISC 微指令流的方式成为 NexGen Nx586、AMD K5 和 Intel Pentium Pro 等处理器的基本技术之一。

20 世纪 80 年代的中期，另一个独立的项目也在 AT&T Bell 实验室展开，目标是设计一个分离的标量流水线，该项目是 C 机器项目的一部分。最后得到结果是 CRISP 的微处理器，参见［Ditzel 和 McLellan, 1987; Ditzel 等人, 1987］中的描述。CRISP 在变长指令经过三段的译码流水线时将其翻译成固定长度的格式，包括次地址域（next-address field）并保存在 32 项的 DIC 中。三段的执行流水线将取出译码之后的指令并执行。Motorola 68060 从该设计中得到了很多启发并使用了 CRISP 的分支隐藏的方法，通过将分支条件和目标地址与按照指令顺序在分支前面的那条指令相关联，从而从译码后的指令流中删除了分支。通过使用这种方法将分支指令的执行和计算指令的执行重叠起来，CRISP 每个时钟周期能执行一条以上的简单指令。

7.1.7 20 世纪 80 年代其他的设计方案

20 世纪 80 年代，人们进行了多种多指令发射方法的研究。H.C. Torng 在 Cornell 大学研究了类似 Cray 机器的多指令发射方式，开发了一种乱序多指令发射的机制，称之为分派栈（dispatch stack）［Acosta 等人, 1986］。1986 年推出了 Stellar GS-1000 图形超级计算机工作站［Sporer 等人, 1988］。GS-1000 使用 4 路多线程，12 段的流水线处理器，指令流中相邻的两条指令可以被打包，从而能在一个时钟周期内执行。

Apollo DN10000 和 Intel 的 i860 是 20 世纪 80 年代晚期推出的双发射处理器，但是由于这两个处理器都是在编译时进行双发射的准备，因此它们被归结为超长指令字体系结构而不是超标量体系

结构。Apollo在整数指令格式中使用了一位并指明是否为相伴浮点指令（紧跟在整数指令后面的浮点指令，这一对指令是双字对齐的），这一对指令应该被双发射。i860在浮点指令格式中增加了一位来表明双操作模式，其中，对齐的整数指令和浮点指令同时取指和执行。由于流水化的出现，i860中这一位的效果将控制下一对指令的双发射。

国家半导体公司（National Semiconductor）的Swordfish处理器于20世纪80年代末在以色列开始研发，其体系结构是预译码和超长指令字微体系结构的组合。其设计特征包括双整数流水线（A和B）和一个多单元的浮点处理器。已译码指令cache中的表项被组织成指令对的形式。在指令cache填充的时候（在设计中称为指令装载），对指令进行预译码，提前计算分支的目标地址并检查所需功能部件，以及双发射寄存器的相关关系。如果双发射是可能的，就对cache中的指令对进行特殊的标记。

无论是否实现了双发射，Swordfish的cache项中的第一条指令总是送入流水线A，第二条指令将会送给流水线B和浮点流水线。程序定序指令只能在流水线B执行。为了支持0周期循环，所有的分支都预测为成功。load指令可以在两条流水线上执行，因此可以将load指令发射到流水线A，同时将分支指令或者浮点指令发射到流水线B。load指令还有一个FIFO缓冲作为保留站，这样在数据cache失效的时候，流水线还可以继续执行。数据cache的填充需要6个周期，如果与之相关的指令进入译码段时填充还没有完成，寄存器记分牌将停止该指令的译码。流水线B和浮点流水线以协同的方式执行。在浮点流水线可能发生自陷的情况下，流水线B将访存段执行的周期数加倍，这样流水线B和浮点流水线就能够同时进入写回段。这样就保证了按序提交和浮点流水线的精确中断。

7.1.8　超标量被广泛接受

1989年，Intel发布了第一块单芯片超标量处理器i960CA，它是i960嵌入式处理器体系结构的一种3发射实现［Hinton,1989］。同样在1989年，IBM发布了第一个超标量工作站RS/6000。关于IBM RS/6000的三篇论文在同年10月的国际计算机设计会议（International Conference on Computer Design）上发表。1990年出现了使用微码技术的Tandem Cyclone，只要有可能就执行特定的微程序，在这种微程序中两条指令同时执行［Horst 等人, 1990］。Motorola在1991年推出了双发射的88110。大型机的制造商也在尝试超标量设计：1991年Univac发布了A19，1992年Lipay［1992］展示了IBM ES/9000 Model 520。20世纪90年代早期发布了很多超标量设计，包括用于个人计算机（PC）的、双发射的Intel Pentium和3发射的PowerPC 601。1995发布了4种主要的微处理器核：HP 8000、Intel P6（Pentium Pro, Pentium II, Pentium III的基础），MIPS R10000和UltraSPARC-I（UltraSPARC-II的基础），它们在过去的7年中是计算机系统的主力军。Intel最近推出了最新的Pentium 4核心，Sun推出了UltraSPARC-III。从1995年的K5开始，AMD积极地开展了多个超标量设计的研究，直到目前的Athlon（K7）和Hammer（K8）。IBM和Motorola也在POWER和PowerPC系列中推出了几种设计方案。然而，一些系统制造商，例如Compaq（现在是HP）和SGI，调整或者取消了它们的超标量设计计划并准备采用Intel Itanium系列处理器，Itanium是一种新的EPIC（Explicitly Parallel Instruction Computing，显式指令并行计算）体系结构。例如，从1992年的双发射21064（EV5）发展起来的Alpha系列处理器，其8发射实现21464（EV8）在2001年被取消。

图7.4显示了在超标量设计上有重要意义的设计、论文和商业处理器的时间表，然而，更多的超标量处理器并没有在这张图中反映出来。

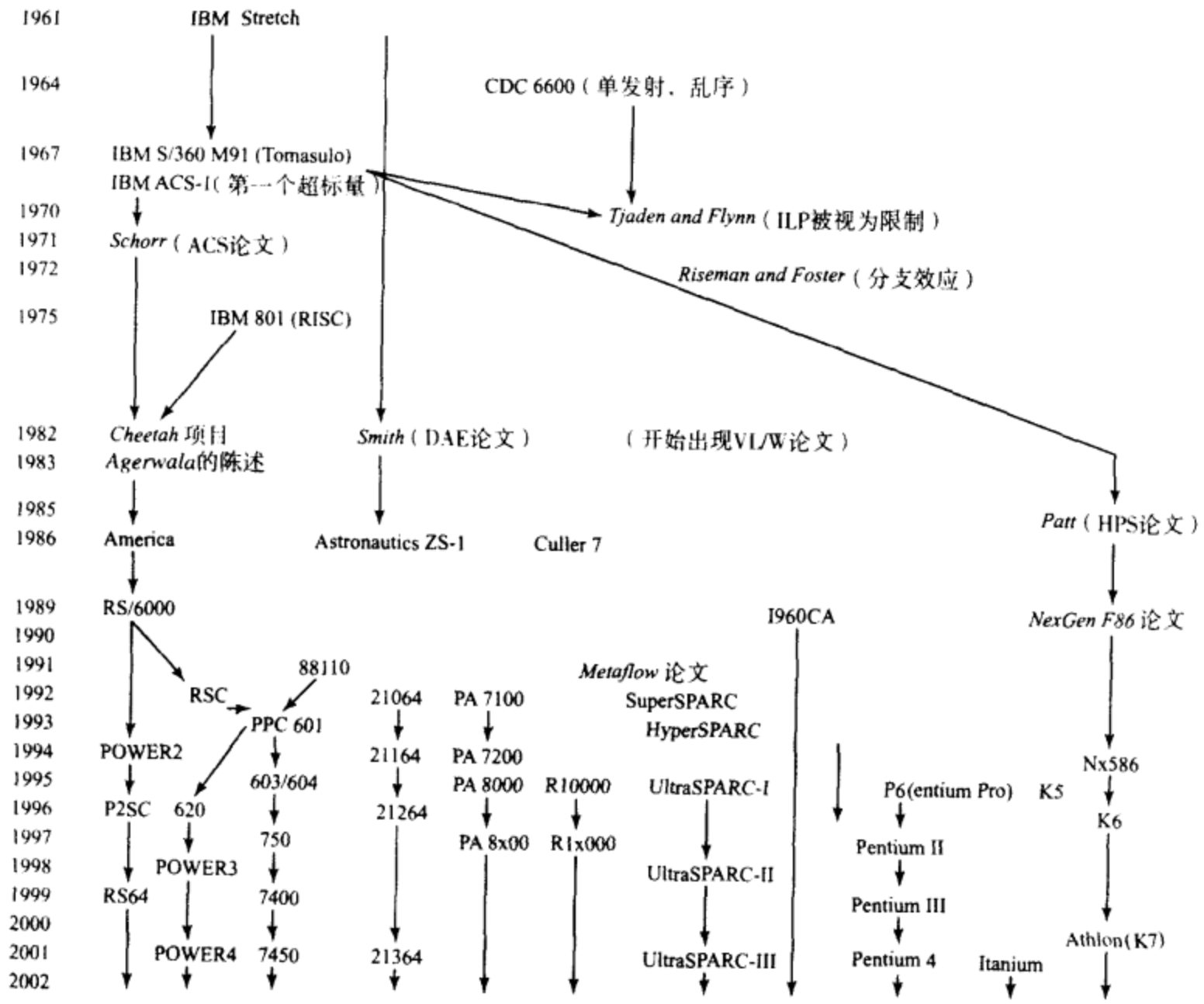


图 7.4 超标量发展的时间表

7.2 对目前设计的分类

这一节将对目前的超标量设计进行分类。一些处理器将超标量技术用于对流水化的ISA进行翻新，另一些处理器和体系结构的设计目标是最快的时钟周期和最少的限制（称为“速度型”），还有一些处理器和体系结构的设计目标是增强指令的多发射（称为“智慧型”），最后一些是对已有体系结构的修订，我们必须区别这些不同设计方式的特点。

7.2.1 对 RISC 和 CISC 的翻新

许多制造商选择在IBM RS/6000的性能水平上进行竞争，通过将超标量技术应用到20世纪80年代的RISC体系结构上，这些RISC体系结构一般都是面向单条整数流水线进行优化，或者将超标量技术应用到传统的CISC体系结构上。这种翻新的方法有6种类型，或者6个级别的复杂程度（改写自 Shen 和 Wolfe [1993]）。这种分类可以看做是设计要点，而不是严格的按照时间先后来排列的。例如，1996 年，QED 选择对 200 MHz 的 MIPS R5000 使用第一个设计要点，用 SPEC95 测试时得到了很好的结果：SPECint95 的性能和 SPECfp95 的性能分别是 200 MHz 的 Pentium Pro（属于第 6 级设计）的 70% 和 85%。

1. **浮点协处理器方式：**

- 这些处理器不能够在一个时钟周期内发射多条整数指令，也不能在同一个时钟周期内发射一条整数指令和一条分支指令；但是发射逻辑允许同时发射一条整数指令和一条浮点指令。这是对流水化的RISC所进行的最早扩展。通过允许整数部件执行浮点load和store操作以及维护索引寄存器和循环控制来提高浮点程序的性能。
- 例子：HP 的 PA-RISC 7100 和 MIPS R5000。

2. **整数指令和分支指令：**

- 这种类型的处理器能够同时发射整数指令和分支指令。
- 例子：Intel 的 i960CA 和 HyperSPARC。

3. **多条整数指令发射：**

- 这些处理器包括多个整数部件，允许整数指令的双发射，或者整数指令和存储器指令的双发射。
- 例子：HP 的 PA-RISC 7100LC，Intel i960MM 和 Intel Pentium。

4. **相关的整数指令发射：**

- 这种类型的处理器使用级联或者 3 输入的 ALU 来实现相关的整数指令的多发射。推测执行是有条件的并且不需要复杂的恢复机制。
- 例子：SuperSPARC, Motorola 68060和IBM POWER2（尽管POWER2是受限的乱序执行）。

5. **支持精确中断的多功能部件：**

- 这种类型的处理器强调精确异常模型，具有复杂的恢复机制，功能部件较多而发射约束较少。一般都支持分支预测和推测执行并使用回写来完成执行。也可能使用分布式保留站进行受限形式的乱序执行。例如，一个功能部件可以将等待执行的指令进行排队，而同时其他的功能部件可以执行后续其他类型的指令。
- 例子：Motorola 88110 和 PowerPC 603 和 604。

6. **扩展的乱序执行：**

- 许多处理器允许受限形式的乱序执行，例如第5级所描述的机制，而这种类型的处理器支持所有指令完全乱序执行。除了一般的流水段，还有一个分派段。在分派段，指令被送入集中的保留站或者一组分布的保留站中。另外还有一个提交段，允许推测指令通过回写来完成执行，但并不修改寄存器的体系结构状态，只有当指令执行到提交段时才会修改寄存器的体系结构状态。这种实现是一种受限数据流的机制。
- 例子：Intel 的 P6 和 HAL 的 SPARC64。

表 7.2 给出了在第 5 级和第 6 级设计中的各种缓冲策略。

表 7.2　乱序机制

处理器	保留站结构（表项数目）	复制操作数	再定序缓冲表项	复制结束
Alpha 21264	分布（20, 15）	无	20（4 指令）	无
HP PA 8000	分布（28, 28）	无	组合的保留站 / 再定缓冲	无
Nx586	分布（14, 14, 14）			
AMD K5	分布（2, 2, 2, 2, 2, 1）	有	16	有
Intel P6	集中（20）	有	40	有
AMD K6	集中（24）			

（续表）

处理器	保留站结构（表项数目）	复制操作数	再定序缓冲表项	复制结束
MIPS R10000	分布队列（16, 16, 16）	无	32项的活动列表	无
PPC 603	分布（1, 1, 1, 1, 1）		5	
PPC 604	分布（2, 2, 2, 2, 2, 2）	有	16	无
PPC 750	分布（1, 1, 1, 1, 2, 2）	有	6	无
SPARC64	分布（8, 8, 8, 12）	有	64项的A环	无

7.2.2 Alpha：一种侧重于时钟周期的体系结构

第二组设计很少受多发射的限制，因为它们的指令级集（ISA）从开始设计的时候就考虑了多发射的问题。DEC的体系结构设计者Richard Sites和Rich Witek明确反对一些在他们看来与RISC实现不相容的设计：延迟分支和单源操作数乘除法以及字符串寄存器[Sites, 1993]。他们还摒弃了模式位、条件码、精确的算术异常和严格的存储器定序。令人惊奇的是，他们还抛弃了字节和16位字的load/store操作，这两种操作要求在存储器和处理器之间有移位和掩码网络，以及一套“读-修改-写”的排序机制。他们的设计使用简单的指令序列并用软件方法来进行字节和字的操作。后来，这被证明是一种错误的设计方案，特别是在Alpha上仿真IA32的时候。在1995年，字节和半字的load/store指令被加入到Alpha体系结构中，并且在21164A上得到支持。

Alpha的主要目标是高时钟频率和快速执行整数指令，在Alpha的第一个版本21064中，使用了一种超流水和超标量相结合的方法。然而，21064属于前面的翻新设计类型的第1级设计，因为每个时钟周期可以发射一条整数指令，而该指令不能和整数分支指令一起发射。后来的21164是第3级的设计，每个时钟周期可以发射两条指令。另外一种观点是，不考虑超流水的结构，而是察看每两个时钟周期完成了什么操作（特别地，Alpha 21064的时钟频率是同时代处理器芯片的2倍甚至更高）。在这种观点下，21064是一个4路发射的设计，同时对相关指令的发射带有中等限制（也就是它们不能在同一个双字以内），这样看来它属于第4级设计。

21264在性能上同样取得了很大的进步，它属于第6级设计并支持乱序发射。后来的21364使用了相同的核心。21464是一个8发射的设计，但在2001年被Compaq公司取消。最近，Compaq和HP实现了合并并将它的高端产品移向了Intel的Itanium系列处理器。

7.2.3 POWER系列：侧重于增强指令的体系结构

第三组处理器和体系结构也具有从零开始的优势。事实上，这一组是开始得最早，也是进行得最为复杂的工作之一。由Rich Oehler领导的IBM POWER的设计人员以Agerwala和Cocke的思想为基础，在体系结构中定义了三个独立的部件，每个部件都有自己的寄存器组。这样，整数部件（IBM称为定点部件）有整数寄存器组，浮点部件有浮点寄存器组，一个体系结构可见的分支部件有特殊的分支寄存器组，从而各个部件可以和其他部件并行执行。这种方法减少了指令发射数目较少的机器的实现难度，因为不同部件的指令不需要进行复杂的相关性检查来识别共享的寄存器（参见Weiss和Smith[1994]）。

快速的浮点运算也是设计的一个目标，结果是引入了乘加指令，这样使用单条指令就可以完成内积运算需要的两个浮点操作。另外，还引入了其他一些复杂的指令，例如字符串操作和load/store指令的索引寄存器自动更新指令，目的是减少目标代码的指令数目并同时减少CPI，而不是单纯地减少CPI。同时，使用了1位模式位来保证精确的算术异常：置为1使得浮点的执行串行化（保证精确异常），置为0使得执行速度更快但是为非精确中断。POWER体系结构是第3级设计，后来的POWER2体系结构是第4级设计，因为它包含一个混合的整数部件并允许相关整数指令的发射。POWER2不是第5级设计，因为它在浮点指令重叠执行的时候不提供精确异常。

POWER的方法是谋求每个时钟内实现平衡的工作性能，而不是像Alpha那样追求尽可能快的时钟频率。IBM也强调上市时间，对于很多的部件采用了标准单元的设计并且电路设计不够优化（尤其是POWER2的设计）。因此，1996年POWER和POWER2实现的最快时钟频率分别是62.5 MHz和71.5 MHz。当时DEC的Alpha 21064A的速度是300 MHz，21164A的速度是500 MHz。Smith和Weiss［1994］提供了对这两种设计哲学的一个有趣的比较。

POWER3和POWER4是从PowerPC 620衍生出来的（参见后面的介绍）。

7.2.4　体系结构修订

很多指令级体系结构已经从32位的基础上迁移到了64位，通过将指令集流水化使得设计可以更加容易地利用超标量技术来实现。

Michael Mahon和Jerry Hauck将精确体系结构（Precision Architecture，1.0版本的设计者是Bill Worley和Michael Mahon）扩展为64位的工作方式。PA-RISC 2.0也包括多媒体扩展（MAX）［Lee and Huck,1996］。对于超标量实现，一个重要的改变是定义了8个浮点状态位，而不是像原来那样只有一个。PA-RISC 2.0加入了基于推断的cache行预取指令，以防止在TLB失效时引起失效操作；在PSW中加入了一位弱排序存储器模型模式位；加入了过程调用和返回时候的提示信息，以维护返回地址预测栈；还加入了一条FMA指令。PA-RISC 2.0进一步地定义了load/store的cache提示位（例如，进行有空间局部性的访问，HP PA 7200中就是这样做的）。PA-RISC 2.0使用了一种独特的静态分支预测的方法：如果在比较指令和分支指令中使用的寄存器是升序，那么分支预测为某一个方向，如果使用的寄存器是降序，那么分支预测为另外一个方向。设计者之所以使用这种方式来传递静态预测的信息，是因为已经没有剩余的操作码可用了。PA 8000是第6级设计并且是PA- RISC 2.0体系结构的一个实现。

Apple，IBM和Motorola在20世纪90年代联合定义了PowerPC指令集。主要的体系结构设计者包括Rich Oehler，Keith Diefendorff，Ron Hochsprung以及John Sell。PowerPC是POWER体系结构的一个修订版，并省略了乘商寄存器（MQ）和几条指令［Diefendorff等人，1994］。第一个PowerPC的实现是PowerPC 601（第3级设计）。PowerPC 603使用了一个5项的再定序缓冲来提供精确异常，因而划分为第5级设计。604和620（64位的版本）也是第5级设计，因为它们不提供完全的乱序执行。

SPARC指令集（最初由Robert Garner定义）也通过放松对多发射的约束进行了修改。这一修改是由一个庞大的委员会通过100多次会议才决定的。主要的负责人是Dave Ditzel（主席），Joel Boney, Steve Chessin, Bill Joy, Steve Kleiman, Steve Kruger, Dave Weaver, Winfried Wilcke以及Robert Yung。第9版体系结构的目标是转移到64位体系结构并避免串行瓶颈。这样，现在SPARC包括4个独立的浮点条件码和一种新类型的整数分支指令，根据整数寄存器的内容进行条件转移，在效果上相当于多个整数条件码。第9版还增加了对无故障推测载入的支持，在分支指令编码格式中增加了分支预测位，还增加了条件传送和支持弱排序存储器模型的存储器栅栏指令（memory-barrier）。UltraSPARC的所有流水线都具有相同的长度，从而保证大多数的指令是按序完成的，并对运行时间长的浮点指令可以预测异常。所以它是第9版体系结构设计的第3级实现。HAL SPARC64是个积极的、乱序执行的处理器，属于第6级设计。

7.3　处理器介绍

这一节将对几种现代的超标量微处理器进行简要的介绍。处理器将根据制造者或者体系结构系列（如AMD、Cyrix和Intel IA32处理器将一起进行描述）的字母顺序排序。这一介绍并不打算

包罗万象，主要是进行一个简要的概述并强调有趣的、不寻常的设计选择。每种设计的更多信息可以从各种参考资料中得到。

7.3.1 Compaq/DEC Alpha

7.3.1.1 概述

DEC的Alpha设计为64位的体系结构，以替代32位的VAX体系结构。正如在7.2.2小节讨论的一样，该设计对于多指令发射、多处理器以及操作系统无关性倾注了特殊的关注。与大部分超标量设计相反的是，Alpha的体系结构专家（Sites和Witek）选择了允许非精确的算术异常，而且不提供用来切换到精确异常的模式位。他们定义了一条自陷栅栏指令（TRAPB以及几乎相同的EXCB）用来串行化所有的实现，这样挂起的异常将被强迫执行。因此就可以通过在每条浮点指令后插入一条TRAPB指令来实现精确中断。一种更为有效的方法是，保证编译器的寄存器分配不允许指令写同一个基本块或者更小的区域中（例如：对应于单条高级语言语句的代码块）的源寄存器。这一约束使得可以通过每个基本块或者更小的区域加入一条TRAPB指令来提供精确异常，异常处理程序将完全可以确定所有目标寄存器的正确值。

Alpha为底层软件提供了一种特殊模式，称为特权体系结构库（PAL，Privileged Architecture Library）。这些例程具有强制的入口点并且执行的时候不允许中断。尽管它们是由一般的Alpha指令所组成的，但也可以访问隐藏实现的寄存器和指令。PAL例程随操作系统的不同而不同，当启动新的操作系统时可以重新载入。VMS操作系统使用的很多复杂的VAX指令（例如：互锁队列操作）都是使用PAL例程来实现的。

7.3.1.2 Alpha 21064（EV4）/1992

21064（参见图7.5）是Alpha体系结构的第一个实现，并由Alpha体系结构设计师Rich Witek负责。指令取指/发射部件在每个时钟周期内读取两条指令，这两条指令是双字对齐的。指令可以根据一些复杂的规则一起发射，这些规则是在设计中分配寄存器文件端口和指令发射路径的直接结果。译码器以消极的方式工作，也就是说，如果指令对中只有第一条指令可以发射，那么就不继续取指或者检查新的指令，直到第二条指令也被发射并离开译码器。然而，它使用了一个整段来将指令交换到合适的发射槽中，从而在指令发射规则中没有排序的约束（Intel i960）。这种指令发射的简单方法是为了追求高时钟频率所做的众多的折中之一。通常，可以将Alpha强调时钟频率的设计思路与IBM POWER体系结构、PowerPC体系结构中的指令打包技术进行对比，参见Smith和Weiss［1994］。也可以参考7.2.2小节中对Alpha超流水技术的评论。

21064积极地预取指令。只要发生指令cache失效，就会检查预取缓冲。如果需要的cache块在预取缓冲中，它将被送入cache并且下一个cache块的数据被读入缓冲。如果需要的cache块不在预取缓冲中，可以从L2 cache中或者存储器中得到相应的cache块，然后将下一个块中的数据取入到预取缓冲中。在8 K直接映像的指令cache中，每条指令还包括1位的分支预测符（在21064A中是2位）。可以通过设置某个控制寄存器来选择基于地址偏移符号的静态预测方法。21064中还包括一个4项的子程序地址预测栈。但是对于跳转指令必须设置提示位，从而指出是否需要压栈、弹栈或者忽略栈的存在。

21064包含三个功能部件：整数部件、load/store部件和浮点部件。对于执行时间长的指令（如移位指令），整数流水线是两级流水的。然而，加法和减法可以完全在第一段中执行完毕。load/store部件与8 KB的数据cache连接并且包含一个4项的写缓冲。虽然cache是写直达的，但写缓冲的每一项的大小和cache块的大小相等，其目的在于实现写合并。由于使用了load旁路，最多可以同时有3个未完成的数据cache失效。

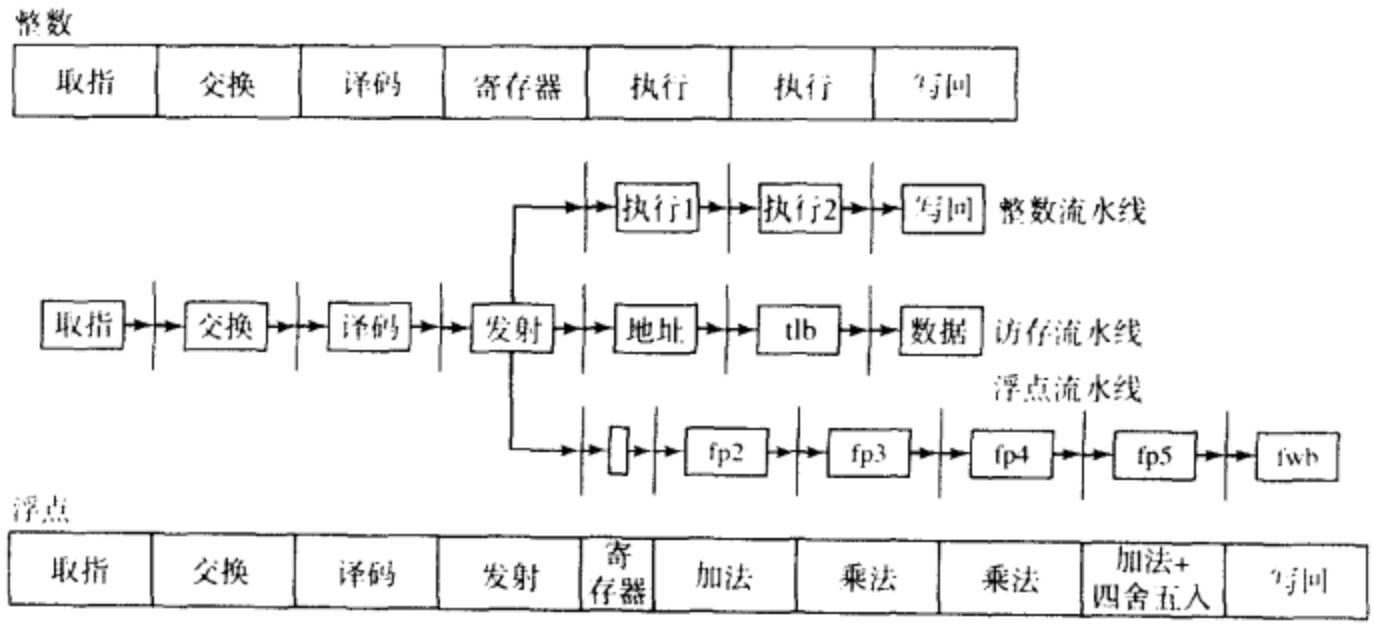


图 7.5　Alpha 21064 流水线

21064处理器在深流水线的设计上是独一无二的，它采用了特殊的设计来防止因为流水线的段数过多而影响数据的流动。事实上，21064上包含45条不同的旁路。后继的芯片（如21064A），不仅包括2位的预测位，还将片上cache的大小增加为16 KB。在21064A上，除法的延迟也减少了。

7.3.1.3　Alpha 21164（EV5）/1995

Alpha 21164是Alpha体系结构的第二个积极的实现。John Edmondson是设计阶段的负责人，Jim Keller是主管高级开发的负责人。Pete Bannon也做出了贡献，他领导了下一个芯片21164PC的设计。21164在片上集成了4个功能部件，3个独立的cache以及一个L3 cache控制器。这些功能部件包括整数部件0，执行整数的移位和读写；整数部件1，执行整数的乘法、整数分支以及整数load（不包括store）；浮点部件0，执行浮点加、减、比较和分支，启动并控制一个浮点除法器；浮点部件1，执行浮点乘法。

设计者们加快了21164的时钟速度，同时减少了移位、整数乘法、浮点乘法和除法的周期数。然而，简单快速但是并不积极的译码器被保留了下来，多发射只能从4字对齐的4指令组里发出，只有在当前的指令组的指令全部发射之后，译码器才能进一步发射其他指令。

4发射周期的正确指令组合是两条不相关的整数指令，一条浮点乘法指令以及一条不相关的非乘法浮点指令。指令组内的4条指令并没有顺序限制，因为流水线引入了一个slotting段，所以可以将指令从两个4指令大小的取指缓冲中路由到译码器。两条整数指令可以都是load指令，每条load指令的目的寄存器既可以是整数寄存器也可以是浮点寄存器。为了使编译器在分支目标地址对齐、生成正确的指令组合上具有更大的灵活性，因此引入了三种nop（空操作）指令：整数部件的nop，浮点部件的nop和消失nop（vanishing nop）。在同时发射比较指令、逻辑指令和与它们相关的条件分支指令时，将遵循特殊的规定。还提供了对分支指令位于4指令组中间时的支持，方法是给译码器中的每条指令增加一个有效位。

异常处理方式与21064相同。当遇到自陷（trap）或者异常指令时，则停止译码器的发射。通过提供8个影子寄存器（shadow register）映射到PAL模式下的整数寄存器组，同时还提供了对PAL入口的支持。这也为PAL代码提供了新的寄存器，使它不会因为寄存器保存/恢复而产生存储器访问。

21164将分支预测位从指令cache中去掉，同时将2048个分支预测位打包放进一个单独的分支历史表。返回地址栈增加到12项，正确预测的成功分支将有1个周期的气泡，但是这个气泡经常

由于前导指令的停顿而消除。21164的流水线控制方法比较有趣，控制逻辑在流水线的前几段检查并处理流水线停顿，但是后来形成的风险（如cache失效和写缓冲溢出等）将在执行段进行处理，这样前面的指令就要重新执行［DEC称为重放（replayed）］，而不是在后面的流水线段中产生一个停顿。这种重新执行的方法据说消除了设计中的几个关键路径，并且对读失效的处理进行了特别的设计，从而使得重新执行没有额外的性能损失。

片上的cache被分为一个L1指令cache（8 KB），一个双端口的L1数据cache（8 KB）和一个统一的L2 cache（96 KB）。L1 cache向流水线提供必要的带宽，但是L1数据cache的容量受限于实现双端口存储器中每一位的晶体管数目。片上的L2 cache弥补了L1的不足。L1的读操作延迟为两个周期，每个周期能够同时支持两次读和一次写；L2的访问周期为8个周期。L1和L2 cache之间有一个失效地址文件（MAF, Miss Address File），一共有6项，用于支持无阻塞的访问。MAF能够合并对同一L2 cache行的不连续的读操作，工作方式和合并对同一cache行时写操作的写缓冲差不多，每个失效地址最多能够记录4个目标寄存器。L2 cache与片外存储器之间也有大小为两项的总线地址文件（BAF, Bus Address File），用于整行填充L2 cache时的无阻塞访问。

21164能够预取当前程序计数器的3个32字节的指令块。预取从一次指令cache失效开始，连续、顺序地进行，直到发生分支或者指令cache发生一次命中。这些预取的块直到需要时才复制到指令cache；这防止了预取的指令错误地替换指令cache中有用的指令。

7.3.1.4 Alpha 21264（EV6）/1997

21264是Alpha体系结构中第一个实现乱序执行的版本，但是流水线的顺序执行部分仍然保持了处理对齐指令组时的效率。该设计将指令对齐时（类似于VLIW的限制）所产生的效率和超标量乱序执行的灵活性结合起来。Jim Keller是21264的主要设计师。

指令cache是一个64 KB两路伪相联的虚地址cache并包括一个行预测器。它与Johnson提出的机制比较类似，用于快速取出对齐的指令组。它使用了一个混合的分支预测器，包括一个基于两位历史计数器的简单预测器，以及一个基于全局分支历史移位寄存器的两级自适应预测器。分支预测逻辑动态决定使用哪一个预测器能够得到最准确的预测。这种方法弥补了自适应预测器训练时间的不足，即对于开始的几百个或几千个分支，自适应预测器不如简单的两位计数器准确。一旦经过充分的训练，就能使用自适应预测器。然而，每一次上下文切换时，历史信息将丢失，因此训练必须重新开始。

取指之后，通过两个12端口的寄存器映射器将4条指令重命名。指令被分派到20项的整数指令队列或者15项的浮点指令队列。总共有80个物理整数寄存器（其中32个体系机构寄存器，8个PAL影子寄存器，40个重命名寄存器），72个物理浮点寄存器（其中32个体系结构寄存器，40个重命名寄存器）。指令组保留在包含20项的再定序缓冲（活动列表）中，最多可以记录多达80条指令以及指令的重命名状态。预测失误的分支需要一个周期恢复到适当的指令组。指令可以在每个周期提交两个4指令组。

整数指令队列每个周期能够发射4条指令，4个整数部件每个对应一条指令。每个整数部件可以执行加法、减法和逻辑指令。另外，一个部件可以执行分支、移位和多媒体指令；一个部件可以执行分支、移位和乘法指令；剩下的两个部件可以执行访存指令。整数寄存器文件实现为两个相同的副本，以提供足够的寄存器端口（比较IBM POWER2）。两个副本之间的一致性是通过向其中一个副本写时相应地更新另一个副本加以实现的，这两个操作之间存在一个周期的延迟。Gieseke等人［1997］估计，分离的整数寄存器文件的性能损失为1%，而统一的整数寄存器文件需要增加22%

的面积、增加47%的数据通路带宽以及增加75%的操作数总线长度。统一的寄存器文件将会限制时钟周期。

浮点（FP）指令队列每个周期最多能发射两条指令，两个浮点功能部件各一条指令。一个浮点部件能够执行加法、减法、除法以及平方根的计算，而另一个浮点部件专门用于乘法。浮点加法、减法和乘法都是流水化的，延迟为4个周期。

当指令以及前面的所有指令都经过可能发生异常和预测失误的地方时，指令就可以提交。这使得指令甚至可以在执行结果被计算出来之前提交。

21264的数据cache为64 KB，两路伪相联。数据cache时钟频率是处理器时钟频率的两倍，因此每个处理器时钟周期能执行两次读操作，一次写操作和一次提取牺牲数据块的操作（victim extract）。同时，还提供了一个32项的load再定序缓冲和32项的store再定序缓冲。

若需要了解21264更多的信息，参见Leibholz和Razdan［1997］，Kessler等人［1998］以及Kessler［1999］。

7.3.1.5　Alpha 21364（EV7）/2001

Alpha 21364使用了一个21364（EV68）的核，同时增加了一个片上的L2 cache，两个存储器控制器以及网络接口。L2 cache采用7路组相联，总共1.75 MB。cache系统还包括16个牺牲缓冲用于L1的替换，16个牺牲缓冲用于L2替换，还有16个L1失效缓冲。存储器控制器支持基于目录的cache一致性协议并提供RAMbus接口。网络接口支持乱序传输并且支持4条链路的自适应路由，每条链路提供6.4 GB/s的带宽。

7.3.1.6　Alpha 21464（EV8）

Alpha 21464是一个积极的8指令宽度的超标量设计，包括4路同时多线程。其设计目标是提高单线程的吞吐率，虽然增加了SMT控制逻辑和重复的资源，但芯片面积的耗费是很小的。在每个时钟周期内，处理器取出两个指令块，每个指令块包含8条指令，每个周期最多处理16个分支预测。预测之后，读取的指令按照每组8条指令分组进入缓冲。每一组发射到16个功能部件：8个整数ALU，4个浮点ALU，两个读操作流水线以及两个写操作流水线。芯片设计包括一个系统间互连路由器，以建立一个满足基于目录的cache一致性协议的NUMA系统，最多可以支持512个处理器。

Alpha处理器的发展（包括21464）终止于2001年6月。从此以后，Compaq转移到Intel Itanium处理器。Seznec等人［2002］描述了21464的分支预测器。Joel Emer在PACT 2001的一次谈话中给出了21464设计的概要情况。

7.3.2　HP的PA-RISC 1.0版本

7.3.2.1　概要

HP的精确体系结构（PA, Precision Architecture）是最早的RISC体系结构之一，它由Bill Worley和Michael Mahon在1981年到1983年间设计并于MIPS和SPARC之前发布。它是一种load/store体系结构，具有许多RISC的特征，但是它的ISA也具备一些VLIW的特征。在某些情况下，一条指令能够指定多个操作。因此，虽然32位PA系列（PA-RISC 1.0版本）的超标量处理器在指令发射带宽上并不出色，但是所有的32位PA处理器都能够在每个时钟周期内执行多个操作。实际上，第一个双发射的PA处理器（PA7100）每个周期能够发射多达4个操作：1个整数ALU操作，1个对ALU结果的条件测试操作，用来确定下一条指令是否无效（谓词执行），1个浮点加法以及1个不相关的浮点乘法。即使前面有一条浮点除法还在执行或者一次cache失效还在处理，这4个操作仍然能够发射。

7300LC之前的32位PA处理器的特征是，每个芯片的晶体管数目相对较少，并将重点放在较大的片外cache的利用上。一个简单的5级流水线则成为每种设计的起点，只要外部cache的SRAM速度允许，就应该使流水线尽可能地快。虽然7100LC和7200（以及以大容量片上cache为特征的7300LC）采用了较小的、专用的片上cache，但是新的64位PA 8000采用了非常先进的乱序执行设计，并且依赖片外cache。特别值得注意的是，HP系统设计者针对某些商业应用（如事务处理）进行了重点考虑，这些应用具有工作集合大的特征，而对于较小的片上cache则局部性很差。

7.3.2.2 PA 7100/1992

7100（参见图7.6）是PA体系结构的第一个超标量实现。它是一个双发射设计，包含一个整数部件和三个独立的浮点部件，每个周期能够发射一条整数部件指令和一条浮点部件指令。整数部件处理整数和浮点的读写，而整数乘法由浮点部件完成。cache填充时对指令进行预译码，以便指令取出后立刻就能够发射到相应的部件。7100的双发射没有排序或者对齐的要求。

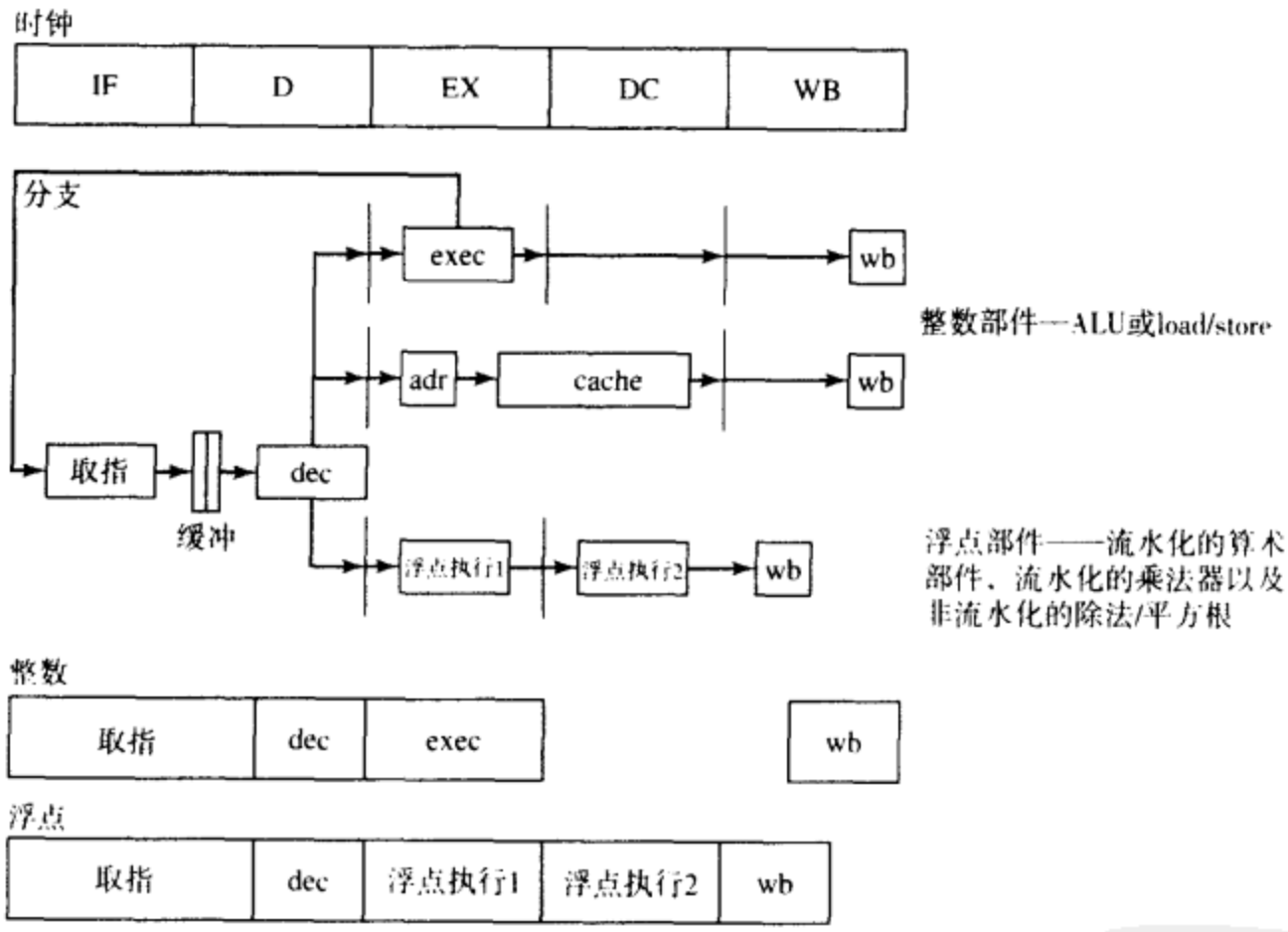


图7.6 AP PA 7100流水线

前面已经提到，对于多操作指令，整数部件每个周期能够执行两个操作，如加法和分支、或者执行并测试结果以确定下一条指令的有效性。浮点部件每个周期也能启动两个独立的操作：加法和乘法或者减法和乘法。不管是单精度还是双精度，浮点加法和乘法的延迟均为两个周期。

7100上的分支预测是静态预测：向前的分支预测为不跳转，向后的分支预测为跳转。对于一个双发射指令对，通过将整数部件的写回操作推迟到浮点部件并成功写回，来保证双发射指令对的精确中断。

7100有一个片上cache控制器，但是cache存储器本身由外部SRAM芯片实现。指令cache是直接映像的，大小从4 KB到1 MB不等。数据cache也是直接映像，大小从4 KB到2 MB不等。每种cache到处理器的数据通路都为64位，因此每个周期能够取两条指令，或者同时执行一个双精度的浮点载入或存储。cache使用虚地址索引和物理地址标识，并且地址首先经过杂凑（hash）映射

以减少冲突次数。除了片上 TLB，还有一个二级 TLB 并由软件在存储器中维护，但当第一级 TLB 失效时，硬件能够直接访问第二级 TLB。二级 TLB 失效将导致一组寄存器切换为 TLB 失效处理时的 8 个影子寄存器，从而有助于将 TLB 失效处理的时间控制在 10 个周期以内。

7100 每周期能够发射一次读操作，如果 cache 命中，读出数据需要两个周期；而写操作命中时需要 3 个周期。若经过特殊匹配，则允许一次相关的浮点写操作与产生该结果的浮点操作在同一周期发射。处理器支持一次 cache 失效下的命中，指令能够继续执行（包括命中的读写操作），直到失效的结果返回或者发生另一次失效。对 R0 的载入也将提供数据预取。

7100 有一个标准的 4 状态 cache 一致性协议，但是还有一个特殊的双处理器耦合方案，当一个处理器拥有所需要的数据时，能够直接进行行到行的数据传输，而不需要通过存储器总线。关于 PA 7100 的更多信息，请参见 Asprey 等人［1993］和 DeLano 等人［1992］的阐述。7150 是 7100 的 125 MHz 实现。

7.3.2.3　PA 7100LC/7300LC/1994/1996

PA 7100LC 是 7100 的扩展，但是成本和功耗更低，主要面向图形和多媒体工作站应用。7100LC 只能作为单处理器使用，降低了双精度浮点操作的性能。同时，独立的片外 cache 提供了第二个整数部件、大小为 1 KB 的片上指令 cache、集成存储器控制器以及支持多媒体的新指令。外部 cache 是指令数据统一的 cache，采用虚地址索引和物理地址标识，大小从 8 KB 到 2 MB。

整数部件是不对称的，只有一个整数部件包含移位和位操作电路。除了每个周期只能有一条移位或位操作指令以外，同一周期可以发射两条整数指令，或者一条整数指令和一次访存，或者一条整数指令和一条浮点指令，或者一次访存和一条浮点指令。同时规定，对于存储器中 64 位对齐的双字，其中两个字的两次读或两次写可以在同一周期中发射。这对于子程序的进入和退出是很有好处的。

分支指令和其他可以改变指令顺序的整数指令只能和它们的上一条指令一起发射，而不能和下一条指令同时发射（如一条延迟的分支指令不能和它的分支延迟槽中的指令一起发射）。跨越 cache 行边界的两条指令能够同时发射，除非这两条指令分别是一条整数指令和一条访存指令。7100LC 上的寄存器记分牌允许“写后写”相关的指令在同一个周期里发射。然而，为了简化控制逻辑，当遇到延迟大于两个周期的操作时，整条流水线将停顿，这些操作包括整数乘法、双精度浮点操作和浮点除法。

集成的 cache 控制器和存储器控制器能够持续地预取指令。存储器控制器改善了对外部 cache 失效的处理，从 TLB 对读写地址的转换中获得页帧号并假设失效，从而推测性地驱动存储器地址线并节省一个周期的失效开销。

关于 PA 7100LC 的更多信息，请参见 Knebel 等人［1993］，Undy 等人［1994］的阐述，以及 1995 年 4 月 Hewlett-Packard 期刊的专题。7300LC 是 7100LC 的一个派生产品，包含大约 900 百万个晶体管，其中大多数用于构造两个 64 KB 的片上 cache［Hollenbeck 等人，1996；Blanchard 和 Tobin，1997；Johnson 和 Undy，1997］。

7.3.2.4　PA 7200/1994

PA 7200 保留了 7100 的指令、数据 cache 控制器、TLB 结构以及三个高性能的浮点部件。7200 同样增加了第二个整数部件和一个 2 KB 的片上辅助数据 cache。指令发射逻辑与 7100LC 比较类似，但是流水线在遇到多周期操作时不会停顿。

7200 的指令 cache 能提供多个顺序的指令预取，指令 cache 容量最大为 1 MB。7200 同样积极地预取数据，允许最多 4 次独立的数据预取；数据 cache 的容量最大也是 1 MB。数据预取根据 load/

store操作的地址寄存器更新的方向和步幅自动进行。两个cache都是直接映像的，使用虚地址索引并以物理地址标识。

7200的译码机制与国家半导体公司（National Semiconductor）的Swordfish类似。指令cache为每个双字增加了6位预译码位，这些预译码位部分用于指示两条指令之间的数据相关，部分用于将指令送往正确的功能部件。预译码位在cache填充时设置。

7200最有趣的设计是使用了一个片上全相联的辅助cache。辅助cache包含64项，每一项为32字节的cache块。所有的数据cache失效和预取都被引向这个辅助cache，辅助cache使用FIFO替换算法与外部数据cache通信。新的load/store操作还提供空间局部性提示（比如块复制操作）。辅助cache中带标记块的替换就可以旁路外部数据cache。也就是说，从辅助cache中清空这些块时，如果这些块是修改过的，就直接写回存储器而不写回外部数据cache，这避免了直接映像外部数据cache时一些不必要的抖动。

关于PA 7200更详细的信息，请参见Kurpanek等人［1994］以及Chan等人［1996］的阐述。

7.3.3 HP的PA RISC 2.0版本

7.3.3.1 HP PA 8000/1996

PA 8000是64位PA-RISC 2.0体系结构（2.0的设计师是Michael Mahon和Jerry Hauck）的第一个实现。PA 8000使用片外cache，这也是HP的传统，因此可以将芯片上的晶体管用于实现两个28项的保留站（结合再定序缓冲）以及8个功能部件。这些功能部件包含两个整数ALU、1个移位/合并部件、1个除法/平方根部件、两个乘法/累加部件以及两个load/store部件。ALU指令被分派到ALU缓冲中，访存指令被分派到访存缓冲（同样是一个28项的地址缓冲）中。一些指令，如载入并修改指令和分支指令，都将分派到ALU缓冲和访存缓冲中去。

组合的保留站/再定序缓冲按照分而治之的方式工作（Gaddis和Lotz，1996）。缓冲中处于偶数槽中的指令发射到一个整数ALU部件或者一个访存部件，处于奇数槽中的指令发射到另一个整数ALU或者访存部件。此外，缓冲的偶数部分和奇数部分分别被细分成4个体（大小分别为4，4，4，2），这样两个28项的缓冲就分成4个组，每个组有4个体。每个组中，包含最靠前指令的体中，第一条就绪指令就是最先获得发射权的指令。这样，每个组每个周期能发射一条指令，最大的发射速率为每个周期两条ALU指令和两条访存指令。当指令被分派到缓冲时，使用了大量的比较器来检查寄存器相关，同样数目的比较器用于匹配等待的指令和寄存器更新。保留站/再定序缓冲使用了特殊的传播逻辑来处理进位和借位相关。

由于使用了片外指令cache，发生跳转的分支包含两个周期的开销。因此使用了一个32项、全相联的分支目标地址cache（BTAC）和一个256项的分支历史表（BHT）。BHT的每一项有一个3位的分支历史寄存器，可以使用该寄存器的值来进行预测。如果BTAC命中，那么预测正确的跳转分支就没有延迟。此外，还可以使用指令格式中的寄存器号码排序机制（参见7.2.4小节）进行静态的分支预测。静态或者动态的预测可以针对不同的页加以选择。HP的建议是，规划动态库的代码并正确地设置静态预测位，同时，对于库代码的页面选择静态预测。在进行库调用时可以在BHT中保存程序的动态历史。

PA 8000可以在发生数据cache失效时继续执行，最多可以发生10次失效。关于PA 8000的深入描述，请参见Hunt［1995］以及Gaddis和Lotz［1996］的著作。

7.3.3.2 PA 8200/1997

PA 8200使用了与PA 8000相同的内核并运行于240 MHz的主频下。8200采取了一些改进措施，例如，将BHT的项扩大到4倍（达到了1024项），还允许一个周期内同时对多个BHT项更新。

TLB 项也从 96 个增加到 120 个。关于 PA 8000 和 PA 8200 的更多信息，请参见 Hewlett-Packard 期刊［1997］的专题。

7.3.3.3　PA 8500/1998

440 MHz 的 PA 8500 是 PA 8000 核心的浓缩版，集成了 0.5 MB 的指令 cache 和 1 MB 的数据 cache。8500 对分支预测方法进行了巨大的改进，在每个 BHT 项中加入了一个两位的计数器。当分支符合静态预测的时候就减少计数器的值，当预测失误时就增加计数器的值。因此，如果计数器的值为 0 和 1，就选择静态预测方向的分支；如果计数器的值为 2 和 3，就选择与静态预测相反方向的分支。BHT 项的数目也增加到 2048 个。Lesartre 和 Hunt［1997］对 PA 8500 做了简要的描述。

7.3.3.4　其他 PA 8x00 处理器

PA 8600 发布于 2000 年，其主频为 550 MHz。8600 进行了容错设计并提供了两个芯片之间的同步操作。PA 8700 发布于 2001 年，其主频为 800 MHz 并增加了 cache 的容量：1.5 MB 的数据 cache 和 0.75 MB 的指令 cache。关于 PA 8700 的详细情况，请参见 HP 的技术报告［2000］。

HP 计划在将它们的产品线切换到 Intel Itanium 系列处理器之前发布了 PA 8800 和 PA 8900。PA 8800 包含两个 PA 8700 的核心，每个都有 0.75 MB 的数据 cache 和 0.75 MB 的指令 cache，以及片上 L2 cache 标识、32 MB 的片外 DRAM L2 cache。

7.3.4　Intel i960

7.3.4.1　概述

i960 体系结构由 Intel 于 1988 年公布，它是一个面向嵌入式系统的 RISC 设计。i960 基本结构只支持整数并包含了 32 个寄存器：分成 16 个全局寄存器和 16 个本地寄存器。i960 基本结构的一个数值扩展版提供了单精度、双精度和扩展精度的浮点运算，编程模型中还加入了 4 个 80 位的寄存器。i960 的负责人是 Glen Myers。

i960 的比较操作进行了细致的设计和流水化实现：

1. 在标准的比较指令之后可以使用一条条件比较指令。如果第一个比较为真，条件比较就不会执行。这时只需要一个条件分支就可以实现范围检查。
2. 提供带增减操作的比较指令，支持快速循环操作。
3. 提供了比较和分支结合的指令，当编译器不能将一条不相关的指令插入到比较指令和条件分支指令之间时，该结合指令可以用于处理延迟槽的情况。

操作码的名字空间同样进行了细致的分配，使得根据指令的前三位能很容易地区分控制指令（C 类型）、寄存器到寄存器的整数指令（R 类型）以及访存指令（M 类型）。这样到不同功能部件的分派就能很快地进行，也不需要预译码位。

7.3.4.2　i960 CA/1989

i960 CA（如图 7.7 所示）发布于 1989 年，它是第一个超标量微处理器，并且是早期超标量设计中现在惟一还能作为产品使用的芯片，主要设计师是 Glenn Hinton 和 Frank Smith。CA 芯片上的模块包括：一个中断控制器，一个 DMA 控制器，一个总线控制器以及 1.5 KB 的存储器，这个存储器可以将一部分分派给寄存器窗口栈，剩下的部分用做低延迟的存储器。主要包含三个功能部件：取指/分支部件（指令排序部件）、整数部件（面向寄存器）和地址产生部件（面向存储器）。整数部件包括一个单周期的整数 ALU 部件和一个流水化的乘法/除法器。地址产生部件控制到存储器的访问，同时也处理到片上存储器的访问。

译码器每两个周期取出4条指令，根据以下规则进行指令发射：

1. 如果可能，将第一条指令发射。
2. 如果第一条指令是面向寄存器的指令，即既不是面向存储器的指令，也不是控制指令，那么就要检查第二条指令。如果是面向存储器的指令，那么（如果可能）就将它发射。
3. 如果已经发射了一条到两条指令，而且没有一条是控制指令，那么就要检查所有剩下的指令。将找到的第一条控制指令发射。

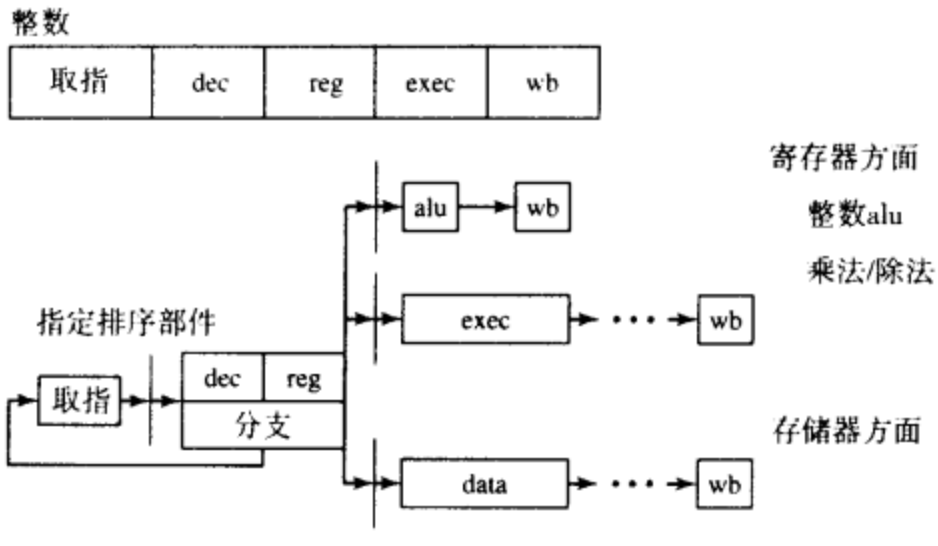


图7.7 Intel i960 CA 流水线

当取出新的4个指令字以后，将包含9种多发射的可能性：

3发射	2发射，非R类型	2发射，非M类型	2发射，非C类型
RMCx	MCxx	RCxx	RMxx
RMxC	MxCx	RxCx	
	MxxC	RxxC	

其中，x表示没有发射的指令。注意，在下面的5种可能中，先执行控制指令，也就是按照乱序的方式执行。然而，指令排序部件要保留当前的4个指令字，直到所有的指令都发射出去。这样，当峰值发射速度能达到每个周期3条指令，最大的持续发射速度则为每个周期两条指令。上述发射规则对指令顺序的限制受到大多数设计者的批评，他们认为这种发射顺序不规则。然而，M类型的有效地址载入指令（lda）是相当普遍的，在许多情况下，一对整数指令可以被指令调度器或者优化器等价地转化为整数指令和lda指令对。参见下面的MM模型，它提供了基于硬件的指令转化。

i960 CA有一个1 KB的两路组相联的指令cache。组相联的设计允许时间敏感的程序同时读取一个或者两个体（通过特殊的指令）并且锁住指令cache，防止失效。而且，通过预测指令cache失效，启动一次推测访存并取出分支目标，从而减少了指令cache失效的开销。异常返回可以由软件处理或者使用异常栅栏指令。

7.3.4.3 i960的其他模型

i960 MM发布于1991年并应用于军事领域。片上集成了一个2 KB的指令cache和2 KB的数据cache[McGeady等人, 1991]。译码器自动将第二条整数指令重写为lda指令，通过预测指令cache失效，自动发出存储器预取并取出分支目标。MM模型还包括一个浮点部件并对i960体系结构的数值运算进行了扩展。CF模型发布于1992年，它集成了4 KB、两路组相联指令cache和1 KB直接映像的数据cache。

i960 Hx 系列模型提供了一个 16 KB 4 路组相联的指令 cache、8 KB 的 4 路组相联的数据 cache 以及 2 KB 的片上 RAM。

7.3.5　Intel IA32

7.3.5.1　概述

在电子计算机（不包括嵌入式微处理器和微控制器）50 年形成的体系结构中，Intel IA32 可能是应用最为广泛的一个。IA32 占领了并将继续占领个人计算机市场，并且已经成为构成大规模多处理机的基础。

1973 年，从 Stanley Mazor，Federico Faggin 和 Masatoshi Shima 等人设计的 8 位微处理器 8080 开始，Intel 发布了好几个扩展版本：16 位的 8086（Stephen Morse 和 Bruce Ravenel）、带保护模式的 286（Bob Childs）以及 32 位的 386（John Crawford 和 Patrick Gelsinger）。每一个后续的设计都以保持代码兼容性作为主要的设计限制。由于既要保持对以前代码的支持，同时又要对指令集进行新的扩展，这使得 IA32 最后的指令集受到强烈的批评。8 个寄存器组成的寄存器组和 CISC 指令集自从 1990 年发布 386 以来相当稳定。后面的 486（由 John Crawford 设计）集成了浮点部件 FPU，仍然使用了扩展的流水线来实现多条指令的单周期执行。

Intel 面临好几个 IA32 的竞争者，这种情况与 20 世纪 70 年代 IBM 在大型机市场面临多个插入兼容（plug-compatible）的竞争者很类似。一些竞争者仅满足于从低端市场分一杯羹，然而另一些竞争者则采用了积极的设计技术在高性能设计中进行竞争。后者的设计工作可以分为 IA32 指令流水线的设计和分离的微体系结构设计。Intel Pentium 属于前面一种类型，而 Intel P6 实现为分离的微体系结构。分离设计的工作最早开始于 1989 年，即 NexGen 公布了 F86（后来称为 Nx586 和 Nx686，形成了现在 AMD K6 的产品线）的时候。我们按照年代顺序，先介绍基于整个指令流水线的设计，然后再考察分离的微体系结构设计。

7.3.5.2　Intel Pentium/1993

Intel 选择了 Pentium 这个名字作为处理器的商标，而不是将它称为 586。实际上有一系列的 Pentium 的实现，它们有不同的尺寸、电源管理技术以及时钟频率。一个新的版本加入了 MMX 多媒体指令集的扩展［Peleg 和 Weiser，1996；Lempel 等人，1997］。Pentium 的负责人是 Don Alpert，协助人员包括 Jack Mills，Bob Dreyer，Ed Grochowski 以及 Uri Weiser。Weiser 在以色列进行了初始设计，这就是双发射 P5 处理器的原始框架，Mills 的研究则对最终形成两条整数流水线的管理方法做出了贡献。

Pentium（如图 7.8 所示）围绕两条整数流水线设计 U 流水线和 V 流水线，这两条流水线以锁步（lockstep）方式工作（一个例外的情况是，成对的指令可能在 V 流水线的执行段停顿，而不需要停顿 U 流水线中的指令）。其流水线中的段与 486 流水线的段很相似。第一个译码段确定指令的长度并检查能否双发射。第二个译码段计算存储器的有效地址，以便在执行时访问数据 cache，该段与 486 中对应段不一样的地方在于，Pentium 能在一个周期内同时读取一个索引寄存器和一个基址寄存器，或者同时处理一个偏移和一个立即数。执行段执行算术和逻辑操作，如果所有的操作数都在寄存器中，操作将在一个周期内完成。但是对于比较复杂的指令，则需要多个周期。例如，一个普通类型的复杂指令将一个寄存器操作数与存储器操作数相加，并需要 3 个周期：第一个周期从 cache 读取存储器操作数，第二个周期执行加法，最后一个周期将结果写回 cache。然而，如果有两条这样的指令（读 - 修改 - 写）组成的指令对，就要增加两个额外的停顿周期。一些整数指令，例如移位、旋转、带进位的加法，只能在 U 流水线中执行。整数乘法在同 U 流水线连接的浮点流水线中完成，它需要 10 个周期。

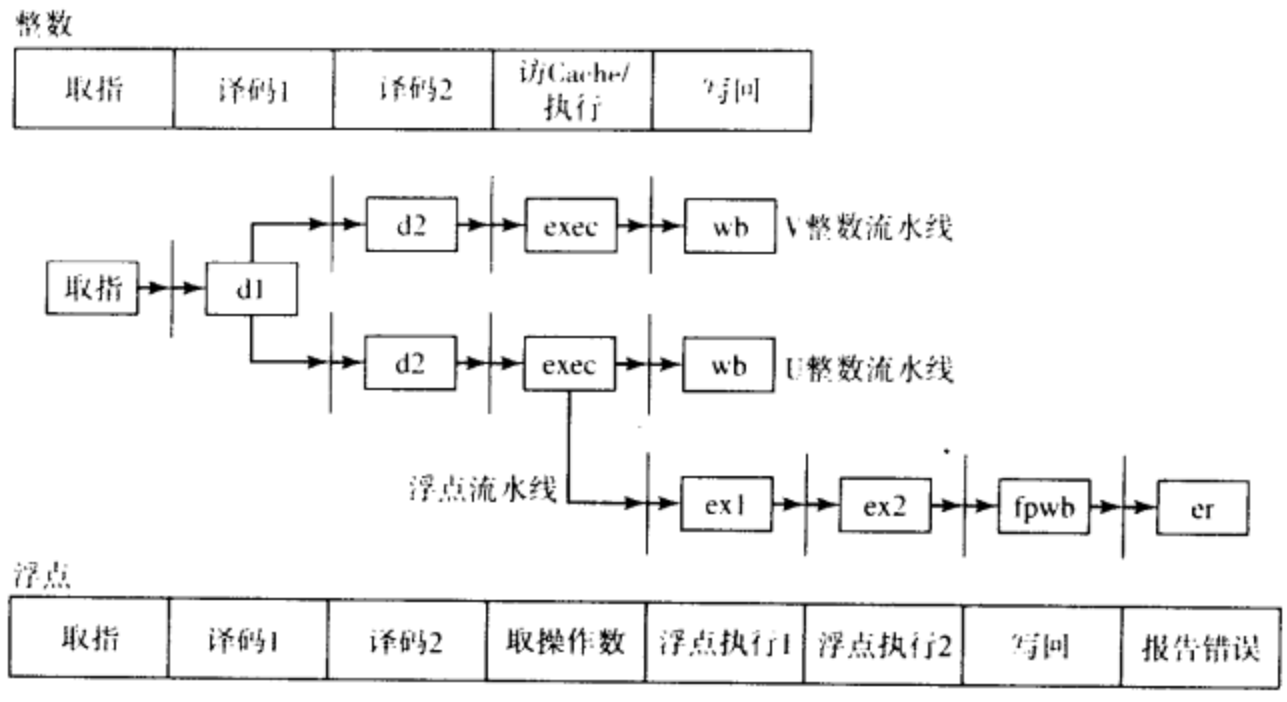


图 7.8 Intel Pentium 流水线

复杂指令（包括浮点指令以及双发射指令对的第一条指令）则是分支指令。在第二条指令与第一条指令存在相关性（不检查 WAR 相关，WAR 也不会限制双发射）的时候，U 流水线是单发射的。复杂的指令在 D1 段产生一系列的控制字并提供几个周期的流水线控制信号。一些指令在组成指令对时要进行特殊处理。例如置标志位指令与相关的分支指令组对时；两条push或者两条pop指令按照顺序（在子程序的进入和退出时）组对时；将一条浮点栈寄存器交换指令（FXCH）和一条浮点算术指令组对时，其相应过程均需要特殊对待。

Pentium 处理器的数据 cache 以 4 字节为边界，8 路交叉，TLB 及标识结构都是真正的双端口，因此 U 流水线和 V 流水线能够并行访问 32 位的双字而不会发生体冲突。cache 不按写分配，因此有时候要在一组顺序写之前插入伪读操作，作为编译器或者手工代码的优化。

浮点数据通路为 80 位宽，直接支持扩展精度运算。486 的延迟异常模型改为预测异常（称为安全指令识别）。浮点指令的单发射限制不像 RISC 体系结构中那么严格，因为 IA32 中的浮点指令可以包含存储器操作数。cache 设计也支持双精度的浮点运算，并使用 U 和 V 流水线分别访问一个 64 位双精度数据的高 32 位和低 32 位。

分支预测在取指段进行，它使用了一个 256 项的分支目标缓冲（BTB），每个项包含分支指令地址、分支目标地址和两位分支历史信息。Pentium（实际上大多数 IA32 实现都如此）没有单独处理无条件分支（jump）。如果一个无条件分支没有记录在 BTB 中，就预测为不跳转。

关于早期 Pentium 的更多信息，参见 Alpert 和 Avnon [1993]。Pentium MMX 的设计包括 57 条新的多媒体指令，它的 cache 更大，分支预测机制（起源于 P6）也做得更好。此外，指令长度译码同样在一个单独的流水线段中进行 [Eden and Kagan, 1997]。

7.3.5.3 Cyrix 6x86（M1）/1994

Cyrix 的工程师 Mark Bluhm 和 Ty Garibay 采取了好几种措施来提高（类似于 Pentium 双流水线设计的）性能，而不仅仅是遵循分离微体系结构的方法。这些措施包括使用寄存器重命名来解决相关（6x86 提供了 32 个物理寄存器），在两条流水线（在 6x86 中称为 X 和 Y）之间增加了定向路径，将两条流水线的执行分离开，而不仅仅是按照 Pentium 流水线的锁步方式前进；能够在译码段之后在两条流水线之间交换指令，以支持动态负载平衡和避免停顿；能够进行更大范围的指令双发射；使用一个 8 项的地址栈，支持返回地址的分支预测，以及能够对跨越最多 4 条分支指令和浮点指令

的任意组合进行推断执行。他们还做出了一些设计选择来更好地支持原来16位IA32的程序以及32位/16位的混合程序，如图 7.9 所示：

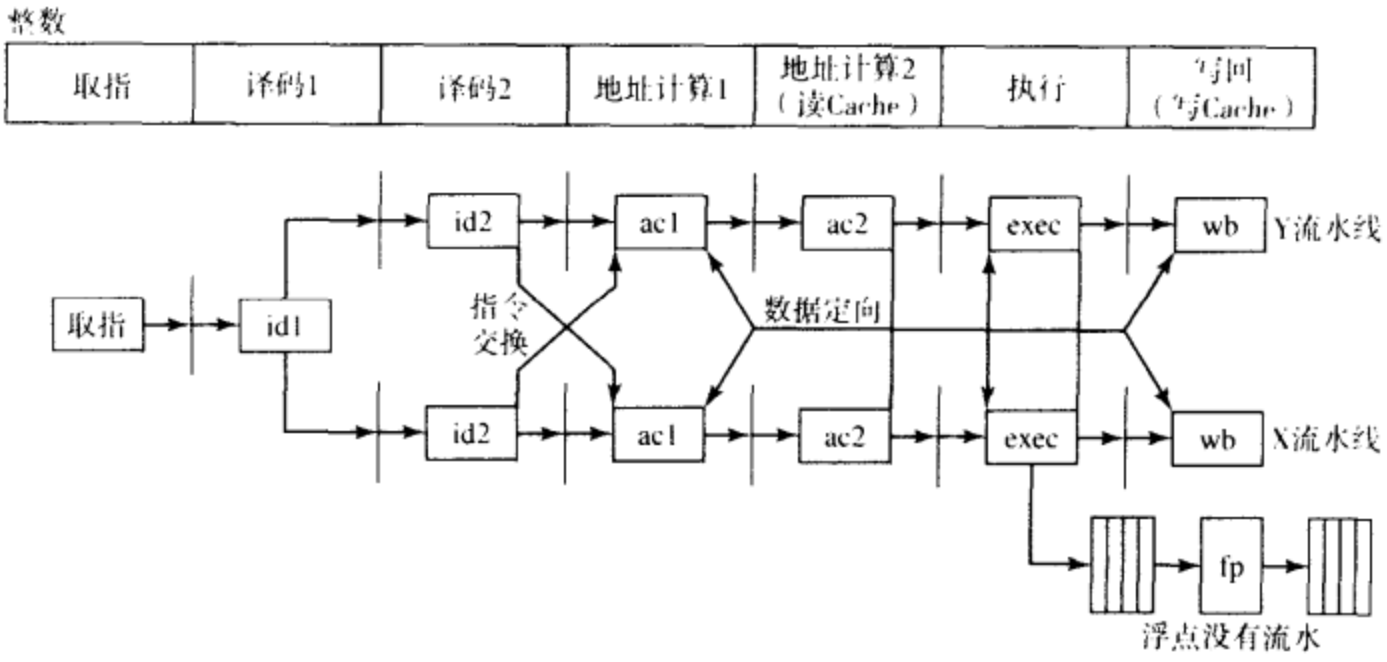


图 7.9　Cyrix 6x86 流水线

6x86 的 X 和 Y 流水线包含 7 段，而 Pentium 的 U 和 V 流水线则包含了 5 段。Pentium 的 D1 和 D2 段在 6x86 中都被分成两段：两个指令译码段和两个地址计算段。指令通过两次 16 字节对齐的取指操作获得，即使对于指令放置的最坏情况，每个周期也能够提供至少 9 个字节的指令。第一个指令译码段确定两条指令的边界，第二个译码段确定操作数。IA32 程序有时将具有 32 位操作数的指令和具有 16 位操作数的指令混合在一起，32 位操作数是默认的，而 16 位的操作数由指令前缀字节指明。6x86 的第二个译码段对处理前缀字节进行了优化，将指示操作数大小的前缀预测为前继指令的形式。

第一个地址计算段进行寄存器重命名，同时也进行地址计算。为了使地址的计算尽早进行，地址计算与寄存器重命名重叠在一起，因此，地址加法器必须从逻辑寄存器文件中获取数据，而执行段的 ALU 必须从重命名后的物理寄存器中获取数据。6x86 使用了一个寄存器记分牌记录任何对地址计算中要使用的逻辑寄存器的更新操作，并强制产生地址计算互锁（AGI, Address Generation Interlock）。而且，对于在地址计算中要改写的寄存器，如栈指针，它们的逻辑和物理两个副本之间的一致性由硬件进行特殊处理。有了硬件的支持，两次 push 或者两次 pop 可以同时执行。段寄存器不进行重命名，但在第一个地址计算段中要维护一个段寄存器记分牌并停止相关的指令，直到段寄存器的改写完成。

第二个地址计算段执行地址转换并访问 cache，从而获取存储器操作数（Pentium 将这一段包含在执行段中）。这一段还进行访存异常处理，因此一些不易预测异常的指令必须单个发射，并在流水线中串行执行。例如某些返回指令、从栈中弹出的目标地址可能导致段异常。对于结果要写回存储器的指令，则在写回段访问 cache（Pentium 在执行段写回结果）。

6x86 在 X 流水线处理分支并且和 Pentium 一样，为一条比较指令和一条相关的条件分支指令提供了特殊的指令配对方式。与 Pentium 不同的是，6x86 能够在 X 流水线发射一条预测为不跳转的分支指令，同时在 Y 流水线发射这条分支指令的后续指令。6x86 同样使用检查点恢复（checkpoint-repair）的方法，允许跨越预测分支和浮点指令并进行推测执行。6x86 提供了 4 级检查点存储。

访存指令通常由 Y 流水线执行，因此当发生 cache 失效时 X 流水线仍然能使用。6x86 对重复和字符串移动指令对提供了特殊的支持，这种指令对要用到两条流水线的资源，移动指令则能够达

到每个周期一个循环的速度。由于X流水线和Y流水线之间加入了Pentium所没有的定向路径，并且将cache的访问放在执行段之外，因此某些组合形式的相关指令对能够在6x86上实现双发射。这些组合形式包括：一条存储器到寄存器的移动指令和要使用该寄存器的算术指令；一条向寄存器写的算术指令和一条要使用该寄存器的、寄存器到存储器的移动指令。

浮点指令由X流水线处理，保存在一个4项的浮点指令队列。当确信浮点指令不会导致访存失效时，将生成一个检查点，指令继续发射并且浮点指令可以乱序执行。6x86也能够双发射一条浮点指令和整数指令。然而，相对于Pentium支持80位扩展精度的浮点计算，6x86浮点部件的数据通路是64位宽并且没有流水化。同样FXCH指令不能与其他浮点指令实现双发射。

Cyrix使用了一个256字节的全相联指令cache和一个16 KB的统一cache（4路组相联），统一cache为16路交叉（以16位为边界，以更好地支持16位代码），并且提供了与Pentium的数据cache类似的双端口，同时支持载入旁路和载入定向。6x86使用了一个128项的直接映像TLB，以尽量减少TLB的访问时间。此外还有一个8项的全相联牺牲TLB，两个相对简单的TLB结构结合起来所提供的命中率与一个大的全相联TLB差不多。同时提供一个4项全相联的目录表项缓冲和支持写合并的12个写缓冲。

关于6x86（当时称为M1）的总体情况请参见Burkhardt［1994］、Gwennap［1993］和Ryan［1994］中的阐述。McMahan等人［1995］提供了更多的细节。以前还有一款Cyrix 5x86，它称为M1 sc并属于标量发射处理器。M2也就是后来的6x86MX，由Doug Beard和Dan Green设计，它采用了MMX指令集扩展，还将统一cache加大到了64 KB。

7.3.5.4 NexGen Nx586（F86）/1994

Nx586表面上看起来像是一个标量处理器，然而，其内在实现方式是超标量方式下的分离式微体系结构。Nx586每个周期将一条IA32指令转换为一条或多条RISC86指令，然后将这些指令分派到三个功能部件：带乘除功能的整数部件、整数部件和地址生成部件。Nx586是IA32指令集的分离式实现中惟一允许跳过IA32转换器、直接从存储器中读取RISC86指令的处理器。

Nx586的每个功能部件都有一个14项的保留站，RISC86指令必须在这里用至少一个周期来进行重命名。每个保留站按照FIFO的方式工作，但是各个功能部件之间可能乱序发射RISC86指令。这种设计主要的缺点是，如果保留站的第一条指令因为数据相关必须停顿，整个保留站都必须停顿。

Nx586需要一个单独的FPU芯片进行浮点运算，但是它包含两个16 KB的片上cache（都是4路组相联，数据cache是写直达的），一个片上L2 cache控制器（4路组相联，写回），一个同时处理三条独立路径的预取缓冲。它使用了自适应分支预测器的动态分支预测机制并支持推断执行，寄存器重命名则使用了22个物理寄存器。NxGen按照这种设计思想发展了几年，三篇相关的文章曾发表于1989年春季的COMPCON上，当时称为F86处理器。Ryan［1994］中有更近一些的信息。Mack McFarland是第一位NexGen的设计师，然后Dave Stiles和Greg Favor设计了Nx586，后来由Korbin Van Dyke负责实际的实现。

7.3.5.5 AMD K5/1995

K5的主要设计师是Mike Johnson。1989年，他在Stanford撰写的博士论文曾作为第一本超标量微处理器的设计教科书加以出版［Johnson, 1991］。K5遵循了很多书中的设计建议，而该书显然是基于超标量AMD 29000的设计思想。

K5采用分离式的设计方式并提供了对IA32的兼容性，IA32指令从存储器中取出后，连同附加的预译码位放到一个16 KB的指令cache中，这些预译码位用于辅助定位指令的域和边界。每个周期最多从指令cache中取出16个字节的指令，根据Johnson书中介绍的分支预测机制，可以将其

合并到一个字节队列中。依照这种机制，每个cache块只能有一条分支指令预测为跳转，因此cache块被限制在16个字节大小。由于没有历史位，最初每个cache块都标记为分支失败。当预测失败时，标记发生改变，以记录最近一次是分支跳转还是不跳转，这样做的效果大致相当于每个cache块有一个历史位。

在K5的指令cache中，每一条IA32指令都标识出了将要产生的微指令或者R-op的数目，这些标识相当于循环次数，以此分派相应数目的译码器来对指令进行译码。因此，每条IA32指令能够立即被发送到一个或者多个两级译码器，而不需要等待控制逻辑在译码器之间传递指令对齐信息。其代价就是增加了进行预译码所带来的指令cache填充时间。K5包含4个译码器，每个译码器每周期产生一条R-op。有趣的是，根据顺序分派到译码器的指令和标识的不同，一条指令的R-op可能被分割在不同的译码周期中。与P6一样，复杂指令不进行常规的译码过程，而是每个周期从控制存储器中取出一条由4条R-op组成的微操作流。

R-op被分派到6个执行部件：两个整数部件、两个load/store部件、一个浮点部件以及一个分支部件。除浮点部件外，每个执行部件都有一个两项保留站，而浮点部件的保留站只有一项。每个保留站每个周期能够发射一条R-op。由于分支保留站包含两项，K5的推断执行能够跨越两个未解析的分支。

R-op将结果写入一个16项的再定序缓冲，每一条R-op都有一个分派时指定的预留表项。写入再定序缓冲具有与寄存器重命名相同的效果，每个周期最多可以按照程序顺序提交4个结果。保留站和再定序缓冲的表项能够处理混合的操作数大小（IA32指令集可能有8、16和32位的操作数），将每个IA32寄存器作为三个独立项（低字节、高字节和扩展字节），每一项都有单独的相关性检查逻辑和独立的重命名标识。

分支部件发现预测失败时，立即启动多个恢复动作：分支部件清空，再定序缓冲中预测失误分支之后的所有表项都标记为失效（但是并不重置“下一项指针”），取指重新从正确的指令地址开始。已经存在于保留站和执行部件的推测R-op并不被清除。相反，这些R-op允许执行完毕并将结果写入相应的、已被作废的再定序缓冲项。由于错误预测的R-op要执行完毕并且确认时进行丢弃，因此某些代码顺序可能产生停顿，不过这些错误预测的R-op往往都是单周期的操作，在从正确的执行路径上开始取指并译码之前这些R-op将被排空。

K5的异常处理表现在两个可选择的恢复方法上。异常并不是在发生时立即处理，而是在再定序缓冲中记下异常状态，只有当再定序缓冲表项提交时处理器才处理该异常。这时，处理器中所有的R-op立即清空（需要多个周期）并从异常处理程序入口处开始取指。由于异常相对很少发生，这种长延迟的恢复动作对性能没有太大的影响。

K5片上集成了一个8 KB的双端口4体数据cache。与大多数处理器一样，写回在R-op提交时进行。与大多数处理器不一样的是，直到执行读操作的R-op成为再定序缓冲头部的R-op时，才开始读取失效时的cache填充。这样设计是为了避免对存储器映像的I/O设备寄存器进行不正确的访问。提前启动cache填充要求特殊的逻辑来处理这些设备寄存器。

K5的cache编址方式是很特别的。cache既有虚地址标识也有实地址标识，虚地址标识用于快速访问，但是在线程切换时必须失效。然而，如果发生失效，首先检查物理地址标识，如果匹配，该cache块可以重置为有效并给定一个新的虚地址标识。这样做避免了对共享存储部件的cache块进行重复填充，同时为进程的重新调度保留了cache轨迹。物理标识也用于总线监听。

K5的性能令人失望，据说是由于设计时没有正确地考虑来自Windows 3.x应用程序的负载特性信息。AMD 1995年与Compaq签订的提供K5芯片的协议失败后，AMD买下了NexGen。关于K5设计的更多信息，参见Gwennap［1994］，Halfhill［1994］以及Christie［1996］。

7.3.5.6 Intel P6 核（Pentium Pro/Pentium II/Pentium III）/1996

Pentium Pro 是第一个基于P6微体系结构的产品，它是一个多芯片的模块，包含一个有550万个晶体管的处理器和1550万个晶体管的二级cache，处理器包括一个8 KB的指令cache和一个8 KB的双端口数据cache，二级cache为256 KB。P6的设计团队包括负责人Bob Colwell，高级设计师Glenn Hinton和Dave Papworth，以及Michael Fetterman和Andy Glew。P6微体系结构的详细描述参见第6章。

P6是Intel第一次使用分离式微体系结构的产品（参见图7.10），其微体系结构按照超流水的方式设计。Intel称转换之后的微指令为μop。取指和转换的流水线为8级，一级用于计算指令地址，两级半用于访问指令cache，两级半用于译码，一级用于寄存器重命名，最后一级为转换之后的μop分配再定序缓冲和保留站表项。再定序缓冲包含40项，保留站包含20项。P6去掉了IA32浮点栈模型的限制，允许FXCH（交换）指令在经过重命名寄存器硬件处理之后标志为完成。

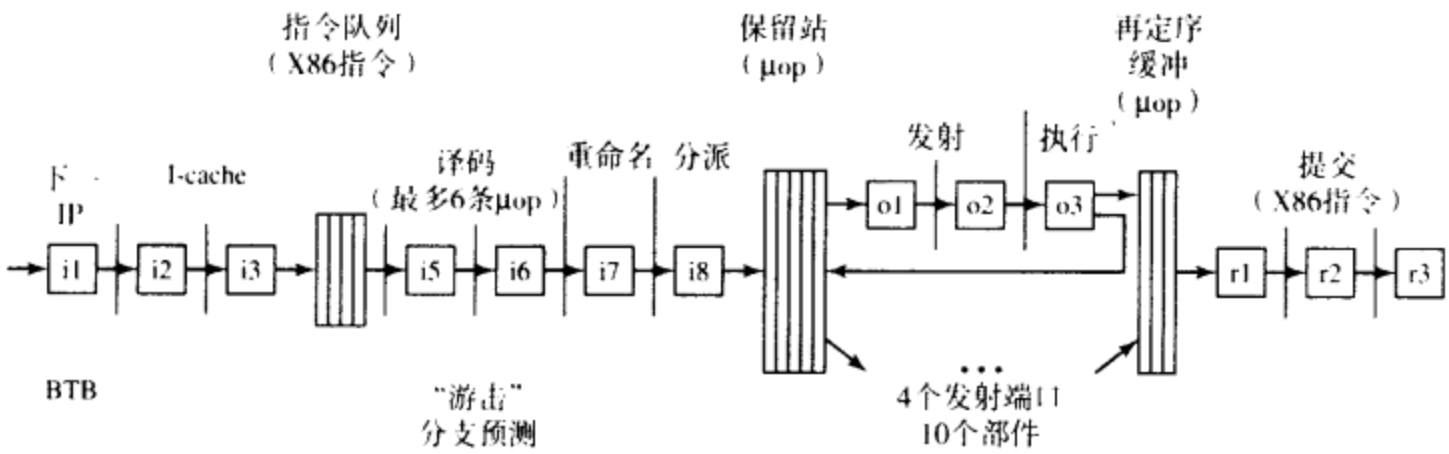


图7.10 Intel P6流水线

在每一个时钟周期内，取指部件取出两个指令cache块（每个32字节），并且提取出一个16字节的指令块放置于译码器的缓冲中。取指过程不仅要求指令cache提供额外的预译码位，还要标出指令边界。译码部件最多可以同时译码3条IA32指令，但是为了维持最大的译码效率，必须对指令进行整理，使得只有第一条指令能够产生多个μop（最多4条），而其他两条指令每条只产生一个μop。带有存储器操作数的指令需要多个μop，因此译码速度限制为每周期一条IA32指令。特别复杂的IA32指令（如PUSHA）需要从控制存储器中取出一长串μop分派到处理器，这需要多个周期。前缀字节以及同一指令中立即数和带偏移寻址模式的组合都会对译码速度产生影响。

每个周期可以完成对三个μop的重命名，然后将它们写入保留站和再定序缓冲。每个周期按照一种伪FIFO的方式扫描保留站，最多可以同时发射4个μop。一个发射端口（实际上是两个读端口和一个写端口的集合）支持宽数据路径并且连接着6个执行部件：整数部件，浮点加法部件，浮点乘法部件，整数除法部件，浮点除法部件以及整数移位部件。第2个发射端口发射μop到第2个整数部件和分支部件。第3个发射端口分派给load操作，而第4个（实际上还有第5个端口）分派给store操作。保留站的扫描和发射需要两个周期，而大多数指令的执行可以在一个周期内结束。发射时需要优先考虑紧随的μop，以增加执行部件间操作数定向的数量。

再定序缓冲每个周期最多可以接受3个结果并提交3个μop。提交操作需要3个周期。整个流水线包含14级，Intel声称其中的某些段可以重叠，因此实际上每条指令的最小端到端延迟为12个周期。

P6的分支处理相当复杂，由于冗长的流水线，预测失败的开销最多为15个周期。P6使用512项BTB的两级自适应分支预测方法。关于P6分支预测机制的详细信息参见第6章。

在P6中，BTB如果能够正确预测不跳转的分支指令，就不会引起停顿，而正确预测发生跳转的分支有一个周期的停顿。BTB的错误预测会产生10个到15个周期的额外开销。当BTB中没有

找到指定的分支时，采用一种“游击”译码器（shortstop decoder）根据偏移标记进行预测。“游击”译码器正确预测的不向前跳转分支不会引起停顿，而正确预测的向后跳转分支存在5个到6个周期的开销。“游击”译码器的错误预测会导致最大 10 个到 15 个周期的开销。P6 增加了一条条件传送指令到 IA32 指令集中，从而避免一些条件分支。而且，P6 还增加了将浮点条件代码传送到整数条件代码的指令。

存储器顺序缓冲包含 20 项，8 个读表项用于提供到数据 cache 的非阻塞读访问，允许载入旁路（load bypass）并执行 load 操作的合并。12 个写表项用于提供写定向和写合并。一级数据 cache 包含 4 个交叉的体，每周期能执行一次写和一次读操作。二级数据 cache 位于专门的后端总线（BSB）上，并以处理器的时钟频率运行。然而，各级 cache 之间没有强制的包含性。

Pentium Pro 发布之初频率为 133 MHz，随后立即提高到 200 MHz，其性能令业界感到惊奇，因为它在 SPEC 整数测试程序上的表现甚至超过了同时代的 300 MHz 的 DEC Alpha 21164。Pentium Pro 是 P6 微体系结构最早的实现，其目标是面向高端工作站市场。在个人电脑市场，其性能表现相对没有那么引人注目。设计者对 16 位程序（如虚拟 8086）的性能进行了折中，从而尽量提高 32 位程序的性能。因此设计者选择串行执行这样一些指令（如远程调用）和其他段寄存器切换指令。P6 微体系结构后续的实现包括 Pentium II 和 Pentium III。Intel 的 Celeron 系列以及 Xeon 的早期系列都是基于 P6 微体系结构。

有关 Pentium Pro（即 P6）的详细描述请参见 Gwennap［1995］和 Halfhill［1996］。Papworth［1996］中谈到了该微体系结构中设计上的折中方案。

7.3.5.7　Pentium 4/2001

Pentium 4 是 Intel 最新的 IA32 微体系结构，其流水线超过 20 级。为了避免 IA32 指令到 μop 的重复转换，转换之后的 μop 存放到一个跟踪 cache 中。关于 Pentium 4 微体系结构的描述参见 Hinton 等人［2000］中的内容。

7.3.5.8　AMD K6（NexGen Nx686）/1996

1995 年，显然是由于 K5 出货的下滑，AMD 买下了 NexGen 并宣称 Nx586 的后续设计都将标记为 AMD K6。Nx586 的后续设计称为 Nx686，由 Greg Favor 完成。Nx686 对 Nx586 的扩展包括集成了一个浮点部件和一个多媒体操作部件，cache 也进行了扩大并且译码速度翻了一番。

K6 有三种类型的译码器并按照相互排斥的方式工作。首先，有一对短译码器（short decoder），每个译码器能单独译码一条 IA32 指令，同时产生一条或者两条 RISC86 操作。另外还有一个长译码器（long decoder），能够处理单条复杂的 IA32 指令，最多可以产生 4 条 RISC86 操作。最后，与 P6 类似，K6 还包括一个向量译码器（vector decoder），提供一个初始的 RISC86 操作集合并开始从控制存储器读出连续的操作集合。K6 最大的译码速率是每周期 4 条 RISC86 操作。AMD 宣称，K6 平均每条 IA32 指令的 RISC86 操作数目比 P6 的 μop 数目少，因此译码速率比 P6 快。K6 的译码器包含预译码位辅助，这些位在指令 cache 填充时产生，并作为指令 cache 块的一部分存储。

K6 将 RISC86 操作分派到一个 24 项的集中式保留站，保留站每个周期最多能发射 6 个操作。寄存器同样要进行重命名，共使用 48 个物理寄存器。K6 对分支的支持包括一个 8192 项的分支历史表，并基于 GAs 实现自适应分支预测。一个 9 位的全局分支历史移位寄存器和来自指令地址的 4 位组合来选择 8192 个 2 位计数器中的一个。同时还提供一个 16 项的目标指令 cache（每行 16 字节）以及一个 16 表项的返回地址栈。

K6 上的数据 cache 实际上每个周期运行两次并提供了双端口，每个周期支持各一次读操作和写操作，这种方法避免了其他 IA32 实现中出现的体冲突的问题。未对齐的存储器访问也比较快，K6 的开销为一个周期，而 Pentium Pro 则为 6 个周期。

关于K6的描述参见Halfhill［1996］。Shriver和Smith［1998］提供了很多关于K6-3D的研究范例，包括对AMD的工程师、设计者以及管理人员的采访录像片段。

7.3.5.9 AMD K7（Athlon）

Dick Meyer是Athlon的主要设计师。Athlon的描述参见Diefendorff［1998］。这是当前AMD产品的旗舰微体系结构。K8（Opteron）于2003年发布，其设计采用了64位扩展地址的IA32结构。

7.3.6 MIPS

7.3.6.1 概述

MIPS体系结构是RISC的精髓，最初来自于Stanford大学的John Hennessy对无互锁流水线的研究工作，而MIPS公司的第一项设计就包括了一个无互锁的读延迟槽。MIPS-I和MIPS-II体系结构分别定义于1986年和1990年，并由Craig Hansen完成。Earl Killian是1991年提出的64位MIPS-III的设计师。Peter Hsu在SGI与MIPS合并以前开始了MIPS-IV的研究，R8000和R10000/12000实现了MIPS-IV。最后一个版本MIPS-V由Earl Killian，Bill Huffman以及Peter Hsu在1995年完成，MIPS-V包括了MIPS数字媒体扩展，并且计划在未来的H1和H2处理器中实现。

MIPS的设计以简洁快速的流水线著称。1992年，当大多数其他公司选择超标量方法时，R4000的设计者（Peter Davies，Earl Killian以及Tom Riordan）选择了超流水（而ALU是单周期的）处理器。通过模拟，对于未进行重编译的整数代码，超流水设计的执行性能优于其内部设计的超标量设计。这是因为超标量设计需要多个整数部件，每个周期才能发射多条整数指令，但是却不能在同一周期中发射相关的整数指令对；相反，超流水设计的一个周期包括两个快速周期，通过使用一个单周期（快速周期）的ALU，能够在两个快速周期内发射相关的整数指令对［Mirapuri等人，1992］。同MIPS相比，SuperSPARC使用的方法比较有意思。SuperSPARC也是1992年设计的，它使用级联的ALU来支持相关指令的发射（MIPS近期的设计都是超标量的）。

7.3.6.2 MIPS R8000（TFP）/1994

MIPS R8000是超标量但不是超流水的，这看起来似乎有点反常。实际上，R8000是TFP（Tremendous Floating Point）设计的最终名字，TFP由Peter Hsu在Silicon Graphics公司设计。R8000是64位的机器并且针对浮点运算。在某些方面看起来是对IBM推出的POWER（RS/6000）处理器的反应。然而，R8000的许多主要思想都是受Cydrome Cydra-5的启示。Peter Hsu是Cydrome的学生，他的设计团队中还有一些人也是：Ross Towle，John Brennan以及Jim Dehnert。Hsu还雇用了Paul Rodman和John Ruttenberg，这两人以前曾在Multiflow工作。

R8000将整数数据和地址同浮点数据分离的技术是很特别的。整数数据可以放到片上16 KB的cache中，而浮点数据不能。因为在很多程序中，浮点数据的时间局部性很差，这样做的目的就是为了避免浮点数据降低cache的效率。R8000提供了一个巨大的二级cache来解决浮点数据所需要的存储器带宽，这个二级cache为两路交叉并且包含5级访问流水线（两级用于芯片间的交叉访问）。通过一个单入口地址来减少体冲突并对cache访问重新排序，以增加组合成奇偶体请求的概率。

当浮点数据和整数数据在同一个数据结构中混合在一起，甚至指定到同一个域中（如union数据结构）时，外部cache和片上cache之间存在一致性问题。为了解决这一问题，片上cache为每个字设置了一位有效位（MIPS体系结构要求访问是对齐的）。cache填充时将所有的有效位置为有效，进行整数和浮点写操作时分别再设置相应的位。

R8000 每个周期最多能将 4 条指令发射到 8 个执行部件：4 个整数部件，两个浮点部件，两个访存部件。整数流水线在译码段之后插入了一个空流水段，使得 ALU 操作的相对位置与 cache 访问在 load/store 流水线中的相对位置相同，这样就不会存在访存延迟，但如果地址生成和 load 的数据相关，则将导致一个周期的停顿。

在发射就绪以前，浮点指令存放在一个浮点队列中。当一条浮点 load 指令（包含 5 个周期的延迟）连同一条相关的浮点指令被分派时，整数流水线也能继续执行。浮点计算的异常是不精确的，但是和 IBM POWER 一样，R8000 提供了一个浮点串行化模式位来帮助调试。

R8000 采用了一种将分支预测和指令对齐结合在一起的机制，这类似于 Mike Johnson 在其著作中提到的机制。cache 中每 4 条指令构成一个指令块，每个指令块有一个分支预测位。预测表项中的位掩码指明分支指令块中有多少指令是有效的，另一个位掩码指明分支目标指令从目标指令块的什么地方开始。如果编译器能够提供支持，以消除同一指令块中有两个可能跳转的分支指令的问题，这将对提高性能很有帮助。R8000 在 cache 外增加了只能以 cache 块为边界的访问逻辑，而 IBM POWER 是在 cache 内部增加逻辑，将这两种机制进行比较会给我们带来很多启示。

Hsu［1994］提供了 R8000 的详细描述。

7.3.6.3　MIPS R10000（T5）/1996

尽管 R8000 是一个多芯片的实现，R10000（以前称为 T5，由 Chris Rowen 和 Ken Yeager 设计）则是一个单芯片的实现，并且每个周期最多能乱序发射 5 条指令。R10000 微体系结构（如图 7.11 所示）值得注意的地方是将操作数和结果流水化，而不像 AMD K5 那样，将操作数的副本存放于保留站并将结果写入再定序缓冲。R10000 只在物理寄存器文件和功能部件之间传递数据，因此比较器的数目以及控制逻辑的数量都得到减少。这也有利于时钟频率的提高，因为数据通路上操作数和结果必须穿越的门级数有所减少。

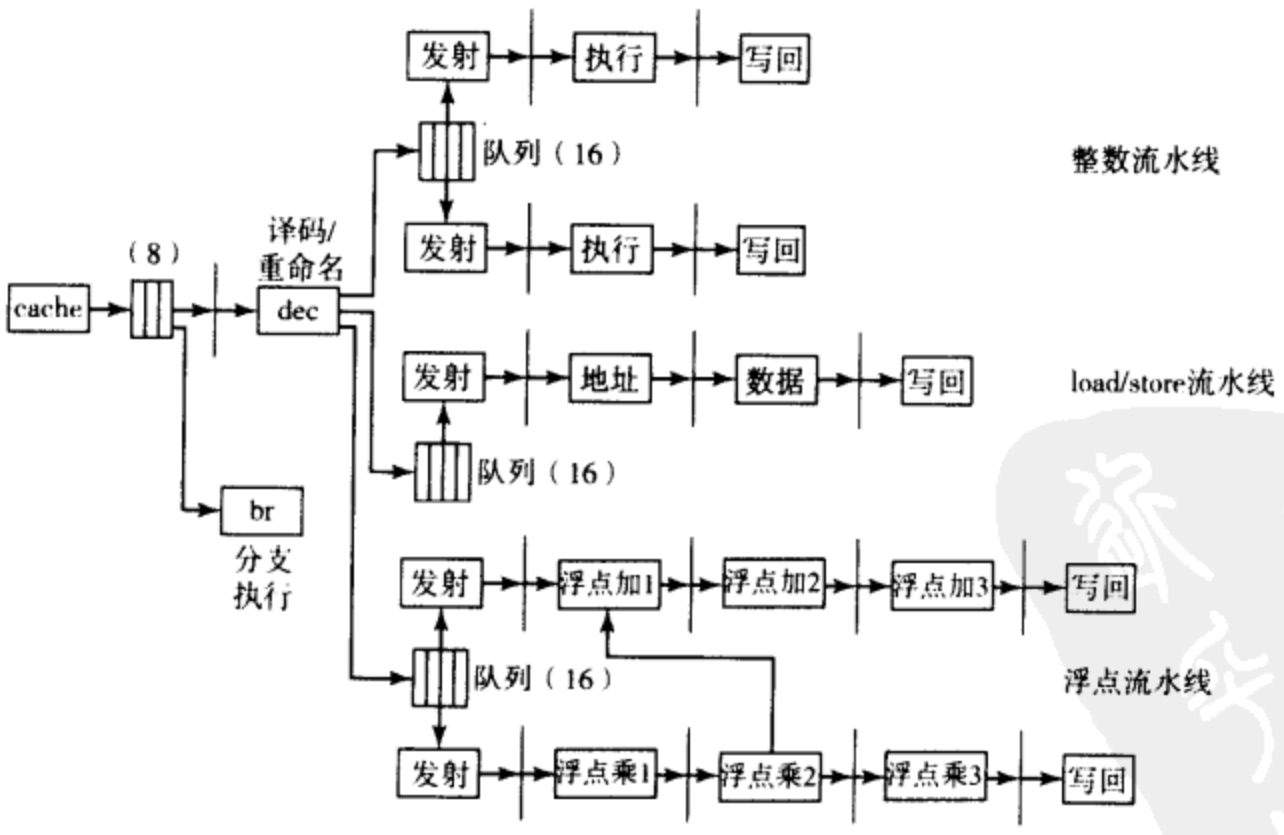


图 7.11　MIPS R10000 流水线

R10000 的指令存放在一个 32 KB 的指令 cache 中，预译码位也同时存于其中，每周期最多能从一个指令 cache 块的任何位置取出 4 条指令。译码速率为每周期 4 条指令，但是指令 cache 与译码器之间有一个 8 项的指令缓冲，这样即使译码发生停顿，取指也能继续进行。

译码器还负责重命名寄存器。虽然MIPS指令集定义了33个整数寄存器（31个寄存器加上两个用于乘除法的特殊寄存器）和31个浮点寄存器，R10000有64个整数物理寄存器和64个浮点物理寄存器。当前的逻辑寄存器与物理寄存器之间的映射由两张映射表维护，一张表用于整数，另一张表用于浮点。

指令从译码器分派到三个指令队列中的一个：整数、访存和浮点队列。这些队列充当保留站的角色，但是它们不包含操作数的值，只有物理寄存器号。当指令从队列中发射时，操作数从物理寄存器文件中读取。每个队列有16个表项，支持乱序发射。每个周期最多能发射5条指令：发射两条整数指令到两个整数部件，这两个整数部件一个能执行分支指令，另一个含有整数乘除法电路；发射一条访存指令到访存部件；还能同时发射一条浮点加法指令和一条浮点乘法指令。

R10000实现了浮点乘加指令，这种类型的指令必须首先通过乘法流水线的前两级执行段，然后发送到加法流水线，完成规格化并写回。其他浮点操作的结果也能在两级执行段之后进行定向。

R10000处理器在一个32项的活动列表中记录已译码指令的物理寄存器分配情况，该列表按照程序顺序维护。每分派一条指令，就在列表中给这条指令分配一个表项，并将该项的完成标志初始化为0。列表中表项的索引也用于对分派到指令队列的指令进行标识。每条指令完成时，将结果写入为其分配的物理寄存器，然后用索引活动列表将完成标志置为1。活动列表通过这种方式充当再定序缓冲的角色并完成顺序提交，MIPS称之为结业（graduation），以区别于在提交时将再定序缓冲的结果写入寄存器的过程。

活动列表的表项包含了在指令中作为目标寄存器的逻辑寄存器号，以及前面分配的物理寄存器号，这样的设计为异常处理提供了一种历史缓冲。当发现异常时则停止分派指令，当前指令允许执行完毕，然后按照逆序检查活动列表，每个周期4条指令并通过重新将以前的寄存器分配写回映射表，取消物理寄存器的映射。为了提供精确的异常处理，这一过程将继续进行直到发生异常的指令取消映射。这时，就可以调用异常处理程序开始处理异常。

为了使分支预测失误能快速恢复，可以使用检查点恢复机制，保存寄存器映射表的副本并为每个分支保存另一条路径的地址。最多可以同时存在4个检查点，因此R10000的推测执行可以跨越4个分支。预测失误的恢复需要一个周期将映射表恢复到分支指令的那一点上，然后再重新从正确的地址取指。错误预测路径上推断执行的指令同样要从处理器中清空。因此R10000为每条已译码的指令增加一个4位的分支掩码。掩码指明一条指令是否是推断执行的，以及这条指令依赖于4个预测分支中的哪一个（可以设置多位）。分支结果出来之后，若预测正确，则要复位处理器中每条指令分支掩码的相应位；相反，错误的预测将导致设置了相应位的指令在一个周期内被清除掉。

整数乘法和除法指令有多个目标寄存器（HI，LO），从而打乱了正常的指令流。R10000的译码器遇到这样的指令将停顿下来，且译码器不会将乘法或除法指令作为一个译码组的第4条指令分派，原因是多个目标寄存器需要特殊处理：每条乘法或除法指令需要在活动列表中分配两个表项。

R10000通过一个特殊的条件寄存器来支持条件分支，其中每一位对应一个物理寄存器，当向物理寄存器文件写入值为0的结果时，相应位被置1。与0比较的条件分支通过检查条件寄存器中的相应位，可以立即决定是否跳转，而不需要从物理寄存器文件中读出数据并检查是否为0。R10000为分支预测维护一个512项的BHT，但是没有缓存分支目标地址，因此正确预测的跳转分支仍需要一个周期的开销。

R10000另一个分支特性是分支4字链，包含跨越最近子程序调用的最多4条指令。这相当于一个返回目标指令缓存并支持从子程序的快速返回。在1994年设计的初期，设计者提出了一种类似cache的结构，用于保存最近4次预测为跳转的分支指令在顺序执行路径上的4条指令。当检测到预测失误时，这一分支恢复cache（branch-resume cache）能够用于快速重新取指，并且1994年

到 1995 年的描述文档都称这一机制为 R10000 独有的特性。然而，相对于另一种简单方法，即从指令 cache 取出顺序执行路径上的指令，该机制在最好的情况下也只能节省一个周期，因此在实际的 R10000 芯片中没有采取这一机制。

为了支持强存储器一致性，访存指令队列按程序顺序来维护。R10000 使用了两个 16 × 16 的矩阵来进行地址匹配，同时提供了载入定向功能以检测并避免冲突。

R10000 的概要情况请参见 Halfhill [1994]。Yeager [1996] 对此进行了较为深入的描述，包括指令队列和活动列表的细节。Vasseghi 等人 [1996] 描述了 R10000 的电路设计细节。

R10000 的后续设计 R12000 将活动列表增加到 48 项，BHT 增加到 2048 项，并且还增加了一个 32 项、两路组相联的分支目标 cache。最新的 R14000 类似于 R12000。MIPS 曾计划在转向 Intel Itanium 之前推出 R16000。

7.3.6.4　MIPS R5000/QED RM7000/1996/1997

R5000 由 QED 设计，QED 是 Earl Killian 和 Tom Riordan 创建的公司，他们也设计了 R4x000 系列处理器中的一部分。R5000 的特性包括 200 MHz 的主频和两个 32 KB 两路组相联的 L1 cache。它的组织结构很简单，并且每个周期只发射一条整数指令和一条浮点指令。然而，其性能却可以与一个扩展的、具有乱序执行能力的设计相提并论。Riordan 将该设计扩展为 300 MHz 的 RM7000，它仍然保留了 R5000 的双发射结构，但是片上集成了一个 16 KB 4 路组相联的 L1 指令 cache，一个 16 KB 4 路组相联 L1 数据 cache，以及一个 256 KB 的 4 路组相联的 L2 统一 cache。

7.3.7　Motorola 68060 / 1993

68060（如图 7.12 所示）是 68000 CISC 体系结构的一个实现，并且是这一系列处理器中第一个推向市场的超标量版本。虽然许多早期的工作站使用的是 680x0 处理器，并且 Apple 为 Macintosh 选择的也是 680x0 处理器，然而 680x0 系列处理器已经被更多的 IA32 和 RISC 设计所取代。实际上，Motorola 在工作站市场已经选择了 PowerPC，因此 68060 被设计为低端、低功耗产品，以进军嵌入式系统市场。68060 的设计师是 Joe Circello。

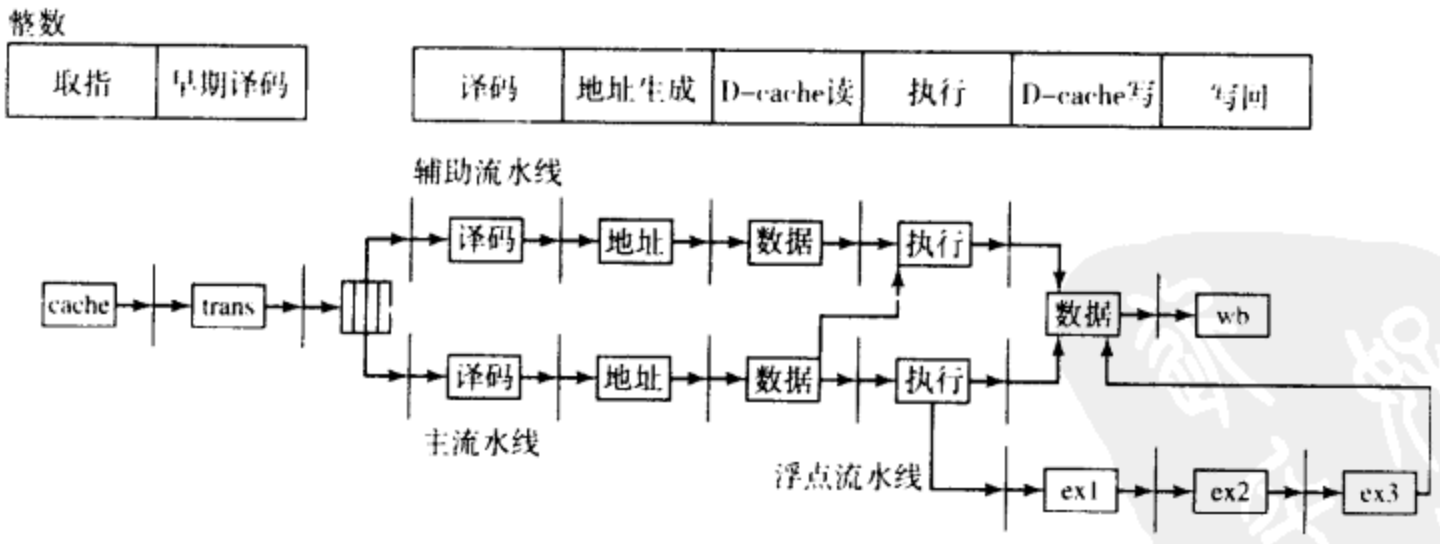


图 7.12　Motorola M68060 流水线

68060 的实现方式是流水化的，它具有分离式的微体系结构，将变长的 68000 指令转换为定长的、能够完全确定所需资源的指令。这样就简化了相关检测和发射逻辑。而且，一些更复杂的 68040 指令类型不由硬件处理，而是通过自陷加以实现的并由软件进行处理。

取指流水线基于分支预测来获得指令，并将转换后的指令存放到一个 16 项的 FIFO 缓冲区中，该缓冲区将取指和指令转换与指令的执行分隔开。指令从缓冲发射，在每个时钟周期内，发射逻辑

尽量按时序发射两条指令到两条4级执行流水线中。缓冲中的每个表项可以容纳一个16位操作码、32位的扩展字以及早期的译码信息。一些复杂的指令需要一个以上的表项。

由于从取出分支指令到分支条件的计算之间需要多个周期，因此68060使用了一个256项、4路组相联的分支cache，同时带有两位预测信息。错误的分支预测将导致7个周期的开销。68060的一个突出特性是分支隐藏（branch folding），并将分支条件、一个地址恢复增量以及分支目标地址一起存放到分支cache中。分支cache的表项用分支指令前一条指令的地址进行了标识，只要条件码满足分支条件，就能将分支从发往FIFO缓冲的指令流中去除掉。分支隐藏技术最早出现在CRISP处理器中［Ditzel和McLellan，1987；Ditzel等人，1987］。

主执行流水线能执行所有的指令，并在一个单独的执行部件中执行浮点指令。辅助执行流水线只执行整数指令。这两条流水线必须按照锁步的方式工作（类似于Pentium），但是控制逻辑要复杂得多。每一条执行流水线由两套取指和执行段组成，Motorola将该机制描述为“背靠背”放置的两个RISC引擎。带有存储器操作数的指令需要这两个引擎：第一对取出地址并用一个ALU部件计算出有效地址（并启动cache读操作），第二对取出寄存器操作数，并使用一个ALU部件计算出操作结果。进一步考虑，通过将计算有效地址的ALU通用化，一些操作可以由主流水线的前两段执行，然后将结果定向到辅助流水线。因为有些指令总是在前两段执行，其他指令根据发射指令对之间的相关性动态地分配。在很多情况下，相关的指令对能够在同一周期里发射。68060同样使用了寄存器重命名来消除假相关。

数据cache是4路交叉，并允许同时执行一次读操作和一次无阻塞的写操作，但不支持一个周期两次读或者两次写。关于68060更详细的描述，请参见Circello和Goodrich［1993］和Circello等人［1995］。

7.3.8 Motorola 88110/1991

88110在它所处的时代是一个非常出色的设计并发布于1991年，就在IBM RS/6000开始流行的时候。88110是Morotola 88K RISC体系结构的双发射实现，并对88K体系结构进行了扩展。88110引入了一个单独的扩展精度（80位）浮点寄存器文件，并增加了图形指令和无延迟的分支。88110值得注意的地方是它包含了10个功能部件，并使用了历史缓冲来提供精确的异常处理和分支预测失败时的恢复机制。Keith Diefendorff是88110的主要设计师，Willie Anderson设计了图形和浮点扩展部分，Bill Moyer设计了存储系统。

10个功能部件分别是取指分支部件，访存部件，位操作部件，浮点加法部件，乘法部件，除法部件，两个整数部件以及两个图形部件。浮点操作使用扩展精度运算，整数和浮点寄存器文件都有两个专门的历史缓冲端口，用于记录结果寄存器的原始值。历史缓冲包含12项，每周期能记录两个寄存器。

一般情况下，88110每周期取两条指令，除非这两条指令跨越了cache块。译码器尽可能地在每个周期发射两条指令。分支部件包含一个单项的保留站，写操作部件包含一个3项的保留站，因此除了分支和写操作，其他指令都是顺序发射。对于跨越预测的分支，其推测指令的条件位将被标记，一旦预测失败，这些指令就要清除掉，而对于任何被带条件位的指令写过的寄存器，需要使用历史缓冲中的值进行恢复。条件存储操作禁止改写数据cache，它必须驻留在保留站，直到预测的分支证明是正确的。分支是静态预测的，包含目标指令cache（TIC）并使用分支指令的地址来索引、返回分支目标地址处的两条指令。TIC能提供最近发生跳转的32次分支的目标指令。TIC实现了全相联、虚地址访问，因此在上下文切换时必须清空。

88110不进行寄存器重命名，但是允许同时发射两条带有写后读相关的指令，也允许同时发射产生结果的指令和相关的写操作。当然，如果写操作保留站有一个空闲的项，可以在结果产生指令

执行过程中的任何时候发射相关的写操作。访存部件包含一个4项的读缓冲，并允许load指令旁路store指令。

异常可以按照程序顺序实现精确定位，异常发生以前的指令允许提交，异常指令之后的指令更新过的寄存器要使用历史缓冲进行恢复。外部中断具有较短的响应延迟，一旦发生中断，就终止所有未完成的指令，并使用历史缓冲来恢复寄存器文件。

10个功能部件共享两条80位的写回总线。由于各个功能部件的延迟不一样，某个周期内可能有两条以上的指令竞争总线。总线仲裁优先级不同于一般的策略，它优先将总线分派给较少时钟周期的操作，从而进一步推迟了长延迟的操作。

所有片上cache都是8 KB、两路组相联并能进行大量的cache控制操作。每个cache都支持失效行或者整个cache，并且能够冻结一个或两个体。数据cache可以根据块地址或页地址清除cache块，也可以清空整个cache。数据cache能够配置为写回/按写分配，或者写直达/不按写分配。88110还有不进行分配的写直达、行预取、行分配等方式，以及不需要访存并直接将内容置为0的指令。数据cache是非阻塞的，能够在一次失效下继续执行并且不妨碍下一次的读写命中。

Apple，Next，Data General，Encore以及Harris设计了配置88110的机器（后面三个公司推出了基于88110的系统），但是Motorola在制造全功能的芯片时出现了困难。因此，88110设计从来没有完全实现过，并且为了支持PowerPC，Motorola取消了88110的修订计划和后续设计。然而，88110的cache、TLB以及总线设计技术则用在了IBM/Motorola PowerPC处理器以及Motorola 68060上。

关于88110的文章请参见Diefendorff和Allen[1992a]，Diefendorff和Allen[1992b]以及Ullah和Holle[1993]。

7.3.9　IBM POWER

7.3.9.1　概述

IBM POWER体系结构在7.2.3小节中进行过简要的描述，在Tilak Agerwala和John Cocke的研究基础上，IBM POWER在逻辑上划分为三个独立的部件，每个部件有自己的寄存器组。最初它是在1989年（作为IBM RISC System/6000）由Rich Oehler等人提出。之后，这一系列处理器就出现了多种模型。基于最初的RIOS芯片组的POWER实现可以通过时钟频率和cache的大小来区别，但也有单芯片的实现，即RSC，以及一个功能更强大的芯片组POWER2。

POWER体系结构面向高性能双精度浮点运算。每一个浮点寄存器宽度为64位，所有的浮点操作都按双精度完成。实际上，单精度的读写需要额外的时间进行单精度到内部双精度格式的转换。POWER提高性能的一个主要因素是乘加指令（FMA），这是一条包含4个操作数的指令并将前两个操作数相乘，再加上第三个操作数，最后将结果存放到第4个操作数指定的部件。在数值计算中会多次遇到这种操作形式。RIOS卓越的逻辑设计将该操作用一个两级流水加以实现。乘加操作合并的一个负面效应是加法操作必须使用比双精度更宽的设计，但这也促进了具备这一特性的数值算法的发展。

提高性能的第二个因素是对load操作中浮点寄存器进行重命名，并允许同一个循环体的多次迭代重叠执行。这是因为下一次迭代的load操作不会因为重用一个存有有效值的寄存器而被延迟。通过指定一个新的物理寄存器，所有后续浮点指令的寄存器引用都被重新映射到正确的物理寄存器（在RIOS中，使用了38个物理寄存器来映射32个逻辑寄存器）。

POWER提高性能的第三个因素是使用超宽译码策略，以及将尽可能多的指令分派到指令缓冲的分派方法。这些方法能够确定指令流中的分支并尽早化解。

对事务处理和数据库应用的重视，使得PWOER在实现以cache块（128字节）为粒度的记录级锁定（record-level locking）时提供了硬件支持。只要操作数完全包含在一个cache块中，硬件就可以支持未对齐的操作数。跨越cache块的访问将导致异常，异常处理程序必须进行多次存储器访问来模拟该操作。减少这种未对齐异常的频率是另一个影响性能的因素，这可以通过增大cache块来实现。

7.3.9.2 RIOS流水线/1989

图7.13描述了早期POWER实现的流水线。这条流水线与Greg Grohoski设计的America处理器的流水线基本上是一样的。指令cache和分支部件每周期能取出4条指令，甚至可以跨越cache块边界。在顺序执行中，这4条指令被放置到一个8项的顺序指令缓冲中。虽然采取了预测分支为不跳转的策略，并实现了顺序指令有条件地发射和分派，但是在分支逻辑检查指令缓冲的前5项时，如果找到了一条分支指令，就从分支目标地址处取出4条指令并存放到一个特殊的缓冲中。如果分支没有跳转，则目标缓冲被清空；如果分支跳转，则顺序缓冲就被清空，目标指令将被移动到顺序缓冲中，同时，推测分派的顺序指令将被清除，并且如果有必要，分支部件寄存器将从历史寄存器中重新装载数据。这样就沿着分支跳转的路径顺序执行下去。

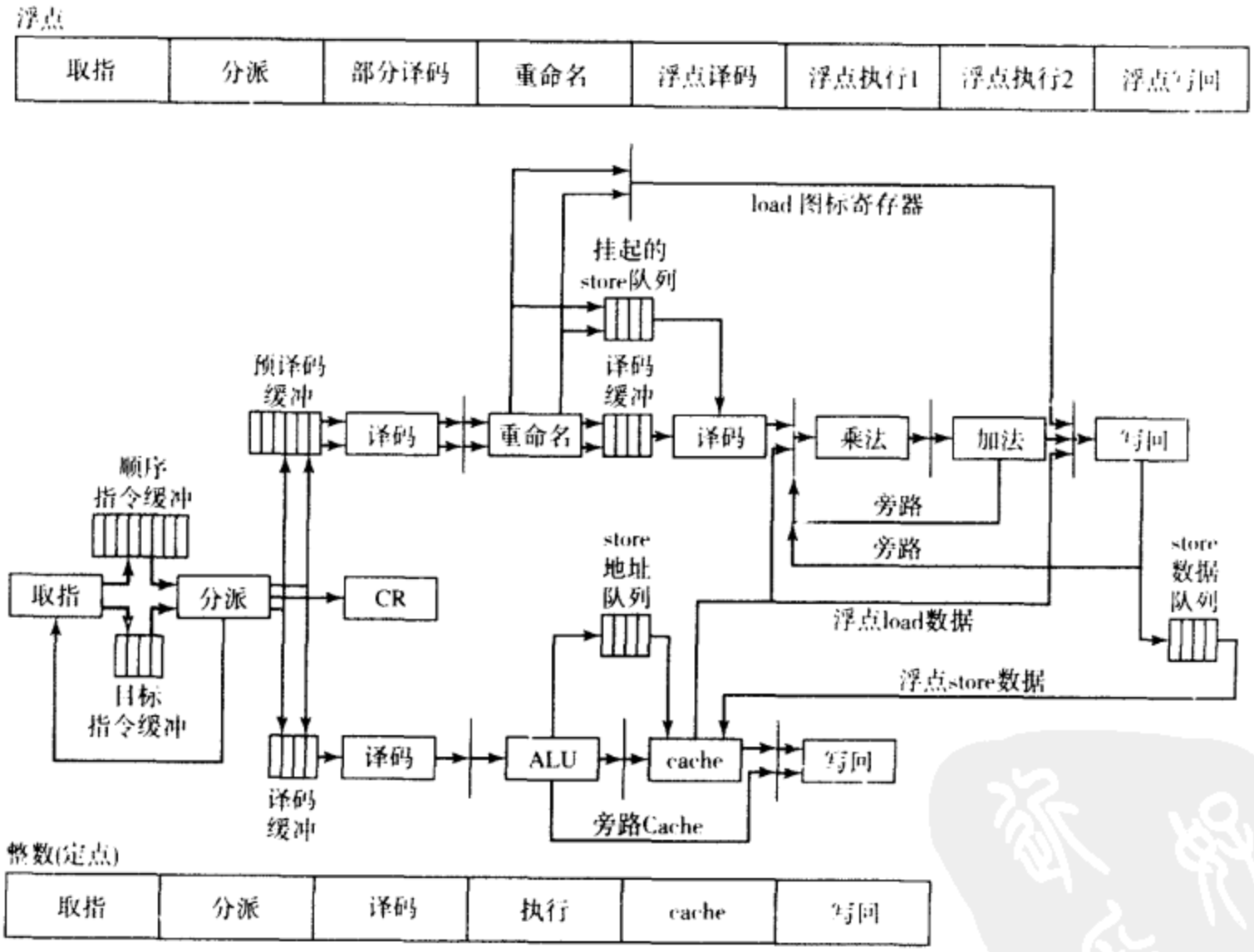


图7.13 IBM POWER（RIOS）流水线

指令cache和分支部件每个周期最多发射/分派4条指令。两条指令能够被发射到分支部件，而两条浮点和整数（在POWER中称为定点）指令则分派到缓冲，这两条指令可以都是浮点指令、整数指令或者每类一条指令。FMA指令则作为一条浮点指令。对于load/store浮点指令，两个部件都可以进行分派，而其他浮点指令会被整数部件丢弃，其他整数指令则会被浮点部件丢弃。如果指令缓冲为空，相应指令类型的第一条指令将被发射到对应部件。这种缓冲区的双分派策略回避了指令组对和排序规则。预译码标识在指令cache填充时将会加到指令上去，用来确定所需要的部件并加速发射和分派。

指令 cache 和分支部件将执行分支条件码逻辑操作。其中包含三个寄存器：一个链接寄存器，保存返回地址；一个计数寄存器，保存循环计数；一个条件寄存器，由 8 个单独的条件码域（CRi）组成。CR0 是整数部件的默认条件码，CR1 是浮点部件的默认条件码。显式的整数和浮点比较指令可以使用 8 个条件码中的任何一个，但是默认的条件码会自动随着相应的指令实现隐式更新。多个条件码提供了重用的功能，并允许编译器将条件码逻辑操作替换为条件转移。然而，Hall 和 O'Brien［1991］指出，XL 编译器并不能从多个条件码中获得好处。另一方面，他们发现使用计数寄存器的分支效果会更加显著。

整数部件流水线包含 4 级：译码、执行、cache 访问以及写回。不同于传统的 ALU 旁路逻辑，POWER 的整数部件直接将 ALU 的结果传递到写回段，写回段每个周期能写回两个整数结果。浮点 store 操作过程首先生成地址，然后将地址放入一个 4 项的 store 地址队列，等待浮点数据到达。浮点 load 操作读出的数据送往浮点写回段的同时也送往第一个浮点执行段。后面这个定向允许一条浮点 load 指令和一条相关的浮点指令在同一周期发射，并且读出的数据到达执行段将没有任何延迟。设计者采取了一些措施对 load/store 操作进行了改进，其中包括利用有效地址更新索引寄存器中的内容。这样从访问存储器所存数组的循环体中去掉了索引寄存器中的增量指令，其代价是整数寄存器文件增加了一个写端口。

浮点部件每个周期内从预译码缓冲中最多接收两条指令，整数指令在预译码段则被丢弃，浮点 load/store 指令在这一段将被识别。浮点部件的第二段将重命名浮点寄存器，每周期可以重命名两条指令中的寄存器标识。新的物理寄存器被分配给浮点 load 指令并作为目标寄存器。这时 load、store 和 ALU 操作被分开，后面两种类型的指令将被送到各自的缓冲。写缓冲允许旁路 load 操作和 ALU 操作。浮点部件的第三段中每周期将译码一条指令，并从寄存器文件中读出需要的操作数。最后三段是乘法、加法以及写回。

浮点异常有两种处理方法并可以任选一种。常规执行模式不能精确定位异常，而顺序执行模式则可以。顺序执行模式一次只完成一条浮点操作。

关于 POWER 体系结构和实现的更多信息，参见“IBM 研究与发展”1990 年 1 月的特别专题、IBM RISC System/6000 技术手册以及 Oehler 和 Blasgen［1991］、Weiss 和 Smith［1994］。

7.3.9.3　RSC/1992

RSC 作为 POWER 体系结构的单芯片实现于 1992 年发布。由于当时存在着百万晶体管的约束，RIOS 芯片的并行度得不到有效的支持。因此 RSC 设计只包含三个功能部件（分支、整数和浮点），每个周期能发射或分派两条指令。RSC 片上集成了 8 KB 的统一 cache 并实现了两路组相联，其块大小为 64 字节，每块分成 4 个扇区，采用了写直达 / 不按写分配策略，此外还集成了 64 项、两路组相联的统一 TLB。

每周期最多能够从 cache 中取出 4 条指令（一个区），这些指令被放入一个 7 表项的指令队列中。每个周期对指令队列中的前三条指令译码，并且前两条中的一条指令能够发射到整数部件，一条指令能够分派到浮点部件。整数部件没有缓冲并由 3 级流水线组成，这 3 级流水线包括译码、执行和写回。cache 访问可以在执行段或写回段进行，以避免对单个 cache 的访问竞争。浮点部件包含一个两项的缓冲，指令首先被分派到缓冲中，使得分派逻辑能够更快扫描到后面的整数和分支指令。与 RIOS 不同的是，RSC 的浮点部件不进行寄存器重命名，并且乘法段和加法段各需要两个周期。

分支指令之后的指令预取根据偏移标识来推测执行，但是分支指令码中有一位能够改变预测的方向。分支部件存在着较多的限制，只能独立地处理依赖于计数寄存器的分支，以及目标地址与分支指令在同一页的分支，其他分支指令必须发射给整数部件。在分支没有解决之前不进行推测执行。

RSC还包含一个带三级优先级的微码序列器（microcode sequencer），供指令处理、DMA、中断控制和TLB失效处理共同使用。该序列器包含一个3 KB的微指令控制存储器，16个通用寄存器以及一个96字的RAM。

Charles Moore是RSC的主要设计师，关于RSC的描述参见他的文章［Moore等人，1989］。

7.3.9.4 POWER2/1994

Greg Grohoski负责对4路发射/分派的POWER结构进行扩展，包括将指令cache增加到32 KB、增加第二个整数部件和第二个浮点部件，这就是6路发射/分派的PWOER2。此外，还要求每个周期处理两条分支并允许发射相关的整数指令。

POWER2对POWER指令集进行了扩展，引入了载入/存储4字（128位）的访存指令、浮点到整数的转换指令以及浮点开方指令。页表表项搜索和PTE缓存规则也做了相应的改变，以减少处理TLB失效过程中cache失效的次数。

POWER2每个周期所取的指令增加到8条，仍然支持跨越cache块边界的取指，并且顺序和目标指令缓冲的表项数目分别增加到16个和8个。在按序分派模式下，指令cache和分支部件每个周期最多发射/分派6条指令，并且分支部件再向前检查两条指令并判断是否为分支指令。在目标分派模式下，指令cache和分支部件最多分派4条整数和浮点指令到整数和浮点部件。第二种模式不进行指令的条件发射/分派，但可以将跳转的分支指令开销减少两个周期。POWER2还有两个独立的分支站，可以检测分支条件并产生必要的分支目标地址。配置两个分支部件的主要好处是可以计算并预取第二个分支的目标地址，而第二个分支之前的一个分支是已经解析的不跳转分支。只有那些在未解析分支之前的、未解析的分支指令（可以是第一个，也可以是第二个）可以推测地发射和执行。如果遇到第二个未解析的分支，将停止发射/分派。

POWER2有两个整数部件，每一个都有自己的整数寄存器文件副本以及维护一致性的硬件。每个部件都能执行简单的整数操作，包括load/store操作。cache控制和特权指令将在第一个部件上执行，而第二个部件执行乘法和除法指令。第二个部件的乘法操作需要两个周期，并能够在一个周期内执行两条相关的加法指令。

整数部件也处理load/store指令。数据cache包括的目录都是双端口的。实际上，cache运行的时钟频率比常规时钟频率快3倍，每个整数部件占用一个时间片（往往读操作在前），cache的填充则占用一个时间片。

POWER2还包含两个浮点部件，每一个都能执行FMA指令并需要两个周期。浮点ALU之前有一个缓冲，因此可以将一条运行时间较长的指令和一条与之相关的指令分派给一个ALU，而将后面一条不相关的指令乱序分派给第二个ALU。每个ALU都有多条到对方的旁路，但只有规格化的浮点数据才能通过这些旁路来传递，没有规格化的数据或者特殊值［如NaN（非数值），无穷］必须通过寄存器文件处理。

POWER2的算术异常是不精确的，所以惟一能产生中断的指令是访存指令和整数自陷指令。当一条能产生中断的指令发射时，浮点部件和整数部件必须进行同步。

关于POWER2的更多信息，参见Barreh等人［1994］，Weiss和Smith［1994］以及White［1994］。POWER2的单芯片实现称为P2SC。

7.3.9.5 PPC-AS和RS64系列

POWER2的一个变种是用于IBM AS/400系列计算机微处理器的基础，这一系列微处理器的第一个实现是7芯片的PPC-AS A10。PPC-AS芯片设计进行了扩展，包括AS/400所需要的72位虚地址并提供了一个单层次的存储系统，但是也可以用于普通的PowerPC模式（参见7.3.10小节）。扩

展版本由 Andy Wottreng 和 Mike Corrigan 设计。A10 的后续设计为 A30 和 A35。A35 也称为 RS64-I 并用于 RS/6000 工作站。这些设计发展成为 Star 系列，其中最新的成员是两路多线程的 SStar，称为 RS64-IV 并用于 RS/6000 产品线上。

早期 AS/400 处理器的深入研究参见 Borkenhagen 等人［1994］和 Bishop 等人［1996］的著作。关于 SStar 的更多信息，参见 Borkenhagen 等人［2000］的著作。

7.3.9.6　POWER3

POWER3 起步于 PowerPC 620 的设计方案，关于它的描述参见 O'Connell 和 White［2000］。

7.3.9.7　POWER4

POWER4 是一个 8 发射的设计，其流水线超过 15 级。译码器将一些复杂的指令分成简单的操作（称为cracking）。POWER4同时也是一种单芯片多处理器设计，一个芯片上有两个处理器核。Jim Kahle 是 POWER4 的主要设计师。IBM 的 pseries 网站上包含了关于 POWER4 微体系结构的白皮书。

7.3.10　PowerPC

7.3.10.1　概述

PowerPC 体系结构是 IBM、Motorola 以及 Apple 之间从 1991 年开始的合作结果。Motorola 和 IBM 在 Texas 的 Austin 建立了联合设计中心，并将 POWER 指令集和 88110 总线接口作为联合设计的开始，定义了 64 位的操作模式和 80 位的虚地址模式。POWER 指令集中加入了单精度浮点，修正了整数乘法和除法、字载入、预约和有条件的字存储，同时包含了对 little-endian 和 big-endian 的支持，还定义了弱排序存储器一致性模型和 I/O 栅栏指令（命名为 eieio 指令）。去掉的内容包括记录级锁定、MQ 寄存器和与之相关的指令以及一些位操作和字符串操作指令。cache 控制指令也进行了修改并提供了更大的灵活性。POWER 体系结构对于短延迟浮点指令和分支指令的尽早解决相当的重视，这一点在 PowerPC 中得以继承。PowerPC 的主要设计师包括 Rich Oehler（IBM），Keith Diefendorff（Motorola），Ron Hochsprung（Apple）以及 John Sell（Apple）。Diefendorff［1994］，Diefendorff 等人［1994］，以及 Diefendorff 和 Silha［1994］包含了更多关于 PowerPC 的合作历史以及对 POWER 改动的信息。

PowerPC 的前三个实现（601，603 和 604）只实现了 32 位模式。第 4 个实现（620）同时实现了 32 位和 64 位模式。Peng 等人［1995］对 64 位的体系结构进行了描述。

7.3.10.2　PowerPC 601/1993

601 是 PowerPC 体系结构的第一个实现，由 Charles Moore 和 John Muhich 设计。由于上市时间紧迫，因此 Moore 用以前在 IBM 完成的 RSC 设计作为原型，同时采用了 Motorola 88110 的总线设计和 cache 一致性机制，以支持 Apple 以前基于 88110 的系统设计。与 RSC 相比较，统一 cache 扩大到 32 KB，TLB 结构遵循了 88110 的设计，同样也提供了一个 4 项的转换影子阵列以减少 TLB 的冲突。三个功能部件的功能设计也和 RSC 相同（分支、整数和浮点）。

PowerPC 601 提供了一个 8 项的指令队列，底部的 4 条指令经过了译码。浮点指令和分支能从这 4 项中的任意一个进行分派，但是整数指令只能从最下面的表项发射。一个特有的标识可以将同一周期发射 / 分派的指令链接在一起。如果某周期没有整数指令发射，则产生一条 nop 操作，使之能构成一个指令包。

601 的浮点部件遵循 RSC 的设计方案并包含一个两项的浮点指令队列，并且不对浮点寄存器进行重命名。由于指令集直接支持单精度，601 使用了 RSC 的浮点乘加流水线。

整数部件也处理 load/store 指令，但是 601 的存储系统比 RSC 要复杂得多。601 在处理器和 cache 之间增加了一个两项的读队列和一个 3 项的写队列。在 cache 和存储器之间，增加了一个 5 项的存

储队列，从而保证了cache是无阻塞的。分支指令的预测方式与RSC一样，跨越未解决的分支指令可以有条件地分派后续指令但不执行。

601保留了RSC的微码引擎（即序列器），但是其作用受到了限制。601中的序列器包括18 KB的微码控制存储器、8个寄存器和32个字的RAM。设计者在601中加入了很多多处理机的功能。数据cache实现了MESI协议，标识也是双端口的（每周期两次读）并支持监听（基于一个区）。cache同时也监听写回队列，从而使填充具有一定的优先级而不会导致一致性问题。然而，当来自另一个处理器的监听命中某项时，该项就专门写回修改过的扇区，该扇区的写回就具有了访存的优先权。

关于PowerPC 601的更多信息，参见Becker等人[1993]、Moore[1993]、Potter等人[1994]以及Weiss和Smith[1994]

7.3.10.3 PowerPC 603/1994

603是PowerPC的低功耗实现版本，由Brad Burgess、Russ Reininger以及Jim Kahle设计。603面向小型单处理器系统，如膝上型电脑。603具有分离的指令和数据cache，大小均为8 KB并包含5个独立的执行部件：分支、整数、系统、访存以及浮点。系统部件执行条件码逻辑操作，以及写或者读系统特殊寄存器的指令

603每个周期从指令cache取出两条指令，同时送往分支部件和一个6表项的指令队列。分支部件能够将不改变分支部件寄存器的分支从指令队列中删除掉，否则，分支将直接穿过系统部件。译码器检查指令队列底部的两条指令，每个周期最多发射/分派两条指令。指令分派包括读寄存器操作数并为目标寄存器指定一个重命名的寄存器。603中总共有5个整数重命名寄存器和4个浮点重命名寄存器。

每个执行部件都有一个保留站，因此指令即使存在数据相关也能分派。指令分派还要求在完成缓冲中为每条分派的指令分配一项，完成缓冲共有5项。指令在完成缓冲中提交的速度是每周期两条。指令提交包括更新寄存器文件，传递所指定的重命名寄存器的值。所有的指令在完成缓冲中按照程序的原始顺序提交，因此所有的异常都能精确定位。

默认的分支预测方法是基于偏移的符号来进行预测，但是编译器能够使用分支指令码中的一位来改变预测方向。系统支持跨越一个条件分支的推测执行，推测执行的路径上可以有无条件分支和计数寄存器递减分支，但是遇到条件分支必须停下来等待结果。分支预测失败的处理是清除所有预测的指令以及完成缓冲中分支指令之后写入的数据。

访存部件支持未对齐操作数执行的多次访存，以及对读多字/写多字和字符串指令的顺序多次访存。读操作实现了流水化并包含两个周期的延迟，写操作则没有流水化。可以采用一种特殊的内部格式支持未规格化的浮点数据，或者使用清零模式。

603有4种电源管理模式：nap、doze、sleep和dynamic。dynamic模式可以在不影响性能的前提下降低空闲执行部件的功耗。

更多的信息参见Burgess等人[1994a]，Burgess等人[1994b]以及ACM中关于“The Making of the PowerPC”的特别专题。关于603的设计权衡问题，Poursepanj等人[1994]提供了一篇其模拟研究的优秀论文。

由Brad Burgess和Robert Golla完成的603e是603之后的一个实现，它将片上cache的大小翻了一番，并使系统部件具有了执行整数加法和比较的能力（因此可以认为603e具有一个功能有限的辅助整数部件）

7.3.10.4 PowerPC 604/1994

PowerPC 604（如图7.14所示）看起来很像Mike Johnson书中的标准处理器设计方案。604包含6个功能部件（每个功能部件都包含一个两项的保留站）以及一个16项的再定序缓冲（即完成

缓冲）。取指部件每周期最多能取出4条指令并放入一个4项的译码缓冲中。然后这些指令被放入一个4项的分派缓冲、读出操作数并执行寄存器重命名。每个周期最多能有4条指令从该缓冲分派到6个功能部件：一个分支部件、两个整数部件、一个整数乘法部件、一个访存部件以及一个浮点部件。每个整数部件都能从任意一个保留站项中发射指令（即乱序执行），而其他部件的保留站则按序发射给定的指令类型，但部件之间不按时序。604支持两个级别的推断执行。再定序缓冲每个周期最多能提交4条指令

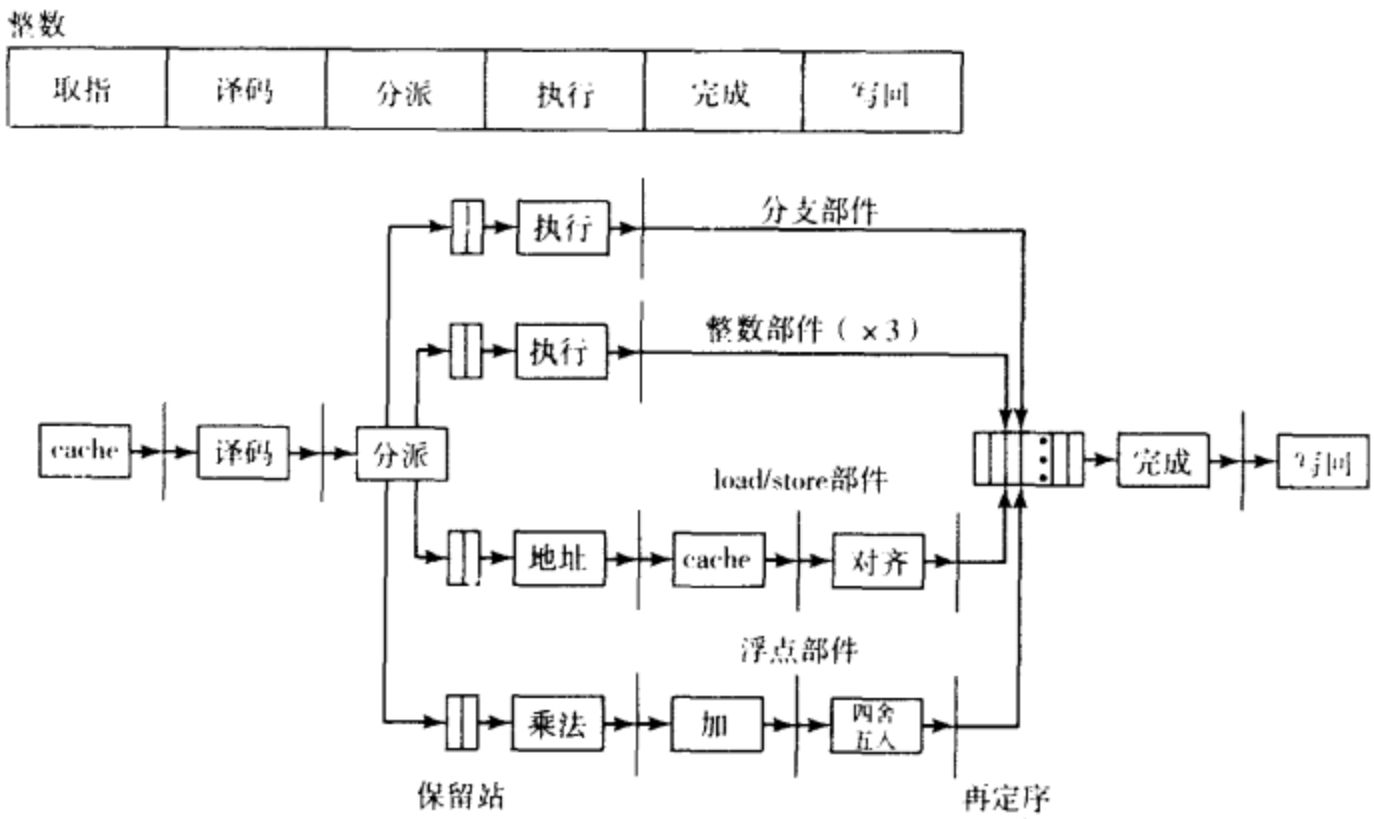


图7.14　IBM/Motorola PowerPC 604流水线

整数寄存器包含一个12项的重命名缓冲，浮点寄存器包含一个8项的重命名缓冲，条件码也有一个8项的重命名缓冲。写存储器操作不能推测执行，但是604也不允许条件寄存器上的逻辑操作以及使用进位的整数算术操作推测执行。

604的分支预测机制包括一个512项的分支历史表（BHT），每项有一个两位预测器；还包括一个64项的全相联分支目标地址cache（BTAC）。译码部件识别并处理无条件分支、在BHT中命中而在BTAC中不命中的分支。对于CTR寄存器的分支则存在特殊的分支预测，这些分支通常用于实现最内层的循环。分派逻辑遇到分支指令时，将停止指令的多发射，所以604一个周期只处理一条分支指令。

存储器接口包括一个4项的读队列，一个6项的写队列以及一个128项的两路组相联TLB。

Peter Song是604的主要设计师。关于604更多的信息参见Song等人[1994]的相关描述。Denman等人[1996]讨论了后续的芯片604e，它是一个低功耗的管脚兼容版本。604e将cache大小翻了一番，并分别为条件寄存器操作和分支提供了单独的执行部件。这两个与分支相关的部件各有一个两项的保留站，但是指令分派仍限制为每周期一条。

7.3.10.5　PowerPC 620/1995

620是PowerPC体系结构中第一个64位的版本，其设计者包括Don Waldecker、Chin Ching Kau以及Dave Levitan。620的组织结构与604比较类似并使用了相同的6个功能部件。然而，620在许多方面更出色一些。例如，620去掉了指令流水线中的译码段并由指令cache填充时的预译码取代。

整数和浮点寄存器的重命名缓冲都是8个，而条件寄存器则有16个重命名缓冲。浮点部件和整数部件的保留站都是两项，而分支部件的保留站有4项，访存部件的保留站有3项（浮点部件的发射是顺序的，而访存部件的发射是乱序的）。为了支持跨越4个分支的推测执行，620的分支部件采用了一个256项的分支目标cache、一个2048项的分支历史表、一个8项的返回地址栈以及一个历史寄存器文件。此外，为了加强访存操作的处理，620采用了一个5项的读队列、一个5项的写操作队列（存放完成地址转换但是还没有接收到数据的写指令），以及一个6项的写操作完成队列（存放完成地址转换并接收到数据的写指令）。

620的实现则进行了一些简化：需要两个源操作数的整数指令只能从指令队列的底部两项中进行发射，再定序缓冲的项必须成对分配和释放。

据报道，PowerPC 620存在着某些设计缺陷，从而导致了多处理机操作具有一定限制，因此极少有系统采用620芯片。关于620设计的更多信息，参见Thompson、Ryan［1994］和Levitan等人［1995］。在关于620性能的研究中，Diep等人［1995］分析了不同版本的620设计并得出了相应的结论，620的单浮点部件和每周期不能发射一条以上的load指令和store指令是主要的瓶颈，应该在以后的实现中加以改进。

7.3.10.6 PowerPC 750（G3 Arthur）/1997

750的主要设计师是Brad Burgess。750是一个低功耗的芯片，其流水线包含4级，缓冲也比604少，但是750的性能却比604高。

750有6个功能部件：两个整数部件、一个系统寄存器部件、一个访存部件、一个浮点部件以及一个分支部件。与604不同的是，除了分支部件和访存部件，其他功能部件的保留站只有一项。750访存部件的保留站有两项。指令在写入L1指令cache之前预译码为36位的格式。取指部件每周期能取出4条指令，最多能分派两条指令，这两条指令来自有6表项指令缓冲的底部两项。分支指令一旦识别出就进行处理，当预测为跳转时，可以通过从目标地址取指来隐藏分支。推测执行可以跨越一个未解析的分支，而推测的取指能够跨越两个未解析的分支。一种节省完成缓冲空间的方法是在分派以前“压缩”指令缓冲中的nop和没有发生转移的分支指令。这两种类型的指令不会分配完成缓冲中的表项，但是未解析的、预测为不跳转的分支会被保留在分支部件中，直到被解析为止，从而保证预测失败时能够恢复。

750采用一个32 KB、8路组相联的指令cache、一个32 KB 8路组相联的数据cache、一个64项4路组相联的BTIC以及一个512项的BHT。此外还有一个两路组相联的二级cache控制器和二级标识，二级cache支持256 KB、512 KB、1 MB的片外SRAM。

Burgess将750设计的特点概括为“适度的发射带宽、短流水线、大容量cache以及高效的分支部件，并将重点放在解决分支而不是预测分支上”。750的圆片面积和功耗（5 W）要比其他具有更宽的发射带宽和更深的流水线的设计小得多，而性能却可与之相提并论，甚至还要高一些。

更多的细节参见Kennedy等人［1997］（740版本不包含L2标识和控制器）。

7.3.11 SPARC第8版

7.3.11.1 概述

SPARC体系结构是一个RISC设计，它起源于David Patterson在加州大学伯克利分校所做的研究工作。这些早期工作区别于其他设计的特征在于，在过程调用时使用寄存器窗口来减少对存储器的访问，并且这一特征被SPARC的主要设计师Robert Garner用到了SPARC中。1986年，第一个SPARC处理器实现了SPARC体系结构的第7版，这是一个流水线的版本，它定义了一组整

数指令，每一条指令都能在一个周期内执行完毕（除了乘法和除法指令），还实现了延迟的分支。SPARC 体系结构手册明确地声明："一个未发生转移的分支所需要的执行时间不比一个发生转移的分支少"。浮点队列（FPQ）也明确定义于体系结构手册中，这是一个再定序缓冲并且能够直接由异常处理软件加以访问。

虽然第7版的体系结构手册中就建议采用子程序实现整数乘法和除法，1990年的第8版体系结构仍实现了整数乘法。SuperSPARC 和 HyperSPARC 处理器实现了 SPARC 第 8 版。从这些实现中积累的经验和对未来 64 位超标量设计的重点考虑，导致了 SPARC 体系结构第 9 版的出现。

7.3.11.2　TI（德州仪器）的 SuperSPARC（Viking）/1992

SuperSPARC由Sun公司的Greg Blanck设计并由TI公司的Steve Krueger监督实现。SuperSPARC每个周期最多能按程序的顺序发射3条指令，围绕一个处理分支的控制部件、一个浮点部件以及一个独特的、包含3个级联ALU的整数部件进行设计。这些级联的ALU允许同时发射相关的整数指令，因此处理器能够很好地执行未进行重编译的 SPARC 二进制代码。

SuperSPARC每个周期取出4条对齐的指令，译码器需要一个半周期并在最后半个周期（TI称为分组段）发射3条指令。尽管有一些指令被定义为单发射的（如寄存器窗口的保存和恢复、整数乘法），分组逻辑最多将两条整数指令、一条访存指令或者一条浮点指令组合到一个指令组中。实际的发射规则很复杂，需要考虑资源限制，如对整数寄存器写端口数目的限制。一个指令组也在任何控制转移指令之后结束。一旦发射出去，指令组则在流水线中锁步前进，但有一个例外，就是浮点指令将被放入一个4项的指令缓冲中，等待浮点部件空闲，然后独立执行并使用SPARC的浮点队列（FPQ）来处理异常。上面已经提到，因为有级联ALU，一条相关的指令（整数、存储或者分支）可以与一条产生操作数的整数指令包含在一个指令组里。但这对于load指令不成立，一条与load指令相关的指令必须放在下一个指令组。

SuperSPARC有两个4指令的取指队列。一个队列用于沿着顺序路径取指，另一个队列用于遇到分支指令时，预取分支目标处的指令。由于一个指令组在控制转移指令之后结束，分支延迟槽指令将包含在顺序路径上的下一个指令组，而这个指令组将被推测发射（所以SuperSPARC实际上是预测为不跳转）。如果分支确实是发生跳转，除了分支延迟槽中的指令，推测指令组中的指令将被清除，而预取的分支目标处的指令则在下一指令组中发射。如果分支不发生跳转，就没有分支开销，否则在分支指令与分支目标指令组之间存在一个单发射周期，分支延迟槽的指令在这个周期发射并执行。

访存指令由整数部件执行并提供了载入旁路。SuperSPARC同样使用了一个8项的写缓冲。二级 cache 控制器提供块复制和块清零的操作。

SuperSPARC的总体描述参见Blanck和Krueger[1992]。这个芯片有点令人失望，问题主要出在cache的设计上，而不在于核心的设计。

7.3.11.3　Ross HyperSPARC（Pinnacle）/1993

HyperSPARC在SuperSPARC之后一到两年推向市场，它在多发射方面不如后者高效。然而，就像 Alpha 和 POWER 的对比一样，HyperSPARC 与 SuperSPARC 相比，其在性能上的胜出再一次证明了一个发射带宽小但时钟快的设计不比一个发射带宽大的设计差。HyperSPARC设计还强调了对 Sun 工作站 Sbus 上的双处理器方式的支持。HyperSPARC 规范由 Raju Vegesna 完成，第一个仿真器则由 Jim Monaco 设计。

HyperSPARC包含4个执行部件：整数、浮点、访存以及分支部件。取指部件每周期能从片上8 KB指令cache中取出两条指令并放入译码器。在访问片上指令cache的同时也访问片外cache，因此指令 cache 失效只有一个周期的延迟。

译码器同时能译码两条指令，且只有当先前取得的指令发射出去之后才接收更多的指令。译码器还负责读取寄存器操作数。译码器能发射三种特殊形式的相关指令：sethi指令和相关的or指令、sethi指令和相关的load/store指令、以及一条设置条件码的整数ALU指令和相关的分支指令。两条浮点指令也能在同一个周期中分派到一个4项的浮点指令队列中，条件是该队列还有空间。有几种可能的情况要求停顿，其中一些情况是由于寄存器文件端口的竞争，因为整数寄存器文件只有两个读端口。此外还有53条要求单发射的指令，包括调用指令、保存指令、恢复指令、乘法除法指令以及浮点比较指令。

整数部件总共有136个寄存器并提供8个重叠的指令窗口，每个指令窗口包含24个寄存器以及8个全局寄存器。整数流水线以及访存和分支流水线除了通常的取指段和译码段以外，还包含另外4段：执行、cache读、cache写以及寄存器更新。整数部件不使用与cache相关的两段，但是这两段也包含在流水线中，因此所有非浮点流水线的长度都是相同的。整数乘法和除法延迟特别长，分别需要18个和37个周期。乘除法将使后面的指令停止发射，直到执行完毕。

浮点部件之前有一个4项的前队列（prequeue），之后有一个3项的后队列（postqueue），它们实现了乱序执行的SPARC FPQ技术。前队列允许译码器尽可能快地分派浮点指令，从而更早地发现分支。一旦前队列满，指令的分派就将停止。

前队列中的浮点指令按顺序进行译码并发射到后队列，后队列中的每一项对应于浮点流水线中的一个执行段（执行-1，执行-2，四舍五入）。浮点的load指令和一条相关的浮点指令可以在同一周期发射，但在读出的数据被传递到执行-1段之前，相关的指令要在前队列中等待两个周期。当一条浮点指令和一条相关的浮点写指令在译码器中组成指令对时，在操作结果进入四舍五入段之前，写指令至少要在译码器中等待两个周期，然后在下一个周期传递到访存部件。

HyperSPARC芯片组包括一个外部cache控制器，用于管理256 KB直接映像的cache，每行32字节并采用写失效的一致性协议。HyperSPARC还提供了一个64项的全相联TLB以及cache读写缓存。cache控制芯片实现了一条块复制指令和一条块填充指令。

Vegesna［1992］提供了一篇关于HyperSPARC的入门级文章。

7.3.11.4 Metaflow Lightning/Thunder/1991/1994

Lightning和Thunder由Bruce Lightner和Val Popescu进行设计，它们使用了一个集中的保留站，设计者称之为DRIS，即延迟调度寄存器重命名指令架（deferred-scheduling register-renaming instruction shelf）。Thunder在1994年的Hot Chips上进行了描述，它是4芯片Lightning的3芯片更新版本，而Lightning设计于1991年。Thunder每个周期最多发射4条指令到8个执行部件：3个整数部件、两个浮点部件、两个访存部件以及1个分支部件。分支预测是动态的并且包括返回地址预测。关于Lightning的文章参见Lightner和Hill［1991］以及Popescu等人［1991］。

7.3.12 SPARC第9版本

7.3.12.1 HAL SPARC64（R1）/1995

HAL策划了SPARC体系结构第9版的设计，而SPARC64是其中的第一个实现。R1是最初的单处理机版本，而R2是带有目录cache一致性协议的多处理机版本。HAL的设计还包括一个独特的三层存储器管理机制（区域、视图以及页），以减少64位地址空间所需要的映射表大小。SPARC64的设计者是Hisashige Ando、Winfried Wilcke以及Mike Shebanow。

R1的寄存器文件设计得很出色。R1共有116个整数寄存器，其中有78个可以在任何时候组成4个SPARC寄存器窗口，剩下的38个用于寄存器重命名。R1还有112个浮点寄存器，其中32个

寄存器用于单精度计算，32个寄存器用于双精度计算，剩下的48个浮点寄存器用于寄存器重命名。整数寄存器文件有10个读端口和4个写端口，而浮点寄存器文件有6个读端口和3个写端口。

R1有4个64 KB的4路组相联虚地址cache（两个用做指令cache，两个用做数据cache，允许每个周期执行两个无冲突的读写操作）。R1还提供了一个实地址表用于数据cache的反映射，并通过8个读缓冲提供对数据cache的无阻塞访问（支持读合并）。为了加快取指速度，R1采用了一个0级4 KB的直接映像指令cache。在这个cache中，SPARC指令被部分译码为一种内部格式，包括已部分计算的分支目标地址。同时0级指令cache中还为每一条指令提供了一个两位的分支历史记录信息。CPU芯片推测取出片外指令cache块并填充两个预取缓冲。

R1每周期最多分派4条指令到4个保留站，具体情况取决于指令的类型。4个整数部件（两个整数ALU，一个整数乘法部件以及一个整数除法部件）包含一个8项的保留站；两个地址生成部件包含一个8项的保留站；两个浮点部件（一个浮点乘加部件和一个浮点除法器）包含一个8项的保留站，两个访存部件包含一个12项的保留站。指令在分派时进行寄存器重命名。访存指令同时分派到地址生成部件和访存部件的保留站。有效地址从地址生成部件送到一个与访存保留站相关联的cache。

某些设计每周期分派、发射、完成以及提交的指令数目相同，而SPARC64则有很大变化。在一个给定的周期内，最多可以分派4条指令，最多可以发射7条指令，最多可以执行10条指令，最多可以完成9条指令，最多可以提交（commit）8条指令，最多可以退出（retire）4条指令。任何时候最多有64条指令处于活跃状态，硬件给每条指令指派一个6位的序列号，在一个称为A-ring的结构中记录活跃的指令。A-ring按照检查点恢复的方式工作，提供分支预测失败时的恢复，总共可以容纳16个检查点（例如，分支或修改无须重命名的控制寄存器）。R1有4个指针用于改写A-ring：ISN，上一次发射的序列号；CSN，上一次提交的序列号；RRP，资源恢复指针；NCSNP，未提交的存储器序列号，它们可以对读写操作进行调度。每条指令都附加了一个指向上一个检查点的指针。对于在检查点处没有对齐的自陷指令，处理器每个周期可以撤销4条指令。

整数指令流水线包含7级：取指、分派、执行、写回、完成、提交以及退出。流水线没有译码段，因为译码主要在指令载入第0级指令cache时完成。完成段检查错误和异常，提交段顺序执行体系结构状态的更新，退出段回收所有的资源。访存指令需要额外的两个执行段。SPARC64使用第9版中新的自陷定义，可以重命名自陷等级，处理器可以推测进入分派时检测到的自陷。

更多的细节参见Chen等人[1995]，Patkar等人[1995]，Simone等人[1995]，Wilcke[1995]以及Williams等人[1995]。Simone的文章详细讨论了几种设计中的折中方案，包括发射条件码修改指令的特殊优先级逻辑。

Fujitsu购买HAL后终止了后续的设计。Mike Shebanow是SPARC64的负责人，并曾经策划过一个使用trace cache和值预测机制的8发射设计。Shebanow在1999年的微处理器论坛（Microprocessor Forum）以及1999年在Stanford大学召开的研讨会上描述了SPARC 64 V的设计，后者的录像可以在微软多所大学的研究实验室的网站（http://murl.microsoft.com/）上获得。

7.3.12.2　Sun UltraSPARC-I/1995

UltraSPARC-I（如图7.15所示）是64位的SPARC体系结构第9版的实现，由Les Kohn、Marc Tremblay、Guillermo Maturana以及Robert Yung设计。UltraSPARC每个周期向9个功能部件（两个整数部件，一个访存部件，一个分支部件，一个浮点加法部件，一个浮点乘法部件，一个浮点除法及开方部件，一个图形加法部件以及一个图形乘法部件）发射4条指令。UltraSPARC引入了30条左右新的图形指令（虚拟指令集，VIS），还提供了块读写指令和寄存器窗口。

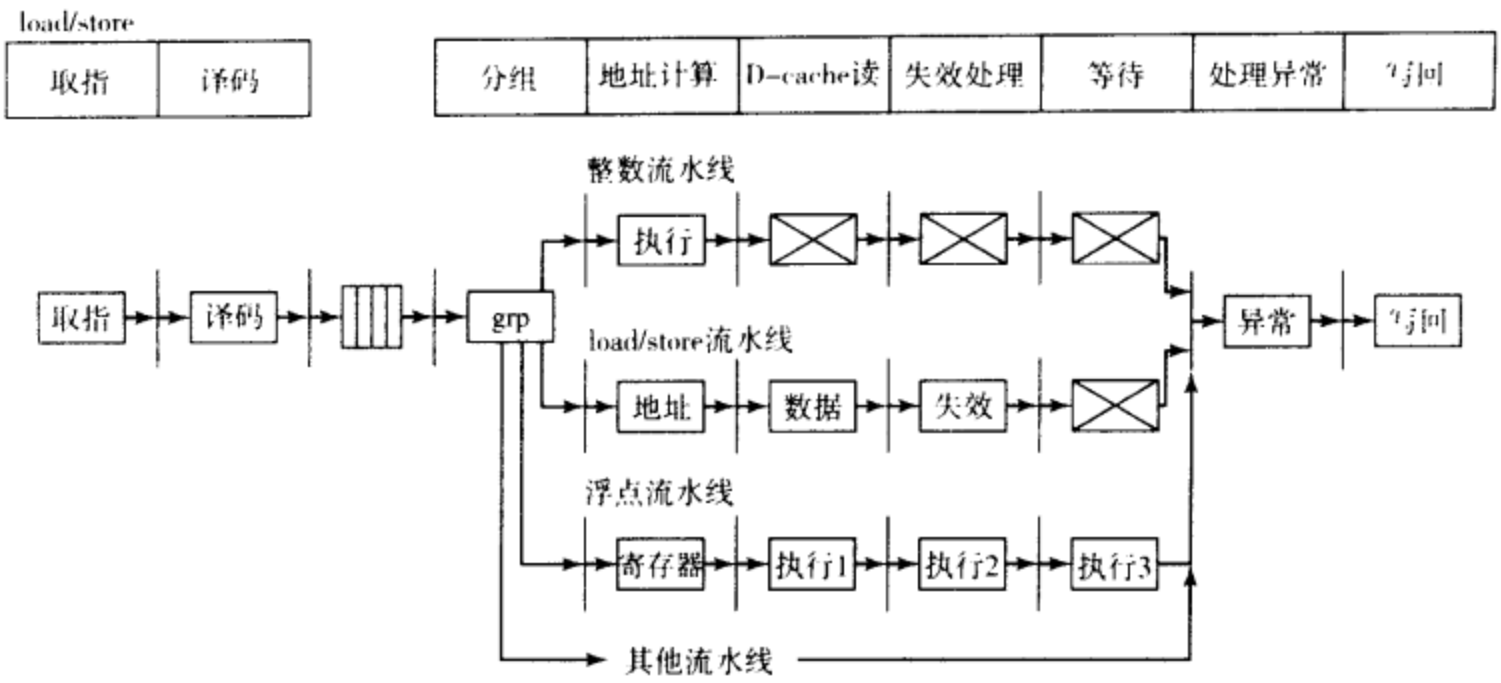


图 7.15 Sun UltraSPARC-I 流水线

UltraSPARC 与许多同时期的设计不同，它不是一个乱序执行的设计。其设计团队广泛模拟了多种设计方案，包括各种乱序执行方式，然后宣称乱序执行方式会带来 20% 的时钟周期开销，并可能使上市时间延长 3~6 个月；相反，可以通过采用推测执行、无故障 load 等措施来获取高性能，并且可以更好地发挥 UltraSPARC 编译器的作用。

因此UltraSPARC每个周期最多从一个12项的指令缓冲中按顺序发射4条指令，并支持精确异常，但是UltraSPARC省略了SPARC传统的浮点队列，每个功能部件的流水线都是4级。UltraSPARC对运行时间较长的指令（如除法，开方）中的异常进行预测。它使用类似Johnson在其著作中建议的分支预测机制来支持指令的推测发射。UltraSPARC的指令cache每块包括8条指令，每一对指令有一个两位的分支历史信息，每4条指令有一个12位的、指向下一个cache块的域。历史信息和下一个cache块标识用于填充指令缓冲，并且使用历史信息从而提高了预测的准确率，消除了映射到传统 BHT 同一项的多个分支之间的冲突。分支的解析是在整数和浮点流水线的第一个执行段之后进行的。

UltraSPARC在存储器接口设计方面相对要更出色一些。指令cache使用了一种组预测方法，能提供直接映像cache的访问速度，同时实现了两路组相联cache的低冲突概率。UltraSPARC还有一个9项的读缓冲和8项的写缓冲，以支持载入旁路和写合并。

关于UltraSPARC I处理器的描述，参见Wayner[1994]，Greenley等人[1995]，Lev等人[1995]以及Tremblay和O'Connor[1996]。Tremblay等人[1995]讨论了在设计该处理器及其存储系统时所做的折中。Goldman和Tirumalai[1996]讨论了UltraSPARC II，UltraSPARC II则对存储系统进行了改进并引入了预取等策略。

7.3.12.3 UltraSPARC III

UltraSPARC III 基本上是一个全新的设计，主要设计师是 Gary Lauterback。关于 UltraSPARC III 的更多信息，参见 Horel 和 Lauterbach [1999] 以及 Lauterbach [1999]。

7.3.13 其他超标量处理器

为了讨论的完整性，应该描述一下本章中没有涉及的设计方案。这些设计包括Intergraph *Clipper* C400（1991），Transputer T9000（1992），Hitachi Super-H SH4（1996）以及IDT C6 Winchip（1997）。

参考文献

Processor manuals are available from the individual manufacturers and are not included in the references. With the growth of the World Wide Web, much information about current processors can be obtained by an electronic visit to the manufacturer's website.

Acosta, R., J. Kjelstrup, and H. Torng: "An instruction issuing approach to enhancing performance in multiple functional unit processors," *IEEE Trans. on Computers*, C-35, 9, September 1986, pp. 815-828.

Allen, F.: "The history of language processor technology in IBM," *IBM Journal of Research and Development*, 25, 5, September 1981, pp. 535-548.

Alpert, D., and D. Avnon: "Architecture of the Pentium microprocessor," *IEEE Micro*, 11, 3, June 1993, pp. 11-21.

Asprey, T., G. Averill, E. DeLano, R. Mason, B. Weiner, and J. Yetter: "Performance features of the PA7100 microprocessor," *IEEE Micro*, 13, 3, June 1993, pp. 22-35.

Bannon, P., and J. Keller: "The internal architecture of Alpha 21164 microprocessor," *Proc. COMPCON*, San Francisco, March 1995, pp. 79-87.

Bannon, P., and Y. Saito: "The Alpha 21164PC microprocessor," *Proc. COMPCON*, San Jose, CA, February 1997, pp. 20-27.

Barreh, J., S. Dhawan, T. Hicks, and D. Shippy: "The POWER2 processor," *Proc. COMPCON*, San Francisco, Feb.-March 1994, pp. 389-398.

Bashe, C., L. Johnson, J. Palmer, and E. Pugh, *IBM's Early Computers*. Cambridge, MA: M.I.T. Press, 1986.

Becker, M., M. Allen, C. Moore, J. Muhich, and D. Tuttle: "The PowerPC 601 microprocessor," *IEEE Micro*, 13, 5, October 1993, pp. 54-67.

Benschneider, B., et al.: "A 300-MHz 64-b quad issue CMOS RISC microprocessor," *IEEE Journal of Solid-State Circuits*, 30, 11, November 1995, pp. 1203-1214. [21164]

Bishop, J., M. Campion, T. Jeremiah, S. Mercier, E. Mohring, K. Pfarr, B. Rudolph, G. Still, and T. White: "PowerPC AS A10 64-bit RISC microprocessor," *IBM Journal of Research and Development*, 40, 4, July 1996, pp. 495-505.

Blanchard, T., and P. Tobin: "The PA 7300LC microprocessor: A highly integrated system on a chip," *Hewlett-Packard Journal*, 48, 3, June 1997.

Blanck, G., and S. Krueger: "The SuperSPARC microprocessor," *Proc. COMPCON*, San Francisco, February 1992, pp. 136-141.

Borkenhagen, J., R. Eickemeyer, R. Kalla, and S. Kunkel: "A multithreaded PowerPC processor for commercial servers," *IBM Journal of Research and Development*, 44, 6, November 2000, pp. 885-898. [SStar/RS64-IV]

Borkenhagen, J., G. Handlogten, J. Irish, and S. Levenstein: "AS/400 64-bit PowerPC-compatible processor implementation," *Proc. ICCD*, Cambridge, MA, October 1994, pp. 192-196.

Bose, P.: "Optimal code generation for expressions on super scalar machines," *Proc. AFIPS Fall Joint Computer Conf.*, Dallas, November 1986, pp. 372-379.

Bowhill, W., et al.: "Circuit implementation of a 300-MHz 64-bit second-generation CMOS Alpha CPU," *Digital Technical Journal*, 7, 1, 1995, pp. 100-118.

Buchholz, W., *Planning a Computer System*. New York: McGraw-Hill, 1962. [IBM Stretch]

Burgess, B., M. Alexander, Y.-W. Ho, S. Plummer Litch, S. Mallick, D. Ogden, S.-H. Park, and J. Slaton: "The PowerPC 603 microprocessor: A high performance, low power, superscalar RISC microprocessor," *Proc. COMPCON*, San Francisco, Feb.-March 1994, pp. 300-306. [cited as 1994a]

Burgess, B., N. Ullah, P. Van Overen, and D. Ogden: "The PowerPC 603 microprocessor," *Communications of the ACM*, 37, 6, June 1994, pp. 34-42. [cited as 1994b]

Burkhardt, B.: "Delivering next-generation performance on today's installed computer base," *Proc. COMPCON*, San Francisco, Feb.-March 1994, pp. 11-16. [Cyrix 6x86]

Chan, K., et al.: "Design of the HP PA 7200 CPU," *Hewlett-Packard Journal*, 47, 1, February 1996.

Chen, C., Y. Lu, and A. Wong: "Microarchitecture of HaL's cache subsystem," *Proc. COMPCON*, San Francisco, March 1995, pp. 267-271. [SPARC64]

Christie, D.: "Developing the AMD-K5 architecture," *IEEE Micro*, 16, 2, April 1996, pp. 16-26.

Circello, J., and F. Goodrich: "The Motorola 68060 microprocessor," *Proc. COMPCON*, San Francisco, February 1993, pp. 73-78.

Circello, J., et al.: "The superscalar architecture of the MC68060," *IEEE Micro*, 15, 2, April 1995, pp. 10-21.

Cocke, J.: "The search for performance in scientific processors," *Communications of the ACM*, 31, 3, March 1988, pp. 250-253.

Cohler, E., and J. Storer: "Functionally parallel architecture for array processors," *IEEE Computer*, 14, 9, Sept. 1981, pp. 28-36. [MAP 200]

DeLano, E., W. Walker, J. Yetter, and M. Forsyth: "A high speed superscalar PA-RISC processor," *Proc. COMPCON*, San Francisco, February 1992, pp. 116-121. [PA 7100]

Denman, M., P. Anderson, and M. Snyder: "Design of the PowerPC 604e microprocessor," *Proc. COMPCON*, Santa Clara, CA, February 1996, pp. 126-131.

Diefendorff, K.: "History of the PowerPC architecture," *Communications of the ACM*, 37, 6, June 1994, pp. 28-33.

Diefendorff, K.: "K7 challenges Intel," *Microprocessor Report*, 12, 14, October 26, 1998, pp. 1, 6-11.

Diefendorff, K., and M. Allen: "The Motorola 88110 superscalar RISC microprocessor," *Proc. COMPCON*, San Francisco, February 1992, pp. 157-162. [cited as 1992a]

Diefendorff, K., and M. Allen: "Organization of the Motorola 88110 superscalar RISC microprocessor," *IEEE Micro*, 12, 2, April 1992, pp. 40-63. [cited as 1992b]

Diefendorff, K., R. Oehler, and R. Hochsprung: "Evolution of the PowerPC architecture," *IEEE Micro*, 14, 2, April 1994, pp. 34-49.

Diefendorff, K., and E. Silha: "The PowerPC user instruction set architecture," *IEEE Micro*, 14, 5, December 1994, pp. 30-41.

Diep, T., C. Nelson, and J.P. Shen: "Performance evaluation of the PowerPC microarchitecture," *Proc. ISCA*, Santa Margherita Ligure, Italy, May 1995, pp. 163-174.

Ditzel, D., and H. McLellan: "Branch folding in the CRISP microprocessor: Reducing branch delay to zero," *Proc. ISCA*, Philadelphia, June 1987, pp. 2-9.

Ditzel, D., H. McLellan, and A. Berenbaum: "The hardware architecture of the CRISP microprocessor," *Proc. ISCA*, Philadelphia, June 1987, pp. 309-319.

Eden, M., and M. Kagan: "The Pentium processor with MMX technology," *Proc. COMPCON*, San Jose, CA, February 1997, pp. 260-262.

Edmondson, J., et al.: "Internal organization of the Alpha 21164, a 300-MHz 64-bit quad-issue CMOS RISC microprocessor," *Digital Technical Journal*, 7, 1, 1995, pp. 119-135. [cited as 1995a]

Edmondson, J., P. Rubinfeld, R. Preston, and V. Rajagopalan: "Superscalar instruction execution in the 21164 Alpha microprocessor," *IEEE Micro*, 15, 2, April 1995, pp 33-43. [cited as 1995b]

Fisher, J.: "Very long instruction word architectures and the ELI-512," *Proc. ISCA*, Stockholm, Sweden, June 1983, pp. 140-150.

Flynn, M.: "Very high-speed computing systems," *Proceedings of the IEEE*, 54, 12, December 1966, pp. 1901-1909.

Gaddis, N., and J. Lotz: "A 64-b quad-issue CMOS RISC microprocessor," *IEEE Journal of Solid-State Circuits*, 31, 11, November 1996, pp. 1697-1702. [PA 8000]

Gieseke, B., et al.: "A 600MHz superscalar RISC microprocessor with out-of-order execution," *Proc. IEEE International Solid-State Circuits Conference*, February 1997. pp. 176-177. [21264]

Goldman, G., and P. Tirumalai: "UltraSPARC-II: The advancement of ultracomputing," *Proc. COMPCON*, Santa Clara, CA, February 1996, pp. 417-423.

Greenley, D., et al.: "UltraSPARC: The next generation superscalar 64-bit SPARC," *Proc. COMPCON*, San Francisco, March 1995, pp. 442-451.

Gronowski, P., et al.: "A 433-MHz 64-b quad-issue RISC microprocessor," *IEEE Journal of Solid-State Circuits*, 31, 11, November 1996, pp. 1687-1696. [21164A]

Gwennap, L.: "Cyrix describes Pentium competitor," *Microprocessor Report*, 7, 14, October 25, 1993, pp. 1-6. [M1/6x86]

Gwennap, L.: "AMD's K5 designed to outrun Pentium," *Microprocessor Report*, 8, 14, October 14, 1994, pp. 1-7.

Gwennap, L.: "Intel's P6 uses decoupled superscalar design," *Microprocessor Report*, 9, 2, February 16, 1995, pp. 9-15.

Halfhill, T.: "AMD vs. Superman," *Byte*, 19, 11, November 1994, pp. 95-104. [AMD K5]

Halfhill, T.: "T5: Brute force," *Byte*, 19, 11, November 1994, pp. 123-128. [MIPS R10000]

Halfhill, T.: "Intel's P6," *Byte*, 20, 4, April 1995, pp. 42-58. [Pentium Pro]

Halfhill, T.: "AMD K6 takes on Intel P6," *Byte*, 21, 1, January 1996, pp. 67-72.

Halfhill, T.: "PowerPC speed demon," *Byte*, 21, 12, December 1996, pp. 88NA1-88NA8.

Hall, C., and K. O'Brien: "Performance characteristics of architectural features of the IBM RISC System/6000," *Proc. ASPLOS-IV*, Santa Clara, CA, April 1991, pp. 303-309.

Hardy, N.: LLNL interview, available online at http://www.llnl.gov/vcm/interviews/norman_hardy_1.html

Hewlett-Packard: "PA-RISC 8x00 family of microprocessors with focus on PA-8700," Hewlett Packard Corporation, Technical White Paper, April 2000.

Hinton, G.: "80960 - Next generation," *Proc. COMPCON*, San Francisco, March 1989, pp. 13-17.

Hinton, G., D. Sager, M. Upton, D. Boggs, D. Carmean, A. Kyker, and P. Roussel: "The microarchitecture of the Pentium 4 processor," *Intel Technology Journal*, Quarter 1, 2001.

Hollenbeck, D., A. Undy, L. Johnson, D. Weiss, P. Tobin, and R. Carlson: "PA7300LC integrates cache for cost/performance," *Proc. COMPCON*, Santa Clara, February 1996, pp. 167-174.

Horel, T., and G. Lauterbach: "UltraSPARC-III: Designing third generation 64-bit performance," *IEEE Micro*, 19, 3, May-June, 1999, pp. 73-84.

Horst, R., R. Harris, and R. Jardine: "Multiple instruction issue in the NonStop Cyclone processor," *Proc. ISCA*, Seattle, May 1990, pp. 216-226.

Hsu, P.: "Designing the TFP microprocessor," *IEEE Micro*, 14, 2, April 1994, pp. 23-33. [MIPS R8000]

Hunt, J.: "Advanced performance features of the 64-bit PA-8000," *Proc. COMPCON*, San Francisco, March 1995, pp. 123-128.

IBM: *IBM RISC System/6000 Technology*. Austin: IBM Corporation, 1990.

Johnson, L., and S. Undy: "Functional design of the PA 7300LC," *Hewlett-Packard Journal*, 48, 3, June 1997.

Jouppi, N., and D. Wall: "Available instruction-level parallelism for superscalar and superpipelined machines," *Proc. ASPLOS-III*, Boston, April 1989, pp. 272-282.

Kennedy, A., et al.: "A G3 PowerPC superscalar low-power microprocessor," *Proc. COMPCON*, San Jose, CA, February 1997, pp. 315-324. [PPC 740/750, but one diagram lists this chip as the 613]

Kessler, R.: "The Alpha 21264 microprocessor," *IEEE Micro*, 19, 2, March-April 1999, pp. 24-36.

Kessler, R., E. McLellan, and D. Webb: "The Alpha 21264 microprocessor architecture," *Proc. ICCD*, Austin, TX, October 1998, pp. 90-95.

Knebel, P., B. Arnold, M. Bass, W. Kever, J. Lamb, R. Lee, P. Perez, S. Undy, and W. Walker: "HP's PA7100LC: A low-cost superscalar PA-RISC processor," *Proc. COMPCON*, San Francisco, February 1993, pp. 441-447.

Kurpanek, G., K. Chan, J. Zheng, E. DeLano, and W. Bryg: "PA7200: A PA-RISC processor with integrated high performance MP bus interface," *Proc. COMPCON*, San Francisco, Feb.-March 1994, pp. 375-382.

Laird, M.: "A comparison of three current superscalar designs," *ACM Computer Architecture News*, 20, 3, June 1992, pp. 14-21.

Laird, M., S. Cox, T.J. Tumlin, and M. Smotherman: "A comparison of several superscalar designs," Technical Report, Dept. of Computer Science, Clemson University, July 1992.

Lauterbach, G.: "Vying for the lead in high-performance processors," *IEEE Computer*, 32, 6, June 1999, pp. 38-41. [UltraSPARC III]

Lee, R., and J. Huck,"64-bit and multimedia extensions in the PA-RISC 2.0 architecture," *Proc. COMPCON*, Santa Clara, CA, February 1996, pp. 152-160.

Leibholz, D., and R. Razdan: "The Alpha 21264: A 500 MHz out-of-order-execution microprocessor," *Proc. COMPCON*, San Jose, CA, February 1997, pp. 28-36.

Lempel, O., A. Peleg, and U. Weiser: "Intel's MMX technology - A new instruction set extension," *Proc. COMPCON*, San Jose, CA, February 1997, pp. 255-259.

Lesartre, G., and D. Hunt: "PA-8500: The continuing evolution of the PA-8000 family," *Proc. COMPCON*, San Jose, CA, February 1997.

Lev, L., et al.: "A 64-b microprocessor with multimedia support," *IEEE Journal of Solid-State Circuits*, 30, 11, November 1995, pp. 1227-1238. [UltraSPARC]

Levitan, D., T. Thomas, and P. Tu: "The PowerPC 620 microprocessor: A high performance superscalar RISC microprocessor," *Proc. COMPCON*, San Francisco, March 1995, pp. 285-291.

Lichtenstein, W.: "The architecture of the Culler 7," *Proc. COMPCON*, San Francisco, March 1986, pp. 467-470.

Lightner, B., and G. Hill: "The Metaflow Lightning chipset," *Proc. COMPCON*, San Francisco, February 1991, pp. 13-18.

Liptay, J.S.: "Design of the IBM Enterprise System/9000 high-end processor," *IBM Journal of Research and Development*, 36, 4, July 1992, pp. 713-731.

McGeady, S.: "The i960CA superscalar implementation of the 80960 architecture," *Proc. COMPCON*, San Francisco, February 1990, pp. 232-240. [cited as 1990a]

McGeady, S.: "Inside Intel's i960CA superscalar processor," *Microprocessors and Microsystems*, 14, 6, July/August 1990, pp. 385-396. [cited as 1990b]

McGeady, S., R. Steck, G. Hinton, and A Bajwa: "Performance enhancements in the superscalar i960MM embedded microprocessor," *Proc. COMPCON*, San Francisco, February 1991, pp. 4-7.

McKinney, D., D. Leibholz, M. Rosenbluth, J. Mullens, K. Chui, M. Bhaiwala, S. Patel, C. Houghton, and D. Ramey: "DECchip 21066: The Alpha AXP chip for cost-focused systems," *Proc. COMPCON*, San Francisco, Feb.-March 1994, pp. 406-413.

McLellan, E.: "The Alpha AXP architecture and 21064 processor," *IEEE Micro*, 11, 3, June 1993, pp. 36-47.

McMahan, S., M. Bluhm, and R. Garibay, Jr.: "6x86: The Cyrix solution to executing x86 binaries on a high performance microprocessor," *Proceedings of the IEEE*, 83, 12, December 1995, pp. 1664-1672.

Mirapuri, S., M. Woodacre, and N. Vasseghi: "The Mips R4000 processor," *IEEE Micro*, 12, 2, April 1992, pp. 10-22.

Moore, C.: "The PowerPC 601 microprocessor," *Proc. COMPCON*, San Francisco, February 1993, pp. 109-116.

Moore, C., D. Balser, J. Muhich, and R. East: "IBM single chip RISC processor (RSC)," *Proc. ICCD*, Cambridge, MA, October 1989, pp. 200-204.

O'Connell, F., and S. White: "POWER3: The next generation of PowerPC processors," *IBM Journal of Research and Development*, 44, 6, November 2000, pp. 873-884.

Oehler, R., and M. Blasgen: "IBM RISC System/6000: Architecture and performance," *IEEE Micro*, 11, 3, June 1991, pp. 54-62.

Papworth, D.: "Tuning the Pentium Pro microarchitecture," *IEEE Micro*, 16, 2, April 1996, pp. 8-15.

Patkar, N., A. Katsuno, S. Li, T. Maruyama, S. Savkar, M. Simone, G. Shen, R. Swami, and D. Tovey: "Microarchitecture of HaL's CPU," *Proc. COMPCON*, San Francisco, March 1995, pp. 259-266. [SPARC64]

Patt, Y., S. Melvin, W-M. Hwu, M. Shebanow, C. Chen, and J. Wei: "Run-time generation of HPS microinstructions from a VAX instruction stream," *Proc. MICRO-19*, New York, December 1986, pp. 75-81.

Peleg, A., and U. Weiser: "MMX technology extension to the Intel architecture," *IEEE Micro*, 16,4, August 1996, pp. 42-50.

Peng, C.R., T. Petersen, and R. Clark: "The PowerPC architecture: 64-bit power with 32-bit compatibility," *Proc. COMPCON*, San Francisco, March 1995, pp. 300-307.

Popescu, V., M. Schultz, J. Spracklen, G. Gibson, and B. Lightner: "The Metaflow architecture," *IEEE Micro*, 11, 3, June 1991, pp. 10-23.

Potter, T., M. Vaden, J. Young, and N. Ullah: "Resolution of data and control-flow dependencies in the PowerPC 601," *IEEE Micro*, 14, 5, October 1994, pp. 18-29.

Poursepanj, A., D. Ogden, B. Burgess, S. Gary, C. Dietz, D. Lee, S. Surya, and M. Peters: "The PowerPC 603 Microprocessor: Performance analysis and design tradeoffs," *Proc. COMPCON*, San Francisco, Feb.-March 1994, pp. 316-323.

Pugh, E., L. Johnson, and J. Palmer, *IBM's 360 and Early 370 Systems*. Cambridge, MA: M.I.T. Press, 1991.

Rau, B., C. Glaeser, and R. Picard: "Efficient code generation for horizontal architectures: Compiler techniques and architectural support," *Proc. ISCA*, Austin, TX, April 1982, pp. 131-139. [ESL machine, later Cydrome Cydra-5]

Riseman, E., and C. Foster: "The inhibition of potential parallelism by conditional jumps," *IEEE Trans. on Computers*, C-21, 12, December 1972, pp. 1405-1411.

Ryan, B.: "M1 challenges Pentium," *Byte*, 19, 1, January 1994, pp. 83-87. [Cyrix 6x86]

Ryan, B.: "NexGen Nx586 straddles the RISC/CISC divide," *Byte*, 19, 6, June 1994, p. 76.

Schorr, H.: "Design principles for a high-performance system," *Proc. of the Symposium on Computers and Automata*, New York, April 1971, pp. 165-192. [IBM ACS]

Seznec, A., S. Felix, V. Krishnan, and Y. Sazeides: "Design tradeoffs for the Alpha EV8 conditional branch predictor," *Proc. ISCA*, Anchorage, AK, May 2002.

Shen, J.P., and A. Wolfe: "Superscalar processor design," Tutorial, 20th Intl. Symp. on Computer Architecture, San Diego, May 1993.

Shriver, B., and B. Smith: *The Anatomy of a High-Performance Microprocessor: A Systems Perspective*. Los Alamitos, CA: IEEE Computer Society Press, 1998. [AMD K6-3D]

Simone, M., A. Essen, A. Ike, A. Krishnamoorthy, T. Maruyama, N. Patkar, M. Ramaswami, M. Shebanow, V. Thirumalaiswamy, and D. Tovey: "Implementation trade-offs in using a restricted data flow architecture in a high performance RISC microprocessor," *Proc. ISCA*, Santa Margherita Ligure, Italy, May 1995, pp. 151-162. [HaL SPARC64]

Sites, R.: "Alpha AXP architecture," *Communications of the ACM*, 36, 2, February 1993, pp. 33-44.

Smith, J.E.: "Decoupled access/execute computer architectures," *Proc. ISCA*, Austin, TX, April 1982, pp. 112-119.

Smith, J.E.: "Decoupled access/execute computer architectures," *ACM Trans. on Computer Systems*, 2, 4, November 1984, pp. 289-308.

Smith, J.E.: "The Astronautics S640 computer system," *Proc.Expanding Horizons Conference*, Minneapolis, April 1986. [ZS-1]

Smith, J.E., G. Dermer, B. Vanderwarn, S. Klinger, C. Rozewski, D. Fowler, K. Scidmore, and J. Laudon: "The ZS-1 central processor," *Proc. ASPLOS-II*, Palo Alto, CA, October 1987, pp. 199-204.

Smith, J.E., and T. Kaminski: "Varieties of decoupled access/execute computer architectures," *Proc. 20th Annual Allerton Conf. on Communication, Control, and Computing*," Monticello IL, October 1982, pp. 577-586.

Smith, J.E., and S. Weiss: "PowerPC 601 and Alpha 21064: A tale of two RISCs," *IEEE Computer*, 27, 6, June 1994, pp. 46-58.

Smith, J.E., S. Weiss, and N. Pang: "A simulation study of decoupled architecture computers," *IEEE Trans. on Computers*, C-35, 8, August 1986, pp. 692-702.

Smith, M., M. Johnson, and M. Horowitz: "Limits on multiple instruction issue," *Proc. ASPLOS-III*, Boston, April 1989, pp. 290-302.

Song, S., M. Denman, and J. Chang: "The PowerPC 604 RISC microprocessor," *IEEE Micro*, 14, 5, October 1994, pp. 8-17.

Special issue, "The IBM RISC System/6000 processor," *IBM Journal of Research and Development*, 34, 1, January 1990.

Special issue, "Alpha AXP architecture and systems," *Digital Technical Journal*, 4, 4, 1992.

Special issue, "Digital's Alpha chip project," *Communications of the ACM*, 36, 2, February 1993.

Special issue, "The making of the PowerPC," *Communications of the ACM*, 37, 6, June 1994.

Special issue, "POWER2 and PowerPC architecture and implementation," *IBM Journal of Research and Development*, 38, 5, September 1994.

Special issue, *Hewlett-Packard Journal*, 46, 2, April 1995. [HP PA 7100LC]

Special issue, *Hewlett-Packard Journal*, 48, 4, August 1997. [HP PA 8000 and PA 8200]

Sporer, M., F. Moss, and C. Mathias: "An introduction to the architecture of the Stellar graphics supercomputer," *Proc. COMPCON*, 1988, pp. 464-467. [GS-1000]

Sussenguth, E.: "Advanced Computing Systems," video-taped talk, Symposium in Honor of John Cocke, IBM T.J. Watson Research Center, Yorktown Heights, NY, June 18, 1990.

Thompson, T., and B. Ryan: "PowerPC 620 soars," *Byte*, 19,11, November 1994, pp. 113-120.

Tjaden, G., and M. Flynn: "Detection of parallel execution of independent instructions," *IEEE Trans. on Computers*, C-19, 10, October 1970, pp. 889-895.

Tremblay, M., D. Greenly, and K. Normoyle: "The design of the microarchitecture of the UltraSPARC I," *Proceedings of the IEEE*, 83, 12, December 1995, pp. 1653-1663.

Tremblay, M., and J.M. O'Connor: "UltraSPARC I: A four-issue processor supporting multimedia," *IEEE Micro*, 16, 2, April 1996, pp. 42-50.

Ullah, N., and M. Holle: "The MC88110 implementation of precise exceptions in a superscalar architecture," *ACM Computer Architecture News*, 21, 1, March 1993, pp. 15-25.

Undy, S., M. Bass, D. Hollenbeck, W. Kever, and L. Thayer: "A low-cost graphics and multimedia workstation chip set," *IEEE Micro*, 14, 2, April 1994, pp. 10-22. [HP 7100LC]

Vasseghi, N., K. Yeager, E. Sarto, and M. Seddighnezhad: "200-MHz superscalar RISC microprocessor," *IEEE Journal of Solid-State Circuits*, 31, 11, November 1996, pp. 1675-1686. [MIPS R10000]

Vegesna, R.: "Pinnacle-1: The next generation SPARC processor," *Proc. COMPCON*, San Francisco, February 1992, pp. 152-156. [HyperSPARC]

Wayner, P.: "SPARC strikes back," *Byte*, 19, 11, November 1994, pp. 105-112. [UltraSPARC]

Weiss, S., and J.E. Smith: *POWER and PowerPC*. San Francisco: Morgan Kaufmann, 1994.

White, S.: "POWER2: Architecture and performance," *Proc. COMPCON*, San Francisco, Feb.-March 1994, pp. 384-388.

Wilcke, W.: "Architectural overview of HaL systems," *Proc. COMPCON*, March 1995, pp. 251-258. [SPARC64]

Williams, T., N. Patkar, and G. Shen: "SPARC64: A 64-b 64-active-instruction out-of-order-execution MCM processor," *IEEE Journal of Solid-State Circuits*, 30, 11, November 1995, pp. 1215-1226.

Wilson, J., S. Melvin, M. Shebanow, W.-M. Hwu, and Y. Patt: "On tuning the microarchitecture of an HPS implementation of the VAX," *Proc. Micro-20*, Colorado Springs, December 1987, pp. 162-167. [This proceedings is hard to obtain, but the paper also appears in reduced size in *SIGMICRO Newsletter*, 19, 3, September 1988, pp. 56-58.]

Yeager, K.: "The MIPS R10000 superscalar microprocessor," *IEEE Micro*, 16, 2, April 1966, pp. 28-40.

第8章　高级寄存器数据流技术

8.1　简介

众所周知，现代处理器的性能主要受限于程序的两个特征：控制流和数据流。在学习先进取指技术时已经详细讨论了前者，例如分支预测、trace cache和其他解决Flynn瓶颈的高带宽解决方案。这些技术已被证明非常有效，许多技术在当前先进处理器的设计中已经得到了广泛的应用。然而，解决控制流给处理器性能带来的限制仍然是处理器研究和开发的一个极端重要的领域。在第9章中将重新讨论这一主题，并主要讨论多个同步控制流技术来解决由于分支预测的不准确和分支解析的低效而引起的瓶颈，这是目前研究的一个很活跃的领域。在此之前，我们先关注一下程序数据流带来的性能限制。

前几章已经讨论了程序中由假相关或称之为命名相关所带来的性能局限的解决方法。假相关是在程序执行过程中由于重复使用存储单元所导致的。程序员和编译器必须为临时操作数指定一个惟一的寄存器标识符，而这些寄存器标识符是有限的，一旦可用的标识符分配完，则需要重复使用它们。而且，即使指令集提供了无限数目的寄存器和寄存器标识符，程序中的循环也会引入存储标识符的重复使用，因为一个静态循环体可以同时展开多次，因此假相关和命名相关是不可避免的。在第4章中已经谈到，解决假相关的基本技术是通过把目的操作数重命名来避免对同一位置的多个写操作进行不必要的串行化。寄存器重命名首先在20世纪60年代末期IBM S/360-91中采用的Tomasulo算法中使用，在第4章中已经进行了详细的讨论。这种方法有效地消除了命名相关，使得指令的执行只需要考虑存在的真相关。对分支预测来说，这种技术十分有用，在过去的40年里，许多高性能的处理器设计实现了不同形式的寄存器重命名技术。

如果处理器执行指令的速率只受限于真数据相关，则称该处理器运行在处理器的极限状态。在程序动态数据流图中，只要指令的操作数就绪则该指令就被执行，在这种情况下，该处理器则达到了数据流极限。因此，指令被调度执行的时间只取决于在数据流图中的位置，而该位置是数据流图中到达该点的最长路径。例如，如图8.1所示，指令C在第二个周期执行，因为它和指令A及指令B之间的真相关关系，决定了它必须在第一个周期执行的指令A和指令B之后执行。在数据流图中，节点表示指令，边表示指令间的数据相关，边上的权值是产生指令的结果延迟。

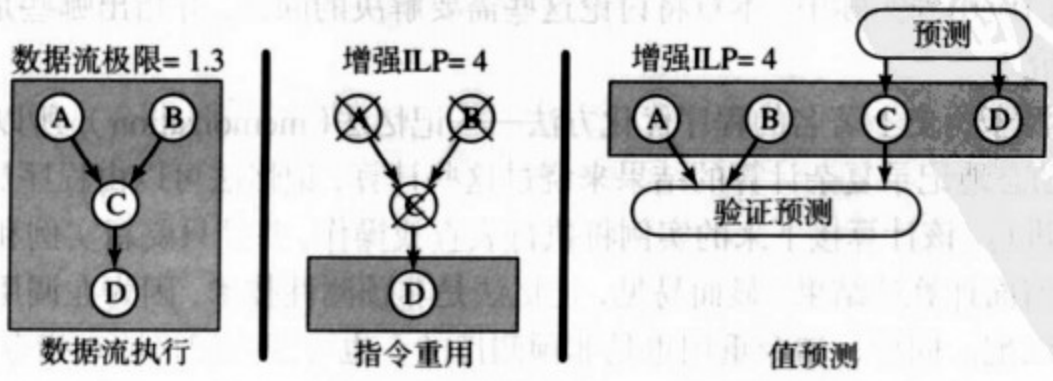


图8.1　超出数据流极限指定的ILP

给定一个数据流图，可以通过计算数据流图的高度（也就是存在的最长路径的长度）来计算程序执行时间的下限。数据流极限就是这个下限，并且决定了指令执行可获得的最大速率（或者说

ILP）。该速率定义为程序中指令的数量除以数据流图的高度。示例参见图8.1中左边所示的简单数据流图，图中由数据流极限决定的、可获得的最大ILP可以计算如下：

（4条指令）/（最长路径上的、3个周期的延迟）=1.3

本章主要讨论两种技术：值预测和指令重用，利用值局部性这一程序特征在典型数据流极限以外加速指令的执行。在这种上下文中，值局部性描述了程序指令所计算的结果（或者是相似的可预测的结果）在程序的执行中会重新出现的可能性。总的说来，程序的值局部性描述了这样一种经验统计：大部分真实程序产生和消耗的值由有限的值集合构成。这种性质同cache和存储层次（memory hierarchy）结构所依赖的时间和空间局部性类似，但它描述的是值本身而不是值的存储位置。

下面讨论的两种技术都使用了值局部性特征，一种技术是重用前面指令计算的结果（指令重用），另一种技术则根据前面一些指令的结果预测将要执行的指令结果（值预测）。两种方法都可以使处理器在数据流图中指令所示位置之前就获得该指令执行的结果，且都可减小数据流图的实际高度，因此可以将指令执行速率提高到超出数据流极限。例如，如图8.1中所示，指令重用机制可以识别出指令A、B以及C表示一个原来的计算，并且可以重用该计算的结果，使得指令D立即执行而不是等待指令A、B以及C的执行结果，使得指令吞吐率达到每个周期4条指令。类似地，图8.1中右边显示了使用值预测机制正确预测指令A、B以及C的结果，把可获得的指令级并行度从较少的1.3提高到了理想的每个周期4条指令。如果指令A和B预测准确，C不需要等待它们的结果就可以执行。同样，如果C预测准确，D也不需要等待C的结果就可以执行。因此，所有这4条指令可以并行执行。

图8.1也描述了指令重用与值预测之间的主要区别。指令重用机制完全不用执行指令A、B、C，值预测则避免了这些指令的串行执行，但是不能不执行。这一差别是由两种技术的一个基本不同点引起的：指令重用保证值局部性，而值预测只是预测它的结果。在后一种情况下，处理器仍然需要执行预测了的指令，并比较执行结果和预测结果来验证预测。这和分支预测很类似，尽管正确的概率很大，预测了方向的分支指令还是需要执行，从而验证预测的正确性。当然，验证消耗了执行带宽，并且要求提供比较机制来验证结果。相反，指令重用之前就提供了正确性的保证，所以不需要验证代码。然而，如8.3.2小节所述，这种正确性保证从表面上看很诱人，但带来的一些负面影响则增加了实现代价，从而降低了指令重用的实用性。

不管是值预测还是指令重用都是最近才开始在文献中出现的，并且还没有在真实的设计中实现过。不过两者对于提高真实程序的性能都表现出相当的潜力，特别是对于那些ILP受限于真数据相关（相对于结构或者控制相关）的程序而言。新的思想都会产生大量需要解决的问题，包括挖掘性能潜力并且将它们应用到实际中。本章将讨论这些需要解决的问题，并指出哪些是可行的，哪些则需要进一步的研究。

指令重用来源于历史上著名的程序优化方法——记忆法（memoization），所以首先讨论这种方法。记忆法通过动态地记录复杂计算的结果来绕过这些计算，记忆法可以由程序员手工进行，或者通过编译器自动进行。该计算接下来的实例将执行表查找操作，并且只要新实例和前面实例的前提条件匹配就重用前面计算的结果。显而易见，记忆法是非预测性技术，因为在调用计算重用之前它要求精确的前提匹配。同样，指令重用也是非预测性的，也可视为记忆法在指令级的硬件实现。

接下来讨论值预测技术，由于它具有预测的特征，所以与指令重用有根本上的不同。值预测试图根据前面的输出来推断指令将来的输出，而不是重用前面所执行的指令。在这一点上，它非常类似于被广泛使用的、基于历史信息的动态分支预测技术（参见第4章），但两者存在一个显著的区

别。分支预测可以收集非常长（可以用几十个分支实例的历史输出来决定后面实例的预测）的历史输出，该技术所预测的属性信息非常少，只需要一个状态位来决定分支是否跳转。相反，值预测试图预测寄存器更新指令的整个32位或者64位结果。显然，精确的值预测需要更宽（全操作数宽度）的历史信息和额外的机制来避免预测错误。另外，产生预测只是实现值预测所需解决问题的一小部分。与分支预测一样，基于预测值的推测执行、预测的验证和预测错误的恢复机制都需要正确地加以处理。

本章首先讨论值局部性，接下来介绍非预测技术（例如指令重用）和预测技术（例如值预测），这两种技术从多个方面利用了值局部性特征。此外还将详细讨论所有这些技术之间的紧密联系，并通过结合指令重用技术和值预测技术来减少验证预测所产生的代价。

8.2　值局部性和冗余执行

本节将讨论值局部性的概念。值局部性定义为以前出现的值在存储单元重复出现的可能性。尽管概念很通用，并能适用于计算机系统中任何存储单元，这里只考虑紧跟在通用寄存器或者浮点寄存器更新指令之后的这些寄存器的值局部性。前面关于动态分支预测的工作只是讨论了值局部性的一个相当有限的应用，即根据以前的行为预测单个条件位。本章中的许多思想可以看成这些工作的逻辑延伸，并将预测单个位扩展为预测整个32位或64位寄存器。

8.2.1　值局部性的缘由

从直觉上说，在寄存器中寻找有用的值局部性似乎很困难。32位寄存器可能包含40亿个值中的任意一个，怎样才能预测出这些值中哪一个才是下一个可能出现的值呢？如果通过考虑单独每条静态指令来缩小预测机制的范围，任务会变得相对容易，并且能够准确地预测出将要写入寄存器文件的值中相当大的一部分。

是什么导致这些值是可预测的呢？通过考察一些真实的程序发现，值局部性的存在主要是由于真实程序、运行时环境以及操作系统在设计上是常规的，即它们的实现不但可以处理偶然事故、异常条件和错误输入等很少出现的情况，同时在设计时也常常考虑将来的扩展和代码重用。即使经过最先进的编译器主动优化过的代码也表现出这些趋势。下列经验统计了来源于对很多真实程序的研究，可以帮助读者理解为什么值局部性会存在：

- **数据冗余**：真实程序的输入构成的集合常常是由变动很小的数据构成。例如包含多个0的稀疏矩阵、包含空格的文本文件以及电子表格中的空白单元。
- **错误检测**：对不常发生的情况的检测也常常被编译进负载单元，而这实际上是运行时的常量。
- **程序变量**：利用立即数来构造程序常量比从存储器载入程序常量更加有效。
- **分支的计算**：为了计算分支的目标地址，例如switch语句，编译器必须产生代码将分支表的基地址装载进寄存器，该地址常常是运行时的常量。
- **虚函数调用**：为了调用虚函数，编译器必须产生代码来装载函数指针，该指针常常是运行时的常量。
- **黏合代码**：考虑到可编址能力和链接约定，编译器必须时常产生黏合代码从一个编译单元调用另一个编译单元。这样的代码包含的load指令与数据的地址往往在程序的整个执行过程中保持不变。
- **可编址能力**：为了获得非自动存储单元的地址可编程能力，编译器必须从表中装载指针，该指针只有在程序装载后才被初始化并始终保持不变。

- **调用子图的恒等性**：函数或者过程总是被固定的且比较小的函数集合调用，同样，函数和过程自身也趋向于调用固定的函数集合，该集合通常比较小。因此，动态调用通常在程序调用图中形成恒等的子图。因此，恢复链接寄存器以及其他调用者保存的寄存器中的load指令有很高的值局部性。
- **存储器别名解析**：编译器必须对可能存在别名的store指令的处理采用保守策略，它会频繁地产生似乎冗余的load指令来解析这些别名。这些装载很可能表现出某种程度的值局部性。
- **寄存器溢出代码**：当编译器使用完所有的寄存器时，便将保持不变的变量写入存储器并重复地从存储器载入。
- **收敛算法**：值局部性常常是由程序员要实现的算法所引起的。常见的例子如收敛算法，该算法在一个数据集合上重复执行，直到达到全局收敛。通常，局部收敛会在全局收敛之前出现，导致收敛域中的重复计算。
- **轮询算法**：算法的选择带来值局部性的另一个例子是使用轮询算法代替效率更高的事件驱动算法。在轮询算法中，最可能出现的结果是所轮询的事件还没有发生，重复检测则引起大量的冗余计算。

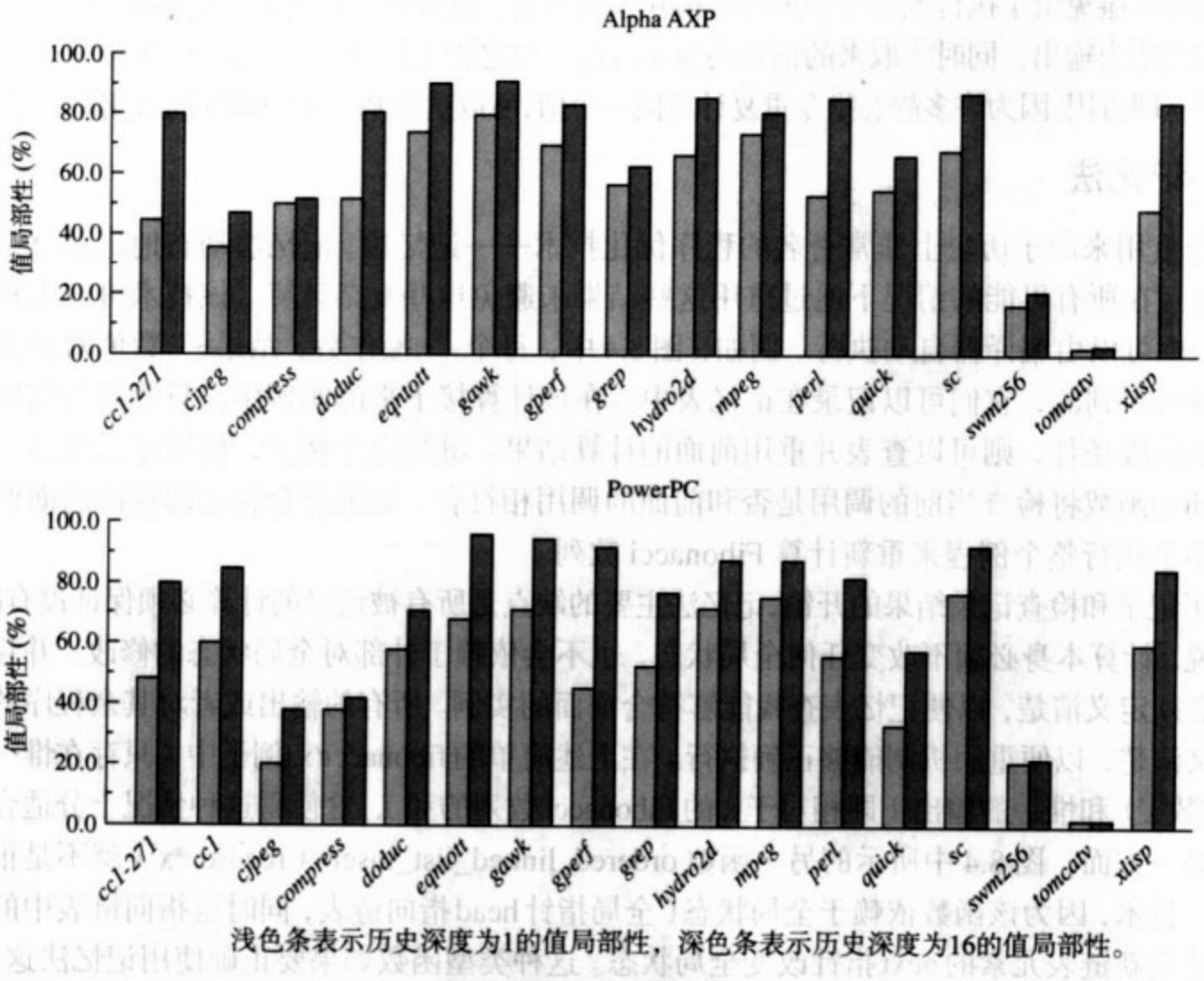
自然，上面很多情况受限于指令级的细节、编译器以及使用的运行时环境，也许有人认为，如果ISA、编译器以及运行时环境发生改变，使用积极的链接或者采用运行时优化可以使得某些值局部性消失。然而，这些改变和提高不会很快出现。目前，上述（以及其他）因素对值局部性的总体影响在我们研究的两种现代RISC指令集上是可以测量并且是相当可观的，这两个指令集都提供了目前水平最高的编译器和运行时系统。值得指出的是，诸如循环展开、循环剥离（peeling）以及尾部复制（tail replication）等编译优化技术可以显著地影响程序中特定静态load的值局部性，因为这些编译优化技术趋向于产生一个load指令的多个实例，这些实例单独以一个带有或高或低值局部性的存储器地址为目标。

8.2.2 量化值局部性

图8.2显示了不同测试程序中load指令的值局部性。对于每个测试程序中的值局部性，是用每条静态load指令从存储器中检索的值符合该静态指令先前所见值的次数，并且除以测试程序中所有动态load次数的和来计算的。图中显示了两组数据，第一组（浅色条）的历史深度为1（即只检测最新检索的值是否符合），第二组（深色条）的历史深度为16（即检测最近16个不同的值）。可以看出，即使历史深度为1，大部分整数程序表现的load值局部性也在50%左右，如果将历史深度扩展到16（假定从16个值中选择正确值的机制是理想的），则值局部性可以提高到80%多。这意味着绝大部分静态load在程序执行过程中装载的值变化很小。遗憾的是，测试程序中有三个程序（cjpeg、swm256和tomcatv）其表现的值局部性较差。

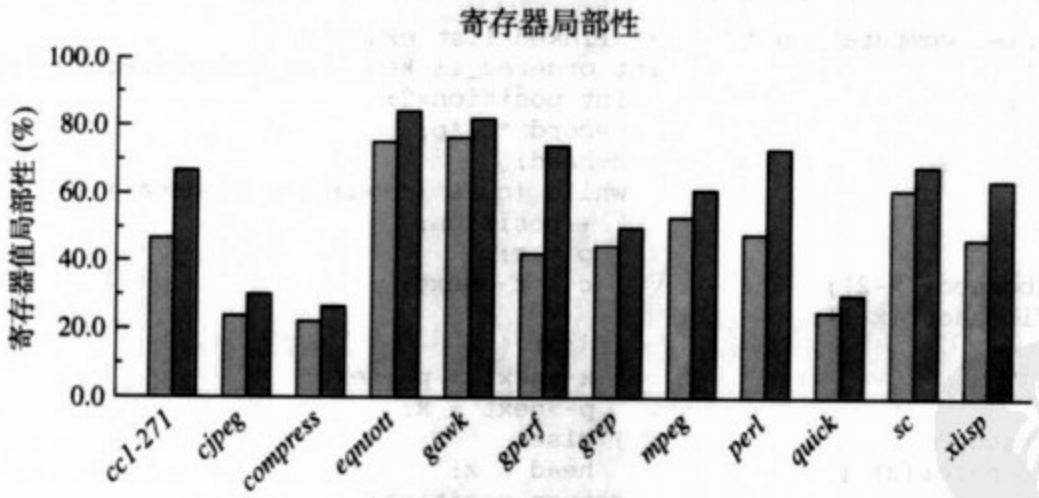
图8.3显示了测试程序中所有写整数寄存器或者浮点寄存器时的平均值局部性（average value locality）。每条静态指令的值局部性计算如下：该静态指令写的值和它前面所见值相符合的次数除以该指令动态出现的总次数。测试程序的平均值局部性是该测试程序中所有值局部性的动态加权平均值。在显示的两组数据中，其中一组（浅色条）表示历史深度为1（即只检测最新写的值是否符合），第二组（深色条）的历史深度为4（即检查最新的4个不同的值）。可以看出，即使历史深度为1，大部分程序显示的值局部性也还是在40%到50%之间（平均51%），如果将历史深度扩展到4（假定从4个值中选择正确值的机制是理想的），则能够将值局部性提高到60%到70%。这意

味着在程序执行过程中，大部分静态指令写入寄存器的值很少有变化。同样，这些测试程序中的 cjpeg、compress 和 quick 表现的值局部性则较弱。



浅色条表示历史深度为1的值局部性，深色条表示历史深度为16的值局部性。

图 8.2　load 值局部性



浅色条表示历史深度为1的值局部性，深色条表示历史深度为16的值局部性。

图 8.3　寄存器值局部性

总体来说，所有在这里研究的程序以及其他程序在 load 指令和写寄存器指令方面都表现出显著的值局部性。该属性至少被不同的指令集、编译器和包括多个用户状态与核心状态的作业独立地验证过。

8.3　非预测的值局部性利用

真实程序中广泛存在的值局部性为提高处理器的性能创造了机会。正如前面所介绍的，预测和非预测技术都可利用。首先介绍使用值局部性的非预测技术，因为与之相关的技术都是非常熟悉的。最

近有一项提议，该提议通过使用指令重用来唤起对该技术的兴趣，这一技术重用前一次执行的结果并且体现了纯硬件技术。在最简单的形式下，只要指令在其历史记录中发现了与之相同的指令，指令重用机制就可以避免由于执行该指令带来的结构和数据相关。这种情况只是简单地重复使用指令重用缓冲中存储的历史输出，同时将取来的指令丢弃不执行。与之相关的指令可以立即流出执行，因为该指令的结果立即可用。因为许多静态指令重复计算同一个值，所以这种基于值局部性的重用常常是可能的。

8.3.1 记忆法

指令重用来源于历史上非常著名的程序优化技术——记忆法。记忆法动态地记录复杂计算的结果，并且在所有可能的情况下通过重用这些结果来避免这些复杂计算。该技术可以由程序员手工执行，也可以由编译器自动执行。例如，图8.4中，每个“<操作数，结果>”数值对是调用函数fibonacci(x)得到的，它们可以记录在记忆表中。在该计算接下来的实例中，只要新的实例符合前面实例的前提条件，则可以查表并重用前面的计算结果。继续这个例子，使用记忆法这一技术的fibonacci(x)函数将检查当前的调用是否和前面的调用相符合，如果符合将立即返回前面调用的结果，而不是执行整个例程来重新计算Fibonacci数列。

除了记录和检查记录结果的开销，记忆法主要的缺点是所有被记录的计算必须保证没有副作用。也就是说，计算本身必须不改变任何全局状态，也不会依赖于外部对全局状态的修改。并且它的所有输入必须定义清楚，以便记忆表查找能够符合前面的实例。所有的输出或者对其余程序的影响也必须定义清楚，以便重用机制能够正确执行。在上述简单的fibonacci(x)例子中，只存在惟一的输入（即操作数x）和惟一的输出（即相对于x的Fibonacci数列的和），这使得这种情况十分适合使用记忆法。另一方面，图8.4中所示的另一函数ordered_linked_list_insert（record *x）就不是很适合记忆法这一技术，因为该函数依赖于全局状态（全局指针head指向链表，同时也指向链表中的节点），并且通过更新链表元素的next指针改变全局状态。这种类型函数如果要正确使用记忆法这一技术，需要检查指针head，同时检查链表中的元素，确定这些元素在上一次请求之后没有发生改变。

```
/* fibonacci series computation */
int fibonacci(x) {
  int result = 0;
  if (x==0)
    result = 0;
  else if (x<3)
    result = 1;
  else {
    result = fibonacci(x-2);
    result += fibonacci(x-1);
  }
  return result;
}
/* memoized version */
int memoized_fibonacci(x) {
  if (seen_before(x))
    return memoized_result(x);
  else {
    int result = fibonacci(x);
    memoize(x,result);
    return result;
  }
}
```

```
/* linked list example */
int ordered_linked_list_insert(record *x) {
  int position=0;
  record *c,*p;
  c=head;
  while (c && (c->data < x->data)) {
    ++position;
    p = c;
    c = c->next;
  }
  if (p) {
    x->next = p->next;
    p->next = x;
  } else
    head = x;
  return position;
}
```

左边所示的对fibonacci(x)的调用很容易被记录，如memoized_fibonacci(x)函数的实现。对于调用了ordered_linked_list(record *x)的fibonacci(x)，实现记忆法将会很困难，因为它依赖于全局变量，并且作为调用的副作用会更新这些全局变量。

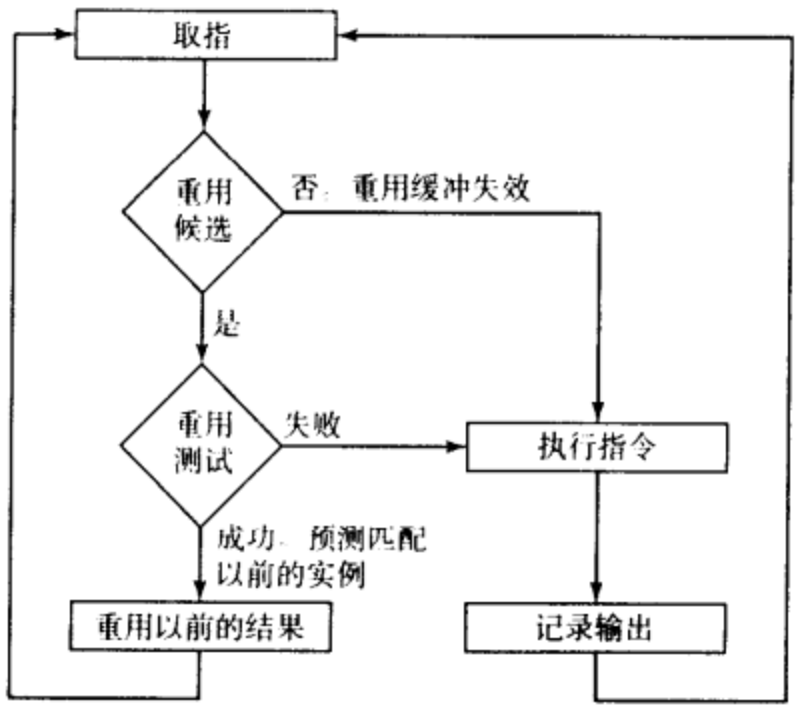
图8.4 *Memoization*示例

无论如何，记忆法是一种强有力的编程技术，它被广泛使用而且非常有效。显而易见，记忆法是一种是非预测性技术，因为该技术要求在调用重用前精确地满足正确的前提。

8.3.2　指令重用

从概念上说，指令重用只不过是记忆法在指令级的硬件实现。这种技术一旦发现“生产者”指令无须执行，则将“生产者”指令和“消费者”指令分离，以发现更多的指令级并行。只要重用机制发现某条指令符合重用历史上的实例，并能够安全使用前一实例的结果，就可以采用这一技术。Sodani和Sohi对指令重用的最先提议主张重用单条机器指令，只要该指令的操作数与该指令前一实例相比没有发生变化就可以重用。同时该提议还描述了一种更高级的机制来记录和重用具有数据相关的指令序列。在该机制中，如果指定数据流区域所有的输入是不变的，并且指令间的数据相关关系存储在重用历史表中，就能够自动重用指令的数据流区域（例如动态数据流图的子图）。接下来的提议还考虑了将重用的范围扩展到包括基本块和从 trace cache 中取出的指令 trace（有关 trace cache 操作的细节请参考第 4 章）。

所有关于重用的提议都使用同一个基本方法：单个指令或者指令集合的执行被记录在历史结构中，该结构存储这些计算的结果以便后面重用。指令集合可以用控制流（如在基本块重用和trace重用中）或者数据流（如数据流区域重用）来定义。该历史结构必须具有某种机制来保证它所保留的内容与程序后面的执行一致。最后，历史结构必须具备查找机制使得后面执行的实例能够与存储的实例进行比较。查找命中或者匹配会触发重用机制，重用机制使得处理器可以跳过可重用段。因此，处理器可以消除由可重用段引起的结构和数据相关，从而能够快速执行后面的程序指令。图8.5总结了该处理过程：



读取一条指令之后，则检查历史机制来确定该指令是否是重用候选。如果是，并且如果指令的前提条件和历史实例匹配，则重用历史实例并丢弃取来的指令。否则，像往常一样执行指令并将其结果记录在历史机制中。

图 8.5　指令重用

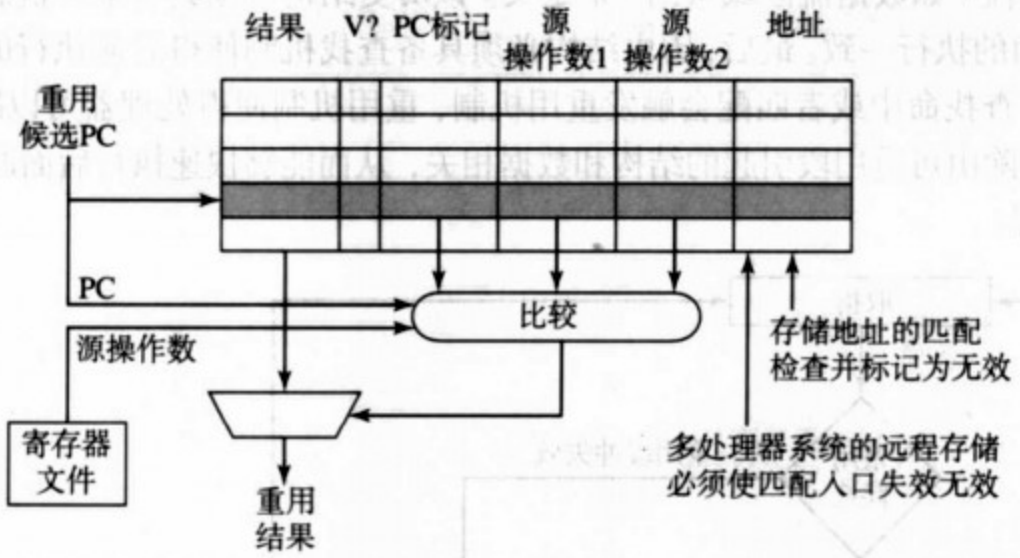
8.3.2.1　重用历史机制

重用机制的所有实现必须具备一种机制来记忆前面所执行的指令或者指令序列。该历史机制必须把前提条件的集合和前面计算出来的结果联系起来。这些前提条件必须精确地指定所要执行的计算以及所有活跃输入（live input），即能够影响计算结果的操作数。对于指令重用，要执行的计算由程序计数器（PC）来指定，程序记数器在进程地址空间中惟一地指定一条静态指令，而活跃输入则是该静态指令的寄存器和存储器操作数。对于块重用来说，计算由基本块中指令的地址范围来指定，活跃输入则是基本块入口处活跃的源寄存器和存储器操作数。对于 trace 重用来说，相应计算

和 trace cache 中的表项相对应，该表项是由取指指令的地址和确定 trace 控制流通路的一组条件分支的输出惟一确定的。另外，也必须指明 trace 入口处活跃的操作数。

保存在重用缓冲中的前提条件包含一个关键属性，即它们惟一地指定一个事件集合，该集合导致了所记录（memoized）结果的计算。因此，如果该集合所定义的所有事件再一次发生，则不需要重新执行计算，可以用记录的结果来代替计算的结果。然而，正如图 8.4 中的例子所示，必须注意到前提条件应该完全指明可能影响计算结果的所有事件。否则，重用机制可能在程序执行时引入错误。

检索和更新重用缓冲 图 8.6 中描述了历史机制或者称为重用缓冲。重用缓冲通常通过 PC 值的低位来检索，该缓冲可以组织成直接映射、组相联或者是全相联的结构。在索引和/或标记中增加输入操作数的若干位能够提供额外的检索信息。这种方法使得同一条静态指令的、具有不同输入操作数的多个实例可以共存于重用缓冲中。当指令或者指令组从执行窗口中退出时，重用缓冲将自动更新，更新可能要求多端口技术或者多体技术来解决高吞吐率的问题。另外，还需要考虑一些常规设计问题，例如替换策略和写回策略（对多级历史结构而言）。



指令重用缓冲存储（为了保证重用指令前面实例的正确性）所需要的前提条件。对ALU和分支指令来说，这包括了PC和源操作数的值。对load和store指令来说，还需要存储存储器地址，对该地址插入的写操作会使得重用项无效。

图 8.6 指令重用缓冲

重用缓冲的组织 重用缓冲可以组织成存储单个指令的历史（即每个表项对应单条指令）、基本块的历史、trace 历史（在 trace cache 中有效地结合重用历史）或者数据流区域历史。当重用缓冲中的每个表项包含大量指令的时候，跟踪活跃输入将存在可扩展性问题。例如，假设一个基本块的大小为 6 条指令或者更多，每条指令包含两个源操作数和一个目标结果。在这种情况下，基本块历史机制可能需要存储 12 条或更多的活跃输入以及 6、7 个结果。trace 重用机制中也存在类似的可扩展性问题，该重用机制试图重用多达 16 条指令的 trace。可以想像，如果将图 8.6 中指令宽度为 1 的结构扩展为所有栏可容纳 16 个实例会出现什么情况。显而易见，构建该宽度的结构和检测重用前提条件的比较器将是一项很困难的任务。

指定活跃输入 可以用名字或者值来指定重用表项的活跃寄存器输入。用名字来指定意味着用所记录的体系结构寄存器编号来表示寄存器操作数，或者用记录的存储器地址来表示存储器操作数。用值来指定则意味着记录的是操作数实际的值而不是它的名字。不管用哪种方法，都必须指定所有的活跃输入以便保证正确性。如果少指定了一个活跃输入，则可能导致错误的重用，因为即使是一条未记录活跃输入的微小变动都会导致不同的结果。Sodani 和 Sohi 研究了利用名字和值来指定

寄存器操作数的机制，但是对存储器源操作数而言，他们只考虑了利用名字来指定的情况。图8.6中重用缓冲的例子是利用值来指定寄存器源操作数，利用名字来指定存储器的位置。

验证活跃输入　为了验证重用候选项的活跃输入的有效性，必须保证保存在重用缓冲表项中的输入与操作数目前的系统值相符，该处理过程称为重用测试。只有所有活跃输入都为有效时才能重用，否则重用的结果可能不正确。对命名的操作数来说，该属性是由一致性机制（一致性机制在下面解释）来保证的。一致性机制对程序所有写操作都检查重用缓冲。对使用值来指定的操作数来说，重用机制必须比较当前系统值和重用表项中的值是否相符。对寄存器操作数而言，这种检查包括从体系结构寄存器文件中读出当前值，并与保存在重用缓冲表项中的值进行比较。注意，这对物理寄存器读端口有着很高的要求，因为必须同时读入所有重用候选项的操作数。对用值来指定的存储器操作数来说，执行重用测试需要从存储器中取出操作数的值并进行比较。显然，这种情况得不到任何好处，因为为了进行比较而从存储器取值的工作量比执行该存储器操作本身的工作量不会少。因此，到目前为止所有关于重用的建议都使用名字来指定存储器操作数，而不是通过值来指定。在图8.6中，每个重用候选项必须从寄存器文件读取到源操作数，并将它们与存储在重用缓冲表项中的值进行比较。

重用缓冲一致性机制　为了保证正确性，重用缓冲必须在插入重用缓冲表项和所有重用该表项的程序之间保持一致。为了保持一致，所有与命名活跃输入冲突的寄存器，或者存储器写操作必须在重用缓冲中有适当的反映。一致性机制负责跟踪程序（或者在多处理器系统情况下跟踪其他处理器运行的其他程序）执行的所有写操作，并且保证与这些写操作相对应的命名活跃输入在重用结构中被标注为无效。这将防止在命名活跃输入发生改变时无效重用的发生。如果活跃输入是由值而不是名字来指定的，则不需要检测插入的写操作，因为活跃输入验证会比较结果的系统值和历史值，并且只有在值匹配时才会触发重用机制。注意，对于命名输入来说，一致性机制必须对程序中的每个写操作都执行重用缓冲中所有活跃输入的相关查找。对于长名字（例如，32位或者64位存储器地址），即便是中等大小的历史表，该相关查找的代价也是难以接受的。在图8.6中，处理器执行的所有存储操作必须检测重用缓冲中是否有可以匹配的项，如果地址与存储操作匹配则必须验证该项。类似地，在多处理器系统中，所有远程写操作必须验证重用缓冲中的匹配表项。

最后，在允许自修改代码的系统中，一致性机制必须能够跟踪保存在重用缓冲中的指令地址的写操作。如果发现有对这些指令的写操作，则必须失效重用缓冲中所有与之匹配的表项。否则，将导致重用与当前存储器映像不一致指令。类似地，对于使指令缓冲中相应项失效的指令（例如PowerPC体系结构中的icbi），其语义必须扩展为同时作废带有匹配标识的重用缓冲表项。

8.3.2.2　重用机制

最后，为了利用重用机制来提高性能，处理器必须能够忽略重用指令的执行，可以直接执行后面的工作，以此来消除或者减少重用指令所带来的数据相关和结构相关。这种做法看起来很直接，但可能要对处理器的数据和控制路径进行很大的修改。首先，重用候选项（不管是单条指令还是一组指令）必须将它们的结果写入处理器的系统状态。在现有的处理器中，只有功能单元才可以将结果写入寄存器文件中，这样做可能需要在已经有多个端口的物理寄存器文件上再增加写端口。第二，必须修改指令唤醒（wakeup）和调度逻辑，以便适应延迟实际上为0的重用指令。第三，为了支持精确异常，重用候选项必须进入处理器的再定序缓冲，同时它必须绕过发射队列或者保留站。这种非标准的行为将给控制通路带来额外的复杂性。最后，为了保持正确的存储器引用顺序，需要在处理器的load/store队列（LSQ）中跟踪重用的存储器指令。因为LSQ队列表项的更新一般是在指令发射之后、根据执行中所产生的地址来进行的，所以为了让更新在更早（发射或者执行之前）的流水段发生，也需要额外的数据路径和LSQ写端口。

总之，实现指令重用需要对现有的现代微处理器设计中的控制路径和数据路径进行相当大的修改，甚至可能需要重新设计。这种要求也许就是重用机制还没有在任何真实设计中出现的原因，其改变之大以至于它们很可能只能在全新的设计中才会被引入。

8.3.3 基本块和 trace 重用

随后的提议扩展了 Sodani 和 Sohi 最初关于指令重用的提案，并将其扩展为包含了由控制流定义的指令集合的重用。在这些提案中，都使用了类似的方法来存储和查找重用历史，只不过操作的粒度是基本块或指令 trace。在这两种情况下，控制流单元（不管是基本块还是 trace）都作为原子重用计算单元。换句话说，因输入操作数部分匹配而采用部分重用是不允许的。因为可以直接旁路大块指令，所以，将指令重用范围扩展到基本块和trace增加了每个重用实例的潜在优势。然而，多个活跃输入都与前面的计算相同，这种概率比在这些组中找到单个可重用指令的概率小得多，所以该技术同时也减小了找到匹配重用表项的概率。如前面所讨论的，基本块和 trace 可能具有大量的活跃输入，对此进行的重用检测会导致可扩展性问题。也许只有时间可以验证在更粗控制粒度上的重用是否比指令级重用更加有效。

8.3.4 数据流区域重用

与随后试图重用基于控制流的指令组不同，Sodani 还提议了一种方法来存储和重用指令的数据流区域（S_{n+d} 和 S_{v+d} 策略）。该方法要求使用簿记（bookkeeping）策略，簿记策略将指针嵌入到重用缓冲中来连接具有数据相关的指令。对数据流图来说，因为重用属性是可以传递的，可以通过遍历这些指针来重用数据流图的整个子图。因此，任何指令，只要它的数据流的前项都是重用候选（即它们都满足重用测试）则它也是重用候选。归纳使用该原则可以构造一个可重用的数据流区域，并产生出一组相互联系的可重用指令。可重用区域是通过跟踪嵌入到重用表中的数据相关指针动态构建的。相关指令由这些边相连，并且所有成功的重用测试结果会沿着这些边传递到相关指令。相关指令的重用测试将检测所有活跃输入参数是否是由前面刚刚重用过的指令产生的，如果不是，则传递重用测试。如果满足了这一条件则意味着所有的操作数没有发生变化，或者是由前面的重用表项所产生的，相关指令可以被重用。重用测试既可以通过名字（在 S_{n+d} 策略中）也可以通过值（在 S_{v+d} 策略中）来执行。

在动态管理结构中维持数据相关指针的完整性（integrity）是一个难以解决的问题。只要替换重用缓冲中的表项，所有指向该表项的指针都将变成陈旧的。必须找到并删除所有陈旧的指针，以便防止随后对重用缓冲的访问导致可重用性的错误传播。Sodani 建议使用相联查找机制，在每次替换表项时自动失效所有这样的指针。显然，相联查找的代价和复杂性以及频繁的替换操作使得这种解决方案是不可伸缩的。另外一种策略是将相关指针存储在一个单独的、较小的结构中，该结构很适合进行相联查找。这种策略是可能的，但是目前并没有得到相应的研究结果。

Connors和Hwu随后的工作提出了严格地用软件来实现区域级重用，通过修改编译器来产生代码，产生的代码将执行编译器所构造的数据流域重用测试。这种方法将检测活跃输入操作数，并且只要发现与之相匹配的历史表项就调用域重用，同时忽略该域的执行，直接用结果更新系统状态。实际上，该工作又回到了基于软件的记忆法这一技术，为记忆法建立历史驱动（profile-driven）技术是可行的，也是最好的。

8.3.5 结论

总之，该节讨论了各种重用前次计算的策略。从概念上说，这些提议都与著名的记忆法这一技术类似，只是在重用的粒度和实现的细节上有所不同。它们都是基于程序的值局部性这一特征，因

为如果没有程序值局部性这个特征，识别重用候选项的概率会非常低。到目前为止，还没有任何真正的设计采用了重用技术，但是如果成功解决了所有的实现问题，重用技术还是具有很大的性能潜力。

8.4　带预测的值局部性利用

在考虑了使用非预测技术来开发值局部性以便增强指令级并行之后，接下来考虑利用预测技术实现相同的功能。在深入研究值预测的细节之前，先考虑一下推断执行的理论基础——弱相关模型（weak dependence model）。

8.4.1　弱相关模型

正如在研究假相关消除技术时所学到的，顺序程序中过于强调了指令间隐含的优先顺序，这些优先顺序并不需要严格满足顺序执行模型的要求。程序实际的语义和指令间的相关性是由控制流图（CFG）和数据流图（DFG）来指定的。只要没有违背CFG和DFG强加的串行化约束，就可以重叠和重排序指令的执行，这样做可以避免强加隐含的但不必要的优先顺序，也就提高了性能。通过Tomasulo算法或者更近一些的、基于再定序缓冲的实现可以获得这种提高。然而，必须要保持指令间的真相关。迄今为止，所有严格保持这种相关的机器都包含下列两个要求：

- 相关是以一种绝对而精确的方式决定的。也就是说，两条指令间的关系不是相关就是非相关，如果存在疑问，则总是悲观地假设指令之间存在相关。
- 在指令执行中保持相关。也就是说，不允许违背相关关系，并且在指令在执行过程中，需要持续保持该相关关系。

这种传统而保守的程序执行方法可以称之为强相关模型（strong dependence model）。传统的强相关模型过分严格，并且不必要地限制了可用的并行性。另一种允许使用诸如值预测等技术的可选模型是弱相关模型，该模型规定：

- 不需要精确确定或者悲观地假定相关关系，而是乐观地近似估计甚至暂时忽略相关关系。
- 只要在影响持久的机器状态前能够执行恢复工作，就可以在指令执行中暂时违背相关关系。

采用弱相关模型的优点是，由CFG和DFG指明的程序语义在机器能处理指令之前不需要完全确定。更进一步地说，只要矫正措施随时可以恢复预测错误，机器就可以主动预测或者暂时违背相关关系。如果很大一部分预测都是正确的话，机器实际上可以超过传统强相关模型所增加的性能极限。

从概念上说，使用弱相关模型的机器有两个相互作用的引擎。前端引擎采用弱相关模型并采用推断执行。它试图推断指令的执行以便积极地处理指令。当推断正确时，这些推断的指令实际上会忽略或者跳过流水线某些阶段的执行。后端引擎仍然使用强相关引擎来验证推断的有效性，从预测错误中恢复并为推断引擎提供历史及指导信息。结合这两种引擎，可以在不违背程序语义的情况下获得空前的指令级并行性。DFG 中表示指令间相关性的边现在只在推断错误时才在关键路径上出现。实际上，这些相关边是随机的，并且只要推断正确就可以消除或者屏蔽由维持这些相关所导致的串行化损失。因此，传统的、DFG中基于关键路径长度的数据流极限不再是不可逾越的硬性极限。

8.4.2　值预测

在8.2.2小节中曾了解到，许多程序的寄存器写操作都表现出了相当程度的值局部性。该发现为微体系结构的设计提供了新的、激动人心的可能性。因为许多指令的结果在发射或者执行前就可

以精确地预测，与之相关的指令不再受操作数数据流的串行化约束所带来的限制。现在，因为压缩了通过相关图的关键路径，所以可以以额外的自由度来推测性地调度指令，以便更好地使用函数单元和硬件缓冲并且经常能够更早地完成操作。然而，为了使用值局部性并获得这些好处，必须实现多种硬件机制：能够准确预测值（值预测单元）的单元；支持利用推断值执行的微体系结构；检测值预测的机制；最后还需有恢复机制，以便在程序执行中发生错误推断的情况下能恢复正确的执行。

8.4.3 值预测单元

值预测单元为处理器核心的推断计算产生正确的预测值。值预测单元的效果主要由两个因素决定：准确性和覆盖率。与覆盖率相关的第三个因素是预测器的范围。准确性度量预测器能够避免预测发生错误的能力，而覆盖率则度量预测器并能够尽可能多地预测指令输出。预测器的范围描述了预测器所确定的指令集合。获得较高的准确性（即基本不发生预测错误）一般意味着减少覆盖率，因为所有消除预测错误的策略很可能会同时删除一些正确的预测。相反，由于同样的原因获得较高的覆盖率很可能会减低准确性：积极地追求每个预测的机会很可能会导致大量的预测错误。

如果考虑两种极端情况，掌握准确性和覆盖率之间的折中就会比较容易。在一种极端情况下，预测器不加选择地预测所有的指令，从而获得100%的覆盖率。这种情况将导致较低的准确性，因为很多指令本身是不可预测的，因而会导致预测错误。在另一种极端情况下，预测器不会预测任何指令，这样也就消除了所有的预测错误，获得了100%的准确性。当然，如果没有任何可预测的指令被执行，那么这种情况导致的覆盖率为0%。设计者的任务就是在这两种极端情况下找出一个点，以便同时获得较高的准确性和覆盖率。

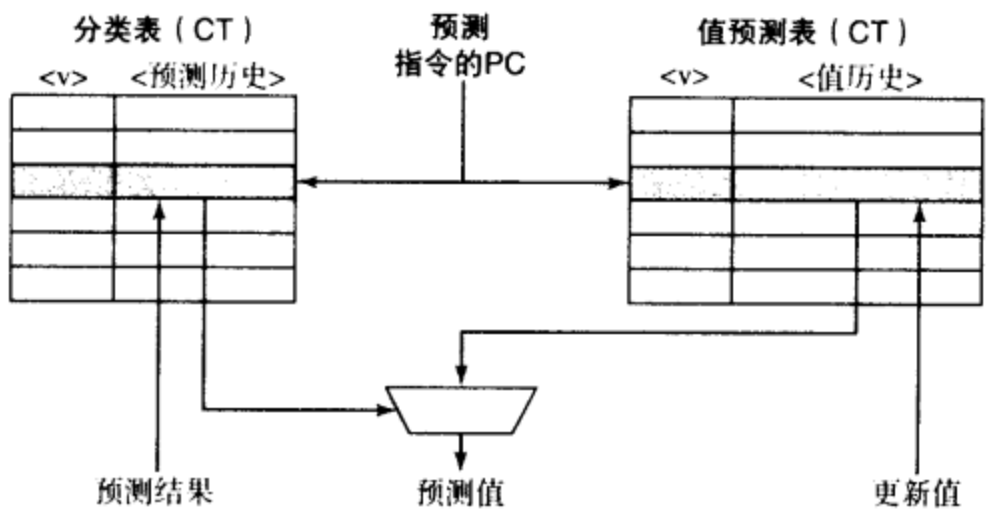
将值预测器的范围限制为某类特定的指令（例如load指令），或者其他动态或静态决定的命令子集，这样做会更容易地提高该子集的准确性和/或覆盖率，特别是在预算固定的情况下更加适用。

为了在准确性和覆盖率之间取得合适的平衡，值预测器的构建需要仔细权衡，必须考虑覆盖率发生变化时对性能的影响（例如，指令调度执行时覆盖率的任意改变，以及动态数据流图高度的变化），以及准确性的变化（例如，更少或者更多的预测错误）对性能的影响。这种分析将随着所考虑的微体系结构的详细结构和时序细节而改变，准确的折中分析需要详细的寄存器传输级（RTL）模拟。更好的覆盖率并不会总是提高性能，因为实际上只有相对较小的预测集合对性能起关键作用，因此这也使得分析进一步复杂。类似地，提高准确性也不会总提高性能，因为所消除的预测错误可能对性能影响也不是很关键。Fieds、Rubin 和 Bodik [Fields et al，2000] 最近的一份研究通过直接测量程序执行的关键路径定量地显示了这一点，该研究表明，较少的准确预测实际上减少了关键路径上的边。他们建议将值预测器的范围限制为程序数据流图中关键路径上的指令（例如最长的）。

8.4.3.1 预测准确性

一种自然的值预测策略就是简单地认可预测策略产生的所有可能预测，将它们作为可预测的操作数提交给执行核心。然而，已发表的一些报告显示，值预测器的准确性变化很大，有时正确预测率只有18%。显然，以自然的方式采用不正确的预测是不明智的，预测错误将会导致性能问题。虽然预测错误恢复策略的实现从理论上说可以达到没有直接的性能损失，但是由于实现上的困难，这种方法实际上是不可行的（在8.4.4.5小节中讨论了一种可能的方法）。因此，自从值预测最初的提议开始，研究者就描述了提高预测器准确性的置信度估计（confidence estimation）技术。

置信度估计　置信度估计技术将置信度和每个值预测联系在一起，并使用这些置信度来过滤不正确的预测，以此来提高预测的准确性。如果预测超过某个置信度阈值后，则处理器核心将采用该预测值；否则，则忽略预测值并不带推测地执行，强迫与之相关的操作等待“生产者”完成计算后的结果。典型情况下，置信度是通过历史机制来建立的，该机制在推测正确时使计数器加1，推测不正确时将计数器减1或者重置计数器。尽管对每个表项对应多个计数器以及每个计数器对应多个表项的情况做过研究，但通常值预测单元中每个表项都和一个计数器相关联。图8.7中所示的分类表是置信度预测机制的一个简单示例。迄今为止，对置信度预测器设计空间的研究非常广泛，它和动态分支预测器的设计空间很类似（参见第4章）：设计参数包括选择单级还是多级历史；索引为预测结果的历史、PC值或者是散列组合；预测器表项状态机中的状态数目和转换函数，等等。即使图8.7中所示的相对简单的置信度预测策略，也能够消除所有预测错误的90%多，而同时只牺牲少于10%的覆盖率。



一个简单值预测单元（VPU）的内部结构。VPU包括两个表：分类表（CT）和值预测表（VPT），两个表都是直接映射的并且都是以预测指令的指令地址（PC）作为索引。CT中的项包含两个域，“有效”域可以是单个表示有效项的位，也可以是部分或者全部标志域，该域和PC的高位匹配来表明有效域；另一个是预测历史，预测历史是由一个或者多个位构成的饱和计数器。只要正确或者不正确地完成了一个预测，预测历史就相应地递增或者递减，它用来将指令划分为可预测的或不可预测的。分类用来决定某条特定指令的结果是否应该预测。增加饱和计数器的位数能够增加分类处理的滞后作用，并且忽略异常值和/或者破坏性干扰能够避免错误分类。

图8.7　值预测单元

8.4.3.2　预测覆盖率

度量值预测单元效果的第二个因素是预测覆盖率。最初提出的简单值预测器只是简单地记住特定静态指令产生的前一个值。图8.7显示的是这种最近值预测器（last value predictor）的一个示例。每次指令执行时都会用产生的结果更新值预测表（VPT）。作为更新操作的一部分，如果前一个值匹配实际结果，则分类表中的置信度会增加，否则减少。下次取同一条静态指令时会检索前一个值和相应的当前置信度。如果置信度超过一个固定阈值，则使用预测的值；否则丢弃该值。

简单的最近值预测器在通用的目标程序集上可以提供大概40%的覆盖率。通过更加复杂的预测器可以提供更好的覆盖率，这些预测器或者可以提供额外的上下文让预测器从多个先前值中选择（基于历史的预测器），或者能够发现可预测的序列并且计算出原来不可见而将来需要的值（计算预测器）。

基于历史的预测器　最简单的基于历史的预测器只记住特定静态指令最近写入的值，并且推测该指令的下一个动态实例会计算相同的结果。更复杂的预测器提供了相应的方法为每条静态指令存储多个不同的值，然后使用一些策略从这些值中选择一个作为预测值。例如，Burtscher和

Zorn提出的最近n值预测器（last-n value predictor），它使用了预测结果历史的策略从存储在值预测表中的n个值中选择一个。另一种策略是，通过Sazeides和Smith提出的有限上下文方法（FCM），预测器也存储多个值，而不是在程序执行时选择最近值的有限上下文，而不是严格按照PC值执行。这个值上下文跟分支历史寄存器所捕捉到的分支结果上下文很类似，该寄存器成功地用于实现两级分支预测器。FCM策略可以捕捉周期性的值序列，例如遍历一个链表所装载的指针地址集合。虽然存储多个值和其上下文的实现代价相当大，对某些负荷来说FCM可以达到超过90%的覆盖率。

计算预测器 计算预测器试图捕捉一条静态指令产生的值序列的可预测模式，然后计算序列中的下一个实例。它们与基于历史的预测器有着根本的区别，因为它们能够预测在前面程序执行中没有发生的值。Gabbay和Mendelson首先提出了跨距预测器（stride predictor），它可以检测到值序列中固定的跨距，通过把观察到的跨距与前一个值相加能够计算下一个值。跨距预测器需要额外的硬件：为了能检测跨距，需要使用一个32位或64位的减法单元来提取出跨距，还需要一个比较器来比较提取出来的跨距和前面的跨距实例。在值预测表中，需要额外的空间来存储跨距值和一些额外的置信度预测位来标识有效的跨距。最后，需要一个加法器将前一个值和跨距相加得到每个新的预测。跨距预测对于某些负载来说非常有效，然而，对于额外的存储、数学单元以及复杂性，其产生的代价是否值得则不十分清楚。

还有其他更先进的计算预测器，但迄今为止没有一种被正式提出。显然，计算预测器的设计空间中存在一个连续区域，该区域位于两种极端状态之间，即没有计算能力的、基于历史的预测和完善的预先执行。在完善的预先执行中，预测器可以得到所有体系结构状态作为上下文，预测器简单地预测实际程序的语义来提取计算结果。后一种极端状态显然既不实际又没有用处，因为它只是简单地复制处理器执行内核的功能，而我们感兴趣的是，是否存在一个中间地带使得至少有一个程序计算的子集能抽象出来，在这点上具有合理代价的预测器将具有很高的准确性。显然，复杂的分支预测器能够从许多程序中抽取95%或者更多的控制流行为。复杂计算性值预测器对程序数据流是否可以达到同样的目标仍然是一个尚未解决的问题。

混合预测器 最后，类似于第4章描述的混合或组合分支预测器，提出了将多个不同预测器组合成单一的不同策略。这种混合预测策略可能组合了最近值预测器、跨距预测器和有限上下文预测器，以便获得每种预测器的优点。混合预测器不但具有更好的整体覆盖率，而且也更加有效，同时能够实现更加高效、紧凑的预测机制，因为它们只是定位于静态指令的子集。

实现问题 一些研究考察了值预测单元不同的实现问题。这些问题包括值预测单元的大小、组织、可访问性以及对更新延迟的灵敏度，这些问题很难解决，尤其对复杂的、带计算的混合预测器更是如此。一些解决方案（例如利用巧妙的散列函数索引表）采用分体结构并实现了多个并行访问，它们都取得了不错的效果。最近的一份建议提出，将复杂的值预测器访问转移到完成时段，并且将该次访问的结果存储到一个简单、直接映射的表中或者直接存储到trace cache中，这能够将访问复杂性从对时间要求严格的流水线前端转移出来。另一个巧妙的建议避免了将值存储在单独的历史结构中，而是假定需要的值存在于寄存器文件中，并且只存储指向相应寄存器的指针。令人惊讶的是该方法相当好，特别是当编译器分配寄存器名并且对于寄存器中保存的值有所了解的时候。

8.4.3.3 预测范围

决定值预测单元效果的最后一个因素是预期的预测范围。值预测最初的提议严格集中在load指令上，将值预测的范围限制为通常被认为对性能影响较关键的指令集合上。通过预测和推测性地

使用那些load指令返回的值可以减少load延迟，这将提高性能并且减少了竞争激烈的cache端口带来的结构相关的影响，而且将被cache失效所阻塞的load指令与cache的失效处理并行进行，从而提高了存储器一级的并行性。

所提出的大部分预测策略都是以所有寄存器写指令为目标的。然而，也存在一些有趣的例外。Sodani和Sohi指出，不应该预测用于解析条件分支的寄存器内容，因为通常这些值预测的准确性比目前复杂分支预测器产生的特定预测的准确性要差。在最初的值预测工作中回避了该问题，最初的工作采用的是PowerPC指令集体系机构，该结构中所有的条件分支由专门的条件寄存器来解决。因为只预测通用寄存器，由错误的值预测推翻正确分支预测所带来的有害影响将保持在最小水平。在类似于MIPS和PISA（Sodani使用的）的指令集中没有条件寄存器，所以预测所有通用寄存器的策略也会预测分支源操作数，并且能够直接对分支解析产生负面影响。

有几位研究者提出，可以将值预测只集中在对性能影响关键的那些数据相关上。这样做有一些优点：避免了无用预测带来的额外工作；可以得到更好的准确性和覆盖率并且预测器的实现代价较低；可以减少或者消除无用预测中发生的预测错误。Fields、Rubin和Bodik［Fields等人，2000］提出了一种新颖的方法，他们通过监视乱序指令的执行导出了数据相关的临界值，从而论证了上述的这些优点。

8.4.4　使用预测值的推断执行

与指令重用一样，值预测也需要微体系结构的支持来使用提前可用的指令结果。然而由于值预测的推断本质，所以在所需要的支持上也有着根本的区别。在指令重用前，需要有一个重用测试来保证其正确性，在8.3.2.2小节中简单描述了微体系结构的变化，主要包括在乱序的超标量处理器中访问簿记结构的额外带宽。相反，值预测（本质上是推测性技术）则要求微体系结构提供更多支持预测错误的检测和恢复方面的支持。因此，值预测意味着对值推断执行、验证预测和预测错误恢复的支持。这里首先描述一种最低限度支持推测执行的方法，然后将讨论更多高级的验证和恢复策略。

8.4.4.1　直接的值预测

初看起来，使用推断值看起来很自然地与现代乱序执行的超标量处理器已经提供的结构相符合。首先，为了支持值预测，需要存储预测值并把它从值预测单元转发到相关指令，已经存在的重命名缓冲和重命名寄存器就非常适合该目的。第二，需要一种机制预测性地发射相关指令。标准的乱序发射逻辑经过少许改动就可以很好地做到这一点。第三，需要发现预测错误值的机制。显而易见的方案就是扩展保留站并为每条指令保存预测的结果，并提供从保留站、功能单元输出到比较器的额外数据通路，比较器验证将这些值是否相等，如果不等则发出预测错误的信号。最后，需要恢复预测错误的方法。如果像对待分支预测错误那样对待值预测错误，就可以简单地重复使用分支预测错误恢复机制来刷新预测指令，并重新取出预测错误指令之后的所有指令。令人惊讶的是，这些小小的改动在单处理器系统[1]中就足够保证其正确性，在预测器高度准确且预测错误相对较少的情况下，这些改动甚至能提供显著的加速。更加复杂的验证和恢复技术能带来性能更好的设计，但是需要增加额外的复杂性。这些技术将在下面的小节中讨论。

---

[1] 最近一份出版物讨论了为什么在保持cache一致性的多处理机中它们是不够的：实际上，在前一条load指令计算地址时，值预测允许相关指令先执行，这样将存在数据相关的load指令间的自然引用顺序给破坏了。对于依赖这种相关顺序来保证正确性的多处理器程序，在此处描述的这种简单的值预测策略下可能失败。对于进一步的讨论，感兴趣的读者可以参考［Martin等人，MICRO 2001］。

8.4.4.2 预测验证

预测验证和保证指令重用正确性的重用测试类似。换句话说，它必须保证对值进行预测的结果与由系统状态和该指令的语义所决定的实际结果匹配。最直接的验证方法就是执行被预测的指令，然后把执行结果和预测值进行比较。这样做必须为每个功能单元添加一个ALU宽度的比较器来验证预测值。因为通过比较器的延迟和通过ALU的延迟相当，所以大多数建议都认为，还需要一个额外的周期来确定预测是否错误。

预测验证有两个目的。首先是在预测发生错误的时候能够激发恢复动作，可能的恢复动作将在8.4.4.4小节中讨论。第二个目的更加微妙并在没有出现预测错误时发生：需要将“验证了一个正确的预测”这一事实告诉使用该预测结果进行推断执行的相关指令。依赖于不同的恢复机制，这些进行完推断执行的指令可能继续占用处理器窗口中的资源，直到发现自己是非推断性指令为止。例如，在传统的乱序微处理器中，指令只能以程序规定的顺序进入发射队列和保留站。一旦它们被发射和执行，则不存在数据通路或者控制通路能够将它们重新放回发射队列重新发射。在这种微体系结构中，如果指令使用了预测的源操作数并推断地发射了，那么该指令应该留在发射队列中或者保留站中，以便将来需要用正确的操作数重新发射。因为发射队列槽是很重要的硬件，它对性能起着关键的作用，所以，及时地将指令的输入操作数没有发生预测错误这一消息通知给指令，对减少结构相关很重要。

正如上面所提到的，预测错误检测最直接的方法就是让指令等待操作数就绪后再开始执行，并且比较执行得到的结果和预测的结果。该方法存在的问题是指令的操作数本身可能就是预测的指令（也就是说，产生该操作数的指令可能是进行了值预测的指令，或者更加微妙：数据流中产生该操作数的指令前面的指令可能使用了预测值）。因为预测的输入操作数产生预测的输出，一个预测的值可以沿数据流图传播，传播的距离仅仅受限于处理器指令窗口的大小，从而产生了推测的操作数。如果预测操作数不正确，在验证一条带有不正确操作数的指令所产生的预测时，可能会导致验证应该不能通过时却发现验证结果正确这种情况，或者是验证应该失败的时候却意外地成功了。不管哪种情况都存在正确性的问题，因为验证导致不正确操作数的预测错误时总可以发现该操作数，所以可以发现前一种情况，然而后一种情况则只会导致不必要的恢复机制的调用。总之，正是这一原因，后者可能导致性能问题，因为指令无谓地被重复执行了。

推断验证 当使用预测的操作数来解析分支指令时也有类似的问题。在这种情形下，由于不正确的值预测，正确预测了的分支可能被解析成不正确的，从而导致分支预测错误并改变了方向。解决这两个问题的最直接的方法就是禁止对带有预测输入的预测进行验证（无论是值预测还是分支预测）。这种解决办法的不足在于，当一个正确的预测输入解析一个预测错误的分支，或者纠正一个值预测时失去了执行机会，对于这种性能影响的重要性并没有最后的答案。然而，当前趋向于实现深的执行流水线，这将对分支的预测错误非常敏感，这使得我们相信，实现任何延迟预测错误的分支解析都是错误的。

传播验证结果 为了将验证推迟到所有输入操作数都是非预测的时候，必须在适当位置提供一种机制，该机制将通知指令其输入操作数是否被验证，这带来了额外的复杂性。这种机制最简单的形式就是再定序缓冲（ROB），一旦成为ROB中最旧的指令，就意味着所有数据流的前项都已经验证过了并且可以验证该指令了。然而，将验证推迟到指令即将提交的时刻隐含着负面的性能影响，特别是对预测错误的条件分支指令。因此，需要一种将操作数验证状态沿数据流图传播的机制。存在两种基本的可选办法：验证指令时，验证状态沿着数据相关的边串行传播或者并行广播。串行传播可以附加在广播的结果上，结果广播用来在乱序执行中唤醒相关指令。并行广播更加昂贵，它意味着给所有预测数据流前面项的操作数添加标记，然后在预测被验证时广播这些标记。并行广播的速度很快，因为所有的相关链在链头部某个长延迟指令（例如cache失效）验证后的一个周期内，

都将变成非预测的。从讨论可以看出，这种即时提交能够立即释放发射队列或者保留站，而不是等待沿数据流图的串行传播，因而减少了结构相关。

8.4.4.3　数据流区域验证

数据流区域的概念为提高值预测验证的效果带来了很好的机会。回忆一下，数据流区域是由指令集合定义的数据流图的子图，这些指令集合从一个活跃输入集合出发是可达的。正如 Sodani 和 Sohi 提出的，只要区域的活跃输入集合符合重用测试，就可以重用该数据流区域。同样的属性也可用于验证数据流区域中发生的所有值预测的正确性。可以在值预测单元中合并进一种类似于8.3.4小节中所讨论的机制，该机制通过在值预测表中存储数据相关性的指针来构造数据流区域。然后，在数据流区域中的值预测调用将导致验证范围的缩小。也就是说，只要数据流区域机制能够保证区域内所有的预测彼此一致，就只需要验证对应于数据流区域中活跃输入的最初预测。一旦用传统方法验证了这些初始的预测，则整个数据流区域就被验证了，并且不再需要执行或者验证区域中剩下的指令。

该方法和数据流区域重用惊人地相似，它在值预测表中需要类似的机制来构造数据流区域信息并保证其一致性（在8.3.4小节中更加详细地讨论了这些问题）。然而，仍存在一个基本的区别：数据流区域重用要求数据流区域的活跃输入没有被干扰（如果重用测试是通过名称进行的），或者是没有变化并且在寄存器文件中可用（如果重用测试是通过值进行的）。将数据流区域和值预测相结合则避免了这些限制，这可以通过将重用测试无限期推迟，直到在处理器的执行窗口中活跃输入就绪加以实现。一旦所有活跃输入都验证完毕，则整个数据流区域都可通知为非预测状态并停止不再执行。在可重用数据流区域执行的指令中，这样可以显著地降低结构相关和功能单元之间的竞争。

8.4.4.4　利用重新取指的预测错误恢复

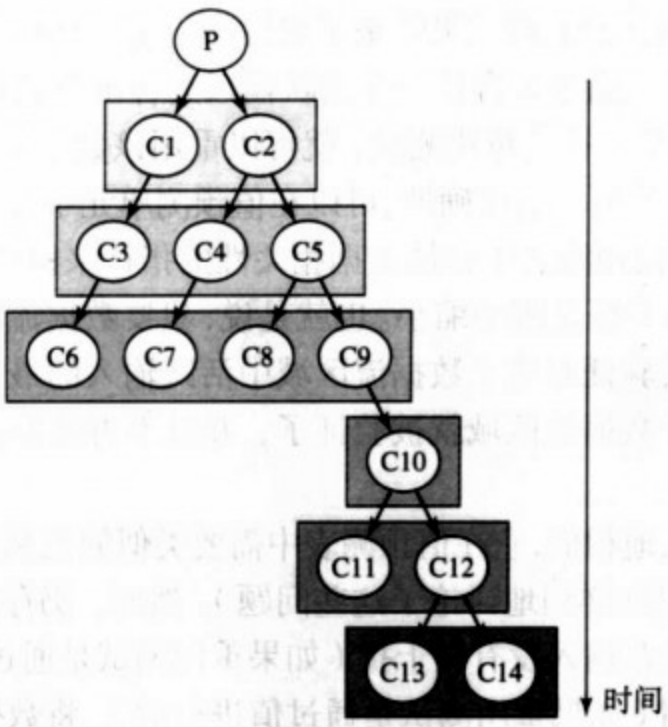
有两种方法可以用来从值预测的错误中恢复：重新取指和选择性重新发射。正如已经提到过的，基于重新取指的恢复建立在分支预测错误的恢复机制之上，目前，几乎所有现代超标量处理器中都存在这种分支预测错误恢复机制。在该方法中，将对值预测错误和分支预测错误同样对待：对于程序中预测错误的指令，它后面的所有指令都从处理器中清除，取指单元被重新定向并读取这些指令。体系结构状态则还原成预测错误指令之后的指令，并且保证重取的指令没有被任何预测错误的值所污染，因为这种预测错误的值在重取时不应该存在。

基于重取的预测错误恢复机制最吸引人的特点在于，假定重定向预测错误分支的机制已经存在的情况下，该机制对处理器所做的改动非常小。另一方面，它也具有明显的缺点，就是预测错误的代价非常大。研究表明，在采用重取策略来恢复预测错误的处理器中，获得性能提高需要高准确性的值预测。如果没有较高的正确值预测（通常由高阈值的置信度机制确定），额外的重取可能导致性能的下降。遗憾的是，高阈值的置信度机制同时也不可避免地减少了预测覆盖率，从而导致处理器设计不能够获得值预测带来的性能优势。

8.4.4.5　利用选择性重新发射的预测错误恢复

选择性重新发射为基于重取的恢复所带来的性能局限提供了可能的解决方案。利用选择性重新发射时，只有那些与预测错误值相关的指令才要求重新发射。实现选择性重新发射需要一种机制将预测错误信息沿数据流图传播到所有相关指令。正如传播验证信息一样，重新发射信息可以通过串行或者并行传播。串行机制很容易附加在乱序处理器中已有的、用于唤醒相关指令的结果总线上。在串行传播的情况下，传递重新发射状态的延迟，与从预测错误到必须重新发射的指令数据流的距离成比例。尽管不是很有把握，但可以想像，重发射的信息永远不可能赶上图8.8中所描述的预测

操作数波阵面，因为它们以同样的某一级速度沿数据流图传播。更进一步说，即使重发射信息最终赶上了预测值的波阵面，重发射信息的串行传播延迟会导致过多的沿预测操作数波阵面的无用执行。因此，研究者提出了基于广播的机制，并行地将重发射指令传递给所有相关指令。



推测性操作数的波阵面作为指令P的预测输出，沿着动态数据流图传播。它的消费者C1和C2将推测属性传递给它们的消费者C3、C4和C5，以此类推。预测验证状态的串行传播以类似的方式沿着数据流图传播。并行传播需要一个标识广播机制，它可以使得所有推测执行的相关指令在单个周期内得知验证状态。

图 8.8　推测性操作数的波阵面

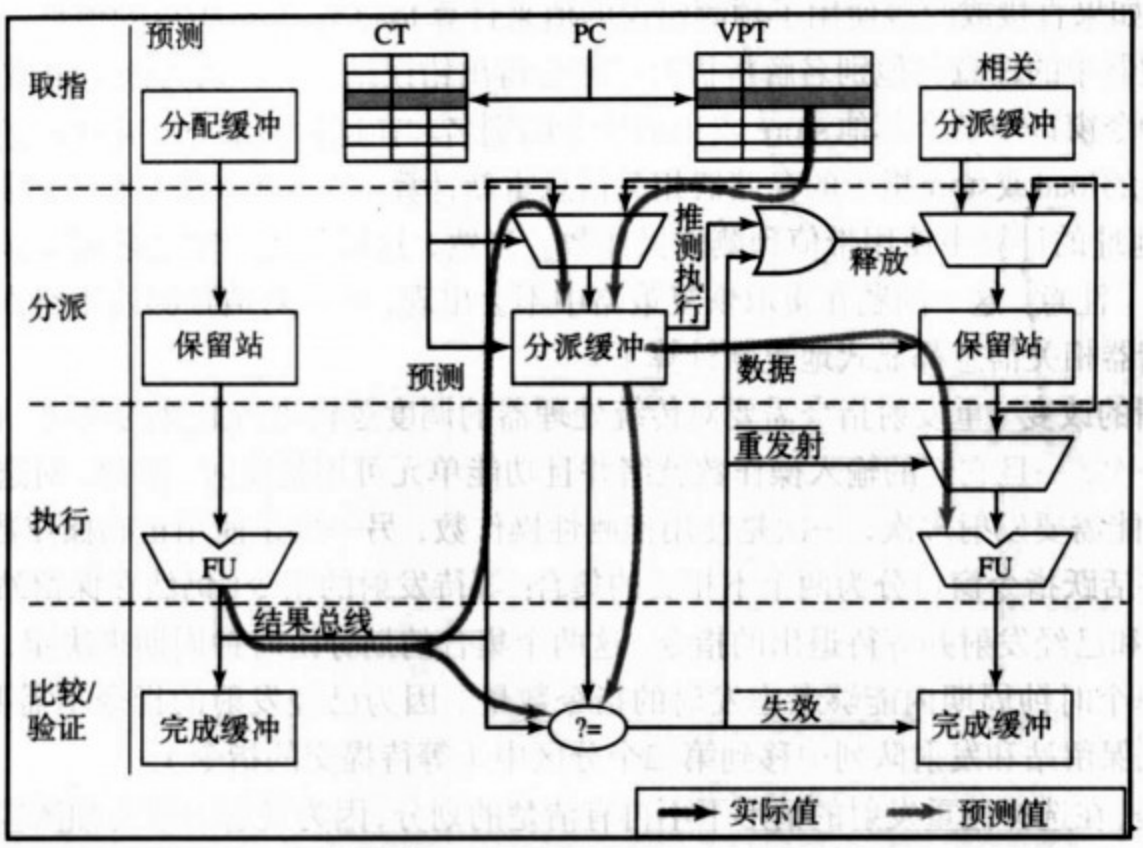
在这种并行机制下，将使用值预测操作数提供一个惟一的标记，所有与该操作数相关的指令必须将这些标记传递给它们的相关指令。如果发生了预测错误，将广播发生预测错误的操作数所对应的标记，以便数据流图的所有子节点都知道它们必须重新发射并用一个新的操作数重新执行。图 8.9 描述了使用并行广播的选择性重发射的一种可能实现。

选择性重发射预测错误的损失　在基于重取的预测错误恢复机制中，发生预测错误的损失和发生分支预测错误的损失相当，在最近流水线很长的处理器中，可以包含 12 个或者更多周期。选择性重发射的目的在于，通过减少从判断是否有预测错误发生到正确地重新执行数据相关指令之间的周期数来缓解损失。假设需要单个额外周期执行预测验证，显然，最好的情况是预测错误的损失为单个周期。也就是说，相关指令比没有预测时应该迟一个周期执行。

如果相关指令已经推测执行了，但还在保留站中等待一个预测操作数被验证时，才会产生相应的损失。因为对值进行比较在流水线结果出来以后又花费了一个周期的延迟，所以相关指令被重新发射并包含正确的值加以执行，这将比没有预测时晚一个时钟周期。另外，早先不正确的推测发射可能引起了结构相关，并妨碍了其他有用指令的调度或者执行。在相关指令仍然没有执行的情况下（因为结构相关或者未解决的数据相关）则不存在损失，因为值只要真正地就绪，相关指令就可以发射并验证值比较操作是否以并行的方式执行。

数据流主动执行（Data Flow Eager Execution）　通过使用数据流主动执行，可以将发生预测错误带来的损失降为 0 周期。在这种策略中，只要非推测性的操作数就绪，相关指令就将推测性地重新执行。换句话说，即相关指令存在第二个“影子发射”（shadow issue），就好像前面没有发生推测发射一样。预测验证和第二次发射并行执行，在预测正确的情况下，影子发射将停止。否则，

影子发射将继续，就像没有发生推测一样继续执行。这样，发生预测错误带来的损失实际上为 0。当然，所有和预测值相关的指令数据流的影子发射消耗了相当多的额外执行资源，并可能消耗完可用的功能单元，从而导致计算变慢。然而，对一个有足够执行资源的多发射处理器，这可能是减少预测错误所带来的损失的一个可行性方案。预测置信度也可以用来控制数据流主动执行。当预测置信度比较高时，积极执行被禁止；置信度比较低时，积极执行可以用来减少预测错误带来的损失。



右边所示的相关指令使用左边指令的预测结果，因而能够在同一周期内发射执行。VP单元在取指段和分派段预测值，然后通过重命名缓冲推测性地将它们传递给随后的相关指令。相关指令能够马上发射和执行，但是在体系结构上并不能实现，并一直占用保留站直到它的输入不再是预测性的为止。推测性传递的值带有标记，表明它们所依赖的没有提交的寄存器写操作，这些标记会传递到相关指令的结果。同时，右边预测的指令执行，并且预测的值与实际值进行比较来验证预测值。一旦验证了一个预测，就会将它的标记传递给所有活跃的指令，所有相关指令可以释放它们的保留站并运行到完成单元（在预测正确的情况下），或者可以利用正确的寄存器值重新执行（如果预测不正确）。

图 8.9　使用选择性重发射的值预测示例

调度延迟的影响　在实现了 Tomasulo 算法的等价算法的正规乱序处理器中，指令调度决定是在选定指令实际执行前的一个周期内做出的。这样一种策略可以使调度器对动态事件做出立即反应，例如发现 store/load 别名或者 cache 失效，则在接下来的周期里发射另外一条无关的指令。然而，对于存储器相邻读写周期的两次起点间的最小时间间隔（cycle time），其限制导致现在的设计放弃了这一性质，并引入了在实际执行几个周期前就产生一个执行调度的指令调度器。这种被称为调度延迟的效应限制了调度器对动态事件的反应能力。当然，对值预测是否产生了错误的检测是动态事件，多种现代处理器设计（例如 Alpha 21264 和 Intel Pentium 4）中都包含了多周期的调度延迟，这必将导致值预测发生错误所带来的损失增大。简而言之，产生值预测所带来的损失实际上是调度延迟和验证延迟之和。因此，在具有 3 个周期的调度延迟和 1 个周期的验证延迟的处理器中，发生值预测错误将带来 4 个周期的延迟。然而，即使是在这种设计中，也可以利用数据流主动执行来减少发生预测错误带来的损失。当然，在这种方法中执行资源被占用的可能性会随着调度延迟增加，因为执行和停止指令数和该延迟成正比。

8.4.4.6 选择性重发射进一步的含义

存储器数据相关 选择性重发射要求发生预测错误之后的所有数据相关指令都重新发射。识别寄存器数据相关是一个相当简单的问题，但存储器数据相关的识别可能导致很微妙的问题。也就是说，存储器数据相关本身就是由寄存器值定义的。如果寄存器的值由于值预测错误被证明不正确，存储器数据相关信息可能需要重新构造来保证正确性，并确定哪条附加的指令实际上和值预测错误有关。例如，如果直接或间接使用了预测错误的值来计算 load 或 store 指令的地址，则 load/store 序列或者该处理器中的任意其他别名解析机制可能会得出错误结论，并认为该 load 或 store 指令是或不是处理器指令窗口中某个其他 store 或 load 指令的别名。在这种情形下，必须注意保证地址被值预测错误污染的load或store指令的存储器相关信息重新计算。另一种方法是在处理器中禁止在load或store指令地址的计算中使用带值预测的操作数。当然，这样做会严重地限制值预测提高存储器级并行的能力。注意，这一问题在重取恢复策略中不会出现，因为对值预测发生错误之后的所有指令而言，存储器相关信息都显式地重新计算了。

调度逻辑的改变 重发射指令需要对传统处理器的调度逻辑进行显著的改动。在正常操作中，指令只发生一次，一旦它们的输入操作数就绪并且功能单元可用就发射。然而，对选择性重新发射而言，指令可能需要发射多次，一次是使用推测性操作数，另一次是使用正确操作数。所有乱序的实际实现都将活跃指令窗口分为两个不相交的集合：等待发射的指令（仍然在保留站或者是发射队列中的指令）和已经发射并等待退出的指令。这两个集合的划分由时钟周期来决定，时钟周期的要求决定了在单个时钟周期内能够考虑发射的指令数量。因为已经发射的指令不需要考虑重发射，所以它们将从保留站和发射队列中移到第二个分区中（等待提交的指令）。

遗憾的是，在选择性重发射的情形下不再有清楚的划分，因为发射时带有预测操作数的指令可能需要重新发射，因此不应该离开发射队列或者保留站。该问题有两种解决方法：将预测性发射的指令从保留站移出，但是同时提供额外的机制在需要重发射时将它们重新插入保留站；或者将它们保留在保留站中直到输入操作数不再是预测的。第一种解决方法为前端控制和数据流带来了很多复杂性，而且必须处理可能的死锁情形。当一条指令需要重新插入保留站进行重发射，而和该指令相关的新指令将保留站全部占满时，就会发生死锁，没有可用的保留站而且也不会再有变为可用的保留站，这是因为所有新的指令都在等待旧指令的执行，而旧指令并不会向前执行，也永远不会释放，从而导致系统死锁。注意，基于重取的恢复没有这种问题，因为所有新的相关指令在预测错误发生时都被清除出了保留站。

因此，后一种手段（即推测发射的指令记录其在保留站的位置）被讨论的更多。然而，该方法需要一种机制将推测的操作数变为非推测状态。类似8.4.4.2小节中描述的并行或者串行机制能够达到这一目的。除了必须要跟踪操作数的验证状态带来的复杂性之外，该解决方案还有一个小问题，即它提高了保留站的占有率。没有值预测时，相关指令在发射的同一个周期就释放了它的保留站，也就是最后一个输入操作数计算出来后的一个周期。在这种策略下，即使带有值预测操作数的指令的发射早得多，但保留站却在操作数就绪之后还被多占用了一个周期，因为保留站直到预测的操作数被验证以后才被释放，也就是操作数计算出来下一个周期。

数据推测的已有支持 注意，对于没有实现值预测但确实支持其他形式数据预测（例如，推测 load 指令和前一个 store 指令没有别名关系）的已有处理器，可能已经对有限的选择性重发射提供了支持。Intel Pentium 4 就是这样一种处理器，它实现了选择性重发射来提供对推测的 cache 命中的恢复。这里，访问cache所得到的数据在没有验证标识匹配前就传递给了相关指令，标识匹配能验证cache是否命中。如果标识不匹配，则选择性地重新发射相关指令。如果这种类型的重发射策略已经存在，就可以使用它来支持值预测错误的恢复。然而，已有的选择性重发射机制一般能够

在较短的周期内就能解决预测情况（例如，标识不匹配或者别名解析）、这种状况减小了能够重用已有选择性重发射机制的可能性。预测窗口只有几个周期使得所预测操作数的波阵面（参见图8.8）只会沿着数据流图传播几个级别，这反过来又限制了能够预测发射的指令数量。如果选择性重发射机制利用了该属性，并以某种方式将其限制为只执行很少的相关操作，则它对值预测没有什么用处，因为值预测的预测窗口能扩展到几十个或者上百个周期（例如，当对某个cache失效的load指令进行了值预测时），并且能够占用处理器整个指令窗口。然而，反过来并不成立，如果处理器实现了支持值预测的选择性重发射，则可以重用这一机制来支持其他形式的数据预测恢复。

8.4.5　值预测的性能

大量已发表的研究报告都曾分析过值预测的性能潜力。所报告的性能影响差别很大，在最坏的情况下甚至能够降低性能，而在最好的情况下可以加速100%或者更多。可获得的性能严重依赖于前面提到的许多因素，包括机器模型的详细细节、流水线结构以及工作负载的选择。下面是一些影响性能的因素：

- 程序或者工作负载中值局部性的程度。
- 正确预测指令和使用结果的指令之间动态相关的距离。如果编译器已经将相关指令调度为距离较远，则使用值预测减少结果的延迟可能不会带来很多好处。
- 通过机器模型得到的取指速率。如果取指速率比流水线的执行速率快，则值预测可以显著提高执行吞吐率。但是，如果流水线受限于取指，值预测则没有什么作用。
- 值预测单元获得的覆盖率。显然，越多的指令得到预测，则可能会产生更多的性能好处。相反，较差的覆盖率带来的好处则是有限的。
- 值预测单元的准确率。因为发生预测错误能够减慢处理器的速度，所以获得正确和不正确预测之间的较高比率对获得重大性能将非常关键。
- 在流水线实现中，发生预测错误后导致的损失。从讨论可知，恢复策略（重取还是重发射）和恢复策略的有效性都能够对预测错误所带来的损失产生很大影响。一般说来，需要预测调度的流水线越长，则发生预测错误所带来的损失越大，也就对这种影响更加敏感。
- 数据流相关性限制程序的程度。如果程序的性能主要不是受数据相关性的影响，则通过值预测消除数据相关性将不会带来很多好处。例如，如果程序是受限于取指、结构相关或是存储器带宽，则值预测不可能提高性能。

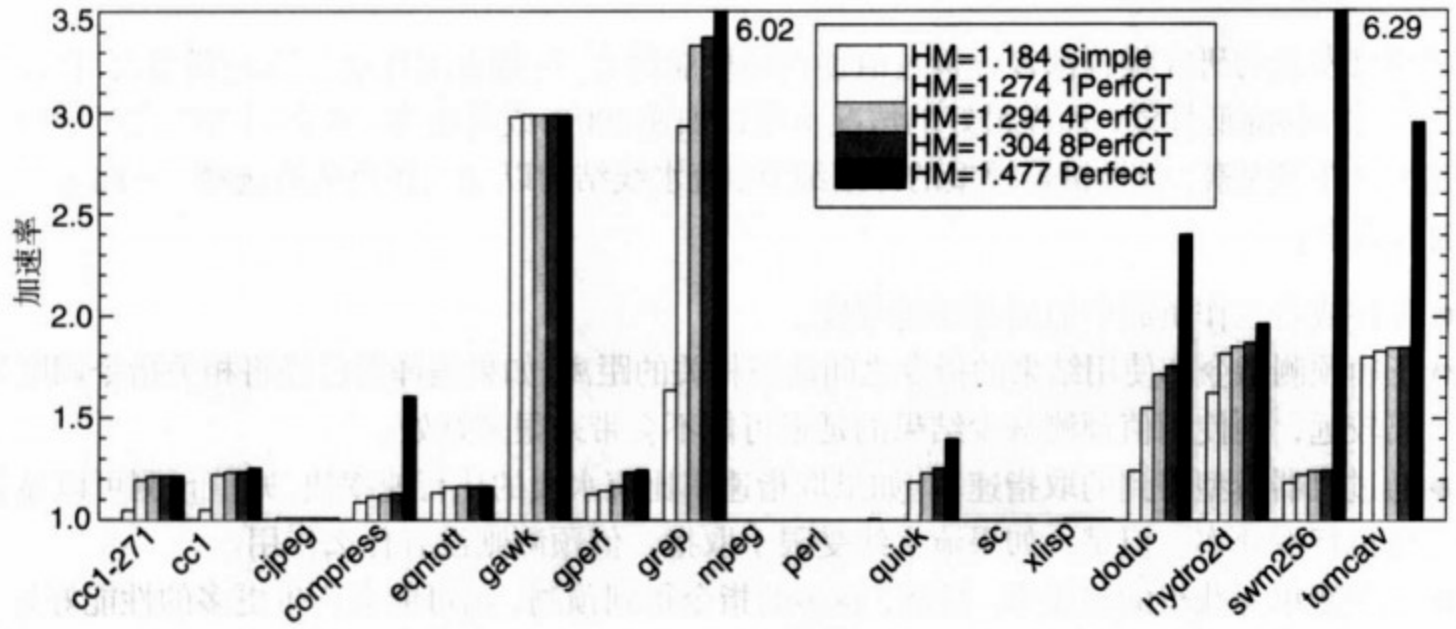
总的说来，值预测带来的性能影响还不是十分清楚。目前得出的结论是，在大量不同指令集、测试程序、机器模型和预测错误恢复策略下，据文献称可以获得非凡的加速比。

作为值预测的性能指标，图 8.10 显示了理想化的机器模型的一些性能结果。这一理想化的机器模型度量了一种可能的数据流限制，因为对于所有实际目标来说，该模型中的并行问题只被以下三个因素所限制：

- 分支预测准确率，该分支预测最小具有 3 周期的重定向损失。
- 取指带宽（每个周期一个成功分支）。
- 数据流相关性。
- 发生错误值预测时带来的一个周期的损失。

该模型反映了多方面的理想化性能，因为它不存在结构相关，并且发生预测错误带来的损失很低。然而，这里只考虑基于历史的值预测，所以后面利用计算或者混合预测器的模型则表现出

了高得多的加速比。图8.10显示了5个不同配置的VP单元的加速比，如表8.1所示。在表8.1中，属性标记为“完美”则表明类似于“完美cache”的行为，即假定一种总是产生正确结果的结构。也就是说，在1PerfCT、4PerfCT和8PerfCT配置中，假定存在一个神奇的置信度表（CT），该表能够正确地识别出所有可预测和不可预测的寄存器写。更进一步，在4PerfCT和8PerfCT的配置中，假定一个完美的机制从存储在值历史表中的4个（或者8个）值中选择一个正确的值。注意，这是Burtscher和Zorn提出的最近n值预测器的理想版本。进一步，假定Perfect配置总是能够正确地预测所有寄存器写的值，并有效地消除了所有数据相关。对这些配置而言，只有Simple是可以实现的VP单元配置，而其他种只是用来度量对VPT和CT预测准确性提高所起的潜在作用。



Simple配置使用了一个简单的最近值预测器。1PerfCT、4PerfCT和8PerfCT配置使用了完美的置信度，消除了所有预测错误并使得覆盖率最大，从1,4或者8个最近值的历史中分别选择一个。Perfect配置消除了所有真数据相关，显示出了整体性能的潜力。

图8.10 理想化机器模型的值预测加速率

表8.1 值预测单元配置

配置	值预测表		置信度表	
	直接映射项	历史深度	直接映射项	位/项
Simple	4096	1	1024	2位的up-down saturating计数器
1PerfCT	4096	1	∞	完美
4PerfCT	4096	4/完美选择器	∞	完美
8PerfCT	4096	8/完美选择器	∞	完美
Perfect	∞	完美	∞	完美

图8.10中的结果清楚地表明，即使是简单预测器也可以获得显著的加速比。Simple和1PerfCT配置之间的区别表明了预测的准确性非常重要，因为在极限情况下，它的加速比增加了50%。4PerfCT和8PerfCT情形表示跟踪多个值的、基于历史的预测器只能取得少量的好处。最后，Perfect配置表明对受限于数据流的测试程序而言，是可以获得惊人的加速比的。

8.4.6 结论

总之，基于值预测的推测执行提出了不同的策略来预测执行。研究者们描述了提高预测准确性和覆盖率的技术，并且将预测器的范围集中在值预测认为是最有用的地方。还研究了很多实现问题，包括预测器的设计和为值预测执行提供有效的微体系结构支持。同时，仍然存在许多未解的难题和未曾研究的领域，并且仍然没有真正集成了值预测的设计出现。值预测显示的性能潜力是否能够平衡其所需要的额外复杂性还有待时间的验证。

8.5　小结

本章研究了推测性的和非推测性的两种技术，并将寄存器数据流提高到标准数据流极限之外。这些技术是基于程序的值局部性属性，该属性描述了操作数在静态指令后面的执行中重新出现的概率。该属性用来从程序动态数据流图中移走计算，因此能够减小树的高度，使得程序具有压缩的执行调度。压缩的执行调度允许指令在数据流图中相应位置之前开始执行。只要这种情形发生，就可以说程序运行在数据流极限之外。数据流极限是由数据流图中指令的数目除以图的高度得出的比率。因为这些技术能够减小高度，所以执行速度就超过了数据流极限。

非推测性技术包括记忆法，记忆法是存储和重新使用无副作用计算的一种编程技术。指令重用是记忆法的指令级实现，只要它们的操作数和目前的实例匹配，就重用以前执行过的指令。指令块、trace和数据流区域的重用技术在控制流或数据流关系的基础上，将指令重用扩展到更大的指令组。这些技术有一个共同的特点，它们只有在保证正确性的时候才被调用，正确性是通过使用重用测试来保证的。相反，这里所研究的基于值局部性的其他技术，例如值预测，它本身具有预测性，只要能够正确地预测出计算的结果，它就会将计算从数据相关图中移走。由于必须提供推测性执行、预测错误的检测以及恢复机制，值预测引入了额外的微体系结构上的复杂性。

在真正的处理器设计中，还没有采用过这些技术。已发表的研究表明，获得惊人的性能提高是可能的，但是工业界似乎发现，扩充已有的设计增加对这些技术的实现不能够带来足够的性能提高，额外的代价和复杂性是不值得的。只有时间能够判断出将来的微体系结构是否会使用这些技术，并真正地获得性能上的提高。

8.6　习题

1. 图 8.1 曾提出，使用诸如指令重用或者值预测之类能够克服真数据相关的技术，能够将 IPC 从 1 提高到 4。然而，介绍这些技术的文献指出的加速为从百分之几到百分之几十。确定并且描述一个制约该加速比的程序属性。
2. 与第 1 题中一样，确定并且描述至少一个因素，使得在最好情况下的加速比不会出现。
3. 假定在 PowerPC 620 中实现了整数指令的指令重用，并且假定需要在分派段执行基于值的重用测试。整数体系结构寄存器文件（ARF）和重命名缓存需要多少个读和写端口？
4. 与第 3 题中一样，假定在 PowerPC 620 中实现了指令重用，并且希望在分派段执行基于值的重用测试。设计重用缓存并将指令重用集成到620的流水线中，该结构需要多少个读/写端口？
5. 假定将在 PowerPC 620 的分派段执行基于名字的重用测试，并通过它建立一种指令重用机制来重用 load 指令。因为目前还不知道在 load 指令前所有正在执行的 store 指令的地址，存在几种设计选择：如果具有未知地址的 store 指令仍然在执行，则禁止 load 指令重用；将重用的 load 指令的分派推迟到 store 指令计算出它们的地址后；向下执行，让这样的 load 指令被重用并依赖于其他机制来保证正确性。从性能的角度考虑这三种方案。
6. 假设条件与第 5 题相同。PowerPC 620 微体系结构的哪些特点可以用来保证第三种情况的正确性？如果选择第三种方案，该指令重用策略还是非推测性的吗？
7. 在第 5 题所描述的情形中，比较 PowerPC 620 5 段流水线和 Intel Pentium 4 20 段流水线中 load指令重用的效果。描述的三种选择方案中哪一种可能在未来长流水线处理器中工作得最好？为什么？
8. 构造一个 load 值结果序列。在该序列中，最近值预测器将比 FCM 预测器或者跨距预测器工作得更好。计算出每个预测器在该序列下的预测率。

9. 构造一个load值结果序列，在该序列中，FCM预测器将比跨距预测器或者最近值预测器工作得更好。计算出每个预测器在该序列下的预测率。
10. 构造一个load值结果序列，在该序列中，跨距预测器将比FCM预测器或者最近值预测器工作得更好。计算出每个预测器在该序列下的预测率。
11. 考虑值预测器和分支预测器之间的相互作用。给定一个跨距值预测器和一个两极Gag分支预测器，该分支预测器的分支历史寄存器为10位。编写一段C程序代码片断，在代码中，跨距预测器可以纠正分支预测器产生的预测错误。
12. 进一步考虑值预测器和分支预测器之间的相互作用。给定一个最近值预测器和一个两极Gag分支预测器，该分支预测器的分支历史寄存器为10位。编写一段C程序代码片断，在代码中，最近值预测器会错误地解析分支预测器正确预测的分支。
13. 假设一个值预测器会错误地重定向正确预测的分支，指出并讨论至少在两种微体系结构方面可供选择的方法来解决这一问题。
14. 假定在PowerPC 620中实现了整数指令的值预测，那么整数体系结构寄存器文件（ARF）和重命名缓存需要多少个额外的读/写端口。
15. 如第14题所示，假定在PowerPC 620中实现了值预测，并决定用全局广播的重发射作为恢复机制。在这种机制中，每个执行中的指令必须精确知道前面哪条指令和它相关，无论该指令是和它直接相关还是数据流图中通过多级间接和它相关。针对PowerPC 620，设计一个RAM/CAM硬件结构来跟踪这个信息，并且在检测到预测错误时能够选择性地重新发射。该结构需要多少个写端口？
16. 对于第15题中的硬件结构，试计算硬件结构的大小（它需要存储的位数目）。在一个更积极的微体系结构中，例如Intel P6，P6允许40条指令同时执行，该硬件结构的大小会发生什么变化？
17. 在图8.10所示数据的基础上，gawk测试程序在使用了更加积极的值预测策略时并没有获得更高的加速比，试解释其中的原因并加以证明。
18. 在图8.10所示数据的基础上，swm256测试程序在Perfect值预测策略下，其加速比并没有出现显著的提高，试解释其中的原因并加以证明。
19. 在图8.3和图8.10所示数据的基础上，解释测试程序sc中存在的矛盾：尽管它的寄存器写大概有60%是可预测的，但实现值预测并没有带来性能的加速。讨论至少两种导致这种情况可能发生的原因。
20. 给出第19题的答案，完成一些可以操作的试验来证明你的假设。
21. 在8.4.4.1小节中描述了死锁的情形。描述一种可能的解决方案，在不需要所有推测发射的指令驻留在保留站的情况下防止死锁发生。比较所提出的解决方案和另一种可选择的方案，该方案将指令驻留在保留站中直到它们是非推测性时为止。

第9章　执行多线程

9.1　介绍

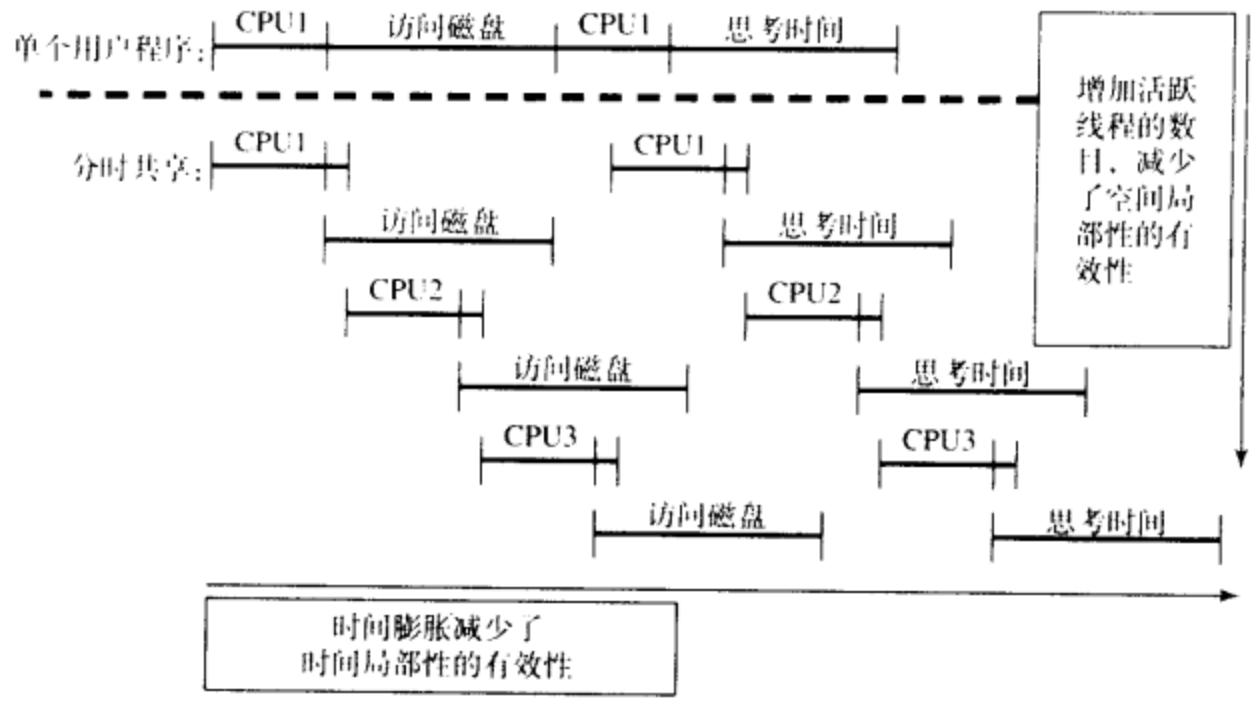
到目前为止，我们对高性能处理器的研究主要集中于加速单个线程的执行速度，也就是说，将重点放在减少单个串行程序的执行时间上。前面已经讨论过，影响执行时间的因素主要包含三个，而且它们是相互关联的：处理器周期时间、可利用的指令级并行以及程序的指令数目。电路设计技术的发展以及对体系结构的权衡，可以减少时钟周期时间。可利用的指令级并行会受编译技术和微体系结构技术的影响。先进的编译技术可以减少指令相关，先进的微体系结构技术，例如分支预测或者值预测，可以减轻控制和数据相关的负面效应。最后，程序的指令数目取决于算法的改进、编译技术的进步以及所采用的指令集的基本特性。所有这些技术都基于执行单个线程的假设，虽然处理器从开始到结束以连续的方式执行完程序的静态控制流，积极化解控制相关和数据相关，但是仍然保留了很大部分的顺序执行。

在这一章中，我们将考虑另一种提高性能的方法。实际上，这种方法已经广泛地应用于真实的系统。这种方法称为线程级并行（thread-level parallelism），它主要用于提高单个处理器或者多个处理器的吞吐率或处理指令的带宽。线程级并行思想来源于一些早期的分时共享计算机系统。这些早期的系统将相对较快的CPU和相对较慢的输入输出设备（最慢的I/O设备是程序员或者终端前的操作员）耦合在一起工作。由于CPU非常昂贵，而慢速的I/O设备如终端相对较便宜，操作系统的开发人员创造出分时共享的概念，允许多个I/O设备连接到一个CPU，以时间片的形式共享这个CPU。这就允许昂贵的CPU在当前线程遇到长延迟的I/O事件（例如，读磁盘或者等待终端用户键入命令）时，将上下文切换到另一个用户线程。因此，只要有其他用户线程等待执行，系统中最昂贵的资源——CPU就保持繁忙。时间分片策略——保证公平访问CPU的时间片，通过软件在操作系统中实现，因此执行时就引入了额外的切换上下文开销。因此，单个线程的执行时间（或者单个用户所看到的延迟）实际上有所增加，因为现在增加了操作系统进行上下文切换管理的开销。然而，处理器总的指令吞吐率将会提高，因为，在等待长延迟的I/O事件结束时，CPU执行了另一个线程的指令，而在以前它将处于空闲的状态。

从微体系结构的观点来看，这种分时共享的方式给处理器的设计者带来了挑战。由于多个独立的线程是交叉执行的，这就可能破坏基于空间和时间局部性原理的cache和其他数据结构。而且，线程内的分支和值预测冲突可能对这些结构造成极大的压力，并降低其效率，特别是当这些结构不是很大的时候。最后，大量的线程（在现代高端分时共享系统中，可能有几万到几十万的线程）很容易超出传统存储系统能提供的容量和带宽，这就要求设计中要有很大的二级和三级cache以及巨大的存储器带宽。这些影响如图9.1所示。

如果多个并行的线程之间存在共享的数据，对共享数据的访问必须按照定义好的顺序进行，而且要能够重复进行，这称为串行化或者同步。否则，程序的行为就不确定，甚至出错。我们将在9.2节中考虑一些简单而且广泛使用的串行化或同步机制，所有这些机制都依赖于硬件提供的原子（atomic）操作。当一个操作的所有子操作按照一个不可分割的整体执行时，我们称这一操作是原子操作。也就是说，这些子操作要么全部执行并且没有被其他操作或者处理器打断，要么一个

都不执行。现代处理器支持的硬件原语能够用于实现多种原子操作，使多个处理机或者线程能够进行正确的同步



在这个例子中，三个用户程序分时共享CPU，这将对CPU的使用与磁盘访问延迟和其他交互用户的思考时间重叠起来。这样使得CPU总是忙，因而提高了总的吞吐率，但是也增加了单个用户程序的延迟。这是由于上下文切换开销和排队延迟（当另一个用户程序占用CPU时的等待延迟）所导致的。分时分享对时间和空间局部性将产生不利的影响。

图 9.1 CPU 的分时共享

从体系结构的观点来看，分时共享的负载为建立具有扩展能力的系统创造了机会。也就是说，大量可以同时执行的线程促使了多处理机系统的建立，因为操作系统能够很容易地将多个线程分布到多个处理机上执行。建立多处理机系统需要设计者解决许多权衡问题，主要是关于存储器子系统的权衡，以及存储器如何给每个处理机提供一致的视图。我们将在 9.2 节中讨论这一问题，并将简要地描述为支持这样的存储器视图，现代多处理机设计中必须提供的一致性接口的主要特点。

除了在多个独立的处理器上同时运行多个线程的系统以外，人们还提出并实现了在单个处理器上交叉执行多个线程的方案。这样的多线程处理器有多种不同的形式，如细粒度多线程，即每个周期或者每几个周期切换一次线程上下文；粗粒度多线程，即只有遇到长延迟的事件，如 cache 失效时才切换线程；还有同时多线程，它允许多个线程的指令在一个乱序执行处理器的执行窗口中同时混合执行。我们将在 9.4 节中讨论多线程处理器的设计和实现中的一些问题。

多处理机系统的提出在并行算法领域开辟了广阔的研究空间，也就是使用多个相互合作的线程更快地解决问题。许多重要的问题，特别是对大量数据进行规则运算，很适合并行实现。然而，这类研究的最终目标（串行程序的自动并行化）还没有实现。但是某些算法的自动并行化已经解决，并且已经大量应用于科学数值计算，这些应用具有可预测的控制流（如具有确定迭代次数的嵌套循环），以及可静态分析的存储器访问模式（如顺序读取一个浮点多维数组）。对于这些应用，并行化编译器能够将总的计算量分解到多个独立的线程中去，比如将总的数据和循环划分成几部分，在线程之间进行分配。自然，划分算法必须注意避免在并行线程之间发生数据相关冲突，可能还需要使用同步原语来保证正确性。科学数值问题的自动并行化多年前就已经解决，并且在这一领域内的许多应用都很成熟。

然而，对于典型的非数值计算串行程序，很难通过编译时的自动并行化来提取线程级的并行。也就是说，对于具有不规则的控制流，或者访问数据的方式不可预测，或者充满了对指针数据结构的访问，很难从原始程序控制流的各个部分静态地确定存储器数据相关。对这样的代码进行自动并

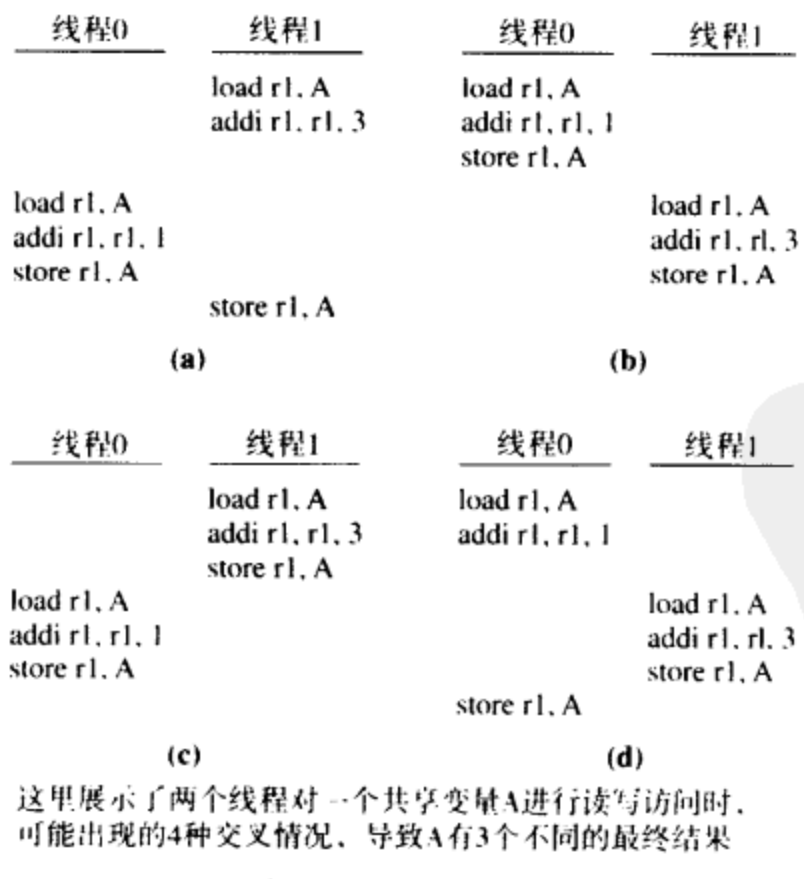
行化则很困难，因为编译时没有关于控制流和数据相关的准确信息，将串行程序划分成多个并行且独立的线程几乎是不可能的。

最近，一些研究人员提出，可以在运行时进行串行程序的自动并行化，而不是在编译时进行；或者提供足够的硬件支持，有效地解决多线程执行的问题。这种方案称为隐式多线程（implicit multithreading）。隐式多线程的方法包括动态多线程（dynamic multithreading）。这种方法完全依赖于硬件自动确定并推测产生隐式线程；同时还包括多标量（multiscalar）方法，这种方法使用硬件和编译相结合的办法达到同样的目的；另外还包含线程级推测（thread-level speculation）。这种方法依赖于编译器创建并行线程，但是由硬件提供对数据相关冲突的检测。我们将在9.5节中讨论隐式多线程的一些实现方法。

研究人员还提出了多线程执行的另外一种机制，即预执行（pre-execution）机制。预执行使用另一个上下文预先执行线程，但只执行关键部分，从而在主线程执行之前预取出数据和指令，从而化解难以预测的条件分支。现在人们已经提出了一个类似的方法并用于错误发现和容错执行。我们将在9.6节中讨论这些方法和相关的实现。

9.2　共享存储器线程的同步

分时共享的负载可能在多个并发的活动进程之间共享数据，对共享数据的访问必须串行化，并按照一种正确定义的可重复的方式进行。否则，负载的行为可能不确定，甚至出错。图9.2说明了两个线程对一个共享变量A进行读写时，可能出现的4种交叉方式。这4种方式的任何一种都可能出现在一个分时共享系统中，这样的系统将交叉执行这两个线程。假设A的初始值为A = 0，根据不同的交叉访问情况，A最终的值可以是3［图9.2（a）］，4［图9.2（b）和图9.2（c）］或者1［图9.2（d）］。显然，正确编写的程序应该有一个可预测和可重复的结果，而不是决定于操作系统的任务分派策略。这个简单的例子说明了在共享存储器线程之间进行正确同步的必要性。



这里展示了两个线程对一个共享变量A进行读写访问时，可能出现的4种交叉情况，导致A有3个不同的最终结果

图9.2　对同步的需求

现代处理器支持硬件原语，可以用于实现各种原子操作，使得多个处理器或者线程能够进行正确的同步。这些原语由硬件提供对原子操作的支持。称一个操作是原子的是指它所有的子操作都按

照一个不可分割的整体执行。也就是说，要么所有的子操作都完成，并且在执行过程中没有被其他操作或线程打断，要么一个都不执行。表9.1总结了三种实现中常用的原语，这些原语都可以用于共享存储器线程的同步。

对于表9.1中第一个原语fetch-and-add，其操作是从存储器读出一个值，加上一个操作数，再将结果写回存储器。硬件保证这个动作序列是原子的。实际上，处理器必须不停地重试，直到在任何其他线程覆盖这个共享变量之前，成功地将结果写回存储器。图9.2中的代码片断可以重写为“fetchadd A, 1”和“fetchadd A, 3”，分别对应于线程0和线程1，如图9.3（a）所示，从而得到一个具有确定可重复行为的共享存储器程序。在这种情况下，惟一可能的结果将是A=4。

表9.1 一些常用的同步原语

原语	语义	注释
fetch-add-add	原子读→加→写回操作	允许原子增量，可用于实现互斥操作的同步锁
compare-and-swap	原子读→比较→条件写回	只有当读操作返回预期的值时才写回
load-linked/store-conditional	原子读→条件写回	只有当读/写指令对是原子操作时才写回，即中间没有插入其他写操作

对于第二个原语compare-and-swap，操作是读出一个值并与另一个操作数比较，如果读出的值与该操作数匹配，就将这个操作数写回存储器。这条原语允许程序员原子性地交换一个寄存器的值和存储器单元的值，而存储器单元中的值就是我们想要的值。如果比较失败，会设置一个条件标志。这条原语可以用来实现相互排斥地执行受保护的关键段。关键段只是一个指令序列，但是必须以原子的方式执行，即要保证在当前执行关键段的线程完成对整个段的执行之前，其他线程都不能进入关键段。例如，图9.2中的代码片断对A进行了改写，可以通过将改写设置为关键段来使操作原子化，同时用另一个锁变量来保护关键段。图9.3（b）说明了这一过程，其中，cmpswp指令检查锁变量AL，如果AL被设置为1，cmpswp指令失败，并且线程将重复执行cmpswp指令（这个过程称为旋转锁）直到成功。一旦cmpswp指令执行成功，线程将进入关键段并原子性地执行读、加以及存操作（因为相互排斥保证了其他线程不能同时执行受同一把锁保护的关键段）。最后，线程向锁变量AL写入0，表明线程完成了关键段的执行。

对于第三个原语load-locked/store-conditional（ll/stc），其操作是读出一个值，执行其他任意操作，然后试图将数据存到同一地址。如果中间插入其他任何一个线程的写操作都将导致写条件失败。然而，如果没有其他对同一地址的写操作，则load/store指令对能够原子性地执行并且能够成功写入。图9.3（c）说明了如何使用ll/stc指令实现对共享的存储器进行重写。在这个例子中，ll指令读出A的当前值并与另一个值相加，然后使用stc指令将和写回存储器。如果stc失败，线程则返回ll指令并直到该指令成功地执行，从而保证了改写的原子性。

图9.3中的例子都保证了相同的最终结果：存储器地址A中的值始终等于4，而不管两个线程什么时候执行，或者它们的访存如何交叉。原语的原子特性保证了结果的惟一性。

(a)

线程0	线程1
fetchadd A, 1	fetchadd A, 3

(b)

线程0	线程1
spin:	spin:
cmpswp AL, 1	cmpswp AL, 1
bfail spin	bfail spin
load r1, A	load r1, A
addi r1, rl, 1	addi r1, rl, 3
store r1, A	store r1, A
store 0, AL	store 0, AL

(c)

线程0	线程1
spin:	spin:
ll r1, A	ll r1, A
addi r1, r1, 1	addi r1, r1, 3
stc r1, A	stc r1, A
bfail spin	bfail spin

图9.3 使用（a）fetch-and-add；（b）compare-and-swap；（c）load-linked/store-conditional进行同步

从实现的角度来看，ll/stc指令对是这三个原语中最具吸引力的一个。因为它很接近已经实现的读写指令，从而能够很好地应用于曾详细讨论过的流水化和超标量实现。而fetch-and-add和compare-and-swap，就不是这样，因为它们要求两次不可分割的访存。因此，它们要求处理器的流水线进行专门处理。

现代指令集如MIPS、PowerPC、Alpha以及IA-64都提供ll/stc原语来同步。ll/stc原语的实现相当容易，惟一增加的语义是，每条ll指令必须记住它读取的地址。所有后续的写操作（包括多处理机系统中远程处理器执行的写操作）必须检查写地址，确定是否与ll指令读取的地址（称为链接地址）匹配。最后，当stc执行时，同样必须检查地址，确定是否与链接地址匹配。如果匹配，stc允许执行；否则，stc就会失败并设置条件码。这些改变要求提供的支持只比标准的读写指令多一点。因此，ll/stc指令很容易实现，并且能提供和fetch-and-add，compare-and-swap以及其他原语同样的同步能力。

总之，要保证共享存储器中多个线程能够正确、可重复地执行，适当的同步是必要的。这不仅适用于那些运行于分时共享的单处理器程序，同样适用于多处理机系统或者多线程处理器上的程序。

9.3　多处理机系统介绍

随着用户计算需求的增长，多处理机系统能够提供一个自然的、不断提高的升级途径。只要核心的用户程序能够提供线程级并行，给系统添加处理器，或者将较小的系统替换为较大的、包含更多处理器的系统，都能给用户提供一个直接增加计算能力的途径。其次，多处理机系统使系统供应商能够将单个微处理器的设计代价分摊到多种设计方案中，这些设计方案能够提供不同层次的性能和扩展能力。最后，使用一致共享存储器的多处理机系统能够提供与分时共享的单处理机兼容的编程模式，便于用户使用以前已经存在的应用程序，也便于开发新的应用程序。在这些系统中，硬件和操作系统软件综合起来提供给用户和程序员的功能界面，实际上是基于以下4点多处理机理想假设：

- **完全共享存储器**　即系统中所有处理机对所有物理存储器的访问都是平等的。
- **单位延迟**　即所有的访存请求都能在一个周期内满足。
- **无竞争**　即一个处理机的访存不会受到另一个处理机访存的影响。
- **写的瞬时传播**　即某个处理机写存储器导致的更新立刻对所有的处理机可见。

系统及处理机的设计者必须努力使系统尽量接近这些理想假设，从而满足用户对于性能和正确性的要求。显然，在实现这些目标时，诸如造价以及可扩展性等因素起着重要作用，但是一个优秀的系统必须很好地满足这些假设。

9.3.1　完全共享存储器，单位延迟以及无竞争

如图9.4所示，大多数提供一致存储器访问（UMA, Uniform Memory Access）的传统共享存储器多处理机系统使用一种“dancehall”的组织结构，一组存储器模块或者存储体通过交叉开关互连网络与一组处理机连接起来，并且每个处理机通过交叉开关访问存储器的延迟是相同的。这种方法的弊端在于交叉开关的价格，该价格会随着处理机和存储体的总数按平方增长，同时每次访存都必须穿过交叉开关。另一种方法是非一致存储器访问（NUMA, Nonuniform Memory Access），许多系统供应商现在都按照这种方法构造系统。在这种方法中，处理机仍然通过一个交叉开关互连网络连

接在一起，但是每个处理机有一个本地的存储器，它的访问延迟要低得多。在NUMA结构中，只有对远程存储器的访问才存在穿越交叉开关的延迟开销。

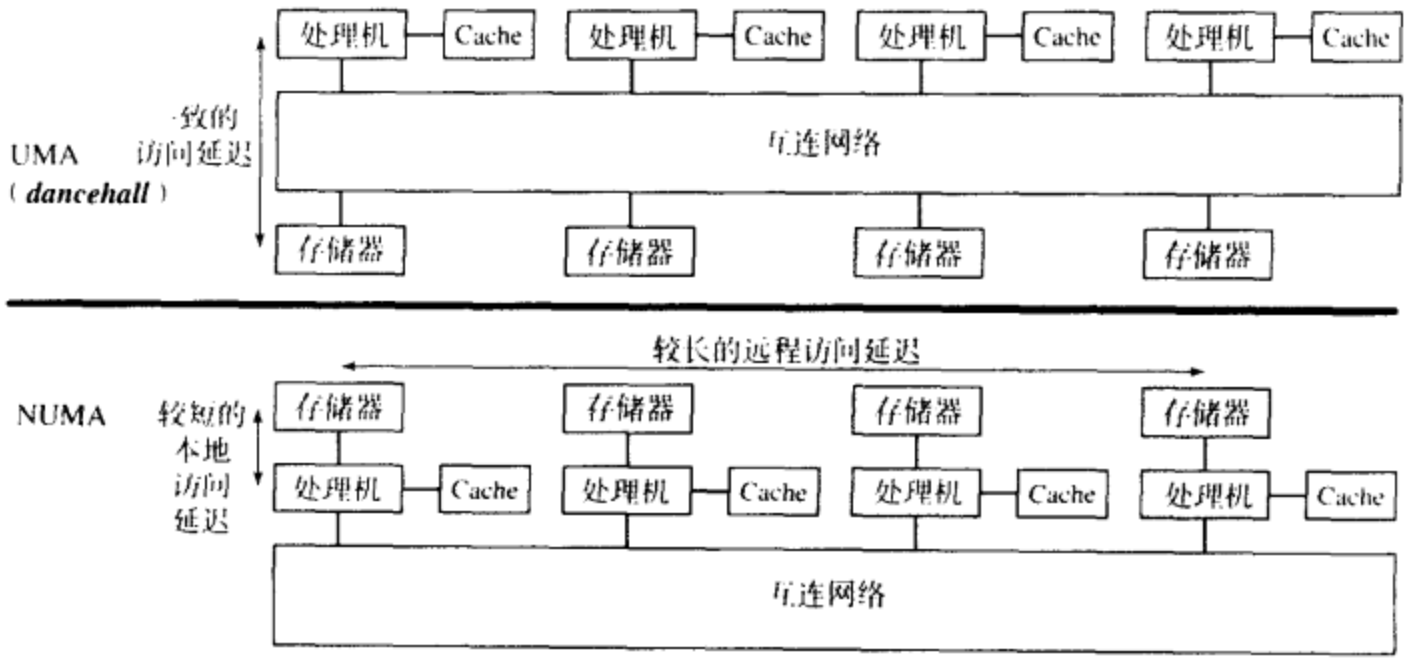


图 9.4 UMA 与 NUMA 多处理机体系结构

在UMA和NUMA系统中，与单处理机系统一样，都是通过cache来近似满足单位延迟的理想假设。cache能够同时满足对本地和远程存储器（NUMA）的访问。类似地，可以使用cache对访存流量进行过滤来减轻存储体之间的竞争。因此，对于在单处理机系统中不可缺少的cache，在多处理机系统中同样非常有用。然而，cache的存在给处理存储器写操作带来了一定的困难，因为存储器写操作必须对其他处理机可见，或者传播给其他处理器机。

9.3.2 写操作的瞬时传播

在一个分时共享的单处理机系统中，如果一个线程改写了存储器某个单元的值，该线程以及任何其他线程都会立刻察看到新的值，因为这个值会存放在处理机的cache中。遗憾的是，对于多处理机系统，不再具备这个立即可见的特性，因为对同一地址的访问可能由其他处理机发出。由于这些处理机有自己的cache，可能包含同一cache行的私有副本，它们可能看不到其他处理机的更新。例如，在图9.5（a）中，处理机P1向存储器地址A写入值1。由于没有一致性的支持，在P2的cache中，存储器地址A的副本并没有被更新，P2读取A将会得到以前的值0。这就是经典的cache一致性问题。要解决这一问题，系统必须提供cache一致性协议，保证系统中所有的处理机能够察看到其他处理机的更新，这样每个处理机拥有一致的存储器视图。实现cache一致性有两种基本的方法：更新协议和作废协议，我们将在后面的章节中简要地讨论这两个协议。图9.5（b）和图9.5（c）分别对这两种协议进行了说明。

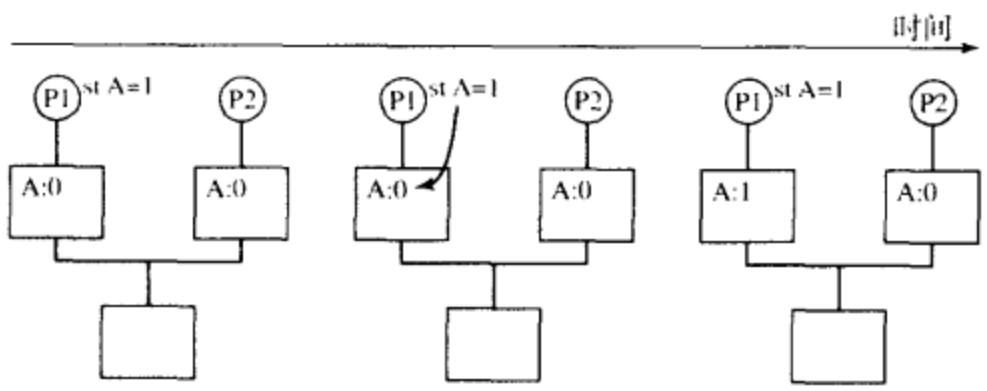
9.3.3 一致的共享存储器

对于共享存储器的多处理机系统，一致的存储器视图很难实现。但如果不能实现一致的视图，共享存储器程序的行为将不可预测，因为不同的处理机从存储器读到的值各不相同。我们已经论述过，一致性问题是由于写操作不能立刻自动地传播给其他处理机的cache造成的。有两类一致性协议可以保证写操作的结果对于其他处理机是可见的。

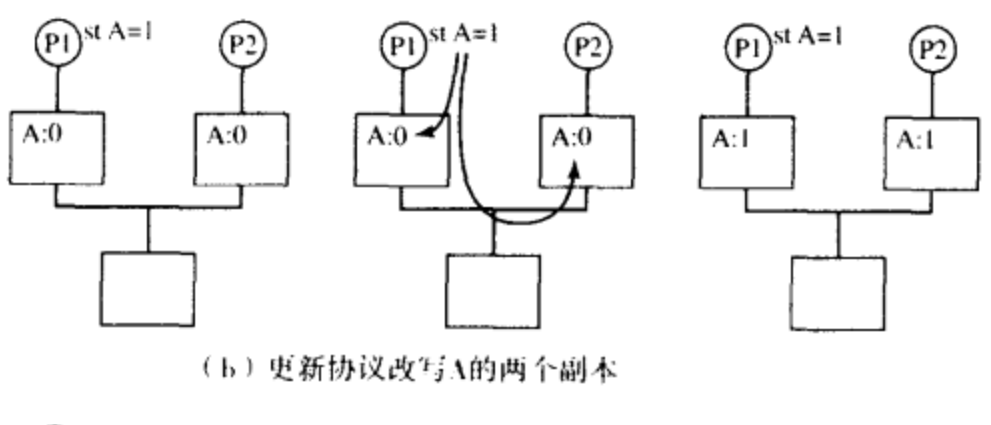
9.3.3.1 更新协议

早期提出的多处理机系统采用一个相当直接的办法来维护cache一致性。在这些系统中，处理机的cache采用写直达策略，所有的写操作不仅写入执行写操作的处理机cache，同时也写入存储

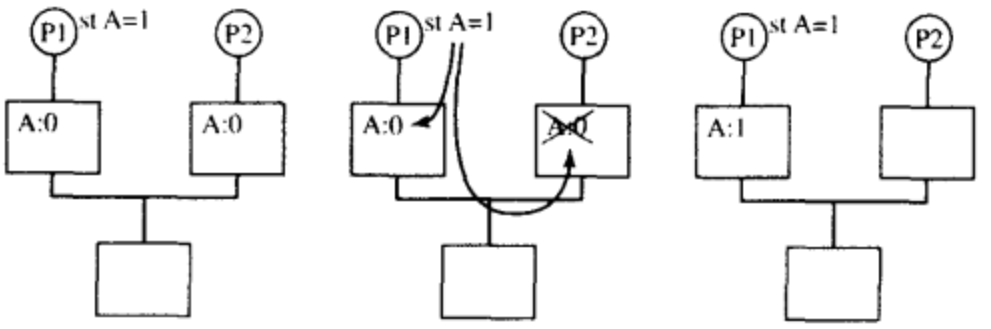
器　这种协议如图9.5（b）所示　由于所有的处理机都连接在一条共享的总线上，这条总线还将所有的处理机与存储器相连，所以所有其他处理机可以在共享总线上监听新的值，并更新它们的数据副本（如果有这样的副本）。这样就能够得到写直达的结果　实际上，这种更新协议是基于广播写直达策略的，即每个处理机的每次写操作都是写直达，不仅写入存储器，还写入任何拥有这个数据副本的处理机cache　显然，这种协议扩展性较差，仅限于数量较少的处理机，否则大量处理机的写直达流量会很快将存储器总线的可用带宽占用完



（a）非一致性协议：P2中A的值过期



（b）更新协议改写A的两个副本



（c）作废协议消除远程的过期副本

更新协议更新所有的远程副本，而作废协议去除远程副本

图9.5　更新和作废协议

可以对该协议进行简单的优化，对私有数据采用写回策略，即写操作只是将数据写入处理机的本地cache，当这一行被替换出cache时，才将结果写回存储器。然而，在这样的协议中，任何对共享cache行（存在于任何其他处理机cache中的行）的写操作仍然需要在总线上广播，以使包含共享副本的处理机能更新它们的副本　而且，一个远程读操作所读的行如果恰好是修改过的，那么在满足读请求之前，要将这一行先写回存储器。

遗憾的是，更新协议对于带宽的苛刻要求使得这种协议几乎消失，因此没有一个现代多处理机系统采用更新协议来维护cache一致性。

9.3.3.2　作废协议

现代共享存储器多处理机系统都采用作废协议来维护cache一致性。作废协议的基本前提很简单：任何时候只允许一个处理机执行写操作（这样的协议经常称为single-writer协议）　在该协议

中，试图对cache中某一行写数据的处理机必须使该行成为系统中惟一有效的副本，而其他处理机cache中的副本都要作废掉（所以称为作废协议）。该协议如图9.5（c）所示。简单地说，处理机执行写之前，首先要检查系统中是否还存在其他副本。如果有，处理机发出消息将其他副本作废。然后，处理机向其私有（专用）副本中写入数据。后续的、对同一行的写操作可以流水化，因为不需要检查是否存在其他副本。和单处理机的写回cache一样，修改过的行直到被替换出去的时候才写回存储器。然而，一致性协议必须记住存在这样一个修改过的副本，以防止其他处理机到存储器读取以前的值。而且，还必须能够从处理机cache中读出修改过的数据，以满足其他处理机对这个惟一副本的远程访问。

作废协议要求cache目录至少要为每行维护两个状态：修改（M）状态和作废（I）状态。如果某一行处于作废状态，那么这一行的数据无效，必须从存储器中读取数据。如果处于修改状态，处理机知道系统中没有别的副本（即本地的副本是专有副本），因此处理机能够对这一行进行读写。注意，如果被替换出去的行处于修改状态，必须写回存储器，因为处理机可能对它进行了写操作。简单的办法是在cache行的状态中加入一个dirty位，使得处理机能够区分哪些是处理机的专有行（通常称为E状态），哪些是专有并且修改过的行（M状态）。Macintosh桌面系统使用的IBM/Motorola PowerPC G3处理器实现了这种作废协议。

注意，如果只有这三种状态（MEI），cache行不能同时出现在多个处理机的cache中。因此，需要解决这一问题，允许可读的副本能够出现在多个处理机的cache中。多数作废协议还包括一个共享状态（S），这个状态表明，系统中可能存在多个可读的副本。如果一个处理机要向处于S状态的cache行写数据，必须首先将这一行提升到M状态并作废其他远程的副本。

图9.6显示了一个简单的MESI一致性协议的状态表和转换图。每一行代表4种状态（M，E，S或者I）中的一种，针对每种类型的总线事件，每一列总结了一致性控制器必须执行的动作。引起cache行发生状态转换的事件包括本地访存（读或写）、远程访存（总线读，总线写或者总线状态升级）以及本地的cache行替换。cache目录或者标识阵列维护每一个cache行的MESI状态。注意，此时允许每个cache行在不同时刻可以具有不同的状态，并使包括私有数据的cache行保持为E状态或者M状态，而包含共享数据的行处于S状态并且可以同时存在于多个cache中。MESI一致性协议支持single-writer并保证了一致性，同时还支持只读数据的共享，以及在发生本地写时，从专有状态（E）提升到修改状态（M）（不需要总线的状态升级消息）。

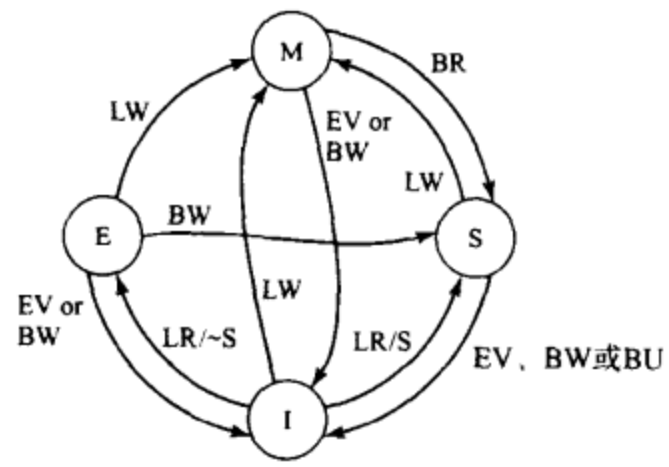
9.3.4 实现cache一致性

维护cache一致性需要有一套记录cache行状态（即MESI）的机制，从而能够正确处理对这些行的访问。保存一致性状态最方便的场所是在cache的标识存储器中，因为必须在cache中对状态信息进行维护。然而，cache行的本地一致性状态要对系统中其他处理机可见，以满足它们对这些行的访问。因此，实现cache一致性时必须提供一种分布访问方式。主要有两种实现方法：监听实现和目录实现。

9.3.4.1 监听实现

实现一致性的最简单的方法是利用监听机制。在监听实现中，所有由一致性协议引起的片外地址事件（即作废协议中的cache失效和作废）将共享总线，这对系统中所有其他处理机都是可见的。在规模较小的系统中，地址总线是电路共享的，当这些命令放到总线上时，每个处理机都能察看到所有其他处理机的命令。更先进的点到点互连机制也能实现监听，它通过层次化的监听互连机制，将所有命令反映给所有的处理机。为简单起见，我们将只讨论地址总线的机制。

当前状态s	事件和本地一致性控制器的响应和动作（S表示下一状态）					
	本地读（LR）	本地写（LW）	本地Eviction（EV）	总线读（BR）	总线写（BW）	总线升级（BU）
作废（I）	发出总线读 如果没有共享则 s' = E 否则 s' = S	发出总线写 s' = M	s' = I	无动作	无动作	无动作
共享（s）	无动作	发出总线升级 s' = M	s' = I	响应共享	s' = I	s' = I
专有（E）	无动作	s' = M	s' = I	响应共享 s' = S	s' = I	错误
修改过（M）	无动作	无动作	将数据写回； s' = I	响应dirty位； 将数据写回； s' = S	响应dirty位； 将数据写回； s' = I	错误



为了响应本地和总线事件，一致性控制器可能要改变cache行的一致性状态，可能还要取出或提供cache行的数据

图9.6　MESIcache一致性协议的例子

在监听实现中，一致性协议规定了一个处理机是否响应总线命令以及如何响应总线命令。例如，一个远程处理机对当前被修改过的cache行发出读请求，这将导致cache控制器将这一行从本地cache中直接传送给发出请求的处理机，也许还需要写回存储器，以这种方式请求将能够得到最新的副本。同样，一个远程处理机对当前处于共享状态的行发出作废请求，将导致控制器更新目录表项并将这一行标记为作废。这将防止本地处理机以后从cache中读取失效的数据。

监听实现的主要缺点在于可扩展性较差。假设系统中每个处理机以一定的速率产生地址总线事务，可以看到，必须监听的输入地址总线事务直接和系统的处理机数目成正比。

$$\text{输出监听速率} = s_o = \langle \text{cache 失效速率} \rangle + \langle \text{作废请求速率} \rangle \quad \text{（公式9.1）}$$

$$\text{输入监听速率} = s_i = n \times s_o \quad \text{（公式9.2）}$$

也就是说，如果每个处理机每秒产生s_o个总线事务（包括cache失效导致的读请求和对共享行写操作导致的作废请求），系统中有n个处理机，那么每个处理机每秒必须监听ns_o个事务。每次监

听都要查找本地的cache目录并判断处理机是否需要做出反应（参见图9.6）。对于较大的n，查找总带宽是不可接受的。同样，对于将系统中的处理机连接在一起的互连网络，其带宽可能很快就被事务流量占用完。实际上，许多基于监听的多处理机系统的性能都受限于地址总线的带宽。基于监听的实现只能扩展到几十个处理机（最大为64个，如Sun Enterprise 10000），如果超过这个数字，就要为提高地址总线带宽付出昂贵的代价。

大规模的、基于监听的多机系统相对于处理机数目较少的系统，还存在访存延迟动态增加的问题。因为访存延迟决定于一致性响应的延迟，而不是DRAM的延迟。换句话说，对于较大的n，进行监听和从所有处理机收集监听响应的时间要比从DRAM读取数据的时间长，甚至对于具有较长远程访存延迟的NUMA结构系统也是如此。如果访存的数据通过推测的方式传给请求方，那么需要一直等到所有的处理机都报告它们不包含改写过的副本后，这些数据才能被认为是有效的。因此，在解决cache失效时，对性能起到决定作用的不是本地或者远程访存延迟，而是监听响应延迟。

9.3.4.2 目录实现

对于解决监听实现的可扩展性和访存延迟问题，常用方法是使用目录。在目录实现中，通过从存储器中分配空间来记录cache行的一致性状态。一致性状态存放在目录中，而目录和存储器放在一起。一致性状态表明某一行当前是否在某个cache中，另外还包括了指向所有位于cache中的副本的指针，这些指针存放在一个称为共享列表或者共享向量的结构中。共享列表可以是精确的（每一个共享副本都被单独指出），也可以是粗略的（多个处理机共享表中的一个表项，该表项表明这些处理机拥有这一行的副本），可以存储为链表或者大小固定的向量。精确共享向量的缺点是巨大的存储开销，特别是对于处理机数目很大的系统，因为共享向量的存储空间与处理机数目成正比。对于一个cache行为64字节、包含512个处理机的大型系统，共享向量的开销将达到100%。

带宽扩展 目录实现的主要好处是，目录的带宽随着存储器的带宽增长而增长：增加一个存储体以支持更大的数据带宽同时也增加了目录的带宽。另一个好处是，通过在目录中过滤掉一些命令，减少了对于地址总线带宽的需求。在目录实现中，地址命令首先送到目录，只有必要时才传给远程的处理机（即当这一行在远程cache中为脏时，或者向远程cache中的共享行写数据时）。因此，每个处理机需要接受地址命令的频率不再与处理机的数目成正比，但是与数据共享的程度成正比，因为只有当处理机拥有所请求cache行的副本时，它才接收地址命令。因此，目录实现可以用于建立包含几十到几百个处理机的系统。

访存延迟 最后，cache失效的延迟能够显著地减小，因为一旦存储器完成对目录的检查，就能提供非推测的数据。特别是对于NUMA系统，这一点显得尤为突出。在NUMA系统中，操作系统已经进行了优化并将私有数据或者非共享数据放置在本地存储器中。由于对本地存储器的访问延迟很低，cache失效能够在几十纳秒中解决，而不是几百纳秒。

通信失效延迟 基于目录系统的主要缺点是，当发生cache失效而所请求的行在远程cache中是dirty时（这种情况称为通信失效或dirty失效），会带来额外的延迟。在基于监听的系统中，通信失效能直接得到满足，因为读请求能被直接传递给拥有数据的cache。在目录实现中，请求先被送到目录，然后再传递给当前的拥有者。这增加了一次处理机/存储器之间的移动，因此增加了延迟。一些数据共享非常集中的应用，如数据库事务处理，对于通信失效延迟非常敏感，在基于目录实现的系统中性能可能会很差。

人们还提出并建立了监听/目录混合系统。例如，Sequent NUMA-Q系统在其4处理机集合中使用了传统的、基于总线的监听协议来维护一致性，但是多个集合之间的一致性是通过建立在可扩

展一致性接口（SCI，Scalable Coherent Interface）标准上的目录协议实现的。混合机制具有目录机制的可扩展的优点，同时，对于能够在本地监听域内满足的通信失效，保持了较低的平均延迟。

9.3.5　多级cache、包含以及虚拟存储器

大多数现代处理机实现了多级cache，对访问延迟和带宽实现了容量和失效率的权衡：第一级（或者称为主cache）相对较小，但是访问时间只有一个或者两个周期，通常具有多体或者多端口；而第二级cache的容量要大得多，但是访问时间需要多个周期，而且一般都是单端口。多级cache的层次设计是在实现低造价、复杂性与最低平均访问延迟之间的平衡。如下面的公式所示，平均访问延迟lat_{avg}可以表示为n级cache中每一级延迟的加权和，其中每一级延迟lat_i的权值为这一级cache所满足的访存概率ref_i：

$$lat_{avg} = \sum_{i=1}^{n} ref_i \times lat_i \qquad \text{（公式9.3）}$$

当然，这样计算出来的平均延迟对于乱序执行的处理器意义不大，因为二级cache的失效延迟通常和其他有用的工作重叠，从而减小对主cache高命中率的依赖性。除了减小平均延迟以外，主cache的另一个主要目标是减少对二级cache的带宽需求。因为一个合理大小的主cache能够满足大多数访存，只有一小部分需要访问第二级cache，所以到二级cache只需要较窄的单端口接口。

在多级cache设计中保证一致性要比一级cache复杂一些。通过在多级cache之间保持包含关系能够得到一些好处，包含关系是指强制驻留在高级cache的行也同时驻留在低级cache中。

非包含的cache　实现多级cache一致性而不要求具有包含关系的简单办法是，在一致性机制中把每个cache作为cache层次中的平等实体，即每一级cache独立地维护一致性。在监听实现中，这意味着每一级cache都要监听所有通过系统地址总线的地址命令。这将导致对一级cache的标识存储器会有极高的带宽要求，因为处理机和内部输入地址总线都会非常频繁地访问标识存储器。IBM Northstar/Pulsar设计采用了非包含的cache并基于监听的一致性协议，第一级标识存储器中有两个副本，实际上提供了双端口的访问。在非包含的目录实现中，共享向量必须为每一个处理机（如果是精确的共享向量）的每一级cache维护单独的表项，或者可以使用粗略的共享机制，即消息必须传送到包含所请求副本的处理机的每一级cache。

包含的cache　除了单独为每一级cache维护一致性之外，另一种方法是维护包含性，使得高级cache中每一行的一致性状态和低级cache保持一致。例如，一个两级cache系统中，cache的层次结构必须保证驻留在一级cache中的每一行，同时也驻留（或者包含）在二级cache中，其状态也必须一致。保持包含性相当简单：当某一行进入一级cache时，也必须同时被放置在二级cache中。同样，当某一行被替换出二级cache时，也必须替换出一级cache。如果保持了包含性，只有低级（二级）cache需要直接参与cache一致性机制。根据定义，属于一级cache行的一致性操作也属于二级cache中的相应行，即如果在二级cache中找到这样的行，那么一致性操作也必须作用于一级cache上。实际上，在二级cache的标识存储器中进行查找起到了过滤器的作用，从而防止不相关的一致性操作在一级cache的标识存储器中进行查找。在基于监听的实现中，地址流量很大，这样做很有好处，因为对标识存储器的访问现在划分为两个分离的部分：90%以上的处理机访问通过一级标识存储器满足，即cache命中；而90%以上的地址总线命令通过二级标识存储器满足。只有一级cache失效时才要求进行二级标识的查找，只有当一致性操作命中共享cache行时才访问一级标识存储器。这种方法不再需要维持一级标识存储器的多个副本或者进行双端口访问

cache一致性和虚拟存储器 几乎所有的现代处理机都实现了虚拟存储器，从而提供访问保护和分页的需求，但这增加了设计的复杂性。引入虚拟存储器之后，用户程序产生的有效地址或者虚拟地址通过操作系统维护的映射被转换为物理地址。通常，地址转换在访问cache之前进行，但是，由于访问时间和容量的原因，一些处理机的主cache使用了虚拟的索引或者标识。虚拟地址cache的访问时间要短一些，因为地址索引和地址转换可以并行处理。然而，由于cache一致性的维护通常都是使用物理地址而不是虚地址，在虚地址cache中进行一致性维护则成为一个难题，必须要有进行反向地址转换的机制。这可以通过单独的反向地址转换（RAT，Reverse Address Translation）表来完成，RAT表记录所有访问的实地址和相应的虚地址。在多级cache中，实现反向地址转换还可以在二级标识存储器中设置指向相应的一级标识存储器表项的指针。不管使用哪一种方法，一致性控制器都能够搜索所有与给定的物理地址相对应的一级标识存储器。在较大的组相联虚地址cache中，其实现代价非常高，因为与给定物理地址对应的虚地址集合可能很大。

9.3.6 存储一致性

多机系统除了提供一致的存储器视图以外，还必须支持预先定义的存储一致性模型（memory consistency model）。一致性模型对处理机访问存储器的顺序进行了规定，它是任何多机系统的指令集体系结构规范中不可分割的部分。多处理机之间的一致性访存顺序对于任何共享存储器的多线程十分重要，如果没有结构化的规则来确定访存的顺序，这样的程序就不能保证线程间同步的正确性和可靠性，从而也不能保证行为的可重复性和可预测性。例如，图9.7显示了一个简单的串行化机制并保证对关键段的执行是相互排斥的，例如对共享数据的改写。Dekker的两个处理机的相互排斥机制是，处理机0设置变量A并测试另一个变量B，然后执行互斥的访问（对于处理机1，变量名要反过来）。只要每个处理机在测试对方的变量之前设置了自己的变量，就能保证互斥。然而，此时如果访存没有一致的顺序，两个处理机则很难判断对方是否进入了关键段。例如，两个处理机同时测试对方的变量，但是都没有看到对方的写操作，所以都进入了关键段，然后对共享的数据进行改写。如果允许处理机重新规定访存的顺序，使得读操作在独立的写操作之前执行（在第4章中称为载入旁路），那么，该方案就是可行的。

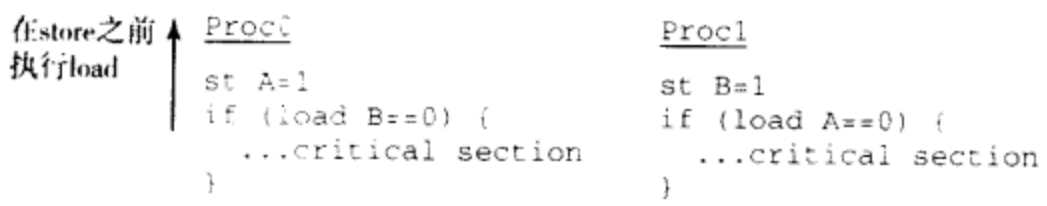
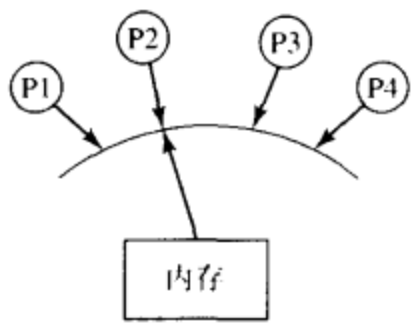


图9.7 Dekker的互斥算法

9.3.6.1 顺序一致性

最简单的一致性模型称为顺序一致性模型，它要求所有处理机的访问都是全序（total order）的。从概念上说，在一个顺序一致（SC，Sequentially Consistent）的系统中，就好像所有的处理机都在顺序地访问共享存储器一样，访存序列是交叉的、完全排序的，而且对于每一个单独的处理机而言是服从程序顺序的。顺序一致性模型如图9.8所示，在原理上与单个分时共享处理机上执行的多个线程对存储器的交叉访问类似。由于这种相似性，程序员容易推断SC系统中的共享存储器程序的行为，因为在分时共享的单处理机上，正确执行的多线程程序通常也能够在顺序一致性的多处理机上正确执行。



每个处理机按程序顺序访存，来自所有处理机的访问被交叉起来，好像存储器一次只满足一个处理机的请求

图 9.8　顺序一致性存储器访问排序［Lamport］

然而，顺序一致性模型的效率较低。全序的访问顺序要求每一次读写操作必须按照程序顺序发射，从而不能发挥出现代乱序执行处理机的性能，而且每一次访存还必须和其他所有处理机进行排序，因此要有很高的互连带宽来建立这种全局的排序。幸运的是，对于允许在乱序执行的处理机中松弛指令顺序的原理，同样也允许放宽对全序访存顺序的要求。也就是说，指令流的顺序执行是过度的要求，并不是实现正确性所必需的。同样，SC 全序也是过于严格的要求，并不是必需的。在乱序执行的处理机中，寄存器重命名和再定序缓冲能够松弛执行顺序，松弛的程度只取决于真数据相关，而维持着顺序执行的假象。同样地，SC 全序也可以进行松弛，只有必须按照顺序进行访问并保证数据相关的访存才按序执行，其他访问可以乱序执行。这保证了希望使用SC 全序的程序仍然能够正确运行，因为只有将共享数据的访问而引起的数据相关暴露给其他处理器时，才会出现失败的情况。

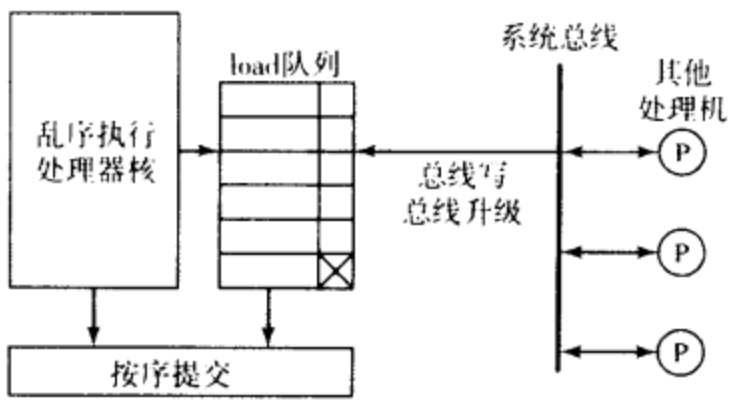
9.3.6.2　顺序一致性的高性能实现

有许多因素使得这种松弛的SC全序模型能够得到有效的实现。首先是cache 和cache 一致性机制的存在。由于cache 一致性保证了每个处理机能够察看到其他处理机的写操作，因此能够发现任何处理机内部的数据相关。也就是说，如果一个处理机的读操作与来自另一个处理机的写操作存在数据相关（即存在一个写后读 RAW 相关），一致性机制必须有所动作来满足相关：首先在发出读操作的处理机的 cache 中作废请求的地址（保证 single-writer 一致性），然后，读操作将失效，发出写操作的处理机将请求的行传送给发出读操作的处理机。

简单地说，如果没有这样的一致性动作（作废或 cache 失效），我们就不知道存在相关。由于大多数存储器访问都能通过 cache 命中满足，这样就不需要上述动作，因此可以安全的放松命中的存储器访问。这时访存排序的问题可以分解为一个局部问题（按照 cache 失效和远程作废请求形成的边界来划分）和一个全局问题（对 cache 失效和作废请求排序）。前者通过增加一个处理机的读写队列实现，该队列除了监视处理机本地地址以外，还监视全局地址事件。cache 失效和作废的排序则通过在系统中某个地方提供一个全局排序点来完成。对于规模较小的、使用共享总线的系统，单个共享总线的仲裁为失效和作废建立了全局的顺序。在目录实现中，命令可以在到达目录时或者在系统互连的其他共享点上进行排序。

然而，仍然必须通过对所有的访问进行排序（根据一致性事件）来解决局部问题。简单地说，必须保证在进行当前的访问之前，所有先进行的访存都必须命中 cache。显然，这就退化到了顺序地执行所有的存储器访问，而不能使用高性能的乱序执行。这里，可以使用推测执行来解决这一问题，使之能够放松要求的顺序。也就是说，可以假设某次访存实际上不需要进行排序，因而推测地执行这次访存。只有当确定这次访存需要排序时，才从推测中进行恢复。由于一个规范的乱序执行处理机已经支持推测及恢复，我们只需要增加一个机制，用于检测排序的冲突以及发生这种情况时的恢复就可以了。

检测排序冲突的最简单的方法是监视全局地址事件，并检查它们是否与本地推测执行的访存有冲突。由于推测执行的访存已经记录在处理机的读写队列中，因此只需要一个简单的机制来检查全局地址事件（与远程写相应的作废消息）和所有未提交的读操作就可以了。如图9.9所示，一次地址的匹配将导致一次读操作被标记，表示可能存在排序冲突。当指令按照程序顺序提交时，将检查是否存在排序冲突。如果发现有标记的读操作，处理机则把它当做预测失误的分支进行处理，并重新取出并执行这条读指令和所有的后续指令。通过重新执行，读操作现在排在了冲突的远程写操作之后。MIPS R10000、HP PA-8000 和 Intel Pentium Pro 及其后续系列均实现了类似的存储一致性模型。



乱序发出读操作，但地址被记录在load队列中。在读操作提交之前发生的任何远程写请求都被监听，并与load队列比较。如果存在地址匹配，说明可能存在冲突，并且当读操作准备提交时，触发基于重新取指的恢复机制

图9.9 记录读操作并检测一致性冲突

9.3.6.3 松弛一致性模型

一种对顺序一致性的改进方法是规定一个松弛一些的一致性模型。人们提出了多种松弛一致性模型（RC），它们存在着各种细微的差别。关于几种一致性模型的详细讨论，有兴趣的读者请参考[Adve和Gharachorloo, 1996]。RC模型提出的动机是对硬件实现化简，其方法是由程序员确定并标记出需要排序的访问，而没有标记的访问则可乱序执行。

存储器栅栏 标记排序访问的最普通而且最实用的方法是，由程序员插入存储器栅栏指令，从而加入排序要求。典型的存储器栅栏语义（如PowerPC指令集中的sync指令）要求栅栏以前的所有访存操作要在任何后续的访存操作开始之前结束。一个简单而实用的实现方法是，停止指令发射直到所有已发射的访存指令结束为止。必须注意，要确定所有的访存指令确实结束。例如，许多处理机将store指令存放在一个store队列中，这可能会延迟store指令的执行。因此，只检查再定序缓冲是否包含store指令并不够，还必须检查推迟执行的store指令所在的队列。而且，尽管写操作已经对本地cache执行过，而且已经从store队列中移走，但作废消息可能还在传输，或者可能还在远程处理机的作废队列中。为了保证正确性，系统必须保证存储器栅栏后的访存操作执行之前，栅栏以前所有的作废指令确实已经执行了，从而防止任何对旧副本的远程访问。显然，这需要很长时间，对于处理机数目很大的系统，甚至需要几百个周期。

松弛一致性协议的主要缺点是给程序员带来了额外的负担，需要确定并标记出进行排序的访存操作。确定多线程程序的正确性是一件困难的事情。给程序员强加细微的而且有时候是违反直觉的正确性规则只会损害程序员的编程效率，并且会增加出错的可能性。

松弛一致性模型的优点 松弛一致性模型的主要优点是硬件简单而性能较高。如果存储器栅栏数目过多，这一优点就可能消失。这时则需要一种更高效的方法，而不仅仅是简单地停止发射指

令并等待所有未完成的访存操作。存储器栅栏的一个更高效的实现类似图9.9所示的作废记录机制，该机制监听所有读操作的地址并与作废消息进行比较，但是只有当存储器栅栏在一次冲突的读操作提交之前结束时，才会引起一次排序冲突。这可以通过在读/写队列中对读操作进行两次标记实现：第一次是在发生作废时，第二次是存储器栅栏提交并且已经存在第一个标记时。当读操作准备提交时，如果两个标记都在，就会触发重新取指，表明读操作从数据cache中读到了一个旧值。

松弛一致性模型最根本的优点是，在没有存储器栅栏的情况下，硬件有更大的自由将写失效的延迟与后续指令的执行重叠起来。在9.3.6.2小节描述的SC执行机制中，这种重叠受限于乱序指令窗口的大小。一旦指令窗口填满，就不能执行更多的指令，直到未完成的写操作完成为止。在RC系统中，写操作可以存放到写操作队列中，后续的指令可以从指令窗口中提交并为新的指令让出空间。这一区别带来的相对好处决定于存储器栅栏的频率。在极端的情况下，每一个写操作后面都跟有一个存储器栅栏。RC则不能带来任何好处，因为在SC系统中，指令窗口被填满的情况在这里也会发生。然而，即使是在关系数据库这种数据共享程度极大的应用中，存储器栅栏也要比写操作少得多。

假设存储器栅栏出现的频率相对较低，松弛模型的性能取决于指令窗口的大小、取指部件使用有效的指令填充指令窗口的能力、以及提交一条store指令的相对延迟。最近的趋势表明，随着更好的预测器和更大的再定序缓冲的出现，指令窗口的大小和取指能力都在增长，但是由于时钟频率的增加、系统中处理机数目的增加以及多级互连网络的复杂化，相应的延迟也在增加。从所了解的情况看，未来RC系统的主要优点能否转化为显著的性能提高还不是很清楚。无论如何，所有最近提出的指令集都支持松弛一致性（Alpha，PowerPC，IA-64）。它们的存在表明，推断程序在松弛一致性模型下的正确性决不是不可克服的困难。

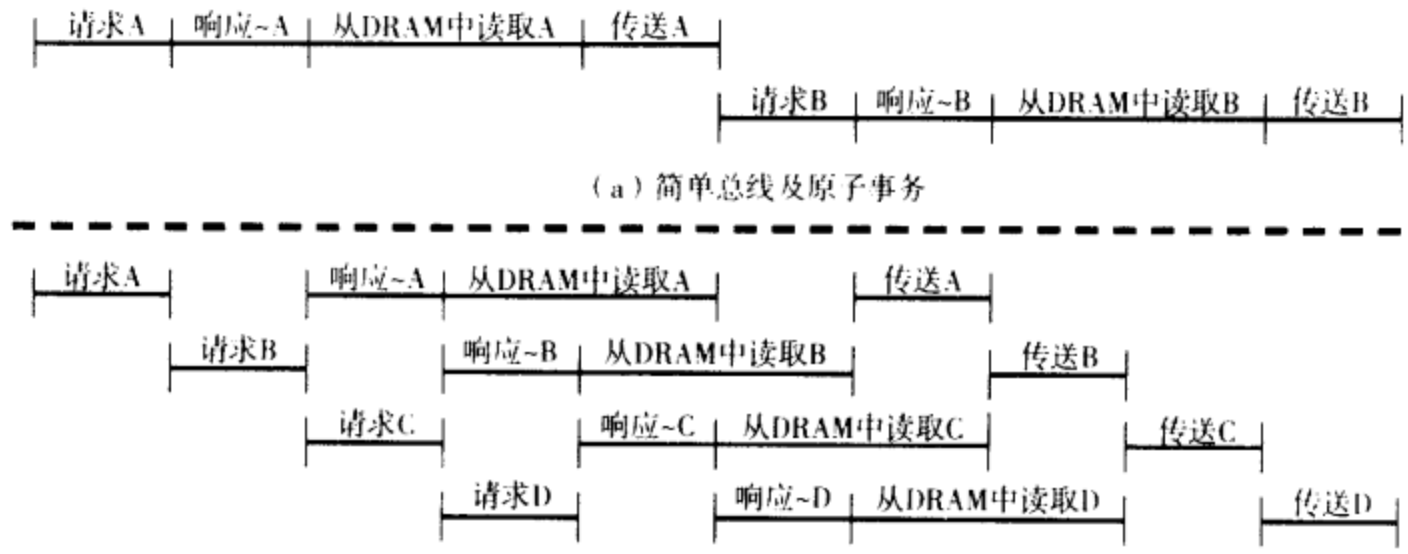
9.3.7　一致性存储器接口

将一个简单的单处理机连接到存储器的接口可以通过总线实现，总线将处理机的读写命令作为原子总线事务。一旦一个处理机成功地控制了总线，它将相应的命令放置在总线上，然后占有总线直到接收到所有数据和地址响应，最后将发出信号结束这次总线事务。许多先进的单处理机都支持分离的总线事务（split transaction），它的请求和响应是分开的，以获得更大的并行性并提高总线的利用率。

在分离事务总线中，处理机发出请求之后，在它从存储器控制器接收到响应之前就释放总线，因此其他处理机也能发出请求并与响应延迟重叠起来。而且，一致性响应的请求也能进行分离，在一致性响应返回之前就释放总线地址。图9.10说明了分离事务总线的优点。在图9.10（a）中，简单总线将两次请求串行化，一次到地址A，一次到地址B：发出请求、监听响应、DRAM读取数据和数据传输。图9.10（b）显示了分离事务总线以一种流水化的方式满足4个请求，该总线将在每次请求之后释放总线，在单独的总线上接收监听响应和数据响应。所用的时间却不超过简单总线满足两次请求的时间。当然，设计也会复杂得多，因为同时存在多个分离的事务，一致性控制器必须一一加以记录。

通常，每一个未完成的事务都与一个惟一的标识关联在一起，该标识比物理地址短得多，并在数据和响应总线上附加信号线或者消息头，来确定并发的一致性或者数据消息。每一项未完成的事务记录在一个失效状态处理寄存器（MSHR，Miss-status Handling Register）中，记录的信息包括失效地址、关键字信息以及重命名寄存器信息。一旦存储控制器返回失效访存所需要的数据，这些信息就用于重新执行。MSHR还用于合并对同一行的多个请求，防止将同一个请求发送多次。另外，

写回缓冲用于推迟被替换的行向存储器的写回操作，直到相应的请求失效被满足。填充缓冲用于收集发往cache行的分组数据应答，最后再写入cache。一个先进的分离事务总线接口的例子如图9.11所示。

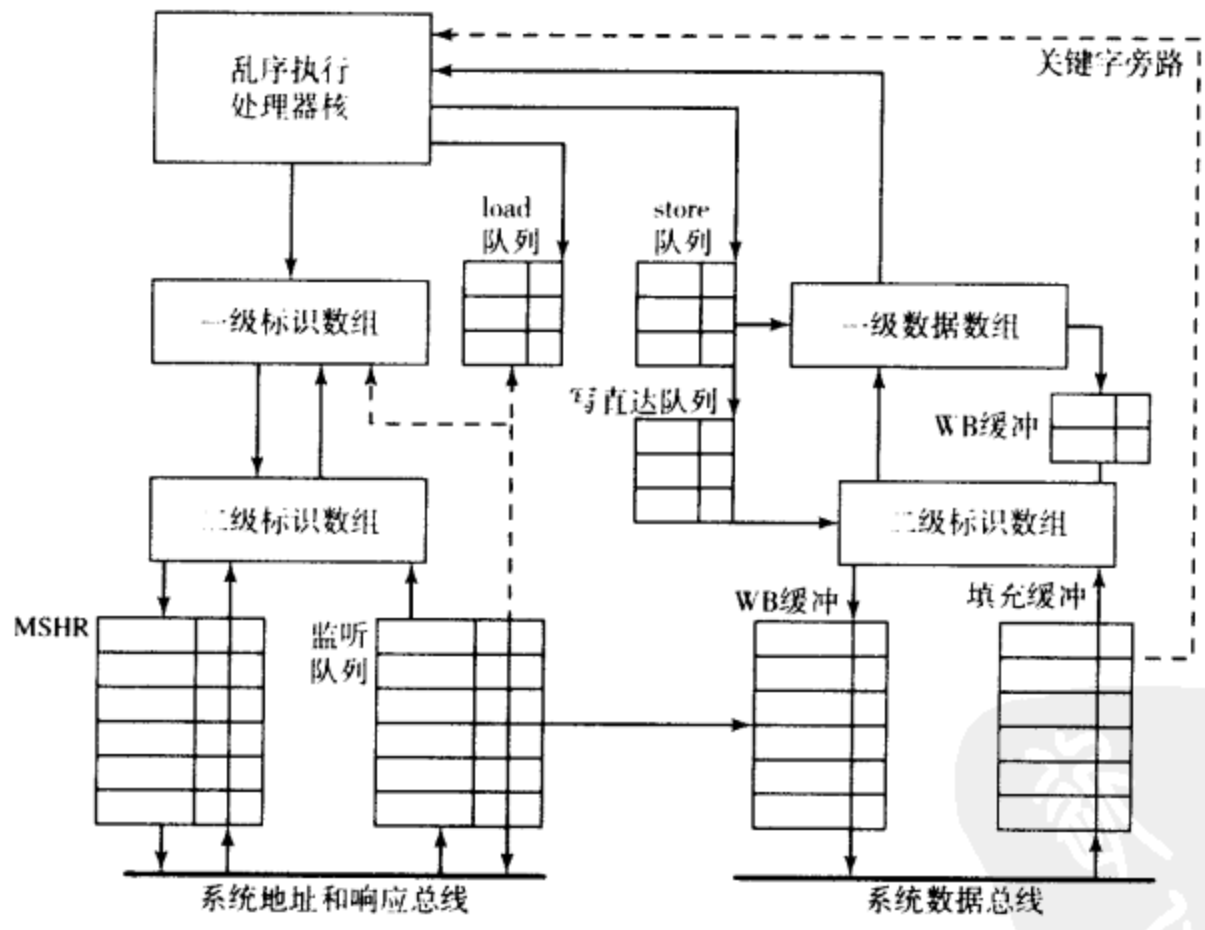


（a）简单总线及原子事务

（b）分离事务总线及分开的请求和响应

分离事物总线通过将请求、响应和数据传送流水化来提高吞吐率

图9.10　简单总线与分离事务总线



一个处理机与存储器的通信可能会经过两级cache、一个load队列、一个store队列、一个写直达队列（如果L1 cache是写直达的）、MSHR（失效状态处理寄存器）监听队列、填充缓冲以及写回缓冲。这些结构的控制逻辑并没有在图中显示。

图9.11　处理机－存储器接口

要保证多机系统中的一致性，必须对这一相对简单的单处理机接口进行改进。首先，必须改进总线仲裁机制以支持多个请求方或者总线控制者。其次，必须提供对来自其他处理机的总线命令的支持。在监听实现中，这些命令都是由其他处理机放置在总线上的，而在目录实现中，命令是从目

录中传过来的。该命令集应提供下列功能：检查处理机的标识存储器并得到某一行的状态；将修改过的数据从cache中替换出去以及作废一行。在早期的微处理器设计中，一致性控制器位于片外，而所有的现代微处理器都将一致性控制器集成在片内。集成在片内的一致性控制器对地址总线上的高层命令（即远程读，独享读以及作废）做出响应，然后向本地cache发出相应的底层命令。为了尽可能地提高并行性，现代处理机采用了监听队列（参见图9.11）并从总线上接收监听命令，然后以流水的方式进行处理。

9.3.8　结论

采用多个处理机并具有一致的存储器视图的系统在市场上取得了巨大的成功。它们为系统供应商和用户带来了显著的优势，包括可扩展性、易用性以及可编程的能力，并提供了一条从入门级系统到企业级系统的升级路径。许多重要的应用都包含大量的线程级并行，这是这些系统成功的关键。随着对性能和可扩展性的要求继续提高，这些系统的设计者在实现cache一致性、共享存储器并最小化通信和访存延迟时，将面临无数的权衡。

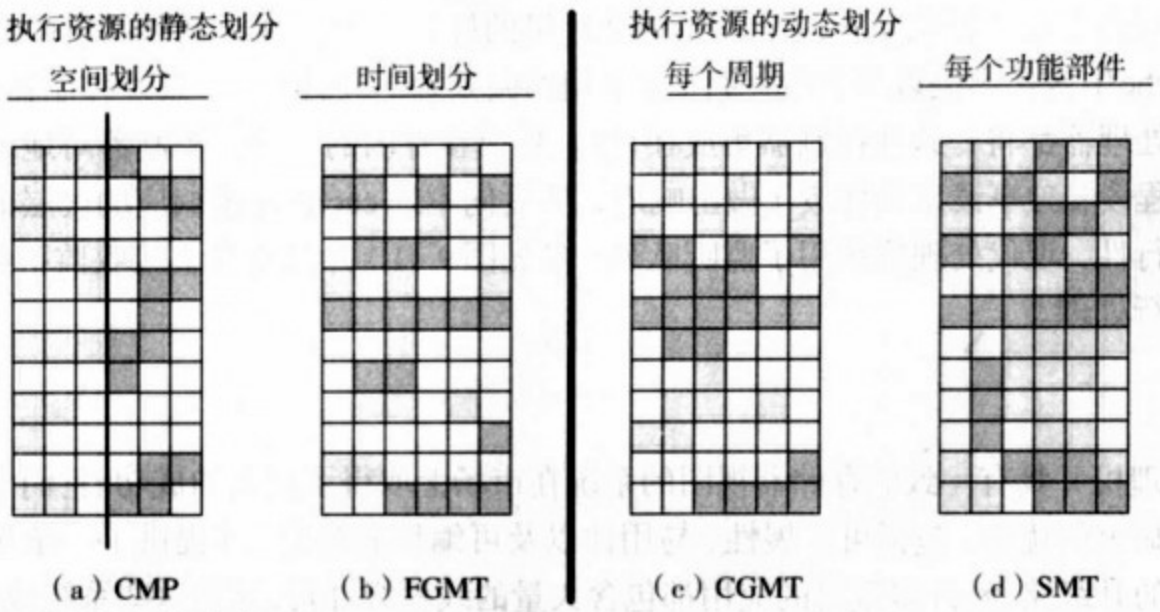
9.4　显式多线程处理器

由于现在流行的应用程序存在着大量的线程级并行性，显然，微处理器的下一步发展就是使每个处理器芯片能够执行多个线程。这样做的主要目的是想进一步提高处理器芯片上昂贵的执行资源的利用率。分时共享的操作系统在当前线程等待具有较长延迟的I/O事件时，将切换到另一个线程执行，从而提高了CPU的利用率（如图9.1所示）。与此类似，执行多个线程的芯片能够在某个线程由于cache失效或者分支预测失败而停顿的情况下，仍然保持处理器资源处于繁忙的状态。实现这种能力最直接的办法是在单个处理器芯片中集成多个处理器核，至少已经有两个通用微处理器设计是这样做的（IBM Power4和Hewlett-Packard PA-8900）。这种设计称为单芯片多处理器（CMP，Chip Multiprocessor）。一些研究人员建议发展单芯片多处理器技术，以支持单线程程序的并行推测执行。我们将在9.5节中讨论这一问题。

通过为每个线程复制一个完整的处理器核，单芯片多处理器提供了一种比较极端的方法来支持单个芯片上多个线程的执行。人们提出了很多在单个处理器核上实现多线程的方法，有一些甚至已经在商用的产品上实现。多线程技术的范围包括细粒度多线程，即多个线程在单个处理器核上的交叉执行是基于时钟周期的；粗粒度多线程，同样也是在单个处理器核上交叉执行多个线程，但是执行时间的边界由具有较长延迟的事件决定，例如cache失效；同时多线程，允许多个处于活跃状态的线程指令同时占有处理器的执行窗口，从而消除了上下文切换的开销。表9.2总结了在几种实现多线程的方法中，上下文切换机制和资源共享的程度。每种机制执行资源的分配情况如图9.12所示。

表9.2　资源共享和线程切换的各种方法

多线程方法	线程间共享的资源	线程切换机制
无	所有资源	由OS进行上下文切换
细粒度	除了寄存器文件和控制逻辑以外	每个时钟周期切换
粗粒度	除了取指缓冲、寄存器文件和控制逻辑外	流水线停顿时切换
SMT	除了取指缓冲、返回地址栈、逻辑寄存器文件、控制逻辑、再定序缓冲、store队列等以外	所有上下文同时处于活跃状态，没有切换
CMP	二级cache，系统互连	所有上下文同时处于活跃状态，没有切换



4种可能的选择包括：
（a）单芯片多处理器，静态划分执行带宽；（b）细粒度多线程，每个周期执行不同的线程；（c）粗粒度多线程，在遇到较长延迟事件时切换线程；（d）同时多线程，将多个不同线程的指令混合执行。CMP和FGMT静态地划分执行资源并使用空间划分，即为每个处理器分派固定数目的资源，或者使用时间划分，即多个线程分时复用同一组资源。CGMT和SMT允许动态划分，CGMT使用基于每个周期的时间划分，SMT使用基于每个功能部件的空间划分。SMT实现了最大的灵活性和最高的资源利用率以及指令吞吐率。

图 9.12　在单个芯片上运行多个线程［Tullsen 1995］

9.4.1　单芯片多处理器

晶体管密度的提高使得在单个芯片上采用更复杂、更占面积的体系结构技术成为可能，例如乱序执行机制、高准确率的分支预测部件，甚至大小可变的二级 cache。最近的设计已经集成了一致性控制器来实现无黏性（glueless）的多处理方案，并集成了大型片外 cache 的标识存储器，以及与 DRAM 进行直接相连的存储控制器。片上系统（system-on-a-chip）在芯片上直接集成了图形控制器、I/O 设备和 I/O 总线接口。随着晶体管密度的继续增加，下一步则需要将多个处理器核集成到同一个硅片上。单芯片多处理器（CMP，Chip Multiprocessor）为系统设计者带来了一些明显的好处: 在单个芯片上集成多个处理器使得多个处理器的封装和互连的物理实现变得容易。紧密的集成减少了片外信号，从而减小了处理器与处理器之间的通信和同步延迟。最后，由于集成了多个处理器，可能需要重新考虑 cache 和一致性接口的共享问题。

共享 cache　CMP 一个明显的设计选择是在多个核之间共享片内或者片外的 cache 存储器（IBM Power4 和 HP PA-8800 就是这样实现的）。这种方法减小了片上处理器之间的通信失效延迟，因为不再需要片外信号来处理通信失效。当然，远程处理器的共享失效仍然是一个问题，尽管发生共享失效的频率应该有所降低。但是，还存在这样一种情况，即如果处理器执行的是不相关的线程并且没有共享数据，那么共享的 cache 就会忙于应付冲突失效。操作系统的任务调度器可以进行处理器绑定（processor affinity）调度，即将相同或者相关的任务调度到共享 cache 的处理器上执行，从而减少这种冲突。

共享一致性接口　另一个明显的选择是与系统的其余部分共享一致性接口。接口的硬件开销由两个处理器承担，并且由于受多个独立的线程驱动，能够获得额外的存储器级的并行，从而提高了利用率。当然，承受两个处理器的流量将使一致性接口的负担比承受单个处理器的流量时更大。因此，设计者必须十分注意，确信一致性接口能够满足多个处理器的带宽要求。另一方面，如果有共享的片上 cache 以及足够的信号带宽，设计者应当重新考虑维持一致性的写直达和基于更新的协议。简单地说，没有理由认为可以使用芯片与芯片之间的一致性协议来维持单芯片内的一致性。类似地，单芯片内多个处理器之间的同步机制也要重新考虑。

CMP的缺点　CMP也有一些缺点。首先，假定使用相同的硅技术，在同样的造价下，得到的单处理器运行单个线程的速度要比CMP快，因为多余的硅片面积可以用于实现更好的分支预测部件、更大的cache或者更多的执行资源。而且，集成多个核很容易导致圆片面积过大，从而带来制造上的一些问题，尤其是在频率很高的情况下更是如此。经验表明，即使CMP的设计频率达到单处理器的水平，实际上也会因为生产的原因带来一些频率缺陷。最后，操作系统和软件的扩展能力给系统中的处理器数目设置了上限，该上限比由物理封装和其他物理限制所决定的上限要低得多。因此，有人可能会认为，从性能价格比的角度来看，CMP并不是一种有效的方法，无论应用于高端还是低端都没有显著的优点。然而，一些厂商已经发布了CMP设计，并且这些设计确实产生了一些令人瞩目的优点，尤其是在应用程序包含大量线程级并行的商用服务器市场。

IBM POWER4　图9.13显示了IBM POWER4的结构。每个处理器芯片包含两个深度流水的、乱序执行的处理器核，每个核包含64 KB的私有一级（L1）指令cache和32 KB的私有一级数据cache。L1数据cache是写直达的，来自两个处理器核的写请求被集中，且合并到共享的L2 cache各个体的存储队列中（图中的P0 STQ和P1 STQ）。存储队列包含4个64字节的表项并用于写合并。L2 cache的三个体每个都是512 KB，每个体都有多个MSHR表项用于记录未完成的事务。同时，还包含了多个写回缓冲、多个监听队列表项，用于处理到达的一致性请求。处理器还与系统中其他处理器共享一致性接口、一个单独的到一致性I/O子系统的接口、一个到片外的L3 cache以及它的片内标识存储器的接口。由于L1数据cache采用写直达策略，所有远程处理器的一致性请求以及片内另一个处理器核的读请求可以从L2 cache得到满足。L2 cache的标识存储器为片内的两个处理器核设置了一个共享向量，用于记录两个核中的哪一个包含了L2 cache的任何一列的共享副本。当本地处理器核中的一个或者远程处理器对某个共享列发出写请求时，就会用到这个共享向量，将向一个或者两个本地处理器核发出作废消息，以保证cache写的一致性。POWER4支持的从L2到处理器核的带宽极大（超过100 GB/s），还为L3 cache和多处理器配置方式下的周边处理器提供多个高带宽接口（每个超过10 GB/s）。

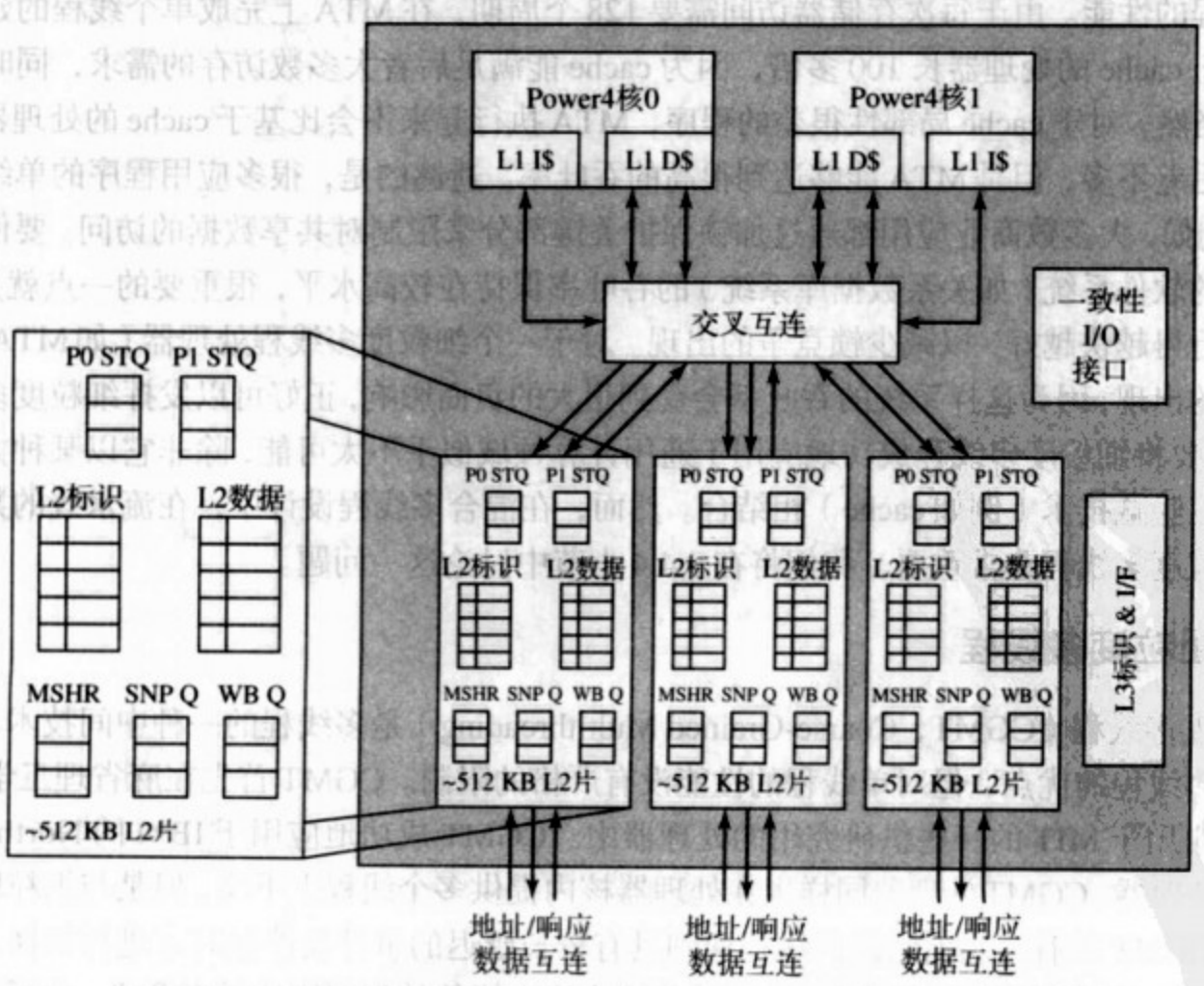


图9.13　IBM POWER4单芯片多处理器的例子

9.4.2 细粒度多线程

细粒度的多线程处理器在片上提供两个或者更多的线程上下文，线程切换的方式是按照一种固定的、细粒度的调度策略，从一个线程切换到下一个线程，通常每个周期将处理不同线程的指令。细粒度多线程的起源可以追溯到20世纪60年代中期、Seymour Cray设计CDC-6600超级计算机的时候。在CDC-6600中，10个I/O处理器以循环的方式共享一个中央处理器，每个I/O处理器交叉占用中央处理器。在20世纪70年代，Burton Smith设计并建造了Denelcor HEP，即第一个真正的多线程处理器，它在单条流水线中交叉执行多个线程上下文的指令，从而隐藏存储器延迟并且不需要检测和化解指令间的相关。

Burton Smith后来设计了一种类似的机器——Tera MTA，他将重点放在尽量提高存储器访问的利用率上，方法是交叉执行存储器访问路径上的多个线程指令。最新的MTA处理器的目标是高端科学计算，并在高带宽低延迟的存储器访问路径上投入了很多努力。实际上，MTA处理器的存储器带宽是系统中最昂贵的资源，因此尽可能提高它的利用率是合理的。MTA处理器是一个细粒度的多线程处理器，即按照一种固定的调度方式，每个时钟周期都进行线程切换。它有足够的寄存器上下文（128个）来隐藏存储器的延迟，因此没有必要使用数据cache。存储器访问路径是完全流水化的，允许128个线程中的每一个线程都含有一个未完成的、对存储器的访问。该处理器最大的优点是没有数据cache，由于没有cache，所有线程访问存储器的延迟是一致的，因此编译器没有必要重新构造访存模式，以增加数据cache层次的利用率。相反，编译器将注意力放在确定计算中相互独立的线程上。例如，在科学计算程序中进行循环展开（doaross loop），以满足128个上下文的调度需要。虽然Tera MTA在早期取得了一些成功，但是由于第二代CMOS的实现（第一代使用镓砷化合物技术）迟迟没有推出，它的未来现在还不确定。

单线程性能 细粒度多线程处理器如Tera MTA的最大缺点是，它们为提高总的吞吐率而牺牲了单线程的性能。由于每次存储器访问需要128个周期，在MTA上完成单个线程的延迟要比传统的、基于cache的处理器长100多倍，因为cache能满足后者大多数访存的需求，同时只需要几个周期。当然，对于cache局部性很差的程序，MTA执行起来不会比基于cache的处理器差，因为访存的延迟差不多，但是MTA能够达到很高的吞吐率。遗憾的是，很多应用程序的单线程性能非常重要，例如，大多数商业应用都通过加锁保护关键部分来限制对共享数据的访问。要使具有大量共享数据的软件系统（如关系数据库系统）的吞吐率保持在较高水平，很重要的一点就是，这些关键部分执行得越快越好，以减少锁竞争的出现。对于一个细粒度多线程处理器（如MTA），反而希望锁竞争的出现，因为这样系统的吞吐率会受到很大的负面影响，正好可以发挥细粒度多线程的作用。因此，要将细粒度多线程成功地应用于通用计算领域似乎不太可能，除非它以某种方式与传统的访存延迟隐藏技术（例如cache）相结合。然而，在混合多线程设计中，在流水线的某些特殊段中实现细粒度多线程非常重要，我们将在9.4.4小节中讨论这一问题。

9.4.3 粗粒度多线程

粗粒度多线程（CGMT，Coarse-Grained Multithreading）是多线程的一种中间技术，它具有很多细粒度多线程的优点，但对单线程的性能没有严格的限制。CGMT首先在麻省理工学院（MIT）提出，并应用于MIT的一些供研究用的处理器中。CGMT成功地应用于IBM的North-star/Pulsar PowerPC处理器。CGMT处理器同样也在处理器核内提供多个线程上下文，但是与细粒度多线程不同的是，CGMT只有在当前活跃的线程遇到具有较长延迟的事件而停顿时才进行切换，例如出现cache失效时。这种方法对于出现cache失效并停止流水线的处理器则比较有意义。为了不使流水线

停顿，另一个线程中准备好的指令将进入流水线，当它出现cache失效时再切换到下一个线程。按照这种方式，两个或多个线程在处理器中交叉执行，从而提高了处理器执行资源的利用率，有效地隐藏了很大一部分cache失效延迟。

线程切换开销　CGMT处理器设计中的一个关键问题是执行线程上下文切换的开销。由于上下文切换取决于一些动态事件，例如cache失效，这些事件只有进入流水线后面的段以后才能发现，简单的上下文切换会引入一些开销。因为失效指令之后的指令可能已经进入流水线，需要将这些指令从流水线中清除掉。同样，新线程的指令只有通过了流水线前面的段才能进入执行段。这将产生一个或者多个流水线气泡，气泡的数目取决于流水线的长度。避免切换开销的一个直接的办法是为每个线程复制流水线寄存器，每次切换的时候保存流水线的当前状态。这样，下一个线程的上下文就能在下一个时钟周期切换回去，消除了流水线气泡（Motorola 88000采用了类似的方法来减少中断延迟）。当然，要覆盖流水线所有的状态，其所占面积和复杂性的代价是需要考虑的。IBM Northstar/Pulsar的流水线相当短，切换开销只需要3个周期，设计者发现不值得去复制寄存器，消除3个周期的切换开销对于提高性能的帮助不大。这是很合理的，因为上下文切换能够隐藏上百个周期的cache失效延迟，再省下几个周期也不能使性能提高多少。当然，对于流水线很长、切换开销很大的设计，其权衡结果可能完全不同，可能需要复制流水线寄存器或者采取其他减少切换开销的方法。

保证公平性　CGMT处理器设计中的挑战之一是如何保证执行资源的分派是公平的，从而避免饿死的情况发生。如果每个线程的cache失效率差不多，各个线程上下文就能比较公平地共享流水线，因为每个线程将按照相同的概率让出CPU。然而，一个线程的cache失效率不是程序员或者操作系统能够轻易控制的，因此，需要提供额外的机制来保证公平性，以防止饿死。可以采用操作系统调度策略中的标准技术：经过一个时间片之后，失效率低的线程可以抢占CPU，强制执行一次线程切换。为了避免因过早抢占而导致的饿死，硬件可以强制规定一个最短时间。

除了要保证公平性之外，当单线程的吞吐率非常重要的时候，CGMT处理器还应该提供一套机制，尽量减少无效执行所占用的带宽，同时要尽量增大有效执行的带宽。前者可能发生在一个线程处于忙等待的状态（例如，反复查询其他线程或者处理器占有的旋转锁）或者进入操作系统的空循环。后者发生在某个线程占有关键资源（例如，一个竞争激烈的锁）而系统中有其他线程在等待该资源释放的时候。在这种情况下，具有高优先级的线程的执行不能被抢占，即使它因为cache失效而停顿。因为其他线程可能直接（由于线程切换开销）或者间接（引起额外的冲突失效和存储器竞争）使高优先级线程慢下来。

线程优先级　CGMT处理器可以通过优先级机制来克服这些性能上的缺陷。优先级至少分为高、中、低3级。注意，这不是操作系统层次上的优先级。在操作系统中，每个线程或者进程都有一个固定的优先级，由操作系统或者系统管理员设置。与之不同的是，这里的优先级是动态变化的，反映了一个线程在当前执行环境下的相对重要性。因此，当线程进行优先级转换时，需要程序员的干预来通知硬件。例如，当线程获得锁后进入关键部分的执行，应该转换到高优先级；相反，当线程退出时，应该降低优先级。同样，当线程进入空循环或者开始反复查询旋转锁时，应该降低优先级。当然，这样的通信机制需要定义一个接口，通常是通过ISA中的特殊指令来指明这些转换，同时还需要程序员将这些指令放在程序的适当位置上。除此之外，还可以使用一种隐式的模式匹配机制，识别出优先级转换的执行序列。IBM Northstar/Pulsar处理器采用了前一种方法，使用特殊编码的NOP指令指明线程优先级。幸运的是，软件的改动集中在操作系统和中间件（例如数据库）中相对较少的地方，通过较少的努力即可实现。

线程切换状态机 图9.14显示了CGMT处理器的一个简单的线程切换状态机。如图所示，每个线程有4种可能的状态：运行、就绪、停顿和换出。线程状态的转换发生在cache失效开始或者结束的时候，此时线程切换逻辑决定切换到下一个线程。对于一个设计优良的CGMT处理器，下列条件能够导致线程发生切换：

- 当前执行的主线程发生cache失效，并且有候选的线程处于就绪状态。
- 主线程进入空循环，并且有非空转的候选线程处于就绪状态。
- 主线程进入同步循环（忙等待），并且有非空转的候选线程处于就绪状态。
- 一个换出的线程转换到就绪状态，并且该线程比当前主线程的优先级高。
- 一个就绪的非空转候选线程在最近n个周期都没有执行过指令（避免饿死）。

最后，如果处于运行状态的线程的执行时间小于某个固定的周期数，就不允许其他线程抢先执行，这样可以保证每个线程总能向下执行。

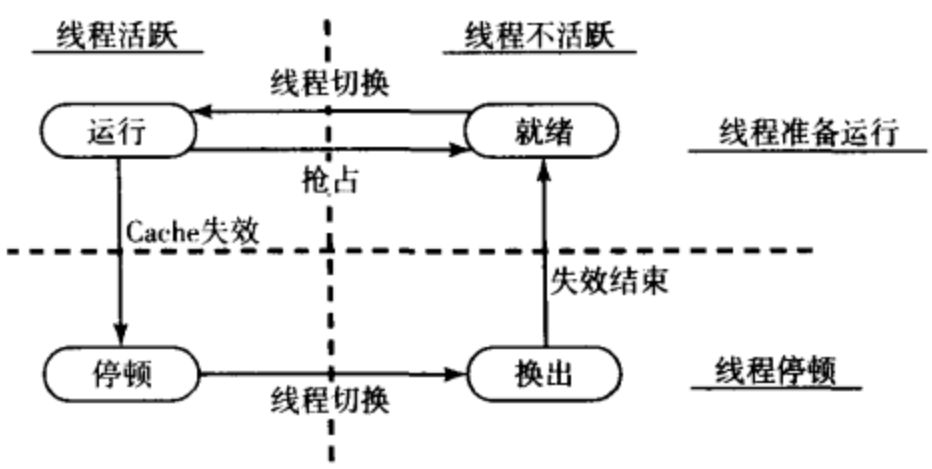


图9.14 CGMT线程切换状态机

性能和价格 CGMT是提高指令吞吐率的一种很有效的技术。根据IBM的报告，Northstar/Pulsar系列处理器通过CGMT将指令吞吐率提高了30%，而圆片面积的增加小于10%，对时钟周期的影响几乎可以忽略。CGMT带来的惟一复杂性在于线程切换和优先级管理控制的复杂性，另外还有至少两倍大小的寄存器文件，以容纳两个线程的上下文而不是一个。最后，为实现线程优先级，要求软件所做的改动要小，这些也必须在计算价格时考虑到。

9.4.4 同时多线程

片内多线程最后一个也是最复杂的实现方法是，允许多个线程在共享的执行资源中以细粒度的方式动态交叉执行指令。这种技术最早是在1995年由Washington大学的研究人员提出，并在最新的Intel Pentium4处理器中得到了实现。他们认为，以前实现多线程的方法在多个线程之间共享硬件的效率不高，因为线程切换的方式使得整个流水线或者流水线的各段只能包含来自一个线程的指令。由于指令级并行的分布是不均匀的，这就导致流水线的某些段存在空闲的指令槽，因而降低了多线程的效率。为了提高效率，他们提出了同时多线程（SMT，Simultaneous Multithreading）技术，允许指令在流水段内部和流水段之间交叉执行，以尽量提高处理器执行资源的利用率。

现代乱序执行处理器的一些特性使之能够有效地实现同时多线程。首先，指令乱序通过流水线中间的段，即与程序或者取指的顺序不一样。这使得不同线程的指令能够在这些段中混合执行，从而使这些段中的资源利用率更高。例如，单个线程内部的数据相关限制了超标量处理器每个时钟周期只能发射一到两条指令，而另一个与之无关的线程指令可以用于填充空闲的发射槽。其次，通过寄存器重命名机制动态化解数据相关，重命名表只需要简单地将每个线程相同的寄存器与另一个不

同的物理寄存器对应起来，其他操作则交给标准的乱序执行硬件处理，相关则通过使用重命名之后的物理寄存器来化解。最后，在乱序处理器中，用于提取和平滑不均匀、不规则指令级并行的大量缓冲（如再定序缓冲、发射队列、load/store队列、提交队列），可以通过采用多线程而提高利用率，因为对于可能导致处理器利用率降低的数据同步和控制相关，现在只会影响到这些指令自身所在的线程，来自其他线程的指令仍然可以将流水线充满。

9.4.4.1　SMT的资源共享

SMT的主要目标是通过在多个线程之间共享处理器资源，提高这些资源的利用率。实际上，多个线程同时执行所带来的并行性可以证明更深更宽流水线的合理性，因为额外增加的资源在SMT中可能更有用。然而，哪些资源应当共享，哪些不应当或者不能共享，却不是很清楚。图9.15展示了一些设计选择，在左边的设计中，除取指段和提交段以外的所有资源都被共享，右边的设计只共享执行段和访存段。无论哪种设计，不同线程的指令必须在有共享资源的流水段之前汇合，在流水线的末尾分开，从而为每一个线程保存精确的异常。

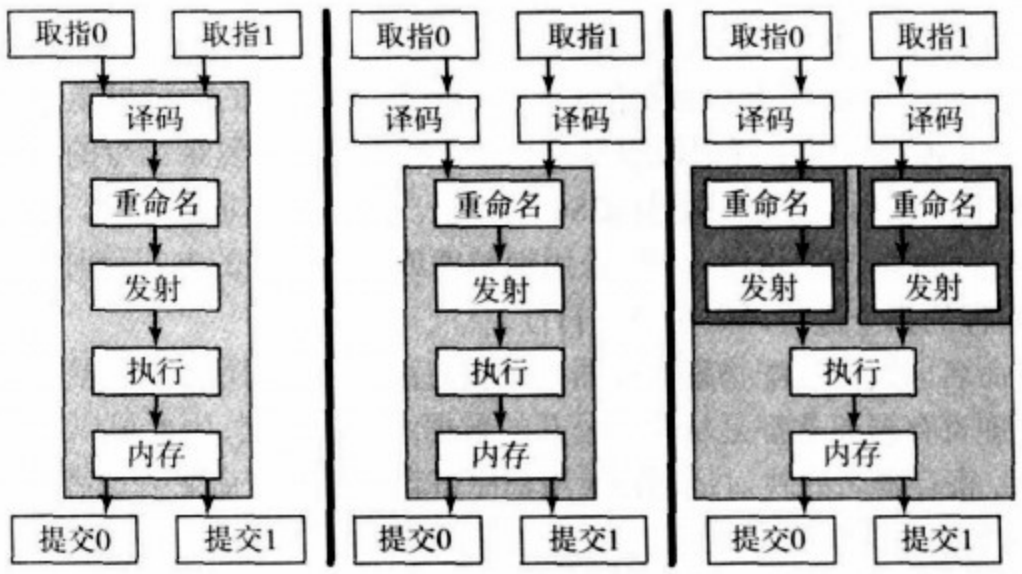


图9.15　SMT资源共享的不同情况

段间缓冲的实现　与超标量处理器设计类似，SMT设计中的关键问题之一是段间缓冲的实现。如果复制了取指段或者译码段，如图9.15的中间和左边的设计所示，处于复制段交汇处的段缓冲必须支持多个写操作同时进行。这会增加设计的复杂性，因为前一个周期只能进行一个写操作。另外，用于记录程序中指令顺序的load/store队列和再定序缓冲必须重新进行设计，以适应多线程的需要。如果这些缓冲被划分为每个线程一份，其设计则与传统结构的设计很类似。当然，这种划分设计会使单线程的性能无法达到最佳状态，因为现在单线程不能占有所有可用的资源。在多个线程之间共享再定序缓冲带来了额外的复杂性，因为必须分别跟踪每个线程的程序顺序，而且再定序缓冲必须能够选择性地清空一些不连续的表项，以支持每个线程的分支预测失误的恢复。这就需要复杂的管理功能，再定序缓冲不能再被看做是简单的循环队列。load/store队列（LSQ）也有同样的问题，由于存储一致性模型已经隐含地规定了LSQ如何化解存储器数据相关，这会使设计进一步复杂化。后面我们将简单地讨论一下这一问题。

SMT的流水段共享　对于如何有效地共享每个流水段中的资源，其影响因素有很多，我们将针对各个流水段讨论其中的一些问题。这些讨论将基于图9.15所示的流水线结构。

- **取指：**指令段最昂贵的资源是指令cache端口，由于cache端口被限制为只能访问相邻的地址，所以要在多个线程之间共享单个cache端口是很困难的，因为多个线程不太可能都从连续的地址甚至只是相对集中的地址取指令。因此，SMT的设计倾向于为每个线程单独提供

个取指段，或者以细粒度或粗粒度的方式分时共享单个端口。双端口设计的造价很高，而且难以证明是有效的，所以实际中的SMT设计倾向于采用分时共享的方式。另一个昂贵的资源是分支预测器。同样，双端口的预测器相当于有效容量减半，所以分时共享的方法可能更有意义。然而，现在的分支预测器的某些原理是基于串行程序的语义，在多个线程任意交叉执行时表现并不好。例如，返回地址栈（RAS, Return Address Stack）依赖于程序调用和返回FIFO（先入先出）的行为方式，如果多个线程的调用和返回是交叉的，那么RAS将不能可靠地工作。同样，多个交叉执行的线程产生的分支使得全局分支历史寄存器（BHR, Branch History Register）的内容很凌乱，因此任何依赖于BHR的分支预测器的表现都很糟糕。因此，在分时共享的分支预测器设计中，至少要为每个线程设置一个BHR这样的部件。

- **译码：** 对于简单的RISC指令集，译码段的主要任务是确定源和目的操作数、化解指令相关。译码宽度（decode group width）为n的译码器其逻辑复杂性为$O(n^2)$，包括操作数标识符比较器和优先级译码器的实现。根据定义，不同线程之间的指令是不存在指令相关的，所以为了降低译码器的复杂性，可以将译码器按照线程进行划分。例如，两个宽度为4的译码器可以同时对两个线程进行译码，它的逻辑要比一个共享的宽度为8的译码器简单一些。当然，在单个线程每个周期能够提供8条指令译码的情况下，这样的设计可能会影响到单个线程的性能。对于CISC指令集，译码段要复杂得多，因为要确定复杂指令的语义，通常还要将复杂指令分解成一系列简单的、类似RISC的原语。由于译码非常复杂，在线程之间共享译码器更有意义。而且，与取指段一样，采用细粒度或者粗粒度的方式分时共享译码器，要比同时对多个线程的指令进行译码更具可行性。
- **重命名：** 重命名段负责分配物理寄存器，并将程序中显式使用的寄存器名映射到物理寄存器名。由于物理寄存器通常都是从一个公共的数据结构中分配，因此可以共享管理空闲列表的逻辑。然而，寄存器名的映射是通过使用程序寄存器索引重命名表或者映射表来完成的，要么更新映射（目的操作数），要么读映射（源操作数）。由于程序寄存器名在各个线程中是分离的，可以根据线程来划分重命名表，这样能够提供较高的带宽，而代价比真正的多端口低得多。然而，这意味着将重命名段按照线程进行划分，与译码段一样，这会影响包含大量指令级并行的程序的单线程性能。
- **发射：** 发射段实现了Tomasulo算法，通过两个步骤对指令进行动态调度，这两个步骤是唤醒并选择。唤醒数据准备完毕的指令，然后从准备好的指令中进行选择，以满足结构相关。显然，如果多个线程同时共享功能部件，选择过程必须包含多个线程的指令。然而，指令的唤醒显然局限于线程内部，即是否唤醒一条指令只取决于同一个线程中前面指令的执行结果。因此，可以按照线程对发射窗口进行划分，因为唤醒操作不会跨越这种划分。当然，与前面的段一样，划分可能会影响单线程的性能。然而，一些研究人员称，在以后的处理器技术中，发射窗口逻辑会成为关键的时间限制路径之一。对这个逻辑进行划分，以利用多个数据流分离的线程的并行性，在固定的时钟周期延迟内可以获得大得多的发射窗口，从而提高SMT的吞吐率。
- **执行：** 执行段在功能部件上执行每条指令，以实现指令的语义。要实现多线程，共享功能部件是一种很直接的方法。此外，还可以利用多线程对旁路逻辑（即定向路径）进行简化，因为来自不同线程的指令之间不需要传递结果。例如，在Alpha 21264的集群化体系结构中，可以修改发射逻辑，将来自同一个线程的指令引导到同一个集群，从而减少了跨越集群传递结果的可能。另外，发射逻辑通过用其他线程的不相关指令来填充相关指令之间的空白，能够避免连续发射相关的指令，因此不再需要从ALU的输出到输入的定向路径。同样，这样的优化也会影响单线程执行的性能，除非优化之后能够达到更高的频率。

- **访存**：访存段访问cache，满足load指令，还负责化解存储器相关，并执行其他一些与存储器有关的记录任务。在多个线程之间共享cache访问端口，尽量提高端口的利用率，这是SMT设计的主要目标之一。这可以通过一个相当直接的方法实现。然而，共享检测和化解相关的硬件要复杂一些。这些硬件由load/store队列组成，它记录程序的访存时序，并检测后面的load是否和前面的store相关。要使load/store队列支持多线程，需要理解存储一致性模型，因为有些模型（如顺序一致性模型，参见9.3.6小节）禁止一个线程去读取另一个线程所存的值。要处理这样的情况，load/store队列必须能够识别线程，在允许操作的时候才能进行值的传递，不允许操作的时候就要停顿相关的取操作。分别为每个线程提供单独的load/store队列可能要简单一些，当然，这样会降低线程间共享资源的程度，也会将单个线程使用的队列大小限制为它所占有的划分的大小。
- **提交**：在流水线的提交段，指令的结果按照程序的顺序提交。这包括检查异常或者其他不正常的情况，然后提交指令的结果。指令结果的提交可以通过更新重命名映射（基于物理寄存器文件的设计），或者将重命名寄存器的值复制到程序所使用的寄存器中（基于重命名寄存器的设计）。不管哪种情况，超标量处理器的提交段必须检测并保证WAW相关（因为上次提交的写操作必须先完成），可能要求重命名表或者寄存器文件具有多个端口。同样，将检测硬件按照线程进行划分将有利于设计的实现，因为WAW相关只可能发生在线程内部，结果的提交在线程之间不会引起冲突。另一个办法是，由于提交段的延迟和带宽不是很关键，可以按细粒度或者粗粒度的方式分时共享提交段。

目前，还没有关于以上任何一种资源共享方式的透彻的研究。从目前了解的有限情况来看，Pentium 4的SMT设计似乎同时共享了发射、执行和访存段的大部分，但是以粗粒度的方式共享流水线的前端，以细粒度的方式共享提交段。因此，Pentium 4显然在SMT思想与实现限制之间进行了折中，SMT思想要求尽可能地共享更多的资源，而在实现中这样做要受到时钟周期和复杂性的限制。

SMT对串行化指令的支持　所有指令集都包含具有串行化语义的指令，比较典型的指令包括影响全局状态（如改变处理器特权级别或者作废一次地址转换）的指令，或者对访存操作加以顺序限制（如在9.3.6.3小节中讨论的存储器栅栏）的指令。这些指令通常采用“暴力”的方式执行，如将当前流水线排空，清除掉所有活跃的指令，再执行这些指令，然后重新开始发射后面的指令。之所以要使用这种方式是因为这种指令相对而言很少出现，即使实现的效率很低也不会对性能有很大的影响。而且，这些指令的语义很微妙，难以用更加有效的方式正确无误地实现。

然而，在SMT设计中，串行化指令出现的频率可能显著增加，因为出现频率是与线程的数目成正比的。例如，在单线程的处理器中，假设串行化指令每600个周期出现一次，而在一个4线程SMT处理器中，假设指令吞吐率是单线程的3倍，这样串行化指令每200个周期就会出现一次。显然，要保持高性能，需要更加有效、更加主动的方法来实现这些指令，因为每200个周期就将流水线排空一次，从而大大降低了性能。改变全局状态的串行化指令的执行，可以通过对全局状态进行重命名实现流水化，这和消除指令间假相关的寄存器重命名一样。一旦全局状态被重命名，只有读取这个状态的后续指令才需要推迟执行，而前面的指令可以继续读取以前的版本。因此，串行化指令前后的指令可以在处理器的指令窗口中混合出现。然而，全局状态的重命名可能不像说起来那么容易。例如，对于TLB缓冲或者其他地址转换和保护结构的串行化更新，需要重命名大量存储器结构。遗憾的是，这会增加访问延迟并且这些访问路径可能都在关键路径上。最后，将存储器栅栏指令流水化需要解决无数与存储一致性模型有关的问题，我们在9.3.6.3小节中已经讨论了其中的一

些问题。一种可能的方法是，有选择地为某个线程排空流水线，但仍然允许其他线程并行执行。显然这也要对再定序缓冲的设计进行相应的变化，同时发射逻辑必须有选择地停止发射某个线程的指令，而允许发射其他线程的指令。无论哪种情况，复杂的实现并非没有价值，目前仍有大量的工作需要完成。

多线程的管理 在9.4.3小节中我们讨论了许多关于粗粒度多线程的问题，在某种程度上SMT也存在同样的问题。也就是说，处理器的调度策略必须保证一定的公平性，必须保证每个活跃的线程都能向下执行。同样，必须提供优先级策略，防止无用指令（旋转循环、空循环）消耗执行资源。另外，还要能够提高优先级、保证性能很关键的线程能够尽量提高吞吐率。然而，由于一个纯粹的SMT设计没有线程切换的概念，实现上述策略的机制就有所不同：SMT通过在发射逻辑的指令选择上设定优先级，从而能够在更细的粒度上管理执行资源的分配，而不是换出一个低优先级的线程或者换入一个高优先级的线程。通过控制线程的取指，可以防止不同优先级上的线程占有处理器的执行窗口超过某个固定数目。处理器中任何动态分配的资源都有类似的限制。例如，限制每个线程对load/store队列的占有，从而限制了线程对存储器的压力；或者限制每个线程对MSHR的占有，从而限制了每个线程未处理完的cache失效的数目；还可以限制每个线程在分支或者值预测结构中的表项，从而为更高优先级的线程留出更多的资源。

SMT的性能和耗费 显然，有多种因素可以影响SMT的性能。例如cache、预测器结构中线程之间的冲突。一些已经公布的资料显示，这种冲突并不过分，特别是在二级cache这样较大的结构中，但是对于主cache和其他较小结构的影响还不是很清楚。到目前为止，关于SMT的性能潜力，惟一确定的证明来自Intel，它宣称当运行包含大量线程级并行的服务器负载时，Pentium 4的吞吐率有16%到28%的提高空间。接下来将总结一些已经公开的Pentium 4的SMT设计细节。由于Pentium 4的并行性有限，它只支持两个线程，而且只在发射、执行和访存段实现了真正的SMT，所以性能的提高比一些研究文献中所描述的要低很多，这并不奇怪。然而，这些文献中所描述的方法是否可行还不是很清楚，也不清楚即使解决了上面讨论的所有实现上的问题后，SMT的设计能否扩展到超过两个线程，比如3个线程。当然，实现SMT的耗费，包括实现的复杂性（如资源重置），目前已经为人们所充分了解。

Pentium 4的混合多线程实现 Intel Pentium 4处理器采用了混合形式的多线程，在逻辑上有两个处理器共享执行资源。Intel的实现方案（称为混合多线程）与学术文献中提出的SMT在概念上比较类似，但实质上是不一样的。从目前了解的有限的情况来看，Pentium 4流水线中的时序部分（如前端的取指译码部件和提交段）是细粒度的多线程。也就是说，两个逻辑线程按时钟周期交叉取指、译码、提交，除非其中的一个线程因为某种原因而停顿。如果有一个线程停顿，另一个线程就占有所有的资源，直到停顿的线程恢复执行。这样的机制也可以称为基于单个周期的粗粒度多线程。Pentium 4还实现了两级调度逻辑，指令被放到第一级逻辑的5个发射队列，然后在第二级逻辑被发射到功能部件。同样，第一级调度是细粒度多线程：在每个给定的周期只有一个线程能够将指令放入发射队列。同样，如果一个线程停顿，另一个线程能够继续将指令放入发射队列，直到停顿的线程恢复执行。实际上，Pentium 4将细粒度和粗粒度多线程结合起来。然而，Pentium 4并没有在发射段以及执行和访存段真正实现同时多线程，即允许两个线程的指令按照任意的方式交叉执行。

Pentium 4的资源共享也有一些复杂。流水线中乱序部分的大多数缓冲（如再定序缓冲、load队列、store队列）被对半分开，而不是任意共享。调度队列的划分不是很固定，但对线程占有所有表项进行了限制。前面已经讨论过，为了在两个线程都活跃时能达到较高的吞吐率，对资源进行这样的划分牺牲了单线程可能达到的最大性能。从较高的层次上来看，当两个线程的行为具有很大的对称性时，这样划分的效果会比较好，但是如果两个线程不对称并对资源有不同的利用要求，其性

能就会很差。然而，Pentium 4 支持单线程模式并可以取消所有的资源划分，使得单个线程能够占有所有的资源。这样就改善了出现这种情况时的性能。

9.5　隐式多线程处理器

到目前为止，我们对多线程处理器和多处理机系统的讨论都是基于显式的、程序员开发的多线程来提高指令吞吐率。然而，存在很多重要的应用，它们的单线程性能极为重要。一种提高单线程应用性能的方法是，将单线程分成多个并行执行的线程。与程序员手工将程序并行化不同，隐式多线程（IMT, Implicit Multithreading）利用程序控制流的特点，自动生成多个线程。

通常，科学计算任务的并行化都是通过编译器自动完成或者是手工完成的，一般都是面向几十到上百个CPU来提取线程级并行，从而获得很高的加速比。与此不同的是，隐式多线程只有几个到十几个线程。规模上的区别主要是由隐式多线程紧密的耦合特性决定的，因为执行的线程相对比较短（数十条指令），并且经常需要与其他线程进行大量的通信，以解决数据和控制相关。而且，由于推测执行的大量使用，需要尽快从推测失误中恢复，这也使得线程之间的耦合度很高。综合这些因素，使得隐式多线程中的线程数目很难提高。不过，对于一些使用传统线程级并行方法效果不太理想的应用，有资料显示，隐式多线程技术表现出了良好的加速性能。

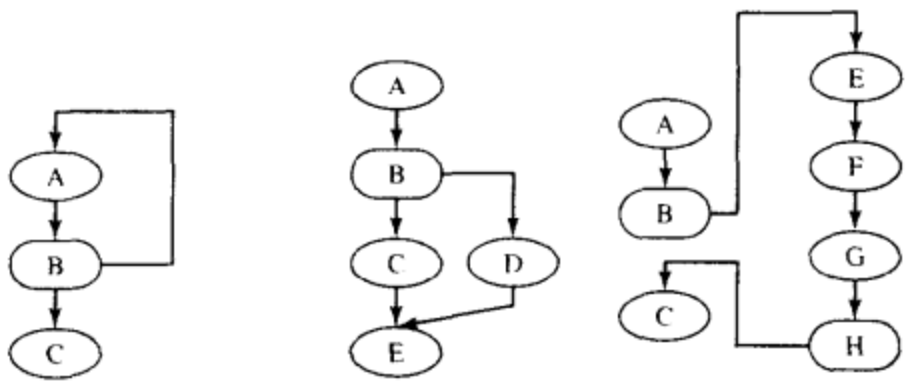
IMT技术产生之初是想尽可能地提取指令级并行，用程序控制流中多个分离位置上的指令填充一个较大的共享执行窗口。另一些IMT技术认为，IMT是一种建立可扩展性更好的指令窗口的方法：多个独立的隐式线程可以分派到独立的处理单元上执行，避免了对集中的共享执行窗口的需求，从而减少了许多实现上的复杂性。当然，这种分散的设计仍然必须提供一种满足多个处理单元间数据相关的方法，目前很多研究都集中在对这个问题的有效解决上。

从根本上说，IMT处理器设计主要有三大挑战。这三大挑战与超标量处理器设计所面临的挑战是相同的，即：化解控制相关、化解寄存器数据相关以及化解存储器数据相关。然而，由于IMT设计的一些独有特性，相关化解要困难得多。一些技术完全依赖于硬件机制来化解相关，而另一些则偏重于对编译器的依赖。我们将对每一个挑战进行讨论，并描述一些文献中给出的解决方案。

9.5.1　化解控制相关

IMT的主要争议之一是，要构造一个足够大的、具有大量指令级并行的线程是非常困难的。构造线程的传统办法是使用分支预测器来推测程序控制流的流向，这种方法受限于累计的分支预测正确率。例如，即使正确率达到95%的分支预测器连续经过10次预测之后，正确率也将下降为60%。由于许多重要的程序在条件分支之间只有5条到6条指令，这使得在分支预测器的失误率不可接受之前，只能构造出一个只有50条到60条指令的线程。因此，提高分支预测的正确性仍然是活跃的研究领域，然而，要进一步改进正确率已经达到95%的分支预测器，所需要的硬件代价将是十分昂贵的。而且，分支预测的正确性是否能够显著提高还不是很清楚。

控制无关　所有已经提出的IMT设计都利用控制相关的特点来增加指令窗口的大小。我们称程序控制流图中的一个节点是控制无关的，无论该节点与当前节点之间的条件分支如何化解，都会执行到这个节点。图9.16说明了程序中几种控制无关的情况。在IMT设计中，隐式线程可以产生于控制流的汇合处、子过程的返回地址处、循环体或者循环跳出处。这些线程的产生是不需要推测的，因为控制无关保证了程序最终都会到达这些线程的起始点。然而，也可以通过推测来产生，这样可以在线程内部包含一些生成线程时还不能完全确定的控制流。例如，遍历链表的循环次数可能与某个数据相关，此时为多个循环体推测地产生线程通常能得到较好的性能，即使后来发现其中一些推断的线程是错误的。



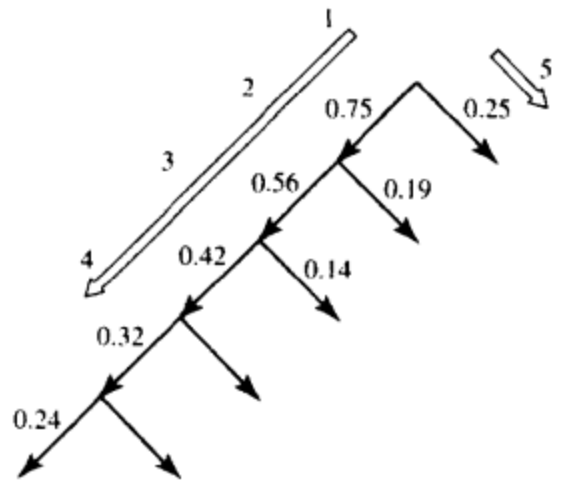
（a）循环关闭　　（b）控制流收敛　　（c）调用/返回

控制无关存在多种来源：在（a）中，由于循环次数是确定的，块C最终将位于块B的后面；在（b）中不管分支如何执行，块E总是在块B之后执行；在（c）中，当调用E的子程序结束后，块C总会在块B之后执行。

图9.16　控制无关的来源

在程序控制流中，在控制无关的节点上产生隐式线程有几个好处。首先，中间任何错误预测的分支指令不会直接影响到控制无关的线程，因为不管程序控制流如何变化，最终都会执行到该节点上。因此，利用控制无关性可以使处理器跳过难以预测的分支，从而发现必然执行的指令。其次，跳过指令具有积极预取的效果。也就是说，从程序控制流中某个将来的节点上读取指令，可以有效地将当前线程的执行和将来的线程可能引起的cache失效重叠在一起。反过来，当前线程也可能发生指令cache失效，也可以和未来线程的执行重叠起来。注意，这种预取效果是传统的单线程执行无法达到的，因为对单线程而言，当前线程和未来线程的取指是串行化的。这种预取的效果是很显著的，Akkary的报告称，DMT（Dynamic Multithreading）处理器的提交指令有40%的指令是在发生指令cache失效时取出的。

主动分离执行　另一种创建隐式线程的方法是在主动分离执行的结构（DEE，Disjoint Eager Execution）中提出来的。传统的主动执行通过同时执行分支的两条路径来克服条件分支。显然，当分支增多时，执行的路径数目会急剧增加。在DEE设计中，对分支判定树的每一个分支的累积分支预测率进行比较，并选择累积预测率最高的分支执行，从而实现了对判定树的剪枝。这一过程如图9.17所示。每个静态分支的预测率可以通过profiling估算，累积的分支预测率通过将达到该分支所经过的所有分支的预测率相乘得到。然而，由于实现的原因，[Uht 1995] 发现，如果每个分支具有统一的静态预测率，将得到一种直接的取指策略，即从分支树上后退固定层数的分支取指，这样只需要一个传统的分支预测器，它提供的主路径将与这些候选路径交叉执行。这些候选的路径在DEE中就称为隐式线程。



假设每个分支预测的准确率是75%，图中显示了累积的分支预测率，经过分支路径1、2、3和4，下一个最高的累积准确率是分支路径5，所以下一步从路径5取指。

图9.17　主动分离执行 [Uht 1995]

表9.3对4种IMT技术进行了总结，列出了它们所利用的控制流特性和隐式线程的来源、创建、排序、执行以及相关的化解。对于线程由编译器创建的情况，可以对程序控制流进行静态分析来决定创建线程的位置。还有一种线程级的推测技术（TLS, Thread-Level Speculation），在编译时，为循环的每个循环体创建一个线程。多标量（Multiscalar）技术在编译器创建线程时的灵活度更大，可以在程序控制流中的任意一点创建线程（在多标量中称为任务）。DEE技术在单个指令窗口内动态地检测并利用控制无关性，但是也通过在分支预测树中后退来创建线程，如图9.17所示。最后，动态多线程技术使用启发式检测技术对硬件进行检测，并在过程调用和向后循环的分支处创建线程，遇到过程调用时，过程调用与其后的程序流同时执行。类似地，遇到循环的返回时，下一个循环体与循环后面的程序流同时执行。

表9.3　几种隐式多线程方法的特点

	多标量	主动分离执行（DEE）	动态多线程（DMT）	线程级推测（TLS）
控制流特点	控制无关性	控制无关性 累积的分支预测失误	控制无关性	控制无关性
隐式线程来源	循环体 控制流汇合点	循环体 控制流汇合点 累积的分支预测失误	循环退出 子程序返回	循环体
线程创建机制	软件/编译器	硬件隐式创建	硬件隐式创建	软件/编译器
线程创建和顺序化	程序顺序	乱序	乱序	程序顺序
线程执行	分布的处理单元	共享处理单元	共享的多线程处理单元	单独的CPU
寄存器数据相关	软件硬件推测支持	硬件，无推测	硬件，数据相关预测和推测	不允许出现，编译器要避免发生
存储器数据相关	基于硬件的推测	硬件	硬件，预测和推测	相关推测，利用MESI一致性的简单扩展检查

线程的乱序创建　动态多线程所独有的一个挑战是线程产生的顺序与程序的顺序不一致。例如，在过程嵌套调用的时候，第一次调用会产生一个线程来执行这次调用，同时还会产生一个线程来执行过程返回处之后的指令，这样就有两个活动的线程。如果被调用过程的代码又遇到嵌套调用，并且产生另一个线程来执行这次调用，这样就有三个活动的线程。然而，该线程虽然是第三个被创建的，实际上在程序顺序中它比第二个线程早出现。结果是，再定序缓冲现在必须能够将任意数目的指令插入到一组活动线程的指令中间。我们还会发现，由于线程的乱序创建，化解寄存器和存储器数据相关的过程也会变得更加复杂，这种方法是否可行还有待考察。

物理组织　显然，构造一个较大的指令窗口只是整个工作的一半，任何试图从这样的指令窗口发现并利用并行性的设计必须证明其可行性。许多IMT技术对处理器的执行资源进行划分，这样每个线程能够独立地执行，从而实现分布的、可扩展的指令级并行提取。由于划分出来的执行资源只需要包含单个线程的指令窗口，因此每个划分的执行能力不需要超过当代的处理器。实际上，它的执行能力可能还弱一些。多个这样的指令窗口重叠执行，可以获得额外的并行性。对于线程级的推测（TLS），每一个划分实际上是一个独立的微处理器核，这与单芯片多处理器（CMP，参见9.4.1小节）十分类似。相反，动态多线程（DMT）依赖于一种类似于同时多线程（SMT）的多线程执行内核，交叉执行多个隐式线程，而不是显式线程。DMT还提出了一个两级再定序缓冲，它能够支持非常大的指令窗口。执行结束但还没有提交的线程将转移到第二级再定序缓冲中，只有当发生预测失误并需要重新执行时，该线程才从第二级缓冲中取出。最后，主动分离执行（DEE）的处理器拥有一个集中的指令窗口，并基于静态的程序结构来组织指令窗口，同时执行多个隐式

线程，而不是只有一条动态的路径。这就是说，DEE的指令窗口看到的是程序的静态视图，硬件同时执行同一静态控制流的多个动态实例。

最后，多标量（multiscalar）处理器的结构可以描述为处理单元的循环队列。队列的尾节点不需要推测，即可执行当前线程或任务；其他节点执行未来的、基于对控制和数据相关的推测任务。尾节点执行结束时退出，下一个节点将成为无推测的尾节点。同时，产生一个新的未来线程并占有尾节点执行结束之后释放的资源。按照这种方式，通过在多个处理单元之间重叠执行，可以获得单个处理单元以外的并行性。

线程的顺序化和提交 IMT处理器设计的主要挑战之一是，控制或者预测硬件必须将线程按照程序的顺序进行顺序化和提交。依赖编译器的线程创建方式则比较容易处理。同样，基于队列的结构组织方式，如多标量，至少可以在概念上简化这一任务。然而，所有的方法都需要控制逻辑确定一个线程或者任务在提交以前没有发生正确性冲突。控制相关相对而言直接一些，当无推测的控制流最终到达需要提交的线程，然后控制流离开该线程并向下一个推断的线程前进时，该线程就可以安全地提交。然而，解决数据相关就要复杂一些，我们将在下一小节中讨论这一问题。

9.5.2 寄存器数据相关化解

寄存器数据相关包括名相关（WAR）、假相关（WAW）和真相关（RAW）。在IMT处理器设计中，与传统的超标量处理器设计一样，前两种相关可以通过寄存器重命名和按序提交来化解。惟一复杂一些的是按序提交必须在多个线程之间进行协调，但是通过按程序的顺序提交线程可以很容易地解决这一问题。

真相关可以分为两种类型：线程内部的相关和线程之间的相关。线程内相关可以通过前面章节中学习的标准技术来化解，因为线程内部的指令是按照程序顺序排列的，可以进行重命名，同时借助于定向路径并使用传统的方法进行提交。然而，对于线程间相关，由于指令的顺序和程序的顺序不一致，所以要复杂一些。由于这一原因，很难确定正确的生产者和消费者关系，当消费者（读寄存器指令）马上就要执行时，生产者（写寄存器指令）可能还没有译码。例如，在一个新线程的开始部分要读一个寄存器，而对这个寄存器的最后一次写要在前面某个线程的末尾才发生，这时就会出现上面的情况。由于前面的线程忙于执行其他指令，执行最后一次写的指令甚至可能还没有取出来。对于这种情况，传统的重命名机制不能正确地检测到真相关，因为生产者还没有更新重命名信息来反映它将要进行的写操作。因此，为了保证正确执行，需要简化编程模型，或者采用更加复杂的重命名机制。

化解线程间寄存器数据相关最简单的方法是简化编程模型，在编译的时候就不允许相关的出现。线程级推测（TLS）采用了这种方法。由于编译器将创建并行执行的线程，所以只需要强制所有共享操作数之间的通信必须通过存储器读/写。这样，寄存器相关都位于线程内部，因此只需要使用我们早已熟悉的技术，例如寄存器重命名或者Tomasulo算法来化解相关，这与单线程处理器一样。

与之不同的是，多标量允许线程间通过寄存器通信，但是也需要编译器的帮助，从而确定线程间的寄存器相关。编译器检查未来线程，确定寄存器文件中哪些寄存器会被写入，并将最后一次写这些寄存器的指令标记出来。这样，当写寄存器发生时，前面线程的处理单元就知道将结果传递给未来线程。编译器将这些信息嵌入到一个写操作掩码中，当未来线程被创建时，该掩码将提供给未来线程。这样，借助编译器提供的帮助信息，就有可能使用相对简单的硬件有效地实现一个分布的、可扩展的相关化解机制。

然而，DEE和DMT没有使用编译器的帮助，而是动态地化解相关。DEE构造一个类似单线程的执行线程，按照程序顺序在这条路径上取指并译码。因此，这条路径上的数据相关相对比较容易确定。另一条候选的主动执行路径，在我们的讨论中作为隐式线程，也具有类似的顺序语义，所以能够化解相关。DEE也通过实现最小控制相关（MCD, Minimal Control Dependences）来检测控制相关。实现MCD的硬件能够确定并化解分支控制流路径之间的数据相关。有兴趣的读者请参考[Uht 1995]，文中对这种硬件机制进行了描述。

另一方面，DMT的指令流都是乱序的，因此，DMT最主要的任务是确定最后一次对某些寄存器的写操作，而将来某个线程将会读这些寄存器，因为执行写的指令可能还没有被取出或译码。一种十分简单的解决方法是，假设所有的寄存器都会被当前线程写，并延迟未来线程对寄存器的读请求，直到当前线程所有的指令都已经取指并译码。当然，这种方式的性能极差。因此，DMT依赖于对数据相关的推测，未来线程假设寄存器操作数已经存放在寄存器文件中，并使用这些操作数进行推测执行。当然，如果前面的线程对这样的寄存器进行了写操作，未来线程必须重新执行这些指令。DMT有一套复杂的相关化解机制，能够在检测到发生相关冲突时进行恢复执行。另外，研究人员已经研究出一种自适应的预测机制，能够根据历史信息来预测相关的寄存器写操作。当预测器确定了这样的写操作后，未来线程中的相关指令将被停顿，这样就能够防止使用陈旧的数据。通过采用值预测机制，寄存器相关问题能够得到简化。在发生相关或者难以确定的时候，可以预测操作数的值，以后再进行验证。在第8章中所讨论的关于值预测、验证和恢复技术可以应用于这种设计方案。

9.5.3　存储器数据相关化解

最后，隐式多线程处理器设计还必须正确化解存储器数据相关。同样，将问题分解成线程内和线程间的存储器数据相关将有助于我们的讨论。线程内的存储器相关与线程内的寄存器相关一样，可以通过前面章节中介绍的传统技术加以解决：WAW和WAR相关通过写缓冲来化解，RAW相关则通过流水线停顿或者重定向化解。

线程间的名相关和假相关（WAR和WAW）也可以通过一种直接的方式化解，即缓存未来线程写操作的结果并在这些线程提交时进行确认。IMT的各种实现方法之间有一些细微的差异。DEE和DMT使用类似传统的load/store队列来缓存写操作的结果。多标量使用一个复杂的机制，称为地址解析缓冲（ARB, Address Resolution Buffer）来缓存写结果。最后，TLS对传统的MESI cache一致性进行了扩展，允许被将来线程写入的cache行存在多个副本。这些副本用数字进行标识，该数字随每个新的线程递增。这个数字附加在cache行地址的后面，从而使传统的MESI一致性支持cache行中多个修改过的副本。提交逻辑负责确认这些修改过的行，只要一个线程成为非推测的，就将它们写回存储器。

线程间的真存储器相关（RAW）要比真寄存器相关复杂得多，尽管它们在概念上比较接近。主要的问题是相同的：由于指令取指和译码的顺序与程序的顺序不一致，后面的读操作不能获得前面写操作的信息，因为这些写操作可能还没有计算出目标地址来，甚至可能还没有取出来。

TLS存储器的RAW相关化解　同样，TLS采用了最简单的方法：将来的线程简单地认为没有发生相关冲突并推测地使用最新的值。这可以通过对传统的、基于监听的cache一致性协议进行简单的扩展而实现：当一个推测执行的线程执行了一个引起cache失效的读操作时，其他处理器的cache按照与程序相反的顺序搜索匹配的地址。通过按照与程序相反的顺序搜索（即按照线程创建的相反顺序），找到最后一次写操作（如果存在），并用该值满足引起失效的读操作。如果没有找到匹配的地址，则只能通过存储器来满足读操作。在效果上，TLS机制假设任何相关的读操作和

写操作距离足够远，当后面的线程进行读操作时，前面的线程已经将结果写回存储器，换句话说，就是当后面的线程发生读失效时，将会产生一个监听命中。只有当读操作和写操作实际上没有按照正确的顺序执行时，才会发生错误的预测。

当然，由于TLS采用了较简单的数据相关推测形式，需要有一套机制来检测并恢复相关冲突。同样，采用的方法是对已经存在的cache一致性协议进行简单地扩展。需要处理的情况有两种：首先，未来线程的读请求是通过存储器满足的；其次，未来线程的读请求通过前面的线程所修改的cache行满足。对于前面一种情况，cache行被放入未来线程的cache中并处于专有状态，因为它是系统中惟一的副本。然后，前面的线程对同一cache行执行了写操作，因此引起了一次潜在的相关冲突。为了执行写操作，前面的线程必须监听其他cache，以获得对这一行的专有访问。此时，未来线程相应行的cache副本被发现了，该线程则因为冲突而作废。对于后面一种情况，即未来线程的读请求通过前面线程所修改的cache行满足，情况非常类似。这一行被放入未来线程的cache中并处于共享状态，同时在前面线程的cache中也变为共享状态。这就是当满足一个远程的读请求时，修改过的cache行所产生的动作，如图9.6所示。当前面的线程再次向这一行写时，必须通知其他处理器的cache作废它们的副本，这时，未来线程的共享副本被发现了，因而触发了一次冲突。恢复机制很简单：线程被作废并重新执行。

DMT的存储器RAW相关化解 DMT处理真相关的方法是在每个线程各自的load/store队列中记录读写操作。这些队列可用于处理线程内的存储器相关，也可以用于处理线程间的存储器相关，方法是一旦发出一个读请求，就搜索前面线程的store队列；而一旦发出一个写请求，就搜索后面线程的load队列。如果搜索到匹配的内容，对于前面一种情况，将把数据传递给相关的读请求；对于后面一种情况，则发生了冲突，因为旧的数据已经被读过了，这将导致后面的线程重新发射读请求以及相关的指令。在实际效果上，DMT的处理机制实现了存储器重命名，因为同一个存储器数据的副本在任何时候都可以访问，只要所有的写操作都已经执行并放到store队列中，相关的读操作就能读到正确的副本。当然，如果前面一个写操作仍然未能执行，这套机制就不能获得正确的相关信息，读操作就可能读到错误的数据，从而在执行到写操作时必须重新进行。

DEE的存储器RAW化解 DEE采用的机制在概念上与DMT类似，但是描述得更加详细一些。DEE使用了一个地址交叉、高吞吐率的结构，能够记录程序顺序并检测RAW相关。同样，由于读写可能没有按照顺序进行，检测机制必须在逻辑上将它们按照程序的顺序排列，只有当冲突发生时，才将其标记出来。由于DEE的取指策略产生的隐式线程同样包含写操作，因此必须对每个线程进行跟踪，这将使检测机制复杂化。

多标量的ARB机制 多标量采用了地址解析缓冲（ARB，Address Resolution Buffer），这是一种集中的、多端口的、地址交叉的结构，能够化解RAW相关。当未来线程发出推测的读请求时，该结构为每个这样的读请求分配一个记录表项，然后当前面的线程执行写操作时，将写操作与这些表项进行比较，如果有匹配的项，就可以确定发生了一次冲突，发生冲突的未来线程将被作废并重新执行。同样，每次读操作也与所有先执行的写操作比较，这些写操作也存放在ARB中。如果存在数据相关，就从ARB中获得数据，而不是从数据cache中获得。需要注意的是，对于WAW的操作顺序，这些先执行的写操作可能给更早的、尚未执行的写操作造成障碍。例如，假设未来线程n+1向地址A写数据，这次写操作被存放到ARB中。然后，未来线程n+2从地址A读数据，这次读操作将从ARB中线程n+1的表项获得数据。最后，当前线程n向地址A执行写操作时，如果实现机制过于简单，将会发现未来线程n+2执行了一次读操作。这样，线程n+2以及以后的线程都会被作废并重新执行。然而，由于线程n+1在中间插入了一次写操作，实际上并不会发生相关冲突，线程n+2没有必要作废。

实现的挑战　对于ARB以及那些记录所有读写操作的集中机制，其最大问题在于可扩展性。由于每个处理单元对这一结构的带宽要求都很大，扩展到数目较多的处理单元则非常困难。TLS使用标准的cache协议过滤了一部分需要记录的操作，从而避免了可扩展性的问题。外界只需要了解cache失效和cache部件状态的改变，并排列和比较一小部分地址引用。线程内部的排序由传统的load/store队列完成。对于ARB，多标量提出了一种基于cache的改进方法，称为推测形式cache（speculative versioning cache）。当然，基于cache的过滤方法将会出现地址粒度导致的错误相关。也就是说，由于cache一致性协议是基于块进行操作的，这些块要比单个字大得多（通常为32字节到128字节）。当向某一块的某一个字写数据，而从同一块的另一个字读数据时，将会错误地引发一次相关冲突，从而导致额外的恢复开销，而这种情况不会出现在细粒度的相关检测机制上。

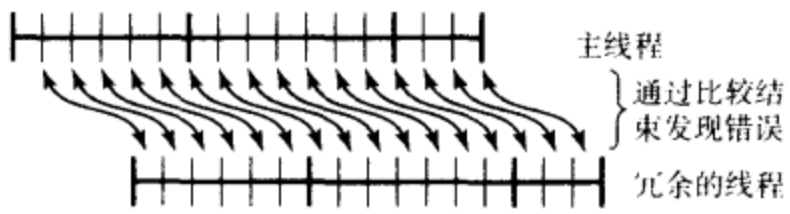
其他有关存储器相关检测的问题都比较简单。例如，有限的缓冲空间会影响有效的推测。同样，提交的带宽也会产生限制，特别是对TLS系统，因为提交包括将修改过的行写回存储器。如果一个推测的线程修改了大量的行，写回带宽则会影响性能，因为要等所有的写回都执行完毕之后，才能产生新的将来线程。最后，TLS以及其他细地址粒度的方法在化解相关时，都存在搜索写操作最新版本的问题，而且这一搜索过程必定是串行化的。TLS是按照与创建线程相反的顺序串行搜索其他处理器的。其他一些IMT技术提出了并行的关联查找，这种方式的速度要快一些，但是其代价也更昂贵，难以扩展到数量很大的处理单元上。

9.5.4　结论

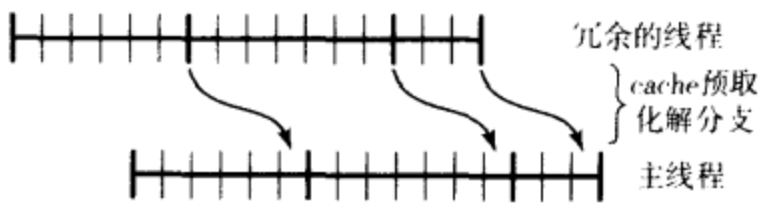
到目前为止，隐式多线程还处于研究的阶段。虽然其动态提高性能的能力超过了单线程，但是目前还不能确定其实现问题，以及其他一些直到真正实现时才会讨论的问题。当然，随着单芯片多处理器技术的广泛应用，很有可能在这样的系统上实现线程级的推测执行。然而，这些改进只对那些具有符合TLS硬件要求的应用，以及能够重新编译以利用TLS硬件的应用具有相应的效果。更复杂的技术，例如，DEE、DMT以及多标量，则需要对现在的处理器实现更大的改动，只有当设计要求更高时才可能采用。

9.6　执行相同的线程

到目前为止，我们讨论了显式多线程和隐式多线程处理器的设计，它们都是同时执行多个线程，以提高处理器的吞吐率。许多研究人员还提出了另一种技术，就是在多个上下文中执行相同的指令。虽然这种技术看上去是违反直觉的，但还是有它的好处。首先提出的方案［Rotenberg，FTCS 1999］是活跃流/冗余流同时多线程（A/R-SMT），主要解决错误发现的问题。通过在不同的线程上下文中将同一指令流执行两次并比较其结果，可以发现流水线中的瞬时错误。也就是说，如果由于一个存储单元的错误导致流水线硬件将某一位翻转，在冗余的指令流中同一位也被翻转的可能性则很小。通过比较线程之间的结果，可以发现很多这样的瞬时错误。这些错误给我们带来了启示：活跃和冗余指令流相互协助可以使得执行更加有效。也就是说，它们可以相互为对方预取存储器数据，还可以相互化解分支预测失误。这些合作效应已经在一些研究方案中进行了探讨。我们将在后面的小节中讨论其中的一些方案，以及它们带来的好处：错误发现、预取以及分支化解。图9.18说明了执行相同线程的情况。图9.18（a）展示了如何利用冗余线程来检查主线程的瞬时错误，图9.18（b）展示了冗余线程如何为主线程预取cache以及如何化解主线程的分支预测失误。



(a)错误发现



(b)预执行

图 9.18 执行相同的线程

9.6.1 错误发现

如前文所述，冗余线程最初来源于这样一种想法，即相同线程两次执行结果之间的不一致可以用于发现瞬时错误。AR-SMT 以 SMT 处理器为基础，对 SMT 流水线的前端进行了改进，将取来的指令流复制进入两个独立的线程上下文。然后两个上下文彼此独立地执行，并将它们的结果存入再定序缓冲。流水线的提交段进行了进一步的改进，以比较指令结果，从而检查是否存在不一致的地方。任何不一致的地方都将用于确定流水线中的瞬时错误。实际的处理器设计采用了一种类似的方法，重点进行错误发现和容错处理。例如，IBM S/390 G5 处理器对所有的指令都进行冗余执行，但实现方法是在片内复制流水线硬件，两条流水线按锁步方式工作。Compaq 的 Tandem 也进行了类似的系统级设计。在这些设计中，两个物理的处理器芯片被耦合在一起，以锁步的方式运行相同的线程并通过比较处理器的结果来发现错误。实际上，这样的设计在容错计算领域已经有很长的历史了。

DIVA 方案［Austin, MICRO 1999］建立在 AR-SMT 的概念之上，但不是在 SMT 处理器上运行两个线程，而是使用一个简单的处理器并重新执行指令流，从而动态地检查另一个复杂处理器的计算结果。表面上看，这一对处理器的吞吐率会受限于简单的处理器，从而导致性能较差。然而，实际上，如果考虑到复杂处理器需要推测执行并化解所有的控制和数据相关，就可以知道，简单的处理器并不比复杂的处理器差。在这种情况下，没有必要将简单处理器上的代码并行化，因为所有的相关都已经消除，即所有的条件分支都已经化解，所有的数据相关都不存在，因为输入输出操作数的值都已经知道。简单处理器要实现的只是独立地验证每一条指令，根据提供的输入执行并得到输出结果，然后与提供的输出进行比较。一旦每一条指令都用这种方式进行了验证，那么根据归纳法，整个指令流都得到了验证。由于比较容易保证简单处理器的正确性，所以可以用简单处理器来检查一个复杂的、很可能存在设计问题的处理器操作。因此，这种方法除了用于发现瞬时错误以外，还可用于检查设计错误。

使用一个简单的、较慢的处理器进行动态验证确实还存在一个缺点，这一缺点在各种文献中还没有被充分地讨论。由于检查处理器只用于验证运算（例如 ALU 操作，存储器访问，分支等），因此可以将检查并行化，因为每一个运算都是独立地进行验证。然而，这依赖于复杂处理器提供正确操作数的能力。换句话说，没有检查复杂处理器内部的操作数通信，因为检查处理器依赖于复杂处理器进行正确的通信。由于操作数通信是现代乱序执行处理器复杂性的主要根源之一，因此有人认为检查处理器是不必要的。换句话说，关于错误检查，有人认为检查通信要比检查运算重要得多，因为对于验证 ALU 以及其他可以看做组合逻辑的计算路径，其正确性相对而言要简单一些。另一

方面，验证复杂的重命名机制和操作数定向机制是非常困难。而且，由于不检查操作数通信，复杂处理器的寄存器文件中的软错误（soft errors）不能通过DIVA检查器检查出来。

为了克服这一缺点，DIVA方案提倡在检查处理器中单独进行操作数通信检查。检查器对每条指令译码，从寄存器文件中读取源操作数，并将结果写回同一个寄存器文件。然而，读写寄存器操作数的过程可能存在写后读（RAW）、读后写（WAR）以及写后写（WAW）相关。它们在第4章中已经详细解释过，这些相关需要复杂的相关检测逻辑进行检测并化解，对于流水深度为n的流水线，相关检测逻辑的复杂度为$O(n^2)$。因此，将检查过程并行化会导致检查器的复杂度与主处理器相当。而且，如果检查器执行速度比主处理器慢，那么它必须支持更宽的流水线，以免成为性能瓶颈。在这种情况下，检查处理器实际上必须具有比它要检查的处理器还要复杂的逻辑。这一问题到底有多么复杂，是否会阻止DIVA成为一种加强容错能力并提高处理器性能的设计技术，还需要进一步的研究来确定。

9.6.2　预取

预取是冗余执行的正面效应之一，因为两个线程都产生相同的指令和数据存储器访问流。当一个线程运行到另一个线程前面的时候，会将有用的指令和数据预取到处理器的cache中，这将产生净加速，因为利用了额外的存储器级的并行。获得高性能的关键是尽量增大提前的程度，也就是两个线程之间的断层。流断层（slipstream）处理器［Rotenberg，ASPLOS 2000］就是这样做的。它对提前线程进行优化并将提前线程分解，去掉不必要的指令，而不是冗余地执行程序的所有指令。不必要的指令是指对于程序的结果没有影响的指令，或者只对化解可预测的分支有作用的指令。由于提前线程不再需要执行这些指令，因此在程序的控制流中，提前线程能更加提前，从而增加了两个线程之间的断层并创造出更好的预取机会。

预执行（preexecution）的方案进一步改善了上述情况。在这些方案中，可以使用profiling信息来确认有问题的指令，如经常预测失误的分支指令，或者经常导致cache失效的读指令。在编译的时候为这些指令构造动态数据流片断。组成这些片断的指令形成了预执行的线程，这个线程在运行时产生并可以使用类似SMT处理器的线程上下文。预执行的片断将会预先计算出问题指令的结果，如果失效，将向存储器发出预取。随后，工作线程（主线程）执行已经预执行过的指令，因而避免了cache失效。

相对于9.5节中讨论的隐式多线程技术，流断层和预执行的主要好处是提前线程不存在正确性的要求。也就是说，因为它只是用于产生预取以及辅助主线程的执行，并且对于程序的状态没有影响，因而产生和执行这样的线程都很容易，不存在关于控制和数据相关的问题需要解决。当然，相关化解的准确性能使提前线程发挥更大的作用，因为这样发出无用预取的可能性更小，比如主线程不可能到达的路径。但这是一个性能的问题，而不是正确性的问题，而且解决起来更加容易。

Intel曾描述过一个Pentium 4 SMT处理器的预执行机制（全功能、软件实现）。在该实现中，提前线程和主线程产生并被分派到同一个物理处理器执行，提前线程为主线程预取指令和数据，对于某些应用程序它得到了可观的加速比。

另一种使用冗余执行进行数据预取的方法是Datascalar体系结构。在该体系结构中，存储器被划分给几个处理器，所有的处理器都执行同一个程序。处理器通过快速广播网络连接起来，使它们能够很快对存储器操作数进行通信。每个处理器负责将程序对其本地存储器引用广播给其他处理器。按照这种方式，每个引用只被广播一次，每个处理器就能够满足程序的所有引用，要么从它的本地存储器获得，要么从一次广播中获得，这次广播由包含相应引用的远程存储器的拥有者发起。通过这种策略，所有对远程存储器的引用通过无请求的方式满足。也就是说，没有一个处理器需要请求

获得存储器数据的副本，如果本地不能满足，处理器只需要等待数据从广播网络上出现，因为拥有数据的远程处理器必将最终执行这一引用并将结果广播出来。结果是平均访存延迟不再包含请求延迟，而只包含广播网络上的传输延迟。在许多方面，这种方法在概念上与前面一种方法使用的冗余流预取很相近

9.6.3 分支化解

流断层和预执行的另一个主要好处是能够尽早地化解相关指令，而这些指令使用传统的方法难以正确地预测。对于流断层，在数据流中先于问题分支指令的指令是必要的，因此要由提前线程执行。分支的结果被传递给主线程，使得当它到达该分支时，可以使用已经计算出来的结果来避免预测失误。类似地，预执行为分支指令构造一个程序片断，产生一个推测的线程来预执行这个片断。尽早化解分支相关在实现上主要的挑战来自于两个线程的同步。对于指令和数据预取，不需要进行同步，因为主线程的取指或者访存将会发现目标就在指令或者数据cache中，而不会经历cache失效。或者说线程通过指令或者数据cache进行了同步，cache能够容忍一定程度的不准确性，这种不准确性包括地址（由于空间局部性）和时间（由于时间局部性）两方面。只要预取及时完成，也就是说既不晚（不能覆盖整个失效延迟）也不早（在主线程访问以前，预取的行被替换掉），预取就能带来好处。

然而，对于分支化解，预执行的分支结果必须和主线程中同一分支正确地同步，否则，如果将分支结果应用于错误的分支，预执行就会失败。线程不能基于静态分支（如PC分支）来同步，因为在两个线程之间的滑动指令窗口中，可能存在同一静态分支的多个动态实例。因此，必须采用一种引用计数机制来确保分支能够被正确的预执行分支化解。引用计数机制必须准确记录每一个静态分支有多少个动态实例。每个实例的分支结果存放在一个顺序队列中，提前线程向队列的一端插入新的分支结果，而主线程从另一端取走结果。如果队列的长度不正确，两个线程则不同步，预测的结果就没有用。构造这样一个队列以及相关的控制逻辑，以及发现预测失误时清空队列的机制，并不是一个简单的问题，目前也还没有令人满意的解决方法。

另外，分支结果可以通过已经存在的分支预测器间接地通信，方法是允许提前线程改变预测器的状态。因此，当主线程进行分支预测时，可以从状态已经改变的预测器中获得好处，因为两个线程通过预测器进行了隐式的同步。然而，提前线程及时并正确地改写预测器的可能性很小，特别是对于包含多级历史纪录的分支预测器。

9.6.4 结论

相同指令的冗余执行已经被提出并实现于错误发现。未来的容错实现技术很有可能在SMT处理器的上下文中采用冗余执行，因为这样做的开销是合理的，得到的错误覆盖也是很有帮助的，特别地，晶体管尺寸使得软错误（soft errors）更容易发生。利用冗余流的执行，产生预取以及尽早化解分支来提高性能的方法还没有到达实际设计阶段。纯软件的冗余流预取很有可能不久就会实现，因为至少理论上可能做到不需要改动任何硬件，然而，完全软件实现的机制在性能上能获得的好处还不十分清楚。假设未来基于SMT和CMP处理器的设计会成为普遍的方式，这可能会慢慢增加对这些机制的需求，因此就有可能部分地采用这些机制。

9.7 小结

本章广泛讨论了执行多线程的设计方案。许多重要的应用，特别是在服务器领域，包含了大量的线程级并行，可以在这样的系统中高效地运行本章所讨论的设计方案。本章还讨论了多处理机系

统和多线程处理器上下文中的显式多线程执行。构造多处理机系统的挑战主要是如何为所有的线程提供一致的存储器视图，同时尽量减少平均访存延迟。通过共享执行资源，多线程处理器可以获得更加高效的设计。共享执行资源的方式可以是芯片级的单芯片多处理器（CMP），也可以是以细粒度或者粗粒度的方式分时共享，还可以是同时多线程（SMT）。多个线程上下文还可以用于加速串行程序的执行，这样的方案包含复杂的硬件实现机制，如隐式多线程，以及软硬件混合的机制，如采用编译器转换和关键硬件辅助相结合的方式。所有这些方法都必须正确处理控制和数据相关，其中还有许多实现问题没有解决。最后，多线程上下文还可以用于冗余执行，既可以用来发现瞬时错误，还可以通过预先执行问题指令进行预取和分支化解，从而提高性能。

其中的许多技术已经被实际的系统所采用，其他还有很多技术仍然处于研究的状态，未来的设计可能会采用其中的部分技术，来克服构造高吞吐率、高主频和低功耗的计算机系统时会遇到的实现挑战

9.8　习题

1. 使用图9.3中的语法，展示如何使用load-linked/store-conditional原语来同步compare-and-swap操作。
2. 使用图9.3中的语法，展示如何使用load-linked/store-conditional原语在进入关键段之前获得锁变量。
3. 假设存在一个处理器，如广泛应用于Apple Macintosh系统的PowerPC G3，主要用于单处理机系统，因此只有一个非常简单的MEI cache一致性协议。为什么单机系统也需要支持cache一致性。G3的MEI协议是以此为目的的吗？试解释其中的原因。
4. Apple曾出售过一个基于G3的双处理机系统，主要用于运行不对称的负载。换句话说，第二个处理器只用于执行部分特定的应用，如Adobe Photoshop，而不是与第一个处理器对称地执行任何准备好的线程或进程。假设有一个多处理机操作系统（MacOS当时还不是），解释为什么对称地使用基于G3的双处理机系统可能会得到很差的性能。提出一个能够缓解这一问题的软件解决方案，并解释该方案。
5. 给定图9.6所示的MESI协议，为更加简单的MEI协议创建一个类似的规范（状态表和转换图），并评价该协议。
6. 许多系统使用MOESI cache一致性协议，其中O状态是指这一行是共享的而且是修改过的：即可能存在多个副本，但是其他副本都处于S状态，当这一行被替换出去的时候，由处于O状态行的cache负责将这一行写回存储器。修改图9.6中的状态表和转换图，以包含O状态。
7. 解释增加O状态至MESI协议有什么好处。
8. 为了支持分离事务总线，真正的一致性控制器除了图9.6所示的状态以外，还包括很多瞬时状态。例如，处理器对已作废的行（I）发出读请求，这一行则被置于IS瞬时状态，直到处理器收到一个有效的数据响应，然后这一行转换为共享状态（S）。给定一个分离事务总线，将每个总线命令（总线读，总线写以及总线升级）分成请求和响应，扩充图9.6的状态表和状态转换图，使之包含所有必要的瞬时状态和总线响应。为简单起见，假设对处于瞬态行的总线命令经过一定的延迟后，将得到一个负确认（NAK）响应，强制它退出。
9. 对于第8题，进一步扩充图9.6的状态表和转换图，通过增加必要的瞬时状态，消除至少三个NAK响应。评价所得到的一致性协议的复杂性。

10. 假设一个处理机包含下列条件：频率为1 GHz、目标CPI为2、每条指令的二级cache失效率为1%、基于监听的cache一致性协议、总共32个处理机、8字节的地址消息（包括命令和监听地址），计算每个处理机节点所需要的监听带宽。
11. 给定第10题的假设，对一个有64个处理机的系统进行改造。目前的二级cache设计为一个单端口的标识存储器，查找延迟是3 ns。该系统有足够的监听带宽吗？如果没有，对系统进行改进以使它具有足够的监听带宽。
12. 使用10.2.5小节中的公式，计算一个三级cache系统的平均访存延迟，一级cache如果命中需要1个周期，二级cache命中需要12个周期，三级cache命中需要50个周期，如果都失效则需要250个周期。假设一级cache失效率为5%，二级cache失效率为2%，三级cache失效率为0.5%。
13. 给定第12题的假设，计算一个没有L3 cache的系统的平均访存延迟，存储器的延迟为200周期（因为不再执行L3的查找）。哪一个系统性能较好？这两个系统的平衡失效率是多少（即两个系统提供相同性能时L3的失效率）？
14. 假设存在一个类似于Hewlett-Packard PA-8500的处理器且只有一级数据cache。假设cache采用虚地址索引而不是实地址标识，并且采用4路组相联，每行128B，总共2 MB。为了从总线监听一致性消息，使用了一个反地址转换表（RAT），存放从实地址到虚地址的映射。假设RAT是全相联的，虚存每页为4 KB，RAT需要多少个表项才能映射整个数据cache？
15. 给定第14题的假设，为RAT描述一个合理的组相联结构，使它仍然能够映射整个数据cache。
16. 给定第14题的假设，说明如果RAT不能映射数据cache中所有可能的表项时，将会出现什么情况。描述当RAT的表项被替换时发生的事件序列。
17. 图9.9解释了记录读操作在高性能顺序一致性多处理机中的应用。如图所示，通过监听读操作队列，可以检测到一个潜在的顺序冲突，并且当确认读指令时，将重新执行被标记的读指令。解释为什么不能在检测到冲突时立刻重新执行读指令，而是要等到指令确认时才重新执行。
18. 给定第17题描述的机制，假共享（远程处理机向一个cache行的低半段写，而本地处理机从高半段读）会导致附加的流水线预取。提出一种硬件机制，消除这种预取方式并对该机制的硬件开销进行量化。
19. 对于第18题，描述一种软件的方法，从而能够达到相同的效果。
20. 单芯片多处理器（CMP）实现了片内和片外一致性协议的组合。讨论以下一致性协议的所有组合情况、实现方法以及优缺点。对于片内，考虑更新和作废协议并使用监听和目录实现；对于片外，只考虑作废协议并使用监听和目录实现。哪一种方案是更有意义的组合？
21. 假设构造一个类似于Tera MTA的细粒度多线程处理器，使用大量并行的线程隐藏访存延迟。假设该处理器支持100个并行执行的线程，访存延迟为100个周期，每个周期为1 ns。同时假设使用传统的DRAM芯片实现存储子系统，DRAM芯片的命令保持（occupancy）时间为30 ns，即每个命令（读或写）在芯片接口保持的时间为30 ns。计算为了防止处理器因为等待访存结果而停顿、存控必须支持的独立DRAM芯片接口的最小数目。
22. 如第21题所述，如果进一步假设DRAM芯片支持页模式，其中8字节的顺序访问每次只需要10 ns。这样，第一次访问需要30 ns，但对同一页（512字节）的顺序访问只需要10 ns。处理器所运行的科学计算程序会顺序访问大数组中的各个元素。给定这种访存模式，为了防止处理器停顿，现在需要支持多少个独立的DRAM芯片接口？
23. IBM Northstar和Pulsar这样的粗粒度多线程处理器只提供顺序执行。如果将支持乱序执行的处理器用于粗粒度多线程，效率是否更高？解释其中的原因。

24. Intel Pentium 4同时多线程处理器支持指令的乱序执行。如果将顺序执行的处理器用于同时多线程，效率是否更高？解释其中的原因。
25. 对于具有分布处理单元的IMT设计（如多标量或者TLS），必须进行负载平衡以保证每个处理单元的负载大致相等。讨论基于硬件和软件的负载平衡机制，评价哪一种机制更适合于多标量和TLS。
26. 隐式多线程处理器（如DMT方案）将指令插入再定序缓冲的顺序与程序不一致。这意味着必须要有一套复杂的空闲列表管理机制，用于记录再定序缓冲中可用的表项。对于目前已有的乱序执行处理器的物理寄存器文件，也需要类似的空闲列表管理机制。说明DMT ROB（Re-Order Buffer）的空闲列表管理机制与物理寄存器的管理机制有何不同。详细描述这种机制，使用图表和伪码实现该管理算法。
27. DEE方案看来依赖于相当均匀的分支预测率，讨论如果错误的分支预测的分布不均匀（例如，一个错误的分支预测可能跟在一个或者多个错误预测之后）会发生什么事情。在这种情况下，DEE的效率如何？用一个例子说明DEE是否会仍然有效。
28. 最近的一项研究表明，TLS方法能从无声存储（silent stores）中获得巨大的好处。无声存储是指这样的store指令，它向存储器写入一个值，而该值已经存放在某一位置。创建一个详细的示范程序，说明无声存储如何提高TLS系统的性能。
29. 冗余线程对条件分支的预执行能够加速分支预测失误的化解，只要两个线程中的分支能正确地同步。提出一个详细的设计方案，使提前线程与主线程能够对分支进行正确的同步。指明设计中存在的挑战，并对其硬件部件的耗费进行量化。